

REPARTO DE PROBLEMAS

Marca qui n ya lo leyo. En Tematica anota de que se trata; en Dificultad, que tan duro se ve y si vale la pena ahora.

Problema	Juan	Felipe	Reyes	Tematica	Dificultad
A	☐	☐	☐		
B	☐	☐	☐		
C	☐	☐	☐		
D	☐	☐	☐		
E	☐	☐	☐		
F	☐	☐	☐		
G	☐	☐	☐		
H	☐	☐	☐		
I	☐	☐	☐		
J	☐	☐	☐		
K	☐	☐	☐		
L	☐	☐	☐		
M	☐	☐	☐		
N	☐	☐	☐		

XXXIX MARATON NACIONAL DE PROGRAMACION ACIS/REDIS 2026

----- NOTEBOOK -----

Universidad de Ibagué

Participantes:

Juan David Reyes Cure
Juan Felipe Rodriguez Barbosa
Juan Esteban Rodriguez Castellanos

September 29, 2026

Entrenador:

Stiven Correa

Contents

1 Data Structures

1.1 Brute Force - Count Majority Subarrays	7
1.2 Compresion de Coordenadas	7
1.3 Conteo de Inversiones con Fenwick - Frosh	8
1.4 Conteo de Inversiones y Redondeo Exacto - Sky Line	8
1.5 Fenwick Tree (Consultas offline) - D Query	9
1.6 Fenwick Tree (Offline Queries) - K Query	9
1.7 Fenwick Tree (Prefix Minimum, Dominancia 3D) - The Day Of The Competitors	10
1.8 Hash Map y Ordenamiento	11
1.9 Hash Set - Dangerous Dive	11
1.10 Heap con Borrado Perezoso - Aquarium Bubble Lights	11
1.11 Mo's Algorithm - Wine Production	12
1.12 Ordered Set (PBDS)	13
1.13 Persistencia (DFS + Rollback) - The Declaration Of Independence	13
1.14 Prefix Sums - Count Majority Subarrays2	14
1.15 Prefix Sums con Primera Aparicion - Accounting Qualifying Con Prefix	14
1.16 Segment Tree - Block Placement Queries	14
1.17 Segment Tree - Monarchy In Vertigo	15
1.18 Segment Tree - Running A Penitentiary	16
1.19 Small-to-Large Merging (DSU on Tree) - Distinct Colors	16
1.20 Sparse Table sobre Bloques Iguales - Frequent	17
1.21 Sqrt Decomposition - Holes	18
1.22 Tree Construction - Binary Tree	18

2 Dynamic Programming

2.1 Binary Search - Patches	19
2.2 Bitmask DP - Music Festival	19
2.3 Bitmask SOS DP - El Juego Del Calamar	19
2.4 Camino en Grilla con Balance de Parentesis - Has Valid Path	20
2.5 Convex Hull Trick	21
2.6 Digit DP	21

2.7 Distancia de Damerau-Levenshtein	22
2.8 Divide and Conquer Optimization	22
2.9 DP sobre Intervalos con Bandera de Inversion - Artic Virus	22
2.10 Interval Scheduling - Yuyuan Market	23
2.11 Prefix Sums - Zig Zag Array	24
2.12 SOS (Sum over Subsets) + Inclusion-Exclusion - How Many Teams	24
2.13 Steiner Tree - Joining Capitals	25
2.14 Subset Sum (Knapsack) - Karamell	25
2.15 Valor Esperado - Revenge Of The Salary Of At Coder Inc	26

3 Graph

3.1 2-SAT (Kosaraju) - Signal Coverage	26
3.2 Articulation Points (Tarjan) - Submerging Islands	28
3.3 BFS - Ladder Length	28
3.4 BFS - Boss	29
3.5 BFS - Bacon Number	30
3.6 BFS Backtracking - Find Ladders	31
3.7 Biconnected Components (Block-Cut Tree) + LCA - Hedge Mazes	31
3.8 Bipartite Matching - Attacking Rooks	33
3.9 Bipartite Matching - Knights In The Board	34
3.10 Bipartite Matching (Algoritmo de Kuhn) - School Dance	35
3.11 Circuito Euleriano (Algoritmo de Hierholzer) - Mail Delivery	35
3.12 Clique Maximo (Bron-Kerbosch con Pivote) - Holy Network	36
3.13 Clique Maximo con Backtracking Iterativo - Holy Network (clique Iterativo)	37
3.14 Componentes Fuente con Kosaraju - Lumina Juan	37
3.15 Componentes Fuente de la Condensacion (Tarjan) - Lumina	39
3.16 Componentes Fuertemente Conexas (Kosaraju) - Planets And Kingdoms	40
3.17 DFS - Artwork	40
3.18 DFS - Denouncing Mafia	41
3.19 DFS - A Mazing!	41
3.20 Dijkstra - Fantastic Robot	42

Contents (cont.)

3.21 Dijkstra con Nodos Virtuales - Chinese Innovation	43	5.15 Numeros Romanos	70
3.22 DSU (enfoque INCORRECTO para este problema) - Holy Network (DSU, Incorrecta)	44	5.16 Optimization - Satoru Gojo	71
3.23 Heavy-Light Decomposition - Los Ratones III	44	5.17 Polynomial Composition - Coach	71
3.24 Kruskal Reconstruction Tree - Trucks	46	5.18 Prefix Sums - Triangles	72
3.25 LCA (Binary Lifting) - Subway Lines	46	5.19 Recurrencia (Fibonacci) - Fractions Are Better When Continued	72
3.26 Max Flow - Download Speed	47	5.20 Recurrencias y busqueda de intervalo factible - Investigating Quadradòmeda	73
3.27 Max Flow - March Of The Penguins	48	5.21 Recursion (Simulacion tipo Josephus) - Metal Detector	73
3.28 Max Flow (Binary Search + Dinic) - Gasoline	49	5.22 Simulacion - Cahokia Ruins	73
3.29 MST con Prim (Grafo Denso) - Brute	50	6 Number Theory	
3.30 State-Space BFS - Folding Machine	51	6.1 BFS sobre Residuos - Only1s0s	74
3.31 Strongly Connected Components (Tarjan) - Proving Equivalences	51	6.2 Big Exponentiation - Uj (cpp)	74
3.32 Tree Degree Analysis - Colmillos	52	6.3 Big Exponentiation - Uj (java)	75
3.33 Tree Reconstruction (BFS por capas) - Expansion Of The Road Network	52	6.4 Criba de Eratostenes (Solo Impares)	75
4 Geometry		6.5 Criba de Factor Primo Mínimo + Inclusion-Exclusion - Ingredients That May Harm You	75
4.1 2D Difference Array (Coordinate Compression) - Office Space	54	6.6 Diferencia de Cuadrados - Square Diff	76
4.2 Area Circulo-Poligono	55	6.7 Divisibilidad - Archmage's Crystals	76
4.3 Convex Hull y Barrido Angular - Guard Deployment	55	6.8 Divisors - Aesthetics In Poetry	76
4.4 Coordenadas Polares	58	6.9 Exponenciacion Modular (Torre de Exponentes) - Exponentiation II	77
4.5 Geometria 1D - Kilometric Intersection	59	6.10 Factorizacion en Digitos	77
4.6 K Vecinos Mutuos y Clasificacion de Componentes - Celestial Veins	59	6.11 GCD and LCM - Journeygrapes	78
4.7 K-Means (Clustering)	60	6.12 Modular Arithmetic - Broul	78
4.8 Primitivas y Convex Hull	62	6.13 Modular Arithmetic - Creating Multiples	78
4.9 Probabilidad de Circulo Contenido - Omens	62	6.14 Modulo de Entero Gigante	78
4.10 Punto en Poligono Convexo ($O(\log n)$)	63	6.15 Modulo de un Binario Gigante - Binary Dozens	79
4.11 Rotating Calipers (Par Mas Lejano)	63	6.16 Number Theory (Criba y Factorizacion) - Counting Divisors	79
4.12 Shoelace (Area de Poligono)	64	6.17 Number Theory (GCD y Divisores) - The Seals Of Shanghai	79
4.13 Triangle Classification - AROD	65	6.18 Number Theory (LCM via Factorizacion de Ciclos) - Permutation Rounds	80
5 Math		6.19 Reconstruccion con Divisores - Remainders	81
5.1 2-Coloreo por Suma Potencia de 2 - Drug Test	66	6.20 Sieve of Eratosthenes - Prime Sieve	81
5.2 Ad-hoc - Attention To Meeting	66	6.21 Ternas Pitagoricas - Pythagorean	82
5.3 Algebra - The Wisdom Of Master Wei	66	7 Combinatorics	
5.4 Bit Manipulation - Meme Wars	67	7.1 Combinatoria (Factoriales Modulares) - Binomial Coefficients	82
5.5 Brute Force - Antiarithmetic	67	7.2 Combinatoria Avanzada (Lucas, Catalan, Stirling)	83
5.6 Construction - Ezraft And Array	67	7.3 Construction - Farmpiggy And Subset Sum	83
5.7 Eliminacion Gaussiana (mod p)	68	7.4 Conteo de Distintos con Inventario de Frecuencias - Total Numbers	84
5.8 Fibonacci (Fast Doubling) - Fibonacci Sum	68	7.5 Conteo de Inversiones (cadena repetida) - Inversions	84
5.9 Fibonacci (Zeckendorf-like Greedy) - Getting In Shape	69	7.6 Euler's Formula - Pizza Cutter	85
5.10 Fibonacci Fast Doubling - Pou	69	7.7 Formula Cerrada con Inverso Modular - Number Of Sets	85
5.11 Greedy - A Healthy Menu	70	7.8 Paridad de Binomiales (Kummer) - Less Coint Tosses	86
5.12 Implementation - Hour For A Run	70	7.9 Permutation Cycles - Lines Of Containers	86
5.13 Logaritmos - Gatuno's Descent Into Psychopathy	70	7.10 Probabilidad (Paradoja del cumpleaños) - Mount Rushmore And Birthdays	87
5.14 Matrix Exponentiation - Buses	70	7.11 Probabilidad (Valor esperado con inverso modular) - Frangolino Ali Na Mesa	87

Contents (cont.)

7.12 Ranking - Palindromic Lexi Small	88	10.14 Puente y Linterna (version con indices) - Bridge Juan	110
7.13 Ranking de Combinaciones - Discrete Catalog	88	10.15 Segment Counting - Cerros	110
7.14 Suma de Subconjuntos (2^n) - Group Work	89	10.16 Simulation - Kathmandu	110
7.15 Tecnica de Contribucion - Bytelandia's Stones	89	10.17 Sliding Window	110
7.16 Triangulos en Poligono Convexo	89	10.18 Sorted Pairing - Balatro	111
8 Game Theory		10.19 Sorting - Finding Forks	111
8.1 Misere Nim y Grundy Acotado	90	10.20 Sorting (Prefix Sums) - Coven Complications	111
8.2 Nim con Segment Tree - Juan Vs Frank	90	10.21 Two Pointers (Container With Most Water)	111
8.3 Posiciones Perdedoras (potencias de 2) - Digitsduel	90	11 Binary Search	
8.4 Sprague-Grundy - Marbles	91	11.1 Greedy (busqueda binaria sobre la respuesta) - Minas Gerais' Walls	112
9 String		11.2 Two Pointers - Black Hills Golden Jewels	112
9.1 Brute Force Matching - Enigma	91	12 Constructive Algorithms	
9.2 Chequeo de Palindromo	92	12.1 Hashing - Handling The Blocks	113
9.3 KMP para Contar Ocurrencias Solapadas - Judgmental Crowd	92	12.2 Secuencias de Langford - Doubled Sequence II	113
9.4 KMP sobre Cadena Recursiva (Duplicacion) - Song Of The Sirens	93	13 Implementation	
9.5 Maximo Solape Sufijo-Prefijo (KMP) - Sign	94	13.1 Ad Hoc - Toilet Seat	114
9.6 Monotonic Stack - Remove Duplicates1	95	13.2 Ad Hoc - Rock Paper Scissors Tournament	114
9.7 Palindromo Ignorando Simbolos - Just Palindromes	95	13.3 Ad Hoc - Zero Or One	115
9.8 Palindromos No Solapados (Greedy) - Max Palindromes	96	13.4 Ad Hoc - Collatz Polynomial	115
9.9 Parentesis Balanceados (Todas las Variantes)	96	13.5 Ad Hoc - Unraveling Monty Hall	115
9.10 Partition and Permutation - Policianacional	98	13.6 Arreglos - Buffoon	115
9.11 Remove Duplicate Letters - Remove Duplicates	98	13.7 Backtracking (Enumeracion de bits desconocidos) - Harmonics With Interference	115
9.12 Rotacion Minima (Dos Punteros) - Fingerprints	99	13.8 Brute Force - Cake Hater	116
9.13 Simulation - Process Str	100	13.9 Brute Force - LLMs	116
9.14 Suffix Array + LCP - Couple Of Bip Bop	100	13.10 Circular Queue - Eeny Meeny	117
9.15 Suffix Automaton - Beautiful Words	101	13.11 Clasificacion de Cadenas por Prefijo - Joker Or Judger	117
9.16 Trie + DP de Segmentacion (Maximo) - Spacebar Tokenizer	102	13.12 Conjuntos - João João	118
9.17 Z-Function - A String Problem	103	13.13 Constructive - A Story About Tea	118
10 Greedy		13.14 Conteo de Records - Harder Horizons	118
10.1 Binary Search - Maratona Brasileira De Popcorn	103	13.15 Cycle Detection - Switches	119
10.2 Bit Manipulation - Lecographically Maximum	104	13.16 Date Parsing - Crochet	119
10.3 Budget Optimization - Posters	104	13.17 Fuerza Bruta $O(n^2)$ sobre Subperiodos - Accounting Qualifying	120
10.4 Buy Low Sell High (Comprar y Vender Acciones)	104	13.18 Generacion de Numeros con Digitos Distintos - Division Digitos Distintos	120
10.5 Constructive Algorithms - Nomad	105	13.19 Greedy - Escalator	121
10.6 DP - Exhibition Of Clownfish	105	13.20 Lectura de Entrada	121
10.7 Huffman	106	13.21 Lights Out (Primera Fila Forzada) - Turnswitch	123
10.8 Huffman sobre Frecuencias Variables en el Tiempo - Efficient Encoding	106	13.22 Simulacion con Parsing deCodigo - Impossible Primebox	124
10.9 Interval Scheduling - Interval Scheduling Greedy	107	13.23 Simulation - Enigma Of The Jewelry Case	125
10.10 Maximo Conjunto a Distancia d en Arbol - Oficinas A Distancia D En Arbol	107	13.24 Template Base	125
10.11 Prefijos sobre Secuencias Balanceadas - Incompatible Pairs	108	14 Formulario	
10.12 Prefix Sums - Maximizing Advertising	108	14.1 Algebra y Ecuaciones	125
10.13 Puente y Linterna - The Bridge At Night	109		

Contents (cont.)

14.2 Areas y Volúmenes	126	17.13 Message Route - BFS de Camino Mínimo con Reconstrucción	144
14.3 Combinatoria	126	17.14 Monsters - BFS Multifuente + BFS de Escape	144
14.4 Cotas y Límites	127	17.15 Planetqueries I - Binary Lifting sobre Grafo Funcional	146
14.5 Datos que no se Deducen	127	17.16 Round Trip - Detección de Ciclo con BFS	146
14.6 Geometría	128	17.17 Round Trip (Stack) - Detección de Ciclo con Pila (DFS Iterativo)	147
14.7 Probabilidad Geométrica y Áreas Raras	129	17.18 Round Trip II - Ciclo en Grafo Dirigido (DFS de Tres Colores)	148
14.8 Rectas y Cónicas	129	17.19 Shortest Routes I - Dijkstra con Priority Queue	149
14.9 Secuencias Notables	130	17.20 Shortest Routes II - Dijkstra desde Cada Nodo	149
14.10 STL Indispensable	131	17.21 Shortest Routes II (Floyd Warshall) - Floyd-Warshall	151
14.11 Sumas y Series	131		
14.12 Tabla ASCII	132	18 CSES: Introductory Problems	
14.13 Teoría de Números	132	18.1 Apple Divisions - Fuerza Bruta 2^n	151
14.14 Trampas de C++	133	18.2 Bit Strings - Exponenciación Modular Rápida	152
14.15 Trigonometría	133	18.3 Chessboard And Queens - Backtracking con Poda por Diagonales	152
14.16 Trucos de Bits	134	18.4 Coin Piles - Invariantes Aritméticos	152
15 Divide and Conquer		18.5 Creating Strings - Backtracking sobre Frecuencias	152
15.1 Conteo de Inversiones con Merge Sort - Frosh Merge Sort	134	18.6 Digit Queries - Búsqueda por Bloques de Longitud	153
16 Uncategorized		18.7 Gray Code - Identidad i XOR (i > 1)	153
16.1 A - Apple Pie	135	18.8 Grid Coloring I - Greedy Local con 4 Colores	153
16.2 B - Balanced Balloons	135	18.9 Grid Path Description - Backtracking con Poda de Zona Encerrada	153
16.3 B - Binary Dozens	135	18.10 Increasing Array - Greedy de Máximo Prefijo	154
16.4 C - Clean Streets	135	18.11 Knight Moves Grid - BFS en Grilla	154
16.5 D - Displaying Decimals	135	18.12 Mex Grid Construction - Identidad MEX = XOR	155
16.6 E - Emergency Rations	135	18.13 Missing Number - Suma de Gauss	155
16.7 F - Fuzzy Factorization	135	18.14 Number Spiral - Fórmula por Capas	155
16.8 G - Gridland Power Gauge	135	18.15 Palindrome Reorder - Conteo de Frecuencias	155
16.9 I - Infiltration Route	135	18.16 Permutations - Construcción Directa	156
16.10 K - Kings Conquest	135	18.17 Rab Game I - Construcción por Rotación de Bloque	156
16.11 L - Lonely Creatures	135	18.18 Repetitions - Barrido Lineal de Rachas	156
17 CSES: Graph Algorithms		18.19 String Reorder - Greedy Lexicográfico con Cota de Factibilidad	156
17.1 Building Roads - DSU (Union-Find)	135	18.20 Tower Of Hanoi - Recursión Clásica	157
17.2 Building Teams - BFS de Bicoloreo (Bipartito)	136	18.21 Trailing Zeros - Fórmula de Legendre	157
17.3 Counting Rooms - Flood Fill en Grilla (BFS)	137	18.22 Two Knights - Conteo por Complemento	157
17.4 Course Schedule - Orden Topológico (Kahn)	137	18.23 Two Sets - Construcción por Bloques de 4	157
17.5 Cycle Finding - Bellman-Ford (Ciclo Negativo)	138	18.24 Weird Algorithm - Simulación Directa	158
17.6 Flight Discount - Dijkstra sobre Grafo de Capas	138		
17.7 Flight Routes - K Caminos Más Cortos	139	19 CSES: Range Queries	
17.8 Game Routes - DP sobre DAG en Orden Topológico	140	19.1 Distinct Value Queries II - Segment Tree sobre Ocurrencia Anterior	158
17.9 High Score - Bellman-Ford (Camino Máximo)	140	19.2 Distinct Values Queries - Mo's Algorithm	160
17.10 Investigation - Dijkstra con Cuatro Cantidades a la Vez	141	19.3 Dynamic Range Minimum Queries - Segment Tree (Mínimo)	160
17.11 Labyrinth - BFS con Reconstrucción de Camino	142	19.4 Dynamic Range Sum Queries - Fenwick Tree (BIT)	161
17.12 Longest Flight Route - Camino Más Largo en DAG (DFS con Memo)	143	19.5 Forest Queries - Prefix Sums 2D	161
		19.6 Forest Queries II - Fenwick 2D	162
		19.7 Hotel Queries - Segment Tree con Descenso	162

Contents (cont.)

19.8 Increasing Array Queries - Monotonic Stack + Binary Lifting con Costos	163
19.9 List Removals - Fenwick con Descenso Binario (k-esimo)	163
19.10 Missing Coin Sum Queries - Cubetas por Potencia de 2 + Segment Tree	164
19.11 Movie Festival Queries - Greedy + Binary Lifting sobre el Tiempo	165
19.12 Pizzeria Queries - Truco de p[i] mas y menos i	165
19.13 Polynomial Queries - Lazy Propagation de Progresiones Aritmeticas	166
19.14 Prefix Sum Queries - Segment Tree con Nodo Compuesto	167
19.15 Range Interval Queries - Barrido Offline con Fenwick	168
19.16 Range Queries And Copies - Segment Tree Persistente	168
19.17 Range Update Queries - Fenwick de Diferencias	169
19.18 Range Updates And Sums - Lazy Propagation con Suma y Asignacion	170
19.19 Range XOR Queries - Prefix XOR	171
19.20 Salary Queries - Fenwick con Compresion Offline	171
19.21 Static Range Minimum Queries - Sparse Table	172
19.22 Static Range Sum Queries - Prefix Sums	172
19.23 Sub Array Sum Queries - Segment Tree de Maximo Subarreglo	173
19.24 Sub Array Sum Queries II - Maximo Subarreglo en un Rango	173
19.25 Visible Building Queries - Monotonic Stack + Binary Lifting	174
20 CSES: String Algorithms	
20.1 All Palindromes - Manacher (Palindromo Mas Largo que Termina en Cada Posicion)	175
20.2 Finding Borders - Funcion de Prefijos (Cadena de Bordes)	175
20.3 Finding Periods - Periodos = n - Bordes	176
20.4 Longest Palindrome - Manacher	176
20.5 Minimal Rotation - Rotacion Minima (Dos Punteros)	177
20.6 Required Substring - DP sobre el Automata de KMP	177
20.7 String Matching - KMP (Busqueda de Patron)	178
20.8 Word Combinations - Trie + DP de Segmentacion (Conteo)	179

1 - Data Structures

1.1 Brute Force - Count Majority Subarrays

C++\LeetCode\countMajoritySubarrays.cpp

Cuenta subarreglos donde "target" es mayoría (aparece en mas de la mitad de las posiciones). Fuerza bruta $O(n^2)$: para cada inicio i expande el final j acumulando cuantas veces aparece target en el subarreglo actual y compara el doble de esa cuenta contra el tamaño del subarreglo.

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 class Solution {
6 public:
7     int countMajoritySubarrays(vector<int>& nums, int target) {
8         int god = 0;
9         for(int i = 0 ; i < nums.size(); i++)
10             {
11                 vector<int> temp;
12                 int tempi = 0;
13                 for(int j = i ; j < nums.size(); j++)
14                     {
15                         temp.push_back(nums[j]);
16                         if(nums[j]==target)
17                             {
18                                 tempi++;
19                             }
20                         if(tempi*2 > temp.size())
21                             {
22                                 god++;
23                             }
24                     }
25             }
26
27         return god;
28     }
29 };
30
```

1.2 Compresion de Coordenadas

C++\Notebook\compresion de Coordenadas.cpp

Cuando los VALORES llegan a $1e9$ pero solo hay $n \leq 2e5$ distintos, se reemplaza cada valor por su POSICION en la lista ordenada de valores distintos. Así un arreglo indexado por valor pasa de $1e9$ casillas a n , y de golpe caben Fenwick, segment tree o arreglos de diferencias. Son tres líneas y siempre las mismas: `sort(v.begin(), v.end()); v.erase(unique(v.begin(), v.end()), v.end());`; `idx = lower_bound(v.begin(), v.end(), x) - v.begin()`; La primera ordena, la segunda deja un solo ejemplar de cada valor (unique NO borra: mueve los repetidos al final y devuelve donde empieza la basura, por eso hace falta el erase), y `lower_bound` da el índice comprimido en $O(\log n)$. PROPIEDADES QUE HAY QUE TENER CLARAS, porque de ellas depende que se pueda usar: - Es MONOTONA: si $a < b$ entonces `comp(a) < comp(b)`. Por eso se puede comprimir y seguir comparando, ordenando o preguntando por rangos de valores. - Los valores IGUALES caen en el mismo índice, y eso casi siempre es lo que uno quiere. - NO preserva las distancias: `comp(1)` y `comp(1000000)` pueden quedar en 0 y 1. Si el problema suma longitudes, áreas o distancias, comprimir a secas da respuestas mal, y hay que guardar los valores originales para multiplicar por el ancho real de cada tramo (es lo que hace un arreglo de diferencias 2D sobre coordenadas comprimidas). Verificado: ida y vuelta exacta sobre 200000 casos aleatorios, monotonia y los duplicados cayendo en el mismo índice, sin un solo fallo. LOS DOS ERRORES QUE CUESTAN EL PROBLEMA: 1) Olvidar el erase despues del unique. El vector queda con basura al final, el tamaño esta mal y `lower_bound` puede caer en la zona muerta. 2) Comprimir solo los valores INICIALES cuando las actualizaciones van a introducir valores nuevos. Si una consulta futura escribe un valor que no esta en la tabla, no tiene índice donde ir. La salida es leer TODAS las consultas primero y meter esos valores en la compresion, lo que obliga a que la solución sea OFFLINE. (Así funciona "Salary Queries" de CSES, que esta en este mismo cuaderno.) CUANDO NO HACE FALTA: si los valores ya son chicos, no se comprime. Y si hacen falta los índices pero el problema es ONLINE y no se pueden juntar todos los valores antes, la alternativa es un segment tree dinámico (nodos creados a demanda) o un map, que cuesta un log mas y bastante mas memoria por elemento.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 // ----- la version corta, la que se escribe en contest -----
5 struct Compresor {
6     vector<long long> v;
7
8     // Se le pasan TODOS los valores que van a aparecer, incluidos los de las
```

```
9 // actualizaciones futuras.
10 void construir(vector<long long> vals) {
11     v = vals;
12     sort(v.begin(), v.end());
13     v.erase(unique(v.begin(), v.end()), v.end());
14 }
15
16 int n() const { return v.size(); }
17
18 // valor -> indice comprimido [0, n). El valor TIENE que existir.
19 int comp(long long x) const {
20     return lower_bound(v.begin(), v.end(), x) - v.begin();
21 }
22
23 // indice -> valor original
24 long long orig(int i) const { return v[i]; }
25
26 // Cuantos valores hay < x. Sirve cuando x puede NO estar en la tabla.
27 int menores(long long x) const {
28     return lower_bound(v.begin(), v.end(), x) - v.begin();
29 }
30
31 // Cuantos son <= x.
32 int menoresIguales(long long x) const {
33     return upper_bound(v.begin(), v.end(), x) - v.begin();
34 }
35 };
36
37 int main() {
38     ios::sync_with_stdio(false);
39     cin.tie(nullptr);
40
41     int n;
42     if (!(cin >> n)) return 0;
43
44     vector<long long> a(n);
45     for (long long &x : a) cin >> x;
46
47     Compresor c;
48     c.construir(a);
49
50     cout << "valores distintos: " << c.n() << '\n';
51
52     for (long long x : a) {
53         cout << x << " -> " << c.comp(x) << '\n';
54     }
55
56     // Uso tipico: el arreglo ya comprimido, listo para cualquier estructura
57     vector<int> comprimido(n);
58     for (int i = 0; i < n; i++) comprimido[i] = c.comp(a[i]);
59
60     cout << "comprimido:";
61     for (int x : comprimido) cout << ' ' << x;
62     cout << '\n';
63     return 0;
64 }
```

1.3 Conteo de Inversiones con Fenwick - Frosh

C++\CCPL\frsh.cpp

Resuelve "Frosh Week" (problema F, CCPL): hay n estudiantes en fila y solo se pueden intercambiar PARES CONSECUTIVOS; se pide el mínimo de intercambios para dejarlos ordenados. LO QUE HAY QUE VER ES LA EQUIVALENCIA, porque el resto es una plantilla: cada intercambio de vecinos cambia el orden relativo de UNA sola pareja, así que arreglar la fila cuesta exactamente tantos intercambios como PAREJAS DESORDENADAS haya, o sea el número de INVERSIONES (pares $i < j$ con $a[i] > a[j]$). No es una cota, es una igualdad: cada inversión hay que deshacerla y cada swap deshace una. COMO SE CUENTAN AQUÍ: se recorre de izquierda a derecha metiendo cada elemento en un Fenwick indexado por VALOR. Al llegar a $a[i]$ ya hay i elementos adentro; de esos, $fw.sum(pos)$ son los menores o iguales, así que $i - fw.sum(pos)$ son los MAYORES, y esos son justo los que quedaron a la izquierda siendo mas grandes: las inversiones que aporta $a[i]$. Se acumula y se inserta. La compresión de coordenadas (el `lower_bound` sobre el arreglo ordenado) no es opcional: los números de estudiante son arbitrarios y el Fenwick se indexa por posición, no por valor. El $+1$ es porque un Fenwick es 1-indexado a la fuerza (con $i = 0$, $i \& -i$ vale 0 y el `for` no avanza nunca). El acumulador va en `long long` y eso TAMPOCO es opcional: con $n = 10^6$ al revés el total es $n*(n-1)/2$, casi $5*10^{11}$, que revienta un `int` sin avisar. HAY OTRA FORMA, con merge sort, que está en "froshMergeSort" en este mismo cuaderno. Medido con $n = 10^6$ en esta máquina, las dos ya arregladas: Fenwick 477 ms aleatorio, 335 invertido, 337 ordenado, ~16 MB merge sort con buffer único 360 ms aleatorio, 311 invertido, 293 ordenado, ~8 MB O sea que para ESTE problema el merge sort gana, porque el Fenwick paga un sort y además guarda tres arreglos (a, el ordenado y el bit de `long long`) contra dos del merge. La razón para quedarse igual con la versión de Fenwick es que se GENERALIZA: el mismo código, cambiando que se pregunta, cuenta cuantos hay menores que x , responde por rangos, o soporta actualizaciones. El merge sort solo sabe contar inversiones y se acaba. Verificado contra el sample y contra la versión de merge sort sobre $n = 10^6$ en los tres casos (aleatorio, invertido y ya ordenado): mismo resultado, 250188280112 en el aleatorio.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Fenwick {
5     int n;
6     vector<long long> bit;
7
8     Fenwick(int n) : n(n), bit(n + 1, 0) {}
9
10    void add(int idx, long long val) {
11        for (; idx <= n; idx += idx & -idx)
12            bit[idx] += val;
13    }
14
15    long long sum(int idx) {
16        long long res = 0;
17
18        for (; idx > 0; idx -= idx & -idx)
19            res += bit[idx];
20
21        return res;
22    }
23 };
24
25 int main() {
26     ios::sync_with_stdio(false);
27     cin.tie(nullptr);
28
29     int n;
30
31     while (cin >> n) {
32
33         vector<int> a(n);
34
35         for (int i = 0; i < n; i++) {
36             cin >> a[i];
37         }
38
39         vector<int> sorted = a;
40         sort(sorted.begin(), sorted.end());
41
42         Fenwick fw(n);
43
44         long long god = 0;
45
46         for (int i = 0; i < n; i++) {
47
48             int pos = lower_bound(
```

```
48             sorted.begin(),
49             sorted.end(),
50             a[i]
51         ) - sorted.begin() + 1;
52
53         god += i - fw.sum(pos);
54
55         fw.add(pos, 1);
56     }
57
58     cout << god << '\n';
59 }
60
61
62 return 0;
63 }
```

1.4 Conteo de Inversiones y Redondeo Exacto - Sky Line

C++\ICPC\2024\K - SkyLine.cpp

Resuelve "Skyline" (problema K, ICPC 2024): cuenta las inversiones de la secuencia de alturas (pares $i < j$ con $h[i] > h[j]$) y responde la fracción que son del total de pares, inversiones / $(N(N-1)/2)$, con 3 decimales. El conteo es el de "frosh" (Frosh Week, CCPL, en este cuaderno): recorriendo de izquierda a derecha, los i elementos anteriores menos los que son $\leq h[i]$ dan los mayores que quedaron a la izquierda. DOS TRAMPAS, Y LA PRIMERA VERSION DE ESTE ARCHIVO CAIA EN LAS DOS: 1) El enunciado dice $-20 < h < 300$, pero su propio sample trae una altura de -29 . La primera versión indexaba el Fenwick con $h + 21$, y con -29 quedaba en -8 : `fw.add(-8)` no termina nunca ($i \& -i$ se hace 0 y el índice deja de avanzar). Se colgaba con el sample. Por eso ahora se comprime, que funciona con cualquier valor. Lección: cuando el sample contradice las cotas, manda el sample. 2) Las reglas generales piden redondear con los empates HACIA ARRIBA. Con `double` y `printf`, un empate exacto como 0.0625 ($N = 32$ con 31 inversiones) se redondea hacia el par y sale 0.062 . Ahora se calcula con enteros: $(2000*HD + PHD) / (2*PHD)$ da las milesimas con el empate hacia arriba. Verificado en 48 empates exactos y 1500 casos generales. El caso $N = 1$ va aparte porque el total de pares sería 0 y la división, $0/0$.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Fenwick {
5     int n;
6     vector<long long> bit;
7
8     Fenwick(int n) : n(n), bit(n + 1, 0) {}
9
10    void add(int idx, long long val) {
11        for (; idx <= n; idx += idx & -idx)
12            bit[idx] += val;
13    }
14
15    long long sum(int idx) {
16        long long res = 0;
17
18        for (; idx > 0; idx -= idx & -idx)
19            res += bit[idx];
20
21        return res;
22    }
23 };
24
25 int main() {
26     ios::sync_with_stdio(false);
27     cin.tie(nullptr);
28
29     int N;
30
31     while (cin >> N && N != 0) {
32
33         vector<int> h(N);
34
35         for (int &x : h)
36             cin >> x;
37
38         if (N == 1) {
39             cout << "0.000\n";
40             continue;
41         }
42     }
```

```

43 // Compresion de coordenadas: no se confia en el rango del enunciado,
44 // porque el propio sample trae una altura de -29.
45 vector<int> v = h;
46 sort(v.begin(), v.end());
47 v.erase(unique(v.begin(), v.end()), v.end());
48
49 Fenwick fw(v.size());
50
51 long long inversions = 0;
52
53 for (int i = 0; i < N; i++) {
54
55     int x = lower_bound(v.begin(), v.end(), h[i]) - v.begin() + 1;
56
57     // Elementos anteriores que son <= h[i]
58     long long leq = fw.sum(x);
59
60     // De los i anteriores, los demas son > h[i]
61     inversions += i - leq;
62
63     fw.add(x, 1);
64 }
65
66 long long totalPairs = 1LL * N * (N - 1) / 2;
67
68 // Redondeo a 3 decimales con enteros: empates hacia arriba, como pide
69 // el enunciado. Con double + printf un empate exacto (0.0625) puede
70 // redondearse hacia el par y dar 0.062 en vez de 0.063.
71 long long mil = (2000 * inversions + totalPairs) / (2 * totalPairs);
72
73 string frac = to_string(mil % 1000);
74 while (frac.size() < 3)
75     frac = "0" + frac;
76
77 cout << mil / 1000 << "." << frac << "\n";
78 }
79
80 return 0;
81 }

```

1.5 Fenwick Tree (Consultas offline) - D Query

C++\Training_Camp_2026\Day 5\H - D - query.cpp

Resuelve "D-query" (Day 5, problema H - Contest 05 [Avanzados]): dada una secuencia a_1, a_n y q consultas (i, j) , para cada una hay que responder cuantos elementos distintos hay en la subsecuencia a_i, a_j . Ordena las consultas por su extremo derecho j (vector order) y las procesa offline avanzando un puntero curJ de izquierda a derecha: por cada nueva posici n, si el valor ya apareci  antes (lastPos) le resta 1 en el BIT a su ocurrencia previa y suma 1 en la posici n actual, de modo que el BIT siempre cuenta, para cada valor, solo su  ltima ocurrencia hasta curJ; la respuesta de cada consulta es $query(j) - query(i-1)$.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int bit_tree[30005];
5 int n;
6
7 void update(int pos, int val){
8     for(; pos <= n; pos += pos & (-pos))
9         bit_tree[pos] += val;
10 }
11
12 int query(int pos){
13     int res = 0;
14     for(; pos > 0; pos -= pos & (-pos))
15         res += bit_tree[pos];
16     return res;
17 }
18
19 int main(){
20     ios_base::sync_with_stdio(false);
21     cin.tie(nullptr);
22
23     cin >> n;
24     vector<int> a(n + 1);
25     for(int i = 1; i <= n; i++)

```

```

26     cin >> a[i];
27
28     int q;
29     cin >> q;
30
31     vector<int> qi(q), qj(q);
32     vector<int> order(q);
33     for(int k = 0; k < q; k++){
34         cin >> qi[k] >> qj[k];
35         order[k] = k;
36     }
37
38     // sort query indices by right endpoint j
39     sort(order.begin(), order.end(), [&](int x, int y){
40         return qj[x] < qj[y];
41     });
42
43     vector<int> answer(q);
44     unordered_map<int,int> lastPos; // value -> last position it was seen
45     lastPos.reserve(1 << 20);
46
47     int curJ = 0; // how far we've processed (1-indexed)
48     for(int idx : order){
49         int j = qj[idx];
50         // advance pointer up to j, updating BIT
51         while(curJ < j){
52             curJ++;
53             int val = a[curJ];
54             auto it = lastPos.find(val);
55             if(it != lastPos.end()){
56                 update(it->second, -1); // remove old occurrence's contribution
57             }
58             update(curJ, 1);
59             lastPos[val] = curJ;
60         }
61         answer[idx] = query(j) - query(qi[idx] - 1);
62     }
63
64     for(int k = 0; k < q; k++)
65         cout << answer[k] << "\n";
66
67     return 0;
68 }

```

1.6 Fenwick Tree (Offline Queries) - K Query

C++\Training_Camp_2026\Day 4\I - K Query.cpp

Resuelve "K-query" (Day 4, problema I - Contest 04 [Avanzados]): dado un arreglo de N n meros y varias consultas (i, j, k) , hay que responder cuantos elementos del subarreglo $[i, j]$ son mayores que k . Ordena las consultas por k descendente y los elementos del arreglo por valor descendente (cmpQuery, cmpElement); recorre las consultas insertando en un Fenwick tree (bit) todos los elementos con valor mayor a k acumulados hasta el momento, de forma que una consulta de rango sobre el Fenwick (rangeQuery) cuenta exactamente los elementos $> k$ dentro de $[l, r]$.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Element {
5     int value;
6     int pos;
7 };
8
9 struct Query {
10     int l, r, k;
11     int id;
12 };
13
14 bool cmpElement(const Element &a, const Element &b) {
15     return a.value > b.value;
16 }
17
18 bool cmpQuery(const Query &a, const Query &b) {
19     return a.k > b.k;
20 }
21

```

```

22 class Fenwick {
23     vector<int> bit;
24     int n;
25
26 public:
27     Fenwick(int n) {
28         this->n = n;
29         bit.assign(n + 1, 0);
30     }
31
32     void update(int idx, int val) {
33         while (idx <= n) {
34             bit[idx] += val;
35             idx += idx & -idx;
36         }
37     }
38
39     int query(int idx) {
40         int sum = 0;
41         while (idx > 0) {
42             sum += bit[idx];
43             idx -= idx & -idx;
44         }
45         return sum;
46     }
47
48     int rangeQuery(int l, int r) {
49         return query(r) - query(l - 1);
50     }
51 };
52
53 int main() {
54     ios::sync_with_stdio(false);
55     cin.tie(nullptr);
56
57     int n;
58     cin >> n;
59
60     vector<Element> arr(n);
61
62     for (int i = 0; i < n; i++) {
63         cin >> arr[i].value;
64         arr[i].pos = i + 1;
65     }
66
67     sort(arr.begin(), arr.end(), cmpElement);
68
69     int q;
70     cin >> q;
71
72     vector<Query> queries(q);
73
74     for (int i = 0; i < q; i++) {
75         cin >> queries[i].l >> queries[i].r >> queries[i].k;
76         queries[i].id = i;
77     }
78
79     sort(queries.begin(), queries.end(), cmpQuery);
80
81     Fenwick bit(n);
82     vector<int> ans(q);
83
84     int idx = 0;
85
86     for (auto &qu : queries) {
87         while (idx < n && arr[idx].value > qu.k) {
88             bit.update(arr[idx].pos, 1);
89             idx++;
90         }
91         ans[qu.id] = bit.rangeQuery(qu.l, qu.r);
92     }
93
94     for (int x : ans)

```

```

95         cout << x << "\n";
96
97     return 0;
98 }

```

1.7 Fenwick Tree (Prefix Minimum, Dominancia 3D) - The Day Of The Competitors

C++\Training_Camp_2026\Day 4\J - The day of the competitors.cpp

Resuelve "The day of the competitors" (Day 4, problema J - Contest 04 [Avanzados]): cada uno de N competidores tiene un ranking en 3 competencias distintas, un competidor es "excelente" si ningun otro lo supera en las tres a la vez, y hay que contar cuantos competidores excelentes hay. Ordena a los competidores por el primer ranking (a) y recorre en ese orden manteniendo un Fenwick tree indexado por el segundo ranking (b) que guarda el minimo tercer ranking (c) visto; para cada competidor consulta el minimo de c entre los que tienen b menor (queryPrefixMin(b-1)): si ese minimo es $\geq$ c, nadie lo domina en las tres competencias y se cuenta como excelente.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 static char buf_[1 << 25];
5 static int buflen_, bufPos_;
6 inline int gc() {
7     if (bufPos_ == buflen_) { buflen_ = fread(buf_, 1, sizeof(buf_), stdin); bufPos_ = 0; if (!buflen_) return -1; }
8     return buf_[bufPos_++];
9 }
10 inline int readInt() {
11     int c = gc();
12     while (c != -1 && (c < '0' || c > '9') && c != '-') c = gc();
13     bool neg = false; if (c == '-') { neg = true; c = gc(); }
14     int x = 0;
15     while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = gc(); }
16     return neg ? -x : x;
17 }
18
19 int N;
20 vector<int> bitArr;
21
22 inline void update(int pos, int val) {
23     for (; pos <= N; pos += pos & (-pos)) {
24         if (val < bitArr[pos]) bitArr[pos] = val;
25     }
26 }
27
28 inline int queryPrefixMin(int pos) {
29     int res = INT_MAX;
30     for (; pos > 0; pos -= pos & (-pos)) {
31         if (bitArr[pos] < res) res = bitArr[pos];
32     }
33     return res;
34 }
35
36 int main() {
37     int t = readInt();
38     string out;
39     while (t--) {
40         N = readInt();
41         vector<array<int,3>> people(N);
42         for (int i = 0; i < N; i++) {
43             int a = readInt(), b = readInt(), c = readInt();
44             people[i] = {a, b, c};
45         }
46         sort(people.begin(), people.end(), [](const array<int,3>& x, const array<int,3>& y) {
47             return x[0] < y[0];
48         });
49         bitArr.assign(N + 1, INT_MAX);
50         int count = 0;
51         for (int i = 0; i < N; i++) {
52             int b = people[i][1];
53             int c = people[i][2];
54             int mn = queryPrefixMin(b - 1);
55             if (mn >= c) count++;
56             update(b, c);
57         }
58         out += to_string(count);

```

```

59     out += "\n";
60 }
61 fwrite(out.data(), 1, out.size(), stdout);
62 return 0;
63 }

```

1.8 Hash Map y Ordenamiento

C++\Notebook\Hash Map y Ordenamiento.cpp

Diferencia clave: *unordered_map* es tabla hash, $O(1)$ promedio pero SIN orden; *map* es árbol rojo-negro, $O(\log n)$ pero siempre ordenado por clave. Ninguno de los dos se puede ordenar por el valor (*second*) directamente, así que el patrón obligado es volcarlo a un vector `<pair<...>>` y ordenar ese vector con un comparador lambda. Aquí están los comparadores más usados: por clave ascendente/descendente, por valor ascendente/descendente, y por valor con desempate por clave (el que piden casi siempre en rankings y conteos de frecuencia). También el truco de recorrer con *structured bindings* para modificar valores *in-place* y el de invertir el mapa.

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 int main()
6 {
7     ios_base::sync_with_stdio(false);
8     cin.tie(nullptr);
9
10    // contar frecuencias
11    unordered_map<string, int> mp = {{ "c", 3}, {"a", 1}, {"b", 2}};
12
13    // para ordenar hay que volcar a un vector de pares
14    vector<pair<string, int>> v(mp.begin(), mp.end());
15
16    // por clave ascendente
17    sort(v.begin(), v.end(), [](auto &l, auto &r){ return l.first < r.first; });
18
19    // por valor descendente
20    sort(v.begin(), v.end(), [](auto &l, auto &r){ return l.second > r.second; });
21
22    // por valor descendente con desempate por clave ascendente
23    sort(v.begin(), v.end(), [](auto &l, auto &r){
24        if (l.second == r.second) return l.first < r.first;
25        return l.second > r.second;
26    });
27
28    for (auto &p : v)
29    {
30        cout << p.first << " -> " << p.second << "\n";
31    }
32
33    // modificar todos los valores in-place (el & es obligatorio, si no se copia)
34    // con C++17 también vale: for (auto &[key, val] : mp) val += 3;
35    for (auto &p : mp)
36    {
37        p.second += 3;
38    }
39
40    // buscar sin crear la clave por accidente (operator[] la crearia)
41    if (mp.count("b"))
42    {
43        cout << "b vale " << mp["b"] << "\n";
44    }
45
46    // invertir el mapa (valor -> clave) para ordenar por valor
47    vector<pair<int, string>> inv;
48    for (auto &p : mp) inv.push_back({p.second, p.first});
49    sort(inv.begin(), inv.end());
50
51    // map ya viene ordenado por clave, no necesita sort
52    map<int, string> m;
53    m[3] = "tres";
54    m[1] = "uno";
55    m[2] = "dos";
56    for (auto &p : m)
57    {
58        cout << p.first << " -> " << p.second << "\n";

```

```

59     }
60
61     return 0;
62 }

```

1.9 Hash Set - Dangerous Dive

C++\Training_Camp_2026\Day 2\E - Dangerous Dive.py

Resuelve "Dangerous Dive" (Day 2, problema E - Maratona de Programação da SBC 2013): de N buzos voluntarios que fueron a una misión, se conocen los R que regresaron; hay que reportar, en orden ascendente, los identificadores de los que NO volvieron (o * si volvieron todos). Guarda los identificadores que regresaron en un set k para poder consultar pertenencia en $O(1)$, recorre los identificadores de 1 a n y va concatenando a la salida los que no están en k (es decir, los que faltan).

```

1 n, m = list(map(int, input().split()))
2 k = set(list(map(int, input().split())))
3 if(n == m):
4     print("")
5 fin = ""
6 for i in range(1, n+1):
7     if(i not in k):
8         fin+=(str(i)+" ")
9 print(fin)

```

1.10 Heap con Borrado Perezoso - Aquarium Bubble Lights

C++\Codeforces\aquariumBubbleLights.cpp

Resuelve "Aquarium Bubble Lights" (Codeforces, problema B): hay n enteros no negativos y q operaciones; en cada una PRIMERO se le resta 1 a todos (sin bajar de cero) y DESPUES, si x no es cero, se hace $a[x] = \max(a[x], v)$. Tras cada operación hay que decir cuantos quedan positivos, y al final imprimir la secuencia. EL TRUCO ES NO DECREMENTAR NUNCA. Restarle 1 a los n elementos en cada una de las q operaciones cuesta $O(n \cdot q) = 4 \cdot 10^4 \cdot 10$ y no pasa ni de lejos. En vez de guardar el valor, se guarda CUANDO se va a volver cero: $\text{expire}[i] = \text{valor} + \text{el instante en que se fija}$. Con eso, $a[i]$ sigue positivo en el instante t es simplemente $\text{expire}[i] > t$, y el valor real en ese momento es $\max(0, \text{expire}[i] - t)$. El decremento global desaparece: no se toca a nadie, solo avanza el reloj. Al inicio el reloj está en 0, así que $\text{expire}[i]$ arranca valiendo $a[i]$, y al final los valores son $\text{expire}[i] - q$. EL BORRADO PEREZOSO ES LA OTRA MITAD. Se mete cada (*expire*, índice) en un heap de mínimos para saber quien expira primero. El problema es que cuando $a[x]$ se actualiza, su entrada VIEJA sigue tirada en el heap, y *std::priority_queue* no sabe borrar un elemento del medio. La salida es no borrarla: se deja ahí y, al sacarla, se compara contra el valor vigente *if* ($\text{expire}[\text{id}] \neq e$) *continue*; y si no coinciden es una entrada muerta y se ignora. Cada operación mete a lo sumo una entrada, así que el heap nunca pasa de $n + q$ elementos y el costo total sigue siendo $O((n+q) \log n)$. Este patrón es el que se usa siempre que haga falta un *decrease-key* y la cola de prioridad no lo tenga: es el mismo "if ($\text{dist} > \text{distancia[actual]}$) *continue*;" de Dijkstra. EL ORDEN DE LOS DOS PASOS NO ES NEGOCIABLE, y el enunciado lo recalca: primero se vacían los que expiran en el instante t (el *while*), y solo después se aplica la asignación. Si se hace al revés, un elemento que acaba de recibir un valor nuevo se contaría como expirado. El contador de positivos se lleva al vuelo, restando cuando alguien expira y sumando cuando uno pasa de cero a positivo; recorrer el arreglo en cada operación sería volver al $O(n \cdot q)$. Sobre los tipos: *expire* llega a lo sumo a $10^9 + 2 \cdot 10^5$, que todavía cabe en *int*, pero en *long long* uno no tiene que pararse a comprobarlo. Verificado contra una simulación directa del enunciado (restar 1 a todos, asignar, contar) en 4000 casos aleatorios con valores chicos para forzar que todo expire seguido: sin una sola diferencia, mas el *sample*. Medido con $n = q = 2 \cdot 10^5$ y valores hasta 10^9 : 0.27 s contra un límite de 1 s, y 0.16 s en el adversario de valores 0..3 donde el heap trabaja todo el tiempo. OJO: usa *structured bindings* (*auto [e, id]*), que piden C++17. En Codeforces compila, pero con un g++ viejo hay que volver a *.first* y *.second*.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 int main() {
7     ios::sync_with_stdio(false);
8     cin.tie(nullptr);
9
10    int n, q;
11    cin >> n >> q;
12
13    vector<ll> expire(n + 1);
14
15    // (expire, index)
16    priority_queue<pair<ll, int>,
17                vector<pair<ll, int>>,
18                greater<pair<ll, int>>> pq;
19
20    for (int i = 1; i <= n; ++i) {
21        ll a;
22        cin >> a;
23

```

```

24     expire[i] = a;
25
26     if (a > 0)
27         pq.push({expire[i], i});
28 }
29
30 int positive = 0;
31
32 // Initially, all a[i] > 0 are positive.
33 for (int i = 1; i <= n; ++i)
34     if (expire[i] > 0)
35         positive++;
36
37 for (int t = 1; t <= q; ++t) {
38     int x;
39     ll v;
40     cin >> x >> v;
41
42     // After the mandatory decrement at operation t,
43     // an element is zero iff expire[i] <= t.
44     while (!pq.empty() && pq.top().first <= t) {
45         auto [e, id] = pq.top();
46         pq.pop();
47
48         // Old heap entry.
49         if (expire[id] != e)
50             continue;
51
52         expire[id] = 0;
53         positive--;
54     }
55
56     if (x != 0) {
57         // Current value after the decrement.
58         ll cur = max(0LL, expire[x] - t);
59
60         ll nw = max(cur, v);
61
62         if (cur == 0 && nw > 0)
63             positive++;
64
65         if (nw > 0) {
66             expire[x] = t + nw;
67             pq.push({expire[x], x});
68         } else {
69             expire[x] = 0;
70         }
71     }
72
73     cout << positive << '\n';
74 }
75
76 // Final values are expire[i] - q.
77 for (int i = 1; i <= n; ++i) {
78     if (i > 1)
79         cout << ' ';
80
81     cout << max(0LL, expire[i] - q);
82 }
83
84 cout << '\n';
85
86 return 0;
87 }

```

1.11 Mo's Algorithm - Wine Production

C++ Training Camp, Day 7, E - Wine Production.cpp

Resuelve "Wine Production" (Day 7, problema E - Contest 07 [Avanzados]): dadas las temperaturas de N d  s y Q consultas de rango $[l, r]$, hay que calcular para cada rango la "calidad" x m  xima tal que existan al menos x temperaturas distintas que se repitan al menos x veces dentro de ese rango (un   ndice tipo h -index sobre las frecuencias de temperatura). Usa el algoritmo de Mo (offline, ordenando las consultas por bloques de tama  o $\sqrt{N}$) para mover los punteros l/r y mantener freqArr (frecuencia de cada valor comprimido) junto con $B[k]$ (cantidad de valores distintos con frecuencia $\geq k$); la variable h se ajusta incrementalmente en $\text{addElem}/\text{removeElem}$ para responder cada consulta en $O(1)$ amortizado.

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int N, Q;
5  vector<int> A; // valores comprimidos, 1-indexado
6  vector<int> freqArr; // frecuencia actual de cada valor comprimido
7  vector<int> B; // B[k] = cantidad de valores distintos con frecuencia >= k
8  int h;
9
10 inline void addElem(int val) {
11     int f = freqArr[val];
12     B[f + 1] += 1;
13     freqArr[val] = f + 1;
14     // solo puede crecer como maximo 1 (demostrado matematicamente),
15     // pero dejamos el while por seguridad/robustez.
16     while (h + 1 < (int)B.size() && B[h + 1] >= h + 1) h++;
17 }
18
19 inline void removeElem(int val) {
20     int f = freqArr[val];
21     B[f] -= 1;
22     freqArr[val] = f - 1;
23     while (h > 0 && B[h] < h) h--;
24 }
25
26 int main() {
27     scanf("%d %d", &N, &Q);
28     vector<int> rawA(N + 1);
29     vector<int> sortedVals(N);
30     for (int i = 1; i <= N; i++) {
31         scanf("%d", &rawA[i]);
32         sortedVals[i - 1] = rawA[i];
33     }
34     sort(sortedVals.begin(), sortedVals.end());
35     sortedVals.erase(unique(sortedVals.begin(), sortedVals.end()), sortedVals.end());
36     int M = (int)sortedVals.size();
37
38     A.assign(N + 1, 0);
39     for (int i = 1; i <= N; i++) {
40         int idx = (int)(lower_bound(sortedVals.begin(), sortedVals.end(), rawA[i]) - sortedVals.begin());
41         A[i] = idx + 1;
42     }
43
44     freqArr.assign(M + 2, 0);
45     B.assign(N + 2, 0);
46     h = 0;
47
48     vector<int> ql(Q), qr(Q), qIdx(Q);
49     for (int i = 0; i < Q; i++) {
50         scanf("%d %d", &ql[i], &qr[i]);
51         qIdx[i] = i;
52     }
53
54     int blockSize = max(1, (int)sqrt((double)N));
55     sort(qIdx.begin(), qIdx.end(), [&](int a, int b) {
56         int ba = ql[a] / blockSize, bb = ql[b] / blockSize;
57         if (ba != bb) return ba < bb;
58         if (ba & 1) return qr[a] > qr[b];
59         return qr[a] < qr[b];
60     });
61
62     vector<int> ans(Q);

```

```

63 int curL = 1, curR = 0;
64 for (int qi : qIdx) {
65     int l = ql[qi], r = qr[qi];
66     while (curR < r) { curR++; addElem(A[curR]); }
67     while (curL > l) { curL--; addElem(A[curL]); }
68     while (curR > r) { removeElem(A[curR]); curR--; }
69     while (curL < l) { removeElem(A[curL]); curL++; }
70     ans[qi] = h;
71 }
72
73 string out;
74 out.reserve(Q * 4);
75 for (int i = 0; i < Q; i++) {
76     out += to_string(ans[i]);
77     out += '\n';
78 }
79 fputs(out.c_str(), stdout);
80 return 0;
81 }

```

1.12 Ordered Set (PBDS)

C++\Notebook\Ordered Set (PBDS).cpp

Estructura basada en políticas de GNU: funciona como un set<> normal pero además esta indexada internamente, lo que da dos operaciones que un set no tiene, ambas en $O(\log n)$: `find_by_order(k)` devuelve un iterador al k -ésimo elemento (0-indexado, o end() si $k \geq \text{size}()$), y `order_of_key(x)` devuelve cuantos elementos hay estrictamente menores que x . Sirve para contar inversiones, rankings dinámicos o "cuantos menores que X llevo hasta ahora" sin montar un Fenwick con compresión de coordenadas. Para permitir duplicados usa `less_equal<int>` en vez de `less<int>` (ojo: con eso erase por valor deja de funcionar y hay que borrar via `find_by_order`). AVISO: es una extensión de GNU, solo compila con g++ (no con MSVC ni clang sin libstdc++). La instalación local de MinGW 6.3 de este equipo tiene los headers de `pb_ds` incompletos y da error al compilar, pero en Codeforces y en la mayoría de jueces funciona sin problema.

```

1 #include <bits/stdc++.h>
2 #include <ext/pb_ds/assoc_container.hpp>
3 #include <ext/pb_ds/tree_policy.hpp>
4
5 using namespace std;
6 using namespace __gnu_pbds;
7
8 typedef tree<int, null_type, less<int>, rb_tree_tag,
9             tree_order_statistics_node_update> ordered_set;
10
11 int main()
12 {
13     ios_base::sync_with_stdio(false);
14     cin.tie(nullptr);
15
16     ordered_set s;
17     s.insert(10);
18     s.insert(20);
19     s.insert(30);
20     s.insert(40);
21
22     cout << *s.find_by_order(0) << "\n"; // 10, el primero
23     cout << *s.find_by_order(2) << "\n"; // 30, el tercero
24
25     cout << s.order_of_key(30) << "\n"; // 2, hay dos menores que 30
26     cout << s.order_of_key(35) << "\n"; // 3, no hace falta que 35 exista
27
28     if (s.find_by_order(99) == s.end())
29     {
30         cout << "fuera de rango\n";
31     }
32
33     return 0;
34 }

```

1.13 Persistencia (DFS + Rollback) - The Declaration Of Independence

C++\Training_Camp_2020\Day 3\0 - The Declaration of Independence.py

Resuelve "The Declaration of Independence" (Day 5, problema D - Contest 05 [Avanzados]): el documento empieza vacío (versión 0) y cada query crea una nueva versión a partir de otra versión v ya existente: "E v x" copia la versión v y agrega la oración x al final, "D v" copia la versión v y elimina la primera oración; para cada query D hay que imprimir la oración eliminada. Modela las versiones como un árbol (dicc[v] guarda los hijos que se crearon a partir de v) y recorre ese árbol con un DFS (función dfs) manteniendo un único deque global "arr" que representa el documento actual; al entrar a un nodo aplica su operación (append o popleft) y al salir la deshace (rollback), simulando así todas las versiones sin copiar el documento.

```

1 import sys
2 from collections import deque
3
4
5 sys.setrecursionlimit(3*10**6)
6 n = int(input())
7 queries = []
8
9 dicc = {0: []}
10 deletes = []
11 for i in range(1, n+1):
12     q = input().split(" ")
13     kind = q[0]
14
15     if kind == 'E':
16         father = int(q[1])
17         num = int(q[2])
18         if father not in dicc:
19             dicc[father] = [(i, num)]
20         else:
21             dicc[father].append((i, num))
22     else:
23         father = int(q[1])
24         deletes.append(i)
25         if father not in dicc:
26             dicc[father] = [(i, kind)]
27         else:
28             dicc[father].append((i, kind))
29     arr = deque([])
30     res = [0] * (n + 1)
31     def dfs(currNode):
32         global arr
33         quitado = 0
34         if currNode[1] == "D":
35             if len(arr) > 0:
36                 quitado = arr.popleft()
37             else:
38                 if currNode[1] != None:
39                     arr.append(currNode[1])
40             res[currNode[0]] = quitado
41
42         if currNode[0] in dicc:
43             for son in dicc[currNode[0]]:
44                 dfs(son)
45
46         if currNode[1] == "D":
47             arr.appendleft(quitado)
48         else:
49             if len(arr) > 0:
50                 arr.pop()
51
52     dfs((0, None))
53
54 for i in deletes:
55     print(res[i], end=" ")

```

1.14 Prefix Sums - Count Majority Subarrays2

C++ LeetCode countMajoritySubarrays2.cpp

Misma idea que countMajoritySubarrays.cpp pero en $O(n)$: transforma el arreglo $a + 1$ (target) / -1 (cualquier otro valor) y cuenta subarreglos con suma positiva. Como el balance (prefijo acumulado desplazado por n) cambia en ± 1 en cada paso, usa un arreglo $pre[]$ indexado por ese balance para llevar, sin necesidad un Fenwick tree, cuantos prefijos anteriores tuvieron un balance menor al actual, actualizando $presum$ incrementalmente en $O(1)$ por posicion.

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 class Solution {
6 public:
7     long long countMajoritySubarrays(vector<int>& nums, int target) {
8         int n = nums.size();
9
10        vector<int> pre(2 * n + 1, 0);
11
12        pre[n] = 1;
13
14        int cnt = n;
15        long long presum = 0;
16        long long ans = 0;
17
18        for (int x : nums) {
19            if (x == target) {
20                presum += pre[cnt];
21
22                ++cnt;
23                ++pre[cnt];
24            } else {
25                --cnt;
26
27                presum -= pre[cnt];
28                ++pre[cnt];
29            }
30
31            ans += presum;
32        }
33
34        return ans;
35    }
36};
```

1.15 Prefix Sums con Primera Aparicion - Accounting Qualifying Con Prefix

C++ LeetCode accountingQualifyingConPrefix.cpp

Resuelve "Account Qualifying" (problema A, ICPC 2025): de una lista de transacciones hay que dar el mayor deposito d , el mayor retiro w (el mas negativo) y r , el largo del subperiodo mas largo con TANTOS depositos como retiros. EL TRUCO DE r : se mapea deposito $a + 1$, retiro $a - 1$ y consulta de saldo a 0. Entonces "tantos depositos como retiros" es "la suma del tramo es 0", y con sumas de prefijo eso es $prefijo[j] == prefijo[i]$. Guardando la PRIMERA posicion donde aparece cada valor de prefijo, el tramo mas largo que termina en j sale restando esa primera aparicion. Por eso el mapa solo se escribe cuando el valor es nuevo: reescribirlo acortaria los tramos. El $first[0] = -1$ no es un detalle menor: cubre los tramos que arrancan en la posicion 0, donde el prefijo vale 0 "antes de empezar". OJO CON D Y W : si no hay depositos d vale 0, y si no hay retiros w vale 0. Tomar el maximo y el minimo del arreglo a secas esta MAL, y el propio sample lo pilla: con 100 200 300 el minimo es 100 pero la respuesta es 0. Por eso van acotados con $\max(0, \dots)$ y $\min(0, \dots)$. La primera version de este archivo fallaba justo ahi. Los ceros (consultas de saldo) no rompen nada: aportan 0 a la suma, asi que un tramo de puros ceros cuenta como valido, que es lo que pide el enunciado. Costo $O(n)$ por caso. Medida: 5 casos de $n = 10000$ en 20 ms. Verificado contra fuerza bruta en 2900 casos, incluidos los que no tienen depositos o no tienen retiros.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int n;
9
10    while (cin >> n && n != 0) {
11
12        vector<int> a(n);
```

```
13
14        for (int &x : a)
15            cin >> x;
16
17        // Si no hay depositos d es 0, y si no hay retiros w es 0: por eso se
18        // acotan con 0 en vez de tomar el maximo y el minimo a secas.
19        int d = max(0, *max_element(a.begin(), a.end()));
20        int w = min(0, *min_element(a.begin(), a.end()));
21
22        // Convertimos:
23        // deposito -> +1
24        // balance -> 0
25        // retiro -> -1
26        for (int &x : a) {
27            if (x > 0) x = 1;
28            else if (x < 0) x = -1;
29        }
30
31        // first[s] = primera posicion donde aparecio
32        // el prefix sum s
33        unordered_map<int, int> first;
34
35        int sum = 0;
36        int r = 0;
37
38        // prefix sum 0 aparece antes de empezar
39        first[0] = -1;
40
41        for (int i = 0; i < n; i++) {
42
43            sum += a[i];
44
45            if (first.count(sum)) {
46                // Mismo prefix sum => entre ambas posiciones
47                // la suma es 0
48                r = max(r, i - first[sum]);
49            } else {
50                // Guardamos solamente la primera aparicion,
51                // porque nos da el segmento mas largo
52                first[sum] = i;
53            }
54        }
55
56        cout << d << ' ' << w << ' ' << r << '\n';
57    }
58
59    return 0;
60}
```

1.16 Segment Tree - Block Placement Queries

C++ LeetCode blockPlacementQueries.cpp

Resuelve "Block Placement Queries": usa un segment tree de maximos y un set de obstaculos para determinar si un bloque de tamaño sz cabe en algun hueco del rango $[0, x]$.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 typedef int T; // tipo de dato del segtree
5 struct segtree {
6     vector<T> st; //, lazy;
7     int n;
8     T neutro = 0; // neutro para MAX (los huecos son >= 0)
9
10    segtree(const vector<T> &v) {
11        n = v.size();
12        st.assign(n*4, 0);
13        //lazy.assign(n*4, neutro);
14        build(1, 0, n-1, v);
15    }
16
17    void build(int p, int L, int R, const vector<T> &v) {
18        if (L == R) st[p] = v[L];
19        else {
```

```

20     int m=(L+R)/2, l=p*2, r=l+1;
21     build(l,L,m,v); build(r,m+1,R,v);
22     st[p] = max(st[l], st[r]); // <- max
23 }
24 }
25
26 /*
27 void propagate(int p, int L, int R, T val) {
28     if (val == neutro) return;
29     st[p] = val;
30     lazy[p] = neutro;
31     if (L == R) return;
32     int l = p*2, r = l+1;
33     lazy[l] = lazy[r] = val;
34 }
35 */
36
37 T query(int i, int j) { return query(1, 0, n-1, i, j); }
38 void upd(int i, int j, T val) { upd(1, 0, n-1, i, j, val); }
39
40 T query(int p, int L, int R, int i, int j) {
41     //propagate(p, L, R, lazy[p]);
42     if (i > R || j < L) return neutro;
43     if (i <= L && R <= j) return st[p];
44     int m=(L+R)/2, l=p*2, r=l+1;
45     T lf = query(l, L, m, i, j);
46     T rg = query(r, m+1, R, i, j);
47     return max(lf, rg); // <- max
48 }
49
50 void upd(int p, int L, int R, int i, int j, T val) {
51     //propagate(p, L, R, lazy[p]);
52     if (i > R || j < L) return;
53     if (i <= L && R <= j) st[p] = val; // cambiar por propagate(p,L,R,val);
54     else {
55         int m=(L+R)/2, l=p*2, r=l+1;
56         upd(l, L, m, i, j, val);
57         upd(r, m+1, R, i, j, val);
58         st[p] = max(st[l], st[r]); // <- max
59     }
60 }
61 };
62
63 class Solution {
64 public:
65
66     vector<bool> getResults(vector<vector<int>>& queries) {
67         int MAX = 0;
68         for (auto &q : queries) MAX = max(MAX, q[1]);
69
70         vector<T> base(MAX + 1, 0); // árbol de huecos, todo en 0
71         segtree seg(base);
72
73         set<int> obs = {0}; // 0 es la pared izquierda
74         vector<bool> res;
75
76         for (auto &q : queries) {
77             if (q[0] == 1) { // colocar obstáculo en x
78                 int x = q[1];
79                 auto it = obs.lower_bound(x); // primer elemento > x
80                 int prev = *std::prev(it);
81                 seg.upd(x, x, x - prev); // hueco que termina en x
82                 if (it != obs.end()) {
83                     int next = *it;
84                     seg.upd(next, next, next - x); // el hueco viejo se acorta
85                 }
86                 obs.insert(x);
87             } else { // consulta [2, x, sz]
88                 int x = q[1], sz = q[2];
89                 auto it = obs.upper_bound(x); // primer elemento > x
90                 int prev = *std::prev(it); // mayor obstáculo <= x

```

```

93     bool ok = (x - prev >= sz) || (seg.query(0, x) >= sz);
94     res.push_back(ok);
95 }
96 }
97 return res;
98
99
100 }
101 };

```

1.17 Segment Tree - Monarchy In Vertigo

C++\Training_Camp_2026\Day 4\H - Monarchy in Vertigo.cpp

Resuelve "Monarchy in Vertigo" (Day 4, problema H - Contest 04 [Avanzados]): se mantiene un árbol genealógico donde nacen hijos (agregados al final de la lista de hijos de su padre) y mueren personas, y tras cada muerte hay que reportar quien es el nuevo monarca recorriendo el árbol con la regla "primero los hijos, de mayor a menor edad". Ese recorrido es exactamente un preorder del árbol final (el orden de nacimiento ya respeta "hijos antes que hermanos, mayor antes que menor"), así que precalcula el rango preorder de cada persona (rankOf) y usa un segment tree con OR para marcar vivos/muertos y encontrar en $O(\log N)$ la primera posición viva del preorder (queryFirstAlive), que es el monarca actual.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 static int N;
5 static vector<int> seg;
6
7 void update(int node, int lo, int hi, int pos, int val) {
8     if (lo == hi) { seg[node] = val; return; }
9     int mid = (lo + hi) >> 1;
10    if (pos <= mid) update(node<<1, lo, mid, pos, val);
11    else update(node<<1, mid+1, hi, pos, val);
12    seg[node] = seg[node<<1] | seg[node<<1+1];
13 }
14
15 int queryFirstAlive(int node, int lo, int hi) {
16     if (seg[node] == 0) return -1;
17     if (lo == hi) return lo;
18     int mid = (lo + hi) >> 1;
19     int r = queryFirstAlive(node<<1, lo, mid);
20     if (r != -1) return r;
21     return queryFirstAlive(node<<1, mid+1, hi);
22 }
23
24 int main(){
25     int Q;
26     scanf("%d", &Q);
27     vector<int> type(Q), arg(Q);
28     for (int i = 0; i < Q; i++) scanf("%d %d", &type[i], &arg[i]);
29
30     int births = 0;
31     for (int i = 0; i < Q; i++) if (type[i] == 1) births++;
32     N = births + 1; // total number of people that will ever exist
33
34     vector<vector<int>> children(N + 1);
35     vector<int> childId(Q, -1);
36     int nextId = 2;
37     for (int i = 0; i < Q; i++) {
38         if (type[i] == 1) {
39             int cid = nextId++;
40             children[arg[i]].push_back(cid);
41             childId[i] = cid;
42         }
43     }
44
45     // Pre-order numbering of the FINAL tree (birth order never changes relative order)
46     vector<int> rankOf(N + 1), idAtRank(N + 1);
47     vector<int> st; st.reserve(N + 1);
48     st.push_back(1);
49     int cur = 0;
50     while (!st.empty()) {
51         int u = st.back(); st.pop_back();
52         rankOf[u] = ++cur;
53         idAtRank[cur] = u;
54         for (int i = (int)children[u].size() - 1; i >= 0; i--) st.push_back(children[u][i]);

```

```

95     }
96
97     seg.assign(4 * (N + 1), 0);
98     update(1, 1, N, rankOf[1], 1);    // Constant starts alive
99
100    string out; out.reserve(Q * 7);
101    for (int i = 0; i < Q; i++) {
102        if (type[i] == 1) {
103            update(1, 1, N, rankOf[childId[i]], 1);
104        } else {
105            update(1, 1, N, rankOf[arg[i]], 0);
106            int pos = queryFirstAlive(1, 1, N);
107            out += to_string(idAtRank[pos]);
108            out += '\n';
109        }
110    }
111    fputs(out.c_str(), stdout);
112    return 0;
113 }

```

1.18 Segment Tree - Running A Penitentiary

C++\Training_Camp_2026\Day 7\D - Running a penitentiary.cpp

Resuelve "Running a penitentiary" (Day 7, problema D - Contest 07 [Avanzados]): N guardias vigilan cada uno un intervalo de celdas $[L, R]$, y hay que soportar Q operaciones: cambiar el intervalo de un guardia, o, dado un rango de guardias $[a, b]$, calcular cuantas celdas son vigiladas simultaneamente por TODOS los guardias de ese rango (la interseccion de sus intervalos). Usa un segment tree doble (treeL guarda el maximo de los L en el rango, treeR el minimo de los R) con point update (updatePos) y range query (query); la interseccion resultante tiene longitud $\max(0, \min R - \max L + 1)$.

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  typedef long long ll;
4
5  int n;
6  vector<ll> treeL, treeR;    // treeL: max de L en el rango, treeR: min de R en el rang
7
8  const ll NEG_INF = -2000000000000000LL;
9  const ll POS_INF = 2000000000000000LL;
10
11 inline void pull(int i) {
12     treeL[i] = max(treeL[2*i], treeL[2*i+1]);
13     treeR[i] = min(treeR[2*i], treeR[2*i+1]);
14 }
15
16 // actualiza el guarda en posicion 0-indexada 'pos' con nuevo intervalo [lVal, rVal]
17 void updatePos(int pos, ll lVal, ll rVal) {
18     pos += n;
19     treeL[pos] = lVal;
20     treeR[pos] = rVal;
21     for (pos >= 1; pos >= 1; pos >= 1) pull(pos);
22 }
23
24 // consulta semiabierto [l, r) 0-indexada -> devuelve (max L, min R) en ese rango
25 pair<ll, ll> query(int l, int r) {
26     ll resL = NEG_INF, resR = POS_INF;
27     for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
28         if (l & 1) {
29             resL = max(resL, treeL[l]);
30             resR = min(resR, treeR[l]);
31             ++l;
32         }
33         if (r & 1) {
34             --r;
35             resL = max(resL, treeL[r]);
36             resR = min(resR, treeR[r]);
37         }
38     }
39     return make_pair(resL, resR);
40 }
41
42 int main() {
43     ios_base::sync_with_stdio(false);
44     cin.tie(nullptr);
45 }

```

```

46 int N, Q;
47 cin >> N >> Q;
48 n = N;
49 treeL.assign(2 * n, 0);
50 treeR.assign(2 * n, 0);
51
52 for (int i = 0; i < n; i++) {
53     ll l, r;
54     cin >> l >> r;
55     treeL[n + i] = l;
56     treeR[n + i] = r;
57 }
58
59 for (int i = n - 1; i >= 1; i--) pull(i);
60
61 string output;
62 output.reserve((size_t)Q * 8);
63
64 for (int qq = 0; qq < Q; qq++) {
65     char op;
66     cin >> op;
67     if (op == 'C') {
68         int i; ll l, r;
69         cin >> i >> l >> r;
70         updatePos(i - 1, l, r);
71     } else { // '?'
72         int a, b;
73         cin >> a >> b;
74         pair<ll, ll> res = query(a - 1, b);
75         ll ans = res.second - res.first + 1;
76         if (ans < 0) ans = 0;
77         output += to_string(ans);
78         output += '\n';
79     }
80 }
81
82 cout << output;
83 return 0;
84 }

```

1.19 Small-to-Large Merging (DSU on Tree) - Distinct Colors

C++\Training_Camp_2026\Day 4\M - Distinct Colors.cpp

Resuelve "Distinct Colors" (Day 4, problema M - Contest 04 [Avanzados]): dado un arbol con raiz en el nodo 1 donde cada nodo tiene un color, hay que calcular para cada nodo la cantidad de colores distintos en su subarbol. Hace un DFS iterativo para obtener un orden donde cada hijo aparece antes que su padre (orden invertido), y procesa los nodos en ese orden fusionando el set de colores de cada hijo (colorSet) dentro del set mas grande entre padre e hijo (swap + insert), tecnica small-to-large que garantiza complejidad $O(N \log^2 N)$ en vez de copiar sets completos en cada fusion.

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main(){
5      ios_base::sync_with_stdio(false);
6      cin.tie(NULL);
7
8      int n;
9      cin >> n;
10
11      vector<int> color(n + 1);
12      for (int i = 1; i <= n; i++) cin >> color[i];
13
14      vector<vector<int>> adj(n + 1);
15      for (int i = 0; i < n - 1; i++) {
16          int a, b;
17          cin >> a >> b;
18          adj[a].push_back(b);
19          adj[b].push_back(a);
20      }
21
22      vector<int> parent(n + 1, 0);
23      vector<int> order;
24      order.reserve(n);
25
26      // DFS iterativo (evita overflow de pila en arboles muy profundos, tipo cadena)

```

```

27 vector<int> stck;
28 stck.reserve(n);
29 vector<char> visited(n + 1, 0);
30 stck.push_back(1);
31 visited[1] = 1;
32 while (!stck.empty()) {
33     int u = stck.back();
34     stck.pop_back();
35     order.push_back(u);
36     for (int v : adj[u]) {
37         if (!visited[v]) {
38             visited[v] = 1;
39             parent[v] = u;
40             stck.push_back(v);
41         }
42     }
43 }
44
45 // Invertimos el preorder: en este orden, cada hijo aparece antes que su padre,
46 // así que al procesar u, todos sus hijos ya estan listos.
47 reverse(order.begin(), order.end());
48
49 vector<set<int>> colorSet(n + 1);
50 vector<int> ans(n + 1, 0);
51
52 for (int u = 1; u <= n; u++) colorSet[u].insert(color[u]);
53
54 // Small-to-large merging: siempre fusionamos el set mas chico dentro del mas grande
55 for (int u : order) {
56     for (int v : adj[u]) {
57         if (v != parent[u]) { // v es hijo de u
58             if (colorSet[u].size() < colorSet[v].size())
59                 colorSet[u].swap(colorSet[v]); // swap de sets es O(1), no copia elementos
60             colorSet[u].insert(colorSet[v].begin(), colorSet[v].end());
61             colorSet[v].clear(); // liberamos memoria que ya no se necesita
62         }
63     }
64     ans[u] = (int)colorSet[u].size();
65 }
66
67 for (int i = 1; i <= n; i++) {
68     cout << ans[i];
69     cout << (i < n ? ' ' : '\n');
70 }
71
72 return 0;
73 }

```

1.20 Sparse Table sobre Bloques Iguales - Frequent

C++ CCPL/frequent.cpp

Resuelve "Frequent Values" (problema E, CCPL): un arreglo de n enteros NO DECRECIENTE y q consultas $[i, j]$; para cada una hay que decir cuantas veces aparece el valor mas repetido de ese rango. LA LLAVE DEL PROBLEMA ES QUE EL ARREGLO VIENE ORDENADO, y sin eso no habria solucion facil. Al estar ordenado, los valores iguales quedan pegados en BLOQUES contiguos, asi que la pregunta deja de ser sobre valores y pasa a ser sobre longitudes de bloque. Si el arreglo llegara desordenado esto no sirve: ahí tocaria Mo, que esta en la ficha de CSES. Un rango $[i, j]$ toca a lo sumo tres cosas: - el PEDAZO del bloque donde cae i , que va de i hasta el final de su bloque - el PEDAZO del bloque donde cae j , desde el inicio de ese bloque hasta j - los bloques COMPLETOS que quedan en el medio. Los dos pedazos se calculan con una resta, y para los bloques completos hay que pedir el maximo de un rango de longitudes, que es exactamente lo que responde una sparse table en $O(1)$. La respuesta es el mayor de los tres. EL CASO QUE SE OLVIDA: si i y j caen en el MISMO bloque no hay medio ni dos pedazos, y la respuesta es $j - i + 1$ directo. Si uno no separa ese caso, el rango de bloques del medio sale invertido ($bi+1 > bj-1$) y la consulta a la sparse table se va a leer basura. Por eso el maxRango devuelve 0 cuando $l > r$, que es el segundo cinturón de seguridad. POR QUE SPARSE TABLE Y NO SEGMENT TREE: el arreglo de longitudes no cambia nunca, y el maximo es idempotente, así que se pueden solapar los dos bloques de la consulta y responder en $O(1)$. Un segment tree daría $O(\log n)$ y mas codigo, y solo hace falta si hubiera actualizaciones. Costo $O(n \log n)$ para construir y $O(1)$ por consulta. Medido con $n = q = 100000$: 0.09 s, incluyendo los adversarios de todos iguales (un solo bloque) y todos distintos (n bloques). Verificado contra fuerza bruta: 19407 consultas sobre 3000 arreglos aleatorios con bloques de repetidos, sin una sola diferencia, mas los tres casos del sample (1, 4, 3). OJO CON LA ENTRADA: los casos vienen uno tras otro y se acaban con una línea con un 0 solo, así que el while lee n y corta cuando es cero, sin leer q en ese ultimo.

```

1 #include <bits/stdc++.h>
2 using namespace std;

```

```

3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int n, q;
9
10    while (cin >> n && n != 0) {
11        cin >> q;
12
13        vector<int> a(n + 1);
14        for (int i = 1; i <= n; i++) cin >> a[i];
15
16        // Como el arreglo viene ordenado, los iguales quedan en BLOQUES
17        // contiguos: se guarda a que bloque pertenece cada posición, y donde
18        // empieza y termina cada bloque.
19        vector<int> bloque(n + 1, ini(n + 2), fin(n + 2), tam(n + 2);
20
21        int b = 0;
22
23        for (int i = 1; i <= n; i++) {
24            if (i == 1 || a[i] != a[i - 1]) {
25                b++;
26                ini[b] = i;
27            }
28            bloque[i] = b;
29            fin[b] = i;
30        }
31
32        for (int t = 1; t <= b; t++) tam[t] = fin[t] - ini[t] + 1;
33
34        // Sparse table de MAXIMOS sobre los tamanos de bloque.
35        int K = 1;
36        while ((1 << K) <= b) K++;
37
38        vector<vector<int>> st(K, vector<int>(b + 1, 0));
39
40        for (int i = 1; i <= b; i++) st[0][i] = tam[i];
41
42        for (int k = 1; k < K; k++) {
43            int len = 1 << k;
44            for (int i = 1; i + len - 1 <= b; i++)
45                st[k][i] = max(st[k - 1][i], st[k - 1][i + len / 2]);
46        }
47
48        // Maximo de tam[1..r]; los dos bloques se solapan y no pasa nada,
49        // porque el maximo es idempotente.
50        auto maxRango = [&](int l, int r) {
51            if (l > r) return 0;
52            int k = 31 - __builtin_clz(r - l + 1);
53            return max(st[k][l], st[k][r - (1 << k) + 1]);
54        };
55
56        while (q--) {
57            int i, j;
58            cin >> i >> j;
59
60            int bi = bloque[i], bj = bloque[j];
61
62            // Todo el rango cae dentro de un mismo bloque.
63            if (bi == bj) {
64                cout << j - i + 1 << '\n';
65                continue;
66            }
67
68            int izq = fin[bi] - i + 1; // cola del bloque de i
69            int der = j - ini[bj] + 1; // cabeza del bloque de j
70            int medio = maxRango(bi + 1, bj - 1);
71
72            cout << max(medio, max(izq, der)) << '\n';
73        }
74    }
75    return 0;

```

```
76 }
```

1.21 Sqrt Decomposition - Holes

C++11 Training_Comp_2020/Day_4/K - Holes.cpp

Resuelve "Holes" (Day 4, problema K - Contest 04 [Avanzados]): hay N agujeros en fila, cada uno con una potencia que hace saltar la bola a la posición $i+a_i$, y hay que soportar M operaciones que cambian la potencia de un agujero o tiran una bola por un agujero, reportando cuantos saltos da y desde que agujero sale de la fila. Divide la fila en bloques de tamaño $\sqrt{N}$ y precalcula por bloque, para cada posición, a donde sale del bloque (`jmp_`), cuantos saltos (`cnt_`) y desde que agujero deja la fila si aplica (`last_`); una actualización solo recalcula su bloque (`recomputeBlock`) y una consulta encadena esos saltos precomputados bloque a bloque en $O(\sqrt{N})$ por operación.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 static char buf_[1 << 22];
5 static int bufLen_, bufPos_;
6 inline int gc() {
7     if (bufPos_ == bufLen_) { bufLen_ = fread(buf_, 1, sizeof(buf_), stdin); bufPos_ = 0; if (!bufLen_) return -1; }
8     return buf_[bufPos_++];
9 }
10 inline int readInt() {
11     int c = gc();
12     while (c != -1 && (c < '0' || c > '9') && c != '-') c = gc();
13     bool neg = false; if (c == '-') { neg = true; c = gc(); }
14     int x = 0;
15     while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = gc(); }
16     return neg ? -x : x;
17 }
18
19 int N, M;
20 vector<int> a_; // power of each hole, 1-indexed
21 vector<int> jmp_; // first position outside the hole's block reached (or N+1 sentinel = exits the row)
22 vector<int> cnt_; // number of individual jumps taken to reach jmp_[i]
23 vector<int> last_; // valid only when jmp_[i] == N+1: last hole visited before leaving the row
24 int blockSize;
25
26 void recomputeBlock(int p) {
27     int blockIdx = (p - 1) / blockSize;
28     int start = blockIdx * blockSize + 1;
29     int end = min(N, start + blockSize - 1);
30     for (int i = end; i >= start; i--) {
31         int nxt = i + a_[i];
32         if (nxt > N) {
33             jmp_[i] = N + 1;
34             cnt_[i] = 1;
35             last_[i] = i;
36         } else if (nxt > end) {
37             // leaves the current block to a later block (nxt > i always since a_[i] >= 1)
38             jmp_[i] = nxt;
39             cnt_[i] = 1;
40             // last_[i] intentionally left as-is; never read while jmp_[i] <= N
41         } else {
42             jmp_[i] = jmp_[nxt];
43             cnt_[i] = cnt_[nxt] + 1;
44             last_[i] = last_[nxt];
45         }
46     }
47 }
48
49 int main() {
50     N = readInt();
51     M = readInt();
52     a_.assign(N + 2, 0);
53     jmp_.assign(N + 2, 0);
54     cnt_.assign(N + 2, 0);
55     last_.assign(N + 2, 0);
56
57     for (int i = 1; i <= N; i++) a_[i] = readInt();
58
59     blockSize = max(1, (int)sqrt((double)N));
60 }
```

```
61 for (int start = 1; start <= N; start += blockSize) {
62     recomputeBlock(start);
63 }
64
65 string out;
66 out.reserve(M * 12);
67 char tmp[32];
68
69 for (int q = 0; q < M; q++) {
70     int type = readInt();
71     if (type == 0) {
72         int pos = readInt(), val = readInt();
73         a_[pos] = val;
74         recomputeBlock(pos);
75     } else {
76         int pos = readInt();
77         long long jumps = 0;
78         int cur = pos;
79         int lastHole = -1;
80         while (cur <= N) {
81             jumps += cnt_[cur];
82             if (jmp_[cur] > N) {
83                 lastHole = last_[cur];
84             }
85             cur = jmp_[cur];
86         }
87         int len = sprintf(tmp, "%d %lld\n", lastHole, jumps);
88         out.append(tmp, len);
89     }
90 }
91 fwrite(out.data(), 1, out.size(), stdout);
92 return 0;
93 }
```

1.22 Tree Construction - Binary Tree

C++11 LeetCode/binaryTree.cpp

Resuelve "Create Binary Tree From Descriptions": dado un arreglo de triplas `[parent, child, isLeft]`, construye el árbol enlazando cada hijo a su padre y devuelve la raíz (el único nodo que nunca aparece como hijo).

```
1 #include <bits/stdc++.h>;
2
3 using namespace std;
4
5 class Solution {
6 public:
7     struct TreeNode {
8         int val;
9         TreeNode *left;
10        TreeNode *right;
11        TreeNode() : val(0), left(nullptr), right(nullptr) {}
12        TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
13        TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
14    };
15
16    TreeNode* createBinaryTree(vector<vector<int>>& descriptions) {
17        unordered_map<int, TreeNode*> nodes;
18        unordered_set<int> children;
19
20        for (auto &d : descriptions) {
21            int parent = d[0];
22            int child = d[1];
23            int isLeft = d[2];
24
25            if (!nodes.count(parent))
26                nodes[parent] = new TreeNode(parent);
27
28            if (!nodes.count(child))
29                nodes[child] = new TreeNode(child);
30
31            if (isLeft)
32                nodes[parent]->left = nodes[child];
33            else
34                nodes[parent]->right = nodes[child];
35        }
```

```

35
36     children.insert(child);
37 }
38
39 for (auto &d : descriptions) {
40     int parent = d[0];
41     if (!children.count(parent))
42         return nodes[parent];
43 }
44
45 return nullptr;
46 }
47 };

```

2 - Dynamic Programming

2.1 Binary Search - Patches

C++\Training_Camp_2026\Day 2\ - Patches.py

Resuelve "Patches" (Day 2, problema I - Maratona de Programacao da SBC 2013): una llanta de circunferencia C tiene N agujeros a distancias F_i desde una marca de referencia; hay dos tipos de parche, de largo T_1 y T_2 , y cada parche cubre de forma continua todos los agujeros que caigan dentro de su longitud. Hay que cubrir todos los agujeros usando el menor largo total de parches. $rec(voy)$ es una recursion memoizada sobre el indice del siguiente agujero sin cubrir: prueba colocar un parche de largo t_1 o de largo t_2 empezando en $l[voy]$, usa $bisect_right$ para hallar en $O(\log N)$ el primer agujero que ya queda fuera del parche, y se queda con el mínimo entre ambas opciones (largo del parche mas lo que falte cubrir despues, $rec(i1)$ o $rec(i2)$).

```

1 from bisect import bisect_right
2
3 memo = {}
4
5 def rec(voy):
6     if voy == n:
7         return 0
8
9     if voy in memo:
10        return memo[voy]
11
12    i1 = bisect_right(l, l[voy] + t1)
13    i2 = bisect_right(l, l[voy] + t2)
14
15    ans = min(
16        t1 + rec(i1),
17        t2 + rec(i2)
18    )
19
20    memo[voy] = ans
21    return ans
22
23
24 n, c, t1, t2 = map(int, input().split())
25 l = list(map(int, input().split()))
26
27 print(rec(0))

```

2.2 Bitmask DP - Music Festival

C++\Training_Camp_2026\Day 8\F - Music Festival.py

Resuelve "Music Festival" (Day 8, problema F - Contest 08 [Avanzados]): hay $N \leq 10$ escenarios, cada uno con varios shows con horario [inicio,fin) y una cantidad de canciones conocidas del artista; hay que elegir un show por cada escenario (viendolo completo, sin faltar a ningun escenario) tal que los horarios elegidos no se superpongan entre sí, maximizando la suma de canciones conocidas, o -1 si no hay forma valida. Ordena todos los shows por inicio, precalcula con $bisect_left$ el siguiente show compatible (next_show) tras terminar cada uno, y usa $dp[i][mask] =$ mejor puntaje considerando shows desde el indice i en adelante con "mask" marcando que escenarios ya quedaron cubiertos, transicionando a $next_show[i]$ al tomar el show i (o saltandolo).

```

1 from bisect import bisect_left
2
3 n = int(input())
4
5 shows = []
6
7 for stage in range(n):
8     data = list(map(int, input().split()))

```

```

9     m = data[0]
10    for i in range(m):
11        s = data[1 + 3 * i]
12        e = data[2 + 3 * i]
13        p = data[3 + 3 * i]
14        shows.append((s, e, stage, p))
15
16    shows.sort()
17
18    m = len(shows)
19
20    starts = [x[0] for x in shows]
21
22    next_show = [0] * m
23    for i in range(m):
24        next_show[i] = bisect_left(starts, shows[i][1])
25
26    FULL = (1 << n) - 1
27    NEG = -10**18
28
29    dp = [[NEG] * (1 << n) for _ in range(m + 1)]
30    dp[m][FULL] = 0
31
32    for i in range(m - 1, -1, -1):
33        for mask in range(1 << n):
34            dp[i][mask] = dp[i + 1][mask]
35
36        _, _, stage, songs = shows[i]
37        bit = 1 << stage
38
39        for mask in range(1 << n):
40            if dp[next_show[i]][mask | bit] != NEG:
41                dp[i][mask] = max(
42                    dp[i][mask],
43                    songs + dp[next_show[i]][mask | bit]
44                )
45
46    ans = dp[0][0]
47
48    print(ans if ans != NEG else -1)

```

2.3 Bitmask SOS DP - El Juego Del Calamar

C++\Training_Camp_2026\Day 9\D - El Juego del Calamar.cpp

Resuelve "El Juego del Calamar" (Day 9, problema D - Contest 08 [Inicial & Avanzado]): una torre de n pisos con $k \leq 15$ habitaciones por piso (capacidad $C[i][j]$) y escaleras que solo suben de piso i a $i+1$; en cada turno se descartan jugadores sobrantes por capacidad y luego los sobrevivientes deben subir por una escalera o son eliminados. Para cada turno t (de forma independiente) se busca el maximo de jugadores que pueden seguir vivos. Como k es chico, representa por bitmask que habitaciones del piso actual se usan, y con una transformada tipo SOS (Sum over Subsets, arreglos $r/h/g/Nmask$) calcula en $O(2^k * k)$ el cuello de botella (min-cut) entre el conjunto de habitaciones alcanzadas y las capacidades del siguiente piso, propagando piso a piso hasta el turno t .

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4 const ll INF = (1ll)4e18;
5
6 int main(){
7     int n, k, m;
8     scanf("%d %d %d", &n, &k, &m);
9
10    // C[i][j]: piso i (1..n), habitacion j (0..k-1)
11    vector<vector<ll>> C(n+1, vector<ll>(k));
12    for(int i = 1; i <= n; i++)
13        for(int j = 0; j < k; j++)
14            scanf("%lld", &C[i][j]);
15
16    // adjOut[i][j]: mascara de habitaciones del piso i+1 alcanzables desde la habitacion j del pis
17    o i
18    vector<vector<int>> adjOut(n+1, vector<int>(k, 0));
19    for(int c = 0; c < m; c++){
20        int f, u, v;
21        scanf("%d %d %d", &f, &u, &v);

```

```

21     adjOut[f][u-1] |= (1 << (v-1));
22 }
23
24 int SZ = 1 << k;
25 int full = SZ - 1;
26
27 vector<ll> r(SZ), h(SZ), g(SZ);
28 vector<int> Nmask(SZ);
29
30 // r[mask] para el piso 1: suma de C[1][j] para j en mask (sin cuello de botella, fuente direct
31 a)
32 r[0] = 0;
33 for(int mask = 1; mask < SZ; mask++){
34     int lb = mask & (-mask);
35     int j = __builtin_ctz(lb);
36     r[mask] = r[mask ^ lb] + C[1][j];
37 }
38
39 vector<ll> ans(n+1);
40 ans[1] = r[full];
41
42 for(int i = 1; i <= n-1; i++){
43     // h[B] = r[complemento de B] = flujo maximo usando SOLO las habitaciones de B
44     for(int B = 0; B < SZ; B++){ h[B] = r[full ^ B];
45
46     // Nmask[B] = union de vecinos (piso i+1) de las habitaciones en B
47     Nmask[0] = 0;
48     for(int B = 1; B < SZ; B++){
49         int lb = B & (-B);
50         int j = __builtin_ctz(lb);
51         Nmask[B] = Nmask[B ^ lb] | adjOut[i][j];
52     }
53
54     // g[N] = mejor (min) h[B] entre los B cuya vecindad es exactamente N
55     fill(g.begin(), g.end(), INF);
56     for(int B = 0; B < SZ; B++){
57         int N = Nmask[B];
58         if(h[B] < g[N]) g[N] = h[B];
59     }
60
61     // transformada tipo SOS: de g a r_{i+1}, procesando cada bit (habitacion del piso i+1)
62     for(int mask = 0; mask < SZ; mask++){ r[mask] = g[mask];
63     for(int b = 0; b < k; b++){
64         int bit = 1 << b;
65         ll cap = C[i+1][b];
66         for(int mask = 0; mask < SZ; mask++){
67             if(mask & bit){
68                 int m0 = mask ^ bit;
69                 ll old0 = r[m0], old1 = r[mask];
70                 r[m0] = min(old0, old1);
71                 r[mask] = min(old0, old1 + cap);
72             }
73         }
74     }
75
76     ans[i+1] = r[full];
77 }
78
79 for(int t = 1; t <= n; t++){
80     printf("%lld%c", ans[t], t==n?'\\n':' ');
81 }
82
83 return 0;
84 }

```

2.4 Camino en Grilla con Balance de Parentesis - Has Valid Path

C++ LeetCodeHasValidPath.cpp

Resuelve "Check if There Is a Valid Parentheses String Path" (LeetCode 2267): en una grilla de '(' y ')' hay que decir si existe un camino de la esquina superior izquierda a la inferior derecha, moviéndose solo abajo o a la derecha, cuya cadena sea de parentesis valida. LA IDEA: del camino recorrido solo importa el BALANCE (+1 por '(', -1 por ')'), no la cadena entera. Dos caminos que llegan a la misma casilla con el mismo balance tienen exactamente el mismo futuro, asi que el estado es (i, j, balance) y se memoiza. Es la misma idea de fondo de la ficha "Parentesis Balanceados" de este cuaderno, llevada a una grilla. El balance nunca pasa del largo del camino, m+n-1, asi que la memo es m * n * (m+n): con la grilla maxima de 100 x 100 son unos 2 millones de estados, que caben de sobra. Un balance negativo se corta en el acto: un ')' de mas ya no lo arregla nada que venga despues. PODAS QUE NO HACEN FALTA PERO CORTAN TRABAJO: si el camino tiene largo impar (m+n-1 impar) la respuesta es false sin mirar nada; si el balance supera las casillas que faltan, (m-1-i) + (n-1-j), ya no se alcanza a cerrar; y la primera casilla tiene que ser '(' y la ultima ')'. OJO: la funcion valid y la cadena s que se arma en dfs no se usan para la respuesta, la memo sobre el balance ya decide todo. Se pueden borrar sin cambiar nada. Verificado con los dos ejemplos de LeetCode y contra la fuerza bruta de probar TODOS los caminos en 3000 grillas de hasta 6x6, sin un solo fallo. Una grilla de 99 x 100 tarda ~20 ms.

```

1  class Solution {
2  public:
3
4      bool valid(string &s) {
5          int balance = 0;
6
7          for (char c : s) {
8              if (c == '(')
9                  balance++;
10             else
11                 balance--;
12
13             if (balance < 0)
14                 return false;
15         }
16
17         return balance == 0;
18     }
19
20     vector<vector<vector<int>>> memo;
21
22     bool dfs(vector<vector<char>>& grid,
23             int i, int j,
24             int balance,
25             string &s) {
26
27         int m = grid.size();
28         int n = grid[0].size();
29
30         // Nunca podemos tener mas ')' que '('
31         if (balance < 0)
32             return false;
33
34         // Ya calculamos este estado
35         if (memo[i][j][balance] != -1)
36             return memo[i][j][balance];
37
38         // Llegamos al final
39         if (i == m - 1 && j == n - 1) {
40             return memo[i][j][balance] = (balance == 0);
41         }
42
43         // Abajo
44         if (i + 1 < m) {
45
46             char c = grid[i + 1][j];
47
48             s.push_back(c);
49
50             int newBalance = balance + (c == '(' ? 1 : -1);
51
52             if (dfs(grid, i + 1, j, newBalance, s))
53                 return memo[i][j][balance] = true;
54
55             s.pop_back();
56         }

```

```

57
58 // Derecha
59 if (j + 1 < n) {
60
61     char c = grid[i][j + 1];
62
63     s.push_back(c);
64
65     int newBalance = balance + (c == '(' ? 1 : -1);
66
67     if (dfs(grid, i, j + 1, newBalance, s))
68         return memo[i][j][balance] = true;
69
70     s.pop_back();
71 }
72
73 return memo[i][j][balance] = false;
74 }
75
76 bool isValidPath(vector<vector<char>>& grid) {
77
78     int m = grid.size();
79     int n = grid[0].size();
80
81     // La longitud maxima del string es m+n-1
82     int maxBalance = m + n;
83
84     memo.assign(
85         m,
86         vector<vector<int>>{
87             n,
88             vector<int>(maxBalance + 1, -1)
89         }
90     );
91
92     string s;
93     s.push_back(grid[0][0]);
94
95     int balance = (grid[0][0] == '(' ? 1 : -1);
96
97     return dfs(grid, 0, 0, balance, s);
98 }
99 };

```

2.5 Convex Hull Trick

C++(Notebook)Convex Hull Trick.cpp

Optimiza DP's de la forma $dp[i] = \min_j (m[j] * x[i] + b[j])$, es decir, cuando cada estado anterior aporta una recta y hay que evaluar la envolvente inferior en $x[i]$. Baja de $O(n^2)$ a $O(n)$ amortizado. Mantiene un deque con solo las rectas que llegan a ser minimas en algun punto: al insertar una recta nueva se descartan por atras las que quedaron "tapadas" (su punto de corte ya no aporta), y al consultar se descartan por adelante las que ya quedaron atras del x pedido. Requiere que las pendientes se inserten monotonas (decrecientes para minimo) y que las x de consulta vengan crecientes; si no se cumple, hay que usar Li Chao Tree o CHT dinamica.

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 struct CHT
6 {
7     struct L { long long m, b; double x; };
8     deque<L> dq;
9
10    double inter(const L &a, const L &c)
11    {
12        return double(c.b - a.b) / double(a.m - c.m);
13    }
14
15    void add(long long m, long long b)
16    {
17        L l = {m, b, 0};
18        while (dq.size() >= 2 && inter(dq[dq.size() - 2], dq.back()) >= inter(dq.back(), l))
19        {
20            dq.pop_back();

```

```

21        }
22        if (dq.empty()) l.x = -1e300;
23        else l.x = inter(dq.back(), l);
24        dq.push_back(l);
25    }
26
27    long long query(long long x)
28    {
29        while (dq.size() >= 2 && dq[1].x <= x)
30        {
31            dq.pop_front();
32        }
33        return dq.front().m * x + dq.front().b;
34    }
35 };
36
37 int main()
38 {
39     ios_base::sync_with_stdio(false);
40     cin.tie(nullptr);
41     return 0;
42 }

```

2.6 Digit DP

C++(Notebook)Digit DP.cpp

Cuenta cuantos numeros de 0 a X cumplen una propiedad definida sobre sus digitos (aqui: que la suma de digitos sea divisible por D). Se recorre el numero digito por digito llevando dos cosas en el estado: el acumulado de la propiedad (s) y el flag $tight$, que indica si el prefijo construido sigue pegado al prefijo de X . El flag $tight$ es la clave: si sigue activo, el digito actual solo puede llegar hasta el digito correspondiente de X ; si ya se rompio (se puso un digito menor), a partir de ahi se puede poner cualquier cosa de 0 a 9 y el subarbol se memoiza y se reutiliza. Para contar en un rango $[A,B]$ se calcula $solve(B) - solve(A-1)$.

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 long long dp[20][200][2];
6 string S;
7 int D;
8
9 long long go(int i, int s, int tight)
10 {
11     if (i == (int)S.size()) return (s % D == 0);
12
13     long long &r = dp[i][s][tight];
14     if (r != -1) return r;
15     r = 0;
16
17     int lim = 9;
18     if (tight) lim = S[i] - '0';
19
20     for (int d = 0; d <= lim; ++d)
21     {
22         r += go(i + 1, (s + d) % D, tight && d == lim);
23     }
24     return r;
25 }
26
27 long long solve_digitdp(long long X, int d)
28 {
29     S = to_string(X);
30     D = d;
31     memset(dp, -1, sizeof dp);
32     return go(0, 0, 1);
33 }
34
35 int main()
36 {
37     ios_base::sync_with_stdio(false);
38     cin.tie(nullptr);
39
40     long long A, B;
41     int d;
42     while (cin >> A >> B >> d)

```

```
43 {
44     cout << solve_digitdp(B, d) - solve_digitdp(A - 1, d) << "\n";
45 }
46
47 return 0;
48 }
```

2.7 Distancia de Damerau-Levenshtein

C++ NotebookDistancia de Damerau-Levenshtein.cpp

Minimo numero de operaciones para convertir a en b, con CUATRO operaciones de costo 1: insertar, borrar, sustituir e intercambiar dos caracteres ADYACENTES. Es la distancia de edicion de toda la vida mas la transposicion, que es justo lo que modela los errores de tipeo. OJO, que aqui esta la trampa: casi todo el mundo escribe la version "facil" (OSA, Optimal String Alignment), que es la tabla de Levenshtein con un if extra para el swap. Esa version prohíbe editar lo que quedo entre los dos caracteres intercambiados, y da respuestas mas grandes. El ejemplo que la separa es "ca" -> "abc": esta rutina responde 2 (intercambiar c y a, insertar la b), mientras que OSA responde 3. Si el problema pide la distancia REAL (la que cumple desigualdad triangular), toca esta version. Verificado contra busqueda exhaustiva. Los dos arreglos auxiliares son lo que permite la version irrestricta: da[c] = ultima FILA i donde aparecio la letra c en a (se actualiza al cerrar la fila i, asi que durante la fila i solo ve filas anteriores) db = ultima COLUMNA j de esta fila donde a[j] == b[j] (se lee antes de actualizarlo) Con k = da[b[j]] y l = db, el par (a[k], b[l]) es el candidato a intercambiar. Su costo es H[k][l] + (i-k-1) + 1 + (j-l-1): lo de antes, borrar los (i-k-1) sobrantes de a, pagar 1 por el intercambio, e insertar los (j-l-1) que faltan de b. Esa es la libertad que OSA no tiene. El desplazamiento en 1 no es capricho: el termino de intercambio necesita una fila y una columna "menos uno" donde caer cuando la letra nunca aparecio (k = 0) o no hubo match en la fila (l = 0). Esas casillas valen INF = n+m, mas que cualquier distancia posible, asi que el min las descarta solas y no hay que escribir ningun if. Costo O(n*m) tiempo y O(n*m) memoria. Si solo hace falta el numero y no reconstruir la edicion, se puede bajar a O(m) guardando tres filas, pero hay que arrastrar tambien la fila del intercambio, asi que rara vez vale la pena. LIMITES de esta implementacion: da[26] con b[j]-a' asume MINUSCULAS a-z; con mayusculas o digitos se sale del arreglo. Y H es 1005x1005 usando indices hasta n+1, o sea n, m <= 1003. Entrada: pares de palabras, una por linea, hasta la linea "***".

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 static int H[1005][1005];
5
6 int main() {
7     ios::sync_with_stdio(false);
8     cin.tie(nullptr);
9
10    string a, b;
11    while (cin >> a >> b) {
12        if (a == "*" && b == "*") break;
13
14        int n = a.size(), m = b.size();
15        int INF = n + m;
16
17        // H desplazada en 1: H[i+1][j+1] = distancia entre a[0..i] y b[0..j]
18        H[0][0] = INF;
19        for (int i = 0; i <= n; i++) { H[i + 1][0] = INF; H[i + 1][1] = i; }
20        for (int j = 0; j <= m; j++) { H[0][j + 1] = INF; H[1][j + 1] = j; }
21
22        int da[26] = {0}; // ultima fila donde aparecio cada letra en a
23
24        for (int i = 1; i <= n; i++) {
25            int db = 0; // ultima columna en esta fila donde a[i]==b[j]
26            for (int j = 1; j <= m; j++) {
27                int k = da[b[j] - 1] - 'a';
28                int l = db;
29                int cost = 1;
30                if (a[i - 1] == b[j - 1]) { cost = 0; db = j; }
31
32                H[i + 1][j + 1] = min({
33                    H[i][j] + cost, // sustituir/igual
34                    H[i + 1][j] + 1, // insertar
35                    H[i][j + 1] + 1, // borrar
36                    H[k][l] + (i - k - 1) + 1 + (j - l - 1) // intercambiar
37                });
38            }
39            da[a[i - 1] - 'a'] = i;
40        }
41
42        cout << H[n + 1][m + 1] << '\n';
43    }
```

```
44     return 0;
45 }
```

2.8 Divide and Conquer Optimization

C++ NotebookDivide and Conquer Optimization.cpp

Optimiza DP's por capas del tipo $dp[i][m] = \min_k (dp[i-1][k] + C(k,m))$, típicos de "partir un arreglo en i grupos". Baja cada capa de $O(n^2)$ a $O(n \log n)$. Se apoya en que el punto de corte optimo es monotono: si $opt[m]$ es el mejor k para m, entonces opt es no decreciente en m. Entonces se resuelve el m del medio buscando su optimo en el rango $[optl, optr]$, y con ese resultado se acota la busqueda de las dos mitades: la izquierda solo puede tener su optimo en $[optl, bestk]$ y la derecha en $[bestk, optr]$. Solo es valido si el costo C cumple la desigualdad cuadrangular (condicion de Monge).

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 const long long INF = (1LL << 62);
6
7 void compute(int l, int r, int optl, int optr,
8             const function<long long(int,int)>& C,
9             vector<long long>& prev, vector<long long>& cur, vector<int>& opt)
10 {
11     if (l > r) return;
12
13     int m = (l + r) / 2;
14     int bestk = optl;
15     long long best = INF;
16
17     for (int k = optl; k <= min(m - 1, optr); ++k)
18     {
19         long long v = prev[k] + C(k, m);
20         if (v < best)
21         {
22             best = v;
23             bestk = k;
24         }
25     }
26
27     cur[m] = best;
28     opt[m] = bestk;
29
30     compute(l, m - 1, optl, bestk, C, prev, cur, opt);
31     compute(m + 1, r, bestk, optr, C, prev, cur, opt);
32 }
33
34 int main()
35 {
36     ios_base::sync_with_stdio(false);
37     cin.tie(nullptr);
38     return 0;
39 }
```

2.9 DP sobre Intervalos con Bandera de Inversion - Artic Virus

C++ ICPC2024 - Artic Virus.cpp

Resuelve "Arctic Virus" (problema A, ICPC 2024): decidir si una cadena es un virus segun la gramatica $\phi ::= A \mid T \mid \phi C \mid A \phi \mid A \phi^{\wedge-1} \mid G \phi^{\wedge-1} C$, donde $\phi^{\wedge-1}$ es phi al revés. Responde simple (la cadena es A o T), mutation (es virus y mide 2 o mas) o doomed. OJO CON EL LARGO T: solo A y T son "simple". Una C o una G sola no la genera ninguna regla y es doomed. La primera version imprimia simple para cualquier letra sola; verificada la corregida contra el lenguaje generado desde la gramatica, en todas las cadenas de largo 1 a 8. Las reglas se deshacen de afuera hacia adentro: mirando la primera y la ultima letra se sabe que regla pudo haber sido la ultima, se quita lo que agrega, y se sigue con lo de adentro. EL TRUCO ES NO INVERTIR NUNCA LA CADENA. Dos de las reglas dejan el interior al revés, y copiar e invertir substrings seria caro y ademas daria estados imposibles de memoizar. En vez de eso el estado es (l, r, rev): el substring s[l..r], leído al derecho o al revés segun la bandera. Invertir es solo cambiar la bandera, y rev decide cual extremo es el "primero" y cual el "ultimo". Asi todos los estados posibles son $O(n^2 * 2)$, cada uno con trabajo $O(1)$. La memo va en un arreglo memo[l][r][rev] con -1 para "sin calcular". La primera version usaba map<tuple>, y con n = 1000 costaba entre 230 y 440 ms por caso; con varios casos grandes en la misma entrada eso ya arriesga el limite de tiempo. CUANDO USAR ESTE ESTADO: cualquier gramatica o juego sobre una cadena donde las operaciones quitan letras de las puntas; y la bandera de inversion en particular siempre que una operacion "de la vuelta" a la cadena.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 string s;
```

```

5 int n;
6
7 // memo[l][r][rev]: -1 = sin calcular, 0 = no, 1 = si. Un arreglo en vez de
8 // map<tuple>; con n = 1000 el map costaba cientos de ms por caso.
9 signed char memo[1001][1001][2];
10
11 bool solve(int l, int r, bool rev) {
12     if (l > r) return false;
13
14     signed char &res = memo[l][r][rev];
15
16     if (res != -1)
17         return res;
18
19     int len = r - l + 1;
20
21     // Llegamos a una configuracion simple
22     if (len == 1) {
23         char c = rev ? s[r] : s[l];
24         res = (c == 'A' || c == 'T');
25         return res;
26     }
27
28     // Obtener extremos de la cadena actual
29     char first = rev ? s[r] : s[l];
30     char last = rev ? s[l] : s[r];
31
32     bool ans = false;
33
34     // phiC
35     // Si termina en C, quitamos el ultimo caracter
36     if (last == 'C') {
37         if (!rev)
38             ans |= solve(l, r - 1, false);
39         else
40             ans |= solve(l + 1, r, true);
41     }
42
43     // Aphi
44     // Si comienza en A, quitamos el primero
45     if (first == 'A') {
46         if (!rev)
47             ans |= solve(l + 1, r, false);
48         else
49             ans |= solve(l, r - 1, true);
50     }
51
52     // Aphi^-1
53     // Quitamos A del inicio y luego invertimos
54     if (first == 'A') {
55         if (!rev)
56             ans |= solve(l + 1, r, true);
57         else
58             ans |= solve(l, r - 1, false);
59     }
60
61     // Gphi^-1C
62     // Debe comenzar en G y terminar en C
63     if (first == 'G' && last == 'C' && len >= 3) {
64         if (!rev)
65             ans |= solve(l + 1, r - 1, true);
66         else
67             ans |= solve(l + 1, r - 1, false);
68     }
69
70     res = ans;
71     return res;
72 }
73
74 int main() {
75     ios::sync_with_stdio(false);
76     cin.tie(nullptr);
77

```

```

78     while (cin >> n >> s) {
79
80         for (int l = 0; l < n; l++)
81             for (int r = 0; r < n; r++)
82                 memo[l][r][0] = memo[l][r][1] = -1;
83
84         // Largo 1: solo A y T son la etapa simple; C y G no son virus.
85         if (n == 1) {
86             if (s[0] == 'A' || s[0] == 'T')
87                 cout << "simple\n";
88             else
89                 cout << "doomed\n";
90         }
91         else {
92             bool ok = solve(0, n - 1, false);
93
94             cout << (ok ? "mutation\n" : "doomed\n");
95         }
96     }
97
98     return 0;
99 }

```

2.10 Interval Scheduling - Yuyuan Market

C++ Training_Camp_2020 Day 10/A - Yuyuan Market.cpp

Resuelve "Yuyuan Market" (Day 10, problema A - Contest 09 [Avanzados]): una fila de $2N$ jianzhi con símbolos repetidos (cada símbolo aparece dos veces), donde solo se pueden comprar pares (posición i , posición $j > i$) del mismo símbolo, sin cruzar hacia atrás una vez elegido un par. Hay que maximizar la cantidad de pares comprados y, entre los que logran el máximo, minimizar la suma de distancias $j-i$. Cada símbolo define un intervalo $[L[v], R[v]]$ (primera y segunda aparición); es DP de intervalos sobre la posición p : $cnt[p]/len[p]$ guardan el mejor (cantidad, costo) usando las primeras p posiciones, y $endAtL[p]$ permite extender el DP tomando el intervalo que termina en p desde el estado anterior a su inicio.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 static char buf_[1 << 22];
5 static int buflen_, bufPos_;
6 inline int gc() {
7     if (bufPos_ == buflen_) { buflen_ = fread(buf_, 1, sizeof(buf_), stdin); bufPos_ = 0; if (!buflen_) return -1; }
8     return buf_[bufPos_++];
9 }
10 inline int readInt() {
11     int c = gc();
12     while (c != -1 && (c < '0' || c > '9') && c != '-') c = gc();
13     bool neg = false; if (c == '-') { neg = true; c = gc(); }
14     int x = 0;
15     while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = gc(); }
16     return neg ? -x : x;
17 }
18
19 int main() {
20     int N = readInt();
21     int M = 2 * N;
22     vector<int> L(N + 1, 0), R(N + 1, 0);
23     vector<bool> seen(N + 1, false);
24     for (int pos = 1; pos <= M; pos++) {
25         int v = readInt();
26         if (!seen[v]) { L[v] = pos; seen[v] = true; }
27         else R[v] = pos;
28     }
29
30     vector<int> endAtL(M + 1, -1); // endAtL[p] = L of the symbol whose R equals p, or -1
31     for (int v = 1; v <= N; v++) endAtL[R[v]] = L[v];
32
33     vector<int> cnt(M + 1, 0);
34     vector<long long> len(M + 1, 0);
35     for (int p = 1; p <= M; p++) {
36         cnt[p] = cnt[p - 1];
37         len[p] = len[p - 1];
38         if (endAtL[p] != -1) {
39             int Lv = endAtL[p];
40             int candCnt = cnt[Lv - 1] + 1;

```

```

41     long long candLen = len[Lv - 1] + (long long)(p - Lv);
42     if (candCnt > cnt[p] || (candCnt == cnt[p] && candLen < len[p])) {
43         cnt[p] = candCnt;
44         len[p] = candLen;
45     }
46 }
47 }
48
49 printf("%d %lld\n", cnt[M], len[M]);
50 return 0;
51 }

```

2.11 Prefix Sums - Zig Zag Array

C++ - LeetCode ZigZagArray.cpp

Cuenta los arreglos "zigzag" de longitud n con valores en $[l, r]$ (cada elemento alterna entre mayor y menor que su vecino), modulo $1e9+7$. DP sobre la longitud manteniendo, para cada valor, cuantas secuencias terminan subiendo (up) o bajando (down); las transiciones se aceleran con sumas de prefijos y sufijos en $O(n * (r-l))$.

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 class Solution {
6 public:
7     static constexpr int MOD = 1'000'000'007;
8
9     int zigZagArrays(int n, int l, int r) {
10         int m = r - l + 1;
11
12         vector<long long> up(m), down(m);
13         vector<long long> pref(m + 1);
14         vector<long long> suff(m + 1);
15         vector<long long> newUp(m);
16         vector<long long> newDown(m);
17
18         for (int i = 0; i < m; i++) {
19             up[i] = i;
20             down[i] = m - 1 - i;
21         }
22
23         for (int len = 3; len <= n; len++) {
24             pref[0] = 0;
25             for (int i = 0; i < m; i++) {
26                 pref[i + 1] = (pref[i] + down[i]) % MOD;
27             }
28
29             suff[m] = 0;
30             for (int i = m - 1; i >= 0; i--) {
31                 suff[i] = (suff[i + 1] + up[i]) % MOD;
32             }
33
34             for (int i = 0; i < m; i++) {
35                 newUp[i] = pref[i];
36                 newDown[i] = suff[i + 1];
37             }
38
39             up.swap(newUp);
40             down.swap(newDown);
41         }
42
43         long long ans = 0;
44         for (int i = 0; i < m; i++) {
45             ans = (ans + up[i] + down[i]) % MOD;
46         }
47
48         return (int)ans;
49     }
50 };

```

2.12 SOS (Sum over Subsets) + Inclusion-Exclusion - How Many Teams

C++ - training_camp_2020/day04 - How many teams.cpp

Resuelve "How many teams?" (Day 6, problema H - Maratona SBC de Programacao 2025): N estudiantes tienen cada uno un subconjunto de K habilidades (representado como bitmask), un equipo de 3 estudiantes tiene como conjunto de habilidades la union de las de sus miembros, y se dan M subconjuntos especiales; para cada uno hay que contar cuantos equipos de 3 estudiantes distintos tienen union de habilidades exactamente igual a ese subconjunto. El código cuenta primero $f[mask]$ = cantidad de estudiantes con esa máscara exacta, aplica SOS (sum over subsets) para que $f[mask]$ pase a contar estudiantes cuya máscara es submáscara de $mask$, calcula $h[mask] = C(f[mask], 3)$ (tripletas cuya union esta contenida en $mask$), y luego aplica Mobius / inclusion-exclusion (SOS inverso) sobre $h[]$ para recuperar el conteo exacto por union, de modo que cada consulta se responde en $O(1)$.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 static char buf_[1 << 22];
5 static int bufLen_, bufPos_;
6 inline int gc() {
7     if (bufPos_ == bufLen_) { bufLen_ = fread(buf_, 1, sizeof(buf_), stdin); bufPos_ = 0; if (!bufLen_) return -1; }
8     return buf_[bufPos_++];
9 }
10 inline int readInt() {
11     int c = gc();
12     while (c != -1 && (c < '0' || c > '9') && c != '-') c = gc();
13     bool neg = false; if (c == '-') { neg = true; c = gc(); }
14     int x = 0;
15     while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = gc(); }
16     return neg ? -x : x;
17 }
18 // Reads a binary string of known length K, skipping any non-'0'/'1' characters first,
19 // and returns the mask with position 1 (first character) as bit 0 (matches problem's
20 // convention, verified against both worked examples).
21 inline int readMaskFixedLen(int K) {
22     int c = gc();
23     while (c != '0' && c != '1') c = gc();
24     int mask = 0;
25     for (int pos = 0; pos < K; pos++) {
26         if (c == '1') mask |= (1 << pos);
27         if (pos + 1 < K) c = gc();
28     }
29     return mask;
30 }
31
32 int main() {
33     int N = readInt(), K = readInt();
34     int full = 1 << K;
35     vector<long long> f(full, 0); // will hold cnt[] then get turned into SOS sums
36
37     for (int i = 0; i < N; i++) {
38         int mask = readMaskFixedLen(K);
39         f[mask]++;
40     }
41
42     // SOS: f[mask] = number of students whose mask is a submask of `mask`
43     for (int bit = 0; bit < K; bit++) {
44         int b = 1 << bit;
45         for (int mask = 0; mask < full; mask++) {
46             if (mask & b) f[mask] += f[mask ^ b];
47         }
48     }
49
50     // g[mask] = C(f[mask], 3)
51     vector<long long> h(full, 0);
52     for (int mask = 0; mask < full; mask++) {
53         long long v = f[mask];
54         h[mask] = (v >= 3) ? (v * (v - 1) * (v - 2) / 6) : 0;
55     }
56
57     // Mobius inversion (inclusion-exclusion) to get exact-union counts
58     for (int bit = 0; bit < K; bit++) {
59         int b = 1 << bit;
60         for (int mask = 0; mask < full; mask++) {

```

```

61         if (mask & b) h[mask] -= h[mask ^ b];
62     }
63 }
64
65 int M = readInt();
66 string out;
67 out.reserve((size_t)M * 8);
68 char tmp[32];
69 for (int q = 0; q < M; q++) {
70     int mask = readMaskFixedLen(K);
71     int len = sprintf(tmp, "%lld\n", h[mask]);
72     out.append(tmp, len);
73 }
74 fwrite(out.data(), 1, out.size(), stdout);
75 return 0;
76 }

```

2.13 Steiner Tree - Joining Capitals

C++\Training_Camp_2026\Day 8\ - Joining Capitals.cpp

Resuelve "Joining Capitals" (Day 8, problema J - Contest 08 [Avanzados]): un reino tiene N ciudades, K de ellas capitales; hay que construir líneas de transmisión (costo = distancia euclidiana) de forma que todas las capitales queden conectadas entre sí por un camino, cada capital este conectada directamente a una sola otra ciudad, y el costo total sea mínimo. Como cada capital debe tener grado 1, el árbol de Steiner solo se ramifica en las ciudades no- capitales: $dp[mask][idx]$ (algoritmo de Dreyfus-Wagner) guarda el costo mínimo de conectar el subconjunto "mask" de capitales usando como punto de unión la no-capital "nonCap[idx]", combinando submáscaras disjuntas (paso MERGE) y extendiendo con un Dijkstra $O(M^2)$ entre no- capitales (paso EXTEND) para propagar el costo por las ciudades intermedias.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int N, K;
6     cin >> N >> K;
7     vector<double> X(N), Y(N);
8     for (int i = 0; i < N; i++) cin >> X[i] >> Y[i];
9
10    vector<vector<double>> dist(N, vector<double>(N, 0.0));
11    for (int i = 0; i < N; i++) {
12        for (int j = 0; j < N; j++) {
13            double dx = X[i] - X[j], dy = Y[i] - Y[j];
14            dist[i][j] = sqrt(dx * dx + dy * dy);
15        }
16    }
17
18    vector<int> nonCap;
19    for (int v = K; v < N; v++) nonCap.push_back(v);
20    int M = (int)nonCap.size();
21
22    const double INF = 1e18;
23    int fullMask = (1 << K) - 1;
24    // dp[mask][idx] : idx indexes into nonCap (0..M-1)
25    vector<vector<double>> dp(1 << K, vector<double>(M, INF));
26
27    // base case: each capital connects directly to each non-capital
28    for (int i = 0; i < K; i++) {
29        for (int idx = 0; idx < M; idx++) {
30            dp[1 << i][idx] = dist[i][nonCap[idx]];
31        }
32    }
33
34    for (int mask = 1; mask <= fullMask; mask++) {
35        // MERGE step: combine two disjoint submasks meeting at the same non-capital
36        for (int sub = (mask - 1) & mask; sub > 0; sub = (sub - 1) & mask) {
37            int other = mask ^ sub;
38            if (sub < other) {
39                for (int idx = 0; idx < M; idx++) {
40                    double cand = dp[sub][idx] + dp[other][idx];
41                    if (cand < dp[mask][idx]) dp[mask][idx] = cand;
42                }
43            }
44        }
45    }

```

```

46    // EXTEND step: simple  $O(M^2)$  Dijkstra among non-capitals only
47    vector<bool> done(M, false);
48    for (int iter = 0; iter < M; iter++) {
49        int u = -1;
50        double best = INF;
51        for (int idx = 0; idx < M; idx++) {
52            if (!done[idx] && dp[mask][idx] < best) { best = dp[mask][idx]; u = idx; }
53        }
54        if (u == -1) break;
55        done[u] = true;
56        for (int w = 0; w < M; w++) {
57            if (!done[w]) {
58                double cand = dp[mask][u] + dist[nonCap[u]][nonCap[w]];
59                if (cand < dp[mask][w]) dp[mask][w] = cand;
60            }
61        }
62    }
63 }
64
65 double answer = INF;
66 for (int idx = 0; idx < M; idx++) answer = min(answer, dp[fullMask][idx]);
67
68 printf("%.5f\n", answer);
69 return 0;
70 }

```

2.14 Subset Sum (Knapsack) - Karamell

C++\Training_Camp_2026\Day 3\K - Karamell.py

Resuelve "Karamell" (Day 3, problema K - Maratona SBC de Programacao 2024): Alice y Bob reciben N bolsas de caramelos que se reparten en el orden dado, entregando cada bolsa a quien tenga menos caramelos en ese momento (en caso de empate, a Alice); hay que encontrar un orden de las bolsas tal que ambos terminen con la misma cantidad de caramelos, o determinar que es imposible. El código primero descarta el caso de suma total impar. Luego usa una DP de subset-sum $dp[i][s]$, con bytearray por fila para ahorrar memoria) para determinar si existe un subconjunto de bolsas cuya suma sea la mitad del total, y reconstruye ese subconjunto (sub). Finalmente separa las bolsas en dos grupos (alice/bob) según pertenezcan o no a ese subconjunto y las intercala de forma greedy, dándole siempre la siguiente bolsa al que menos lleva acumulado, reproduciendo la regla de reparto del enunciado para garantizar el empate final.

```

1 import sys
2 input = sys.stdin.readline
3
4 n = int(input())
5 ka = sorted(map(int, input().split()))
6 suma = sum(ka)
7
8 if suma % 2 != 0:
9     print(-1)
10 else:
11     mit = suma // 2
12
13     # ---- subset-sum DP: reemplaza el rec exponencial ----
14     # dp[i][s] = con los primeros i elementos se puede formar la suma s
15     dp = [bytearray(mit + 1) for _ in range(n + 1)]
16     dp[0][0] = 1
17     for i in range(1, n + 1):
18         w = ka[i - 1]
19         cur = dp[i]
20         prev = dp[i - 1]
21         for s in range(mit + 1):
22             if prev[s] or (s >= w and prev[s - w]):
23                 cur[s] = 1
24
25     if not dp[n][mit]:
26         print(-1)
27     else:
28         # reconstruyo el subconjunto que suma mit (queda como sub ordenado, igual que f[0])
29         sub = []
30         s = mit
31         for i in range(n, 0, -1):
32             w = ka[i - 1]
33             if s >= w and dp[i - 1][s - w]: # este elemento entra al subconjunto
34                 sub.append(w)
35                 s -= w

```

```
36 sub.reverse() # ascendente, como lo esperaba tu código
37
38 # ---- de aquí en adelante: TU MISMA reconstrucción ----
39 alice = []
40 bob = []
41 pos = 0
42 for j in range(len(ka)):
43     if pos < len(sub) and sub[pos] == ka[j]:
44         alice.append(j)
45         pos += 1
46     else:
47         bob.append(j)
48
49 alicel = 0
50 bob1 = 0
51 posA = 0
52 posB = len(bob) - 1
53 imp = []
54 for _ in range(len(ka)):
55     if alicel < bob1:
56         v = ka[alice[posA]]; posA += 1
57         alicel += v
58     else:
59         v = ka[bob[posB]]; posB -= 1
60         bob1 += v
61     imp.append(str(v))
62 print(" ".join(imp))
```

2.15 Valor Esperado - Revenge Of The Salary Of AtCoder Inc

C++11 Training_Camp_2020/Day 10/N - Revenge of The Salary of AtCoder Inc.cpp

Resuelve "Revenge of The Salary of AtCoder Inc." (Day 10, problema N - Contest 09 [Avanzados]): con un dado de N caras y $x=0$, en cada ronda se tira el dado (resultado y); si $x < y$ se cobra A_y yen y x pasa a valer y , si no el proceso termina. Hay que hallar el valor esperado del salario total, modulo 998244353. Define $E[i]$ = valor esperado de pago futuro estando en $x=i$; por linealidad de la esperanza, $E[i] = (1/N) \cdot \sum_{y>i} (A_y + E[y])$, ya que tirar $y < i$ termina el proceso sin pago adicional. El código calcula esto de atrás hacia adelante (i de N a 1) manteniendo $S = \sum_{y>i} A_y + E[y]$ para $y > i$, de modo que $E[i] = S \cdot \text{invN}$ (invN es el inverso modular de N via Fermat); el resultado final es $E[0] = S \cdot \text{invN}$ tras procesar todo el arreglo.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5 const ll MOD = 998244353;
6
7 ll binpow(ll a, ll e) {
8     ll r = 1;
9     while (e) {
10         if (e & 1) r = r * a % MOD;
11         a = a * a % MOD;
12         e >>= 1;
13     }
14     return r;
15 }
16
17 ll inv(ll x) {
18     return binpow(x, MOD - 2);
19 }
20
21 int main() {
22     ios::sync_with_stdio(false);
23     cin.tie(nullptr);
24
25     int N;
26     cin >> N;
27
28     vector<ll> A(N + 1);
29     for (int i = 1; i <= N; i++)
30         cin >> A[i];
31
32     ll invN = inv(N);
33
34     vector<ll> E(N + 2, 0);
35
36     ll S = 0;
```

```
37
38     for (int i = N; i >= 1; i--) {
39         E[i] = S * invN % MOD;
40         S = (S + A[i] + E[i]) % MOD;
41     }
42
43     ll ans = S * invN % MOD;
44
45     cout << ans << '\n';
46 }
```

3 - Graph

3.1 2-SAT (Kosaraju) - Signal Coverage

C++11 ICPC2020 - Signal Coverage.cpp

Resuelve "Signal Coverage" (problema G, ICPC 2024). Cada antena se instala o no; hay parejas de las que AL MENOS UNA debe instalarse, y dos antenas que coinciden en tiempo (sus intervalos se cruzan) y en espacio (sus círculos se tocan) NO pueden instalarse las dos. El código decide si se puede y da una asignación. Todas las restricciones son de dos variables, y eso es 2-SAT; al menos una: $x \vee y$ no las dos: $\neg x \vee \neg y$ Cada clausula ($a \vee b$) se vuelve dos implicaciones, $\neg a \rightarrow b$ y $\neg b \rightarrow a$. Con la codificación $2i$ es instalada, $2i+1$ no instalada, negar es un xor con 1. Se sacan las componentes fuertemente conexas con Kosaraju. Hay solución si y solo si ninguna variable queda en la misma componente que su negación (eso sería x implica $\neg x$ implica x). LA TRAMPA DE LA ASIGNACIÓN: Kosaraju numera las componentes en ORDEN TOPOLOGICO, y la regla es x verdadera si $\text{comp}[x] > \text{comp}[\neg x]$. Con Tarjan la numeración sale al revés y la desigualdad se voltea. Mezclar las dos da asignaciones que violan restricciones sin avisar. El choque de círculos va en enteros: $\text{dist}^2 \leq (r_1 + r_2)^2$, sin raíz cuadrada. El borde cuenta (tangentes chocan) y los días también: activa de b a $e+1$, así que chocan si $\max(b) \leq \min(e)$. OJO CON LA LECTURA: cada antena viene como " r x y b e", con el radio PRIMERO. La primera versión leía " x y r b e" y respondía "11" en el primer caso del sample, instalando dos antenas que se cubren. Corregida, se valida contra fuerza bruta en 1800 casos (incluye tangentes y días que se tocan), sin una sola salida inválida. RIESGO DE MEMORIA: se revisan todos los pares, y si muchas antenas se solapan las cláusulas crecen como n^2 . Medido con todas solapadas: 6000 antenas piden 371 MB, y con las 10000 del enunciado sería del orden de 1 GB. Si el juez trae un caso así de denso, la salida es no guardar las aristas de conflicto y recalculas al recorrer, como hace "J - Lumina".

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Antenna {
5     long long x, y, r;
6     int b, e;
7 };
8
9 int main() {
10     ios::sync_with_stdio(false);
11     cin.tie(nullptr);
12
13     int T;
14     cin >> T;
15
16     while (T--) {
17
18         int n, m;
19         cin >> n >> m;
20
21         vector<Antenna> a(n);
22
23         for (int i = 0; i < n; i++) {
24             // el enunciado da el RADIO primero: r x y b e
25             cin >> a[i].r >> a[i].x >> a[i].y
26                 >> a[i].b >> a[i].e;
27         }
28
29         /*
30          2-SAT:
31
32          2*i    -> antena i instalada
33          2*i + 1 -> antena i NO instalada
34
35          negacion:
36          x ^ 1
37          */
38     }
```

```

39     int V = 2 * n;
40
41     vector<vector<int>> g(V), gr(V);
42
43     auto addEdge = [&](int u, int v) {
44         g[u].push_back(v);
45         gr[v].push_back(u);
46     };
47
48     // (u OR v)
49     auto addClause = [&](int u, int v) {
50
51         // !u -> v
52         addEdge(u ^ 1, v);
53
54         // !v -> u
55         addEdge(v ^ 1, u);
56     };
57
58     // -----
59     // Restricciones dadas por las parejas
60     // -----
61
62     for (int i = 0; i < m; i++) {
63
64         int u, v;
65         cin >> u >> v;
66
67         --u;
68         --v;
69
70         // Al menos una debe instalarse
71         // xu OR xv
72         addClause(2 * u, 2 * v);
73     }
74
75     // -----
76     // Restricciones entre antenas
77     // -----
78
79     for (int i = 0; i < n; i++) {
80
81         for (int j = i + 1; j < n; j++) {
82
83             // Sus periodos se intersectan?
84             bool sameTime =
85                 max(a[i].b, a[j].b) <=
86                 min(a[i].e, a[j].e);
87
88             if (!sameTime)
89                 continue;
90
91             // Sus círculos se intersectan?
92             long long dx = a[i].x - a[j].x;
93             long long dy = a[i].y - a[j].y;
94
95             long long dist2 = dx * dx + dy * dy;
96
97             long long radius = a[i].r + a[j].r;
98
99             bool samePlace =
100                 dist2 <= radius * radius;
101
102             if (samePlace) {
103
104                 /*
105                  No pueden instalarse las dos:
106
107                  !(xi AND xj)
108
109                  = !xi OR !xj
110
111                  */

```

```

102             addClause(
103                 2 * i + 1,
104                 2 * j + 1
105             );
106         }
107     }
108
109     // -----
110     // Kosaraju - primera pasada
111     // -----
112
113     vector<bool> visited(V, false);
114     vector<int> order;
115
116     function<void(int)> dfs1 = [&](int u) {
117
118         visited[u] = true;
119
120         for (int v : g[u]) {
121             if (!visited[v])
122                 dfs1(v);
123         }
124
125         order.push_back(u);
126     };
127
128     for (int i = 0; i < V; i++) {
129         if (!visited[i])
130             dfs1(i);
131     }
132
133     // -----
134     // Kosaraju - segunda pasada
135     // -----
136
137     vector<int> comp(V, -1);
138
139     function<void(int, int)> dfs2 =
140         [&](int u, int id) {
141
142         comp[u] = id;
143
144         for (int v : gr[u]) {
145             if (comp[v] == -1)
146                 dfs2(v, id);
147         }
148     };
149
150     reverse(order.begin(), order.end());
151
152     int components = 0;
153
154     for (int u : order) {
155
156         if (comp[u] == -1) {
157             dfs2(u, components++);
158         }
159     }
160
161     // -----
162     // Comprobar si existe solucion
163     // -----
164
165     bool possible = true;
166
167     for (int i = 0; i < n; i++) {
168
169         int on = 2 * i;
170         int off = 2 * i + 1;
171
172         if (comp[on] == comp[off]) {
173             possible = false;

```

```

85         break;
86     }
87 }
88
89 if (!possible) {
90     cout << "Impossible\n";
91     continue;
92 }
93
94 // -----
95 // Construir respuesta
96 // -----
97
98 string ans(n, '0');
99
100 for (int i = 0; i < n; i++) {
101
102     int on = 2 * i;
103     int off = 2 * i + 1;
104
105     ans[i] = (comp[on] > comp[off]) ? '1' : '0';
106 }
107
108 cout << ans << '\n';
109 }
110
111 return 0;
112 }

```

3.2 Articulation Points (Tarjan) - Submerging Islands

C++(Training_Camp_2026\Day 4\L - Submerging Islands.cpp)

Resuelve "Submerging Islands" (Day 4, problema L - Contest 04 [Avanzados]): dado un grafo de N islas conectadas por M puentes, hay que contar cuantas islas, al sumergirse, desconectar  partes de la ciudad. Es el cl sico problema de puntos de articulaci n: hace un DFS (funcion dfs) calculando disc/low de cada nodo, marca como articulaci n a todo nodo no raiz cuyo hijo v cumpla $low[v] \geq disc[u]$, y a la raiz si tiene mas de un hijo en el  rbol DFS, contando al final cuantos nodos quedaron marcados (soporta grafos no conexos probando DFS desde cada nodo no visitado).

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const int MAXN = 10005;
5
6 vector<int> adj[MAXN];
7 int disc[MAXN], low[MAXN], timer;
8 bool visited[MAXN], articulation[MAXN];
9
10 void dfs(int u, int parent) {
11     visited[u] = true;
12     disc[u] = low[u] = ++timer;
13     int children = 0;
14
15     for (int v : adj[u]) {
16         if (!visited[v]) {
17             children++;
18             dfs(v, u);
19
20             low[u] = min(low[u], low[v]);
21
22             if (parent != -1 && low[v] >= disc[u])
23                 articulation[u] = true;
24         }
25         else if (v != parent) {
26             low[u] = min(low[u], disc[v]);
27         }
28     }
29
30     if (parent == -1 && children > 1)
31         articulation[u] = true;
32 }
33
34 int main() {
35     ios::sync_with_stdio(false);
36     cin.tie(nullptr);

```

```

37     int N, M;
38
39     while (cin >> N >> M && (N || M)) {
40
41         for (int i = 1; i <= N; i++) {
42             adj[i].clear();
43             visited[i] = false;
44             articulation[i] = false;
45         }
46
47         timer = 0;
48
49         for (int i = 0; i < M; i++) {
50             int u, v;
51             cin >> u >> v;
52             adj[u].push_back(v);
53             adj[v].push_back(u);
54         }
55
56         // Por si el grafo no fuera conexo.
57         for (int i = 1; i <= N; i++)
58             if (!visited[i])
59                 dfs(i, -1);
60
61         int ans = 0;
62         for (int i = 1; i <= N; i++)
63             if (articulation[i])
64                 ans++;
65
66         cout << ans << '\n';
67     }
68
69     return 0;
70 }
71 }

```

3.3 BFS - Ladder Length

C++(LeetCode\ladderLength.cpp)

Resuelve "Word Ladder": halla la longitud de la transformaci n mas corta de beginWord a endWord cambiando una letra a la vez, usando solo palabras de wordList. BFS por niveles donde cada palabra es un nodo y dos palabras se conectan si difieren en exactamente una letra.

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 class Solution {
6 public:
7     int ladderLength(string beginWord, string endWord, vector<string>& wordList) {
8         unordered_set<string> words(wordList.begin(), wordList.end());
9
10        if (!words.count(endWord))
11            return 0;
12
13        queue<pair<string, int>> q;
14        q.push({beginWord, 1});
15
16        while (!q.empty()) {
17            auto [word, dist] = q.front();
18            q.pop();
19
20            if (word == endWord)
21                return dist;
22
23            for (int i = 0; i < word.size(); i++) {
24                char original = word[i];
25
26                for (char c = 'a'; c <= 'z'; c++) {
27                    if (c == original) continue;
28
29                    word[i] = c;
30
31                    if (words.count(word)) {

```

```

32         q.push({word, dist + 1});
33         words.erase(word); // mark as visited
34     }
35 }
36
37     word[i] = original;
38 }
39 }
40
41     return 0;
42 }
43 };

```

3.4 BFS - Boss

C++ Training_Camp_2020/Day 2/C - Boss.cpp

Resuelve "Boss" (Day 2, problema C - Maratona de Programacao da SBC 2013): en un organigrama de N empleados donde X maneja directamente a Y, la operacion "T A B" intercambia las POSICIONES que ocupan A y B dentro del organigrama (que nunca cambia), y "P E" pregunta la edad del jefe mas joven, directo o indirecto, de E. Guarda el grafo invertido rev[Y] = jefes directos de una posicion, y los arreglos empPos/posEmp para saber que empleado ocupa cada posicion despues de cada swap. Cada consulta hace BFS desde la posicion de E sobre rev, visita todas las posiciones jefe (ancestros) y se queda con la edad minima entre los empleados que hoy las ocupan.

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      ios::sync_with_stdio(false);
6      cin.tie(nullptr);
7
8      int N, M, I;
9
10     // El problema puede contener varios casos de prueba
11     while (cin >> N >> M >> I) {
12
13         // age[i] = edad del empleado i
14         vector<int> age(N + 1);
15         for (int i = 1; i <= N; i++)
16             cin >> age[i];
17
18         /*
19          rev representa el grafo invertido.
20
21          Si X maneja directamente a Y:
22              X -> Y
23
24          entonces almacenamos:
25
26              rev[Y].push_back(X)
27
28          porque durante una consulta queremos subir por la cadena
29          de mando (buscar todos los jefes).
30          */
31         vector<vector<int>> rev(N + 1);
32
33         for (int i = 0; i < M; i++) {
34             int X, Y;
35             cin >> X >> Y;
36             rev[Y].push_back(X);
37         }
38
39         /*
40          El grafo nunca cambia.
41
42          Lo único que cambia es qué empleado ocupa cada posición
43          dentro del organigrama.
44
45          empPos[e] = posición donde está actualmente el empleado e.
46
47          posEmp[p] = empleado que ocupa actualmente la posición p.
48          */
49         vector<int> empPos(N + 1);
50         vector<int> posEmp(N + 1);

```

```

51
52     // Inicialmente cada empleado está en su propia posición.
53     for (int i = 1; i <= N; i++) {
54         empPos[i] = i;
55         posEmp[i] = i;
56     }
57
58     while (I--) {
59
60         char op;
61         cin >> op;
62
63         // Cambio de posiciones
64         if (op == 'T') {
65
66             int A, B;
67             cin >> A >> B;
68
69             // Posiciones actuales de ambos empleados
70             int posA = empPos[A];
71             int posB = empPos[B];
72
73             // Intercambiar las posiciones de los empleados
74             swap(empPos[A], empPos[B]);
75
76             // Actualizar qué empleado queda en cada posición
77             swap(posEmp[posA], posEmp[posB]);
78         }
79
80         // Consulta
81         else {
82
83             int E;
84             cin >> E;
85
86             // Posición donde se encuentra actualmente el empleado E
87             int start = empPos[E];
88
89             vector<int> vis(N + 1, 0);
90             queue<int> q;
91
92             q.push(start);
93             vis[start] = true;
94
95             int ans = INT_MAX;
96
97             /*
98              Hacemos BFS sobre el grafo invertido.
99
100             Desde la posición del empleado vamos visitando
101             todos los jefes (directos e indirectos).
102
103             Cada nodo del BFS representa una POSICIÓN.
104
105             Para conocer la edad del jefe debemos averiguar
106             qué empleado ocupa actualmente esa posición.
107             */
108             while (!q.empty()) {
109
110                 int u = q.front();
111                 q.pop();
112
113                 // Revisar todos los jefes directos
114                 for (int bossPos : rev[u]) {
115
116                     if (vis[bossPos])
117                         continue;
118
119                     vis[bossPos] = true;
120                     q.push(bossPos);
121
122                     // Empleado que ocupa esa posición
123                     int bossEmp = posEmp[bossPos];

```

```

85
86         // Actualizar la edad mínima encontrada
87         ans = min(ans, age[bossEmp]);
88     }
89 }
90
91 // Si nunca encontramos un jefe
92 if (ans == INT_MAX)
93     cout << "\n";
94 else
95     cout << ans << "\n";
96 }
97 }
98 }
99
100 return 0;
101 }

```

3.5 BFS - Bacon Number

C++ Training_Camp_2020/Day 3/Bacon Number.cpp

Resuelve "Bacon Number" (Day 3, problema B - Maratona SBC de Programacao 2024): dadas N películas con sus actores, hay que responder Q consultas que piden conectar dos actores x e y mediante una secuencia alternada actor-película-actor-...-película-actor, donde cada actor consecutivo comparte la película que los une con el siguiente, o determinar que no es posible. Construye un grafo cuyos nodos son películas y donde hay arista entre dos películas si comparten al menos un actor (agrupando películas por actor en actorMovies), y corre un BFS desde cada película (dist, parent) para precalcular caminos mínimos entre películas; luego, para cada consulta, prueba todas las combinaciones de películas donde aparecen x e y y reconstruye el camino de actores/películas siguiendo el arreglo parent.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const int INF = 1e9;
5
6 int main() {
7     ios::sync_with_stdio(false);
8     cin.tie(nullptr);
9
10    int N, M;
11    cin >> N >> M;
12
13    vector<vector<int>> movieActors(N + 1);
14    unordered_map<int, vector<int>> actorMovies;
15
16    for (int i = 1; i <= N; i++) {
17        int k;
18        cin >> k;
19        movieActors[i].resize(k);
20        for (int j = 0; j < k; j++) {
21            cin >> movieActors[i][j];
22            actorMovies[movieActors[i][j]].push_back(i);
23        }
24    }
25
26    // actor común entre películas
27    vector<vector<int>> common(N + 1, vector<int>(N + 1, 0));
28    vector<vector<int>> adj(N + 1);
29
30    for (auto &it : actorMovies) {
31        auto &v = it.second;
32        for (int i = 0; i < (int)v.size(); i++) {
33            for (int j = i + 1; j < (int)v.size(); j++) {
34                int a = v[i], b = v[j];
35                if (!common[a][b]) {
36                    common[a][b] = common[b][a] = it.first;
37                    adj[a].push_back(b);
38                    adj[b].push_back(a);
39                }
40            }
41        }
42    }
43
44    vector<vector<int>> dist(N + 1, vector<int>(N + 1, INF));

```

```

45    vector<vector<int>> parent(N + 1, vector<int>(N + 1, -1));
46
47    for (int s = 1; s <= N; s++) {
48        queue<int> q;
49        dist[s][s] = 0;
50        q.push(s);
51
52        while (!q.empty()) {
53            int u = q.front();
54            q.pop();
55
56            for (int v : adj[u]) {
57                if (dist[s][v] == INF) {
58                    dist[s][v] = dist[s][u] + 1;
59                    parent[s][v] = u;
60                    q.push(v);
61                }
62            }
63        }
64    }
65
66    int Q;
67    cin >> Q;
68
69    while (Q--) {
70        int x, y;
71        cin >> x >> y;
72
73        if (!actorMovies.count(x) || !actorMovies.count(y)) {
74            cout << -1 << '\n';
75            continue;
76        }
77
78        int bestA = -1, bestB = -1, best = INF;
79
80        for (int a : actorMovies[x])
81            for (int b : actorMovies[y])
82                if (dist[a][b] < best) {
83                    best = dist[a][b];
84                    bestA = a;
85                    bestB = b;
86                }
87
88        if (best == INF) {
89            cout << -1 << '\n';
90            continue;
91        }
92
93        vector<int> movies;
94        int cur = bestB;
95        while (cur != bestA) {
96            movies.push_back(cur);
97            cur = parent[bestA][cur];
98        }
99        movies.push_back(bestA);
100        reverse(movies.begin(), movies.end());
101
102        vector<int> out;
103
104        out.push_back(x);
105
106        if (x != common[movies[0]][movies[0]])
107            ; // nada
108
109        int currentActor = x;
110
111        if (actorMovies[x][0] != movies[0]) {
112            // x aparece en movies[0] por construcción
113        }
114
115        for (int i = 0; i + 1 < (int)movies.size(); i++) {
116            int actor = common[movies[i]][movies[i + 1]];
117            if (currentActor != actor) {

```

```

18         out.push_back(movies[i]);
19         out.push_back(actor);
20         currentActor = actor;
21     } else {
22         out.push_back(movies[i]);
23     }
24 }
25
26 out.push_back(movies.back());
27
28 if (currentActor != y)
29     out.push_back(y);
30
31 int actors = 0;
32 for (int i = 0; i < (int)out.size(); i += 2)
33     actors++;
34
35 cout << actors << '\n';
36
37 for (int i = 0; i < (int)out.size(); i++) {
38     if (i) cout << ' ';
39     cout << out[i];
40 }
41 cout << '\n';
42 }
43 }

```

3.6 BFS Backtracking - Find Ladders

C++ <LeetCode/FindLadders.cpp>

Resuelve "Word Ladder II": encuentra TODAS las secuencias de transformacion mas cortas de beginWord a endWord cambiando una letra a la vez. BFS por niveles guardando los padres de cada palabra, y luego backtracking desde endWord para reconstruir todos los caminos minimos.

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 class Solution {
6 public:
7     vector<vector<string>> ans;
8     unordered_map<string, vector<string>> parents;
9
10    void buildPaths(string word,
11                  string beginWord,
12                  vector<string>& path) {
13
14        if (word == beginWord) {
15            vector<string> cur = path;
16            reverse(cur.begin(), cur.end());
17            ans.push_back(cur);
18            return;
19        }
20
21        for (auto& p : parents[word]) {
22            path.push_back(p);
23            buildPaths(p, beginWord, path);
24            path.pop_back();
25        }
26    }
27
28    vector<vector<string>> findLadders(string beginWord,
29                                    string endWord,
30                                    vector<string>& wordList) {
31
32        unordered_set<string> words(wordList.begin(), wordList.end());
33
34        if (!words.count(endWord))
35            return {};
36
37        queue<string> q;
38        q.push(beginWord);
39
40        unordered_set<string> visited;

```

```

41        visited.insert(beginWord);
42
43        bool found = false;
44
45        while (!q.empty() && !found) {
46
47            int sz = q.size();
48
49            unordered_set<string> levelVisited;
50
51            while (sz--) {
52
53                string word = q.front();
54                q.pop();
55
56                string original = word;
57
58                for (int i = 0; i < word.size(); i++) {
59
60                    char old = word[i];
61
62                    for (char c = 'a'; c <= 'z'; c++) {
63
64                        if (c == old) continue;
65
66                        word[i] = c;
67
68                        if (!words.count(word))
69                            continue;
70
71                        if (!visited.count(word)) {
72
73                            if (!levelVisited.count(word)) {
74                                q.push(word);
75                                levelVisited.insert(word);
76                            }
77
78                            parents[word].push_back(original);
79
80                            if (word == endWord)
81                                found = true;
82                        }
83                    }
84
85                    word[i] = old;
86                }
87            }
88
89            for (auto& w : levelVisited)
90                visited.insert(w);
91        }
92
93        if (!found)
94            return {};
95
96        vector<string> path = {endWord};
97        buildPaths(endWord, beginWord, path);
98
99        return ans;
100    }
101 };

```

3.7 Biconnected Components (Block-Cut Tree) + LCA - Hedge Mazes

C++ <Training_Camp_2026/Day 4/ - Hedge Mazes.cpp>

Resuelve "Hedge Mazes" (Day 4, problema N - Contest 04 [Avanzados]): dado un laberinto de R salas y C corredores, para cada consulta (S, T) hay que decidir si existe exactamente un camino simple entre S y T (si hay un ciclo en el medio, existiría mas de un camino simple). Construye las componentes biconexas (buildBiconnected, via low-link) y arma el arbol de bloques y puntos de corte (block-cut tree), donde cada bloque de tamaño ≥ 3 vertices representa un ciclo; usando LCA con binary lifting (bfsBuildLCA/queryPath) calcula el maximo tamaño de bloque en el camino entre S y T, y responde 'Y' solo si ese maximo es ≤ 2 (solo aristas, sin ciclos).

```

1 #include <bits/stdc++.h>
2 using namespace std;

```

```

3
4 static char buf_[1 << 22];
5 static int buflen_, bufPos_;
6 inline int gc() {
7     if (bufPos_ == buflen_) { buflen_ = fread(buf_, 1, sizeof(buf_), stdin); bufPos_ = 0; if (!buflen_) return -1; }
8     return buf_[bufPos_++];
9 }
0 inline int readInt() {
1     int c = gc();
2     while (c != -1 && (c < '0' || c > '9') && c != '-') c = gc();
3     bool neg = false; if (c == '-') { neg = true; c = gc(); }
4     int x = 0;
5     while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = gc(); }
6     return neg ? -x : x;
7 }
8
9 int R, C, Q;
10 vector<pair<int,int>> edges; // edge id -> (u,v)
11 vector<vector<pair<int,int>>> adj; // adj[u] -> list of (v, edgeId)
12 vector<int> disc_, low_;
13 vector<char> visited_, isCut;
14 int timer_;
15
16 vector<int> posOfVertex; // "position" node id in block-cut tree for each original vertex
17 // for a cut vertex v: position = v itself (1..R)
18 // for a non-cut vertex v: position = its unique block's node id (R+1..
19
20 R+numBlocks)
21 vector<int> blockWeight; // weight[nodeId] : block-size for block nodes, 0 for cut-vertex nodes (
22 irrelevant)
23 vector<vector<int>> treeAdj; // adjacency list over node ids 1..(R+numBlocks)
24 vector<int> treeCompId; // which tree (of the block-cut forest) each node belongs to
25 int numBlocks;
26
27 vector<int> lastMarked; // lastMarked[v] = block index that last touched vertex v (avoids duplicate
28 adds)
29
30 void buildBiconnected() {
31     disc_.assign(R + 1, -1);
32     low_.assign(R + 1, -1);
33     visited_.assign(R + 1, 0);
34     isCut_.assign(R + 1, 0);
35     posOfVertex.assign(R + 1, -1);
36     lastMarked.assign(R + 1, -1);
37     timer_ = 0;
38     numBlocks = 0;
39
40     vector<vector<int>> blockVerts; // blockVerts[b] = list of vertices in block b (0-indexed b)
41     vector<int> edgeStack;
42
43     for (int s = 1; s <= R; s++) {
44         if (visited_[s]) continue;
45         // iterative DFS
46         vector<array<int,4>> stk; // {u, nextIdx, parentEdge, parentVertex}
47         visited_[s] = 1;
48         disc_[s] = low_[s] = timer_++;
49         stk.push_back({s, 0, -1, -1});
50         int rootChildren = 0;
51
52         while (!stk.empty()) {
53             auto &fr = stk.back();
54             int u = fr[0];
55             if (fr[1] < (int)adj[u].size()) {
56                 int v = adj[u][fr[1]].first;
57                 int eid = adj[u][fr[1]].second;
58                 fr[1]++;
59                 if (eid == fr[2]) continue; // skip edge back to parent
60                 if (!visited_[v]) {
61                     visited_[v] = 1;
62                     disc_[v] = low_[v] = timer_++;
63                     edgeStack.push_back(eid);
64                     if (u == s) rootChildren++;
65                     stk.push_back({v, 0, eid, u});
66                 }
67             }
68         }
69     }
70 }

```

```

71     } else if (disc_[v] < disc_[u]) {
72         edgeStack.push_back(eid);
73         low_[u] = min(low_[u], disc_[v]);
74     }
75 } else {
76     int parentEdge = fr[2];
77     int parentVertex = fr[3];
78     int uLow = low_[u];
79     stk.pop_back();
80     if (!stk.empty()) {
81         int p = parentVertex;
82         low_[p] = min(low_[p], uLow);
83         if (uLow >= disc_[p]) {
84             if (p != s) isCut[p] = 1;
85             // pop edges up to and including parentEdge -> forms one block
86             int b = numBlocks++;
87             blockVerts.push_back({});
88             while (!edgeStack.empty()) {
89                 int e = edgeStack.back();
90                 edgeStack.pop_back();
91                 int a1 = edges[e].first, a2 = edges[e].second;
92                 if (lastMarked[a1] != b) { lastMarked[a1] = b; blockVerts[b].push_back(
93                     a1); }
94                 if (lastMarked[a2] != b) { lastMarked[a2] = b; blockVerts[b].push_back(
95                     a2); }
96                 if (e == parentEdge) break;
97             }
98         }
99     }
100 }
101 if (rootChildren > 1) isCut[s] = 1;
102 }
103
104 // assign block weights and non-cut vertex positions
105 blockWeight.assign(1 + R + numBlocks, 0);
106 for (int b = 0; b < numBlocks; b++) {
107     int nodeId = R + 1 + b;
108     blockWeight[nodeId] = (int)blockVerts[b].size();
109     for (int v : blockVerts[b]) {
110         if (!isCut[v]) {
111             posOfVertex[v] = nodeId;
112         }
113     }
114 }
115
116 for (int v = 1; v <= R; v++) {
117     if (isCut[v]) posOfVertex[v] = v;
118     else if (posOfVertex[v] == -1) {
119         // isolated vertex (no edges at all) -> give it its own trivial "block" node conceptual
120         ly.
121         // We reuse it as its own position id (acts like a size-1 pseudo node, weight 0, never
122         matches
123         // any other vertex's position unless that vertex IS this same isolated vertex, which c
124         an't
125         // happen for a distinct query since S != T is guaranteed... but S or T might independe
126         ntly
127         // be isolated vertices in different queries).
128         posOfVertex[v] = v; // treat as its own node; weight will be 0 (irrelevant) and it will
129         simply
130         // have no tree edges, forming its own singleton tree.
131     }
132 }
133
134 // Build tree adjacency
135 treeAdj.assign(1 + R + numBlocks, {});
136 for (int b = 0; b < numBlocks; b++) {
137     int nodeId = R + 1 + b;
138     for (int v : blockVerts[b]) {
139         if (isCut[v]) {
140             treeAdj[v].push_back(nodeId);
141             treeAdj[nodeId].push_back(v);
142         }
143     }
144 }

```

```

88     }
89 }
90
91 // LCA with binary lifting, tracking max blockWeight along the path (node weights only matter for b
92 lock nodes;
93 // cut-vertex nodes / isolated singleton nodes have weight 0 which never triggers the "bad" conditi
94 on).
95 int LOG_;
96 vector<vector<int>> up_;
97 vector<vector<int>> upMax_; // upMax_[k][v] = max weight among the 2^k nodes starting at v going up
98 ward (v itself
99 // included at k=0 as weight[v], combined appropriately)
100 vector<int> depth_;
101
102 void bfsBuildLCA(int totalNodes) {
103     LOG_ = 1;
104     while ((1 << LOG_) < totalNodes + 1) LOG_++;
105     up_.assign(LOG_, vector<int>(totalNodes + 1, 0));
106     upMax_.assign(LOG_, vector<int>(totalNodes + 1, 0));
107     depth_.assign(totalNodes + 1, -1);
108     treeCompId.assign(totalNodes + 1, -1);
109
110     int curComp = -1;
111     for (int start = 1; start <= totalNodes; start++) {
112         if (depth_[start] != -1) continue;
113         curComp++;
114         queue<int> bfsq;
115         depth_[start] = 0;
116         up_[0][start] = 0; // root's parent = 0 (sentinel "no parent")
117         upMax_[0][start] = blockWeight[start];
118         treeCompId[start] = curComp;
119         bfsq.push(start);
120         while (!bfsq.empty()) {
121             int u = bfsq.front(); bfsq.pop();
122             for (int w : treeAdj[u]) {
123                 if (depth_[w] == -1) {
124                     depth_[w] = depth_[u] + 1;
125                     up_[0][w] = u;
126                     upMax_[0][w] = max(blockWeight[w], blockWeight[u]);
127                     treeCompId[w] = curComp;
128                     bfsq.push(w);
129                 }
130             }
131         }
132     }
133 }
134
135 for (int k = 1; k < LOG_; k++) {
136     for (int v = 1; v <= totalNodes; v++) {
137         int mid = up_[k-1][v];
138         if (mid == 0) { up_[k][v] = 0; upMax_[k][v] = upMax_[k-1][v]; }
139         else {
140             up_[k][v] = up_[k-1][mid];
141             upMax_[k][v] = max(upMax_[k-1][v], upMax_[k-1][mid]);
142         }
143     }
144 }
145
146 // returns {lca, maxWeightOnPath} or {-1, -1} if different components
147 pair<int,int> queryPath(int u, int v) {
148     if (treeCompId[u] != treeCompId[v]) return {-1, -1};
149     int best = max(blockWeight[u], blockWeight[v]);
150     if (depth_[u] < depth_[v]) swap(u, v);
151     int diff = depth_[u] - depth_[v];
152     for (int k = 0; k < LOG_; k++) {
153         if ((diff >> k) & 1) {
154             best = max(best, upMax_[k][u]);
155             u = up_[k][u];
156         }
157     }
158     if (u == v) return {u, best};
159     for (int k = LOG_ - 1; k >= 0; k--) {
160         if (up_[k][u] != up_[k][v]) {

```

```

208         best = max({best, upMax_[k][u], upMax_[k][v]});
209         u = up_[k][u];
210         v = up_[k][v];
211     }
212 }
213 best = max({best, blockWeight[up_[0][u]], upMax_[0][u], upMax_[0][v]});
214 return {up_[0][u], best};
215 }
216
217 int main() {
218     while (true) {
219         R = readInt(); C = readInt(); Q = readInt();
220         if (R == 0 && C == 0 && Q == 0) break;
221
222         edges.assign(C, {0,0});
223         adj.assign(R + 1, {});
224         for (int i = 0; i < C; i++) {
225             int a = readInt(), b = readInt();
226             edges[i] = {a, b};
227             adj[a].push_back({b, i});
228             adj[b].push_back({a, i});
229         }
230
231         buildBiconnected();
232         int totalNodes = R + numBlocks;
233         bfsBuildLCA(totalNodes);
234
235         string out;
236         for (int q = 0; q < Q; q++) {
237             int s = readInt(), t = readInt();
238             int ps = posOfVertex[s], pt = posOfVertex[t];
239             auto [lca, mx] = queryPath(ps, pt);
240             if (lca == -1) {
241                 out += "N\n";
242             } else {
243                 out += (mx <= 2) ? "V\n" : "N\n";
244             }
245         }
246         out += "-\n";
247         fwrite(out.data(), 1, out.size(), stdout);
248     }
249     return 0;
250 }

```

3.8 Bipartite Matching - Attacking Rooks

C++\Training_Camp_2026\Day 7\1 - Attacking rooks.cpp

Resuelve "Attacking rooks" (Day 7, problema 1 - Contest 07 [Avanzados]): en un tablero $N \times N$ con algunas casillas ocupadas por peones ('X'), dos torres se atacan si comparten fila o columna sin un peon entre medio; hay que colocar el maximo numero de torres en casillas vacias sin que se ataquen entre si. Modela cada tramo horizontal maximal de casillas vacias entre peones como un nodo "fila" (rowId) y cada tramo vertical como un nodo "columna" (colId); cada casilla vacia conecta su tramo de fila con su tramo de columna. El maximo de torres no atacantes es entonces el maximo matching bipartito entre tramos-fila y tramos-columna, calculado con el algoritmo de Hopcroft-Karp (BFS por capas + DFS de caminos aumentantes, struct HopcroftKarp).

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct HopcroftKarp {
5     int n, m;
6     vector<vector<int>> adj;
7     vector<int> matchL, matchR, dist;
8     const int INF = INT_MAX;
9     HopcroftKarp(int n=0, int m=0): n(n), m(m), adj(n), matchL(n,-1), matchR(m,-1), dist(n) {}
10    void addEdge(int u,int v){ adj[u].push_back(v); }
11    bool bfs(){
12        queue<int> q;
13        for(int u=0;u<n;u++){
14            if(matchL[u]==-1){ dist[u]=0; q.push(u); }
15            else dist[u]=INF;
16        }
17        bool found=false;
18        while(!q.empty()){

```

```

18         int u=q.front();q.pop();
19         for(int v: adj[u]){
20             int w = matchR[v];
21             if(w!=-1){ found=true; }
22             else if(dist[w]==INF){ dist[w]=dist[u]+1; q.push(w); }
23         }
24     }
25     return found;
26 }
27 bool dfs(int u){
28     for(int v: adj[u]){
29         int w = matchR[v];
30         if(w!=-1 || (dist[w]==dist[u]+1 && dfs(w))){
31             matchL[u]=v; matchR[v]=u;
32             return true;
33         }
34     }
35     dist[u]=INF;
36     return false;
37 }
38 int maxMatching(){
39     int result=0;
40     while(bfs()){
41         for(int u=0;u<n;u++){
42             if(matchL[u]==-1){
43                 if(dfs(u)) result++;
44             }
45         }
46     }
47     return result;
48 }
49 };
50
51 int main(){
52     ios::sync_with_stdio(false);
53     cin.tie(nullptr);
54     int N;
55     while(cin >> N){
56         vector<string> grid(N);
57         for(auto &s: grid) cin >> s;
58
59         vector<vector<int>> rowId(N, vector<int>(N,-1));
60         vector<vector<int>> colId(N, vector<int>(N,-1));
61         int R=0, C=0;
62
63         // etiqueta cada tramo horizontal de casillas vacías
64         for(int i=0;i<N;i++){
65             int j=0;
66             while(j<N){
67                 if(grid[i][j]=='X'){ j++; continue; }
68                 while(j<N && grid[i][j]=='.'){ rowId[i][j]=R; j++; }
69                 R++;
70             }
71         }
72         // etiqueta cada tramo vertical de casillas vacías
73         for(int j=0;j<N;j++){
74             int i=0;
75             while(i<N){
76                 if(grid[i][j]=='X'){ i++; continue; }
77                 while(i<N && grid[i][j]=='.'){ colId[i][j]=C; i++; }
78                 C++;
79             }
80         }
81
82         HopcroftKarp hk(R,C);
83         for(int i=0;i<N;i++){
84             for(int j=0;j<N;j++){
85                 if(grid[i][j]=='.')
86                     hk.addEdge(rowId[i][j], colId[i][j]);
87             }
88         }
89         cout << hk.maxMatching() << "\n";
90     }
91     return 0;

```

```

92 }

```

3.9 Bipartite Matching - Knights In The Board

C++(transmisión camp 2025 Day 7) - Knights in the board.cpp

Resuelve "Knights In The Board" (Day 7, problema K - Contest 07 [Avanzados]): dado un tablero NxN con K caballos ya colocados, hay que hallar el minimo numero de caballos a remover para que ninguno ataque a otro (movimiento en L de ajedrez). El grafo de ataques entre caballos es bipartito, ya que un caballo en una casilla de un color (segun (fila+columna)%2, ver color_) solo puede atacar caballos del color opuesto. Por el teorema de Konig, el minimo vertex cover (minimo de caballos a remover para romper todos los conflictos) equivale al maximo matching bipartito. Construye adj solo desde los caballos de color 0 hacia sus vecinos alcanzables con movimiento de caballo, y calcula el maximo matching con el algoritmo de Kuhn (tryKuhn), que es directamente la respuesta pedida.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int N, K;
5 map<pair<int,int>, int> pos2idx;
6 vector<int> R, C, color_;
7 vector<vector<int>> adj;
8 vector<int> matchRight;
9 vector<bool> visited;
10
11 int dr[8] = {1,1,-1,-1,2,2,-2,-2};
12 int dc[8] = {2,-2,2,-2,1,-1,1,-1};
13
14 bool tryKuhn(int u) {
15     for (int v : adj[u]) {
16         if (!visited[v]) {
17             visited[v] = true;
18             if (matchRight[v] == -1 || tryKuhn(matchRight[v])) {
19                 matchRight[v] = u;
20                 return true;
21             }
22         }
23     }
24     return false;
25 }
26
27 int main(){
28     cin >> N >> K;
29     R.resize(K); C.resize(K); color_.resize(K);
30     for (int i = 0; i < K; i++) {
31         cin >> R[i] >> C[i];
32         pos2idx[{R[i], C[i]}] = i;
33         color_[i] = (R[i] + C[i]) % 2;
34     }
35     adj.assign(K, {});
36     for (int i = 0; i < K; i++) {
37         if (color_[i] != 0) continue;
38         for (int d = 0; d < 8; d++) {
39             auto it = pos2idx.find({R[i]+dr[d], C[i]+dc[d]});
40             if (it != pos2idx.end()) adj[i].push_back(it->second);
41         }
42     }
43     matchRight.assign(K, -1);
44     int matching = 0;
45     for (int i = 0; i < K; i++) {
46         if (color_[i] != 0 || adj[i].empty()) continue;
47         visited.assign(K, false);
48         if (tryKuhn(i)) matching++;
49     }
50     cout << matching << "\n";
51 }

```

3.10 Bipartite Matching (Algoritmo de Kuhn) - School Dance

C++\Training_Camp_2020\Day 5\1 - School Dance.cpp

Resuelve "School Dance" (Day 5, problema G - Contest 05 [Avanzados]): hay N chicos y M chicas, y una lista de K parejas potenciales (chico, chica) dispuestos a bailar juntos; hay que hallar el máximo número de parejas de baile que se pueden formar (cada chico y cada chica en a lo sumo una pareja) y mostrar una asignación que lo logre. Implementa matching bipartito con el algoritmo de Kuhn: para cada chico intenta encontrar, vía la función `tryMatch` (DFS de caminos aumentantes sobre el arreglo `matchGirl`), una chica libre o reasignable, marcando `visited` para no repetir chicos durante la búsqueda de un mismo camino aumentante.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int n, m, k;
5 vector<vector<int>> pairs;
6 vector<int> matchGirl;
7 vector<bool> visited;
8
9 bool tryMatch(int boy) {
10     if (visited[boy]) return false;
11     visited[boy] = true;
12
13     for (int girl : pairs[boy]) {
14         if (matchGirl[girl] == 0 || tryMatch(matchGirl[girl])) {
15             matchGirl[girl] = boy;
16             return true;
17         }
18     }
19     return false;
20 }
21
22 int main() {
23     ios::sync_with_stdio(false);
24     cin.tie(nullptr);
25
26     cin >> n >> m >> k;
27
28     pairs.assign(n + 1, {});
29
30     for (int i = 0; i < k; i++) {
31         int boy, girl;
32         cin >> boy >> girl;
33         pairs[boy].push_back(girl);
34     }
35
36     matchGirl.assign(m + 1, 0);
37
38     int maximumPairs = 0;
39
40     for (int boy = 1; boy <= n; boy++) {
41         visited.assign(n + 1, false);
42         if (tryMatch(boy)) {
43             maximumPairs++;
44         }
45     }
46
47     cout << maximumPairs << endl;
48
49     for (int girl = 1; girl <= m; girl++) {
50         if (matchGirl[girl] != 0) {
51             cout << matchGirl[girl] << " " << girl << endl;
52         }
53     }
54
55     return 0;
56 }
```

3.11 Circuito Euleriano (Algoritmo de Hierholzer) - Mail Delivery

C++\Training_Camp_2020\Day 5\1 - Mail Delivery.cpp

Resuelve "Mail Delivery" (Day 5, problema J - Contest 05 [Avanzados]): dado un plano de n cruces y m calles bidireccionales, hay que encontrar una ruta que empiece y termine en la oficina postal (cruce 1) y recorra cada calle exactamente una vez, o determinar que es imposible. Primero verifica las condiciones de existencia de un circuito euleriano: todo vértice debe tener grado par, y todos los vértices con aristas deben ser alcanzables desde el vértice 1 (BFS). Luego construye el circuito con el algoritmo de Hierholzer de forma iterativa (pila `st` y arreglo `ptr` por vértice, marcando `usedEdge`), y revierte el resultado para obtener la ruta final.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main(){
5     ios_base::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int n, m;
9     cin >> n >> m;
10
11     vector<vector<pair<int,int>>> adj(n + 1); // (to, edge_id)
12     vector<int> deg(n + 1, 0);
13
14     for(int i = 0; i < m; i++){
15         int a, b;
16         cin >> a >> b;
17         adj[a].push_back({b, i});
18         adj[b].push_back({a, i});
19         deg[a]++;
20         deg[b]++;
21     }
22
23     // 1) every vertex must have even degree
24     for(int v = 1; v <= n; v++){
25         if(deg[v] % 2 != 0){
26             cout << "IMPOSSIBLE\n";
27             return 0;
28         }
29     }
30
31     // 2) all vertices that have edges must be reachable from vertex 1
32     // (this also catches the case where vertex 1 itself has no edges but m > 0)
33     vector<bool> visited(n + 1, false);
34     queue<int> bfsQ;
35     bfsQ.push(1);
36     visited[1] = true;
37     int reached = 0;
38     while(!bfsQ.empty()){
39         int u = bfsQ.front(); bfsQ.pop();
40         reached++;
41         for(auto &pr : adj[u]){
42             int v = pr.first;
43             if(!visited[v]){
44                 visited[v] = true;
45                 bfsQ.push(v);
46             }
47         }
48     }
49
50     int totalWithEdges = 0;
51     for(int v = 1; v <= n; v++){
52         if(deg[v] > 0) totalWithEdges++;
53     }
54
55     if(reached < totalWithEdges){
56         cout << "IMPOSSIBLE\n";
57         return 0;
58     }
59
60     // 3) Hierholzer's algorithm (iterative) to build the Eulerian circuit
61     vector<int> ptr(n + 1, 0);
62     vector<bool> usedEdge(m, false);
63     vector<int> circuit;
```

```

84 circuit.reserve(m + 1);
85
86 vector<int> st;
87 st.reserve(m + 1);
88 st.push_back(1);
89
90 while(!st.empty()){
91     int v = st.back();
92     if(ptr[v] < (int)adj[v].size()){
93         auto [to, eid] = adj[v][ptr[v]];
94         ptr[v]++;
95         if(usedEdge[eid]) continue;
96         usedEdge[eid] = true;
97         st.push_back(to);
98     } else {
99         circuit.push_back(v);
100        st.pop_back();
101    }
102 }
103
104 reverse(circuit.begin(), circuit.end());
105
106 // circuit must use exactly m edges -> m+1 vertices, and start/end at 1
107 if((int)circuit.size() != m + 1 || circuit.front() != 1 || circuit.back() != 1){
108     cout << "IMPOSSIBLE\n";
109     return 0;
110 }
111
112 for(int i = 0; i < (int)circuit.size(); i++){
113     cout << circuit[i];
114     if(i + 1 < (int)circuit.size()) cout << ' ';
115 }
116 cout << "\n";
117
118 return 0;
119 }

```

3.12 Clique Maximo (Bron-Kerbosch con Pivote) - Holy Network

ICPC2025A1 - Holy Network.cpp

Resuelve "Holy Network" (problema H, ICPC 2025): dados N numeros, hallar el grupo mas grande en el que CADA PAR comparta un factor mayor que 1. LA TRAMPA ES "CADA PAR". Que el grupo este conectado NO alcanza: hay que pedir que todos con todos sean compatibles, o sea un CLIQUE, no una componente conexa. El contraejemplo mas chico es {6, 10, 35}: 6 y 10 comparten el 2, 10 y 35 comparten el 5, pero 6 y 35 son coprimos. La componente conexa mide 3 y el grupo valido mide 2. La primera version de este archivo usaba DSU y devolvía el tamaño de la componente: pasaba los tres casos del sample (donde ambas cosas coinciden) pero fallaba 762 de 1500 casos aleatorios contra fuerza bruta. Esta version, con clique maximo, no falla ninguno. EL CLIQUE MAXIMO ES NP-DIFICIL, pero aqui $N \leq 50$ y Bron-Kerbosch con pivote lo resuelve rapido: se mantiene R (el clique en curso), P (candidatos) y X (ya descartados), todo en mascarar de 64 bits porque N cabe en un unsigned long long. El PIVOTE es lo que evita la explosion: se elige el vertice de $P \setminus X$ con mas vecinos en P y solo se ramifica por los candidatos que NO son sus vecinos, porque cualquier clique maximal o contiene al pivote o contiene a alguno de esos. La poda extra (si lo que llevamos mas todos los candidatos no supera al mejor, se corta) ayuda bastante cuando el grafo es denso. POR QUE NO EXPLOTA CON ESTOS DATOS: cada numero es $\leq 10^{12}$, así que tiene a lo sumo 11 primos distintos ($2 \cdot 3 \cdot 5 \cdot \dots \cdot 31$ ya pasa de $2 \cdot 10^{11}$). El grafo es entonces la union de unos pocos cliques, uno por primo, y esa estructura deja pocos cliques maximales. Medido con 200 casos de $N = 50$: 17 ms si todos comparten factor, 20 ms con numeros de 11 primos, y 84 ms en el adversario que mas cliques maximales logre construir. El mínimo es 1: un grupo de un solo ingrediente es armonico porque no hay pares que revisar. Ojo con el 1 como entrada: $\gcd(1, x) = 1$, así que un 1 no es compatible con nada. Se usa `__gcd` en vez de `std::gcd` para no depender de C++17 ni de incluir `<numeric>`.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 typedef unsigned long long ull;
5
6 int n;
7 ull adj[64];
8 int mejor;
9
10 // Bron-Kerbosch con pivote sobre mascarar de 64 bits.
11 // R = clique que llevamos, P = candidatos, X = ya descartados
12 void bron(ull R, ull P, ull X) {
13     if (P == 0 && X == 0) {
14         mejor = max(mejor, __builtin_popcountll(R));

```

```

6         return;
7     }
8
9     // Cota: ni tomando todos los candidatos se mejora lo que ya hay.
10    if (__builtin_popcountll(R) + __builtin_popcountll(P) <= mejor)
11        return;
12
13    // Pivote: el de P \ X con mas vecinos en P. Solo hace falta ramificar por
14    // los candidatos que NO son vecinos suyos.
15    ull PX = P | X;
16    int piv = __builtin_ctzll(PX);
17    int mejorGrado = -1;
18
19    for (ull t = PX; t; t &= t - 1) {
20        int u = __builtin_ctzll(t);
21        int g = __builtin_popcountll(adj[u] & P);
22        if (g > mejorGrado) {
23            mejorGrado = g;
24            piv = u;
25        }
26    }
27
28    for (ull t = P & ~adj[piv]; t; t &= t - 1) {
29        int v = __builtin_ctzll(t);
30        ull bit = 1ULL << v;
31
32        bron(R | bit, P & adj[v], X & adj[v]);
33
34        P &= ~bit;
35        X |= bit;
36    }
37
38    int main() {
39        ios::sync_with_stdio(false);
40        cin.tie(nullptr);
41
42        while (cin >> n && n != 0) {
43
44            vector<long long> a(n);
45
46            for (auto &x : a)
47                cin >> x;
48
49            for (int i = 0; i < n; i++)
50                adj[i] = 0;
51
52            // Dos ingredientes son compatibles si comparten un factor mayor que 1.
53            for (int i = 0; i < n; i++)
54                for (int j = i + 1; j < n; j++)
55                    if (__gcd(a[i], a[j]) > 1) {
56                        adj[i] |= 1ULL << j;
57                        adj[j] |= 1ULL << i;
58                    }
59
60            // Un grupo de UN solo ingrediente siempre es armonico: no hay pares
61            // que revisar. Por eso la respuesta nunca baja de 1.
62            mejor = 1;
63
64            ull todos = (n == 64) ? ~0ULL : ((1ULL << n) - 1);
65            bron(0, todos, 0);
66
67            cout << mejor << '\n';
68        }
69
70        return 0;
71    }

```

3.13 Clique Maximo con Backtracking Iterativo - Holy Network (clique Iterativo)

C++\ICPC\2025\1 - Holy Network (clique Iterativo).cpp

La otra solución correcta del problema H de ICPC 2025 ("Holy Network"): el grupo mas grande donde CADA PAR comparte un factor mayor que 1, o sea un clique maximo. A diferencia de la version con Bron-Kerbosch (el archivo "H - Holy Network" de esta carpeta), esta hace un backtracking directo y ademas ITERATIVO, con una pila explicita de estados (candidatos que quedan, tamaño del grupo). Vale tenerla porque el patron sirve para cualquier busqueda que se quiera sin recursion. LA PODA ES LO QUE LA HACE VIABLE: los candidatos se recorren en orden y, si lo que ya se llevo mas los que faltan por mirar no supera al mejor encontrado, se corta el ciclo de una (actual.tamGrupo + quedan <= mejorGrupo). Sin esa linea seria 2^n . Cada vez que se elige un candidato, los nuevos candidatos son SOLO los que vienen despues y ademas son compatibles con el elegido: así todo lo que quede en la pila ya es compatible con todo el grupo, y no hay que revalidar nada. El gcd se calcula a mano con el algoritmo de Euclides en vez de llamar a __gcd, lo que evita depender de extensiones del compilador o de <numeric> de C++17. Verificado contra fuerza bruta 2^n en 1500 casos, sin fallos, y pasa el sample (4, 5, 1) y el contraejemplo {6, 10, 35} que tumba la version con DSU.

```
1 #include <iostream>
2 #include <vector>
3 #include <algorithm>
4 #include <stack>
5
6 using namespace std;
7
8 struct Estado
9 {
10     vector<long long> candidatos;
11     long long tamGrupo;
12 };
13
14 int main()
15 {
16     ios::sync_with_stdio(false);
17     cin.tie(nullptr);
18
19     long long n;
20
21     while(cin >> n && n != 0)
22     {
23         vector<long long> resonancia(n);
24
25         for(long long i = 0; i < n; i++)
26         {
27             cin >> resonancia[i];
28         }
29
30         vector<vector<bool>> compatibles(n, vector<bool>(n, false));
31
32         for(long long a = 0; a < n; a++)
33         {
34             for(long long b = a + 1; b < n; b++)
35             {
36                 long long x = resonancia[a];
37                 long long y = resonancia[b];
38
39                 while(y != 0)
40                 {
41                     long long resto = x % y;
42                     x = y;
43                     y = resto;
44                 }
45
46                 if(x > 1)
47                 {
48                     compatibles[a][b] = true;
49                     compatibles[b][a] = true;
50                 }
51             }
52         }
53
54         vector<long long> todos(n);
55
56         for(long long i = 0; i < n; i++)
57         {
```

```
58             todos[i] = i;
59         }
60
61         stack<Estado> pila;
62         pila.push({todos, 0});
63
64         long long mejorGrupo = 0;
65
66         while(!pila.empty())
67         {
68             Estado actual = pila.top();
69             pila.pop();
70
71             mejorGrupo = max(mejorGrupo, actual.tamGrupo);
72
73             for(long long i = 0; i < (long long)actual.candidatos.size(); i++)
74             {
75                 long long quedan = actual.candidatos.size() - i;
76
77                 if(actual.tamGrupo + quedan <= mejorGrupo)
78                 {
79                     break;
80                 }
81
82                 long long elegido = actual.candidatos[i];
83
84                 vector<long long> nuevosCandidatos;
85
86                 for(long long j = i + 1; j < (long long)actual.candidatos.size(); j++)
87                 {
88                     if(compatibles[elegido][actual.candidatos[j]])
89                     {
90                         nuevosCandidatos.push_back(actual.candidatos[j]);
91                     }
92                 }
93
94                 pila.push({
95                     nuevosCandidatos,
96                     actual.tamGrupo + 1
97                 });
98             }
99         }
100
101         cout << mejorGrupo << '\n';
102     }
103
104     return 0;
105 }
```

3.14 Componentes Fuente con Kosaraju - Lumina Juan

C++\ICPC\2024\Lumina Juan.cpp

La otra solución del problema J de ICPC 2024 ("Lumina"): el mínimo de gemas a encender a mano para que la reacción en cadena las encienda todas. La respuesta es la cantidad de componentes fuertemente conexas SIN aristas entrantes desde otra componente. A diferencia de "J - Lumina" de esta carpeta, que usa Tarjan, esta usa KOSARAJU: un DFS sobre el grafo normal que apila los nodos por orden de terminación, y después un DFS sobre el grafo INVERSO tomando los nodos en orden inverso de esa pila. Cada árbol del segundo recorrido es una componente. CUAL CONVIENE: Kosaraju es mas facil de recordar (dos DFS y un reverse) pero necesita construir el grafo inverso, o sea el doble de memoria. Tarjan hace una sola pasada y no lo necesita, pero la logica de low[] y la pila es mas facil de escribir mal. Para un cuaderno vale tener las dos. OJO CON EL ORDEN DE NUMERACION: Kosaraju numera las componentes en orden topologico y Tarjan al reves. Aquí solo se cuentan las fuentes, así que da igual, pero en 2-SAT esa diferencia volteia la regla de asignación (ver "G - Signal Coverage" de esta carpeta). El grafo es DIRIGIDO e IMPLICITO: u alcanza a v si v cae dentro del radio de u, y como cada gema tiene su propio radio eso no es simétrico. Las aristas no se guardan, se recalculan con $\text{dist}^2 \leq r^2$ en enteros. Verificado con los tres casos del sample (1, 2, 2).

```
1 #include <iostream>
2 #include <vector>
3 #include <queue>
4 #include <stack>
5
6 using namespace std;
7
8 struct Circulo
9 {
```

```

0   long long x;
1   long long y;
2   long long r;
3   };
4
5   int main()
6   {
7       ios::sync_with_stdio(false);
8       cin.tie(nullptr);
9
10      long long n;
11
12      while(cin >> n && n != 0)
13      {
14          vector<Circulo> circulos(n);
15
16          for(long long i = 0; i < n; i++)
17          {
18              cin >> circulos[i].x >> circulos[i].y >> circulos[i].r;
19          }
20
21          vector<vector<int>> conexion(n);
22
23          for(long long a = 0; a < n; a++)
24          {
25              for(long long b = 0; b < n; b++)
26              {
27                  if(a == b)
28                  {
29                      continue;
30                  }
31
32                  long long dx = circulos[a].x - circulos[b].x;
33                  long long dy = circulos[a].y - circulos[b].y;
34
35                  long long distancia = dx * dx + dy * dy;
36
37                  if(distancia <= circulos[a].r * circulos[a].r)
38                  {
39                      conexion[a].push_back(b);
40                  }
41              }
42          }
43
44          vector<bool> visitado(n, false);
45          vector<long long> ordenFinal;
46
47          for(long long i = 0; i < n; i++)
48          {
49              if(visitado[i])
50              {
51                  continue;
52              }
53
54              stack<pair<long long, long long>> pila;
55              visitado[i] = true;
56              pila.push({i, 0});
57
58              while(!pila.empty())
59              {
60                  long long actual = pila.top().first;
61                  long long posicion = pila.top().second;
62
63                  if(posicion == (long long)conexion[actual].size())
64                  {
65                      ordenFinal.push_back(actual);
66                      pila.pop();
67                      continue;
68                  }
69
70                  long long siguiente = conexion[actual][posicion];
71                  pila.top().second++;
72

```

```

83                  if(!visitado[siguiente])
84                  {
85                      visitado[siguiente] = true;
86                      pila.push({siguiente, 0});
87                  }
88              }
89          }
90
91          vector<bool> encontrado(n, false);
92
93          long long contador = 0;
94
95          for(long long k = n - 1; k >= 0; k--)
96          {
97              long long inicio = ordenFinal[k];
98
99              if(encontrado[inicio])
100              {
101                  continue;
102              }
103
104              queue<long long> cola;
105              cola.push(inicio);
106              encontrado[inicio] = true;
107              contador++;
108
109              while(!cola.empty())
110              {
111                  long long actual = cola.front();
112                  cola.pop();
113
114                  for(long long siguiente : conexion[actual])
115                  {
116                      if(!encontrado[siguiente])
117                      {
118                          encontrado[siguiente] = true;
119                          cola.push(siguiente);
120                      }
121                  }
122              }
123          }
124
125          cout << contador << '\n';
126      }
127
128      return 0;
129  }

```

3.15 Componentes Fuente de la Condensacion (Tarjan) - Lumina

Resuelve "Lumina" (problema J, ICPC 2024). Cada gema ilumina a las que caen dentro de SU radio, y el código cuenta el mínimo de gemas que hay que encender a mano para que al final todas queden encendidas. EL GRAFO ES DIRIGIDO aunque venga de círculos: u alcanza a v si v esta dentro del radio de u, y como cada gema tiene su propio radio, que u ilumine a v no implica lo contrario. Además es IMPLICITO: las aristas no se guardan, se recalculan con $\text{dist}^2 \leq r^2$ (en enteros, sin raíz) cada vez que hacen falta. Son $O(n^2)$ aristas. LA IDEA: dentro de una componente fuertemente conexa, encender una gema enciende a todas. Así que se comprime cada SCC a un nodo y queda un DAG (la condensacion). En ese DAG, una componente a la que no le llega ninguna arista desde otra (grado de entrada 0) no la puede encender nadie de afuera: hay que encender una de sus gemas a mano. Y basta con eso, porque toda componente del DAG es alcanzable desde alguna fuente. La respuesta es la cantidad de componentes FUENTE. Las SCC salen con Tarjan: `disc[]` es el orden de descubrimiento, `low[]` el menor `disc` alcanzable sin salir de la pila, y cuando `low[u] == disc[u]`, u es la raíz de una componente y se saca de la pila hasta llegar a el. Las aristas que cruzan entre componentes distintas son las que marcan `hasIncoming`. CUANDO USAR: "mínimo de puntos de partida para alcanzarlo todo", "a cuantos hay que avisarle para que el rumor llegue a todos". Si además piden el mínimo de aristas a AGREGAR para que el grafo quede fuertemente conexo, la respuesta es $\max(\#fuentes, \#sumideros)$, salvo que ya sea una sola componente. OJO: Tarjan recursivo llega a profundidad n. Con aristas $O(n^2)$, n no puede ser muy grande y no hay problema, pero en un grafo con millones de nodos habría que hacerlo iterativo. AMBIGÜEDAD DEL ENUNCIADO: una frase define "cerca" de forma SIMETRICA ("si una gema esta dentro del radio de la otra"), y con esa lectura la respuesta sería solo el número de componentes conexas. Pero el ejemplo y la figura hablan del aura de CADA gema alcanzando a otras, que es la lectura dirigida de este código. Los tres samples dan igual con las dos. El caso que las separa: gemas en $(-10^9, 0)$ radio 10^9 , $(0, 0)$ radio 0 y $(10^9, 0)$ radio 10^9 da 2 dirigido y 1 simétrico. Verificado bajo la lectura dirigida contra fuerza bruta en 700 casos; $N = 5000$ en ~200 ms.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Gem {
5     long long x, y, r;
6 };
7
8 int n;
9 vector<Gem> gems;
10
11 vector<int> disc, low, st, component;
12 vector<bool> inStack;
13
14 int timer;
15 int sccCount;
16
17 void tarjan(int u) {
18     disc[u] = low[u] = timer++;
19     st.push_back(u);
20     inStack[u] = true;
21
22     for (int v = 0; v < n; v++) {
23         if (u == v) continue;
24
25         long long dx = gems[u].x - gems[v].x;
26         long long dy = gems[u].y - gems[v].y;
27
28         // u puede iluminar v
29         if (dx * dx + dy * dy > gems[u].r * gems[u].r) continue;
30
31         if (disc[v] == -1) {
32             tarjan(v);
33             low[u] = min(low[u], low[v]);
34         }
35         else if (inStack[v]) {
36             low[u] = min(low[u], disc[v]);
37         }
38     }
39 }
```

```
40 if (low[u] == disc[u]) {
41     while (true) {
42         int v = st.back();
43         st.pop_back();
44
45         inStack[v] = false;
46         component[v] = sccCount;
47
48         if (v == u) break;
49     }
50     sccCount++;
51 }
52
53 int main() {
54     ios::sync_with_stdio(false);
55     cin.tie(nullptr);
56
57     while (cin >> n && n != 0) {
58         gems.resize(n);
59
60         for (auto &gem : gems)
61             cin >> gem.x >> gem.y >> gem.r;
62
63         disc.assign(n, -1);
64         low.assign(n, 0);
65         component.assign(n, -1);
66         inStack.assign(n, false);
67
68         st.clear();
69
70         timer = 0;
71         sccCount = 0;
72
73         // Encontrar SCC
74         for (int i = 0; i < n; i++) {
75             if (disc[i] == -1)
76                 tarjan(i);
77         }
78
79         // inDegree de cada SCC
80         vector<bool> hasIncoming(sccCount, false);
81
82         // Revisar todas las aristas
83         for (int u = 0; u < n; u++) {
84             for (int v = 0; v < n; v++) {
85                 if (u == v) continue;
86
87                 long long dx = gems[u].x - gems[v].x;
88                 long long dy = gems[u].y - gems[v].y;
89
90                 // u -> v
91                 if (dx * dx + dy * dy <= gems[u].r * gems[u].r) {
92                     if (component[u] != component[v]) {
93                         hasIncoming[component[v]] = true;
94                     }
95                 }
96             }
97         }
98
99         int answer = 0;
100     }
```

```

82     for (int c = 0; c < sccCount; c++) {
83         if (!hasIncoming[c])
84             answer++;
85     }
86
87     cout << answer << '\n';
88 }
89
90 return 0;
91 }

```

3.16 Componentes Fuertemente Conexas (Kosaraju) - Planets And Kingdoms

C++\Training_Camp_2026\Day 5\I - Planets and Kingdoms.py

Resuelve "Planets and Kingdoms" (Day 5, problema I - Contest 05 [Avanzados]): dados n planetas y m teleportadores dirigidos, dos planetas pertenecen al mismo reino si y solo si existe una ruta de a a b y de b a a ; hay que hallar todos los reinos (componentes fuertemente conexas) y asignarle a cada planeta la etiqueta de su reino. Implementa Kosaraju con DFS iterativos: primero un DFS sobre el grafo original (matrix) que llena la pila "cam" en orden de finalizaci n, luego recorre "cam" en reversa haciendo DFS sobre el grafo transpuesto (matrix2, funci n dfs_gay) para etiquetar cada componente conexa como un reino distinto en "respuesta".

```

1 import sys
2 sys.setrecursionlimit(300000)
3
4 n, m = map(int, input().split())
5
6 matrix = [[] for _ in range(n)]
7 matrix2 = [[] for _ in range(n)]
8
9 for _ in range(m):
10     p, s = map(int, input().split())
11     p -= 1
12     s -= 1
13
14     matrix[p].append(s)
15     matrix2[s].append(p)
16
17
18 visited = [False] * n
19 cam = []
20
21 def dfs(start):
22     stack = [(start, 0)]
23     visited[start] = True
24
25     while stack:
26         nodo, estado = stack[-1]
27
28         if estado < len(matrix[nodo]):
29             vecino = matrix[nodo][estado]
30             stack[-1] = (nodo, estado + 1)
31
32             if not visited[vecino]:
33                 visited[vecino] = True
34                 stack.append((vecino, 0))
35
36         else:
37             cam.append(nodo)
38             stack.pop()
39
40
41 for i in range(n):
42     if not visited[i]:
43         dfs(i)
44
45
46 visited = [False] * n
47 respuesta = [0] * n
48
49 def dfs_gay(start, reino):
50     stack = [start]
51     visited[start] = True
52
53     while stack:

```

```

54         nodo = stack.pop()
55         respuesta[nodo] = reino
56
57         for vecino in matrix2[nodo]:
58             if not visited[vecino]:
59                 visited[vecino] = True
60                 stack.append(vecino)
61
62
63 voy = 0
64
65 for i in reversed(cam):
66     if not visited[i]:
67         voy += 1
68         dfs_gay(i, voy)
69
70
71 print(voy)
72 print(*respuesta)

```

3.17 DFS - Artwork

C++\Training_Camp_2026\Day 1\A - Artwork.py

Resuelve "Artwork" (Day 1, problema A - Maratona de Programacao da SBC 2019): un ladr n debe cruzar una sala rectangular $M \times N$, desde la esquina de entrada (0,0) hasta la esquina opuesta donde est  la obra, sin acercarse a menos de S metros de ning n sensor de movimiento i . Modela los sensores que se tocan entre s  (distancia entre centros $\leq$ suma de radios) como componentes conexas; si una componente que toca un borde de la sala llega a tocar el borde opuesto o perpendicular, bloquea el paso del ladr n. DFS desde cada sensor que toca un borde, propagandose por los sensores superpuestos y marcando visitados, para decidir si queda un camino libre ('S') o no ('N').

```

1 m, n, s = list(map(int, input().split()))
2 sensors = []
3
4 for i in range(s):
5     sensors.append(list(map(int, input().split())))
6 ans = "S"
7 visited = set()
8 def rec(x,y,r, start):
9     global ans, m, n
10
11     if ans == "N":
12         return
13     for xi,yi,ri in sensors:
14         if (xi, yi, ri) in visited:
15             continue
16         dist = ((x-xi)**2+(y-yi)**2)**(1/2)
17         if dist <= (r+ri):
18             if (start == "l" and (xi+ri >= m or yi-ri <= 0)) or (start == "d" and (xi-ri <= 0 or yi
19 +ri >= n)) or (start == "u" and (xi-ri >= m or yi-ri <= 0)) or (start == "r" and (yi+ri >= n or
20 x-ri <= 0)):
21                 ans = "N"
22                 break
23         visited.add((xi,yi,ri))
24         rec(xi,yi, ri, start)
25
26 for i, sensor in enumerate(sensors):
27     r = sensor[2]
28     x = sensor[0]
29     y = sensor[1]
30     if (x + r >= m and x - r <= 0) or (y + r >= n and y - r <= 0) or (x - r <= 0 and y - r <= 0) or
31 (x + r >= m and y + r >= n):
32         ans = "N"
33         break
34     if x-r<=0 or y-r<=0 or y+r>=n or x+r>=m:
35         dir = ""
36         if x-r<=0:
37             dir = "l"
38         elif y-r<=0:
39             dir = "d"
40         elif y+r>=n:
41             dir = "u"
42         elif x+r>=m:
43             dir = "r"

```

```

42     if dir != "":
43         visited = set()
44         rec(x,y,r, dir)
45 print(ans)

```

3.18 DFS - Denouncing Mafia

C++\Training_Camp_2026\Day 1\D - Denouncing_Mafia.py

Resuelve "Denouncing Mafia" (Day 1, problema D - Maratona de Programacao da SBC 2019): un arbol enraizado de N miembros de mafia (el 1 es el jefe, cada quien tiene un superior directo) donde interrogar a alguien obliga a subir denunciando a su superior hasta llegar al jefe; con un vidente que solo puede senalar a K miembros iniciales, se busca el maximo numero de mafiosos que se pueden arrestar en total. DFS que para cada nodo se queda con la rama de mayor profundidad para "seguir gratis" hacia la raiz, y registra el costo extra (diferencia de profundidad) de las demas ramas como candidatas; al final toma las K ramas de mayor costo y las suma.

```

1 import copy
2 import sys
3
4 sys.setrecursionlimit(10**6)
5 m, k = list(map(int, input().split()))
6 entry = list(map(int, input().split()))
7
8 currMemb = 1
9 dicc = {1: []}
10 for node in entry:
11     currMemb += 1
12     if node in dicc:
13         dicc[node] = dicc[node] + [currMemb]
14     else:
15         dicc[node] = [currMemb]
16
17 visited = set()
18 ans = []
19
20 def rec(curr, depth, prev):
21     global visited, ans
22     if curr in dicc:
23         children = dicc[curr]
24         first = True
25         leafsDepth = []
26         for son in children:
27             newDep = rec(son, depth + 1, curr)
28             leafsDepth.append(newDep)
29             leafsDepth.sort(reverse=True)
30             max = leafsDepth.pop(0)
31             for i in leafsDepth:
32                 ans.append(i - depth)
33             return max
34     else:
35         return depth
36 ans.append(rec(1, 1, None))
37 ans.sort(reverse=True)
38 sum = 0
39
40 curr = 0
41 while(curr < k and curr < len(ans)):
42     sum+=ans[curr]
43     curr+=1
44
45 print(sum)

```

3.19 DFS - A Mazing!

C++\Training_Camp_2026\Day 11 - CCPL - A Mazing!.py

Resuelve "A Mazing!" (Day 11, problema L - CCPL (Kattis)): un problema interactivo donde hay que explorar un laberinto de celdas cuadradas moviendose (up/down/left/right) segun las respuestas del juez ("wall", "ok", "solved"), sin conocer el mapa de antemano, hasta encontrar una salida o determinar que no existe ninguna (reportando "no way out"). Implementa un DFS iterativo sobre el grafo implicito de celdas: `todo` guarda por celda las direcciones aun no probadas (en un deque, excluyendo la direccion opuesta a la usada para entrar a esa celda) y `parent` guarda la celda predecesora; al agotar las direcciones de la celda actual, retrocede (backtrack) hacia su padre repitiendo el movimiento inverso, como una pila explicita de DFS.

```

1 import sys
2 from collections import deque
3

```

```

4 dirs = {
5     "up": (0, 1),
6     "down": (0, -1),
7     "left": (-1, 0),
8     "right": (1, 0),
9 }
10
11 opposite = {
12     "up": "down",
13     "down": "up",
14     "left": "right",
15     "right": "left",
16 }
17
18 curr = (0, 0)
19
20 todo = {
21     curr: deque(["down", "right", "left", "up"])
22 }
23
24 parent = {}
25
26 while True:
27
28     if todo[curr]:
29         move = todo[curr].popleft()
30
31         print(move, flush=True)
32         ans = input()
33
34         if ans == "solved" or ans == "wrong":
35             break
36
37         if ans == "wall":
38             continue
39
40         dx, dy = dirs[move]
41         nxt = (curr[0] + dx, curr[1] + dy)
42
43         if nxt not in todo:
44             todo[nxt] = deque(["down", "right", "left", "up"])
45             # Ya usamos la direccion opuesta para llegar aqui
46             todo[nxt].remove(opposite[move])
47             parent[nxt] = curr
48
49         curr = nxt
50
51     else:
52
53         if curr == (0, 0):
54             print("no way out", flush=True)
55             ans = input()
56             break
57
58         par = parent[curr]
59
60         dx = par[0] - curr[0]
61         dy = par[1] - curr[1]
62
63         if dx == 1:
64             move = "right"
65         elif dx == -1:
66             move = "left"
67         elif dy == 1:
68             move = "up"
69         else:
70             move = "down"
71
72         print(move, flush=True)
73         ans = input()
74
75         if ans == "solved" or ans == "wrong":
76             break

```

```

77
78     curr = par

```

3.20 Dijkstra - Fantastic Robot

C++ Training_Camp_2020/Day 9/E - Fantastic Robot.cpp

Resuelve "Fantastic Robot" (Day 9, problema E - Contest 08 [Inicial & Avanzado]): HERBIE debe llegar desde una celda a otra en una matriz binaria (0 = libre, 1 = pared) pero no puede moverse instrucción por instrucción, sino que debe ejecutar completa una de sus "ordenes" preprogramadas (cadenas de movimientos U/D/L/R), donde cada movimiento bloqueado por una pared o el borde simplemente no cuenta como paso. Modela cada orden como una arista de costo variable (los "pasos" realmente efectivos) entre la celda inicial y la celda final de aplicar esa orden completa (funcion generar_vecinos), y corre Dijkstra con priority_queue sobre ese grafo implícito para hallar el mínimo total de movimientos hasta la celda objetivo.

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  using ll = long long;
5
6  const ll INF = 1e18;
7
8
9  int n,m;
10
11 vector<vector<int>> mat;
12 vector<string> comandos;
13
14
15 vector<tuple<int,int,int>> generar_vecinos(int x, int y){
16
17     vector<tuple<int,int,int>> vecinos;
18
19     for(string &cmd : comandos){
20
21         int xi=x;
22         int yi=y;
23         int pasos=0;
24
25
26         for(char c: cmd){
27
28             if(c=='U'){
29                 pasos++;
30                 xi--;
31
32                 if(xi<0 || mat[xi][yi]==1){
33                     pasos--;
34                     xi++;
35                 }
36
37             }else if(c=='D'){
38
39                 pasos++;
40                 xi++;
41
42                 if(xi>n || mat[xi][yi]==1){
43                     pasos--;
44                     xi--;
45                 }
46
47             }else if(c=='L'){
48
49                 pasos++;
50                 yi--;
51
52                 if(yi<0 || mat[xi][yi]==1){
53                     pasos--;
54                     yi++;
55                 }
56
57             }else{
58
59                 pasos++;
60                 yi++;
61

```

```

62
63         if(yi>m || mat[xi][yi]==1){
64             pasos--;
65             yi--;
66         }
67     }
68 }
69
70
71     vecinos.push_back({xi,yi,pasos});
72 }
73
74
75     return vecinos;
76 }
77
78
79
80 int main(){
81
82     ios::sync_with_stdio(false);
83     cin.tie(nullptr);
84
85     cin>>n>>m;
86
87     mat.assign(n,vector<int>(m));
88
89     for(int i=0;i<n;i++){
90
91         string s;
92         cin>>s;
93
94         for(int j=0;j<m;j++){
95             mat[i][j]=s[j]-'0';
96         }
97
98     int sx,sy;
99     cin>>sx>>sy;
100
101     int tx,ty;
102     cin>>tx>>ty;
103
104     int k;
105     cin>>k;
106
107     comandos.resize(k);
108
109     for(auto &x:comandos)
110         cin>>x;
111
112
113     vector<vector<ll>> dist(n, vector<ll>(m,INF));
114
115     priority_queue<
116         tuple<ll,int,int>,
117         vector<tuple<ll,int,int>>,
118         greater<tuple<ll,int,int>>
119     > pq;
120
121     dist[sx][sy]=0;
122
123     pq.push({0,sx,sy});

```

```

85
86
87 while(!pq.empty()){
88
89
90     auto [d,x,y]=pq.top();
91     pq.pop();
92
93     if(d!=dist[x][y])
94         continue;
95
96     if(x==tx && y==ty)
97         break;
98
99
100     for(auto [nx,ny,costo] : generar_vecinos(x,y)){
101
102         if(dist[nx][ny] > d+costo){
103
104             dist[nx][ny]=d+costo;
105
106             pq.push({
107                 dist[nx][ny],
108                 nx,
109                 ny
110             });
111         }
112     }
113
114     if(dist[tx][ty]==INF)
115         cout<<-1<<"\n";
116     else
117         cout<<dist[tx][ty]<<"\n";
118 }

```

3.21 Dijkstra con Nodos Virtuales - Chinese Innovation

C++ Training_Camp_2020_Day 10/C - Chinese Innovation.cpp

Resuelve "Chinese Innovation" (Day 10, problema C - Contest 09 [Avanzados]): un imperio de n ciudades con m caminos bidireccionales con costo, mas k tipos de teletransportadores; dos ciudades con teletransportador del mismo tipo se conectan instantaneamente, pero el costo de usarlo depende de la ciudad de origen (cada mayor cobra su propia tarifa de salida). Hay que hallar el costo m nimo de viajar de la ciudad 1 a la ciudad n . Modela cada tipo de teletransportador t como un nodo virtual ($\text{virtual_node} = n + t$): cada ciudad con ese tipo agrega una arista dirigida hacia el nodo virtual con peso igual a su tarifa de salida, y el nodo virtual agrega aristas de costo 0 hacia todas esas ciudades; as , saltar entre ciudades del mismo tipo cuesta solo la tarifa de la ciudad de origen. Corre Dijkstra estandar (con *priority_queue* y distancias en un *unordered_map*) desde 1 hasta n sobre este grafo ampliado.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using pii = pair<long long, int>;
5
6 const long long INF = 1e18;
7
8 long long dijkstra(unordered_map<int, vector<pair<int, int>>> &graph, int start, int target) {
9     unordered_map<int, long long> dist;
10    unordered_set<int> visited;
11
12    priority_queue<pii, vector<pii>, greater<pii>> pq;
13
14    dist[start] = 0;
15    pq.push({0, start});
16
17    while (!pq.empty()) {
18        auto [d, u] = pq.top();

```

```

9        pq.pop();
10
11        if (visited.count(u))
12            continue;
13        visited.insert(u);
14
15        if (u == target)
16            return d;
17
18        for (auto [v, w] : graph[u]) {
19
20            long long nd = d + w;
21
22            if (!dist.count(v) || nd < dist[v]) {
23                dist[v] = nd;
24                pq.push({nd, v});
25            }
26        }
27    }
28
29    if (dist.count(target))
30        return dist[target];
31    return INF;
32 }
33
34 int main() {
35     ios::sync_with_stdio(false);
36     cin.tie(nullptr);
37
38     int n, m, k;
39     cin >> n >> m >> k;
40
41     unordered_map<int, vector<pair<int, int>>> graph;
42
43     for (int i = 0; i < m; i++) {
44         int a, b, w;
45         cin >> a >> b >> w;
46         graph[a].push_back({b, w});
47         graph[b].push_back({a, w});
48     }
49
50     for (int i = 1; i <= n; i++) {
51         int tel;
52         cin >> tel;
53
54         for (int j = 0; j < tel; j++) {
55             int t, w;
56             cin >> t >> w;
57
58             int virtual_node = n + t;
59
60             graph[i].push_back({virtual_node, w});
61             graph[virtual_node].push_back({i, 0});
62         }
63     }
64
65     long long costo = dijkstra(graph, 1, n);
66     cout << costo << "\n";
67
68     return 0;
69 }

```

3.22 DSU (enfoque INCORRECTO para este problema) - Holy Network (DSU, Incorrecta)

C++ ICPC2025H - Holy Network (DSU, incorrecta).cpp

Intento del problema H de ICPC 2025 ("Holy Network") con DSU: une dos numeros cuando comparten un factor, y responde el tamaño de la componente mas grande. ESTA VERSION ESTA MAL Y SE GUARDA JUSTAMENTE POR ESO. El enunciado pide que CADA PAR del grupo comparta un factor, no que el grupo este conectado. El contraejemplo mínimo es {6, 10, 35}: 6-10 comparten el 2 y 10-35 comparten el 5, así que la componente mide 3, pero 6 y 35 son coprimos y el grupo valido mide 2. Medido: falla 872 de 1500 casos aleatorios contra fuerza bruta, aunque pasa los tres casos del sample, donde ambas cosas coinciden. La leccion que vale guardar: "todos conectados" y "todos compatibles entre si" son cosas distintas. Lo primero es una componente conexa (fácil, DSU o BFS); lo segundo es un CLIQUE (NP-difícil en general). Si un enunciado dice "cada par", "todos con todos" o "mutuamente", casi seguro es clique y DSU no sirve. La version correcta esta en "H - Holy Network" (Bron-Kerbosch con pivote) y en "H - Holy Network (clique iterativo)", ambas en esta carpeta. Lo unico reutilizable de aqui es como se arman las aristas: en vez de comparar todos los pares con gcd, se factoriza cada numero probando divisores hasta la raíz y se une cada numero con sus factores, usando un map para los primos que aparecen. Eso da $O(n * \sqrt{V})$ en vez de $O(n^2 \log V)$, que ayuda cuando n es grande, aunque aqui $n \leq 50$ y no hace falta.

```
1 #include <iostream>
2 #include <vector>
3 #include <map>
4
5 using namespace std;
6
7 long long buscar(map<long long, long long>& padre, long long x)
8 {
9     if(padre[x] == x)
10     {
11         return x;
12     }
13
14     return padre[x] = buscar(padre, padre[x]);
15 }
16
17 void unir(map<long long, long long>& padre, map<long long, long long>& tam, long long a, long long b)
18 {
19     a = buscar(padre, a);
20     b = buscar(padre, b);
21
22     if(a == b)
23     {
24         return;
25     }
26
27     if(tam[a] < tam[b])
28     {
29         swap(a, b);
30     }
31
32     padre[b] = a;
33     tam[a] += tam[b];
34 }
35
36 int main()
37 {
38     ios::sync_with_stdio(false);
39     cin.tie(nullptr);
40
41     long long n;
42
43     while(cin >> n && n != 0)
44     {
45         vector<long long> numeros(n);
46
47         map<long long, bool> esInput;
48         map<long long, long long> padre;
49         map<long long, long long> tam;
50
51         for(long long i = 0; i < n; i++)
52         {
53             cin >> numeros[i];
54
55             esInput[numeros[i]] = true;
```

```
56         padre[numeros[i]] = numeros[i];
57         tam[numeros[i]] = 1;
58     }
59
60     for(long long x : numeros)
61     {
62         for(long long i = 2; i * i <= x; i++)
63         {
64             if(x % i == 0)
65             {
66                 long long z = x / i;
67
68                 if(!padre.count(i))
69                 {
70                     padre[i] = i;
71                     tam[i] = 1;
72                 }
73
74                 if(!padre.count(z))
75                 {
76                     padre[z] = z;
77                     tam[z] = 1;
78                 }
79
80                 unir(padre, tam, x, i);
81                 unir(padre, tam, x, z);
82             }
83         }
84     }
85
86     map<long long, long long> familias;
87     long long maxValor = 0;
88
89     for(long long x : numeros)
90     {
91         long long raiz = buscar(padre, x);
92
93         familias[raiz]++;
94
95         maxValor = max(maxValor, familias[raiz]);
96     }
97
98     cout << maxValor << '\n';
99 }
100
101 return 0;
102 }
```

3.23 Heavy-Light Decomposition - Los Ratones III

C++ Training_Camp_2020Day 9K - Los Ratones III.cpp

Resuelve "Los Ratones III" (Day 9, problema K - Contest 08 [Inicial & Avanzado]): un arbol de n casas conectadas por calles con valor, donde Q cartas piden la suma de los valores de las aristas en el camino entre u y v despues de multiplicar TODO ese camino por x, y las modificaciones son persistentes (afectan a las cartas siguientes). Usa Heavy-Light Decomposition (dfs para pesos/hijo pesado, decompose para aplanar el arbol en posiciones de arreglo) sobre un segment tree con lazy propagation MULTIPLICATIVO (push, seg[node].lazy) para poder multiplicar y sumar rangos de aristas en el camino en $O(\log^2 N)$ por consulta (path_update/path_query), todo modulo $1e9+7$.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 const ll MOD = 1000000007LL;
7 const int MAXN = 100005;
8
9 struct Edge{
10     int to;
11     ll w;
12 };
13
14 int N,Q;
15
16 vector<Edge> g[MAXN];
```

```

7
8 int parent[MAXN], depth[MAXN], heavy[MAXN], sz[MAXN];
9 int head[MAXN], pos[MAXN];
10 ll val[MAXN];
21 int curPos;
22
23 struct Node{
24     ll sum,lazy;
25 };
26
27 Node seg[4*MAXN];
28
29 int dfs(int v,int p){
30     parent[v]=p;
31     sz[v]=1;
32     heavy[v]=-1;
33     int mx=0;
34     for(auto e:g[v]){
35         if(e.to==p) continue;
36         depth[e.to]=depth[v]+1;
37         val[e.to]=e.w;
38         dfs(e.to,v);
39         sz[v]+=sz[e.to];
40         if(sz[e.to]>mx){
41             mx=sz[e.to];
42             heavy[v]=e.to;
43         }
44     }
45     return sz[v];
46 }
47
48 ll base[MAXN];
49
50 void decompose(int v,int h){
51     head[v]=h;
52     pos[v]++;curPos;
53     base[curPos]=val[v];
54
55     if(heavy[v]!=-1)
56         decompose(heavy[v],h);
57
58     for(auto e:g[v]){
59         if(e.to==parent[v] || e.to==heavy[v]) continue;
60         decompose(e.to,e.to);
61     }
62 }
63
64 void build(int node,int l,int r){
65     seg[node].lazy=1;
66     if(l==r){
67         seg[node].sum=base[l]%MOD;
68         return;
69     }
70     int mid=(l+r)>>1;
71     build(node<<1,l,mid);
72     build(node<<1|1,mid+1,r);
73     seg[node].sum=(seg[node<<1].sum+seg[node<<1|1].sum)%MOD;
74 }
75
76 void push(int node,int l,int r){
77     if(seg[node].lazy==1) return;
78     ll x=seg[node].lazy;
79     seg[node].sum=seg[node].sum*x%MOD;
80     if(l!=r){
81         seg[node<<1].lazy=seg[node<<1].lazy*x%MOD;
82         seg[node<<1|1].lazy=seg[node<<1|1].lazy*x%MOD;
83     }
84     seg[node].lazy=1;
85 }
86
87 void update(int node,int l,int r,int ql,int qr,ll x){
88     push(node,l,r);
89     if(r<ql||qr<l) return;

```

```

90     if(ql<=l&&r<=qr){
91         seg[node].lazy=x%MOD;
92         push(node,l,r);
93         return;
94     }
95     int mid=(l+r)>>1;
96     update(node<<1,l,mid,ql,qr,x);
97     update(node<<1|1,mid+1,r,ql,qr,x);
98     seg[node].sum=(seg[node<<1].sum+seg[node<<1|1].sum)%MOD;
99 }
100
101 ll query(int node,int l,int r,int ql,int qr){
102     push(node,l,r);
103     if(r<ql||qr<l) return 0;
104     if(ql<=l&&r<=qr) return seg[node].sum;
105     int mid=(l+r)>>1;
106     return (query(node<<1,l,mid,ql,qr)+
107             query(node<<1|1,mid+1,r,ql,qr))%MOD;
108 }
109
110 void path_update(int a,int b,ll x){
111     while(head[a]!=head[b]){
112         if(depth[head[a]]<depth[head[b]]) swap(a,b);
113         update(1,1,N,pos[head[a]],pos[a],x);
114         a=parent[head[a]];
115     }
116     if(depth[a]>depth[b]) swap(a,b);
117     if(pos[a]+1<=pos[b])
118         update(1,1,N,pos[a]+1,pos[b],x);
119 }
120
121 ll path_query(int a,int b){
122     ll ans=0;
123     while(head[a]!=head[b]){
124         if(depth[head[a]]<depth[head[b]]) swap(a,b);
125         ans=(ans+query(1,1,N,pos[head[a]],pos[a]))%MOD;
126         a=parent[head[a]];
127     }
128     if(depth[a]>depth[b]) swap(a,b);
129     if(pos[a]+1<=pos[b])
130         ans=(ans+query(1,1,N,pos[a]+1,pos[b]))%MOD;
131     return ans;
132 }
133
134 int main(){
135     ios::sync_with_stdio(false);
136     cin.tie(nullptr);
137
138     cin>>N>>Q;
139
140     for(int i=1;i<N;i++){
141         int u,v;
142         ll c;
143         cin>>u>>v>>c;
144         g[u].push_back({v,c});
145         g[v].push_back({u,c});
146     }
147
148     depth[1]=0;
149     val[1]=0;
150
151     dfs(1,0);
152     decompose(1,1);
153
154     build(1,1,N);
155
156     while(Q--){
157         int u,v;
158         ll x;
159         cin>>u>>v>>x;
160
161         path_update(u,v,x);
162         cout<<path_query(u,v)%MOD<<"\n";

```

```

63     }
64
65     return 0;
66 }

```

3.24 Kruskal Reconstruction Tree - Trucks

C++\Training_Camp_2026\Day 2U - Trucks.cpp

Resuelve "Trucks" (Day 2, problema J - Maratona de Programacao da SBC 2013): Nlogonia es un archipiélago de N islas unidas por M puentes, cada uno con un límite máximo de peso W ; para S sitios de competencia hay que hallar el mayor peso que puede transportarse por camión entre el almacén H y el sitio L , es decir, el máximo sobre todos los caminos posibles del mínimo W en ese camino (cuello de botella máximo). Ordena los puentes por peso decreciente y los va uniendo con un DSU al estilo Kruskal, pero en vez de solo unir arma un árbol binario de reconstrucción (`nodeRepr/leftChild/rightChild`) donde cada fusión crea un nodo interno con el peso del puente. El ancestro común mas bajo (LCA, calculado con binary lifting) de dos hojas en ese árbol es justamente el nodo cuyo peso es la respuesta a la consulta.

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  static char buf_[1 << 22];
5  static int buflen_, bufPos_;
6  inline int gc() {
7      if (bufPos_ == buflen_) { buflen_ = fread(buf_, 1, sizeof(buf_), stdin); bufPos_ = 0; if (!buflen_) return -1; }
8      return buf_[bufPos_++];
9  }
10 inline int readInt() {
11     int c = gc();
12     while (c != -1 && (c < '0' || c > '9') && c != '-') c = gc();
13     bool neg = false; if (c == '-') { neg = true; c = gc(); }
14     int x = 0;
15     while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = gc(); }
16     return neg ? -x : x;
17 }
18
19 int N, M, S;
20 vector<int> dsuParent;
21 int find(int x) {
22     while (dsuParent[x] != x) { dsuParent[x] = dsuParent[dsuParent[x]]; x = dsuParent[x]; }
23     return x;
24 }
25
26 int main() {
27     N = readInt(); M = readInt(); S = readInt();
28
29     vector<array<int,3>> edgeList(M); // {W, A, B} for easy sort by weight
30     for (int i = 0; i < M; i++) {
31         int a = readInt(), b = readInt(), w = readInt();
32         edgeList[i] = {w, a, b};
33     }
34     sort(edgeList.begin(), edgeList.end(), [](const array<int,3>&x, const array<int,3>&y){
35         return x[0] > y[0]; // decreasing weight
36     });
37
38     int totalNodes = 2 * N; // leaves 1..N, internal nodes N+1..2N-1 (at most N-1 of them)
39     dsuParent.assign(totalNodes + 1, 0);
40     iota(dsuParent.begin(), dsuParent.end(), 0);
41
42     vector<int> nodeRepr(totalNodes + 1, 0);
43     for (int i = 1; i <= N; i++) nodeRepr[i] = i;
44
45     vector<int> leftChild(totalNodes + 1, 0), rightChild(totalNodes + 1, 0), weightOf(totalNodes + 1, 0);
46
47     int nextInternal = N + 1;
48     int merges = 0;
49     for (auto &e : edgeList) {
50         if (merges == N - 1) break;
51         int w = e[0], a = e[1], b = e[2];
52         int ra = find(a), rb = find(b);
53         if (ra == rb) continue;
54         int newNode = nextInternal++;
55         weightOf[newNode] = w;

```

```

56         leftChild[newNode] = nodeRepr[ra];
57         rightChild[newNode] = nodeRepr[rb];
58         dsuParent[ra] = rb; // union
59         nodeRepr[rb] = newNode; // rb is now the representative root (arbitrary choice), pointing to
60         o new node
61         merges++;
62     }
63
64     int root = nodeRepr[find(1)];
65
66     int totalUsed = nextInternal - 1; // highest node id actually used
67     int LOG = 1;
68     while ((1 << LOG) < totalUsed + 1) LOG++;
69     vector<vector<int>> up(LOG, vector<int>(totalUsed + 1, 0));
70     vector<int> depth(totalUsed + 1, 0);
71
72     // iterative pre-order traversal to set depth & up[0]
73     {
74         vector<int> stk;
75         stk.push_back(root);
76         depth[root] = 0;
77         up[0][root] = 0;
78         while (!stk.empty()) {
79             int u = stk.back(); stk.pop_back();
80             if (u <= N) continue; // leaf, no children
81             int lc = leftChild[u], rc = rightChild[u];
82             depth[lc] = depth[u] + 1; up[0][lc] = u; stk.push_back(lc);
83             depth[rc] = depth[u] + 1; up[0][rc] = u; stk.push_back(rc);
84         }
85     }
86     for (int k = 1; k < LOG; k++) {
87         for (int v = 1; v <= totalUsed; v++) {
88             up[k][v] = up[k-1][up[k-1][v]];
89         }
90     }
91
92     auto lca = [&](int u, int v) -> int {
93         if (depth[u] < depth[v]) swap(u, v);
94         int diff = depth[u] - depth[v];
95         for (int k = 0; k < LOG; k++) if ((diff >> k) & 1) u = up[k][u];
96         if (u == v) return u;
97         for (int k = LOG - 1; k >= 0; k--) {
98             if (up[k][u] != up[k][v]) { u = up[k][u]; v = up[k][v]; }
99         }
100         return up[0][u];
101     };
102
103     string out;
104     out.reserve(S * 8);
105     for (int i = 0; i < S; i++) {
106         int L = readInt(), H = readInt();
107         int anc = lca(L, H);
108         out += to_string(weightOf[anc]);
109         out += "\n";
110     }
111     fwrite(out.data(), 1, out.size(), stdout);
112     return 0;
113 }

```

3.25 LCA (Binary Lifting) - Subway Lines

C++\Training_Camp_2026\Day 8U - Subway Lines.cpp

Resuelve "Subway Lines" (Day 8, problema L - Contest 08 [Avanzados]): en un sistema de subte con forma de árbol, las líneas de tren van y vuelven entre pares de estaciones terminales (hojas), pasando por el único camino entre ellas; dadas Q consultas con dos pares de terminales (A,B) y (C,D) , hay que calcular cuantas estaciones tienen en común los caminos que definen esas dos líneas. Precalcula profundidad y ancestros con binary lifting (`bfsSetup/up_`) para responder `lca(u,v)` y `dist(u,v)` en $O(\log N)$, y solveQuery halla la intersección de los dos caminos comparando las profundidades de `lca(a,b)` y `lca(c,d)`, usando `lca/isAncestor` para ubicar los extremos de la intersección y devolver su longitud (`dist+1`) o 0 si los caminos no se cruzan.

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  static char buf_[1 << 22];
5  static int buflen_, bufPos_;
6  inline int gc() {

```

```

    if (bufPos_ == bufLen_) { bufLen_ = fread(buf_, 1, sizeof(buf_), stdin); bufPos_ = 0; if (!bufL
    en_) return -1; }
    return buf_[bufPos_++];
}
inline int readInt() {
    int c = gc();
    while (c != -1 && (c < '0' || c > '9') && c != '-') c = gc();
    bool neg = false; if (c == '-') { neg = true; c = gc(); }
    int x = 0;
    while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = gc(); }
    return neg ? -x : x;
}
int N, Q;
vector<vector<int>> adj;
vector<int> depth_;
int LOG_;
vector<vector<int>> up_;
void bfsSetup(int root) {
    depth_.assign(N + 1, -1);
    up_.assign(LOG_, vector<int>(N + 1, root));
    depth_[root] = 0;
    up_[0][root] = root;
    queue<int> q;
    q.push(root);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : adj[u]) {
            if (depth_[v] == -1) {
                depth_[v] = depth_[u] + 1;
                up_[0][v] = u;
                q.push(v);
            }
        }
    }
    for (int k = 1; k < LOG_; k++) {
        for (int v = 1; v <= N; v++) {
            up_[k][v] = up_[k - 1][up_[k - 1][v]];
        }
    }
}
int lca(int u, int v) {
    if (depth_[u] < depth_[v]) swap(u, v);
    int diff = depth_[u] - depth_[v];
    for (int k = 0; k < LOG_; k++) if ((diff >> k) & 1) u = up_[k][u];
    if (u == v) return u;
    for (int k = LOG_ - 1; k >= 0; k--) {
        if (up_[k][u] != up_[k][v]) { u = up_[k][u]; v = up_[k][v]; }
    }
    return up_[0][u];
}
inline int dist(int u, int v) {
    return depth_[u] + depth_[v] - 2 * depth_[lca(u, v)];
}
inline bool isAncestor(int u, int v) {
    return lca(u, v) == u;
}
int solveQuery(int a, int b, int c, int d) {
    int l1 = lca(a, b), l2 = lca(c, d);
    if (depth_[l1] > depth_[l2]) {
        swap(a, c); swap(b, d); swap(l1, l2);
    }
    // is l2 on path(a,b)?
    if (dist(a, l2) + dist(l2, b) != dist(a, b)) return 0;
    if (l1 == l2) {
        int lac = lca(a, c), lad = lca(a, d);
        int deepA = (depth_[lac] >= depth_[lad]) ? lac : lad;

```

```

        int lbc = lca(b, c), lbd = lca(b, d);
        int deepB = (depth_[lbc] >= depth_[lbd]) ? lbc : lbd;
        return dist(deepA, deepB) + 1;
    } else {
        if (isAncestor(l2, a)) {
            int lac = lca(a, c), lad = lca(a, d);
            int deepA = (depth_[lac] >= depth_[lad]) ? lac : lad;
            return dist(l2, deepA) + 1;
        } else {
            int lbc = lca(b, c), lbd = lca(b, d);
            int deepB = (depth_[lbc] >= depth_[lbd]) ? lbc : lbd;
            return dist(l2, deepB) + 1;
        }
    }
}
int main() {
    N = readInt(); Q = readInt();
    adj.assign(N + 1, {});
    for (int i = 0; i < N - 1; i++) {
        int u = readInt(), v = readInt();
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    LOG_ = 1;
    while ((1 << LOG_) < N + 1) LOG_++;
    bfsSetup(1);
    string out;
    out.reserve(Q * 7);
    char tmp[16];
    for (int q = 0; q < Q; q++) {
        int a = readInt(), b = readInt(), c = readInt(), d = readInt();
        int res = solveQuery(a, b, c, d);
        int len = sprintf(tmp, "%d\n", res);
        out.append(tmp, len);
    }
    fwrite(out.data(), 1, out.size(), stdout);
    return 0;
}

```

3.26 Max Flow - Download Speed

C++ Training Camp 2026 (Day 7) - Download Speed.cpp

Resuelve "Download Speed" (Day 7, problema H - Contest 07 [Avanzados]); dada una red de n computadoras y m conexiones dirigidas con capacidad c , hay que hallar la velocidad m xima a la que Kotivalo (computadora n) puede descargar datos del servidor (computadora 1) usando la red. Es el problema cl sico de flujo m ximo: la struct `network` implementa Dinic (BFS por niveles para construir el grafo por capas + DFS de flujo bloqueante con aristas residuales), y `max_flow(1, a)` calcula el flujo m ximo entre el servidor y la computadora de Kotivalo, que es exactamente la velocidad de descarga pedida.

```

#include <bits/stdc++.h>
typedef long long ll;
using namespace std;
struct edge { ll v, cap, inv, flow; };
struct network
{
    ll n, s, t;
    vector<ll> lvl;
    vector<vector<edge>> g;
    network(ll n) : n(n), lvl(n), g(n) {}
    void add_edge(ll u, ll v, ll c)
    {
        g[u].push_back({v, c, (ll)g[v].size(), 0});
        g[v].push_back({u, 0, (ll)g[u].size()-1, c});
    }
    bool bfs()
    {
        fill(lvl.begin(), lvl.end(), -1);

```

```

24 queue<ll> q;
25 lvl[s] = 0;
26 for(q.push(s); q.size(); q.pop())
27 {
28     ll u = q.front();
29     for(auto &e : g[u]) {
30         if(e.cap > 0 && lvl[e.v] == -1)
31         {
32             lvl[e.v] = lvl[u]+1;
33             q.push(e.v);
34         }
35     }
36 }
37 return lvl[t] != -1;
38 }
39
40 ll dfs(ll u, ll nf)
41 {
42     if(u == t) return nf;
43     ll res = 0;
44     for
45     (auto &e : g[u])
46     {
47         if(e.cap > 0 && lvl[e.v] == lvl[u]+1)
48         {
49             ll tf = dfs(e.v, min(nf, e.cap));
50             res += tf; nf -= tf; e.cap -= tf;
51             g[e.v][e.inv].cap += tf;
52             g[e.v][e.inv].flow -= tf;
53             e.flow += tf;
54             if(nf == 0) return res;
55         }
56     }
57     if(!res) lvl[u] = -1;
58     return res;
59 }
60
61 ll max_flow(ll so, ll si, ll res = 0) {
62     s = so; t = si;
63     while(bfs()) res += dfs(s, 1000000000000);
64     return res;
65 }
66 };
67
68 int main()
69 {
70
71     ll a,b; cin >> a >> b;
72     network pcs(a+1);
73     for(int i = 0 ; i < b;i++)
74     {
75         ll x,y,z;cin >> x >> y >> z;
76         pcs.add_edge(x,y,z);
77     }
78     cout << pcs.max_flow(1,a)<< endl;
79
80     return 0;
81 }

```

3.27 Max Flow - March Of The Penguins

C++11 Training_Camp_2026/Day 7/J - March of the Penguins.cpp

Resuelve "March of the Penguins" (Day 7, problema J - Contest 07 [Avanzados]): hay N témpanos de hielo con coordenadas (x,y) , cada uno con n_i pingüinos y un límite m_i de saltos que se pueden dar desde el (un pingüino puede saltar de un témpano a otro si la distancia entre ellos es $\leq D$); hay que hallar todos los témpanos donde TODOS los pingüinos puedan terminar reunidos sin que ningún témpano se quede sin saltos. Para cada témpano candidato t , arma una red de flujo con nodos entrada/salida por témpano ($in=i$, $out=N+i$) donde la arista $in \rightarrow out$ tiene capacidad m_i (presupuesto de saltos), una fuente S conectada a cada témpano con capacidad n_i (pingüinos que parten de ahí), y aristas de capacidad infinita entre témpanos alcanzables saltando; corre Dinic (maxflow) de S a t , si el flujo maximo iguala al total de pingüinos, t es una respuesta valida.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3

```

```

4 struct Dinic {
5     struct Edge { int to; int cap; int flow; };
6     int n;
7     vector<Edge> edges;
8     vector<vector<int>> g;
9     vector<int> level, it;
10    Dinic(int n=0): n(n), g(n) {}
11    int addEdge(int from,int to,int cap){
12        g[from].push_back((int)edges.size());
13        edges.push_back({to,cap,0});
14        g[to].push_back((int)edges.size());
15        edges.push_back({from,0,0});
16        return (int)edges.size()-2;
17    }
18    void resetFlow(){ for(auto &e: edges) e.flow = 0; }
19    bool bfs(int s,int t){
20        level.assign(n,-1);
21        queue<int>q; level[s]=0;q.push(s);
22        while(!q.empty()){
23            int u=q.front();q.pop();
24            for(int id: g[u]){
25                auto &e=edges[id];
26                if(e.cap - e.flow>0 && level[e.to]<0){
27                    level[e.to]=level[u]+1;
28                    q.push(e.to);
29                }
30            }
31        }
32        return level[t]>=0;
33    }
34    int dfs(int u,int t,int f){
35        if(u==t) return f;
36        for(int &i=it[u]; i<(int)g[u].size(); i++){
37            int id=g[u][i];
38            auto &e=edges[id];
39            if(e.cap-e.flow>0 && level[e.to]==level[u]+1){
40                int d=dfs(e.to,t,min(f,e.cap-e.flow));
41                if(d>0){ e.flow+=d; edges[id^1].flow-=d; return d; }
42            }
43        }
44        return 0;
45    }
46    int maxflow(int s,int t){
47        int flow=0;
48        while(bfs(s,t)){
49            it.assign(n,0);
50            int f;
51            while((f=dfs(s,t,INT_MAX))>0) flow+=f;
52        }
53        return flow;
54    }
55 };
56
57 int main(){
58     ios::sync_with_stdio(false);
59     cin.tie(nullptr);
60
61     int T;
62     if(!(cin >> T)) return 0;
63     while(T--){
64         int N; double D;
65         cin >> N >> D;
66         vector<double> x(N), y(N);
67         vector<int> n(N), m(N);
68         for(int i=0;i<N;i++) cin >> x[i] >> y[i] >> n[i] >> m[i];
69
70         int totalPenguins = 0;
71         for(int i=0;i<N;i++) totalPenguins += n[i];
72
73         vector<vector<bool>> canJump(N, vector<bool>(N,false));
74         const double eps = 1e-6;
75         for(int i=0;i<N;i++){
76             for(int j=i+1;j<N;j++){

```

```

77         double dx = x[i]-x[j], dy = y[i]-y[j];
78         double dist = sqrt(dx*dx+dy*dy);
79         if(dist <= D + eps) canJump[i][j] = canJump[j][i] = true;
80     }
81
82     // in_node(i)=i, out_node(i)=N+i, S=2N
83     int S = 2*N;
84     int numNodes = 2*N+1;
85     Dinic dinic(numNodes);
86
87     for(int i=0;i<N;i++) dinic.addEdge(i, N+i, m[i]); // presupuesto de saltos
88     for(int i=0;i<N;i++) dinic.addEdge(S, i, n[i]); // pingüinos que parten de i
89
90     const int INF = 1000000;
91     for(int i=0;i<N;i++){
92         for(int j=i+1;j<N;j++){
93             if(canJump[i][j]){
94                 dinic.addEdge(N+i, j, INF);
95                 dinic.addEdge(N+j, i, INF);
96             }
97         }
98     }
99     vector<int> answer;
100     for(int t=0;t<N;t++){
101         dinic.resetFlow();
102         int f = dinic.maxflow(S, t); // sink = in_node(t)
103         if(f == totalPenguins) answer.push_back(t);
104     }
105     if(answer.empty()) cout << -1 << "\n";
106     else {
107         for(size_t i=0;i<answer.size();i++){
108             if(i) cout << ' ';
109             cout << answer[i];
110         }
111         cout << "\n";
112     }
113 }
114 return 0;
115 }

```

3.28 Max Flow (Binary Search + Dinic) - Gasoline

C++ Training_Camp_2026/Day 8/G - Gasoline.cpp

Resuelve "Gasoline" (Day 8, problema G - Contest 08 [Avanzados]): hay P gasolineras con demanda D_i y R refineras con stock E_j , con solo algunos pares (refineria, gasolinera) habilitados por contrato y un tiempo de viaje T conocido para esos pares; hay que hallar el menor tiempo t tal que, usando solo rutas con $T \leq t$, se pueda abastecer completamente a todas las gasolineras (repartiendo entre varias refineras si hace falta), o -1 si nunca alcanza. Hace búsqueda binaria sobre los tiempos distintos y, para cada candidato t , $feasible(t)$ arma una red fuente->refineria->gasolinera->sumidero (capacidades = stock, infinito solo si tiempo $\leq t$, y demanda) y corre Dinic (network::max_flow) comparando contra sumD.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct edge {
5     int v, cap, inv, flow;
6 };
7
8 struct network {
9     int n, s, t;
10    vector<int> lvl;
11    vector<vector<edge>> g;
12    network(int n) : n(n), lvl(n), g(n) {}
13    void add_edge(int u, int v, int c) {
14        g[u].push_back({v, c, (int)g[v].size(), 0});
15        g[v].push_back({u, 0, (int)g[u].size()-1, c});
16    }
17    bool bfs() {
18        fill(lvl.begin(), lvl.end(), -1);
19        queue<int> q;
20        lvl[s] = 0;
21        for(q.push(s); q.size(); q.pop()) {
22            int u = q.front();
23            for (auto &e : g[u]) {
24                if(e.cap > 0 && lvl[e.v] == -1) {

```

```

25                lvl[e.v] = lvl[u]+1;
26                q.push(e.v);
27            }
28        }
29    }
30    return lvl[t] != -1;
31    }
32    int dfs(int u, int nf) {
33        if(u == t) return nf;
34        int res = 0;
35        for (auto &e : g[u]) {
36            if(e.cap > 0 && lvl[e.v] == lvl[u]+1) {
37                int tf = dfs(e.v, min(nf, e.cap));
38                res += tf;
39                nf -= tf;
40                e.cap -= tf;
41                g[e.v][e.inv].cap += tf;
42                g[e.v][e.inv].flow -= tf;
43                e.flow += tf;
44                if(nf == 0) return res;
45            }
46        }
47        if(!res) lvl[u] = -1;
48        return res;
49    }
50    int max_flow(int so, int si, int res = 0) {
51        s = so; t = si;
52        while(bfs()) res += dfs(s, INT_MAX);
53        return res;
54    }
55 };
56
57 int P, R, C;
58 vector<int> D, E; // D[1..P], E[1..R]
59 vector<array<int,3>> edges; // I (station), J (refinery), T
60 long long sumD;
61
62 // nodos: 0 = source
63 // 1..R = refineras
64 // R+1..R+P = gasolineras
65 // R+P+1 = sumidero
66 bool feasible(int t) {
67     int n = R + P + 2;
68     int S = 0, T = R + P + 1;
69     network net(n);
70
71     // source -> refineria, capacidad = stock
72     for (int j = 1; j <= R; j++) {
73         net.add_edge(S, j, E[j]);
74     }
75     // gasolinera -> sumidero, capacidad = demanda
76     for (int i = 1; i <= P; i++) {
77         net.add_edge(R + i, T, D[i]);
78     }
79     // refineria -> gasolinera, solo si T_edge <= t
80     int INF = 1000000000;
81     for (auto &e : edges) {
82         int i = e[0], j = e[1], time = e[2];
83         if (time <= t) {
84             net.add_edge(j, R + i, INF);
85         }
86     }
87
88     long long f = net.max_flow(S, T);
89     return f >= sumD;
90 }
91
92 int main(){
93     ios::sync_with_stdio(false);
94     cin.tie(nullptr);
95
96     cin >> P >> R >> C;
97     D.assign(P+1, 0);

```

```

98 E.assign(R+1, 0);
99 for (int i = 1; i <= P; i++) cin >> D[i];
100 for (int j = 1; j <= R; j++) cin >> E[j];
101
102 edges.resize(C);
103 vector<int> times(C);
104 for (int k = 0; k < C; k++) {
105     int i, j, t;
106     cin >> i >> j >> t;
107     edges[k] = {i, j, t};
108     times[k] = t;
109 }
110
111 sumD = 0;
112 for (int i = 1; i <= P; i++) sumD += D[i];
113
114 // valores distintos de tiempo, ordenados
115 sort(times.begin(), times.end());
116 times.erase(unique(times.begin(), times.end()), times.end());
117
118 // primero chequeamos si es factible usando TODOS los arcos (t = maximo)
119 if (times.empty() || !feasible(times.back())) {
120     cout << -1 << "\n";
121     return 0;
122 }
123
124 // binary search: encontrar el menor t factible
125 int lo = 0, hi = (int)times.size() - 1;
126 while (lo < hi) {
127     int mid = (lo + hi) / 2;
128     if (feasible(times[mid])) {
129         hi = mid;
130     } else {
131         lo = mid + 1;
132     }
133 }
134
135 cout << times[lo] << "\n";
136 return 0;
137 }

```

3.29 MST con Prim (Grafo Denso) - Brute

C++ CCPL brute.cpp

Resuelve "Anti-brute Force Lock" (problema B, CCPL 2026): una caja fuerte con N llaves de 4 dígitos que hay que desbloquear todas. Los diales arrancan en 0000, cada rueda gira de 0 a 9 dando la vuelta (de 9 se pasa a 0), y hay un botón JUMP que lleva los diales GRATIS a cualquier llave YA desbloqueada. Se pide el mínimo total de giros. El costo entre dos combinaciones es la suma, rueda por rueda, de $\min(|x-y|, 10-|x-y|)$: girar por el lado corto del círculo. Esa es toda la función distancia. EL JUMP ES LO QUE CONVIERTE ESTO EN UN MST. Sin él, habría que recorrer las llaves en orden y el problema sería un TSP, o sea NP-difícil. Con el JUMP gratis, para abrir una llave nueva uno se para en la llave ya abierta que más le convenga y paga solo ese tramo. Entonces el costo total es la suma de las aristas de un árbol que conecta todas las llaves: el mínimo es el árbol de expansión mínima. EL DETALLE FINO, Y ES LO QUE HACE A ESTE PROBLEMA MÁS QUE UN MST DE LIBRO: el 0000 NO es una llave desbloqueada, así que el JUMP nunca puede devolverte ahí. El 0000 sirve una sola vez, para la primerísima llave, y después deja de existir. Por eso NO entra como nodo del grafo: se paga una vez $\min d(0000, \text{llave})$ y aparte se calcula el MST de las llaves SOLAS. Y se puede separar así porque el MST de las llaves no depende de cual se abra primero: el peso del árbol es el mismo empiece Prim donde empiece. Entonces el total es $\min_k d(0000, k) + \text{MST}(\text{llaves})$ y minimizar cada parte por separado da el óptimo global. SI EL 0000 SI FUERA SALTABLE la respuesta sería otra y MENOR: sería el MST del grafo con el 0000 adentro como un nodo más, que puede gastar VARIAS aristas desde el 0000. Con las llaves {1000, 9000}: $d(0000, 1000)=1$, $d(0000, 9000)=1$, $d(1000, 9000)=2$. Como el 0000 no es saltable la respuesta es $1+2=3$; si lo fuera, sería $1+1=2$. Vale la pena tener claro el ejemplo, porque meter el origen al grafo por costumbre es el error natural aquí. POR QUE PRIM Y NO KRUSKAL: el grafo es COMPLETO, las aristas son N^2 . Prim con arreglo simple es $O(N^2)$, que con $N \leq 500$ son 250000 pasos y ni se siente. Kruskal tendría que ordenar las N^2 aristas, $O(N^2 \log N)$, y además construirlas todas en memoria. La regla: grafo denso, Prim con arreglo; grafo disperso, Kruskal con DSU o Prim con priority_queue. Verificado: los 4 casos del sample dan 16, 20, 26 y 17, y además 100000 casos aleatorios (incluidos adversarios con llaves pegadas al 0000) contra una DP de máscara de bits que simula las reglas del enunciado al pie de la letra, sin una sola diferencia.

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 // Giros para pasar de la combinacion a a la b, por el lado corto de cada rueda.

```

```

6 int distancia(const string& a, const string& b)
7 {
8     int dis = 0;
9
10    for (int i = 0; i < 4; i++)
11    {
12        int x = a[i] - '0';
13        int y = b[i] - '0';
14
15        dis += min(abs(x - y), 10 - abs(x - y));
16    }
17
18    return dis;
19 }
20
21 int main()
22 {
23     ios::sync_with_stdio(false);
24     cin.tie(nullptr);
25
26     int T;
27     cin >> T;
28
29     while (T--)
30     {
31         int N;
32         cin >> N;
33
34         vector<string> locks(N);
35
36         for (int i = 0; i < N; i++)
37             cin >> locks[i];
38
39         // Costo de llegar inicialmente desde 0000
40         // a cada una de las llaves.
41         int respuestaInicial = INT_MAX;
42
43         for (int i = 0; i < N; i++)
44         {
45             respuestaInicial = min(
46                 respuestaInicial,
47                 distancia("0000", locks[i])
48             );
49         }
50
51         // Prim solamente entre las llaves.
52         // 0000 NO forma parte del grafo.
53         const int INF = INT_MAX;
54
55         vector<int> mejor(N, INF);
56         vector<bool> visitado(N, false);
57
58         // Podemos comenzar el MST desde cualquier llave,
59         // porque el costo inicial desde 0000 ya lo escogimos
60         // como el mínimo posible.
61         mejor[0] = 0;
62
63         int total = respuestaInicial;
64
65         for (int k = 0; k < N; k++)
66         {
67             int u = -1;
68
69             for (int i = 0; i < N; i++)
70             {
71                 if (!visitado[i] &&
72                     (u == -1 || mejor[i] < mejor[u]))
73                 {
74                     u = i;
75                 }
76             }
77
78             visitado[u] = true;

```

```

78         total += mejor[u];
79
80     for (int v = 0; v < N; v++)
81     {
82         if (!visitado[v])
83         {
84             int peso = distancia(locks[u], locks[v]);
85
86             mejor[v] = min(mejor[v], peso);
87         }
88     }
89 }
90
91 cout << total << '\n';
92 }
93
94 return 0;
95 }

```

3.30 State-Space BFS - Folding Machine

C++(Training_Camp_2026\Day 2\0 - Folding Machine.cpp)

Resuelve "Folding Machine" (Day 2, problema D - Maratona de Programacao da SBC 2013): una cinta de N celdas se puede doblar por cualquier punto entre celdas adyacentes (incluso en los extremos, lo que invierte la cinta), sumando los valores de las celdas que quedan superpuestas. Dado un tape de entrada y uno de salida, determina si existe una secuencia de dobleces que transforme el primero en el segundo. Como cada doblez real reduce estrictamente el largo de la cinta, genera todos los estados alcanzables agrupandolos por longitud (`states[len]`), probando cada pliegue posible de h celdas desde la izquierda o la derecha; `normalized()` reduce cada estado a su forma canonica (el minimo entre la cinta y su reverso) para no explorar duplicados. Al final solo compara si el output normalizado aparece entre los estados generados de longitud M .

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 // A fold at either end may reverse the tape, so both directions
7 // represent the same state.
8 vector<ll> normalized(vector<ll> tape) {
9     vector<ll> reversed = tape;
10    reverse(reversed.begin(), reversed.end());
11    return min(tape, reversed);
12 }
13
14 int main() {
15     ios::sync_with_stdio(false);
16     cin.tie(nullptr);
17
18     int n;
19     cin >> n;
20     vector<ll> input(n);
21     for (ll &x : input) cin >> x;
22
23     int m;
24     cin >> m;
25     vector<ll> output(m);
26     for (ll &x : output) cin >> x;
27
28     // The sum of all cells never changes after a fold.
29     if (accumulate(input.begin(), input.end(), 0LL) !=
30         accumulate(output.begin(), output.end(), 0LL)) {
31         cout << "N\n";
32         return 0;
33     }
34
35     vector<set<vector<ll>>> states(n + 1);
36     states[n].insert(normalized(input));
37
38     // Every real fold lowers the tape length. Thus a state only needs
39     // to be generated once, in the set indexed by its length.
40     for (int len = n; len > m; --len) {
41         for (const vector<ll> &tape : states[len]) {
42             for (int h = 1; h <= len / 2; ++h) {
43                 // Fold the h cells at the left end over the rest.

```

```

44         vector<ll> next;
45         next.reserve(len - h);
46         for (int j = 0; j < h; ++j)
47             next.push_back(tape[h + j] + tape[h - 1 - j]);
48         for (int j = 2 * h; j < len; ++j)
49             next.push_back(tape[j]);
50         if (len - h >= m)
51             states[len - h].insert(normalized(next));
52
53         // Fold the h cells at the right end over the rest.
54         next.clear();
55         const int leftSize = len - h;
56         for (int j = 0; j < leftSize - h; ++j)
57             next.push_back(tape[j]);
58         for (int j = 0; j < h; ++j)
59             next.push_back(tape[leftSize - h + j] + tape[len - 1 - j]);
60         if (len - h >= m)
61             states[len - h].insert(normalized(next));
62     }
63 }
64
65 cout << (states[m].count(normalized(output)) ? "S\n" : "N\n");
66 }

```

3.31 Strongly Connected Components (Tarjan) - Proving Equivalences

C++(Training_Camp_2026\Day 4\0 - Proving Equivalences.cpp)

Resuelve "Proving Equivalences" (Day 4, problema O - Contest 04 [Avanzados]): dadas n proposiciones y m implicaciones ya probadas entre ellas, hay que calcular el minimo numero de implicaciones adicionales necesarias para probar que todas son equivalentes entre si. Calcula las componentes fuertemente conexas con el algoritmo de Tarjan (`tarjanSCC`), condensa el grafo en un DAG de SCCs y cuenta cuantas componentes no tienen aristas entrantes (`hasIn`) ni salientes (`hasOut`) en ese DAG; la respuesta es el maximo entre la cantidad de fuentes y sumideros (0 si ya es una unica SCC, que es la minima cantidad de aristas para volverlo fuerte).

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 static char buf[1 << 20];
5 static int bufLen_, bufPos_;
6 inline int gc() {
7     if (bufPos_ == bufLen_) { bufLen_ = fread(buf_, 1, sizeof(buf_), stdin); bufPos_ = 0; if (!bufLen_) return -1; }
8     return buf_[bufPos_++];
9 }
10 inline int readInt() {
11     int c = gc();
12     while (c != -1 && (c < '0' || c > '9') && c != '-') c = gc();
13     bool neg = false; if (c == '-') { neg = true; c = gc(); }
14     int x = 0;
15     while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = gc(); }
16     return neg ? -x : x;
17 }
18
19 int n, m;
20 vector<vector<int>>> adj;
21 vector<int> disc_, low_, comp_;
22 vector<char> onStack_;
23 vector<int> tarjanStack;
24 int timer_, sccCount;
25
26 // Iterative Tarjan's SCC algorithm.
27 void tarjanSCC() {
28     disc_.assign(n + 1, -1);
29     low_.assign(n + 1, -1);
30     comp_.assign(n + 1, -1);
31     onStack_.assign(n + 1, 0);
32     tarjanStack.clear();
33     timer_ = 0;
34     sccCount = 0;
35
36     for (int s = 1; s <= n; ++s) {
37         if (disc_[s] != -1) continue;
38         vector<array<int, 2>> stk; // {vertex, nextIdx}

```

```

39     stk.push_back({s, 0});
40     disc_[s] = low_[s] = timer++;
41     tarjanStack.push_back(s);
42     onStack_[s] = 1;
43
44     while (!stk.empty()) {
45         auto &fr = stk.back();
46         int u = fr[0];
47         if (fr[1] < (int)adj[u].size()) {
48             int v = adj[u][fr[1]];
49             fr[1]++;
50             if (disc_[v] == -1) {
51                 disc_[v] = low_[v] = timer++;
52                 tarjanStack.push_back(v);
53                 onStack_[v] = 1;
54                 stk.push_back({v, 0});
55             } else if (onStack_[v]) {
56                 low_[u] = min(low_[u], disc_[v]);
57             }
58         } else {
59             stk.pop_back();
60             if (!stk.empty()) {
61                 int p = stk.back()[0];
62                 low_[p] = min(low_[p], low_[u]);
63             }
64             if (low_[u] == disc_[u]) {
65                 // pop the SCC
66                 while (true) {
67                     int w = tarjanStack.back();
68                     tarjanStack.pop_back();
69                     onStack_[w] = 0;
70                     comp_[w] = sccCount;
71                     if (w == u) break;
72                 }
73                 sccCount++;
74             }
75         }
76     }
77 }
78
79 int main() {
80     int T = readInt();
81     string out;
82     while (T--) {
83         n = readInt(); m = readInt();
84         adj.assign(n + 1, {});
85         vector<pair<int,int>> rawEdges(m);
86         for (int i = 0; i < m; i++) {
87             int a = readInt(), b = readInt();
88             rawEdges[i] = {a, b};
89             adj[a].push_back(b);
90         }
91     }
92     tarjanSCC();
93
94     vector<char> hasIn(sccCount, 0), hasOut(sccCount, 0);
95     for (auto &e : rawEdges) {
96         int ca = comp_[e.first], cb = comp_[e.second];
97         if (ca != cb) {
98             hasOut[ca] = 1;
99             hasIn[cb] = 1;
100         }
101     }
102
103     int ans;
104     if (sccCount <= 1) {
105         ans = 0;
106     } else {
107         int sources = 0, sinks = 0;
108         for (int i = 0; i < sccCount; i++) {
109             if (!hasIn[i]) sources++;
110             if (!hasOut[i]) sinks++;
111         }

```

```

102     }
103     ans = max(sources, sinks);
104 }
105 out += to_string(ans);
106 out += "\n";
107 }
108 fwrite(out.data(), 1, out.size(), stdout);
109 return 0;
110 }

```

3.32 Tree Degree Analysis - Colmillos

C++\Codeforces\colmillos.cpp

Calcula cuántos pares de nodos pueden removerse de un árbol para desconectarlo. Es n menos la suma de los dos mayores grados de los nodos.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int n;
9     cin >> n;
10
11     vector<int> deg(n + 1, 0);
12
13     for (int i = 0; i < n - 1; i++) {
14         int a, b;
15         cin >> a >> b;
16
17         deg[a]++;
18         deg[b]++;
19     }
20
21     int mx1 = 0, mx2 = 0;
22
23     for (int i = 1; i <= n; i++) {
24         if (deg[i] >= mx1) {
25             mx2 = mx1;
26             mx1 = deg[i];
27         } else if (deg[i] > mx2) {
28             mx2 = deg[i];
29         }
30     }
31
32     cout << n - mx1 - mx2 << '\n';
33
34     return 0;
35 }

```

3.33 Tree Reconstruction (BFS por capas) - Expansion Of The Road Network

C++\Training_Camp_2026\Day 6\E - Expansion of the road network.py

Resuelve "Expansion of the road network" (Day 6, problema E - Maratona SBC de Programacao 2025): el mapa actual de N ciudades y M caminos pudo haber surgido de un árbol original (un camino único entre cada par de ciudades) al que se le agregó una carretera directa entre cada par de ciudades cuyo camino original pasaba por exactamente una ciudad intermedia (distancia 2 en el árbol). Hay que decidir si el mapa actual es consistente con ese proceso y, de serlo, reconstruir un árbol original válido (o imprimir "*" si es imposible). El código hace BFS desde el nodo 1 para ubicar una hoja candidata del árbol original (la de menor grado en la última capa), luego hace BFS desde esa hoja para obtener las capas "layer[]", identifica los posibles vértices "soporte" intersectando pares de padres de los nodos de profundidad ≥ 2 , construye el árbol candidato asignando "parent[]" capa por capa según la paridad de profundidad, y por último "verify" comprueba que los caminos de distancia 1 y 2 de ese árbol coincidan exactamente con las M aristas dadas antes de imprimirlo.

```

1 import sys
2 from collections import deque
3
4 def solve(data):
5     n = int(data[0]); m = int(data[1])
6     B = n + 1
7     adj = [[] for _ in range(B)]
8     eset = set()
9     ptr = 2

```

```

0  for _ in range(m):
1  a = int(data[ptr]); b = int(data[ptr + 1]); ptr += 2
2  adj[a].append(b); adj[b].append(a)
3  eset.add(a * B + b if a < b else b * B + a)
4
5  # Grafo completo <=> el arbol original era una estrella
6  if m == n * (n - 1) // 2:
7  return '\n'.join('1 %d' % i for i in range(2, n + 1))
8
9  def bfs(src):
10 dist = [-1] * B
11 dist[src] = 0
12 q = deque([src])
13 while q:
14 u = q.popleft()
15 du = dist[u] + 1
16 for v in adj[u]:
17 if dist[v] < 0:
18 dist[v] = du
19 q.append(v)
20 return dist
21
22 # 1) Hoja garantizada del arbol original:
23 # el de grado minimo dentro de la ultima capa BFS
24 d1 = bfs(1)
25 maxd = max(d1[1:])
26 leaf = -1; best = float('inf')
27 for v in range(1, n + 1):
28 if d1[v] == maxd and len(adj[v]) < best:
29 best = len(adj[v]); leaf = v
30
31 # 2) Capas BFS desde la hoja: la capa k contiene las profundidades
32 # 2k-2 y 2k-1 del arbol enraizado en el soporte de la hoja
33 layer = bfs(leaf)
34 inK = bytearray(B)
35 for v in adj[leaf]:
36 inK[v] = 1
37
38 # 3) Candidatos a soporte: todo vertice externo con exactamente 2 vecinos
39 # dentro de N(hoja) es "nieto" del soporte y esos 2 vecinos son
40 # {soporte, su padre}. Interseccion de esos pares => <= 2 candidatos.
41 cand = None
42 for z in range(1, n + 1):
43 if layer[z] >= 2:
44 c = 0; p1 = 0; p2 = 0
45 for y in adj[z]:
46 if inK[y]:
47 c += 1
48 if c == 1: p1 = y
49 elif c == 2: p2 = y
50 else: break
51 if c == 2:
52 if cand is None:
53 cand = {p1, p2}
54 else:
55 cand.intersection_update((p1, p2))
56 if not cand:
57 return ''
58
59 if not cand:
60 return ''
61
62 maxl = max(layer[1:])
63 layers = [[] for _ in range(maxl + 1)]
64 for v in range(1, n + 1):
65 layers[layer[v]].append(v)
66
67 def build(w):
68 parent = [0] * B
69 role = bytearray(B) # 1 = prof. par ("arriba"), 2 = prof. impar ("abajo")
70 role[w] = 1
71 parent[leaf] = w
72 for v in layers[1]:
73 if v != w:

```

```

83 role[v] = 2
84 parent[v] = w
85 for k in range(2, maxl + 1):
86 lows = []
87 km1 = k - 1
88 for x in layers[k]:
89 c = 0; a = 0; b = 0
90 for y in adj[x]:
91 if layer[y] == km1:
92 c += 1
93 if c == 1: a = y
94 elif c == 2: b = y
95 else: return None
96 if c == 2:
97 ra = role[a]
98 if ra == role[b]: return None
99 if ra == 1: a, b = b, a
100 if parent[a] != b: return None # el "arriba" debe ser el abuelo
101 role[x] = 1
102 parent[x] = a
103 elif c == 1:
104 if role[a] != 2: return None # unico vecino previo = abuelo
105 role[x] = 2
106 lows.append((x, a))
107 else:
108 return None
109 for x, g in lows:
110 p = 0
111 for y in adj[x]:
112 if layer[y] == k and role[y] == 1 and parent[y] == g:
113 p = y; break
114 if not p: return None
115 parent[x] = p
116 return parent
117
118 def verify(parent, root):
119 degT = [0] * B
120 for v in range(1, n + 1):
121 if v != root:
122 degT[v] += 1
123 degT[parent[v]] += 1
124 extra = m - (n - 1)
125 s = 0
126 for v in range(1, n + 1):
127 d = degT[v]
128 s += d * (d - 1) // 2
129 if s > extra: return False
130 if s != extra: return False
131 ch = [[] for _ in range(B)]
132 for v in range(1, n + 1):
133 if v != root:
134 p = parent[v]
135 ch[p].append(v)
136 key = v * B + p if v < p else p * B + v
137 if key not in eset: return False
138 for v in range(1, n + 1):
139 nb = ch[v]
140 if v != root:
141 nb = nb + [parent[v]]
142 ln = len(nb)
143 for i in range(ln - 1):
144 x = nb[i]
145 for j in range(i + 1, ln):
146 y = nb[j]
147 key = x * B + y if x < y else y * B + x
148 if key not in eset: return False
149 return True
150
151 for w in cand:
152 parent = build(w)
153 if parent is not None and verify(parent, w):
154 out = []
155 ap = out.append

```

```

56         for v in range(1, n + 1):
57             if v != w:
58                 ap['%d %d' % (v, parent[v])]
59             return '\n'.join(out)
60     return '*'
61
62 def main():
63     data = sys.stdin.buffer.read().split()
64     sys.stdout.write(solve(data) + '\n')
65
66 if __name__ == '__main__':
67     main()

```

4 - Geometry

4.1 2D Difference Array (Coordinate Compression) - Office Space

C++ Training_Camp_2026 Day 11 - CCPLK * Office Space.cpp

Resuelve "Office Space" (Day 11, problema K - CCPL (Kattis)): en una oficina rectangular de $w \times h$ pies, hasta 20 empleados piden un cubículo rectangular (esquinas suroeste y noreste); pide reportar el área total, el área sin reclamar, el área en conflicto (solicitada por 2 o mas empleados) y, para cada empleado, el área que tiene garantizada (lo que pidió menos lo que se solapa con otros). Comprime las coordenadas x/y de las esquinas de todos los rectángulos en 'xs'/ys', y usa un arreglo de diferencias 2D ('grid') para contar cuantas solicitudes cubren cada celda comprimida, junto con un arreglo paralelo 'sumIdx' que acumula (índice+1) del empleado para identificar sin ambigüedad al único dueño cuando una celda queda cubierta exactamente una vez; una suma prefija 2D final obtiene el conteo y dueño real de cada celda.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4
5 static char buf_[1 << 22];
6 static int bufLen_, bufPos_;
7 inline int gc() {
8     if (bufPos_ == bufLen_) { bufLen_ = fread(buf_, 1, sizeof(buf_), stdin); bufPos_ = 0; if (!bufLen_) return -1; }
9     return buf_[bufPos_++];
10 }
11 inline bool readInt(int &out) {
12     int c = gc();
13     while (c != -1 && (c < '0' || c > '9') && c != '-') c = gc();
14     if (c == -1) return false;
15     bool neg = false; if (c == '-') { neg = true; c = gc(); }
16     int x = 0;
17     while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = gc(); }
18     out = neg ? -x : x;
19     return true;
20 }
21 inline bool isSpaceChar(int c) { return c == ' ' || c == '\n' || c == '\r' || c == '\t'; }
22 inline bool readWord(string &s) {
23     s.clear();
24     int c = gc();
25     while (c != -1 && isSpaceChar(c)) c = gc();
26     if (c == -1) return false;
27     while (c != -1 && !isSpaceChar(c)) { s.push_back((char)c); c = gc(); }
28     return true;
29 }
30
31 int main() {
32     int W, H;
33     string out;
34     while (readInt(W)) {
35         readInt(H);
36         int n;
37         readInt(n);
38
39         vector<string> names(n);
40         vector<int> X1(n), Y1(n), X2(n), Y2(n);
41         vector<int> xs, ys;
42         xs.reserve(2 * n + 2);
43         ys.reserve(2 * n + 2);
44         xs.push_back(0); xs.push_back(W);
45         ys.push_back(0); ys.push_back(H);
46

```

```

47         for (int i = 0; i < n; i++) {
48             readWord(names[i]);
49             readInt(X1[i]); readInt(Y1[i]); readInt(X2[i]); readInt(Y2[i]);
50             xs.push_back(X1[i]); xs.push_back(X2[i]);
51             ys.push_back(Y1[i]); ys.push_back(Y2[i]);
52         }
53
54         sort(xs.begin(), xs.end());
55         xs.erase(unique(xs.begin(), xs.end()), xs.end());
56         sort(ys.begin(), ys.end());
57         ys.erase(unique(ys.begin(), ys.end()), ys.end());
58
59         int M = (int)xs.size() - 1; // number of cell columns
60         int K = (int)ys.size() - 1; // number of cell rows
61         int W2 = K + 1; // row width for flat indexing over the (M+1) x (K+1) corner grid
62
63         auto xIdx = [&](int v) { return (int)(lower_bound(xs.begin(), xs.end(), v) - xs.begin()); };
64         auto yIdx = [&](int v) { return (int)(lower_bound(ys.begin(), ys.end(), v) - ys.begin()); };
65         auto at = [&](int i, int j) { return i * W2 + j; };
66
67         // Flat 2D difference array (row-major) over the (M+1) x (K+1) grid of "corner"
68         // points, computed in place into the count grid (avoids a second full grid, and
69         // flat 1D storage avoids the per-row bookkeeping overhead of nested vectors).
70         vector<int> grid((size_t)(M + 1) * (K + 1), 0);
71         // Parallel difference array summing (employee_index + 1) instead of 1. When a
72         // cell's final count is exactly 1, this sum unambiguously equals that single
73         // covering employee's (index + 1), letting us identify the owner without a
74         // separate large prefix-sum array.
75         vector<ll> sumIdx((size_t)(M + 1) * (K + 1), 0);
76
77         for (int i = 0; i < n; i++) {
78             int a = xIdx(X1[i]), b = xIdx(X2[i]);
79             int c = yIdx(Y1[i]), d = yIdx(Y2[i]);
80             grid[at(a, c)] += 1;
81             grid[at(b, c)] -= 1;
82             grid[at(a, d)] -= 1;
83             grid[at(b, d)] += 1;
84
85             ll idPlus1 = (ll)i + 1;
86             sumIdx[at(a, c)] += idPlus1;
87             sumIdx[at(b, c)] -= idPlus1;
88             sumIdx[at(a, d)] -= idPlus1;
89             sumIdx[at(b, d)] += idPlus1;
90         }
91
92         // 2D prefix sum in place -> grid[.] becomes the count of rectangles covering each
93         // cell, and sumIdx[.] becomes the sum of (index+1) of covering rectangles.
94         for (int i = 0; i <= M; i++) {
95             int rowBase = i * W2;
96             int prevRowBase = rowBase - W2;
97             for (int j = 0; j <= K; j++) {
98                 int idx = rowBase + j;
99                 if (i > 0) { grid[idx] += grid[prevRowBase + j]; sumIdx[idx] += sumIdx[prevRowBase
100 + j]; }
101                 if (j > 0) { grid[idx] += grid[idx - 1]; sumIdx[idx] += sumIdx[idx - 1]; }
102                 if (i > 0 && j > 0) { grid[idx] -= grid[prevRowBase + j - 1]; sumIdx[idx] -= sumIdx
103 [prevRowBase + j - 1]; }
104             }
105
106             ll total = (ll)W * H;
107             ll unalloc = 0, contested = 0;
108             vector<ll> guaranteed(n, 0);
109
110             for (int i = 0; i < M; i++) {
111                 ll colWidth = (ll)(xs[i + 1] - xs[i]);
112                 int rowBase = i * W2;
113                 for (int j = 0; j < K; j++) {
114                     ll cellArea = colWidth * (ll)(ys[j + 1] - ys[j]);
115                     int c = grid[rowBase + j];

```

```

16         if (c == 0) {
17             unalloc += cellArea;
18         } else if (c >= 2) {
19             contested += cellArea;
20         } else { // c == 1
21             int owner = (int)(sumIdx[rowBase + j] - 1);
22             guaranteed[owner] += cellArea;
23         }
24     }
25
26     out += "Total " + to_string(total) + "\n";
27     out += "Unallocated " + to_string(unalloc) + "\n";
28     out += "Contested " + to_string(contested) + "\n";
29
30     for (int i = 0; i < n; i++) {
31         out += names[i] + " " + to_string(guaranteed[i]) + "\n";
32     }
33     out += "\n";
34 }
35
36 fwrite(out.data(), 1, out.size(), stdout);
37 return 0;
38 }

```

4.2 Area Circulo-Poligono

C++\Notebook\Area Circulo-Poligon.cpp

Area de la interseccion entre un circulo y un poligono, usando el truco de sumar areas con signo arista por arista: se traslada toda para dejar el circulo en el origen y se recorre el poligono sumando triangleCircleArea de cada arista, de modo que las contribuciones de fuera se cancelan solas y al final basta el valor absoluto. Funciona con el poligono en cualquier orientacion y sin importar si el circulo queda dentro, fuera o a medias. Para cada arista p-q se resuelve $|p + t*d|^2 = r^2$, una cuadratica en t, para hallar donde corta la circunferencia; esos cortes parten el segmento en tramos. Cada tramo se clasifica mirando su punto medio: si cae dentro del circulo aporta el area del triangulo (cross/2), y si cae fuera aporta la del sector circular ($r^2 \cdot \text{angulo}/2$, con atan2 para quedarse con el angulo con signo correcto). Aqui se usa con un rectangulo [0,w]x[0,h] para calcular la probabilidad de que un punto uniforme caiga dentro del circulo, y con eso el valor esperado de la suma de los v.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const double EPS = 1e-12;
5
6 struct Pt { double x, y; };
7
8 double cross(Pt a, Pt b) { return a.x * b.y - a.y * b.x; }
9 double dot(Pt a, Pt b) { return a.x * b.x + a.y * b.y; }
10 Pt sub(Pt a, Pt b) { return {a.x - b.x, a.y - b.y}; }
11 Pt add(Pt a, Pt b) { return {a.x + b.x, a.y + b.y}; }
12 Pt mul(Pt a, double t) { return {a.x * t, a.y * t}; }
13 double norm2(Pt a) { return dot(a, a); }
14
15 // Area con signo de la interseccion entre el circulo centrado en (0,0) de radio r
16 // y el triangulo (0,0)-p-q.
17 double triangleCircleArea(Pt p, Pt q, double r) {
18     Pt d = sub(q, p);
19
20     //  $|p + t*d|^2 = r^2 \rightarrow A t^2 + B t + C = 0$ 
21     double A = norm2(d);
22     double B = 2.0 * dot(p, d);
23     double C = norm2(p) - r * r;
24
25     vector<double> cuts = {0.0, 1.0};
26
27     if (A > EPS) {
28         double disc = B * B - 4.0 * A * C;
29         if (disc > EPS) {
30             double s = sqrt(disc);
31             double t1 = (-B - s) / (2.0 * A);
32             double t2 = (-B + s) / (2.0 * A);
33             if (t1 > EPS && t1 < 1.0 - EPS) cuts.push_back(t1);
34             if (t2 > EPS && t2 < 1.0 - EPS) cuts.push_back(t2);
35         }
36     }

```

```

37     sort(cuts.begin(), cuts.end());
38
39     double ans = 0.0;
40     for (size_t i = 0; i + 1 < cuts.size(); i++) {
41         double l = cuts[i], rr = cuts[i + 1];
42         double mid = (l + rr) / 2.0;
43
44         Pt a = add(p, mul(d, l));
45         Pt b = add(p, mul(d, rr));
46         Pt m = add(p, mul(d, mid));
47
48         if (norm2(m) <= r * r) {
49             ans += cross(a, b) / 2.0; // tramo dentro: triangulo
50         } else {
51             ans += r * r * atan2(cross(a, b), dot(a, b)) / 2.0; // tramo fuera: sector
52         }
53     }
54     return ans;
55 }
56
57 double circleRectangleArea(double cx, double cy, double r, double w, double h) {
58     // Trasladar para que el circulo quede en el origen.
59     Pt rect[4] = {{-cx, -cy}, {w - cx, -cy}, {w - cx, h - cy}, {-cx, h - cy}};
60
61     double area = 0.0;
62     for (int i = 0; i < 4; i++) area += triangleCircleArea(rect[i], rect[(i + 1) % 4], r);
63     return fabs(area);
64 }
65
66 int main() {
67     ios::sync_with_stdio(false);
68     cin.tie(nullptr);
69
70     int n;
71     double r, w, h;
72     cin >> n >> r >> w >> h;
73
74     double ans = 0.0;
75     for (int i = 0; i < n; i++) {
76         double x, y, v;
77         cin >> x >> y >> v;
78         ans += v * circleRectangleArea(x, y, r, w, h) / (w * h);
79     }
80
81     cout << fixed << setprecision(15) << ans << "\n";
82     return 0;
83 }

```

4.3 Convex Hull y Barrido Angular - Guard Deployment

C++\ICPC\2025\G - Guard Deployment.cpp

Resuelve "Guard Deployment" (problema G, ICPC 2025): hay edificios rectangulares con altura, y hay que dar el perimetro de la cerca convexa mas corta que los encierra y la altura minima comun de las torres para que todas se vean entre si. Son dos problemas sueltos pegados: 1) EL PERIMETRO es el del convex hull de las cuatro esquinas de todos los edificios. La cerca mas corta que encierra un conjunto de puntos es siempre su envolvente convexa. 2) LA ALTURA depende de que edificios se atraviesan. Una linea de vista entre dos torres pasa por encima de un edificio si el segmento que las une (visto desde arriba) toca ese rectangulo; y si pasa, hay que ir al menos un metro mas alto que el. La respuesta es el maximo de $h+1$ sobre los edificios atravesados, y nunca menos de 3. LO INTERESANTE ES COMO DECIDE SI UN EDIFICIO SE ATRAVIESA, sin probar los $O(H^2)$ pares de torres: desde un vertice del hull, todas las direcciones que dan al rectangulo forman un INTERVALO ANGULAR, delimitado por sus cuatro esquinas. Si otro vertice del hull cae dentro de ese intervalo, el segmento entre ambos pasa por el edificio. Teniendo los angulos hacia los demas vertices ordenados, cada consulta es una busqueda binaria. El intervalo se obtiene al revés de lo que uno esperaria: se ordenan los angulos a las cuatro esquinas, se busca el HUECO mas grande entre angulos consecutivos (en circulo), y el intervalo bueno es el complemento de ese hueco. Es la forma de manejar el caso en que el intervalo cruza el 0 sin escribir casos especiales. Verificado contra una referencia que prueba cada par de torres contra cada edificio con aritmetica entera exacta: 1600 casos con rectangulos alineados y rotados 45 grados, perimetro y altura correctos en todos, ademas del sample. Medida: 10 casos de $N = 1000$ edificios en 200 ms.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ld = long double;

```

```

5
6 const ld PI = acos(-1.0L);
7 const ld EPS = 1e-12L;
8
9 struct Point {
10     ld x, y;
11
12     bool operator<(const Point& p) const {
13         if (fabs(x - p.x) > EPS)
14             return x < p.x;
15         return y < p.y;
16     }
17
18     bool operator==(const Point& p) const {
19         return fabs(x - p.x) < EPS &&
20             fabs(y - p.y) < EPS;
21     }
22 };
23
24 struct Building {
25     Point p[4];
26     int h;
27 };
28
29 ld cross(Point a, Point b, Point c) {
30     return (b.x - a.x) * (c.y - a.y)
31         - (b.y - a.y) * (c.x - a.x);
32 }
33
34 // -----
35 // CONVEX HULL
36 // -----
37
38 vector<Point> convexHull(vector<Point> p) {
39     sort(p.begin(), p.end());
40
41     p.erase(unique(p.begin(), p.end()), p.end());
42
43     int n = p.size();
44
45     if (n <= 1)
46         return p;
47
48     vector<Point> h(2 * n);
49     int k = 0;
50
51     // Lower
52     for (int i = 0; i < n; i++) {
53         while (k >= 2 &&
54             cross(h[k - 2], h[k - 1], p[i]) <= EPS)
55             k--;
56         h[k++] = p[i];
57     }
58
59     // Upper
60     for (int i = n - 2, t = k + 1; i >= 0; i--) {
61         while (k >= t &&
62             cross(h[k - 2], h[k - 1], p[i]) <= EPS)
63             k--;
64         h[k++] = p[i];
65     }
66
67     h.resize(k - 1);
68
69     return h;
70 }
71

```

```

72
73 // -----
74 // DISTANCIA
75 // -----
76
77 ld dist(Point a, Point b) {
78     ld dx = a.x - b.x;
79     ld dy = a.y - b.y;
80
81     return sqrt(dx * dx + dy * dy);
82 }
83
84 // -----
85 // PUNTO SOBRE RECTANGULO
86 // Como los edificios son convexos, basta revisar sus 4 lados.
87 // -----
88
89 bool pointOnSegment(Point a, Point b, Point p) {
90     if (fabs(cross(a, b, p)) > EPS)
91         return false;
92
93     return p.x >= min(a.x, b.x) - EPS &&
94         p.x <= max(a.x, b.x) + EPS &&
95         p.y >= min(a.y, b.y) - EPS &&
96         p.y <= max(a.y, b.y) + EPS;
97 }
98
99 bool pointInBuilding(const Building& b, Point p) {
100     for (int i = 0; i < 4; i++) {
101         Point a = b.p[i];
102         Point c = b.p[(i + 1) % 4];
103
104         if (pointOnSegment(a, c, p))
105             return true;
106     }
107
108     bool pos = false;
109     bool neg = false;
110
111     for (int i = 0; i < 4; i++) {
112         ld cr = cross(
113             b.p[i],
114             b.p[(i + 1) % 4],
115             p
116         );
117
118         if (cr > EPS) pos = true;
119         if (cr < -EPS) neg = true;
120     }
121
122     return !(pos && neg);
123 }
124
125 // -----
126 // ANGULO
127 // -----
128
129 ld angle(Point a, Point b) {
130     ld x = b.x - a.x;
131     ld y = b.y - a.y;
132
133     ld ang = atan2(y, x);
134
135     if (ang < 0)
136         ang += 2 * PI;
137 }
138

```

```

61     return ang;
62 }
63
64
65 // -----
66 // EL EDIFICIO ES ATRAVESADO POR ALGUN SEGMENTO
67 // ENTRE DOS VERTICES DEL HULL?
68 //
69 // Para un vertice v del hull:
70 //
71 // todas las direcciones que desde v alcanzan el rectangulo
72 // forman un intervalo angular.
73 //
74 // Los cuatro corners del rectangulo determinan ese intervalo.
75 // Si existe otro vertice del hull dentro de ese intervalo,
76 // el segmento entre ambos atraviesa/toca el rectangulo.
77 // -----
78
79 bool buildingBlocks(
80     const vector<Point>& hull,
81     const Building& b,
82     const vector<vector<ld>>& hullAngles
83 ) {
84
85     int H = hull.size();
86
87     for (int v = 0; v < H; v++) {
88
89         // Si el vertice del hull pertenece al edificio,
90         // claramente este edificio participa.
91         if (pointInBuilding(b, hull[v]))
92             return true;
93
94         // Angulos hacia las 4 esquinas del edificio
95         vector<ld> a(4);
96
97         for (int k = 0; k < 4; k++)
98             a[k] = angle(hull[v], b.p[k]);
99
100         sort(a.begin(), a.end());
101
102         // Buscamos el mayor hueco circular.
103         // El intervalo complementario es el menor intervalo
104         // que contiene las 4 esquinas.
105         int bestGap = 0;
106         ld maxGap = -1;
107
108         for (int k = 0; k < 4; k++) {
109
110             ld next = (k == 3 ? a[0] + 2 * PI : a[k + 1]);
111
112             ld gap = next - a[k];
113
114             if (gap > maxGap) {
115                 maxGap = gap;
116                 bestGap = k;
117             }
118         }
119
120         ld L = a[(bestGap + 1) % 4];
121         ld R = a[bestGap];
122
123         if (R < L)
124             R += 2 * PI;
125
126         const vector<ld>& angles = hullAngles[v];
127
128         // Tenemos los angulos de los demas vertices.
129         // Probamos tambien una copia desplazada 2PI para
130         // resolver el caso en que el intervalo cruza 0.
131         auto check = [&](ld x) {
132             return x + EPS >= L && x <= R + EPS;
133         };
134     }

```

```

224
225     auto it = lower_bound(
226         angles.begin(),
227         angles.end(),
228         L - EPS
229     );
230
231     if (it != angles.end() && check(*it))
232         return true;
233
234     // Si L > 2PI, usamos los angulos originales + 2PI.
235     if (L > 2 * PI - EPS) {
236
237         ld L2 = L - 2 * PI;
238         ld R2 = R - 2 * PI;
239
240         auto it2 = lower_bound(
241             angles.begin(),
242             angles.end(),
243             L2 - EPS
244         );
245
246         if (it2 != angles.end() &&
247             *it2 <= R2 + EPS)
248             return true;
249     }
250 }
251
252 return false;
253 }
254
255 // -----
256 // MAIN
257 // -----
258
259 int main() {
260
261     ios::sync_with_stdio(false);
262     cin.tie(nullptr);
263
264     int T;
265     cin >> T;
266
267     while (T--) {
268
269         int N;
270         cin >> N;
271
272         vector<Building> buildings(N);
273
274         vector<Point> allPoints;
275
276         for (int i = 0; i < N; i++) {
277
278             for (int j = 0; j < 4; j++) {
279
280                 cin >> buildings[i].p[j].x
281                     >> buildings[i].p[j].y;
282
283                 allPoints.push_back(buildings[i].p[j]);
284             }
285
286             cin >> buildings[i].h;
287         }
288
289         // -----
290         // 1. Convex Hull
291         // -----
292
293         vector<Point> hull = convexHull(allPoints);
294
295         int H = hull.size();

```

```

297
298 // -----
299 // 2. Perimetro
300 // -----
301
302 ld perimeter = 0;
303
304 for (int i = 0; i < H; i++)
305     perimeter += dist(
306         hull[i],
307         hull[(i + 1) % H]
308     );
309
310 // -----
311 // 3. Angulos de los vertices del hull
312 //
313 // Para cada vertice guardamos los angulos hacia
314 // todos los demas vertices.
315 // -----
316
317 vector<vector<ld>> hullAngles(H);
318
319 for (int i = 0; i < H; i++) {
320
321     for (int j = 0; j < H; j++) {
322
323         if (i == j)
324             continue;
325
326         hullAngles[i].push_back(
327             angle(hull[i], hull[j])
328         );
329     }
330
331     sort(
332         hullAngles[i].begin(),
333         hullAngles[i].end()
334     );
335 }
336
337 // -----
338 // 4. Altura minima
339 // -----
340
341 int minHeight = 3;
342
343 for (int i = 0; i < N; i++) {
344
345     if (buildingBlocks(
346         hull,
347         buildings[i],
348         hullAngles
349     )) {
350
351         minHeight = max(
352             minHeight,
353             buildings[i].h + 1
354         );
355     }
356 }
357
358 cout << fixed << setprecision(6)
359      << (double)perimeter << ' '
360      << minHeight << '\n';
361 }
362
363 return 0;
364 }

```

4.4 Coordenadas Polares

C++ Notebook\Coordenadas Polares.cpp

Pasar de (x, y) a (r, θ) y al contrario. Las formulas son de colegio, pero en competitiva lo que tumba la solucion son los detalles de implementacion, y por eso vale la ficha. 1) EL ANGULO SE SACA CON $\text{atan2}(y, x)$, NUNCA CON $\text{atan}(y/x)$. $\text{atan}(y/x)$ pierde el cuadrante (manda $(-1, -1)$ al mismo angulo que $(1, 1)$) y explota cuando $x = 0$. atan2 mira los signos de los dos argumentos por separado y acierta en los cuatro cuadrantes. OJO CON EL ORDEN DE LOS ARGUMENTOS: es $\text{atan2}(y, x)$, la y va PRIMERO. Invertirlos da el angulo respecto al eje y, y el error pasa desapercibido porque para 45 grados da lo mismo. 2) EL RADIO CON $\text{hypot}(x, y)$, no con $\sqrt{x^2 + y^2}$. hypot escala internamente y no pierde el paso intermedio. Medido en esta maquina: con $x = y = 1e-200$, $\sqrt{x^2 + y^2}$ da 0 porque el cuadrado se va a cero por UNDERFLOW, mientras hypot da $1.41e-200$. Por arriba, con $1e200$, aqui las dos dieron $1.41e200$ (el x87 calcula con 80 bits y salva el caso), asi que el problema real de $\sqrt{}$ es el underflow, no el overflow. Igual hypot sale gratis. 3) atan2 DEVUELVE EL RANGO $(-\pi, \pi]$, no $[0, 2\pi)$. Los angulos de abajo del eje x salen NEGATIVOS. Si el problema necesita $[0, 2\pi)$ hay que sumar 2π a los negativos, y si se olvida, el barrido angular arranca en el eje x NEGATIVO en vez del positivo. LO QUE DE VERDAD IMPORTA: PARA ORDENAR POR ANGULO, NO USES atan2 . El uso numero uno de las polares en competitiva es el barrido angular. Con atan2 funciona, pero es la version fragil, y conviene tener claro en que falla exactamente: - El orden ANGULAR sale bien con los dos metodos. Medido con 200000 puntos enteros aleatorios: las dos ordenaciones son monotonas en angulo y NO hay ni un solo par cuyo angulo verdadero quede en el orden equivocado. - Donde atan2 se rompe es en los COLINEALES. De las 15156 posiciones en que las dos listas no coinciden, las 15156 son entre puntos del MISMO angulo exacto: atan2 les da doubles que difieren en el ultimo bit, el desempate por distancia nunca se ejecuta y quedan revueltos. Con 200 puntos colineales barajados que deben salir por distancia, el comparador exacto los ordena bien y el de atan2 los saca MAL, tanto con coordenadas chicas como de $2e8$. - Y hay algo peor que el orden: comparar angulos double con epsilon rompe la transitividad, y un comparador no transitivo en std::sort es comportamiento indefinido. La version exacta usa solo ENTEROS: se parte el plano en dos mitades (arriba o abajo del eje x) y dentro de la misma mitad se compara con el producto cruz. Cero punto flotante. SU LIMITE: cross y la norma multiplican coordenadas, y $2^c \wedge 2$ tiene que caber en long long, o sea $|\text{coord}|$ hasta $\sim 2.1e9$ (medido: con $2e9$ cabe, con $3e9$ la norma ya sale negativa). Para las coordenadas de $1e9$ tipicas queda un factor 4 de margen. CUANDO SI CONVIENEN LAS POLARES: rotar (sumar al angulo), simetria circular, sectores y arcos, y cualquier enunciado que ya venga en (r, θ) . Para SUMAR vectores hay que volver a cartesianas: en polares la suma no es componente a componente. Y para rotar un punto exacto conviene la matriz de rotacion sobre las cartesianas, que no pasa por atan2 ni acumula el error de ida y vuelta (medido: el viaje cartesiano -> polar -> cartesiano sobre 300000 puntos de hasta $1e6$ pierde a lo sumo $4.7e-10$, suficiente para un eps de $1e-9$ pero no gratis).

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const double PI = acos(-1.0);
5
6 // ----- conversion -----
7 struct Polar { double r, th; };
8
9 Polar aPolar(double x, double y) {
10     return {hypot(x, y), atan2(y, x)}; // th queda en (-pi, pi]
11 }
12
13 pair<double, double> aCartesiano(double r, double th) {
14     return {r * cos(th), r * sin(th)};
15 }
16
17 // Lleva cualquier angulo al rango [0, 2pi).
18 double normalizar(double th) {
19     th = fmod(th, 2 * PI);
20     if (th < 0) th += 2 * PI;
21     return th;
22 }
23
24 double grados(double rad) { return rad * 180.0 / PI; }
25 double radianes(double deg) { return deg * PI / 180.0; }
26
27 // ----- barrido angular exacto, sin atan2 -----
28 struct P {
29     long long x, y;
30 };
31
32 long long cross(const P &a, const P &b) { return a.x * b.y - a.y * b.x; }
33
34 // semiplano: 0 = angulo en [0, pi), 1 = en [pi, 2pi). El (0,0) cae en 0.
35 int mitad(const P &p) {
36     if (p.y > 0) return 0;
37     if (p.y < 0) return 1;
38     if (p.x >= 0) return 0; // sobre el eje x: el +x va en la 1a
39     return 1; // mitad y el -x en la 2a

```

```

40 }
41
42 // Ordena por angulo en [0, 2pi) arrancando en el +x. Los colineales quedan
43 // juntos y entre ellos ordenados por distancia al origen.
44 bool menorAngulo(const P &a, const P &b) {
45     int ha = mitad(a), hb = mitad(b);
46     if (ha != hb) return ha < hb;
47
48     long long c = cross(a, b);
49     if (c != 0) return c > 0;          // a va antes si b queda a su izq
50
51     return a.x * a.x + a.y * a.y < b.x * b.x + b.y * b.y;
52 }
53
54 int main() {
55     ios::sync_with_stdio(false);
56     cin.tie(nullptr);
57     cout << fixed << setprecision(6);
58
59     double x, y;
60     while (cin >> x >> y) {
61         Polar p = aPolar(x, y);
62         pair<double, double> q = aCartesiano(p.r, p.th);
63
64         cout << "r = " << p.r
65              << " theta = " << p.th
66              << " rad = " << grados(p.th) << " grados"
67              << " [0,2pi) = " << normalizar(p.th)
68              << " vuelta = (" << q.first << ", " << q.second << ")\n";
69     }
70     return 0;
71 }

```

4.5 Geometria 1D - Kilometric Intersection

C++ Training_Camp_2020_Day 9J - Kilometric Intersection.py

Resuelve "Kilometric Intersection" (Day 9, problema J - Contest 08 [Inicial & Avanzado]): dados dos intervalos cerrados $[a, b]$ y $[c, d]$ sobre una recta (tramos de carretera en kilometros), se pide la longitud de su interseccion, siendo 0 si no se superponen o solo se tocan en un punto. Calcula el intervalo de interseccion tomando $l = \max(a, c)$ y $r = \min(b, d)$, e imprime $\max(0, r - l)$, que es exactamente la longitud del solapamiento (o 0 si $l > r$).

```

1 import sys
2
3 n = int(input())
4
5 for _ in range(n):
6     a, b, c, d = map(int, input().split())
7
8     l = max(a, c)
9     r = min(b, d)
10
11     print(max(0, r - l))

```

4.6 K Vecinos Mutuos y Clasificacion de Componentes - Celestial Veins

C++ ICPC 2025 - Celestial Veins.cpp

Resuelve "Celestial Veins" (problema C, ICPC 2025): cada estrella mira a sus K vecinas mas cercanas, dos estrellas quedan unidas solo si CADA UNA esta entre las K de la otra (vinculo mutuo), y hay que clasificar cada componente conexa en una de cinco formas y contarlas. TRES COSAS QUE HAY QUE HACER BIEN Y SON FACILES DE ARRUINAR: 1) EL DESEMPATE. Si dos candidatas quedan a la misma distancia, gana la que aparece antes en la entrada. Ordenando pares (distancia al cuadrado, indice) eso sale solo, porque el indice desempata. Con coordenadas hasta 10^9 , $dx^2 dx + dy^2 dy$ llega a $8 \cdot 10^{18}$ y cabe JUSTO en long long: en int o en double se rompe. 2) EL VINCULO ES MUTUO. Que A tenga a B entre sus K no basta; hay que comprobar tambien al revés. Como cada estrella tiene a lo sumo 5 vecinas, revisarlo es $O(K)$. 3) EL ORDEN DE LA CLASIFICACION, que el enunciado fija: Cluster, Ouroboros, Claw, Path, Nebula. Importa porque las formas se solapan. Un triangulo es a la vez completo y un ciclo, y se cuenta como Cluster; un camino de 3 estrellas es a la vez garra y camino, y se cuenta como Claw. Evaluarlas en otro orden cambia las respuestas. Cada forma se reconoce contando aristas y grados, sin recorrer nada mas: completo: $E = V \cdot (V-1)/2$ ciclo: $E = V$ y todos los grados valen 2 garra: $E = V-1$, un grado $V-1$ y los demas 1 camino: $E = V-1$ y ningun grado pasa de 2 Una estrella sola ($V = 1$) es Nebula, y cualquier otra cosa tambien. El if de $V == 1$ va primero porque con $V = 1$ la formula del completo tambien se cumple ($E = 0$) y la clasificaria mal como Cluster. Costo $O(n^2)$ por mapa para hallar las vecinas, que con $n = 1500$ son unos 2 millones de pares. Medido: 5 mapas de $n = 1500$ con $K = 5$ en 93 ms, tanto con puntos al azar como en rejilla (que es el caso con mas empates de distancia). Verificado contra una implementacion independiente del enunciado en 1800 mapas, incluidos puntos alineados y en rejilla para forzar empates, sin una sola diferencia, ademas de los dos casos del sample.

```

1 #include <iostream>
2 #include <vector>
3 #include <queue>
4 #include <algorithm>
5
6 using namespace std;
7
8 int main()
9 {
10     ios::sync_with_stdio(false);
11     cin.tie(nullptr);
12
13     long long n, k;
14
15     while (cin >> n >> k && !(n == 0 && k == 0))
16     {
17         vector<long long> coordX(n), coordY(n);
18
19         for (long long i = 0; i < n; i++)
20         {
21             cin >> coordX[i] >> coordY[i];
22         }
23
24         vector<vector<long long>> cercanas(n);
25
26         long long cuantasVecinas = min(k, n - 1);
27
28         for (long long estrella = 0; estrella < n; estrella++)
29         {
30             vector<pair<long long, long long>> candidatas;
31
32             for (long long otra = 0; otra < n; otra++)
33             {
34                 if (otra == estrella)
35                 {
36                     continue;
37                 }
38
39                 long long dx = coordX[estrella] - coordX[otra];
40                 long long dy = coordY[estrella] - coordY[otra];
41
42                 long long distancia2 = dx * dx + dy * dy;
43
44                 candidatas.push_back({distancia2, otra});
45             }
46
47             partial_sort(candidatas.begin(),
48                         candidatas.begin() + cuantasVecinas,
49                         candidatas.end());
50
51             for (long long p = 0; p < cuantasVecinas; p++)
52             {

```

```

53     cercanas[estrella].push_back(candidatas[p].second);
54 }
55 }
56
57 vector<vector<long long>> vinculos(n);
58
59 for(long long a = 0; a < n; a++)
60 {
61     for(long long b : cercanas[a])
62     {
63         if(b < a)
64         {
65             continue;
66         }
67
68         bool bTieneA = false;
69
70         for(long long c : cercanas[b])
71         {
72             if(c == a)
73             {
74                 bTieneA = true;
75             }
76         }
77
78         if(bTieneA)
79         {
80             vinculos[a].push_back(b);
81             vinculos[b].push_back(a);
82         }
83     }
84 }
85
86 long long travelersPath = 0;
87 long long cosmicOuroboros = 0;
88 long long stellarCluster = 0;
89 long long dragonsClaw = 0;
90 long long amorphousNebula = 0;
91
92 vector<bool> visitada(n, false);
93
94 for(long long inicio = 0; inicio < n; inicio++)
95 {
96     if(visitada[inicio])
97     {
98         continue;
99     }
100
101     vector<long long> grupo;
102     queue<long long> cola;
103
104     cola.push(inicio);
105     visitada[inicio] = true;
106
107     while(!cola.empty())
108     {
109         long long actual = cola.front();
110         cola.pop();
111
112         grupo.push_back(actual);
113
114         for(long long vecina : vinculos[actual])
115         {
116             if(!visitada[vecina])
117             {
118                 visitada[vecina] = true;
119                 cola.push(vecina);
120             }
121         }
122     }
123
124     long long estrellas = grupo.size();
125     long long sumaGrados = 0;

```

```

86     long long conGrado1 = 0;
87     long long conGrado2 = 0;
88     long long gradoMaximo = 0;
89
90     for(long long estrella : grupo)
91     {
92         long long grado = vinculos[estrella].size();
93
94         sumaGrados += grado;
95         gradoMaximo = max(gradoMaximo, grado);
96
97         if(grado == 1) conGrado1++;
98         if(grado == 2) conGrado2++;
99     }
100
101     long long enlaces = sumaGrados / 2;
102
103     if(estrellas == 1)
104     {
105         amorphousNebula++;
106     }
107     else if(enlaces == estrellas * (estrellas - 1) / 2)
108     {
109         stellarCluster++;
110     }
111     else if(enlaces == estrellas && conGrado2 == estrellas)
112     {
113         cosmicOuroboros++;
114     }
115     else if(enlaces == estrellas - 1 &&
116             gradoMaximo == estrellas - 1 &&
117             conGrado1 == estrellas - 1)
118     {
119         dragonsClaw++;
120     }
121     else if(enlaces == estrellas - 1 && gradoMaximo <= 2)
122     {
123         travelersPath++;
124     }
125     else
126     {
127         amorphousNebula++;
128     }
129 }
130
131 cout << travelersPath << ' '
132      << cosmicOuroboros << ' '
133      << stellarCluster << ' '
134      << dragonsClaw << ' '
135      << amorphousNebula << '\n';
136 }
137
138 return 0;
139 }

```

4.7 K-Means (Clustering)

C++\Notebook\K-Means.cpp

Agrupar n puntos (de cualquier dimensi n d) en k clusters minimizando el SSE: la suma de las distancias al cuadrado de cada punto al centroide de su cluster. Algoritmo de Lloyd: (1) asignar cada punto al centroide mas cercano, (2) mover cada centroide al promedio de sus puntos, y repetir hasta que ninguna asignaci n cambie. Cada paso solo puede bajar el SSE, as  que siempre converge, pero a un  ptimo LOCAL que depende de los centroides iniciales. Por eso se siembra con k -means++ (cada centroide nuevo sale con probabilidad proporcional a su distancia² al centroide mas cercano, lo que los reparte) y se corre con varias semillas quedandose con el menor SSE. Lo que rompe la versi n ingenua: (a) un cluster puede quedar vac o y su promedio no existe; aqu  se re-siembra en el punto que peor encaja; (b) sin tope de iteraciones no hay cota pr ctica de tiempo. Costo: $O(\text{reinicios} * \text{iteraciones} * n * k * d)$. OJO: el  ptimo exacto es NP-hard. Si el problema es 1D y pide el  ptimo, ordenar y hacer DP de partici n en segmentos contiguos ($O(k * n^2)$), o menos con Divide and Conquer Optimization).

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 typedef vector<double> Vec;

```

```

6 double dist2(const Vec &a, const Vec &b)
7 {
8     double s = 0;
9     for (size_t t = 0; t < a.size(); t++) s += (a[t] - b[t]) * (a[t] - b[t]);
10    return s;
11 }
12
13 struct KMeans
14 {
15     vector<Vec> c; // centroides
16     vector<int> label; // cluster de cada punto
17     double sse; // suma de distancias^2 de cada punto a su centroide
18 };
19
20 // Centroide mas cercano a x (en empate, el de menor indice)
21 int nearest(const Vec &x, const vector<Vec> &c)
22 {
23     int best = 0;
24     double bd = dist2(x, c[0]);
25     for (int j = 1; j < (int)c.size(); j++)
26     {
27         double d = dist2(x, c[j]);
28         if (d < bd)
29         {
30             bd = d;
31             best = j;
32         }
33     }
34     return best;
35 }
36
37 KMeans kmeansOnce(const vector<Vec> &p, int k, mt19937 &rng, int maxIter)
38 {
39     int n = p.size(), d = p[0].size();
40
41     // Semilla k-means++
42     vector<Vec> c(1, p[uniform_int_distribution<int>(0, n - 1)(rng)]);
43     vector<double> md(n, 1e300); // distancia^2 al centroide elegido mas cercano
44     while ((int)c.size() < k)
45     {
46         double total = 0;
47         for (int i = 0; i < n; i++)
48         {
49             md[i] = min(md[i], dist2(p[i], c.back()));
50             total += md[i];
51         }
52         int pick = n - 1;
53         if (total > 0)
54         {
55             double r = uniform_real_distribution<double>(0, total)(rng);
56             for (int i = 0; i < n; i++)
57             {
58                 r -= md[i];
59                 if (r <= 0)
60                 {
61                     pick = i;
62                     break;
63                 }
64             }
65         }
66         else pick = uniform_int_distribution<int>(0, n - 1)(rng); // todos repetidos
67         c.push_back(p[pick]);
68     }
69
70     // Lloyd
71     vector<int> label(n, -1);
72     for (int it = 0; it < maxIter; it++)
73     {
74         bool changed = false;
75         for (int i = 0; i < n; i++)
76         {
77             int j = nearest(p[i], c);

```

```

78             if (j != label[i])
79             {
80                 label[i] = j;
81                 changed = true;
82             }
83         }
84         if (!changed) break;
85
86         vector<Vec> sum(k, Vec(d, 0.0));
87         vector<int> cnt(k, 0);
88         for (int i = 0; i < n; i++)
89         {
90             cnt[label[i]]++;
91             for (int t = 0; t < d; t++) sum[label[i]][t] += p[i][t];
92         }
93
94         // Para re-sembrar un cluster vacio: el punto mas lejano a su centroide actual
95         vector<double> err(n);
96         for (int i = 0; i < n; i++) err[i] = dist2(p[i], c[label[i]]);
97
98         for (int j = 0; j < k; j++)
99         {
100             if (cnt[j] > 0)
101             {
102                 for (int t = 0; t < d; t++) c[j][t] = sum[j][t] / cnt[j];
103             }
104             else
105             {
106                 int far = max_element(err.begin(), err.end()) - err.begin();
107                 c[j] = p[far];
108                 err[far] = -1; // que otro cluster vacio no tome el mismo punto
109             }
110         }
111     }
112 }
113
114 KMeans res;
115 res.c = c;
116 res.label.assign(n, 0);
117 res.sse = 0;
118 for (int i = 0; i < n; i++)
119 {
120     res.label[i] = nearest(p[i], c);
121     res.sse += dist2(p[i], c[res.label[i]]);
122 }
123 return res;
124 }
125
126 // Mejor de 'restarts' corridas. Semilla fija: resultado reproducible
127 KMeans kmeans(const vector<Vec> &p, int k, int restarts = 10, int maxIter = 300,
128               unsigned seed = 7)
129 {
130     k = max(1, min(k, (int)p.size()));
131     mt19937 rng(seed);
132     KMeans best;
133     best.sse = numeric_limits<double>::infinity();
134     for (int r = 0; r < restarts; r++)
135     {
136         KMeans cur = kmeansOnce(p, k, rng, maxIter);
137         if (cur.sse < best.sse) best = cur;
138     }
139     return best;
140 }
141
142 int main()
143 {
144     ios_base::sync_with_stdio(false);
145     cin.tie(nullptr);
146
147     int n, k, d;
148     cin >> n >> k >> d;
149     vector<Vec> p(n, Vec(d));
150     for (auto &x : p)
151         for (auto &v : x) cin >> v;

```

```

62
63 KMeans r = kmeans(p, k);
64
65 cout << fixed << setprecision(6) << r.sse << "\n";
66 for (auto &cc : r.c)
67 {
68     for (int t = 0; t < d; t++) cout << cc[t] << (t + 1 < d ? ' ' : '\n');
69 }
70 for (int i = 0; i < n; i++) cout << r.label[i] << (i + 1 < n ? ' ' : '\n');
71 return 0;
72 }

```

4.8 Primitivas y Convex Hull

C++ Notebook Primitivas y Convex Hull.cpp

Base de todo problema geometrico: struct Pt con operadores, producto punto y producto cruz. El cruz es la herramienta central: su signo dice si tres puntos giran a la izquierda, a la derecha o son colineales, y con eso se resuelve casi todo. Nunca comparar doubles con == : usar sgn() con EPS. Incluye interseccion de segmentos (dos pruebas de orientacion opuesta, mas los casos degenerados colineales via onSeg), convex hull por monotone chain en $O(n \log n)$ (ordena por coordenada y arma la cadena inferior y la superior descartando giros no convexos), y area de poligono simple por la formula del cordon (shoelace), que vale para cualquier poligono cerrado sin auto-intersecciones y devuelve el doble del area en valor absoluto entre dos.

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 const double EPS = 1e-9;
6
7 int sgn(double x)
8 {
9     return (x > EPS) - (x < -EPS);
10 }
11
12 struct Pt
13 {
14     double x, y;
15     Pt() {}
16     Pt(double x, double y) : x(x), y(y) {}
17     Pt operator+(const Pt &o) const { return Pt(x + o.x, y + o.y); }
18     Pt operator-(const Pt &o) const { return Pt(x - o.x, y - o.y); }
19     Pt operator*(double k) const { return Pt(x * k, y * k); }
20 };
21
22 double dot(Pt a, Pt b) { return a.x * b.x + a.y * b.y; }
23 double cross(Pt a, Pt b) { return a.x * b.y - a.y * b.x; }
24 double cross(Pt a, Pt b, Pt c) { return cross(b - a, c - a); }
25
26 // p esta sobre el segmento a-b (asumiendo colinealidad)
27 bool onSeg(Pt a, Pt b, Pt p)
28 {
29     return sgn(cross(a, b, p)) == 0 && sgn(dot(p - a, p - b)) <= 0;
30 }
31
32 bool segIntersect(Pt a, Pt b, Pt c, Pt d)
33 {
34     int o1 = sgn(cross(a, b, c)), o2 = sgn(cross(a, b, d));
35     int o3 = sgn(cross(c, d, a)), o4 = sgn(cross(c, d, b));
36     if (o1 * o2 < 0 && o3 * o4 < 0) return true;
37     return onSeg(a, b, c) || onSeg(a, b, d) || onSeg(c, d, a) || onSeg(c, d, b);
38 }
39
40 // Monotone chain: devuelve el casco convexo en sentido antihorario
41 vector<Pt> convexHull(vector<Pt> p)
42 {
43     sort(p.begin(), p.end(), [](const Pt &a, const Pt &b){
44         if (a.x == b.x) return a.y < b.y;
45         return a.x < b.x;
46     });
47
48     vector<Pt> lo, up;
49     for (auto &pt : p)
50     {
51         while (lo.size() >= 2 && sgn(cross(lo[lo.size() - 2], lo.back(), pt)) <= 0)
52             lo.pop_back();
53         while (up.size() >= 2 && sgn(cross(up[up.size() - 2], up.back(), pt)) <= 0)
54             up.pop_back();
55         lo.push_back(pt);
56         up.push_back(pt);
57     }
58     lo.pop_back();
59     up.pop_back();
60     lo.insert(lo.end(), up.begin(), up.end());
61     return lo;
62 }
63
64 // Formula del cordon (shoelace) para poligono simple
65 double polygonArea(const vector<Pt> &P)
66 {
67     long double s = 0;
68     int n = P.size();
69     for (int i = 0; i < n; i++)
70     {
71         s += (long double)P[i].x * P[(i + 1) % n].y - (long double)P[i].y * P[(i + 1) % n].x;
72     }
73     return fabs((double)s) * 0.5;
74 }
75
76 int main()
77 {
78     ios_base::sync_with_stdio(false);
79     cin.tie(nullptr);
80     return 0;
81 }

```

```

53         lo.pop_back();
54     }
55     lo.push_back(pt);
56 }
57 for (int i = (int)p.size() - 1; i >= 0; --i)
58 {
59     Pt pt = p[i];
60     while (up.size() >= 2 && sgn(cross(up[up.size() - 2], up.back(), pt)) <= 0)
61     {
62         up.pop_back();
63     }
64     up.push_back(pt);
65 }
66
67 lo.pop_back();
68 up.pop_back();
69 lo.insert(lo.end(), up.begin(), up.end());
70 return lo;
71 }
72
73 // Formula del cordon (shoelace) para poligono simple
74 double polygonArea(const vector<Pt> &P)
75 {
76     long double s = 0;
77     int n = P.size();
78     for (int i = 0; i < n; i++)
79     {
80         s += (long double)P[i].x * P[(i + 1) % n].y - (long double)P[i].y * P[(i + 1) % n].x;
81     }
82     return fabs((double)s) * 0.5;
83 }
84
85 int main()
86 {
87     ios_base::sync_with_stdio(false);
88     cin.tie(nullptr);
89     return 0;
90 }

```

4.9 Probabilidad de Circulo Contenido - Omens

C++ VCPY(2024) - Omens.cpp

Resuelve "Omens" (problema I, ICPC 2024): caen dos piedras uniformemente al azar dentro de un rectangulo $L \times W$ y se dibuja el circulo que las tiene como DIAMETRO (centro en el punto medio, radio la mitad de la distancia). Se pide la probabilidad de que ese circulo quede completamente dentro del rectangulo, a 4 decimales. De entrada parece una integral en CUATRO dimensiones y da miedo. Lo que la desarma es una caracterizacion de geometria, no de programacion: el circulo de diametro AB cabe dentro de un convexo K si y solo si TODO punto de la frontera de K ve al segmento AB con angulo de a lo sumo 90 grados porque el circulo de diametro AB es justo el lugar donde ese angulo vale exactamente 90 (arco capaz), y por fuera del circulo el angulo es menor. Aplicandolo al lado $x = 0$ con un punto $Q = (0, q)$, "angulo $\leq 90^\circ$ " es $(A-Q) \cdot (B-Q) \geq 0$, o sea $x_1^2 x_2^2 + (y_1 - q)(y_2 - q) \geq 0$ y como hay que pedirlo para TODO q , se minimiza: el minimo cae en $q = (y_1 + y_2)/2$ y vale $x_1^2 x_2^2 - (y_1 - y_2)^2 / 4$. Repitiendo en los cuatro lados quedan cuatro desigualdades limpias, sin una sola raiz cuadrada: $4^* x_1^2 x_2^2 \geq (y_1 - y_2)^2$, $4^* (W - x_1)^2 (W - x_2) \geq (y_1 - y_2)^2$, $4^* y_1^2 y_2^2 \geq (x_1 - x_2)^2$, $4^* (L - y_1)(L - y_2) \geq (x_1 - x_2)^2$. Verificado: esas cuatro equivalen a la condicion directa (centro y radio) en 3000000 de pruebas aleatorias, sin una sola diferencia. DE AHI SALE UNA FORMULA CERRADA. Las desigualdades son homogeneas de grado 2, asi que escalar el rectangulo no cambia la probabilidad: solo depende de la PROPORCION entre los lados. Con $t = \min(L, W) / \max(L, W)$, la respuesta es $P = (\pi/6) * t * (2 - t)$. Con el cuadrado ($t = 1$) queda $\pi/6 = 0.5236$, y con proporcion 1:2 queda $\pi/8 = 0.3927$, que son justo los dos primeros casos del enunciado. HONESTIDAD SOBRE ESTA FORMULA: salio de conjeturar desde los ejemplos y esta VERIFICADA NUMERICAMENTE, no demostrada. Se contrasto en 8 proporciones distintas (de 0.001 hasta 1) integrando exacto la ultima variable para bajar la varianza, y todas las diferencias quedaron en $2^* 10^{-5}$, dentro del error y sin sesgo. Si en un problema parecido la formula no cuadra, lo que si esta demostrado y se puede usar siempre son las cuatro desigualdades de arriba: con ellas se integra numericamente y se sale igual. El lado util para llevarse: cuando un enunciado pida "probabilidad de que una figura quepa", conviene traducir el "cabe" a desigualdades sobre las coordenadas antes de pensar en integrar. Casi siempre se simplifican, y a veces colapsan a una formula de una linea como aqui.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7     cout << fixed << setprecision(4);
8
9     double L, W;

```

```

0
1 // La entrada termina con la linea "0 0", que no se procesa.
2 while (cin >> L >> W && (L != 0 || W != 0)) {
3     double t = min(L, W) / max(L, W);
4
5     cout << acos(-1.0) / 6.0 * t * (2 - t) << '\n';
6 }
7
8 return 0;
9 }

```

4.10 Punto en Poligono Convexo ($O(\log n)$)

C++(Notebook\Punto en Poligono Convexo.cpp)

Arma el casco convexo de un conjunto de puntos y luego responde muchas consultas del tipo "este punto esta dentro?" en $O(\log n)$ cada una, en vez de $O(n)$ revisando arista por arista. Todo se hace con enteros (el cross devuelve long long), asi que no hay error de precision. El casco usa monotone chain de Andrew; descartar con $\text{cross} \leq 0$ elimina los puntos colineales y deja solo los vertices reales, devolviendo el hull en sentido antihorario. La consulta triangula mentalmente el poligono en abanico desde $h[0]$: primero descarta el punto si cae fuera de las dos aristas extremas ($h[0]-h[1]$ y $h[0]-h[n-1]$), y si no, busca binariamente el sector ($h[0]$, $h[lo]$, $h[lo+1]$) donde cae y comprueba de que lado esta de esa arista. Los cross con ≥ 0 hacen que el borde cuente como dentro. Contempla los casos degenerados en que el casco queda reducido a un punto o a un segmento.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4
5 struct P { ll x, y; };
6
7 ll cross(const P&o, const P&a, const P&b){
8     return (A.x-o.x)*(B.y-o.y) - (A.y-o.y)*(B.x-o.x);
9 }
10
11 vector<P> convexHull(vector<P> p){
12     int n = p.size(), k = 0;
13     sort(p.begin(), p.end(), [](const P&a, const P&b){
14         return a.x != b.x ? a.x < b.x : a.y < b.y;
15     });
16     p.erase(unique(p.begin(), p.end(), [](const P&a, const P&b){
17         return a.x==b.x && a.y==b.y;
18     }), p.end());
19     n = p.size();
20     if (n < 3) return p;
21
22     vector<P> h(2*n);
23     for (int i = 0; i < n; i++){ // cadena inferior
24         while (k >= 2 && cross(h[k-2], h[k-1], p[i]) <= 0) k--;
25         h[k++] = p[i];
26     }
27     for (int i = n-2, t = k+1; i >= 0; i--){ // cadena superior
28         while (k >= t && cross(h[k-2], h[k-1], p[i]) <= 0) k--;
29         h[k++] = p[i];
30     }
31     h.resize(k-1); // quita el repetido final
32     return h; // CCW, sin colineales
33 }
34
35 bool inside(const vector<P>& h, const P& q){
36     int n = h.size();
37     if (n == 1) return h[0].x==q.x && h[0].y==q.y;
38     if (n == 2){ // segmento degenerado
39         if (cross(h[0], h[1], q) != 0) return false;
40         return min(h[0].x, h[1].x) <= q.x && q.x <= max(h[0].x, h[1].x)
41             && min(h[0].y, h[1].y) <= q.y && q.y <= max(h[0].y, h[1].y);
42     }
43     if (cross(h[0], h[1], q) < 0) return false;
44     if (cross(h[0], h[n-1], q) > 0) return false;
45
46     int lo = 1, hi = n-1; // busca el sector
47     while (hi - lo > 1){
48         int mid = (lo + hi) / 2;
49         if (cross(h[0], h[mid], q) >= 0) lo = mid;
50         else hi = mid;
51     }

```

```

52     return cross(h[lo], h[lo+1], q) >= 0;
53 }
54
55 int main(){
56     ios::sync_with_stdio(false);
57     cin.tie(nullptr);
58     int L;
59     while (cin >> L){
60         vector<P> big(L);
61         for (auto &b : big) cin >> b.x >> b.y;
62         vector<P> h = convexHull(big);
63
64         int S; cin >> S;
65         int cnt = 0;
66         for (int i = 0; i < S; i++){
67             P q; cin >> q.x >> q.y;
68             if (inside(h, q)) cnt++;
69         }
70         cout << cnt << "\n";
71     }
72     return 0;
73 }

```

4.11 Rotating Calipers (Par Mas Lejano)

C++(Notebook\Rotating Calipers.cpp)

Par de puntos mas lejano (diametro del conjunto) en $O(n \log n)$. El par optimo siempre son dos vertices del casco convexo, asi que se arma el casco y se recorre con dos punteros: para cada arista ($i, i+1$) se avanza j mientras el triangulo ($i, i+1, j$) crezca en area. Donde deja de crecer, j es el vertice mas lejano a la recta de esa arista (su antipodal). Como j solo avanza y da a lo sumo una vuelta, el recorrido cuesta $O(n)$. Detalle que la version ingenua olvida: el antipodal se mide contra LOS DOS extremos de la arista (i e $i+1$); medir solo contra i puede perder el par optimo. Todo en enteros con distancia AL CUADRADO: exacto, sin sqrt ni EPS (coordenadas hasta $\sim 1e9$ para que no desborde long long). El casco sale sin repetidos ni colineales, que es lo que necesita el while (con colineales el area deja de ser estrictamente unimodal). Casos borde: un solo punto distinto $\rightarrow 0$; todos colineales $\rightarrow$ el casco tiene 2 puntos. El mismo recorrido de antipodales resuelve: ancho minimo del conjunto (minimo, sobre las aristas, de la distancia a su antipodal), rectangulo de area minima que lo cubre y distancia entre dos poligonos convexos.

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 typedef long long ll;
6
7 struct P
8 {
9     ll x, y;
10     bool operator<(const P &o) const { return x != o.x ? x < o.x : y < o.y; }
11     bool operator==(const P &o) const { return x == o.x && y == o.y; }
12 };
13
14 P operator-(P a, P b) { return {a.x - b.x, a.y - b.y}; }
15 ll cross(P a, P b) { return a.x * b.y - a.y * b.x; }
16 ll cross(P o, P a, P b) { return cross(a - o, b - o); }
17 ll dist2(P a, P b)
18 {
19     P d = a - b;
20     return d.x * d.x + d.y * d.y;
21 }
22
23 // Monotone chain: casco en sentido antihorario, sin repetidos ni colineales
24 vector<P> convexHull(vector<P> p)
25 {
26     sort(p.begin(), p.end());
27     p.erase(unique(p.begin(), p.end(), p.end()), p.end());
28     int n = p.size();
29     if (n <= 2) return p;
30
31     vector<P> h(2 * n);
32     int k = 0;
33     for (int i = 0; i < n; i++){
34         {
35             while (k >= 2 && cross(h[k - 2], h[k - 1], p[i]) <= 0) k--;
36             h[k++] = p[i];
37         }

```

```

38 for (int i = n - 2, lo = k + 1; i >= 0; i--)
39 {
40     while (k >= lo && cross(h[k - 2], h[k - 1], p[i]) <= 0) k--;
41     h[k++] = p[i];
42 }
43 h.resize(k - 1);
44 return h;
45 }
46
47 // {distancia^2 maxima, par de puntos}. h = casco antihorario (el de convexHull)
48 pair<ll, pair<P, P>> diameter(const vector<P> &h)
49 {
50     int n = h.size();
51     if (n == 1) return {0, {h[0], h[0]}};
52     if (n == 2) return {dist2(h[0], h[1]), {h[0], h[1]}};
53
54     ll best = -1;
55     P a = h[0], b = h[0];
56     for (int i = 0, j = 1; i < n; i++)
57     {
58         int ni = (i + 1) % n;
59         // avanzar j mientras el area del triangulo (i, ni, j) siga creciendo
60         while (cross(h[i], h[ni], h[(j + 1) % n]) > cross(h[i], h[ni], h[j])) j = (j + 1) % n;
61
62         if (dist2(h[i], h[j]) > best)
63         {
64             best = dist2(h[i], h[j]);
65             a = h[i];
66             b = h[j];
67         }
68         if (dist2(h[ni], h[j]) > best)
69         {
70             best = dist2(h[ni], h[j]);
71             a = h[ni];
72             b = h[j];
73         }
74     }
75     return {best, {a, b}};
76 }
77
78 int main()
79 {
80     ios_base::sync_with_stdio(false);
81     cin.tie(nullptr);
82
83     int n;
84     cin >> n;
85     vector<P> p(n);
86     for (auto &q : p) cin >> q.x >> q.y;
87
88     auto r = diameter(convexHull(p));
89
90     cout << r.first << "\n"; // distancia al cuadrado (exacta)
91     cout << fixed << setprecision(6) << sqrt((double)r.first) << "\n";
92     cout << r.second.first.x << " " << r.second.first.y << " "
93         << r.second.second.x << " " << r.second.second.y << "\n";
94     return 0;
95 }

```

4.12 Shoelace (Area de Poligono)

C++\Notebook\Shoelace.cpp

Area de CUALQUIER poligono simple en $O(n)$, convexo o concavo, dando solo sus vertices en orden. Suma cruzada de cada arista con la siguiente: $\sum (x_i y_{i+1} - x_{i+1} y_i)$, y el area es la mitad del valor absoluto. El truco que importa en competencia es no dividir entre 2: se devuelve el DOBLE del area como long long, que con coordenadas enteras es exacto y evita por completo el punto flotante. Si el problema pide el area y puede ser .5, imprime $a2/2$ y $a2\%2$ por separado, o multiplica todo por 2 desde el principio. El signo tambien sirve: positivo significa que los vertices vienen en sentido antihorario, negativo en sentido horario, asi que de paso detecta la orientacion gratis. Requisitos: los vertices deben ir en orden alrededor del poligono y el borde no puede cruzarse consigo mismo. No hace falta repetir el primer vertice al final.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;

```

```

4
5 struct P { ll x, y; };
6
7 // Doble del area CON SIGNO. Exacto con coordenadas enteras.
8 // > 0 antihorario, < 0 horario, = 0 degenerado (todos colineales).
9 ll area2(const vector<P> &p) {
10     ll s = 0;
11     int n = p.size();
12     for (int i = 0; i < n; i++) {
13         int j = (i + 1) % n;
14         s += p[i].x * p[j].y - p[j].x * p[i].y;
15     }
16     return s;
17 }
18
19 // Area de verdad. Solo aqui aparece el punto flotante.
20 double area(const vector<P> &p) {
21     return llabs(area2(p)) / 2.0;
22 }
23
24 // Puntos de coordenadas enteras SOBRE el borde.
25 // Cada arista aporta gcd(|dx|, |dy|) puntos sin contar dos veces los vertices.
26 ll puntosBorde(const vector<P> &p) {
27     ll b = 0;
28     int n = p.size();
29     for (int i = 0; i < n; i++) {
30         int j = (i + 1) % n;
31         b += __gcd(llabs(p[i].x - p[j].x), llabs(p[i].y - p[j].y));
32     }
33     return b;
34 }
35
36 // Puntos enteros ESTRICTAMENTE interiores, despejando Pick: A = I + B/2 - 1.
37 // Con 2A entero la cuenta sale exacta y sin dividir.
38 ll puntosInteriores(const vector<P> &p) {
39     return (llabs(area2(p)) - puntosBorde(p) + 2) / 2;
40 }
41
42 // Centroid (centro de masa) del poligono. No es el promedio de los vertices.
43 pair<double, double> centroide(const vector<P> &p) {
44     int n = p.size();
45     double a = 0, cx = 0, cy = 0;
46     for (int i = 0; i < n; i++) {
47         int j = (i + 1) % n;
48         double cr = (double)p[i].x * p[j].y - (double)p[j].x * p[i].y;
49         a += cr;
50         cx += (p[i].x + p[j].x) * cr;
51         cy += (p[i].y + p[j].y) * cr;
52     }
53     a /= 2;
54     return {cx / (6 * a), cy / (6 * a)};
55 }
56
57 int main() {
58     ios::sync_with_stdio(false);
59     cin.tie(nullptr);
60
61     int n;
62     if (!(cin >> n)) return 0;
63
64     vector<P> p(n);
65     for (auto &v : p) cin >> v.x >> v.y;
66
67     ll a2 = area2(p);
68
69     // Imprimir sin perder el medio punto: parte entera y el .5 aparte.
70     cout << llabs(a2) / 2;
71     if (llabs(a2) % 2) cout << ".5";
72     cout << "\n";
73
74     return 0;
75 }
76

```

```

77 // 030 con el overflow: si las coordenadas llegan a 1e9, cada producto
78 // llega a 1e18 y la suma de varios se pasa de long long. En ese caso
79 // acumula en __int128, o traslada todos los puntos restando el primer
80 // vertice antes de sumar (las coordenadas se vuelven chicas y el area
81 // no cambia).

```

4.13 Triangle Classification - AROD

C++ (RPC, AROD, C++)

Clasifica triángulos en una grilla de $p \times q$ puntos como agudos, rectos, obtusos o degenerados. Utiliza análisis de productos para determinar el tipo de ángulo en cada triángulo.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using int64 = long long;
5
6 static inline int64 comb3(int64 n) {
7     if (n < 3) return 0;
8     return n * (n - 1) * (n - 2) / 6;
9 }
10
11 int main() {
12     ios::sync_with_stdio(false);
13     cin.tie(nullptr);
14
15     int p, q;
16     cin >> p >> q;
17
18     if (p > q) swap(p, q); // p = lado pequeño, q = lado grande
19
20     const int W = p + 1; // cantidad de columnas de puntos
21     const int H = q + 1; // cantidad de filas de puntos
22
23     // gtProd[s][k] = cantidad de alpha en [0..s] con alpha*(s-alpha) > k
24     // eqProd[s][k] = cantidad de alpha en [0..s] con alpha*(s-alpha) = k
25     int maxS = p;
26     int maxK = maxS * maxS / 4;
27
28     vector<vector<int>> gtProd(maxS + 1, vector<int>(maxK + 1, 0));
29     vector<vector<int>> eqProd(maxS + 1, vector<int>(maxK + 1, 0));
30
31     for (int s = 0; s <= maxS; s++) {
32         int localMax = s * s / 4;
33         vector<int> freq(localMax + 1, 0);
34
35         for (int alpha = 0; alpha <= s; alpha++) {
36             int val = alpha * (s - alpha);
37             freq[val]++;
38         }
39
40         int suf = 0;
41         for (int k = localMax; k >= 0; k--) {
42             gtProd[s][k] = suf;
43             suf += freq[k];
44             eqProd[s][k] = freq[k];
45         }
46     }
47
48     auto getGt = [&](int s, int64 k) -> int64 {
49         int64 localMax = 1LL * s * s / 4;
50         if (k > localMax) return 0;
51         return gtProd[s][(int)k];
52     };
53
54     auto getEq = [&](int s, int64 k) -> int64 {
55         int64 localMax = 1LL * s * s / 4;
56         if (k > localMax) return 0;
57         return eqProd[s][(int)k];
58     };
59
60     int64 acute = 0, right = 0, obtuse = 0, degenerate = 0;
61
62     // -----

```

```

63 // 1) Todos en la misma fila -> degenerados
64 // -----
65 degenerate += 1LL * H * comb3(W);
66
67 // -----
68 // 2) Triángulos usando exactamente 2 filas
69 //
70 // Si la distancia vertical es h:
71 // - caso "punto solo entre la base"
72 // - caso "punto solo afuera de la base"
73 // -----
74 for (int h = 1; h <= q; h++) {
75     int64 verticalWays = 2LL * (H - h); // escoger las dos filas y cuál tiene 2 puntos
76
77     for (int s = 1; s <= p; s++) {
78         int64 waysPlaceX = W - s;
79         int64 weight = verticalWays * waysPlaceX;
80
81         // Punto solo entre los dos de la base
82         int64 rightBetween = 2 + getEq(s, 1LL * h * h); // extremos + posibles internos
83         int64 obtuseBetween = getGt(s, 1LL * h * h);
84         int64 acuteBetween = (s + 1) - rightBetween - obtuseBetween;
85
86         // Punto solo fuera del intervalo horizontal de la base
87         // todos esos son obtusos, salvo el caso alineado con un extremo,
88         // que ya quedó contado en "between" con alpha=0 o beta=0
89         int64 obtuseOutside = 2LL * (s - 1);
90
91         right += weight * rightBetween;
92         obtuse += weight * (obtuseBetween + obtuseOutside);
93         acute += weight * acuteBetween;
94     }
95 }
96
97 // -----
98 // 3) Triángulos usando exactamente 3 filas distintas
99 //
100 // Distancias verticales:
101 // fila1 -> fila2 = a
102 // fila2 -> fila3 = b
103 //
104 // Se cuenta por span horizontal total s.
105 // -----
106 for (int a = 1; a < q; a++) {
107     for (int b = 1; a + b <= q; b++) {
108         int64 verticalWays = H - (a + b);
109         int64 ab = 1LL * a * b;
110         int64 A = 1LL * a * (a + b);
111         int64 B = 1LL * b * (a + b);
112
113         for (int s = 0; s <= p; s++) {
114             int64 waysPlaceX = W - s;
115             int64 weight = verticalWays * waysPlaceX;
116
117             // Caso monótono:
118             // x1<x2<x3 o x1>x2>x3
119             // Todo no degenerado aquí es obtuso.
120             int64 orient = (s == 0 ? 1 : 2);
121
122             int64 degHit = 0;
123             if ((1LL * a * s) % (a + b) == 0) {
124                 degHit = 1;
125             }
126
127             degenerate += weight * orient * degHit;
128             obtuse += weight * orient * ((s + 1) - degHit);
129
130             if (s == 0) continue;
131
132             // -----
133             // Caso donde x1 queda estrictamente "en medio"
134             // u < 0 y u+v > 0
135             // alpha en [1 .. s-1]

```

```

86 // -----
87 {
88     int64 total = s - 1;
89     if (total > 0) {
90         int64 low = min<int64>(total, (ab - 1) / s);
91         int64 rightLin = ((ab % s == 0) && (1 <= ab / s) && (ab / s <= s - 1)) ? 1
92 : 0;
93
94         int64 obt1 = low + getGt(s, A);
95         int64 rig1 = rightLin + getEq(s, A);
96         int64 acu1 = total - obt1 - rig1;
97
98         obtuse += weight * 2 * obt1;
99         right += weight * 2 * rig1;
100        acute += weight * 2 * acu1;
101    }
102 }
103 // -----
104 // Caso donde x3 queda en medio o empata con x1
105 // v < 0 y u+v >= 0
106 // alpha en [0 .. s-1]
107 // -----
108 {
109     int64 total = s;
110     int64 low = min<int64>(total, (ab - 1) / s);
111     int64 rightLin = ((ab % s == 0) && (1 <= ab / s) && (ab / s <= s)) ? 1 : 0;
112
113     int64 obt3 = low + getGt(s, B);
114     int64 rig3 = rightLin + getEq(s, B);
115     int64 acu3 = total - obt3 - rig3;
116
117     obtuse += weight * 2 * obt3;
118     right += weight * 2 * rig3;
119     acute += weight * 2 * acu3;
120 }
121 }
122 }
123 }
124
125 cout << acute << '\n';
126 cout << right << '\n';
127 cout << obtuse << '\n';
128 cout << degenerate << '\n';
129
130 return 0;
131 }

```

5 - Math

5.1 2-Coloreo por Suma Potencia de 2 - Drug Test

C++\ICPC\2024\D - Drug Test.cpp

Resuelve "Drug Test" (problema D, ICPC 2024). El código le asigna a cada x de 1 a N un tipo, A o P, con el 0 fijo en P, de forma que x y $p - x$ queden con tipos OPUESTOS, siendo p la menor potencia de 2 que es mayor o igual a x (o sea, x y su complemento suman potencia de 2). LA OBSERVACION QUE LO VUELVE UN FOR: el complemento $p - x$ siempre es MENOR que x . Como p es la menor potencia de 2 que alcanza a x , $p/2 < x$, o sea $p < 2x$, y entonces $p - x < x$ (verificado para x hasta 2^*10^5). Así que cada x depende de un numero mas chico, y recorriendo en orden creciente su complemento ya tiene tipo cuando se le necesita. Visto como grafo: cada x tiene exactamente un "padre" menor que el, así que las restricciones forman un ARBOL con raíz en 0. Un arbol siempre se puede 2-colorear y la coloracion queda determinada por la raíz, por eso no hay que probar nada ni hacer backtracking. El caso "x es potencia de 2" es redundante: ahí el complemento es 0, que es P, y la regla general ya le da A. COMO LEER BIEN EL ENUNCIADO: al pie de la letra solo pide que cada tamaño tenga AL MENOS UN companero de tipo opuesto, y así hay muchas asignaciones validas (con $N = 12$ salen 66), lo que choca con que el enunciado diga que el diseño es unico. La lectura que lo hace unico es la del ejemplo, que lista TODOS los pares de tamanos distintos que suman potencia de 2: ninguno puede quedar con el mismo tipo. El código solo mira un companero por numero (el del arbol), pero esa coloracion resulta respetar tambien todos los demas pares: comprobado para N hasta 9999, sin un conflicto, y enumerando todas las asignaciones para N hasta 15 es la unica.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {

```

```

5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int N, Q;
9
10    while (cin >> N >> Q && (N || Q)) {
11
12        vector<char> tipo(N + 1);
13
14        // 0 siempre es placebo
15        tipo[0] = 'P';
16
17        for (int x = 1; x <= N; x++) {
18
19            int p = 1;
20
21            while (p < x)
22                p *= 2;
23
24            if (p == x) {
25                // x + 0 = potencia de 2
26                tipo[x] = 'A';
27            }
28            else {
29                int complement = p - x;
30
31                // x y complement tienen tipos opuestos
32                tipo[x] = (tipo[complement] == 'A' ? 'P' : 'A');
33            }
34        }
35
36        while (Q--) {
37            int x;
38            cin >> x;
39            cout << tipo[x];
40        }
41
42        cout << '\n';
43    }
44
45    return 0;
46 }

```

5.2 Ad-hoc - Attention To Meeting

C++\Training_Camp_2026\Day 3A - Attention to Meeting.py

Resuelve "Attention to the Meeting" (Day 3, problema A - Maratona SBC de Programacao 2024): Vinicius quiere saber la duracion maxima que puede tener cada uno de los N discursos de una reunion, sabiendo que todos duran lo mismo, que hay una pausa de 1 minuto entre discursos consecutivos, y que la reunion completa no debe superar K minutos. El problema se reduce a una simple desigualdad lineal: $N*T + (N-1) \leq K$, así que el código despeja T directamente con division entera, $T = (K - (N-1)) // N$, sin busqueda ni iteracion.

```

1 n = int(input())
2 t = int(input())
3
4 print((t - (n-1))/n)

```

5.3 Algebra - The Wisdom Of Master Wei

C++\Training_Camp_2026\Day 10E - The Wisdom of Master Wei.py

Resuelve "The Wisdom of Master Wei" (Day 10, problema E - Contest 09 [Avanzados]): el maestro Wei empezo a programar en el año W y el discípulo Kai en el año K ($W < K$); hay que hallar el año exacto Y en el que la experiencia del maestro ($Y-W$ años) sea el doble de la de Kai ($Y-K$ años). Despeja la ecuacion $Y-W = 2*(Y-K)$, que da $Y = 2K - W$, y la imprime directamente como $m + (m - n)$ usando $n=W$ y $m=K$, sin necesidad de busqueda ni iteracion.

```

1 n,m = list(map(int,input().split()))
2 print(m+(m-n))

```

5.4 Bit Manipulation - Meme Wars

C++ Training Camp_2020/Day 7/10 - Meme Wars.py

Resuelve "Meme Wars" (Day 7, problema G - Contest 07 [Avanzados]): la palabra $S(x)$ se define recursivamente como $S(x) = S(p(x)) + x + S(p(x))$ (con $S('a') = 'a'$, y $p(x)$ la letra anterior), y hay que hallar la letra en la posición n de la enorme palabra $S('z')$. Esta es la clásica "ruler sequence": la letra en la posición n depende únicamente de la mayor potencia de 2 que divide exactamente a n (su valuación 2-adica). El diccionario `dic` mapea cada potencia de 2 a su letra correspondiente, y el bucle recorre las potencias en orden creciente guardando en 'cual' la última potencia que divide a n , hasta encontrar la primera que no lo divide; en ese momento imprime la letra asociada a esa última potencia (equivalente a contar los ceros finales de n en binario).

```
1  dic = {
2      1: "a",
3      2: "b",
4      4: "c",
5      8: "d",
6      16: "e",
7      32: "f",
8      64: "g",
9      128: "h",
10     256: "i",
11     512: "j",
12     1024: "k",
13     2048: "l",
14     4096: "m",
15     8192: "n",
16     16384: "o",
17     32768: "p",
18     65536: "q",
19     131072: "r",
20     262144: "s",
21     524288: "t",
22     1048576: "u",
23     2097152: "v",
24     4194304: "w",
25     8388608: "x",
26     16777216: "y",
27     33554432: "z"
28 }
29 n = int(input())
30 cual = 1
31 for clave, valor in dic.items():
32     if (n % clave == 0):
33         cual = clave
34         continue
35     else:
36         print(dic[cual])
37         break
```

5.5 Brute Force - Antiarithmetic

C++ Training Camp_2020/Day 7/11 - CCPL (Kattis) - Antiarithmetic.cpp

Resuelve "Antiarithmetic?" (Day 11, problema H - CCPL (Kattis)): dada una permutación de $0..n-1$, determina si es antiaritmética, es decir si no existen tres índices $i < j < k$ tales que los valores p_i, p_j, p_k formen una progresión aritmética (ninguna subsecuencia de longitud mayor a 2 puede ser una progresión aritmética). Para cada par de índices (i, k) calcula el valor promedio $v = (p_i + p_k) / 2$ (cuando la suma es par) y usa el arreglo inverso 'pos' (posición de cada valor dentro de la permutación) para ubicar en $O(1)$ donde está v ; si esa posición j queda entre i y k , existe una progresión aritmética y la permutación no es antiaritmética.

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  static char buf_[1 << 22];
5  static int bufLen_, bufPos_;
6  inline int gc() {
7      if (bufPos_ == bufLen_) { bufLen_ = fread(buf_, 1, sizeof(buf_), stdin); bufPos_ = 0; if (!bufLen_) return -1; }
8      return buf_[bufPos_++];
9  }
10 inline bool readInt(int &out) {
11     int c = gc();
12     while (c != -1 && (c < '0' || c > '9') && c != '-') c = gc();
13     if (c == -1) return false;
14     bool neg = false; if (c == '-') { neg = true; c = gc(); }
15     int x = 0;
```

```
6     while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = gc(); }
7     out = neg ? -x : x;
8     return true;
9 }
10 // Reads up to (and past) the colon before the first number is read normally, since ':' is
11 // simply skipped as a non-digit character by readInt's own skip loop.
12
13 bool isAntiarithmetic(int n, const vector<int> &p) {
14     vector<int> pos(n);
15     for (int idx = 0; idx < n; idx++) pos[p[idx]] = idx;
16     for (int i = 0; i < n; i++) {
17         for (int k = i + 1; k < n; k++) {
18             int s = p[i] + p[k];
19             if (s % 2 == 0) {
20                 int v = s / 2;
21                 int j = pos[v];
22                 if (i < j && j < k) return false;
23             }
24         }
25     }
26     return true;
27 }
28
29 int main() {
30     int n;
31     string out;
32     while (readInt(n)) {
33         if (n == 0) break;
34         vector<int> p(n);
35         for (int i = 0; i < n; i++) readInt(p[i]);
36         out += isAntiarithmetic(n, p) ? "yes\n" : "no\n";
37     }
38     fwrite(out.data(), 1, out.size(), stdout);
39     return 0;
40 }
```

5.6 Construction - Ezraft And Array

C++ Codeforces (ezraft and Array.cpp)

Problema de construcción estilo Codeforces: para cada n , imprime un arreglo de n enteros positivos (o -1 si $n=2$, caso imposible). Para $n=1$ usa $[1]$; para $n \geq 3$ arranca con $[1, 2, 3]$ y cada elemento siguiente se hace igual a la suma acumulada de todos los anteriores (duplicándola en cada paso), construyendo una secuencia donde cada nuevo término "rompe" el balance de las sumas de prefijos previas.

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      ios_base::sync_with_stdio(false);
6      cin.tie(NULL);
7
8      int t;
9      cin >> t;
10     while (t--) {
11         int n;
12         cin >> n;
13
14         if (n == 2) {
15             cout << -1 << "\n";
16             continue;
17         }
18
19         vector<long long> a;
20         if (n == 1) {
21             a = {1};
22         } else {
23             a = {1, 2, 3};
24             long long S = 6;
25             for (int i = 4; i <= n; i++) {
26                 a.push_back(S);
27                 S *= 2;
28             }
29         }
30     }
```

```

31     for (size_t i = 0; i < a.size(); i++) {
32         cout << a[i];
33         if (i + 1 != a.size()) cout << ' ';
34     }
35     cout << "\n";
36 }
37 return 0;
38 }

```

5.7 Eliminación Gaussiana (mod p)

C++\Notebook\Eliminacion Gaussiana.cpp

Resuelve sistemas lineales $Ax = b$ sobre Z_p con p PRIMO, que es lo que aparece cuando el problema pide contar soluciones de un sistema, resolver ecuaciones modulares, o sacar el determinante y el rango de una matriz modulo un primo. La única diferencia con el Gauss de toda la vida es que no se puede dividir: para normalizar el pivote se multiplica por su inverso modular (Fermat, $a^{(p-2) \bmod p}$). Como p es primo, TODO elemento distinto de cero tiene inverso, así que no hace falta pivoteo parcial por estabilidad numérica como en double; basta tomar el primer pivote no nulo. Devuelve el rango. Con m incógnitas, si el sistema es compatible hay exactamente $p^{(m - \text{rango})}$ soluciones: rango == m significa solución única, y rango < m deja variables libres (aquí se fijan en 0 para entregar una solución cualquiera). Costo $O(n * m * \min(n, m))$. Si p no es primo la normalización falla; y si el sistema es sobre $GF(2)$, conviene la versión con bitset, que va 64 veces más rápido.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4
5 ll mpow(ll a, ll e, ll p) {
6     ll r = 1; a %= p;
7     while (e) { if (e & 1) r = r * a % p; a = a * a % p; e >>= 1; }
8     return r;
9 }
10
11 ll inverso(ll a, ll p) { return mpow((a % p + p) % p, p - 2, p); }
12
13 // Matriz AUMENTADA de n filas por (m+1) columnas: las m incógnitas y el término
14 // independiente al final. Devuelve el rango; deja una solución en sol.
15 int gauss(vector<vector<ll>> a, int m, vector<ll>& sol, bool& incompatible, ll p) {
16     int n = a.size(), fila = 0;
17     vector<int> donde(m, -1); // donde[col] = fila que tiene el pivote de esa columna
18
19     for (int col = 0; col < m && fila < n; col++) {
20         int piv = -1;
21         for (int i = fila; i < n; i++)
22             if (a[i][col] % p) { piv = i; break; }
23         if (piv < 0) continue; // columna sin pivote: variable libre
24
25         swap(a[piv], a[fila]);
26
27         ll iv = inverso(a[fila][col], p);
28         for (int j = col; j <= m; j++)
29             a[fila][j] = a[fila][j] * iv % p;
30
31         for (int i = 0; i < n; i++)
32             if (i != fila && a[i][col] % p) {
33                 ll f = a[i][col];
34                 for (int j = col; j <= m; j++)
35                     a[i][j] = ((a[i][j] - f * a[fila][j]) % p + p) % p;
36             }
37
38         donde[col] = fila++;
39     }
40
41     sol.assign(m, 0); // las variables libres quedan en 0
42     for (int j = 0; j < m; j++)
43         if (donde[j] >= 0) sol[j] = a[donde[j]][m] % p;
44
45     // Una fila 0 = c con c != 0 significa que no hay solución.
46     incompatible = false;
47     for (int i = fila; i < n; i++) {
48         bool cero = true;
49         for (int j = 0; j < m; j++)
50             if (a[i][j] % p) { cero = false; break; }
51         if (cero && a[i][m] % p) { incompatible = true; break; }
52     }

```

```

53     return fila;
54 }
55
56 // Determinante mod p de una matriz cuadrada. 0 significa que es singular.
57 ll determinante(vector<vector<ll>> a, ll p) {
58     int n = a.size();
59     ll det = 1;
60     for (int c = 0; c < n; c++) {
61         int piv = -1;
62         for (int i = c; i < n; i++)
63             if (a[i][c] % p) { piv = i; break; }
64         if (piv < 0) return 0;
65         if (piv != c) { swap(a[piv], a[c]); det = (p - det) % p; } // cada swap cambia el signo
66
67         det = det * a[c][c] % p;
68         ll iv = inverso(a[c][c], p);
69         for (int i = c + 1; i < n; i++) {
70             ll f = a[i][c] * iv % p;
71             for (int j = c; j < n; j++)
72                 a[i][j] = ((a[i][j] - f * a[c][j]) % p + p) % p;
73         }
74     }
75     return det;
76 }
77
78 int main() {
79     ios::sync_with_stdio(false);
80     cin.tie(nullptr);
81
82     const ll P = 1000000007;
83     int n, m;
84     if (!(cin >> n >> m)) return 0;
85
86     vector<vector<ll>> a(n, vector<ll>(m + 1));
87     for (int i = 0; i < n; i++)
88         for (int j = 0; j <= m; j++) { cin >> a[i][j]; a[i][j] = ((a[i][j] % P) + P) % P; }
89
90     vector<ll> sol;
91     bool incompatible;
92     int rango = gauss(a, m, sol, incompatible, P);
93
94     if (incompatible) {
95         cout << "sin solución\n";
96     } else {
97         cout << "rango " << rango << ", soluciones = P^" << (m - rango) << "\n";
98         for (int j = 0; j < m; j++) cout << sol[j] << " \n"[j == m - 1];
99     }
100     return 0;
101 }

```

5.8 Fibonacci (Fast Doubling) - Fibonacci Sum

C++\Training_Camp_2026\Day 10\M - Fibonacci Sum.cpp

Resuelve "Fibonacci Sum" (Day 10, problema M - Contest 09 [Avanzados]): para T casos con N y M (hasta $1e9$), calcula la suma $F(N) + F(N+1) + \dots + F(M)$ modulo $1e9+7$. Usa la identidad telescópica de que la suma de Fibonacci $\text{sum}_{i=0}^k F(i) = F(k+2) - 1$, por lo que la suma pedida es $F(M+2) - F(N+1) \bmod \text{MOD}$ (visible en la línea `fibmod(b+2) - fibmod(a+1)`). Cada $F(n)$ se calcula en $O(\log n)$ con el algoritmo de duplicación rápida (fast doubling), que recorre los bits de n de mas a menos significativo aplicando las fórmulas $F(2k) = F(k)^2 + F(k+1)^2$ y $F(2k+1) = F(k)^2 + F(k+1)^2$ en aritmética modular.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 typedef long long ll;
5 const ll MOD = 1000000007;
6
7 ll fibmod(ll n, ll mod) {
8     ll a = 0, b = 1, c;
9     for (int i = 63 - __builtin_clzll(n); i >= 0; i--) {
10         c = a;
11         a = (c * (2 * b - c + mod)) % mod;
12         b = (c * c + b * b) % mod;
13
14         if ((n >> i) & 1) {
15             c = (a + b) % mod;

```

```

6         a = b;
7         b = c;
8     }
9 }
20 return a;
21 }
22
23 int main() {
24     ios::sync_with_stdio(false);
25     cin.tie(nullptr);
26
27     int q;
28     cin >> q;
29
30     while (q--) {
31         ll a, b;
32         cin >> a >> b;
33
34         ll ans = (fibmod(b + 2, MOD) - fibmod(a + 1, MOD) + MOD) % MOD;
35         cout << ans << '\n';
36     }
37 }

```

5.9 Fibonacci (Zeckendorf-like Greedy) - Getting In Shape

C++\Training_Camp_2020\Day 4\D - Getting In Shape.cpp

Resuelve "Getting in Shape" (Day 4, problema D - Contest 04 [Avanzados]): dado N , hay que construir una secuencia de ejercicios tipo A/B (terminada en B) tal que la cantidad de formas de completarla saltandose opcionalmente el ejercicio siguiente a cada A sea exactamente N , y devolver la lexicograficamente menor, o "IMPOSSIBLE" si no existe. El numero de formas de un bloque $A^k B$ resulta ser $Fib(k+1)$, y las formas de sufijos se suman; solveSum busca en forma greedy el mayor $Fib(k+2)$ que divida el resto S para minimizar el largo del bloque de A's (dando el prefijo lexicografico mas chico), y solve arma la respuesta de forma recursiva y memoizada (memo) para evitar recomputo exponencial entre llamadas repetidas.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 // fib[i] = Fib(i+1), i.e. fib[0]=Fib(1)=1, fib[1]=Fib(2)=1, fib[2]=Fib(3)=2, ...
5 vector<long long> fib;
6
7 void buildFib() {
8     fib.push_back(1);
9     fib.push_back(1);
10    while (true) {
11        long long a = fib[fib.size() - 1];
12        long long b = fib[fib.size() - 2];
13        if (a > (long long)4e18 - b) break; // stop well before overflow
14        fib.push_back(a + b);
15        if (fib.back() > (long long)4e15) break; // more than enough headroom past 1e15
16    }
17 }
18
19 bool solve(long long N, string &result);
20
21 unordered_map<long long, pair<bool, string>> memo;
22
23 // Find lex-smallest V (ending in B) with h_V(1) + h_V(2) = S.
24 bool solveSum(long long S, string &result) {
25     // Try the largest k such that Fib(k+2) divides S exactly; Fib(k+2) = fib[k+1].
26     for (int k = (int)fib.size() - 2; k >= 1; k--) {
27         long long fibK2 = fib[k + 1]; // Fib(k+2)
28         if (fibK2 > S) continue;
29         if (S % fibK2 == 0) {
30             long long m = S / fibK2;
31             if (m == 1) {
32                 result = string(k - 1, 'A') + "B";
33                 return true;
34             } else {
35                 string sub;
36                 if (solve(m, sub)) {
37                     result = string(k - 1, 'A') + "B" + sub;
38                     return true;
39                 }
40             }
41         }
42     }
43 }

```

```

44 }
45 return false;
46 }
47
48 // Find lex-smallest string (ending in B) with h(1) = N.
49 // Memoized: the same sub-value N can be reached via many different divisor
50 // orderings when the original input has several Fibonacci-related factors,
51 // so without caching this can blow up (verified: >5,000,000 redundant calls
52 // on some inputs without memoization, vs ~1 with it).
53 bool solve(long long N, string &result) {
54     if (N == 1) {
55         result = "B";
56         return true;
57     }
58     auto it = memo.find(N);
59     if (it != memo.end()) {
60         if (it->second.first) result = it->second.second;
61         return it->second.first;
62     }
63     string sub;
64     bool ok = solveSum(N, sub);
65     string ans = ok ? ("A" + sub) : "";
66     memo[N] = {ok, ans};
67     if (ok) result = ans;
68     return ok;
69 }
70
71 int main() {
72     ios_base::sync_with_stdio(false);
73     cin.tie(nullptr);
74     buildFib();
75
76     long long N;
77     cin >> N;
78
79     string result;
80     if (solve(N, result)) {
81         cout << result << "\n";
82     } else {
83         cout << "IMPOSSIBLE\n";
84     }
85     return 0;
86 }

```

5.10 Fibonacci Fast Doubling - Pou

C++\Codeforces\pou.cpp

Calcula el n - simo n mero de Fibonacci modulo 2026 usando exponenciaci n r pida de matriz.

```

1 #include <bits/stdc++.h>
2 typedef long long ll;
3 using namespace std;
4
5 int fibmod(ll n, int m)
6 {
7     int a = 0, b = 1, c;
8     for (int i = 63-__builtin_clzll(n); i >= 0; i--)
9     {
10         c = a;
11         a = (11l*c*(21l*b-c+m)) % m;
12         b = (11l*c*c + 11l*b*b) % m;
13         if ((n>>i) & 1)
14         {
15             c = (a+b) % m; a = b; b = c;
16         }
17     }
18     return a;
19 }
20
21 int main() {
22
23     int n;
24     cin >> n;
25

```

```
26 cout << fibmod(n+1,2026) << endl;
27
28 return 0;
29 }
```

5.11 Greedy - A Healthy Menu

C++[Training_Camp_2026/Day 6/A - A healthy menu.cpp]

Resuelve "A healthy menu" (Day 6, problema A - Maratona SBC de Programacao 2025): un colegio tiene M clases y N frutas, y para cada fruta i y clase j se conoce $G[i][j]$, la cantidad de estudiantes de esa clase a quienes les gusta esa fruta (un estudiante puede gustarle varias frutas). Hay que hallar el menor numero total de estudiantes posible en todo el colegio, consistente con esos conteos. Para cada clase, la cota minima de estudiantes es el mayor $G[i][j]$ entre todas las frutas i (y siempre es alcanzable dejando que un mismo estudiante le guste mas de una fruta). El codigo recorre cada clase i , calcula "maxxx" como el maximo de $G[i][j]$ sobre todas las frutas j , y acumula esos maximos en "maxi", que es la respuesta final.

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 int main()
6 {
7     int n; cin >> n;
8     int m; cin >> m;
9
10    vector<vector<int>> > clases;
11    long long maxi = 0;
12
13    for(int i = 0 ; i < n; i++)
14    {
15        vector<int> pp;
16        for(int j = 0 ; j < m; j++)
17        {
18            int a; cin>>a;
19            pp.push_back(a);
20        }
21        clases.push_back(pp);
22    }
23    for(int i = 0 ; i < n; i++)
24    {
25        long long maxxx = 0;
26        for(int j = 0 ; j < m; j++)
27        {
28            if(clases[j][i] >= maxxx)
29            {
30                maxxx = clases[j][i];
31            }
32        }
33        maxi += maxxx;
34    }
35
36    cout << maxi<< endl;
37
38    return 0;
39 }
```

5.12 Implementation - Hour For A Run

C++[Training_Camp_2026/Day 1/H - Hour for a run.py]

Resuelve "Hour for a Run" (Day 1, problema H - Maratona de Programacao da SBC 2019): Vinicius corre V vueltas a una pista con N carteles igualmente espaciados y quiere saber, para cada decil (10%, 20%, ..., 90% del entrenamiento), cuantos carteles debe contar como minimo para garantizar que ya completo esa fraccion. Calcula $\text{ceil}(V*N*i/10)$ para i de 1 a 9 e imprime los 9 valores.

```
1 import math
2
3 n,c = list(map(int, input().split()))
4
5 mult = c*n
6 res = ""
7 for i in range(1, 10):
8     res += str(math.ceil((mult*i)/10))+ " "
9 print(res)
```

5.13 Logaritmos - Gatuno's Descent Into Psychopathy

C++[Training_Camp_2026/Day 9/Gatuno's Descent into Psychopathy.py]

Resuelve "Gatuno's Descent into Psychopathy" (Day 9, problema F - Contest 08 [Inicial & Avanzado]): el corazon de Gatuno de tamaño $H1$ se reduce con cada mordida segun $H_n = H1 * ((B-1)/B)^n$, donde B es el "factor de brutalidad"; se pide el minimo numero de mordidas n para que H_n sea menor o igual a $H2$. Despeja n algebraicamente tomando logaritmo en base $r = (B-1)/B$ de la razon $H2/H1$ (o/h en el codigo) con $\text{math.log}(o/h, r)$, y redondea hacia arriba con math.ceil para obtener el minimo entero de mordidas que cumple la condicion.

```
1 import math
2 n = int(input())
3
4 for _ in range(n):
5     h, o, r = list(map(int, input().split()))
6
7     r = (r-1)/r
8
9     print(math.ceil(math.log(o/h, r)))
```

5.14 Matrix Exponentiation - Buses

C++[Training_Camp_2026/Day 2/H - Buses.py]

Resuelve "Buses" (Day 2, problema H - Maratona de Programacao da SBC 2013): dada una fila de longitud N metros formada por buses (10 m) y minibuses (5 m), con K colores posibles para minibus y L colores para bus, cuenta de cuantas formas distintas se puede armar la fila, entregando los ultimos 6 digitos del resultado. El numero de formas para una fila de largo 5^m sigue la recurrencia $f(m) = k*f(m-1) + l*f(m-2)$ (agregar al final un minibus de cualquier color o un bus de cualquier color), como un Fibonacci ponderado. Arma la matriz de transicion $M = \begin{bmatrix} k & l \\ l & 1 \end{bmatrix}$ y usa exponenciacion rapida de matrices (mat_pow) para obtener $f(m)$ en $O(\log m)$ modulo 10^6 , ya que N puede ser hasta 10^{15} .

```
1 def mat_mult(A, B, MOD):
2     return [
3         [(A[0][0]*B[0][0] + A[0][1]*B[1][0]) % MOD,
4          (A[0][0]*B[0][1] + A[0][1]*B[1][1]) % MOD],
5         [(A[1][0]*B[0][0] + A[1][1]*B[1][0]) % MOD,
6          (A[1][0]*B[0][1] + A[1][1]*B[1][1]) % MOD],
7     ]
8
9 def mat_pow(M, p, MOD):
10    R = [[1, 0], [0, 1]] # identidad
11    while p > 0:
12        if p & 1:
13            R = mat_mult(R, M, MOD)
14            M = mat_mult(M, M, MOD)
15            p >>= 1
16    return R
17
18 n, k, l = map(int, input().split())
19 MOD = 1000000
20 m = n // 5
21
22 M = [[k % MOD, l % MOD], [l, 0]]
23 P = mat_pow(M, m, MOD)
24 # f(m) = P[1][0]*f(1) + P[1][1]*f(0) = P[1][0]*k + P[1][1]
25 res = (P[1][0] * (k % MOD) + P[1][1]) % MOD
26 print(f"{res:06d}")
```

5.15 Numeros Romanos

C++[Notebook/Numeros Romanos.cpp]

Convierte un entero a numeral romano con un greedy sobre una tabla de valores ordenada de mayor a menor. El truco esta en incluir en la tabla los seis casos sustractivos (CM, CD, XC, XL, IX, IV) como si fueran simbolos propios: asi el greedy de "restar el mayor valor que quepa" nunca se equivoca y no hace falta ningun caso especial. Funciona para 1..3999.

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 string toRoman(int num)
6 {
7     vector<pair<int, string>> roman = {
8         {1000, "M"}, {900, "CM"}, {500, "D"}, {400, "CD"},
9         {100, "C"}, {90, "XC"}, {50, "L"}, {40, "XL"},
10        {10, "X"}, {9, "IX"}, {5, "V"}, {4, "IV"},
11        {1, "I"}
12    };
13 }
```

```

13 string res;
14 for (const auto &p : roman)
15 {
16     while (num >= p.first)
17     {
18         res += p.second;
19         num -= p.first;
20     }
21 }
22 return res;
23 }
24
25 int main()
26 {
27     ios_base::sync_with_stdio(false);
28     cin.tie(nullptr);
29
30     int n;
31     while (cin >> n)
32     {
33         cout << toRoman(n) << "\n";
34     }
35
36     return 0;
37 }

```

5.16 Optimization - Satoru Gojo

C++1(Codeforces)\SatoruGojo.cpp

Problema estilo Codeforces: dado S y M , halla el costo mínimo de escoger k enteros positivos cuyo producto sea al menos S , donde usar más de un factor ($k > 1$) cuesta M por cada factor extra ($(k-1)*M$). Para cada k candidato calcula la suma mínima de k factores con producto $\geq S$ usando la raíz k -ésima entera de S (los factores óptimos son r y $r+1$, ya que por AM-GM la suma se minimiza haciéndolos lo más parejos posible) y prueba todos los k de 1 a 62 para quedarse con el mejor total.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 long long S;
5
6 // Multiply acc by base, but cap at S+1 (i.e. "definitely enough") to avoid overflow;
7 // once acc exceeds S we no longer care about the exact value, only that it's >= S.
8 inline long long capMul(long long acc, long long base) {
9     if (acc > S) return acc; // already known to be enough
10    // check for overflow before multiplying: if acc * base would exceed a safe bound, cap
11    if (base != 0 && acc > (S + 5) / base + 1) {
12        return S + 5; // definitely exceeds S, safe sentinel
13    }
14    long long r = acc * base;
15    if (r > S) r = S + 5; // cap
16    return r;
17 }
18
19 // largest r (>=1) such that r^k <= S
20 long long integerKthRoot(int k) {
21     long long lo = 1, hi = S;
22     while (lo < hi) {
23         long long mid = lo + (hi - lo + 1) / 2;
24         long long val = 1;
25         bool exceeds = false;
26         for (int i = 0; i < k; i++) {
27             val = capMul(val, mid);
28             if (val > S) { exceeds = true; break; }
29         }
30         if (!exceeds) lo = mid;
31         else hi = mid - 1;
32     }
33     return lo;
34 }
35
36 // minimal sum of k positive integers with product >= S
37 long long minSumFork(int k) {
38     long long r = integerKthRoot(k);
39     // try j = 0..k bumps from r to r+1, find smallest j with r^(k-j)*(r+1)^j >= S
40     for (int j = 0; j <= k; j++) {

```

```

41         long long val = 1;
42         for (int t = 0; t < k - j; t++) {
43             val = capMul(val, r);
44         }
45         for (int t = 0; t < j; t++) {
46             val = capMul(val, r + 1);
47         }
48         if (val >= S) {
49             return (long long)k * r + j;
50         }
51     }
52     // fallback (shouldn't happen): all r+1
53     return (long long)k * (r + 1);
54 }
55
56 int main() {
57     long long M;
58     cin >> S >> M;
59
60     long long best = LLONG_MAX;
61     // k up to ~62 is more than enough: 2^62 far exceeds 1e12, and the natural optimum
62     // (ignoring M) is around ln(S) <= ~28 for S<=1e12; checking generously up to 62
63     // safely covers the region where increasing k could still help when M is small.
64     int maxK = 62;
65     for (int k = 1; k <= maxK; k++) {
66         long long sum = minSumFork(k);
67         long long total = sum + (long long)(k - 1) * M;
68         if (total < best) best = total;
69     }
70
71     cout << best << "\n";
72     return 0;
73 }

```

5.17 Polynomial Composition - Coach

C++1(Codeforces)\coach.cpp

Calcula la composición de polinomios $P(Q(x))$ usando multiplicación polinómica modular. El resultado es modular 998244353 y tiene grado máximo $n*m$.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const long long MOD = 998244353;
5
6 // Multiplica dos polinomios A y B
7 // A[i] representa el coeficiente de x^i
8 // B[j] representa el coeficiente de x^j
9 vector<long long> multiply(const vector<long long>& A, const vector<long long>& B) {
10     vector<long long> result(A.size() + B.size() - 1, 0);
11
12     for (int i = 0; i < (int)A.size(); i++) {
13         for (int j = 0; j < (int)B.size(); j++) {
14             // A[i] * x^i multiplicado por B[j] * x^j
15             // queda en el grado i + j
16             result[i + j] += A[i] * B[j];
17             result[i + j] %= MOD;
18         }
19     }
20
21     return result;
22 }
23
24 int main() {
25     ios::sync_with_stdio(false);
26     cin.tie(nullptr);
27
28     int n, m;
29     cin >> n >> m;
30
31     // P tiene grado n, por eso tiene n + 1 coeficientes
32     vector<long long> P(n + 1);
33
34     // Q tiene grado m, por eso tiene m + 1 coeficientes

```

```

35 vector<long long> Q(m + 1);
36
37 for (int i = 0; i <= n; i++) {
38     cin >> P[i];
39 }
40
41 for (int i = 0; i <= m; i++) {
42     cin >> Q[i];
43 }
44
45 // El grado máximo de P(Q(x)) es n * m
46 // Por eso la respuesta tiene n*m + 1 coeficientes
47 vector<long long> ans(n * m + 1, 0);
48
49 // power representa Q^i
50 // Al inicio i = 0, entonces Q^0 = 1
51 vector<long long> power = {1};
52
53 // Calculamos:
54 // P(Q(x)) = p0*Q^0 + p1*Q^1 + p2*Q^2 + ... + pn*Q^n
55 for (int i = 0; i <= n; i++) {
56
57     // Sumamos P[i] * Q^i a la respuesta
58     for (int j = 0; j < (int)power.size(); j++) {
59         ans[j] += P[i] * power[j];
60         ans[j] %= MOD;
61     }
62
63     // Preparamos la siguiente potencia:
64     // si ahora power = Q^i,
65     // después de multiplicar por Q queda power = Q^(i+1)
66     if (i < n) {
67         power = multiply(power, Q);
68     }
69 }
70
71 // Imprimimos los coeficientes r0, r1, ..., r(n*m)
72 for (int i = 0; i <= n * m; i++) {
73     cout << ans[i] << " ";
74 }
75
76 cout << "\n";
77
78 return 0;
79 }

```

5.18 Prefix Sums - Triangles

C++\Training_Camp_2026\Day 2\F - Triangles.cpp

Resuelve "Triangles" (Day 2, problema F - Maratona de Programacao da SBC 2013): dados N puntos sobre un círculo, descritos por las longitudes de arco entre puntos consecutivos, cuenta cuantos triangulos equiláteros distintos se pueden formar usando esos puntos como vertices. Convierte los arcos en posiciones absolutas sobre el círculo con una suma prefija (tot), marcando cada posición ocupada en el arreglo booleano nums[]. Un triangulo equilátero solo puede existir si el perímetro total es múltiplo de 3 ($tri = tot/3$); para cada posición marcada i , verifica si $i+tri$ e $i+2*tri$ también están marcadas, y de ser así cuenta un triangulo y desmarca las tres posiciones para no reutilizarlas.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main()
5 {
6     long long n; cin >> n;
7     long long tot = 0;
8     long long god = 0;
9     vector<long long> nums1;
10    for(long long i = 0 ; i < n ; i ++){
11        {
12            long long a; cin >>a;
13            tot+= a;
14            nums1.push_back(tot);
15        }
16    }
17    vector<long long> nums(1000000,0);
18    for(long long i = 0 ; i < nums1.size() ; i++)
19    {

```

```

8        nums[nums1[i]]=1;
9    }
10
11    if(tot%3!=0)
12    {
13        cout<< 0 <<endl;
14    }
15    else
16    {
17        long long tri = tot/3;
18        for(long long i = 0 ; i < (1000000) ; i++)
19        {
20
21            if(nums[i]==1 && nums[i]==nums[i+tri]&&nums[i]==nums[i+tri+tri]&&i<1000000&&i+tri<1000000&&i+tri+tri<1000000)
22            {
23
24                god++;
25                nums[i]=0;
26                nums[i+tri] = 0;
27                nums[i+tri+tri] = 0;
28            }
29        }
30
31        cout << god << endl;
32
33        return 0;
34    }
35 }

```

5.19 Recurrencia (Fibonacci) - Fractions Are Better When Continued

C++\Training_Camp_2026\Day 3\F - Fractions are better when continued.cpp

Resuelve "Fractions are better when continued" (Day 3, problema F - Maratona SBC de Programacao 2024): dado un entero N , hay que hallar el numerador (en forma irreducible) de la fracción continua p_N definida recursivamente como $p_0 = 1$, $p_1 = 1/(1+1)$, $p_2 = 1/(1+1/(1+1))$, y así sucesivamente, agregando un nivel más de fracción en cada paso. Al expandir la recurrencia, los numeradores de p_N resultan ser exactamente los números de Fibonacci (1, 1, 2, 3, 5, 8, ...). El código aprovecha esto y calcula el resultado con un simple bucle iterativo que va sumando e intercambiando num y dem en cada paso (equivalente a num, dem = dem, num+dem), evitando aritmética de fracciones o recursion explícita.

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 int main()
6 {
7     long long n ; cin >> n;
8
9     if(n==1)
10    {
11        cout<<1<<endl;
12    }
13    else if(n == 2)
14    {
15        cout<<2<<endl;
16    }
17    else
18    {
19        long long num = 1;
20        long long dem = 2;
21        long long temp = 0;
22        long long temp2 = 0;
23        while(n>1)
24        {
25            temp2 = num + dem;
26            num = temp2;
27
28            temp = num;
29            num = dem;
30            dem = temp;
31
32            n--;
33        }
34
35        cout<<num<<endl;

```

```

36     }
37
38     return 0;
39 }

```

5.20 Recurrencias y búsqueda de intervalo factible - Investigating Quadradômeda

C++\Training_Camp_2026\Day 6\ - Investigating Quadradômeda.cpp

Resuelve "Investigating Quadradômeda" (Day 6, problema I - Maratona SBC de Programacao 2025): N estrellas alineadas consecutivamente en horizontal o vertical deben orbitarse con radios enteros $R_i \geq 1$, cada uno estrictamente menor que la distancia d_i a la siguiente estrella, y la condicion de tangencia al pasar de una orbita a la siguiente obliga a que $R_{i+1} = d_i - R_i$ (recurrencia alternante). Hay que hallar el mayor R_1 entero para el cual existe una asignacion valida de todos los R_i (o -1 si no existe ninguna). El codigo acumula C_i via $C_{i+1} = d_i - C_i$ (con $R_i = +-R_1 + C_i$ segun la paridad de i), y va combinando, para cada indice, una cota inferior L (de los i impares, $R_1 \geq 1 - C_i$) o superior U (de los i pares, $R_1 \leq C_i - 1$); si $L \leq U$ al final, la respuesta es U , si no, -1.

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main(){
5      ios_base::sync_with_stdio(false);
6      cin.tie(nullptr);
7
8      int n;
9      cin >> n;
10
11     vector<long long> x(n), y(n);
12     for(int i = 0; i < n; i++) cin >> x[i] >> y[i];
13
14     // R_i = (-1)^(i-1) * R_1 + C_i, with C_1 = 0 and C_{i+1} = d_i - C_i
15     // Need R_i >= 1 for all i:
16     //   i odd -> R_1 >= 1 - C_i   (lower bound)
17     //   i even -> R_1 <= C_i - 1   (upper bound)
18     // Also R_i < d_i for i < n is automatically implied by R_{i+1} >= 1.
19
20     long long C = 0; // C_i, starts at C_1 = 0
21     long long L = LLONG_MIN;
22     long long U = LLONG_MAX;
23
24     for(int i = 1; i <= n; i++){
25         if(i % 2 == 1){
26             // odd index: lower bound constraint
27             L = max(L, 1 - C);
28         } else {
29             // even index: upper bound constraint
30             U = min(U, C - 1);
31         }
32         if(i < n){
33             long long d;
34             if(x[i - 1] == x[i]){
35                 d = llabs(y[i] - y[i - 1]);
36             } else {
37                 d = llabs(x[i] - x[i - 1]);
38             }
39             C = d - C;
40         }
41     }
42
43     if(L > U){
44         cout << -1 << "\n";
45     } else {
46         cout << U << "\n";
47     }
48     return 0;
49 }

```

5.21 Recursion (Simulacion tipo Josephus) - Metal Detector

C++\Training_Camp_2026\Day 5\ - Metal detector.py

Resuelve "Metal detector" (Day 5, problema C - Contest 05 [Avanzados]): en una fila de N personas, un detector defectuoso se activa en forma alternada (la persona 1 no lo activa, la 2 sí, la 3 no, y así sucesivamente), cada persona que lo activa se manda al final de la fila, y para la persona que esta en la posicion i hay que hallar en que turno j finalmente pasara sin activarlo. Simula el proceso por rondas sin construir la fila explicita: la variable "cual" indica si en esta ronda pasan sin activar las posiciones pares o impares, acumula en "llevo" cuantas personas ya pasaron libres, y reduce n y p a la mitad (floor) para continuar con la sub- fila restante hasta que la posicion p caiga del lado que no activa el detector.

```

1  import math
2  t = int(input())
3  for i in range(t):
4      n, p = list(map(int,input().split()))
5      if(p%2 != 0):
6          print(math.ceil(p/2))
7          continue
8      llevo = 0
9      cual = 1
10     q = n
11     while True:
12         if(cual == 1):
13             if(p%2!=0):
14                 print(llevo + math.ceil(p/2))
15                 break
16             a = math.ceil(n/2)
17             llevo += a
18             n = math.floor(n/2)
19             p = math.floor(p/2)
20         else:
21             if(p%2!=1):
22                 print(llevo + math.ceil(p/2))
23                 break
24             a = math.ceil(n/2)
25             llevo += a
26             n = math.floor(n/2)
27             p = math.floor(p/2)
28
29     if(q%2==0):
30         cual = 1
31     else:
32         cual = 2
33
34     q = n

```

5.22 Simulacion - Cahokia Ruins

C++\Training_Camp_2026\Day 5\A - Cahokia ruins.py

Resuelve "Cahokia ruins" (Day 5, problema A - Contest 05 [Avanzados]): al entrar a una sala rectangular $H \times W$, las paredes este y oeste empiezan a moverse una hacia la otra a 1 m/s cada una; cada fila i tiene un ancho de pared (arreglos `left` y `right` en el codigo) medido desde su borde respectivo, y hay que calcular en cuanto tiempo chocan las paredes por primera vez. Para cada fila calcula el hueco libre restante, $w - \text{right}[i] - \text{left}[i]$, y se queda con el minimo (variable `mins`), ya que esa fila es la que se cierra primero; como ambas paredes se acercan a 1 m/s, el tiempo de colision es ese hueco minimo dividido entre 2 (velocidad de cierre combinada).

```

1  h, w = list(map(int,input().split()))
2
3  left = list(map(int,input().split()))
4  right = list(map(int,input().split()))
5
6  mins = float("inf")
7  for i in range(len(right)):
8      x = w - right[i]
9      mins = min(mins, x-left[i])
10
11  if mins // 2 == mins / 2:
12      print(int(mins/2))
13  else:
14      print(mins/2)

```

6 - Number Theory

6.1 BFS sobre Residuos - Only1s0s

C++\ICPC\2024\H - Only1s0s.cpp

Resuelve "Only1s0s" (problema H, ICPC 2024). El código busca el menor múltiplo M de N que se escribe solo con unos y ceros, y responde M / N . M puede tener muchísimos dígitos, así que no se busca entre números sino entre RESIDUOS módulo N : dos números con el mismo residuo se comportan igual de ahí en adelante (agregarles un dígito lleva al mismo residuo), así que basta quedarse con el primero que llega a cada residuo. Hay a lo sumo N estados, y el BFS termina en $O(N)$. Siempre existe: entre 1, 11, 111, ... $(N+1)$ números dos tienen el mismo residuo, y su resta es un múltiplo de N hecho de unos y ceros. POR QUE SALE EL MENOR: el BFS encuentra primero los mas cortos, y dentro del mismo largo, como se agrega el 0 antes que el 1 y la cola ya viene ordenada, los números salen en orden creciente. Así que la primera vez que se alcanza el residuo 0 es con el menor M . Se reconstruye con `parent[]` y `digit[]`. Al final M se divide entre N como en la escuela, dígito por dígito con el resto, porque no cabe en ningún tipo entero. El 10LL en la transición evita el desborde de `cur * 10`.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 string solve(int N) {
5
6     vector<int> parent(N, -1);
7     vector<char> digit(N);
8     vector<bool> visited(N, false);
9
10    queue<int> q;
11
12    // Empezamos con el numero 1
13    int r = 1 % N;
14
15    visited[r] = true;
16    digit[r] = '1';
17
18    q.push(r);
19
20    while (!q.empty()) {
21
22        int cur = q.front();
23        q.pop();
24
25        // Encontramos un multiplo de N
26        if (cur == 0)
27            break;
28
29        // Agregar 0
30        int r0 = (cur * 10LL) % N;
31
32        if (!visited[r0]) {
33
34            visited[r0] = true;
35            parent[r0] = cur;
36            digit[r0] = '0';
37
38            q.push(r0);
39        }
40
41        // Agregar 1
42        int r1 = (cur * 10LL + 1) % N;
43
44        if (!visited[r1]) {
45
46            visited[r1] = true;
47            parent[r1] = cur;
48            digit[r1] = '1';
49
50            q.push(r1);
51        }
52    }
53
54    // Reconstruir M
55    string M;
56
57    int cur = 0;
58
59    while (cur != -1) {
60        M.push_back(digit[cur]);
```

```
61        cur = parent[cur];
62    }
63
64    reverse(M.begin(), M.end());
65
66    return M;
67 }
68
69 // Divide un numero gigante representado como string
70 // entre N.
71 string divide(string M, int N) {
72
73     string ans;
74     long long rem = 0;
75
76     for (char c : M) {
77
78         rem = rem * 10 + (c - '0');
79
80         ans += char('0' + rem / N);
81
82         rem %= N;
83     }
84
85     // Quitar ceros iniciales
86     int pos = 0;
87
88     while (pos + 1 < (int)ans.size() &&
89            ans[pos] == '0') {
90         pos++;
91     }
92
93     return ans.substr(pos);
94 }
95
96
97 int main() {
98
99     ios::sync_with_stdio(false);
100    cin.tie(nullptr);
101
102    int N;
103
104    while (cin >> N && N != 0) {
105
106        string M = solve(N);
107
108        string D = divide(M, N);
109
110        cout << D << '\n';
111    }
112
113    return 0;
114 }
115 }
```

6.2 Big Exponentiation - Uj (cpp)

C++\CCP\3.cpp

Calcula p^q usando aritmética de precisión arbitraria (`boost::multiprecision`) para manejar resultados con exponentes muy grandes.

```
1 #include <bits/stdc++.h>
2 #include <boost/multiprecision/cpp_int.hpp>
3
4 using namespace std;
5 using namespace boost::multiprecision;
6
7 int main() {
8     while (true) {
9         long long p, q;
10        cin >> p >> q;
11
12        if (p == 0 && q == 0) return 0;
```

```

3
4     cpp_int b = 1;
5
6     for (long long i = 0; i < q; i++) {
7         b *= p;
8     }
9
10    cout << b << "\n";
11    }
12 }

```

6.3 Big Exponentiation - Uj (java)

JAVA/CCPL/uj.java

Calcula N^D usando BigInteger para manejar resultados con exponentes muy grandes.

```

1 import java.util.*;
2 import java.math.*;
3
4 class Main {
5     public static void main (String[] args) throws Exception {
6         Scanner sc = new Scanner(System.in);
7
8         while (sc.hasNext()) {
9             int N = sc.nextInt();
10            int D = sc.nextInt();
11
12            if (N == 0 && D == 0) break;
13
14            System.out.println(BigInteger.valueOf(N).pow(D));
15        }
16    }
17 }

```

6.4 Criba de Eratostenes (Solo Impares)

C++/Notebook/Criba de Eratostenes (Solo Impares).cpp

La criba que uno debería escribir por defecto: como el 2 es el único primo par, no vale la pena reservar ni recorrer las posiciones pares. El arreglo solo guarda los impares, con el índice k representando al número $2k+1$, así que ocupa la MITAD de memoria y hace la mitad del trabajo. Medido en esta máquina, mediana de 3 corridas, contra la criba normal de n bits: $n=10^7$ $n=3 \cdot 10^7$ $n=10^8$ Criba clásica (n bits) 30 ms 98 ms 427 ms Esta (solo impares) 15 ms 46 ms 170 ms Dos veces y media mas rápida en 10^8 , y con la mitad de RAM. La ganancia no es solo por hacer menos operaciones: al ocupar la mitad, el arreglo cabe mejor en cache, y una criba es un recorrido de memoria casi puro. LAS DOS CONVERSIONES QUE HAY QUE TENER CLARAS, que es donde se equivoca uno: valor $2k+1 \rightarrow$ índice $k = (\text{valor} - 1) / 2$ se empieza a marcar en $p \cdot p$, que es impar, y su índice es $(p \cdot p - 1) / 2$ se avanza de p en p SOBRE LOS INDICES, no de $2p$ en $2p$ sobre los valores. Es lo mismo: subir un índice p equivale a subir $2p$ en el valor, que es justo saltarse el múltiplo par de p . Ejemplo para $p = 3$: se arranca en el índice $(9-1)/2 = 4$, que es el valor 9. El siguiente índice es $4+3 = 7$, o sea el valor 15. Se salta el 12 solito, porque es par y no está en el arreglo. Empezar en $p \cdot p$ y no en $2p$ es lo de siempre: todo múltiplo de p menor que $p \cdot p$ ya lo marco un primo mas chico. Y el bucle interno se corta por sí mismo cuando $(p \cdot p - 1) / 2 > m$, así que no hace falta ningún if extra ni una raíz cuadrada. Costo $O(n \log \log n)$ en tiempo y $n/2$ bits de memoria. Con $n = 10^8$ son unos 6 MB. Verificado contra la criba clásica: lista de primos idéntica para todo n hasta 200000, y el mismo conteo en 10^6 , 10^7 , $3 \cdot 10^7$ y 10^8 (664579 primos hasta 10^7 , 5761455 hasta 10^8). CUANDO USAR ESTA Y CUANDO OTRA COSA: - Hace falta la lista de primos, o saber si un número es primo, hasta 10^7 o 10^8 : esta. - Hace falta el MENOR FACTOR PRIMO de cada número para factorizar rápido: criba de spf, que es el mismo costo pero guarda un int por posición en vez de un bit (y ahí sí conviene el arreglo completo, porque hay que indexar por el número). - Un solo número hasta 10^{12} y solo saber si es primo: no se criba, se prueba dividir hasta la raíz. Mas grande que eso, Miller-Rabin. - El rango es $[a,b]$ con b hasta 10^{12} pero $b-a$ chico: criba segmentada, cribando primero hasta $\sqrt{b}$ con esta misma y después marcando el segmento. OJO: vector<bool> es un bitset empaquetado, y eso es lo que da los $n/2$ bits. Si se cambia por vector<char> el consumo se multiplica por 8, aunque a veces corra mas rápido por no desempacar bits. Con $n = 10^8$ la diferencia es 6 MB contra 50 MB, así que suele no haber opción.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 // Todos los primos <= n, en orden.
5 vector<int> criba(int n) {
6     if (n < 2) return {};
7
8     int m = (n - 1) / 2; // el índice k es el impar 2k+1
9     vector<bool> compuesto(m + 1, false);
10
11     vector<int> primos;
12     primos.push_back(2); // el unico par, va aparte
13 }

```

```

14     for (int k = 1; k <= m; k++) {
15         if (compuesto[k]) continue;
16
17         long long p = 2LL * k + 1; // long long: p*p se pasa de int
18         primos.push_back((int)p);
19
20         for (long long j = (p * p - 1) / 2; j <= m; j += p)
21             compuesto[j] = true;
22     }
23
24     return primos;
25 }
26
27 // Version que deja la tabla para consultar despues si x es primo en O(1).
28 // esPrimo(x) = x == 2, o x impar con !compuesto[(x-1)/2].
29
30 int main() {
31     ios::sync_with_stdio(false);
32     cin.tie(nullptr);
33
34     int n;
35     if (!cin >> n) return 0;
36
37     vector<int> primos = criba(n);
38
39     cout << primos.size() << " primos hasta " << n << '\n';
40     for (int p : primos) cout << p << ' ';
41     cout << '\n';
42     return 0;
43 }

```

6.5 Criba de Factor Primo Mínimo + Inclusion-Exclusion - Ingredients That May Harm You

C++/Training_Camp_2026/Day-3/ - Ingredients that may Harm You.cpp

Resuelve "Ingredients that may Harm You" (Day 3, problema I - Maratona SBC de Programacao 2024): en un restaurante self-service con N platillos identificados por numeros (donde cada primo que los divide representa un ingrediente basico, con multiplicidad), cada uno de Q comensales tiene un numero X que identifica sus alergias (los primos que dividen a X); para cada comensal hay que contar cuantos subconjuntos de platillos se pueden combinar sin que ninguno de ellos contenga un ingrediente al que sea alergico. El codigo precalcula el menor factor primo (spf) hasta $1e6$ con una criba, para factorizar rapido cualquier numero, y para cada platillo enumera por mascara de bits (mask) todos los divisores libres de cuadrados formados por sus primos, acumulando cnt[d]. Para cada consulta factoriza X y aplica inclusion-exclusion sobre los divisores de X (sumando o restando cnt[d] segun la paridad de bits del mask) para contar cuantos platillos son coprimos con X ; la respuesta final es $2^{(\text{coprimos}) \bmod 1e9+7}$, ya que cada platillo compatible puede incluirse o no libremente en el plato.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const int MAXV = 1000000;
5 const int MOD = 1000000007;
6
7 int spf[MAXV + 1];
8 int cnt[MAXV + 1];
9 int pw[100005];
10
11 int main() {
12     ios::sync_with_stdio(false);
13     cin.tie(nullptr);
14
15     // SPF
16     for (int i = 0; i <= MAXV; i++) spf[i] = i;
17     for (int i = 2; i * i <= MAXV; i++) {
18         if (spf[i] == i) {
19             for (int j = i * i; j <= MAXV; j += i)
20                 if (spf[j] == j) spf[j] = i;
21         }
22     }
23
24     int N;
25     cin >> N;
26
27     pw[0] = 1;
28     for (int i = 1; i <= N; i++)
29         pw[i] = (2LL * pw[i - 1]) % MOD;

```

```

30
31 while (N--) {
32     int x;
33     cin >> x;
34
35     vector<int> p;
36     while (x > 1) {
37         int pr = spf[x];
38         p.push_back(pr);
39         while (x % pr == 0) x /= pr;
40     }
41
42     int m = p.size();
43
44     for (int mask = 0; mask < (1 << m); mask++) {
45         int d = 1;
46         for (int i = 0; i < m; i++)
47             if (mask & (1 << i))
48                 d *= p[i];
49         cnt[d]++;
50     }
51 }
52
53 int Q;
54 cin >> Q;
55
56 while (Q--) {
57     int x;
58     cin >> x;
59
60     vector<int> p;
61     while (x > 1) {
62         int pr = spf[x];
63         p.push_back(pr);
64         while (x % pr == 0) x /= pr;
65     }
66
67     int m = p.size();
68
69     int coprime = 0;
70
71     for (int mask = 0; mask < (1 << m); mask++) {
72         int d = 1;
73         int bits = 0;
74
75         for (int i = 0; i < m; i++) {
76             if (mask & (1 << i)) {
77                 d *= p[i];
78                 bits++;
79             }
80         }
81
82         if (bits & 1)
83             coprime -= cnt[d];
84         else
85             coprime += cnt[d];
86     }
87
88     cout << pw[coprime] << '\n';
89 }
90
91 return 0;
92 }

```

6.6 Diferencia de Cuadrados - Square Diff

C++(ICPC2024) - SquareDiff.cpp

Resuelve "SquareDiff" (problema C, ICPC 2024). El código responde si n se puede escribir como diferencia de dos cuadrados, y toda la solución es una condición: $n = a^2 - b^2$ tiene solución $\Leftrightarrow n \bmod 4 \neq 2$. Por que: $a^2 - b^2 = (a - b)(a + b)$, y esos dos factores tienen la MISMA paridad (se diferencian en 2b). Dos impares dan un producto impar, y dos pares dan un múltiplo de 4; nunca sale algo que sea par pero no múltiplo de 4. Y al revés siempre se puede: un impar $2k+1 = (k+1)^2 - k^2$, y un múltiplo de 4, $4m = (m+1)^2 - (m-1)^2$. Verificado por fuerza bruta para n de 1 a 3000. El reflejo que deja este problema: ante "se puede escribir como...", factorizar la expresión y mirar la PARIDAD de los factores suele matarlo en una línea. OJO si n pudiera ser negativo: en C++ $n \% 4$ conserva el signo, así que $-2 \% 4$ da -2 y no 2 .

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int n;
9
10    while (cin >> n && n != 0) {
11        cout << (n % 4 == 2 ? 'N' : 'Y') << '\n';
12    }
13
14    return 0;
15 }

```

6.7 Divisibilidad - Archmage's Crystals

C++(Training_Camp_2026)Day 9/A - Archmage's Crystals.cpp

Resuelve "Archmage's Crystals" (Day 9, problema A - Contest 08 [Inicial & Avanzado]): un archimago tiene n cristales con potencias a_i ; cada hechizo de fusión toma dos cristales x, y y los reemplaza por uno de poder $x+y$, reduciendo la cantidad de cristales en 1. Se busca el mínimo número de hechizos para que el promedio de los cristales restantes sea un entero. La suma total nunca cambia con las fusiones, así que solo hace falta hallar el mayor divisor i de la suma con $i \leq n$ (recorriendo i desde n hacia 1 con la variable c contando los intentos fallidos); la respuesta es $n - i$, es decir, la cantidad de fusiones necesarias para llegar a ese divisor.

```

1 n = int(input())
2 l = list(map(int, input().split()))
3
4 suma = sum(l)
5 c = 0
6 for i in range(n, 0, -1):
7     if (suma % i == 0):
8         break
9     c += 1
10 print(c)

```

6.8 Divisors - Aesthetics In Poetry

C++(Training_Camp_2026)Day 7/A - Aesthetics in poetry.cpp

Resuelve "Aesthetics in poetry" (Day 7, problema A - Contest 07 [Avanzado]): dado un poema de N versos con longitudes $l_1 \dots l_N$, hay que hallar el menor entero $K > 1$ (K -elegante) tal que N sea múltiplo de K y, para cada resto $i=0..K-1$ al dividir las longitudes por K , haya exactamente N/K versos con ese resto (o -1 si no existe tal K). Genera todos los divisores de N (función divisores), los ordena ascendentemente y para cada candidato $k \geq 2$ cuenta en $\text{cnt}[v \% k]$ cuantos versos caen en cada resto, verificando que todos los conteos sean iguales a n/k ; el primer k que cumple es la respuesta mínima.

```

1 #include <bits/stdc++.h>
2 typedef long long ll;
3 using namespace std;
4
5 vector<ll> divisores(ll n)
6 {
7     vector<ll> div;
8     for (ll i = 1; i * i <= n; i++)
9     {
10        if (n % i == 0)
11        {
12            div.push_back(i);
13            if (i != n/i) div.push_back(n/i);
14        }
15    }
16    return div;
17 }

```

```

8
9 int main()
20 {
21     ll n; cin >> n;
22     vector<ll> xd(n);
23     for(int i = 0; i < n; i++)
24     {
25         cin >> xd[i];
26     }
27
28     vector<ll> juan = divisores(n);
29     sort(juan.begin(), juan.end());
30
31     for(ll k : juan)
32     {
33         if(k >= 2)
34         {
35
36             vector<ll> cont(k, 0);
37             for(ll v : xd)
38             {
39                 cont[v % k]++;
40             }
41
42             bool ok = true;
43             for(ll c : cont)
44             {
45                 if(c != n/k) { ok = false; break; }
46             }
47
48             if(ok)
49             {
50                 cout << k << endl;
51                 return 0;
52             }
53         }
54     }
55
56     cout << -1 << endl;
57     return 0;
58 }

```

6.9 Exponenciación Modular (Torre de Exponentes) - Exponentiation II

C++ Training Camp 2018 Day 10 K - Exponentiation II.cpp

Resuelve "Exponentiation II" (Day 10, problema K - Contest 09 [Avanzados]): calcula a^{b^c} modulo $1e9+7$ para n consultas, con a, b, c hasta $1e9$, donde el exponente b^c es demasiado grande para calcularlo directamente. Usa el Pequeño Teorema de Fermat: como MOD es primo, $a^{MOD-1} = 1$ (para a no múltiplo de MOD), así que el exponente de la torre puede reducirse modulo MOD-1 antes de aplicarlo; el código llama `expmod(b, c, MOD-1)` para obtener ese exponente reducido y luego `expmod(a, ese_valor, MOD)` para el resultado final, reutilizando la misma exponenciación binaria (fast power) para ambos niveles.

```

1 #include <iostream>
2 using namespace std;
3 typedef long long ll;
4 ll MOD = 1000000007;
5
6 ll expmod(ll b, ll e, ll m)
7 {
8     ll ans = 1;
9     while (e) {
10         if (e&1) ans = (1ll*ans*b) % m;
11         b = (1ll*b*b) % m;
12         e /= 2;
13     }
14     return ans;
15 }
16
17
18 int main()
19 {
20     ll n; cin >> n;
21
22     while(n--)
23     {

```

```

24         ll a, b, c; cin >> a >> b >> c;
25
26         cout << expmod(a, expmod(b, c, MOD-1), MOD) << endl;
27     }
28
29     return 0;
30 }

```

6.10 Factorización en Dígitos

C++ Notebook Factorization en Digits.cpp

Halla el menor número cuyos dígitos multiplicados dan N . La idea greedy: dividir siempre por el mayor factor posible entre 9 y 2, porque usar factores grandes minimiza la cantidad de dígitos, y al final ordenar los dígitos de menor a mayor da el número más chico posible con esa multiset de dígitos. Si al terminar el producto de los dígitos no coincide con N , es que N tiene un factor primo mayor que 9 (11, 13, ...) y la respuesta es -1. Casos borde: $N < 2$ se responde con el propio N .

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 string recursive(long long num, string sol = "")
6 {
7     for (int i = 9; i > 1; --i)
8     {
9         if (num % i == 0)
10         {
11             return recursive(num / i, sol + to_string(i));
12         }
13     }
14     return sol;
15 }
16
17 int main()
18 {
19     ios::sync_with_stdio(false);
20     cin.tie(nullptr);
21
22     int t;
23     cin >> t;
24     while (t--)
25     {
26         long long n;
27         cin >> n;
28
29         if (n < 2)
30         {
31             cout << n << "\n";
32             continue;
33         }
34
35         string factors = recursive(n);
36
37         long long mult = 1;
38         for (char ch : factors)
39         {
40             mult *= (ch - '0');
41         }
42
43         if (mult == n)
44         {
45             sort(factors.begin(), factors.end());
46             cout << factors << "\n";
47         }
48         else
49         {
50             cout << -1 << "\n";
51         }
52     }
53
54     return 0;
55 }

```

6.11 GCD and LCM - Journeygrapes

C++(Codeforces)journeygrapes.cpp

Calcula a dividido entre mcd(a,b), que es el multiplicador para obtener lcm(a,b) desde b.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     long long a, b;
6     cin >> a >> b;
7     //c++17
8     cout << a / gcd(a, b) << endl;
9
10    return 0;
11 }
```

6.12 Modular Arithmetic - Broul

C++(CCPL)broul.cpp

Calcula el número mínimo de saltos de tamaño $3*b$ necesarios para cubrir la distancia vertical $abs(a-c)$, o determina si no es posible porque la distancia no es múltiplo de b .

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 int main() {
6     while(true){
7         int a,b,c;
8         cin >> a >> b >> c;
9         if(a==0 && b ==0 && c ==0)
10            {
11                return 0;
12            }
13
14        int god = abs(a-c);
15        if(god%b!=0)
16            {
17                cout << "No accounting tablet" << "\n";
18            }
19        else
20            {
21                if((god/b) % 3 ==0)
22                {
23                    cout << (god/b)/3<< "\n";
24                }
25                else
26                {
27
28                    cout << ((god/b)/3)+1<< "\n";
29                }
30            }
31
32
33    }
34
35
36
37
38    return 0;
39 }
```

6.13 Modular Arithmetic - Creating Multiples

C++(Training_Camp_2026)Day 4/8 - Creating Multiples.cpp

Resuelve "Creating Multiples" (Day 4, problema B - Contest 04 [Avanzados]): dado un número N de L dígitos en base B , hay que reducir (nunca aumentar) como mucho un dígito para que el número resultante M sea múltiplo de $B+1$, eligiendo la solución que minimice M . Usa que B es congruente con -1 módulo $B+1$, por lo que el resto de N módulo $B+1$ es la suma alternada de sus dígitos (variable rem); recorre los dígitos de izquierda a derecha calculando cuanto hay que restarle a cada uno (δ) para anular ese resto, y toma la primera posición válida, que por ser la más significativa da el M más chico posible.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
```

```
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int B, L;
9     cin >> B >> L;
10
11    vector<int> D(L);
12    for (int &x : D) cin >> x;
13
14    int mod = B + 1;
15
16    // Calculate N modulo (B + 1).
17    // B ≡ -1 (mod B+1), so powers alternate.
18    int rem = 0;
19    for (int i = 0; i < L; i++) {
20        if ((L - 1 - i) % 2 == 0)
21            rem += D[i];
22        else
23            rem -= D[i];
24        rem %= mod;
25    }
26    if (rem < 0) rem += mod;
27
28    if (rem == 0) {
29        cout << "0\n";
30        return 0;
31    }
32
33    // Try positions from left to right. The first possible position
34    // gives the smallest resulting number.
35    for (int i = 0; i < L; i++) {
36        int sign = ((L - 1 - i) % 2 == 0) ? 1 : -1;
37
38        // Need a decrease delta such that:
39        // rem - delta * sign ≡ 0 (mod B+1)
40        int delta;
41        if (sign == 1)
42            delta = rem;
43        else
44            delta = (mod - rem) % mod;
45
46        // Digits are smaller than B+1, so this is the only possible delta.
47        if (delta <= D[i] && delta > 0) {
48            cout << i + 1 << ' ' << D[i] - delta << '\n';
49            return 0;
50        }
51    }
52
53    cout << "-1\n";
54    return 0;
55 }
```

6.14 Modulo de Entero Gigante

C++(Notebook)Modulo de Entero Gigante.cpp

Cuando el número de entrada tiene miles de dígitos no cabe en `long long`, así que se lee como `string` y se calcula el resto dígito por dígito con la regla de Horner: se arrastra el resto parcial y en cada paso se hace $rem = (rem * 10 + \text{dígito}) \% N$. Como rem siempre es menor que N , el producto $rem * 10$ nunca desborda mientras N quepa cómodo en `long long`. Sirve para problemas de divisibilidad con números gigantes, o como paso previo antes de aplicar exponenciación modular sobre un exponente enorme.

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 long long modString(const string& P, long long N)
6 {
7     long long rem = 0;
8     for (char c : P)
9     {
10        int d = c - '0';
11        rem = (rem * 10 + d) % N;
12    }
13    return rem;
14 }
```

```

6
7 int main()
8 {
9     ios_base::sync_with_stdio(false);
10    cin.tie(nullptr);
11
12    string P;
13    long long N;
14    while (cin >> P >> N)
15    {
16        cout << modString(P, N) << "\n";
17    }
18
19    return 0;
20 }

```

6.15 Modulo de un Binario Gigante - Binary Dozens

C++(ICPC2025)B - Binary Dozens.cpp

Resuelve "Binary Dozens" (problema B, ICPC 2025): dado un numero en binario de hasta 500 bits, dar su resto al dividir por 12. 500 bits son numeros de hasta 10^{150} , asi que no hay tipo entero que los guarde. Pero el resto si se puede calcular leyendo los digitos de izquierda a derecha con el metodo de Horner: si el numero visto hasta ahora vale v , agregarle un bit b lo convierte en $2v + b$, y tomando modulo en cada paso el acumulador nunca pasa de 24, $rem = (rem * 2 + bit) \% 12$ Funciona porque el modulo es compatible con sumas y productos: se puede reducir en cada paso en vez de al final. La misma idea sirve para cualquier base y cualquier modulo, no solo binario y 12, y es la forma estandar de sacar el resto de un numero que llega como cadena. Que 12 no sea primo da igual: esto no usa inversos ni nada por el estilo. Verificado contra los enteros exactos de Python en 3004 cadenas, incluidas las de 500 bits todos en 1 y todos en 0. El archivo B - Binary Dozens.py de esta carpeta hace lo mismo aprovechando que Python tiene enteros de precision arbitraria: `int(b, 2) \% 12`. Vale tenerlo presente cuando el problema sea justamente de numeros gigantes y el lenguaje este permitido.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     string b;
6
7     while (cin >> b && b != "") {
8
9         int rem = 0;
10
11         for (char c : b) {
12             rem = (rem * 2 + (c - '0')) % 12;
13         }
14
15         cout << rem << '\n';
16     }
17
18     return 0;
19 }

```

6.16 Number Theory (Criba y Factorizacion) - Counting Divisors

C++(Training_Camp_2026)Day10\ - Counting Divisors.cpp

Resuelve "Counting Divisors" (Day 10, problema I - Contest 09 [Avanzados]): dados n enteros, imprime la cantidad de divisores de cada uno. Precomputa con una criba de Eratosthenes (sieve) la lista de primos hasta $MX=1e6$; para cada consulta k , la funcion `divisors` factoriza k probando solo los primos precomputados (hasta $\sqrt{k}$) y acumula el conteo de divisores con la formula producto de $(exponente_i + 1)$ para cada primo, multiplicando por 2 al final si queda un factor primo residual mayor a 1 (el propio n si es primo grande).

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 const int MX = 1000000;
7
8 bool marked[MX + 1];
9 vector<int> primes;
10
11 void sieve() {
12     marked[0] = marked[1] = true;
13
14     for (int i = 2; i <= MX; i++) {
15         if (marked[i]) continue;

```

```

16         primes.push_back(i);
17
18         if (1LL * i * i > MX) continue;
19         for (ll j = 1LL * i * i; j <= MX; j += i)
20             marked[j] = true;
21     }
22 }
23
24 ll divisors(ll n) {
25     ll ans = 1;
26
27     for (ll p : primes) {
28         if (p * p > n) break;
29
30         if (n % p == 0) {
31             int cnt = 0;
32             while (n % p == 0) {
33                 n /= p;
34                 cnt++;
35             }
36             ans *= (cnt + 1);
37         }
38     }
39
40     if (n > 1) ans *= 2;
41
42     return ans;
43 }
44
45 int main() {
46     ios::sync_with_stdio(false);
47     cin.tie(nullptr);
48
49     sieve();
50
51     int t;
52     cin >> t;
53
54     while (t--) {
55         ll k;
56         cin >> k;
57         cout << divisors(k) << '\n';
58     }
59
60     return 0;
61 }

```

6.17 Number Theory (GCD y Divisores) - The Seals Of Shanghai

C++(Training_Camp_2026)Day10\ - The Seals of Shanghai.cpp

Resuelve "The Seals of Shanghai" (Day 10, problema B - Contest 09 [Avanzados]): eligiendo un modulo fijo $M > 1$, cada sello con numero A_i se desbloquea en la ronda de su residuo $A_i \bmod M$; hay que hallar el minimo numero de residuos distintos (rondas) necesarias y cuantos M distintos logran ese minimo. Usa que dos elementos caen en el mismo residuo sii M divide su diferencia: calcula $D = \gcd$ de todas las diferencias contra $A[0]$; si $D=0$ el costo minimo es 1 para todo M ($MAXM-1$ valores); si $D \geq 2$ el costo minimo es 1 y la cuenta es el numero de divisores de D mayores a 1 (`countDivisorsGt1`); si $D=1$ el minimo es 2, y por pigeonhole sobre un triple de valores distintos, todo M valido debe dividir alguna de sus 3 diferencias, por lo que basta probar los divisores de esas 3 diferencias (candidates) y, para cada uno, contar residuos distintos con una tabla hash reutilizable.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4 typedef unsigned long long ull;
5
6 static char buf_[1 << 22];
7 static int buflen_, bufPos_;
8 inline int gc() {
9     if (bufPos_ == buflen_) { buflen_ = fread(buf_, 1, sizeof(buf_), stdin); bufPos_ = 0; if (!buflen_) return -1; }
10    return buf_[bufPos_++];
11 }
12 inline ll readLL() {
13     int c = gc();

```

```

14 while (c != -1 && (c < '0' || c > '9') && c != '-') c = gc();
15 bool neg = false; if (c == '-') { neg = true; c = gc(); }
16 ll x = 0;
17 while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = gc(); }
18 return neg ? -x : x;
19 }
20
21 const ll MAXM = 1000000000LL;
22
23 ll gcdll(ll a, ll b) { while (b) { a %= b; swap(a, b); } return a; }
24
25 // Counts divisors of d that are > 1.
26 ll countDivisorsGt1(ll d) {
27     if (d <= 1) return 0;
28     ll cnt = 0;
29     for (ll i = 1; i * i <= d; i++) {
30         if (d % i == 0) {
31             if (i > 1) cnt++;
32             ll other = d / i;
33             if (other != i && other > 1) cnt++;
34         }
35     }
36     return cnt;
37 }
38
39 void collectDivisorsGt1(ll d, vector<ll> &out) {
40     if (d <= 1) return;
41     for (ll i = 1; i * i <= d; i++) {
42         if (d % i == 0) {
43             if (i > 1) out.push_back(i);
44             ll other = d / i;
45             if (other != i && other > 1) out.push_back(other);
46         }
47     }
48 }
49
50 int main() {
51     int N = (int)readLL();
52     vector<ll> A(N);
53     for (auto &x : A) x = readLL();
54
55     if (N == 1) {
56         printf("1 %lld\n", MAXM - 1);
57         return 0;
58     }
59
60     ll D = 0;
61     for (int i = 1; i < N; i++) D = gcdll(D, llabs(A[i] - A[0]));
62
63     if (D == 0) {
64         printf("1 %lld\n", MAXM - 1);
65         return 0;
66     }
67     if (D >= 2) {
68         printf("1 %lld\n", countDivisorsGt1(D));
69         return 0;
70     }
71
72     // D == 1: minimum is provably exactly 2 (M=2 always splits into exactly the two parity
73     // classes, and D=1 rules out cost=1 for any M>1). We still need to COUNT every M that
74     // achieves cost=2. Key fact used for counting: for ANY 3 elements, since there are only
75     // 2 residue classes under an optimal M, by pigeonhole at least 2 of the 3 must share a
76     // class, so M must divide at least one of their 3 pairwise differences. This holds for
77     // every valid M and every triple, so scanning divisors of one arbitrary triple's 3
78     // differences is guaranteed to contain the full set of valid M's.
79     vector<ll> distinctVals;
80     {
81         vector<ll> tmp = A;
82         sort(tmp.begin(), tmp.end());
83         tmp.erase(unique(tmp.begin(), tmp.end()), tmp.end());
84         distinctVals = tmp;
85     }
86

```

```

87 if ((int)distinctVals.size() < 3) {
88     // Only 2 distinct values overall, differing by exactly 1 (since D=1): every M>1
89     // fails to merge them, so cost=2 for every single M from 2..MAXM.
90     printf("2 %lld\n", MAXM - 1);
91     return 0;
92 }
93
94 ll a = distinctVals[0], b = distinctVals[1], c = distinctVals[2];
95 ll d1 = llabs(a - b), d2 = llabs(b - c), d3 = llabs(a - c);
96
97 vector<ll> candidates;
98 collectDivisorsGt1(d1, candidates);
99 collectDivisorsGt1(d2, candidates);
100 collectDivisorsGt1(d3, candidates);
101 sort(candidates.begin(), candidates.end());
102 candidates.erase(unique(candidates.begin(), candidates.end()), candidates.end());
103
104 // Lightweight open-addressing hash set, sized by N (not by M), reused across all
105 // candidates via a version stamp. This keeps memory at O(N) regardless of how large
106 // M gets (unlike indexing directly by residue, which would need an array of size M,
107 // and M can be up to just under 1e9 here).
108 int tableSize = 1;
109 while (tableSize < 4 * N + 16) tableSize <= 1;
110 int mask = tableSize - 1;
111 vector<ll> tableKey(tableSize);
112 vector<int> tableStamp(tableSize, -1);
113
114 int bestCost = INT_MAX;
115 ll bestCount = 0;
116
117 for (int idx = 0; idx < (int)candidates.size(); idx++) {
118     ll M = candidates[idx];
119     int distinctCount = 0;
120     for (ll x : A) {
121         ll r = x % M;
122         ull h = (ull)r * 0x9E3779B97F4A7C15ULL;
123         int pos = (int)(h >> 40) & mask;
124         while (tableStamp[pos] == idx && tableKey[pos] != r) {
125             pos = (pos + 1) & mask;
126         }
127         if (tableStamp[pos] != idx) {
128             tableStamp[pos] = idx;
129             tableKey[pos] = r;
130             distinctCount++;
131         }
132     }
133     if (distinctCount < bestCost) {
134         bestCost = distinctCount;
135         bestCount = 1;
136     } else if (distinctCount == bestCost) {
137         bestCount++;
138     }
139 }
140
141 printf("%d %lld\n", bestCost, bestCount);
142 return 0;
143 }

```

6.18 Number Theory (LCM via Factorizacion de Ciclos) - Permutation Rounds

C++ Training_Camp_2020 Day 10 - Permutation Rounds.cpp

Resuelve "Permutation Rounds" (Day 10, problema L - Contest 09 [Avanzados]); partiendo del arreglo ordenado $[1..n]$, en cada ronda todo elemento en la posición i se mueve a la posición p_i según una permutación fija; hay que hallar tras cuantas rondas el arreglo vuelve a estar ordenado por primera vez, módulo $1e9+7$. La respuesta es el LCM de las longitudes de todos los ciclos de la permutación vista como grafo funcional. El código recorre cada posición no visitada siguiendo $p[i]$ para medir la longitud de su ciclo (length), factoriza esa longitud en primos y actualiza $\text{maxPower}[\text{primo}]$ con el mayor exponente visto entre todos los ciclos (equivalente a construir el LCM por factorización en vez de multiplicar longitudes directamente, evitando overflow); al final multiplica $\text{primo}^{\text{maxPower}} \bmod \text{MOD}$ para cada primo usando exponenciación modular (power_mod).

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4

```

```

5 static char buf_[1 << 22];
6 static int bufLen_, bufPos_;
7 inline int gc() {
8     if (bufPos_ == bufLen_) { bufLen_ = fread(buf_, 1, sizeof(buf_), stdin); bufPos_ = 0; if (!bufL
9         en_) return -1; }
10    return buf_[bufPos_++];
11 }
12 inline int readInt() {
13     int c = gc();
14     while (c != -1 && (c < '0' || c > '9') && c != '-') c = gc();
15     bool neg = false; if (c == '-') { neg = true; c = gc(); }
16     int x = 0;
17     while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = gc(); }
18     return neg ? -x : x;
19 }
20 const ll MOD = 1000000007LL;
21
22 ll power_mod(ll base, ll exp, ll mod) {
23     base %= mod;
24     ll result = 1;
25     while (exp > 0) {
26         if (exp & 1) result = (result * base) % mod;
27         base = (base * base) % mod;
28         exp >>= 1;
29     }
30     return result;
31 }
32
33 int main() {
34     int n = readInt();
35     vector<int> p(n + 1);
36     for (int i = 1; i <= n; i++) p[i] = readInt();
37
38     vector<char> visited(n + 1, 0);
39     unordered_map<int, int> maxPower; // prime -> max exponent seen across all cycle lengths
40
41     for (int i = 1; i <= n; i++) {
42         if (visited[i]) continue;
43         int length = 0;
44         int j = i;
45         while (!visited[j]) {
46             visited[j] = 1;
47             j = p[j];
48             length++;
49         }
50         // factorize `length`, updating maxPower
51         int x = length;
52         for (int d = 2; (ll)d * d <= x; d++) {
53             if (x % d == 0) {
54                 int exp = 0;
55                 while (x % d == 0) { x /= d; exp++; }
56                 auto it = maxPower.find(d);
57                 if (it == maxPower.end() || it->second < exp) maxPower[d] = exp;
58             }
59         }
60         if (x > 1) {
61             auto it = maxPower.find(x);
62             if (it == maxPower.end() || it->second < 1) maxPower[x] = 1;
63         }
64     }
65
66     ll result = 1;
67     for (auto &pr : maxPower) {
68         result = (result * power_mod(pr.first, pr.second, MOD)) % MOD;
69     }
70
71     printf("%lld\n", result);
72     return 0;
73 }

```

6.19 Reconstruccion con Divisores - Remainders

C++\NPC\Remainders.cpp

Reconstruye el multiconjunto oculto K a partir de la sucesi n $S(n)$ = suma de $(n \bmod k)$ para todo k en K , dados $S(1)..S(N)$. Se apoya en dos observaciones: como todo k es al menos 2, $1 \bmod k = 1$ y por tanto $S(1)$ es directamente el tama o $|K|$; y al mirar la diferencia $S(n) - S(n-1)$, cada k que NO divide a n aporta +1 mientras que cada k que s  divide a n aporta $-(k-1)$, de modo que $D = |K| - (S(n) - S(n-1))$ termina siendo exactamente la suma de los elementos de K que dividen a n . Con eso se procesa n de menor a mayor: se le restan a D los aportes $k \cdot \text{cnt}[k]$ de los divisores propios ya conocidos, y lo que sobra es $\text{cnt}[n] \cdot n$, de donde sale la multiplicidad de n en K . Al final se expande el conteo en la lista ordenada de elementos.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int N;
6     if (!(cin >> N)) return 0;
7
8     vector<long long> S(N + 1, 0); // S[0] = 0
9     for (int i = 1; i <= N; i++) cin >> S[i];
10
11     long long m = S[1]; // |K| = S(1)
12     vector<long long> cnt(N + 1, 0);
13
14     for (int n = 2; n <= N; n++) {
15         long long D = m - (S[n] - S[n - 1]); // suma de los k de K que dividen a n
16         for (int k = 2; k < n; k++)
17             if (n % k == 0) D -= (long long)k * cnt[k];
18         if (D > 0) cnt[n] = D / n; // lo que queda es cnt[n]*n
19     }
20
21     vector<int> K;
22     for (int k = 2; k <= N; k++)
23         for (int i = 0; i < cnt[k]; i++) K.push_back(k);
24
25     cout << K.size();
26     for (int x : K) cout << ' ' << x;
27     cout << '\n';
28     return 0;
29 }

```

6.20 Sieve of Eratosthenes - Prime Sieve

C++\Training_Camp_2026\Day 11 - CCPL\E - Prime Sieve.cpp

Resuelve "Prime Sieve" (Day 11, problema E - CCPL (Kattis)): dado un l mite n (hasta 10^8) y q consultas x , pide primero cuantos n meros primos hay menores o iguales a n , y luego para cada consulta si x es primo (1) o compuesto (0). Implementa una criba de Eratosthenes cl sica sobre el vector "marked" (marca compuestos empezando en i^2 para evitar overflow con $1LL \cdot i \cdot i$), contando los primos menores o iguales al l mite pedido en la variable "cont" mientras criba, y responde cada consulta en $O(1)$ revisando "marked[aa]".

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6
7
8 const int MX = 100000000;
9
10
11
12 vector<bool> marked(MX + 1, false);
13 int cont = 0;
14
15
16
17 void sieve(int uwu) {
18     marked[0] = marked[1] = true;
19
20     for (int i = 2; i <= MX; i++) {
21         if (marked[i]) continue;
22
23         if (i <= uwu)
24             cont++;
25     }
26 }

```

```

26         if (1LL * i * i > MX) continue;
27         for (1ll j = 1LL * i * i; j <= MX; j += i)
28             marked[j] = true;
29     }
30 }
31
32 int main() {
33     ios::sync_with_stdio(false);
34     cin.tie(nullptr);
35
36     int t;
37     cin >> t;
38
39     sieve(t);
40
41     int n; cin >> n;
42     cout << cont << endl;
43     while (n--) {
44         int aa; cin >> aa;
45
46         if(marked[aa]==false)
47         {
48             cout << 1 << endl;
49         }
50         else
51         {
52             cout << 0 << endl;
53         }
54     }
55
56     return 0;
57 }

```

6.21 Ternas Pitagoricas - Pythagorean

C++\RPC\Pythagorean.cpp

Dado n , cuenta en cuantas ternas pitagoricas participa, separando cuatro casos: n como hipotenusa (primitivas y no primitivas) y n como cateto (primitivas y no primitivas). Para n como hipotenusa recorre cada cateto a y verifica si $n^2 - a^2$ es cuadrado perfecto, ajustando la raíz con sqrtl mas dos while para corregir el error de punto flotante (nunca confiar en sqrt directo con enteros grandes); exige $b > a$ para no contar la misma terna dos veces, y $\text{gcd}(a,b) == 1$ la marca como primitiva. Para n como cateto usa la factorizacion $(c-b)(c+b) = n^2$: cada divisor $d < n$ de n^2 da $e = n^2/d$, y si d y e tienen la misma paridad se recupera $b = (e-d)/2$ y $c = (e+d)/2$.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4
5 int main() {
6     ll n;
7     if (!(cin >> n)) return 0;
8
9     ll hypPrim = 0, hypNon = 0, legPrim = 0, legNon = 0;
10    ll n2 = n * n;
11
12    // --- n como hipotenusa ---
13    for (1ll a = 1; a < n; a++) {
14        ll t = n2 - a * a;
15        ll b = (1ll)sqrtl((long double)t);
16        while (b * b < t) b++;
17        while (b * b > t) b--;
18        if (b * b == t && b > a) { // b > a evita contar dos veces
19            if (__gcd(a, b) == 1) hypPrim++;
20            else hypNon++;
21        }
22    }
23
24    // --- n como cateto: (c-b)(c+b) = n^2 ---
25    for (1ll d = 1; d < n; d++) { // d = c-b, d < e obliga d < n
26        if (n2 % d) continue;
27        ll e = n2 / d; // e = c+b
28        if (((e - d) & 1LL)) continue; // misma paridad
29        ll b = (e - d) / 2;
30        ll c = (e + d) / 2;
31        if (b <= 0) continue;

```

```

32        if (__gcd(n, b) == 1) legPrim++;
33        else legNon++;
34    }
35
36    cout << hypPrim << ' ' << hypNon << ' '
37        << legPrim << ' ' << legNon << '\n';
38    return 0;
39 }

```

7 - Combinatorics

7.1 Combinatoria (Factoriales Modulares) - Binomial Coefficients

C++\Training_Camp_2026\Day 10J - Binomial Coefficients.cpp

Resuelve "Binomial Coefficients" (Day 10, problema J - Contest 09 [Avanzados]): calcula n coeficientes binomiales $C(a,b)$ modulo $1e9+7$. Precomputa $\text{fact}[i] = i! \bmod \text{MOD}$ para todo i hasta $\text{MAXN}=1e6$, y su inverso modular $\text{invFact}[i]$ de atras hacia adelante en $O(\text{MAXN})$ (calculando solo una exponenciacion modular $\text{invFact}[\text{MAXN}] = \text{fact}[\text{MAXN}]^{(\text{MOD}-2)}$ via Fermat, y derivando el resto con $\text{invFact}[i-1] = \text{invFact}[i] * i$). Cada consulta responde en $O(1)$ con la formula $\text{fact}[a] * \text{invFact}[b] * \text{invFact}[a-b] \bmod \text{MOD}$.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 const int MOD = 1000000007;
7 const int MAXN = 1000000;
8
9 vector<ll> fact(MAXN + 1), invFact(MAXN + 1);
10
11 ll modPow(ll a, ll e) {
12     ll res = 1;
13     while (e > 0) {
14         if (e & 1) res = res * a % MOD;
15         a = a * a % MOD;
16         e >>= 1;
17     }
18     return res;
19 }
20
21 int main() {
22     ios::sync_with_stdio(false);
23     cin.tie(nullptr);
24
25     fact[0] = 1;
26     for (int i = 1; i <= MAXN; i++)
27         fact[i] = fact[i - 1] * i % MOD;
28
29     invFact[MAXN] = modPow(fact[MAXN], MOD - 2);
30
31     for (int i = MAXN; i >= 1; i--)
32         invFact[i - 1] = invFact[i] * i % MOD;
33
34     int n;
35     cin >> n;
36
37     while (n--) {
38         int a, b;
39         cin >> a >> b;
40
41         ll ans = fact[a];
42         ans = ans * invFact[b] % MOD;
43         ans = ans * invFact[a - b] % MOD;
44
45         cout << ans << '\n';
46     }
47
48     return 0;
49 }

```

7.2 Combinatoria Avanzada (Lucas, Catalan, Stirling)

C++\Notebook\Combinatoria Avanzada.cpp

Formulas combinatorias que van mas alla del $C(n,k)$ basico con factoriales precomputados (ese caso ya esta resuelto en "Binomial Coefficients"). Aqui van cuatro herramientas; Lucas, para cuando n y k son gigantes (hasta $1e18$) pero el modulo es un primo pequeno, descomponiendo n y k en base p y multiplicando los binomiales de cada digito; multinomial, para repartir n objetos en grupos de tamanos distintos; Catalan, que cuenta estructuras balanceadas (parentesis validos, arboles binarios, caminos de Dyck); y Stirling de segunda especie, que cuenta en cuantas formas se parten n elementos en k grupos no vacios. Las tres primeras necesitan que el modulo sea primo (usan el inverso via Fermat).

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 using ll = long long;
6
7 ll modpow(ll a, ll e, ll m)
8 {
9     ll r = 1 % m;
10    a %= m;
11    while (e)
12    {
13        if (e & 1) r = r * a % m;
14        a = a * a % m;
15        e >>= 1;
16    }
17    return r;
18 }
19
20 // C(n,k) mod p directo en O(k), para n y k chicos (uso interno de Lucas)
21 ll Csmall(ll n, ll k, ll p)
22 {
23     if (k < 0 || k > n) return 0;
24     ll num = 1, den = 1;
25     for (ll i = 1; i <= k; i++)
26     {
27         num = num * ((n - k + i) % p) % p;
28         den = den * i % p;
29     }
30     return num * modpow(den, p - 2, p) % p;
31 }
32
33 // Teorema de Lucas: C(n,k) mod p con n, k enormes y p primo pequeno
34 ll C_lucas(ll n, ll k, ll p)
35 {
36     ll res = 1;
37     while (n || k)
38     {
39         ll ni = n % p, ki = k % p;
40         res = res * Csmall(ni, ki, p) % p;
41         n /= p;
42         k /= p;
43         if (!res) return 0;
44     }
45     return res;
46 }
47
48 const int MOD = 1e9 + 7;
49 const int MAXN = 1000000;
50 vector<ll> fact(MAXN + 1), invFact(MAXN + 1);
51
52 void precomputar()
53 {
54     fact[0] = 1;
55     for (int i = 1; i <= MAXN; i++) fact[i] = fact[i - 1] * i % MOD;
56     invFact[MAXN] = modpow(fact[MAXN], MOD - 2, MOD);
57     for (int i = MAXN; i >= 1; i--) invFact[i - 1] = invFact[i] * i % MOD;
58 }
59
60 ll C(int n, int k)
61 {
62     if (k < 0 || k > n) return 0;
63     return fact[n] * invFact[k] % MOD * invFact[n - k] % MOD;
```

```
64 }
65
66 // n! / (a1! * a2! * ... * ak!) : repartir n objetos en grupos de tamanos dados
67 ll multinomial(const vector<int>& a)
68 {
69     ll n = 0;
70     for (int x : a) n += x;
71     ll r = fact[n];
72     for (int x : a) r = r * invFact[x] % MOD;
73     return r;
74 }
75
76 // Catalan(n) = C(2n,n) / (n+1) : parentesis balanceados, arboles binarios, caminos de Dyck
77 ll catalan(int n)
78 {
79     return C(2 * n, n) * modpow(n + 1, MOD - 2, MOD) % MOD;
80 }
81
82 // Stirling de segunda especie: partir n elementos en k grupos no vacios
83 vector<vector<ll>> stirling2(int n, int k)
84 {
85     vector<vector<ll>> S(n + 1, vector<ll>(k + 1, 0));
86     S[0][0] = 1;
87     for (int i = 1; i <= n; i++)
88     {
89         for (int j = 1; j <= k; j++)
90         {
91             S[i][j] = (S[i - 1][j - 1] + (ll)j * S[i - 1][j]) % MOD;
92         }
93     }
94     return S;
95 }
96
97 int main()
98 {
99     ios_base::sync_with_stdio(false);
100    cin.tie(nullptr);
101    return 0;
102 }
```

7.3 Construction - Farmpiggy And Subset Sum

C++\Codeforces\Farmpiggy And Subset Sum.cpp

Problema de construccion (el nombre sugiere relacion con subconjuntos de suma prohibida): arma una permutacion de $1..n$ intercambiando cada pareja de consecutivos ($p[1]=2, p[2]=1, p[3]=4, p[4]=3, \dots$), de forma que $p[i] \neq i$ para todo i (derangement simple) y cada indice queda emparejado con su vecino inmediato.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios_base::sync_with_stdio(false);
6     cin.tie(NULL);
7
8     int t;
9     cin >> t;
10    while (t--) {
11        int n;
12        cin >> n;
13
14        vector<int> p(n + 1);
15        for (int i = 1; i <= n; i += 2) {
16            p[i] = i + 1;
17            p[i + 1] = i;
18        }
19
20        for (int i = 1; i <= n; i++) {
21            cout << p[i];
22            if (i != n) cout << ' ';
23        }
24        cout << "\n";
25    }
26
27    return 0;
```

```
28 }
```

7.4 Cuento de Distintos con Inventario de Frecuencias - Total Numbers

C++[LeetCodeTotalNumbers.cpp]

Resuelve "Unique 3-Digit Even Numbers" (LeetCode 3483): dado un arreglo de dígitos, contar cuantos números DISTINTOS de tres cifras se pueden armar usando tres de ellos, sin cero inicial y que el resultado sea par. La trampa es "distintos". Si uno cuenta permutaciones de posiciones, [6,6,6] daría 6 en vez de 1, porque los tres seises son intercambiables. El arreglo cnt da la vuelta al problema: en vez de recorrer POSICIONES del arreglo, recorre VALORES. Cada terna (first, second, last) se visita exactamente una vez, así que cada número se cuenta una sola vez por construcción, sin sets ni deduplicación posterior. Lo que hace que sea correcto y no solo "distinto" es el préstamo: cnt[last]-- antes de abrir el ciclo de first, y cnt[first]-- antes del de second. Eso reserva la copia física del dígito que ya se usó, para que cnt[second] > 0 signifique "queda todavía un ejemplar libre". Sin ese préstamo, [2,0,0] contaría 220 aunque solo haya un dos. Cada decremento se devuelve al salir del ciclo, así que cnt vuelve intacto en cada vuelta. El último dígito manda la paridad, por eso se fija primero y solo con {0,2,4,6,8}; first va de 1 a 9 porque no puede haber cero inicial; second es libre de 0 a 9. Costo $O(n + 5 \cdot 9 \cdot 10) = O(n)$ con 450 iteraciones fijas, sin memoria extra más allá de cnt[10]. El patrón sirve para cualquier "cuantos resultados distintos salen de un multiconjunto": iterar sobre los valores posibles y usar el arreglo de frecuencias como inventario que se presta y se devuelve.

```
1 class Solution {
2 public:
3     int totalNumbers(vector<int>& digits) {
4         int cnt[10] = {};
5
6         for (int x : digits) {
7             cnt[x]++;
8         }
9
10        int ans = 0;
11
12        for (int last : {0, 2, 4, 6, 8}) {
13            if (cnt[last] == 0) continue;
14
15            cnt[last]--;
16
17            for (int first = 1; first <= 9; first++) {
18                if (cnt[first] == 0) continue;
19
20                cnt[first]--;
21
22                for (int second = 0; second <= 9; second++) {
23                    if (cnt[second] > 0) ans++;
24                }
25
26                cnt[first]++;
27            }
28
29            cnt[last]++;
30        }
31
32        return ans;
33    }
34 }
```

7.5 Cuento de Inversiones (cadena repetida) - Inversions

C++[CodeforcesInversions.cpp]

Cuenta las inversiones de S repetida N veces (N hasta $1e12$) módulo $1e9+7$, es decir pares $i < j$ donde la letra de i va después en el alfabeto que la de j . No se puede construir la cadena, así que se cuenta la contribución de cada posición por separado. Una posición i con letra c aporta dos cosas: dentro de su propia copia aporta $\text{suf} = \text{cuantas letras menores que } c \text{ hay de } i \text{ en adelante}$, y eso ocurre igual en las N copias ($N \cdot \text{suf}$); contra las copias posteriores le sirven TODAS las letras menores que c de la cadena completa ($\text{freqGod}[c]$), una vez por cada par ordenado de copias, o sea $N(N-1)/2$. $\text{freqAtras}[i]$ guarda cuantas letras menores que cada letra quedan desde i hasta el final, y freqGod acumula las frecuencias globales. La división entre 2 va con inverso modular (Fermat, $\text{modpow}(b, \text{MOD}-2)$) porque $N(N-1)/2$ ya se calcula en módulo. Nota: el if de $\text{freqGod} == \text{freqAtras}$ es redundante, las dos ramas dan el mismo valor (cuando $\text{suf} == \text{freqGod}[c]$, $N(N+1)/2 \cdot \text{freqGod} == N(N-1)/2 \cdot \text{freqGod} + N \cdot \text{suf}$).

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 long long MOD = 1000000007;
5
6 long long modpow(long long a, long long e) {
7     long long ans = 1;
8
9     while (e) {
10        if (e & 1)
11            ans = ans * a % MOD;
12        a = a * a % MOD;
13        e >>= 1;
14    }
15    return ans;
16 }
```

```
9     while (e) {
10        if (e & 1)
11            ans = ans * a % MOD;
12        a = a * a % MOD;
13        e >>= 1;
14    }
15    return ans;
16 }
17
18 long long division_mod(long long a, long long b) {
19     return a % MOD * modpow(b, MOD - 2) % MOD;
20 }
21
22 int main()
23 {
24     string s; cin >> s;
25     long long n; cin >> n;
26
27     vector<vector<long long>> freqAtras;
28     vector<long long> freq(26,0);
29     vector<long long> vectorsito(26,0);
30
31     for(long long i = s.size()-1; i >=0; i--)
32     {
33         char letra = s[i];
34         freq[letra-'0'-49]++;
35         for(long long j = 0; j < 26; j++)
36         {
37             if(letra-'0'-49 < j)
38             {
39                 vectorsito[j]++;
40             }
41         }
42         freqAtras.push_back(vectorsito);
43     }
44
45     vector<long long> freqGod(26,0);
46     long long sum = 0;
47     for(long long i = 1; i < 26; i++)
48     {
49         sum += freq[i-1];
50         freqGod[i] = sum;
51     }
52
53     long long GOD = 0;
54     for(long long i = 0; i < s.size(); i++)
55     {
56         if(freqGod[s[i]-'0'-49] == freqAtras[s.size()-i-1][s[i]-'0'-49])
57         {
58             GOD = (GOD + division_mod(((n % MOD) * ((n + 1) % MOD) % MOD , 2) * freqGod[s[i]-'0'-49] % MOD) % MOD;
59         }
60         else
61         {
62             GOD = (GOD + division_mod(((n-1) % MOD) * ((n) % MOD) % MOD , 2) * freqGod[s[i]-'0'-49] % MOD) % MOD;
63             GOD = (GOD + (freqAtras[s.size()-i-1][s[i]-'0'-49] * n) % MOD) % MOD;
64         }
65     }
66
67     cout << GOD << endl;
68
69     return 0;
70 }
```

7.6 Euler's Formula - Pizza Cutter

C++ Training_Camp_2020/Day 01C - Pizza Cutter.cpp

Resuelve "Pizza Cutter" (Day 8, problema C - Contest 08 [Avanzados]): el abuelo Giuseppe corta una pizza rectangular con H cortes que entran por el lado izquierdo y salen por el derecho, y V cortes que entran por abajo y salen por arriba (cada corte es una curva continua, sin tres cortes concurrentes ni cortes tangentes); dado donde entra y sale cada corte, hay que contar en cuantas piezas queda dividida la pizza. Usa la formula de Euler para arreglos de curvas en el plano (piezas = $1 + \#cortes + \#cruces$): cada par de cortes de tipos opuestos siempre se cruza una vez (aporta $H*V$ cruces), y los cruces entre cortes del mismo tipo se cuentan como inversiones (hh, vv) del orden de sus extremos derecho/superior via `countInversions` (merge sort), sumando todo en "answer".

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 static char buf_[1 << 22];
5 static int bufLen_, bufPos_;
6 inline int gc() {
7     if (bufPos_ == bufLen_) { bufLen_ = fread(buf_, 1, sizeof(buf_), stdin); bufPos_ = 0; if (!bufLen_) return -1; }
8     return buf_[bufPos_++];
9 }
10 inline long long readLL() {
11     int c = gc();
12     while (c != -1 && (c < '0' || c > '9') && c != '-') c = gc();
13     bool neg = false; if (c == '-') { neg = true; c = gc(); }
14     long long x = 0;
15     while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = gc(); }
16     return neg ? -x : x;
17 }
18
19 // Counts inversions in `a` (pairs i<j with a[i] > a[j]) via merge sort. O(n log n).
20 long long countInversions(vector<long long>& a) {
21     int n = (int)a.size();
22     if (n <= 1) return 0;
23     vector<long long> buf(n);
24     long long inv = 0;
25     for (int width = 1; width < n; width *= 2) {
26         for (int i = 0; i < n; i += 2 * width) {
27             int mid = min(i + width, n);
28             int end = min(i + 2 * width, n);
29             int p = i, q = mid, k = i;
30             while (p < mid && q < end) {
31                 if (a[p] <= a[q]) buf[k++] = a[p++];
32                 else { buf[k++] = a[q++]; inv += (mid - p); }
33             }
34             while (p < mid) buf[k++] = a[p++];
35             while (q < end) buf[k++] = a[q++];
36             for (int t = i; t < end; t++) a[t] = buf[t];
37         }
38     }
39     return inv;
40 }
41
42 int main() {
43     // X, Y (pizza dimensions) are read for input-format purposes only; the piece
44     // count depends solely on H, V, and the relative order of cut endpoints.
45     (void)readLL(); (void)readLL();
46     long long H = readLL(), V = readLL();
47
48     vector<pair<long long, long long>> hcuts(H), vcuts(V);
49     for (auto& p : hcuts) { p.first = readLL(); p.second = readLL(); }
50     for (auto& p : vcuts) { p.first = readLL(); p.second = readLL(); }
51
52     sort(hcuts.begin(), hcuts.end());
53     vector<long long> y2seq;
54     y2seq.reserve(H);
55     for (auto& p : hcuts) y2seq.push_back(p.second);
56     long long hh = countInversions(y2seq);
57
58     sort(vcuts.begin(), vcuts.end());
59     vector<long long> x2seq;
60     x2seq.reserve(V);
61     for (auto& p : vcuts) x2seq.push_back(p.second);
```

```
62     long long vv = countInversions(x2seq);
63
64     long long answer = 1 + H + V + hh + vv + H * V;
65     printf("%lld\n", answer);
66     return 0;
67 }
```

7.7 Formula Cerrada con Inverso Modular - Number Of Sets

C++ LeetCode/numberOfSets.cpp

Resuelve "Number of Sets of K Non-Overlapping Line Segments" (LeetCode 1621): con n puntos alineados hay que contar en cuantas formas se dibujan k segmentos que no se solapen (si pueden compartir un extremo), modulo $1e9+7$. Toda la DP se colapsa en un solo binomial: la respuesta es $C(n+k-1, 2k)$. La idea es que elegir k segmentos equivale a escoger $2k$ extremos de entre $n+k-1$ posiciones, porque cada extremo compartido se "desdobra" agregando $k-1$ posiciones ficticias. Como hay que dividir en modular, se precomputan factoriales y sus inversos: una sola exponenciación modular para $\text{inverso}[\text{limite}] = \text{factorial}[\text{limite}]^{(MOD-2)}$ por Fermat, y el resto baja con $\text{inverso}[i-1] = \text{inverso}[i] * i$. Así cada binomial sale en $O(1)$ en vez de pagar un modpow por consulta. Costo total $O(n + k + \log MOD)$. OJO con el índice: si $2k > \text{limite}$, $\text{limite} - 2k$ se sale del arreglo. No pasa dentro de las restricciones del problema ($k \leq n-1$ obliga a $2k \leq n+k-1$), pero si reusas este patron en otro problema, valida el rango antes.

```
1 class Solution {
2 public:
3     long long MOD = 1000000007;
4
5     long long modpow(long long a, long long e) {
6         long long ans = 1;
7
8         while (e) {
9             if (e & 1)
10                 ans = ans * a % MOD;
11
12             a = a * a % MOD;
13             e >>= 1;
14         }
15
16         return ans;
17     }
18
19     long long division_mod(long long a, long long b) {
20         return a % MOD * modpow(b, MOD - 2) % MOD;
21     }
22
23     int numberOfSets(int n, int k) {
24         long long limite = n + k - 1;
25
26         vector<long long> factorial(limite + 1);
27         vector<long long> inverso(limite + 1);
28
29         factorial[0] = 1;
30
31         for (long long i = 1; i <= limite; i++) {
32             factorial[i] = factorial[i - 1] * i % MOD;
33         }
34
35         inverso[limite] = modpow(factorial[limite], MOD - 2);
36
37         for (long long i = limite; i >= 1; i--) {
38             inverso[i - 1] = inverso[i] * i % MOD;
39         }
40
41         long long GOD = factorial[limite];
42
43         GOD = GOD * inverso[2 * k] % MOD;
44         GOD = GOD * inverso[limite - 2 * k] % MOD;
45
46         return GOD;
47     }
48
49     // Lo mismo sin la variable limite, escribiendo C(n+k-1, 2k) completo:
50     //
51     //     long long GOD = factorial[n + k - 1];
52     //
53     //     GOD = GOD * inverso[2 * k] % MOD;
54     //     GOD = GOD * inverso[n - k - 1] % MOD;
```

```

94 //
95 //     return GOD;
96 //
97 // Es identico porque limite - 2k = (n + k - 1) - 2k = n - k - 1.
98 // Ayuda a ver que esto es literalmente C(arriba, abajo) con la forma
99 // factorial[arriba] * inverso[abajo] * inverso[arriba - abajo].
100 }
101 };

```

7.8 Paridad de Binomiales (Kummer) - Less Coint Tosses

C++\Training_Camp_2026\Day 1\L - Less_coint_tosses.py

Resuelve "Less Coin Tosses" (Day 1, problema L - Maratona de Programacao da SBC 2019): hay que repartir las 2^N cadenas binarias de longitud N entre Carla y Daniel de forma que la probabilidad de ganar sea igual sin importar el sesgo de la moneda, lo cual solo se logra emparejando equitativamente las cadenas dentro de cada nivel de peso de Hamming k (ya que $P(s)$ depende solo de k), y pide el minimo numero de cadenas que quedan sin asignar. Por el teorema de Kummer, $C(N,k)$ es impar solo cuando k es submascara de los bits de N , y hay exactamente $2^{\text{popcount}(N)}$ de esos k ; por eso basta calcular 2 elevado a la cantidad de bits en 1 de N .

```

1 import math
2 n = int(input())
3 m = bin(n)[2:]
4 j = m.count("1")
5 print(2**j)

```

7.9 Permutation Cycles - Lines Of Containers

C++\Training_Camp_2026\Day 2\L - Lines of Containers.cpp

Resuelve "Lines of Containers" (Day 2, problema G - Maratona de Programacao da SBC 2013): un cargamento de L filas por C columnas de contenedores, numerados 1..LC en orden, fue desordenado moviendo filas y columnas completas con una grua; hay que determinar si la grua puede devolverlos a su posicion original y, de ser posible, el minimo numero de swaps de filas y columnas necesario. Para cada fila (y cada columna) actual deduce a que fila/columna ORIGINAL deberia pertenecer segun los ids, usando $(id-1)/C$ y $(id-1)\%C$, y valida que sea consistente y forme una permutacion valida. Con eso arma rowPerm y colPerm; minSwaps() descompone cada permutacion en ciclos y suma $(\text{tamano_del_ciclo} - 1)$ por ciclo, que es el minimo de swaps para ordenarla.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 /*
5  Calcula el mínimo número de swaps necesarios para convertir
6  una permutación en la identidad.
7
8  Si una permutación tiene un ciclo de tamaño k,
9  se necesitan k-1 intercambios para corregirlo.
10 */
11 int minSwaps(vector<int> &perm) {
12
13     int n = perm.size();
14
15     vector<bool> vis(n, false);
16
17     int ans = 0;
18
19     for (int i = 0; i < n; i++) {
20
21         // Si ya fue visitado o ya está en su lugar
22         if (vis[i] || perm[i] == i)
23             continue;
24
25         int cur = i;
26         int cycleSize = 0;
27
28         // Recorremos el ciclo completo
29         while (!vis[cur]) {
30             vis[cur] = true;
31             cur = perm[cur];
32             cycleSize++;
33         }
34
35         // Un ciclo de tamaño k necesita k-1 swaps
36         ans += cycleSize - 1;
37     }
38 }

```

```

39     return ans;
40 }
41
42 int main() {
43
44     ios::sync_with_stdio(false);
45     cin.tie(nullptr);
46
47     int L, C;
48     cin >> L >> C;
49
50     // Matriz con la configuración actual
51     vector<vector<int>> a(L, vector<int>(C));
52
53     for (int i = 0; i < L; i++)
54         for (int j = 0; j < C; j++)
55             cin >> a[i][j];
56
57     /*
58     rowPerm[i] = fila ORIGINAL a la que realmente pertenece
59     la fila actual i.
60
61     Ejemplo:
62     rowPerm = [3,1,2,0]
63
64     significa:
65     la fila actual 0 debería ser la fila 3,
66     la fila actual 1 debería ser la fila 1,
67     etc.
68     */
69     vector<int> rowPerm(L, -1);
70
71     // Revisamos fila por fila
72     for (int i = 0; i < L; i++) {
73
74         /*
75         La fila original de un contenedor se obtiene como
76
77             (id-1)/C
78
79         porque cada fila contiene exactamente C elementos.
80         */
81         int expectedRow = (a[i][0] - 1) / C;
82
83         // Todos los elementos de la fila deben pertenecer
84         // a la misma fila original.
85         for (int j = 1; j < C; j++) {
86
87             if ((a[i][j] - 1) / C != expectedRow) {
88                 cout << " *\n";
89                 return 0;
90             }
91         }
92
93         rowPerm[i] = expectedRow;
94     }
95
96     /*
97     Además debe ser una permutación válida.
98
99     No pueden existir dos filas actuales que correspondan
100     a la misma fila original.
101     */
102     vector<bool> used(L, false);
103
104     for (int x : rowPerm) {
105
106         if (used[x]) {
107             cout << " *\n";
108             return 0;
109         }
110
111         used[x] = true;

```

```

12     }
13
14     /*
15         colPerm[j] = columna ORIGINAL a la que corresponde
16         la columna actual j.
17     */
18     vector<int> colPerm(C, -1);
19
20     // Revisamos columna por columna
21     for (int j = 0; j < C; j++) {
22
23         /*
24             La columna correcta de un contenedor es
25
26             (id-1)%C
27         */
28         int expectedCol = (a[0][j] - 1) % C;
29
30         // Todos los elementos de la columna deben pertenecer
31         // a la misma columna original.
32         for (int i = 1; i < L; i++) {
33
34             if ((a[i][j] - 1) % C != expectedCol) {
35                 cout << "\n";
36                 return 0;
37             }
38         }
39
40         colPerm[j] = expectedCol;
41     }
42
43     // Verificamos nuevamente que sea una permutación válida
44     used.assign(C, false);
45
46     for (int x : colPerm) {
47
48         if (used[x]) {
49             cout << "\n";
50             return 0;
51         }
52
53         used[x] = true;
54     }
55
56     /*
57         Si llegamos aquí, existe una secuencia de intercambios.
58
59         El número mínimo es:
60
61         swaps para ordenar las filas
62         +
63         swaps para ordenar las columnas
64     */
65     cout << minSwaps(rowPerm) + minSwaps(colPerm) << "\n";
66
67     return 0;
68 }

```

7.10 Probabilidad (Paradoja del cumpleaños) - Mount Rushmore And Birthdays

C++\Training_Camp_2020\Day 3\ - Mount Rushmore and Birthdays.cpp

Resuelve "Mount Rushmore and Birthdays" (Day 5, problema F - Contest 05 [Avanzados]): dado N, el numero de días que tiene un año, hay que hallar la mínima cantidad de personas necesarias en un cuarto para que la probabilidad de que dos compartan cumpleaños sea mayor a 50%. Calcula iterativamente, persona por persona, la probabilidad "prob" de que todos los cumpleaños sigan siendo distintos (multiplicando (n-personas)/n en cada paso, la generalización de la formula clasica de la paradoja del cumpleaños), y se detiene apenas 1 - prob supera 0.5, devolviendo esa cantidad de personas.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4
5 int main() {
6     ios::sync_with_stdio(false);
7     cin.tie(nullptr);

```

```

8     double n; cin >>n;
9     double prob = 1;
10    double persons = 2;
11    prob *= (n-1)/n;
12
13    if(n==1)
14    {
15        cout << 2 << endl;
16        return 0;
17    }
18
19
20    while(true)
21    {
22        prob *= (n-persons)/n;
23
24        persons++;
25        if(1-prob>0.5)
26        {
27            cout<< persons << endl;
28            return 0;
29        }
30
31    }
32
33
34
35
36
37    return 0;
38 }

```

7.11 Probabilidad (Valor esperado con inverso modular) - Frangolino Ali Na Mesa

C++\Training_Camp_2020\Day 6\ - Frangolino ali na mesa.cpp

Resuelve "Frangolino ali na mesa" (Day 6, problema F - Maratona SBC de Programacao 2025): un robot mesero recibe Q comandos, cada uno con un argumento X_i , pero cada comando es en realidad "ir a la mesa X_i " o "pedir X_i milanesas en la mesa actual" con 50% de probabilidad cada uno (empezando en la mesa 1); hay que calcular, para cada mesa, el valor esperado de milanesas servidas, modulo $1e9+7$. El código usa linealidad de la esperanza: arma $\text{coeff}[i] = X_i \cdot \text{inv}2^i$, calcula sumas por sufijo "suffix[i]" de esos coeficientes, inicializa $\text{ans}[1]$ con $\text{suffix}[1]$ (caso en que el robot nunca se movio) y le suma a $\text{ans}[X_i]$ el termino $\text{pow}2[i-1] \cdot \text{suffix}[i+1]$ (la contribucion esperada de que el comando i sea el ultimo movimiento hacia esa mesa), todo con aritmetica modular precomputada de potencias e inversos de 2.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 static const int MOD = 1000000007;
5
6 int main() {
7     ios::sync_with_stdio(false);
8     cin.tie(nullptr);
9
10    int N, Q;
11    cin >> N >> Q;
12
13    vector<int> X(Q + 1);
14
15    long long inv2 = (MOD + 1) / 2;
16
17    vector<long long> invPow(Q + 1), pow2(Q + 1);
18    invPow[0] = 1;
19    pow2[0] = 1;
20    for (int i = 1; i <= Q; i++) {
21        invPow[i] = invPow[i - 1] * inv2 % MOD;
22        pow2[i] = pow2[i - 1] * 2 % MOD;
23    }
24
25    vector<long long> coeff(Q + 1);
26    for (int i = 1; i <= Q; i++) {
27        cin >> X[i];
28        coeff[i] = 1LL * X[i] * invPow[i] % MOD;
29    }
30

```

```

31 vector<long long> suffix(Q + 2, 0);
32 for (int i = Q; i >= 1; i--) {
33     suffix[i] = (suffix[i + 1] + coeff[i]) % MOD;
34 }
35
36 vector<long long> ans(N + 1, 0);
37
38 // initial value at table 1
39 ans[1] = suffix[1];
40
41 // increments
42 for (int i = 1; i <= Q; i++) {
43     long long add = pow2[i - 1] * suffix[i + 1] % MOD;
44     ans[X[i]] += add;
45     if (ans[X[i]] >= MOD) ans[X[i]] -= MOD;
46 }
47
48 for (int i = 1; i <= N; i++)
49     cout << ans[i] % MOD << "\n";
50
51 return 0;
52 }

```

7.12 Ranking - Palindromic Lexi Small

C++17 `lexiCode/palindromicLexiSmall.cpp`

Construye el k -ésimo palindromo mas pequeno lexicograficamente que se puede formar reordenando el string s (o "" si no existen k reordenamientos posibles). Solo arma la mitad izquierda del palindromo (mas el caracter central si el largo es impar) y la refleja: para cada posicion prueba las letras disponibles de menor a mayor y usa `comb()/perms()` (conteo de permutaciones del multiset restante, saturado a $1e6$ para evitar overflow) para saber, sin generar todas las combinaciones, cuantos reordenamientos empiezan con esa letra y saltarselos si cae mas adelante.

```

1 class Solution {
2 public:
3     const long long LIM = 1000000;
4
5     // C(n, r) saturado a LIM+1 si el valor real lo supera.
6     // O(r), con división exacta garantizada en cada paso (identidad de binomiales),
7     // en vez de armar vectores de tamaño r y simplificar con gcd (eso era O(r^2) y el cuello de bo
8     // tella real).
9     long long comb(long long n, long long r) {
10         if (r < 0 || r > n) return 0;
11         r = min(r, n - r);
12         long long result = 1;
13         for (long long i = 1; i <= r; i++) {
14             result = result * (n - i + 1) / i;
15             if (result > LIM) return LIM + 1;
16         }
17         return result;
18     }
19
20     // Permutaciones distintas del multiset cnt[0..25] (n = total restante), saturado a LIM+1.
21     long long perms(long long n, const array<int, 26>& cnt) {
22         long long ans = 1;
23         long long restantes = n;
24         for (int c = 0; c < 26; c++) {
25             if (cnt[c] == 0) continue;
26             long long ways = comb(restantes, cnt[c]);
27             if (ways > LIM) return LIM + 1;
28             ans *= ways;
29             restantes -= cnt[c];
30             if (ans > LIM) return LIM + 1;
31         }
32         return ans;
33     }
34
35     string smallestPalindrome(string s, int k) {
36         int n = s.size();
37         bool mid = (n % 2 != 0);
38         char midi = mid ? s[n / 2] : '\0';
39         string half = s.substr(0, n / 2);
40
41         array<int, 26> cnt{};

```

```

42     long long tot = 0;
43     for (char c : half) { cnt[c - 'a']++; tot++; }
44
45     if ((long long)k > perms(tot, cnt)) return "";
46
47     string god;
48     god.reserve(half.size());
49     long long cont = 0;
50     long long curTot = tot;
51
52     for (size_t i = 0; i < half.size(); i++) {
53         for (int c = 0; c < 26; c++) {
54             if (cnt[c] == 0) continue;
55             cnt[c]--;
56             curTot--;
57             long long permi = perms(curTot, cnt);
58             if (cont + permi < k) {
59                 cont += permi;
60                 cnt[c]++;
61                 curTot++;
62             } else {
63                 god += char('a' + c);
64                 break;
65             }
66         }
67     }
68
69     string sol = god;
70     if (mid) sol += midi;
71     reverse(god.begin(), god.end());
72     sol += god;
73     return sol;
74 }
75 };

```

7.13 Ranking de Combinaciones - Discrete Catalog

C++17 `ICPC2025/D - Discrete Catalog.cpp`

Resuelve "Discrete Catalog" (problema D, ICPC 2025): dado un subconjunto de tamaño k escrito en orden creciente, decir que posición ocupa en la lista de TODOS los subconjuntos de tamaño k ordenados lexicograficamente, contando desde 0. LA IDEA ES CONTAR LO QUE VA ANTES, sin generar nada. Recorriendo el subconjunto de izquierda a derecha, cuando en la posición i esta el valor $a[i]$, todos los subconjuntos que en esa posición tienen un valor MENOR (y coinciden en lo anterior) van antes. Para cada candidato j menor que $a[i]$, lo que queda por elegir son $k-i-1$ valores de entre los $n-j$ que siguen, o sea $C(n-j, k-i-1)$. Se suman todos esos y se pasa a la siguiente posición. Ese patron de "contar los que van antes prefijo por prefijo" es el ranking, y sirve igual para permutaciones o para cualquier orden lexicografico: cambia solo la formula de cuantos completan. El nCr usado multiplica y divide alternando ($r = r * (n-i+1) / i$), que mantiene los numeros chicos porque en cada paso r es exactamente un binomial. Con $n \leq 60$ no se desborda: el mayor ID posible es $C(60, 30) - 1 = 118264581564861423$, que cabe holgado en long long. Ojo que esa misma funcion SI se desborda a partir de $n = 62$ aunque el resultado quepa. Verificado de dos formas: contra la lista completa ordenada para n hasta 12 (6556 escenarios) y contra el ranking exacto calculado aparte para n hasta 60 (4005 escenarios). Sin fallos, y los nueve casos del sample dan exacto.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 ll nCr(int n, int k) {
7     if (k < 0 || k > n)
8         return 0;
9
10    k = min(k, n - k);
11
12    ll r = 1;
13
14    for (int i = 1; i <= k; i++) {
15        r = r * (n - i + 1) / i;
16    }
17
18    return r;
19 }
20
21 int main() {
22     ios::sync_with_stdio(false);

```

```
23 cin.tie(nullptr);
24
25 int n, k;
26
27 while (cin >> n >> k) {
28
29     vector<int> a(k);
30
31     for (int &x : a)
32         cin >> x;
33
34     ll god = 0;
35
36     int anterior = 0;
37
38     for (int i = 0; i < k; i++)
39     {
40         for (int j = anterior + 1; j < a[i]; j++)
41         {
42             god += nCr(n - j, k - i - 1);
43         }
44         anterior = a[i];
45     }
46
47     cout << god << endl;
48 }
49
50
51 return 0;
52 }
```

7.14 Suma de Subconjuntos (2^n) - Group Work

C++(Training_Camp_2026/Day 7/C - Group work.py)

Resuelve "Group work" (Day 7, problema C - Contest 07 [Avanzados]): dada una clase de N estudiantes, cuenta cuantos grupos distintos de al menos 2 estudiantes se pueden formar (dos grupos son distintos si difieren en al menos un estudiante). La respuesta es la suma de combinaciones $C(n,m)$ para $m=2..n$ (equivalente a 2^n menos los subconjuntos de tamaño 0 y 1); el código usa `math.comb` para sumar directamente esas combinaciones.

```
1 import math
2 n = int(input())
3
4 suma = 0
5 for m in range(2,n+1):
6     suma+=math.comb(n,m)
7 print(suma)
```

7.15 Técnica de Contribución - Bytelandia's Stones

C++(Training_Camp_2026/Day 9/B - Bytelandia's stones.cpp)

Resuelve "Bytelandia's stones" (Day 9, problema B - Contest 08 [Inicial & Avanzado]): dado un arreglo $a_1..a_n$, se pide la suma de $S(L,R)$ = suma de a_i en $[L,R]$ sobre TODOS los pares (L,R) posibles. En vez de calcular cada subarreglo, usa la técnica de contribución: cada elemento a_i aparece en exactamente $i*(n-i+1)$ subarreglos $[L,R]$ (eligiendo L en $[1,i]$ y R en $[i,n]$), así que la respuesta es la suma de $x * i * (n-i+1)$ para cada valor x leído en la posición i , acumulada en $O(n)$.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int n;
9     cin >> n;
10
11     long long ans = 0;
12
13     for (int i = 1; i <= n; i++)
14     {
15         long long x;
16         cin >> x;
17         ans += x * 1LL * i * (n - i + 1);
18     }
19
20     cout << ans << endl;
```

```
21     return 0;
22 }
```

7.16 Triangulos en Poligono Convexo

C++(Notebook/Triangulos en Poligono Convexo.cpp)

Dado n , cuantos triangulos distintos salen escogiendo 3 vertices de un poligono regular convexo de n lados. La respuesta es simplemente $C(n,3) = n*(n-1)*(n-2)/6$. Con $n = 5$ da 10, que son exactamente los diez del dibujo del enunciado. TODO EL PROBLEMA ESTA EN VER POR QUE NO HAY QUE RESTAR NADA. Contar triangulos con vertices de un conjunto de puntos NO es $C(n,3)$ en general: hay que quitar las ternas COLINEALES, que dan un triangulo degenerado de area cero. Lo que salva este problema es la palabra CONVEXO: en un poligono convexo ningun vertice cae sobre el segmento que une a otros dos, así que no existe una sola terna colineal y las $C(n,3)$ ternas sirven todas. Si el enunciado dijera "puntos en el plano" en vez de "poligono convexo", la solución sería otra: contar las ternas colineales agrupando por dirección y restarlas. Y se cuentan CONJUNTOS de vertices, no formas. En el pentagono los 10 incluyen repetidos por congruencia (5 con dos lados del poligono y 5 con un lado y dos diagonales), y el dibujo los muestra como distintos. Si pidieran formas distintas salvo rotación y reflexión, sería otro problema completamente, de conteo de Burnside. La sucesión son los números tetraedricos: $n = 3, 4, 5, \dots$ da 1, 4, 10, 20, 35, 56, 84, 120, 165, 220. Sirve para reconocerla si aparece en una salida sin saber de donde viene. Verificado contra fuerza bruta (contar las ternas una por una) para n de 0 a 300, y contra enteros exactos para n hasta 2000: sin un solo fallo. LO UNICO QUE PUEDE FALLAR ES EL DESBORDE, y vale saber donde exactamente (medido): - Escribiendo $n*(n-1)*(n-2)/6$ tal cual, el PRODUCTO intermedio se pasa de long long con $n = 2097154$. O sea que la fórmula directa aguanta hasta $n = 2097153$. - Dividiendo ANTES de multiplicar (que es lo que hace c3 aquí abajo) se llega hasta $n = 3810779$, y ahí el que ya no cabe es el RESULTADO, no un paso intermedio. - Conviene tener claro que dividir temprano solo gana un factor 1.82 en n : el techo duro lo pone el resultado, y contra eso no hay orden de operaciones que ayude. Si n es mas grande toca `__int128`, `unsigned`, o responder modulo algo. POR QUE LA DIVISION TEMPRANA ES EXACTA: entre tres enteros consecutivos siempre hay uno múltiplo de 3 y al menos uno par, así que se puede dividir cada factor por separado sin perder nada. Lo que NO se puede es dividir a lo bruto: hay que buscar CUAL de los tres es el múltiplo de 3, porque no siempre es el mismo. Si el problema pide la respuesta modulo un primo p , no se divide: se multiplica por el inverso de 6, o sea $C(n,3) = n*(n-1)*(n-2) * inv6 \% p$, con $inv6 = modpow(6, p-2, p)$.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4
5 // C(n,3) sin formar el producto completo.
6 ll c3(ll n) {
7     if (n < 3) return 0;
8
9     ll a = n, b = n - 1, c = n - 2;
10
11     // Entre tres consecutivos hay exactamente un múltiplo de 3.
12     if (a % 3 == 0) {
13         a /= 3;
14     } else if (b % 3 == 0) {
15         b /= 3;
16     } else {
17         c /= 3;
18     }
19
20     // Y al menos uno par (puede que el de arriba ya haya cambiado la paridad).
21     if (a % 2 == 0) {
22         a /= 2;
23     } else if (b % 2 == 0) {
24         b /= 2;
25     } else {
26         c /= 2;
27     }
28
29     return a * b * c;
30 }
31
32 int main() {
33     ios::sync_with_stdio(false);
34     cin.tie(nullptr);
35
36     ll n;
37     while (cin >> n) {
38         cout << c3(n) << '\n';
39     }
40     return 0;
41 }
```

8 - Game Theory

8.1 Misere Nim y Grundy Acotado

C++\Notebook\Misere Nim y Grundy Acotado.cpp

Dos variantes del Nim que NO se resuelven con el XOR normal (ese caso ya est  cubierto en "Marbles" con Sprague-Grundy).

Misere Nim invierte la condici n de victoria: pierde quien hace el  ltimo movimiento, y la regla cambia solo en el caso degenerado en que todos los montones valen 1, donde gana el primero si la cantidad de montones es par; en cualquier otro caso se aplica el XOR de siempre. Grundy acotado es el atajo para cuando se pueden quitar entre 1 y k piedras: el n mero de Grundy resulta ser exactamente $n \% (k+1)$, as  que no hace falta calcular el mex, y se pierde justo cuando n es m ltiplo de k+1.

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 // devuelve true si gana el primer jugador (pierde quien mueve al final)
6 bool misere_nim(vector<int> a)
7 {
8     bool all_ones = all_of(a.begin(), a.end(), [](int x){ return x == 1; });
9     int xorsum = 0;
10    for (int v : a) xorsum ^= v;
11
12    if (all_ones)
13    {
14        return (a.size() % 2 == 0);
15    }
16    return (xorsum != 0);
17 }
18
19 // se pueden quitar entre 1 y k piedras: Grundy(n) = n % (k+1)
20 int grundy_bounded(int n, int k)
21 {
22     return n % (k + 1);
23 }
24
25 int main()
26 {
27     ios_base::sync_with_stdio(false);
28     cin.tie(nullptr);
29
30     int n, k;
31     cin >> n >> k;
32
33     vector<int> pilas(n);
34     for (int i = 0; i < n; i++) cin >> pilas[i];
35
36     cout << (misere_nim(pilas) ? "First" : "Second") << "\n";
37
38     // en la versi n acotada, la suma de subjuegos sigue siendo el XOR de los Grundy
39     int total = 0;
40     for (int x : pilas) total ^= grundy_bounded(x, k);
41     cout << (total ? "First" : "Second") << "\n";
42
43     return 0;
44 }
```

8.2 Nim con Segment Tree - Juan Vs Frank

C++\Training_Camp_2026\Day 9\Juan Vs Frank.cpp

Resuelve "Juan vs Frank" (Day 9, problema I - Contest 08 [Inicial & Avanzado]): Frank y Juan juegan duelos de Nim sobre un rango de columnas [l,r], donde cada columna es un mont n de fragmentos y en cada turno se retiran fragmentos de una sola columna; pierde quien no puede mover. Por la teor a de Nim, el primer jugador (Frank) gana si y solo si el XOR de los valores del rango es distinto de cero. Mantiene un segment tree que guarda el XOR de rangos (arreglo seg) con soporte de update puntual (rituales que suman x a una columna) y query de rango (profec as), respondiendo "FRANK" si el XOR del rango es != 0 y "JUAN" si es 0.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 int main() {
7     ios::sync_with_stdio(false);
```

```
8     cin.tie(nullptr);
9
10    int N, Q;
11    cin >> N >> Q;
12
13    vector<ll> a(N);
14
15    int SZ = N;
16    vector<ll> seg(2 * SZ);
17
18    for (int i = 0; i < N; i++) {
19        cin >> a[i];
20        seg[SZ + i] = a[i];
21    }
22
23    for (int i = SZ - 1; i > 0; i--)
24        seg[i] = seg[i << 1] ^ seg[i << 1 | 1];
25
26    auto update = [&](int pos, ll val) {
27        pos--;
28        a[pos] += val;
29        int p = pos + SZ;
30        seg[p] = a[pos];
31        while (p >>= 1)
32            seg[p] = seg[p << 1] ^ seg[p << 1 | 1];
33    };
34
35    auto query = [&](int l, int r) {
36        ll ans = 0;
37        l = l - 1 + SZ;
38        r = r + SZ;
39        while (l < r) {
40            if (l & 1) ans ^= seg[l++];
41            if (r & 1) ans ^= seg[--r];
42            l >>= 1;
43            r >>= 1;
44        }
45        return ans;
46    };
47
48    while (Q--) {
49        char op;
50        cin >> op;
51
52        if (op == 'P') {
53            int l, r;
54            cin >> l >> r;
55            if (query(l, r) == 0)
56                cout << "JUAN\n";
57            else
58                cout << "FRANK\n";
59        } else {
60            int k;
61            ll x;
62            cin >> k >> x;
63            update(k, x);
64        }
65    }
66
67    return 0;
68 }
```

8.3 Posiciones Perdedoras (potencias de 2) - Digitsdel

C++\Codigoes\digitsdel.cpp

Determina el ganador de un juego analizando si $N+2$ es potencia de 2. Si lo es, el segundo jugador gana ("mastermei"), si no, el primero ("the greatest").

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 bool isPowerOfTwo(long long x) {
5     return x > 0 && (x & (x - 1)) == 0;
6 }
```

```

7
8 int main() {
9     ios::sync_with_stdio(false);
10    cin.tie(nullptr);
11
12    int T;
13    cin >> T;
14
15    while (T--) {
16        long long N;
17        cin >> N;
18
19        if (isPowerOfTwo(N + 2)) {
20            cout << "the greatest\n";
21        } else {
22            cout << "mastermei\n";
23        }
24    }
25
26    return 0;
27 }

```

8.4 Sprague-Grundy - Marbles

C++ Training, Camp 2020 Day 8 E - Marbles.cpp

Resuelve "Marbles" (Day 8, problema B - Contest 08 [Avanzados]): en un tablero indexado por filas y columnas, un jugador elige una canica en (l, c) y un entero $u > 0$ y la mueve a $(l-u, c)$, $(l, c-u)$ o $(l-u, c-u)$ sin salir del tablero; el primero en llevar una canica a $(0, 0)$ gana y el Emperador siempre juega primero. Dado el reparto inicial de N canicas, hay que decidir si el Emperador puede ganar jugando optimo. Modela el juego como la suma de N subjuegos independientes (teoria de Sprague-Grundy): precomputeGrundy calcula el numero de Grundy de cada posicion (l, c) via mex de posiciones alcanzables (marcando INSTA cuando $l=0, c=0$ o $l=c$, movimiento ganador inmediato), y el XOR de los Grundy de todas las canicas (mas el chequeo instaWin) decide si hay victoria.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 static char buf_[1 << 16];
5 static int bufLen_, bufPos_;
6 inline int gc() {
7     if (bufPos_ == bufLen_) { bufLen_ = fread(buf_, 1, sizeof(buf_), stdin); bufPos_ = 0; if (!bufLen_) return -1; }
8     return buf_[bufPos_++];
9 }
10 inline int readInt() {
11     int c = gc();
12     while (c != -1 && (c < '0' || c > '9') && c != '-') c = gc();
13     bool neg = false; if (c == '-') { neg = true; c = gc(); }
14     int x = 0;
15     while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = gc(); }
16     return neg ? -x : x;
17 }
18
19 const int MAXV = 101; // coordinates go from 0 to 100
20 const int INSTA = 1000000; // sentinel for "instant win" positions: (i,0), (0,i), (i,i)
21 int grundy_[MAXV][MAXV];
22
23 void precomputeGrundy() {
24     // process states in order of increasing max(l,c): every move strictly decreases
25     // max(l,c) or keeps l the same while decreasing c (or vice versa), so processing
26     // by increasing (l+c) is a safe topological order (all moves reduce l+c).
27     vector<int> seen(3 * MAXV + 10, -1); // seen[v] == stamp means v currently marked reachable
28     int stamp = 0;
29
30     vector<pair<int,int>> order;
31     for (int l = 0; l < MAXV; l++)
32         for (int c = 0; c < MAXV; c++)
33             order.push_back({l, c});
34     sort(order.begin(), order.end(), [](const pair<int,int>&a, const pair<int,int>&b){
35         return (a.first + a.second) < (b.first + b.second);
36     });
37
38     for (auto &pr : order) {
39         int l = pr.first, c = pr.second;

```

```

40         if (l == 0 || c == 0 || l == c) {
41             grundy_[l][c] = INSTA;
42             continue;
43         }
44         stamp++;
45         for (int u = 1; u <= max(l, c); u++) {
46             int g;
47             if (l - u >= 0) {
48                 g = grundy_[l - u][c];
49                 if (g < (int)seen.size()) seen[g] = stamp;
50             }
51             if (c - u >= 0) {
52                 g = grundy_[l][c - u];
53                 if (g < (int)seen.size()) seen[g] = stamp;
54             }
55             if (l - u >= 0 && c - u >= 0) {
56                 g = grundy_[l - u][c - u];
57                 if (g < (int)seen.size()) seen[g] = stamp;
58             }
59         }
60         int mex = 0;
61         while (mex < (int)seen.size() && seen[mex] == stamp) mex++;
62         grundy_[l][c] = mex;
63     }
64 }
65
66 int main() {
67     precomputeGrundy();
68
69     int N = readInt();
70     vector<pair<int,int>> marbles(N);
71     for (auto &m : marbles) { m.first = readInt(); m.second = readInt(); }
72
73     bool instaWin = false;
74     for (auto &m : marbles) {
75         if (m.first == m.second) { instaWin = true; break; }
76     }
77
78     if (instaWin) {
79         printf("Y\n");
80         return 0;
81     }
82
83     int x = 0;
84     for (auto &m : marbles) {
85         x ^= grundy_[m.first][m.second];
86     }
87     printf(x != 0 ? "Y\n" : "N\n");
88     return 0;
89 }

```

9 - String

9.1 Brute Force Matching - Enigma

C++ Training, Camp 2020 Day 8 E - Enigma.cpp

Resuelve "Enigma" (Day 8, problema E - Contest 08 [Avanzados]): dado un mensaje cifrado y un crib (palabra que se asume esta en el mensaje original), hay que contar en cuantas posiciones puede empezar el crib dentro del mensaje cifrado, sabiendo que la maquina Enigma nunca reemplaza una letra por si misma (o sea, en una posicion valida ninguna letra del crib puede coincidir con la letra del mensaje en esa misma posicion). Para cada posicion i de arranque, compara letra a letra el crib contra el substring del mensaje de ese largo (variable "temp"). y descarta la posicion apenas encuentra una letra igual (bandera "ok"); cuenta en "god" las posiciones donde ninguna letra coincidio, en fuerza bruta $O(n*m)$.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 int main()
4 {
5     string s1,s2;
6     cin >> s1 >> s2;
7     int god=0;
8
9     for(int i = 0 ; i <= s1.size()-s2.size();i++)
10    {

```

```

11     string tempi = s1.substr(i,s2.size());
12     bool ok = false;
13     for(int j = 0 ; j < s2.size() && ok == false;j++)
14     {
15         if(tempi[j]==s2[j])
16         {
17             ok = true;
18         }
19     }
20     if(ok == false)
21     {
22         god++;
23     }
24 }
25
26 cout << god << endl;
27
28 return 0;
29 }

```

9.2 Chequeo de Palindromo

C++\Notebook\Chequeo de Palindromo.cpp

Las tres formas de preguntar "esto es palindromo", segun cuantas veces lo vayas a preguntar. Para una consulta suelta basta el chequeo de dos punteros en $O(n)$ y sin memoria extra. Si el problema pregunta por muchos rangos $[l,r]$ distintos, conviene precalcular la tabla $pal[i][j]$ en $O(n^2)$ y responder cada consulta en $O(1)$. Y si n es grande y hace falta el palindromo mas largo, ahi ya toca Manacher en $O(n)$. La regla para elegir: cuenta cuantas consultas vas a hacer. Una sola, dos punteros; muchas sobre $n \leq 2000$, la tabla; n de 10^5 o mas, Manacher.

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  // 1) Consulta suelta:  $O(n)$  tiempo,  $O(1)$  memoria.
5  // Se pasa por referencia para no copiar el string en cada llamada.
6  bool esPalindromo(string &s, int l, int r) {
7      while (l < r) {
8          if (s[l] != s[r]) return false;
9          l++;
10         r--;
11     }
12     return true;
13 }
14
15 // El string completo, con la STL:
16 // equal(s.begin(), s.begin() + s.size() / 2, s.rbegin())
17
18 // 2) Muchas consultas de rango: tabla  $O(n^2)$ , cada consulta  $O(1)$ .
19 // pal[i][j] depende del interior [i+1][j-1], asi que i va de DERECHA a
20 // IZQUIERDA para que ese interior ya este resuelto.
21 vector<vector<bool>> tablaPalindromos(const string &s) {
22     int n = s.size();
23     vector<vector<bool>> pal(n, vector<bool>(n, false));
24     for (int i = n - 1; i >= 0; i--)
25         for (int j = i; j < n; j++)
26             if (s[i] == s[j] && (j - i < 2 || pal[i + 1][j - 1]))
27                 pal[i][j] = true;
28     return pal;
29 }
30
31 // 3) Palindromo mas largo con n grande: Manacher,  $O(n)$ .
32 // d1[i] = cuantos palindromos IMPARES centrados en i
33 // d2[i] = cuantos palindromos PARES centrados entre i-1 e i
34 // El impar mas largo centrado en i mide  $2*d1[i]-1$ , el par mide  $2*d2[i]$ .
35 pair<vector<int>, vector<int>> manacher(const string &s) {
36     int n = s.size();
37     vector<int> d1(n), d2(n);
38     for (int i = 0, l = 0, r = -1; i < n; i++) {
39         int k = (i > r) ? 1 : min(d1[l + r - i], r - i + 1);
40         while (i - k >= 0 && i + k < n && s[i - k] == s[i + k]) k++;
41         d1[i] = k--;
42         if (i + k > r) { l = i - k; r = i + k; }
43     }
44     for (int i = 0, l = 0, r = -1; i < n; i++) {
45         int k = (i > r) ? 0 : min(d2[l + r - i + 1], r - i + 1);

```

```

46         while (i - k - 1 >= 0 && i + k < n && s[i - k - 1] == s[i + k]) k++;
47         d2[i] = k--;
48         if (i + k > r) { l = i - k - 1; r = i + k; }
49     }
50     return {d1, d2};
51 }
52
53 int main() {
54     ios::sync_with_stdio(false);
55     cin.tie(nullptr);
56
57     string s;
58     while (cin >> s) {
59         int n = s.size();
60         cout << (esPalindromo(s, 0, n - 1) ? "SI" : "NO") << "\n";
61     }
62     return 0;
63 }
64
65 // OJO con los casos borde del chequeo de dos punteros: con l == r (un solo
66 // caracter) el while no entra y devuelve true, que es lo correcto; con un
67 // rango vacio (l > r) tambien devuelve true. Si en tu problema el palindromo
68 // vacio NO cuenta, filtra el largo ANTES de llamar a la funcion.

```

9.3 KMP para Contar Ocurrencias Solapadas - Judgmental Crowd

C++\VCPC\Regionals 2025\J - Judgmental Crowd.cpp

Resuelve "Judgmental Crowd" (problema J, Regionals 2025): la reaccion del publico se calcula contando cuantas veces aparecen tres palabras en la cadena y combinandolas con pesos, ha - boooo + 3 * bravo EL DETALLE QUE DECIDE EL PROBLEMA es contar ocurrencias SOLAPADAS. Tras un match no se reinicia $j = 0$ sino $j = pi[j-1]$, que deja el automata en el sufijo mas largo que todavia sirve. Con "haha" hay DOS ocurrencias de "ha" y no una; reiniciando en 0 se perderia la segunda cuando los patrones se pisan (con "aa" dentro de "aaa" pasa lo mismo). Se llama a la misma funcion KMP tres veces, una por palabra, porque los patrones son fijos y cortos. Si fueran muchos patrones convendria Aho-Corasick, que los busca todos en una sola pasada. La funcion de prefijos se calcula sobre el PATRON, no sobre el texto: $pi[i]$ es el borde mas largo de $p[0..i]$, y es lo que permite retroceder sin volver atras en el texto. Por eso todo el conteo es $O(|s| + |p|)$. Comprobado: con "haha" da 2, que son las dos ocurrencias de "ha".

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int KMP(const string& s, const string& p) {
5
6      int n = s.size();
7      int m = p.size();
8
9      // Prefix function del patron
10     vector<int> pi(m);
11
12     for (int i = 1; i < m; i++) {
13
14         int j = pi[i - 1];
15
16         while (j > 0 && p[i] != p[j])
17             j = pi[j - 1];
18
19         if (p[i] == p[j])
20             j++;
21
22         pi[i] = j;
23     }
24
25     // Buscar p dentro de s
26     int ans = 0;
27     int j = 0;
28
29     for (int i = 0; i < n; i++) {
30
31         while (j > 0 && s[i] != p[j])
32             j = pi[j - 1];
33
34         if (s[i] == p[j])
35             j++;
36

```

```

37     if (j == m) {
38         ans++;
39     }
40     // Importante para permitir solapamientos
41     j = pi[j - 1];
42 }
43 }
44
45 return ans;
46 }
47
48 int main() {
49     ios::sync_with_stdio(false);
50     cin.tie(nullptr);
51
52     string s;
53
54     cin >> s;
55
56     cout << KMP(s, "ha") - KMP(s, "boooo") + 3*KMP(s, "bravo") << '\n';
57
58     return 0;
59 }

```

9.4 KMP sobre Cadena Recursiva (Duplicacion) - Song Of The Sirens

C++(Codeforces)songOfTheSirens.cpp

Resuelve "Song of the Sirens" (Codeforces, problema G): hay $n+1$ canciones definidas por $s_{i+1} = s_i + t[i] + s_i$, y cada consulta pide cuantas veces aparece un nombre w dentro de s_k , modulo $1e9+7$. La cadena crece al doble en cada paso, así que s_n puede medir $2^n \cdot 100000$ y construirla esta descartado de entrada. LA RECURRENCIA ES TODO: $f(i+1) = 2 \cdot f(i) + g(t[i])$. Las apariciones de w en s_{i+1} son las de la copia izquierda, las de la derecha (de ahí el $2 \cdot f(i)$) y las que CRUZAN el caracter del medio. Esas últimas son g . POR QUE g SOLO DEPENDE DE LA LETRA Y NO DE LA POSICION, que es el paso que hay que ver: una aparicion que cruza el centro tiene largo m , así que no puede meterse mas de $m-1$ caracteres hacia cada lado. O sea que solo depende del SUFIJO de largo $m-1$ de s_i , de la letra del medio, y del PREFIJO de largo $m-1$ de s_i . Y como s_{i+1} empieza y termina con s_i , esos prefijos y sufijos YA NO CAMBIAN en cuanto $|s_i| \geq m-1$. Entonces g se calcula una sola vez para cada una de las 26 letras, contando apariciones de w en (sufijo + letra + prefijo). Por eso primero se hace crecer $cur = s_k$ hasta que mida al menos m . Como se duplica cada vez, eso son a lo sumo unos 20 pasos con $|w|$ hasta 10^6 , no n pasos. DESENNROLLANDO la recurrencia desde ese punto k : $f(n) = f(k) \cdot 2^{n-k} + \text{suma desde } i=k+1 \text{ hasta } n \text{ de } g(t[i-1]) \cdot 2^{n-i}$. Esa suma tiene hasta n terminos, y con q consultas sería $O(n \cdot q) = 10^6 \cdot 10$. El truco para bajarla: como $2^{n-j-1} = 2^{n-1} \cdot 2^{-j}$, se saca el 2^{n-1} afuera y lo que queda, suma de 2^{-j} sobre las posiciones j donde $t[j]$ es la letra c , se precalcula UNA vez en prefijos por letra. Así cada consulta cuesta $O(26)$ en esa parte en vez de $O(n)$. INV2 = 500000004 es el inverso de 2 modulo $1e9+7$, que sale de $(MOD+1)/2$ y evita tener que llamar a una exponenciacion modular. Los 2^{-j} son potencias de ese inverso. DETALLES DEL KMP: despues de un match se hace $j = pi[j-1]$ en vez de $j = 0$, que es lo que permite contar apariciones SOLAPADAS (con $w = "aa"$ en $"aaa"$ hay dos, no una). Y el mismo arreglo pi se reusa para el conteo en cur y para los 26 strings de g , que por eso se pasa como parametro en vez de recalcularlo. Los límites cuadran porque la suma de todos los $|w|$ no pasa de 10^6 : cur mide $\sim 2m$ y cada uno de los 26 strings de g mide $2m-1$, así que el trabajo por consulta es $O(m)$ veces una constante. Si al terminar de crecer todavía $|cur| < m$, es que ni s_n alcanza el largo de w y la respuesta es 0. Este archivo entro al cuaderno sin la verificación habitual, a pedido: ya venia aceptado.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 const ll MOD = 1000000007;
7 const ll INV2 = 500000004;
8
9 vector<int> build_pi(const string& p) {
10     int m = p.size();
11     vector<int> pi(m);
12
13     for (int i = 1; i < m; i++) {
14         int j = pi[i - 1];
15
16         while (j > 0 && p[i] != p[j])
17             j = pi[j - 1];
18
19         if (p[i] == p[j])
20             j++;
21
22         pi[i] = j;

```

```

23     }
24
25     return pi;
26 }
27
28 ll count_occ(const string& s, const string& p, const vector<int>& pi) {
29     int j = 0;
30     ll ans = 0;
31
32     for (char c : s) {
33         while (j > 0 && c != p[j])
34             j = pi[j - 1];
35
36         if (c == p[j])
37             j++;
38
39         if (j == (int)p.size()) {
40             ans++;
41             j = pi[j - 1];
42         }
43     }
44
45     return ans;
46 }
47
48 int main() {
49     ios::sync_with_stdio(false);
50     cin.tie(nullptr);
51
52     int N, Q;
53     cin >> N >> Q;
54
55     string s0, t;
56     cin >> s0 >> t;
57
58     // pow2[i] = 2^i
59     // invpow2[i] = 2^(-i)
60     vector<ll> pow2(N + 1), invpow2(N + 1);
61
62     pow2[0] = 1;
63     invpow2[0] = 1;
64
65     for (int i = 1; i <= N; i++) {
66         pow2[i] = pow2[i - 1] * 2 % MOD;
67         invpow2[i] = invpow2[i - 1] * INV2 % MOD;
68     }
69
70     /*
71     pref[c][i] =
72     suma de 2^(-j) para j < i donde t[j] == c
73     */
74
75     vector<vector<ll>> pref(26, vector<ll>(N + 1));
76
77     for (int c = 0; c < 26; c++) {
78         for (int i = 0; i < N; i++) {
79             pref[c][i + 1] = pref[c][i];
80
81             if (t[i] - 'a' == c)
82                 pref[c][i + 1] =
83                     (pref[c][i + 1] + invpow2[i]) % MOD;
84         }
85     }
86
87     while (Q--) {
88         int n;
89         string w;
90
91         cin >> n >> w;
92
93         int m = w.size();
94
95         vector<int> pi = build_pi(w);

```

```

96
97  /*
98     Construimos s_k hasta que:
99
100     |s_k| >= |w|
101
102     o hasta llegar a s_n si n es menor.
103  */
104
105  string cur = s0;
106
107  int k = 0;
108
109  while (k < n && (int)cur.size() < m) {
110      cur = cur + t[k] + cur;
111      k++;
112  }
113
114  /*
115     Si todavia |cur| < |w|, entonces estamos en s_n
116     y simplemente hacemos KMP directamente.
117  */
118
119  if ((int)cur.size() < m) {
120      cout << 0 << '\n';
121      continue;
122  }
123
124  /*
125     cur = s_k
126  */
127
128  ll base = count_occ(cur, w, pi);
129
130  /*
131     Si k == n, ya tenemos exactamente s_n.
132  */
133
134  if (k == n) {
135      cout << base % MOD << '\n';
136      continue;
137  }
138
139  /*
140     Para calcular g(c):
141
142     suffix(|w|-1) + c + prefix(|w|-1)
143
144     Todas las apariciones de w en este string
145     contienen obligatoriamente el caracter central c.
146  */
147
148  string prefix = cur.substr(0, m - 1);
149  string suffix = cur.substr(cur.size() - (m - 1), m - 1);
150
151  vector<ll> g(26);
152
153  for (int c = 0; c < 26; c++) {
154      string x;
155      x.reserve(2 * m - 1);
156
157      x += suffix;
158      x += char('a' + c);
159      x += prefix;
160
161      g[c] = count_occ(x, w, pi);
162  }
163
164  /*
165     f(n,w) =
166     f(k,w) * 2^(n-k)
167     +
168     sum_{i=k+1}^n g(i,w) * 2^(n-i)

```

```

69
70     Como g(i,w) depende unicamente de t[i-1]:
71
72     g(i,w) = g[t[i-1]]
73
74     */
75
76     ll ans = base * pow2[n - k] % MOD;
77
78     /*
79     Queremos:
80
81     sum_{j=k}^{n-1}
82     g[t[j]] * 2^(n-j-1)
83
84     = 2^(n-1) *
85     sum_{j=k}^{n-1}
86     g[t[j]] * 2^(-j)
87
88     */
89
90     ll sum = 0;
91
92     for (int c = 0; c < 26; c++) {
93         ll x = (pref[c][n] - pref[c][k] + MOD) % MOD;
94         sum = (sum + g[c] * x) % MOD;
95     }
96
97     ans = (ans + sum * pow2[n - 1]) % MOD;
98
99     cout << ans << '\n';
100 }
101
102 return 0;
103 }

```

9.5 Maximo Solape Sufijo-Prefijo (KMP) - Sign

C++/CCPL/sign.cpp

Resuelve "Scrolling Sign" (problema H, CCPL): un letrero muestra k caracteres; en cada paso todo se corre una posicion a la izquierda y entra una letra nueva por la derecha. Hay que mostrar w palabras de k letras en orden, metiendo la menor cantidad de letras posible. TODA LA REDUCCION ESTA EN UNA OBSERVACION: si estando en la palabra A se meten t letras, el letrero queda mostrando $A[t..k-1]$ seguido de las t nuevas. Para que eso sea B hace falta que $A[t..k-1]$ sea igual a $B[0..k-t-1]$, o sea que un SUFJO de A de largo $k-t$ coincida con el PREFIJO de B de ese mismo largo. Entonces el costo minimo de pasar de A a B es $k -$ (mayor L tal que el sufijo de A de largo L es prefijo de B) y el total es k por la primera palabra (el letrero arranca vacio) mas esos costos. Que el enunciado permita mostrar palabras intermedias que no son del mensaje no cambia nada: esas son justamente los estados por los que se pasa mientras entran las t letras. Y lo de "si una palabra se repite seguida, se muestra una sola vez" sale gratis: si $A == B$ el solape es k y el costo es 0 , sin escribir ningun caso especial. COMO SE SACA EL SOLAPE SIN PAGAR $O(k^2)$: se arma la cadena $B + \# + A$ y se calcula la funcion de fallo de KMP; su ultimo valor es el mayor prefijo de B que ademas es sufijo de A , que es exactamente L . El separador tiene que ser un caracter que NO aparezca en las palabras (aqui son mayusculas, asi que '#' sirve), porque si no el fallo podria cruzar de una mitad a la otra y devolver un valor mayor que k , que no corresponde a ningun solape real. Probar los L de k hacia abajo comparando substrings seria $O(k^2)$ por par y $O(w*k^2)$ en total; con $k = w = 1500$ eso son $3*10^9$ y no pasa. Con KMP es $O(k)$ por par, $O(w*k) = 2.25*10^6$. Medido en el peor caso del enunciado ($k = 1500$, $w = 1500$, palabras casi identicas para que el fallo retroceda lo mas posible), 20 casos completos: 512 ms. Verificado contra una simulacion REAL del letrero, con BFS sobre los estados posibles, en 1500 casos con k de 1 a 4; y el solape de KMP contra comparar substrings a lo bruto en 20000 pares. Cero diferencias en ambas. Los dos samples dan 5 y 5, y el ejemplo del enunciado (CAT ATE TED con $k = 3$, que se logra metiendo CATED) da 5.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 // Mayor L tal que los ultimos L caracteres de a son los primeros L de b.
5 int solape(const string &a, const string &b) {
6     string s = b + "#" + a;
7     int m = s.size();
8
9     vector<int> f(m, 0);
10
11     for (int i = 1; i < m; i++) {
12         int j = f[i - 1];
13
14         while (j > 0 && s[i] != s[j])
15             j = f[j - 1];
16

```

```

7         if (s[i] == s[j])
8             j++;
9
10        f[i] = j;
11    }
12
13    return f[m - 1];
14 }
15
16 int main() {
17     ios::sync_with_stdio(false);
18     cin.tie(nullptr);
19
20     int casos;
21     cin >> casos;
22
23     while (casos--) {
24         int k, w;
25         cin >> k >> w;
26
27         vector<string> pal(w);
28         for (int i = 0; i < w; i++) cin >> pal[i];
29
30         // El letrero arranca vacío, así que la primera palabra cuesta k.
31         long long total = k;
32
33         for (int i = 1; i < w; i++) {
34             total += k - solape(pal[i - 1], pal[i]);
35         }
36
37         cout << total << '\n';
38     }
39     return 0;
40 }

```

9.6 Monotonic Stack - Remove Duplicates1

C++ \LeetCode\removeDuplicates1.cpp

Resuelve "Remove Duplicate Letters": devuelve el menor string lexicográfico que contiene cada letra de s exactamente una vez.

Usa una pila monotona, sacando letras mayores mientras todavia aparezcan mas adelante (cnt > 0) y no esten ya en el resultado.

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 class Solution {
6 public:
7     string removeDuplicateLetters(string s) {
8         // cnt for storing frequency of characters
9         // vis for marking visited characters
10        vector<int> cnt(26,0) , vis(26,0);
11
12        string res = "";
13        int n = s.size();
14
15        for(int i = 0; i<n; ++i)
16            cnt[s[i] - 'a']++;
17
18        for(int i = 0; i<n; ++i)
19        {
20            // decrease cnt of current character
21            cnt[s[i] - 'a']--;
22
23            // If character is not already
24            // in answer
25            if(!vis[s[i]- 'a'])
26            {
27                // Last character > s[i]
28                // and its count > 0
29                while(res.size() > 0 && res.back() > s[i] && cnt[res.back() - 'a'] > 0)
30                {
31                    // marking letter visited
32                    vis[res.back() - 'a'] = 0;
33                    res.pop_back();

```

```

34                }
35
36                // Add s[i] in res and
37                // mark it visited
38                res += s[i];
39                vis[s[i] - 'a'] = 1;
40            }
41        }
42        // return resultant string
43        return res;
44    }
45 };

```

9.7 Palindromo Ignorando Símbolos - Just Palindromes

C++ \ICPC\2023J - Just Palindromes.cpp

Resuelve "Just Palindromes!" (problema J, ICPC 2025): decir si una frase es palindroma ignorando espacios, puntuacion y mayusculas. Se filtra dejando solo letras y pasandolas a minúscula, y despues es el chequeo de dos punteros de siempre. Lo unico que hay que cuidar es la lectura: la frase trae ESPACIOS, así que va con getline y no con >>, que cortaria en el primer espacio.

DOS BORDES QUE EL ENUNCIADO PERMITE Y QUE CONVIENE PROBAR: - La línea VACÍA (dice $0 \leq N$). Al filtrar queda la cadena vacía, que es palindroma: Y. - Una línea de puros símbolos, como "----" del sample. También queda vacía: Y. Con la cadena vacía, $r = t.size() - 1$ vale -1 y el while no entra, así que responde Y sin tocar memoria. Sale bien por como esta escrito el ciclo, pero vale saber que $t.size()$ es SIN SIGNO: si en vez del while hubiera un for con $\leq$ y restas, ese -1 se volvería un número enorme. El enunciado acota los caracteres a ASCII imprimible, así que isalpha y tolower no dan problemas de localización.

Verificado contra fuerza bruta en 5008 frases, incluidas la vacía, las de solo símbolos y palindromos armados a proposito con basura intercalada y mayusculas mezcladas.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     string s;
9
10    while (getline(cin, s) && s != "") {
11
12        string t;
13
14        // Dejamos unicamente letras y pasamos a minusculas
15        for (char c : s) {
16            if (isalpha(c))
17                t += tolower(c);
18        }
19
20        bool ok = true;
21
22        int l = 0;
23        int r = t.size() - 1;
24
25        while (l < r) {
26            if (t[l] != t[r]) {
27                ok = false;
28                break;
29            }
30
31            l++;
32            r--;
33        }
34
35        cout << (ok ? 'Y' : 'N') << '\n';
36    }
37
38    return 0;
39 }

```

9.8 Palindromos No Solapados (Greedy) - Max Palindromes

C++ (LeetCode) maxPalindromes.cpp

Resuelve "Maximum Number of Non-overlapping Palindrome Substrings" (LeetCode 2472): dado s y un k , hay que escoger la mayor cantidad posible de subcadenas palindromas que no se solapen y que midan AL MENOS k . Barre r de izquierda a derecha y, apenas encuentra un palindromo valido que termine en r y empiece en o despues de "inicio", lo toma y salta con inicio = $r+1$. El greedy es optimo porque cerrar lo antes posible nunca quita opciones: deja el sufijo mas largo disponible para lo que venga. Cual l se escoja dentro del mismo r da igual, porque el corte queda en r . Detalle: r arranca en $k-1$, que es la primera posicion donde cabe algo de largo k . Sin tabla DP a proposito. Parece $O(n^3)$ pero en la practica es lo mas rapido: el chequeo de dos punteros aborta al primer par que no coincide (medido: ~2 pasos por llamada en promedio), y una coincidencia profunda implicaria un palindromo largo, que el greedy se llevaria de una, avanzando inicio. Medido con $n = 2000$: el peor adversario encontrado son 7.5 ms, contra 1-4 ms constantes de la version con tabla $pal[i][j]$, que ademas reserva n^2 bits de memoria. Si necesitas consultar MUCHOS rangos sueltos en vez de barrer una vez, ahí sí conviene la tabla: esta en la ficha "Chequeo de Palindromo".

```
1 class Solution {
2 public:
3     bool esPalindromo(string &s, int l, int r) {
4         while (l < r) {
5             if (s[l] != s[r]) return false;
6             l++;
7             r--;
8         }
9         return true;
10    }
11
12    int maxPalindromes(string s, int k) {
13        int n = s.size();
14        int ans = 0;
15        int inicio = 0;
16
17        for (int r = k - 1; r < n; r++) {
18            for (int l = inicio; l <= r - k + 1; l++) {
19                if (esPalindromo(s, l, r)) {
20                    ans++;
21                    inicio = r + 1;
22                    break;
23                }
24            }
25        }
26
27        return ans;
28    }
29 };
```

9.9 Parentesis Balanceados (Todas las Variantes)

C++ (Notebook) Parentesis Balanceados.cpp

Todo problema de parentesis sale de UNA idea: el BALANCE, +1 por cada '(' y -1 por cada ')'. Una cadena es valida si y solo si el balance nunca baja de 0 y termina en 0. Las variantes de abajo son esa misma idea mirada desde distintos angulos. Todas verificadas contra fuerza bruta. CUAL USAR SEGUN LO QUE PIDAN: 1 valido un solo tipo: basta un contador, sin pila, $O(n)$ y $O(1)$ de memoria. 2 validoTipos $O(n)$: ahí SI hace falta pila, porque "()" tiene balance bien en cada tipo por separado y aun así es invalida. 3 parejas/profundidad la pareja de cada parentesis (para recorrer la estructura como un arbol) y el anidamiento maximo, que es el balance maximo. 4 minInserciones los ')' que llegan sin '(' abierto piden uno, y los '(' que quedan abiertos al final piden un ')'. Suma de sueltos. 5 minVolteos cambiar '(' por ')' o al revés. Tras cancelar las parejas queda "" ("": con c cerrados y a abiertos sueltos son $\text{ceil}(c/2) + \text{ceil}(a/2)$. Con largo impar es imposible. 6 borrarMinimo se marcan los sueltos (los ')' sin pareja y los '(' que quedan en la pila) y se borran. Sirve aunque la cadena tenga letras mezcladas. 7 masLarga la pila guarda el indice ANTES de donde empieza el tramo valido actual; arranca en -1 y cada ')' suelto pasa a ser el nuevo tope. 8 contarSubcadenas si el ')' de i casa con el '(' de j , las validas que terminan en i son (...) mas cada valida que terminaba justo en $j-1$: $dp[i+1] = dp[j+1]$. 9 validoComodines '*' vale '(', ')' o nada: se lleva el balance MINIMO y el MAXIMO posibles. Si el maximo baja de 0 no hay forma; el minimo se sube a 0 porque nunca conviene pasar por negativo. Al final el minimo debe ser 0. 10 generar backtracking: se pone '(' mientras queden, y ')' si hay mas abiertos que cerrados. Sale en orden lexicografico y son Catalan(n). 11 kEsima sin generarlas todas: formas $[i][b]$ = cuantas maneras hay de completar i caracteres desde balance b . Si k cabe en las que empiezan con '(', se pone '('; si no, se descuentan y se pone ')'. Hasta n de unos 30. 12 SegTree "subsecuencia balanceada mas larga en $s[L..r]$ " con muchas consultas. Cada nodo guarda (parejas, '(' sobrantes, ')' sobrantes); al unir, los '(' sobrantes de la izquierda casan con los ')' sobrantes de la derecha. 13 maxSubsecuenciaTipos con varios tipos y hay que BORRAR o INSERTAR, el greedy de pila ya no es optimo: DP de intervalos $O(n^3)$. El minimo de inserciones para balancear es n menos ese largo. Contar las secuencias validas de n pares es Catalan(n): ver la ficha de Combinatoria.

```
1 #include <bits/stdc++.h>
2 using namespace std;
```

```
3 typedef long long ll;
4
5 // 1) Un solo tipo.
6 bool valido(const string &s) {
7     int bal = 0;
8     for (char c : s) {
9         if (c == '(') bal++;
10        else bal--;
11        if (bal < 0) return false;
12    }
13    return bal == 0;
14 }
15
16 // 2) Varios tipos: la pila guarda los que abren.
17 bool validoTipos(const string &s) {
18     string pila;
19     for (char c : s) {
20         if (c == '(' || c == '[' || c == '{') {
21             pila.push_back(c);
22             continue;
23         }
24         char abre = '(';
25         if (c == ']') abre = '[';
26         if (c == '}') abre = '{';
27         if (pila.empty() || pila.back() != abre) return false;
28         pila.pop_back();
29     }
30     return pila.empty();
31 }
32
33 // 3) Pareja de cada parentesis (-1 si queda suelto) y anidamiento maximo.
34 vector<int> parejas(const string &s) {
35     vector<int> match(s.size(), -1), pila;
36     for (int i = 0; i < (int)s.size(); i++) {
37         if (s[i] == '(') {
38             pila.push_back(i);
39         } else if (!pila.empty()) {
40             match[i] = pila.back();
41             match[pila.back()] = i;
42             pila.pop_back();
43         }
44     }
45     return match;
46 }
47
48 int profundidad(const string &s) { // para cadenas validas
49     int bal = 0, mx = 0;
50     for (char c : s) {
51         if (c == '(') bal++;
52         else bal--;
53         mx = max(mx, bal);
54     }
55     return mx;
56 }
57
58 // 4) Minimo de inserciones para balancear.
59 int minInserciones(const string &s) {
60     int abiertos = 0, faltan = 0;
61     for (char c : s) {
62         if (c == '(') abiertos++;
63         else if (abiertos > 0) abiertos--;
64         else faltan++; // ')' sin pareja: pide un '('
65     }
66     return abiertos + faltan; // cada '(' abierto pide un ')'
67 }
68
69 // 5) Minimo de volteos '(' <-> ')' para balancear; -1 si es imposible.
70 int minVolteos(const string &s) {
71     if (s.size() % 2) return -1;
72     int abiertos = 0, cerrados = 0;
73     for (char c : s) {
74         if (c == '(') abiertos++;
75         else if (abiertos > 0) abiertos--;
```

```

76     else cerrados++;
77 }
78 return (cerrados + 1) / 2 + (abiertos + 1) / 2;
79 }
80
81 // 6) Borrar el mínimo para que quede valida (devuelve una de las optimas).
82 string borrarMinimo(const string &s) {
83     vector<int> pila;
84     vector<bool> borrar(s.size(), false);
85     for (int i = 0; i < (int)s.size(); i++) {
86         if (s[i] == '(') {
87             pila.push_back(i);
88         } else if (s[i] == ')') {
89             if (!pila.empty()) pila.pop_back();
90             else borrar[i] = true;
91         }
92     }
93     for (int i : pila) borrar[i] = true;
94     string res;
95     for (int i = 0; i < (int)s.size(); i++)
96         if (!borrar[i]) res += s[i];
97     return res;
98 }
99
100 // 7) Largo de la subcadena valida mas larga.
101 int masLarga(const string &s) {
102     vector<int> pila = {-1}; // indice antes del tramo valido
103     int mejor = 0;
104     for (int i = 0; i < (int)s.size(); i++) {
105         if (s[i] == '(') {
106             pila.push_back(i);
107             continue;
108         }
109         pila.pop_back();
110         if (pila.empty()) pila.push_back(i); // ')' suelto: nuevo tope
111         else mejor = max(mejor, i - pila.back());
112     }
113     return mejor;
114 }
115
116 // 8) Cantidad de subcadenas validas (distintas por posicion).
117 ll contarSubcadenas(const string &s) {
118     int n = s.size();
119     vector<ll> dp(n + 1, 0); // dp[i+1] = validas que terminan en i
120     vector<int> pila;
121     ll total = 0;
122     for (int i = 0; i < n; i++) {
123         if (s[i] == '(') {
124             pila.push_back(i);
125             continue;
126         }
127         if (pila.empty()) continue;
128         int j = pila.back();
129         pila.pop_back();
130         dp[i + 1] = dp[j] + 1;
131         total += dp[i + 1];
132     }
133     return total;
134 }
135
136 // 9) Con comodines '*' que valen '(' , ')' o nada.
137 bool validoComodines(const string &s) {
138     int lo = 0, hi = 0; // balance minimo y maximo posibles
139     for (char c : s) {
140         if (c == '(') {
141             lo++;
142             hi++;
143         } else if (c == ')') {
144             lo--;
145             hi--;
146         } else {
147             lo--;
148             hi++;
149         }
150     }
151     return lo <= 0 & hi <= 0;
152 }

```

```

153 }
154 if (hi < 0) return false;
155 lo = max(lo, 0);
156 }
157 return lo == 0;
158 }
159
160 // 10) Todas las validas de n pares, en orden lexicografico.
161 void generar(int n, int abiertos, int cerrados, string &cur, vector<string> &res) {
162     if ((int)cur.size() == 2 * n) {
163         res.push_back(cur);
164         return;
165     }
166     if (abiertos < n) {
167         cur.push_back('(');
168         generar(n, abiertos + 1, cerrados, cur, res);
169         cur.pop_back();
170     }
171     if (cerrados < abiertos) {
172         cur.push_back(')');
173         generar(n, abiertos, cerrados + 1, cur, res);
174         cur.pop_back();
175     }
176 }
177
178 // 11) La k-esima valida de n pares (k desde 1), en orden lexicografico.
179 string kEsima(int n, ll k) {
180     vector<vector<ll>> formas(2 * n + 1, vector<ll>(2 * n + 2, 0));
181     formas[0][0] = 1;
182     for (int i = 1; i <= 2 * n; i++)
183         for (int b = 0; b <= 2 * n; b++) {
184             formas[i][b] = formas[i - 1][b + 1]; // poner '('
185             if (b > 0) formas[i][b] += formas[i - 1][b - 1]; // poner ')'
186         }
187     string res;
188     int b = 0;
189     for (int i = 2 * n; i > 0; i--) {
190         ll conAbre = formas[i - 1][b + 1];
191         if (k <= conAbre) {
192             res += '(';
193             b++;
194         } else {
195             k -= conAbre;
196             res += ')';
197             b--;
198         }
199     }
200     return res;
201 }
202
203 // 12) Subsecuencia balanceada mas larga en s[l..r], con muchas consultas.
204 struct Nodo {
205     int par, abre, cierra;
206 };
207
208 Nodo unir(Nodo a, Nodo b) {
209     int nuevos = min(a.abre, b.cierra);
210     return {a.par + b.par + nuevos, a.abre + b.abre - nuevos,
211             a.cierra + b.cierra - nuevos};
212 }
213
214 struct SegTree {
215     int n;
216     vector<Nodo> t;
217
218     SegTree(const string &s) : n(s.size()), t(4 * s.size()) {
219         build(1, 0, n - 1, s);
220     }
221
222     void build(int p, int l, int r, const string &s) {
223         if (l == r) {
224             if (s[l] == '(') t[p] = {0, 1, 0};
225             else t[p] = {0, 0, 1};
226         }

```

```

202     return;
203 }
204 int m = (1 + r) / 2;
205 build(2 * p, l, m, s);
206 build(2 * p + 1, m + 1, r, s);
207 t[p] = unir(t[2 * p], t[2 * p + 1]);
208 }
209
210 Nodo query(int p, int l, int r, int ql, int qr) {
211     if (ql <= l && r <= qr) return t[p];
212     int m = (1 + r) / 2;
213     if (qr <= m) return query(2 * p, l, m, ql, qr);
214     if (ql > m) return query(2 * p + 1, m + 1, r, ql, qr);
215     return unir(query(2 * p, l, m, ql, qr), query(2 * p + 1, m + 1, r, ql, qr));
216 }
217
218 int maxBalanceada(int l, int r) { // indices desde 0
219     return 2 * query(1, 0, n - 1, l, r).par;
220 }
221 };
222
223 // 13) Varios tipos: subsecuencia balanceada mas larga (DP de intervalos).
224 bool casan(char a, char b) {
225     return (a == '(' && b == ')') || (a == '[' && b == ']') || (a == '{' && b == '}');
226 }
227
228 int maxSubsecuenciaTipos(const string &s) {
229     int n = s.size();
230     if (n == 0) return 0;
231     vector<vector<int>> dp(n, vector<int>(n, 0));
232     for (int len = 2; len <= n; len++)
233         for (int i = 0; i + len - 1 < n; i++) {
234             int j = i + len - 1, mejor = 0;
235             if (casan(s[i], s[j])) mejor = dp[i + 1][j - 1] + 2;
236             for (int k = i; k < j; k++) mejor = max(mejor, dp[i][k] + dp[k + 1][j]);
237             dp[i][j] = mejor;
238         }
239     return dp[0][n - 1];
240 }
241
242 int main() {
243     ios::sync_with_stdio(false);
244     cin.tie(nullptr);
245
246     string s;
247     while (cin >> s) {
248         cout << s << ": valida=" << valido(s)
249             << " inserciones=" << minInserciones(s)
250             << " masLarga=" << masLarga(s)
251             << " subcadenas=" << contarSubcadenas(s)
252             << " queda=" << borrarMinimo(s) << '\n';
253     }
254     return 0;
255 }

```

9.10 Partition and Permutation - Policia Nacional

C++\Codeforces\policia Nacional.cpp

Divide una cadena en 3 partes, las reorganiza y verifica si forma otra cadena objetivo. Si es posible, imprime ambas descomposiciones ordenadas.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int n;
9     cin >> n;
10
11     string s1, s2;
12     cin >> s1 >> s2;
13

```

```

14 // Proban todos los cortes posibles:
15 // A = [0,i)
16 // B = [i,j)
17 // C = [j,n)
18
19 for(int i = 1; i < n-1; i++)
20 {
21     for(int j = i+1; j < n; j++)
22     {
23         string A = s1.substr(0, i);
24         string B = s1.substr(i, j-i);
25         string C = s1.substr(j);
26
27         vector<string> p = {A,B,C};
28
29         sort(p.begin(), p.end());
30
31         do
32         {
33             string formed = p[0] + p[1] + p[2];
34
35             if(formed == s2)
36             {
37                 cout << "YES\n";
38
39                 // Descomposición original del pasaporte
40                 cout << A << " " << B << " " << C << "\n";
41
42                 // Descomposición de la visa
43                 cout << p[0] << " " << p[1] << " " << p[2] << "\n";
44
45                 return 0;
46             }
47         } while(next_permutation(p.begin(), p.end()));
48     }
49 }
50
51 cout << "NO\n";
52
53 return 0;
54
55 }

```

9.11 Remove Duplicate Letters - Remove Duplicates

C++\LeetCode\removeDuplicates.cpp

Intento manual (greedy) de "Remove Duplicate Letters": busca construir el menor string lexicografico que contiene cada letra una sola vez, escogiendo la siguiente letra mas pequena posible que aun permita completar las letras restantes. Version explicativa de la idea greedy; la implementacion limpia con pila monotona esta en removeDuplicates1.cpp.

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 class Solution {
6 public:
7     string removeDuplicateLetters(string s) {
8         vector<int> nums(s.size());
9         bool letras[26] = {false};
10         int cont[26] = {0};
11         for(char c : s)
12         {
13             nums.push_back(c - '0');
14             cont[c-'a']++;
15             letras[c - 'a'] = true;
16         }
17
18         int letmax = 0;
19         for (int i = 0; i < 26; i++)
20         {
21             letmax += letras[i];
22         }
23
24         char ini;

```

```

25     for (int i = 0; i < 26; i++)
26     {
27         if(cont[i] > 0)
28         {
29             ini = i + 'a';
30             break;
31         }
32     }
33
34
35     string god="";
36     int idx = 0;
37     bool contGod[26] = {false};
38     for(int i = 0 ; i < s.size() || god.size() < letmax; i++)
39     {
40         if(s[i] == ini && contGod[s[i]-'a'] == false)
41         {
42             idx = i + 1;
43             for (int i = ini - 'a' + 1; i < 26; i++)
44             {
45                 if(cont[i] > 0 && contGod[i] == false)
46                 {
47                     ini = i + 'a';
48                     break;
49                 }
50             }
51             god+=s[i];
52             contGod[s[i]-'a'] = true;
53         }
54         else if(cont[s[i]-'a']==1 && contGod[s[i]-'a'] == false)
55         {
56             char mini = s[i];
57             int idxMini = i;
58             while(idx < i)
59             {
60                 for(int j = idx; j < i; j++)
61                 {
62                     if(s[j] < mini && contGod[s[j]-'a'] == false)
63                     {
64                         mini = s[j];
65                         idxMini = j;
66                     }
67                 }
68                 if(mini != s[i])
69                 {
70                     god+= s[idxMini];
71                     contGod[s[idxMini]-'a'] = true;
72
73                     idx = idxMini + 1;
74                     mini = s[i];
75                     idxMini = i;
76
77                 }
78                 else
79                 {
80                     break;
81                 }
82             }
83
84             for(int j = idx; j <= i; j++)
85             {
86                 if(s[j] == s[i])
87                 {
88                     god+=s[j];
89                     idx = j + 1;
90                     contGod[s[j]-'a'] = true;
91                     break;
92                 }
93             }
94         }
95     }
96     cont[s[i]-'a']--;
97 }

```

```

98
99     return god;
100 }
101 };

```

9.12 Rotacion Minima (Dos Punteros) - Fingerprints

C++\ICPC\2025\F - Fingerprints.cpp

Resuelve "Fingerprints" (problema F, ICPC 2025): las huellas se guardan de forma circular, así que ABCD, BCDA, CDAB y DABC son la misma. Hay que quedarse con una sola copia de cada huella distinta, representada por su rotación lexicográficamente menor. La rotación mínima sale en $O(n)$ con dos candidatos i y j sobre la cadena duplicada: se comparan carácter por carácter y, al diferir en la posición k , el perdedor salta $k+1$ posiciones de una. Ese salto es lo que la hace lineal, y esta explicada con detalle en la ficha "Minimal Rotation" de CSES, en este mismo cuaderno. EL ERROR QUE TENIA ESTE ARCHIVO ERA DE ORDEN, NO DE ALGORITMO: guardaba las canonicas en un set<string> y las imprimía recorriendolo, o sea en orden ALFABETICO. El enunciado pide el orden en que aparecen en la ENTRADA, y el sample lo delata: con ABCD, BCDA, XYZ, FFFF, ZXY la respuesta es ABCD XYZ FFFF y el set devolvía ABCD FFFF XYZ. La solución es la de siempre para "sin repetidos pero en orden de llegada": un set aparte solo para saber si ya se vio, y un vector que conserva el orden. El insert(...) second dice en una sola operación si era nueva. Verificado contra fuerza bruta (rotación mínima comparando todas las rotaciones) en 500 casos. Medida: 3 casos de 100 huellas de 5000 caracteres en 33 ms.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 string minimalRotation(string s) {
5     int n = s.size();
6
7     s += s;
8
9     int i = 0, j = 1;
10
11     while (i < n && j < n) {
12         int k = 0;
13
14         while (k < n && s[i + k] == s[j + k])
15             k++;
16
17         if (k == n)
18             break;
19
20         if (s[i + k] > s[j + k])
21             i = i + k + 1;
22         else
23             j = j + k + 1;
24
25         if (i == j)
26             j++;
27     }
28
29     int pos = min(i, j);
30
31     return s.substr(pos, n);
32 }
33
34 int main() {
35     ios::sync_with_stdio(false);
36     cin.tie(nullptr);
37
38     int n;
39
40     while (cin >> n && n != 0) {
41
42         // El set solo sirve para detectar repetidas. El ORDEN de salida es el
43         // de aparición en la entrada, así que se guarda aparte en un vector:
44         // imprimir el set directamente las sacaría en orden alfabético.
45         set<string> vistas;
46         vector<string> ans;
47
48         for (int i = 0; i < n; i++) {
49             int m;
50             string s;
51
52             cin >> m >> s;
53

```

```

54         string canon = minimalRotation(s);
55
56         if (vistas.insert(canon).second)
57             ans.push_back(canon);
58     }
59
60     for (const string &s : ans)
61         cout << s << '\n';
62 }
63
64 return 0;
65 }

```

9.13 Simulation - Process Str

C++11 LeetCode/processStr.cpp

Procesa una cadena aplicando operaciones sobre un string acumulador: '*' borra el ultimo caracter, '#' duplica el contenido actual, '%' lo invierte, y cualquier otro caracter se agrega al final. Devuelve el string resultante.

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  class Solution {
6  public:
7      string processStr(string s) {
8          string god = "";
9          for(char a : s)
10             {
11                 if(a == '*')
12                     {
13                         if(god.size()>0)
14                             {
15                                 god.erase(god.size()-1);
16                             }
17                     }
18                 else if(a == '#')
19                     {
20                         god += god;
21                     }
22                 else if(a == '%')
23                     {
24                         reverse(god.begin(),god.end());
25                     }
26                 else
27                     {
28                         god += a;
29                     }
30             }
31         return god;
32     }
33 };

```

9.14 Suffix Array + LCP - Couple Of Bip Bop

C++11 Training_Camp_2026/Day_3/C - Couple of Bip Bop.cpp

Resuelve "Couple of BipBop" (Day 3, problema C - Maratona SBC de Programacao 2024): Bob y Charlie empiezan a bailar cada uno en un verso aleatorio de una cancion de N versos y siguen la coreografia hasta desincronizarse o llegar al final de la cancion; hay que calcular el valor esperado de versos que bailaran en sincronia, expresado como fraccion irreducible P/Q , si ambos eligen su verso inicial de forma uniforme e independiente. El valor esperado equivale a sumar, sobre todos los pares de posiciones iniciales (i,j) , el largo del prefijo comun de los sufijos que empiezan en i y en j . El codigo construye el suffix array (suffix_array) y el arreglo de LCP (kasai) de la secuencia de movimientos, y luego usa una pila monotona para sumar en $O(N)$ la suma de minimos de LCP sobre todos los subarreglos (sumPairs), equivalente a la suma de LCP entre todos los pares de sufijos.

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  using ll = long long;
5
6  ll gcdll(ll a, ll b) {
7      while (b) {
8          a %= b;
9          swap(a, b);
10     }
11     return a;

```

```

12 }
13
14 vector<int> suffix_array(vector<int> s) {
15     int n = s.size();
16     vector<int> sa(n), rnk(n), tmp(n);
17
18     for (int i = 0; i < n; i++) {
19         sa[i] = i;
20         rnk[i] = s[i];
21     }
22
23     for (int k = 1;; k <= 1) {
24         auto cmp = [&](int a, int b) {
25             if (rnk[a] != rnk[b])
26                 return rnk[a] < rnk[b];
27
28             int ra = a + k < n ? rnk[a + k] : -1;
29             int rb = b + k < n ? rnk[b + k] : -1;
30             return ra < rb;
31         };
32
33         sort(sa.begin(), sa.end(), cmp);
34
35         tmp[sa[0]] = 0;
36
37         for (int i = 1; i < n; i++)
38             tmp[sa[i]] = tmp[sa[i-1]] + cmp(sa[i-1], sa[i]);
39
40         rnk = tmp;
41
42         if (rnk[sa[n-1]] == n - 1)
43             break;
44     }
45
46     return sa;
47 }
48
49 vector<int> kasai(vector<int> &s, vector<int> &sa) {
50     int n = s.size();
51
52     vector<int> rank(n);
53     for (int i = 0; i < n; i++)
54         rank[sa[i]] = i;
55
56     vector<int> lcp(n - 1);
57
58     int h = 0;
59
60     for (int i = 0; i < n; i++) {
61         int r = rank[i];
62
63         if (r == n - 1)
64             continue;
65
66         int j = sa[r + 1];
67
68         while (i + h < n && j + h < n && s[i+h] == s[j+h])
69             h++;
70
71         lcp[r] = h;
72
73         if (h)
74             h--;
75     }
76
77     return lcp;
78 }
79
80 int main() {
81     ios::sync_with_stdio(false);
82     cin.tie(nullptr);
83 }

```

```

85 int N;
86 cin >> N;
87
88 vector<int> a(N);
89
90 for (int i = 0; i < N; i++)
91     cin >> a[i];
92
93 auto sa = suffix_array(a);
94 auto lcp = kasai(a, sa);
95
96
97 // suma de LCP entre pares distintos
98 ll sumPairs = 0;
99
100 vector<pair<ll,ll>> st;
101 ll accumulated = 0;
102
103 for (int i = 0; i < (int)lcp.size(); i++) {
104
105     ll cnt = 1;
106     ll val = lcp[i];
107
108     while (!st.empty() && st.back().first >= val) {
109         accumulated -= st.back().first * st.back().second;
110         cnt += st.back().second;
111         st.pop_back();
112     }
113
114     st.push_back({val, cnt});
115
116     accumulated += val * cnt;
117
118     sumPairs += accumulated;
119 }
120
121 ll numerator = 1LL * N * (N + 1) / 2 + 2 * sumPairs;
122 ll denominator = 1LL * N * N;
123
124 ll g = gcdll(numerator, denominator);
125
126 cout << numerator / g << '/' << denominator / g << '\n';
127
128 return 0;
129 }

```

9.15 Suffix Automaton - Beautiful Words

C++ Training_Camp_2025/Day_4/A - Beautiful Words.cpp

Resuelve "Beautiful Words" (Day 4, problema A - Contest 04 [Avanzados]): dado un string A de N caracteres y un conjunto S de M strings, el score de una permutación cíclica B_i de A es el largo del substring mas largo de B_i que tambien aparece como substring de algun string de S ; hay que hallar el minimo score sobre todas las N permutaciones ciclicas posibles. Construye un suffix automaton (struct SAM) con las cadenas de S invertidas y separadores entre ellas, recorre el string A duplicado e invertido para obtener en $G[i]$ el largo del match mas largo terminado en cada posicion, y usa busqueda binaria sobre la respuesta (funcion feasible) para encontrar la minima cota que deje una ventana de N caracteres sin matches mas largos.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 // Suffix automaton over an alphabet of 26 lowercase letters + 1 separator symbol.
5 struct SAM {
6     static const int ALPHA = 27;
7     vector<array<int, ALPHA>> trans;
8     vector<int> len, link_;
9     int last;
10
11     void init(int maxStates) {
12         trans.clear(); len.clear(); link_.clear();
13         trans.reserve(maxStates);
14         len.reserve(maxStates);
15         link_.reserve(maxStates);
16         newNode();
17         len[0] = 0; link_[0] = -1;

```

```

8         last = 0;
9     }
10
11     int newNode() {
12         array<int, ALPHA> a; a.fill(-1);
13         trans.push_back(a);
14         len.push_back(0);
15         link_.push_back(-1);
16         return (int)trans.size() - 1;
17     }
18
19     void extend(int c) {
20         int cur = newNode();
21         len[cur] = len[last] + 1;
22         int p = last;
23         while (p != -1 && trans[p][c] == -1) {
24             trans[p][c] = cur;
25             p = link_[p];
26         }
27         if (p == -1) {
28             link_[cur] = 0;
29         } else {
30             int q = trans[p][c];
31             if (len[p] + 1 == len[q]) {
32                 link_[cur] = q;
33             } else {
34                 int clone = newNode();
35                 len[clone] = len[p] + 1;
36                 link_[clone] = link_[q];
37                 trans[clone] = trans[q];
38                 while (p != -1 && trans[p][c] == q) {
39                     trans[p][c] = clone;
40                     p = link_[p];
41                 }
42                 link_[q] = clone;
43                 link_[cur] = clone;
44             }
45         }
46         last = cur;
47     }
48 };
49
50 inline int charIdx(char c) {
51     if (c >= 'a' && c <= 'z') return c - 'a';
52     return 26; // separator
53 }
54
55 int main() {
56     ios_base::sync_with_stdio(false);
57     cin.tie(nullptr);
58
59     int N, M;
60     cin >> N >> M;
61     string A;
62     cin >> A;
63     vector<string> S(M);
64     long long totalLen = 0;
65     for (auto& s : S) { cin >> s; totalLen += (long long)s.size(); }
66
67     int maxStates = 2 * (int)(totalLen + M + 5) + 10;
68     SAM sam;
69     sam.init(maxStates);
70
71     for (int i = 0; i < M; i++) {
72         string rev = S[i];
73         reverse(rev.begin(), rev.end());
74         for (char c : rev) sam.extend(charIdx(c));
75         if (i + 1 < M) sam.extend(26); // separator between strings
76     }
77
78     int n2 = 2 * N;
79     string doubledA = A + A;
80     string revDoubledA(doubledA.rbegin(), doubledA.rend());
81
82     vector<int> G(n2);

```

```

91 {
92     int v = 0, l = 0;
93     for (int i = 0; i < n2; i++) {
94         int c = charIdx(revDoubledA[(size_t)i]);
95         while (v != 0 && sam.trans[v][c] == -1) {
96             v = sam.link_[v];
97             l = sam.len[v];
98         }
99         if (sam.trans[v][c] != -1) {
100             v = sam.trans[v][c];
101             l++;
102         } else {
103             l = 0;
104         }
105         G[i] = l;
106     }
107 }
108
109 vector<int> C(n2);
110 for (int k = 0; k < n2; k++) C[k] = G[n2 - 1 - k];
111
112 auto feasible = [&](int ans) -> bool {
113     int W = N - ans;
114     if (W <= 0) return true;
115     int runStart = -1;
116     for (int k = 0; k <= n2; k++) {
117         bool bad = (k == n2) || (C[k] > ans);
118         if (!bad) {
119             if (runStart == -1) runStart = k;
120         } else {
121             if (runStart != -1) {
122                 int runEnd = k - 1;
123                 int runLen = runEnd - runStart + 1;
124                 if (runLen >= W && runStart <= N - 1) return true;
125             }
126             runStart = -1;
127         }
128     }
129     return false;
130 };
131
132 int lo = 0, hi = N;
133 while (lo < hi) {
134     int mid = (lo + hi) / 2;
135     if (feasible(mid)) hi = mid; else lo = mid + 1;
136 }
137 cout << lo << "\n";
138 return 0;
139 }

```

9.16 Trie + DP de Segmentacion (Maximo) - Spacebar Tokenizer

C++ (ICPC, 2024) E - Spacebar Tokenizer.cpp

Resuelve "Spacebar Tokenizer" (problema E, ICPC 2024): partir cada cadena en tokens para que la suma de puntajes sea MAXIMA. Cada token del diccionario vale su puntaje, y el enunciado dice que CUALQUIER otro token vale 0: se puede cortar donde sea. $dp[i]$ = mejor puntaje para el prefijo de largo i . Hay dos transiciones: avanzar una letra sumando 0 (esa letra queda dentro de un token no reconocido) y, por cada palabra del diccionario que empiece en i y termine en j , proponer $dp[i] + \text{puntaje para } dp[j+1]$. LA TRAMPA DEL PROBLEMA ES ESA TRANSICION DE 0. Sin ella se exige que TODA la cadena se parta en palabras del diccionario, y el segundo caso del sample ("theonlytokenizeripo", donde "theonly" no esta) sale imposible en vez de 13. La primera version de este archivo tenia ese error: daba -1000000000 en el sample. Verificada la corregida contra todas las formas de cortar en 1015 frases chicas, sin fallos. Las palabras van en un trie para que desde cada posicion se camine una sola vez siguiendo la cadena: cada nodo con puntaje en el camino es una palabra que encaja, y el recorrido se corta en cuanto el trie no tiene la letra. Es exactamente el esqueleto de "Word Combinations" de CSES (que esta en este cuaderno) con max en vez de suma. Vale reconocer que "segmentar en palabras de un diccionario" es siempre esto, cambie o no lo que se optimiza. Con la transicion de 0 toda posicion es alcanzable, asi que ya no hace falta un valor de "imposible". Y dp va en long long porque el enunciado no acota los puntajes. El trie se reconstruye por caso de prueba, ya que el diccionario cambia.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Node {
5     int next[26];
6     int score;

```

```

7
8     Node() {
9         fill(next, next + 26, -1);
10        score = -1;
11    }
12 };
13
14 int main() {
15     ios::sync_with_stdio(false);
16     cin.tie(nullptr);
17
18     int m, s;
19
20     while (cin >> m >> s && (m || s)) {
21
22         vector<Node> trie(1);
23
24         // Insertar palabras
25         for (int i = 0; i < m; i++) {
26
27             string word;
28             int value;
29
30             cin >> word >> value;
31
32             int node = 0;
33
34             for (char c : word) {
35
36                 int x = c - 'a';
37
38                 if (trie[node].next[x] == -1) {
39                     trie[node].next[x] = trie.size();
40                     trie.emplace_back();
41                 }
42
43                 node = trie[node].next[x];
44             }
45
46             trie[node].score = value;
47         }
48
49         while (s--) {
50
51             string a;
52             cin >> a;
53
54             int n = a.size();
55
56             vector<long long> dp(n + 1, 0);
57
58             for (int i = 0; i < n; i++) {
59
60                 // a[i] como parte de un token NO reconocido: suma 0
61                 dp[i + 1] = max(dp[i + 1], dp[i]);
62
63                 int node = 0;
64
65                 for (int j = i; j < n; j++) {
66
67                     int x = a[j] - 'a';
68
69                     if (trie[node].next[x] == -1)
70                         break;
71
72                     node = trie[node].next[x];
73
74                     if (trie[node].score != -1) {
75
76                         dp[j + 1] = max(
77                             dp[j + 1],
78                             dp[i] + trie[node].score
79                         );

```

```

80         }
81     }
82 }
83
84     cout << dp[n] << '\n';
85 }
86 }
87
88 return 0;
89 }

```

9.17 Z-Function - A String Problem

C++ \Codeforces\AStringProblem.cpp

Resuelve "a String Problem" (Codeforces): cuenta cuantos $t \neq "a"$ permiten partir s en trozos que sean t o $"a"$, usando al menos un t . Si s es todo 'a', cualquier bloque de 2 a n letras sirve, o sea $n-1$. Si no, todo t valido tiene que cubrir el primer caracter distinto de 'a', asi que se recorta el prefijo de 'a' (largo p) y se prueba como candidato cada prefijo $x[0..len-1]$ del resto. La Z-function decide en $O(1)$ si el candidato reaparece en una posicion ($z[pos] \geq len$), y $nxt[i]$ salta de un bloque de 'a' al siguiente caracter distinto de 'a' tambien en $O(1)$. Como el recorrido avanza de len en len , cada longitud cuesta $O(m/len)$ y el total queda $O(m \log m)$. mn guarda el bloque de 'a' mas corto que antecede a alguna ocurrencia (contando el prefijo p): el candidato puede llevar hasta mn letras 'a' pegadas al frente, y por eso cada len que funciona aporta $mn+1$ en vez de 1.

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  vector<int> z_function(string s) {
5      int n = s.size();
6      vector<int> z(n);
7
8      for (int i = 1, l = 0, r = 0; i < n; i++) {
9          if (i <= r)
10             z[i] = min(r - i + 1, z[i - l]);
11
12         while (i + z[i] < n && s[z[i]] == s[i + z[i]])
13             z[i]++;
14
15         if (i + z[i] - 1 > r) {
16             l = i;
17             r = i + z[i] - 1;
18         }
19     }
20
21     return z;
22 }
23
24 int main() {
25     ios::sync_with_stdio(false);
26     cin.tie(nullptr);
27
28     int T;
29     cin >> T;
30
31     while (T--) {
32         string s;
33         cin >> s;
34
35         int n = s.size();
36         int p = 0;
37
38         while (p < n && s[p] == 'a')
39             p++;
40
41         if (p == n) {
42             cout << n - 1 << '\n';
43             continue;
44         }
45
46         string x = s.substr(p);
47         int m = x.size();
48
49         vector<int> z = z_function(x);
50
51         vector<int> nxt(m + 1, m);

```

```

52
53         for (int i = m - 1; i >= 0; i--) {
54             if (x[i] != 'a')
55                 nxt[i] = i;
56             else
57                 nxt[i] = nxt[i + 1];
58         }
59
60         long long ans = 0;
61
62         for (int len = 1; len <= m; len++) {
63             int pos = 0;
64             int mn = p;
65             bool ok = true;
66
67             while (true) {
68                 if (pos + len > m) {
69                     ok = false;
70                     break;
71                 }
72
73                 if (pos != 0 && z[pos] < len) {
74                     ok = false;
75                     break;
76                 }
77
78                 pos += len;
79
80                 int next = nxt[pos];
81
82                 if (next == m)
83                     break;
84
85                 mn = min(mn, next - pos);
86                 pos = next;
87             }
88
89             if (ok)
90                 ans += mn + 1;
91         }
92
93         cout << ans << '\n';
94     }
95
96     return 0;
97 }

```

10 - Greedy

10.1 Binary Search - Maratona Brasileira De Popcorn

C++ \Training_Camp_2026\Day 1\10 - Maratona Brasileira de Popcorn.cpp

Resuelve "Maratona Brasileira de Popcorn" (Day 1, problema M - Maratona de Programacao da SBC 2019): N bolsas de palomitas en fila deben repartirse en C tramos contiguos (uno por competidor), cada competidor come a lo sumo T unidades por segundo, y se quiere minimizar el tiempo total (el maximo de todas las bolsas que come cada competidor, dividido T). Búsqueda binaria sobre la cota maxima S que puede cargar un competidor: la funcion `gruposNecesarios` hace un particionado greedy (como el problema clasico de asignacion de libros/pintores) contando cuantos tramos contiguos hacen falta para que ninguno supere S , y se busca el menor S factible con a lo sumo C tramos. La respuesta final es $\text{ceil}(S / T)$.

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main(){
5      ios::sync_with_stdio(false);
6      cin.tie(nullptr);
7
8      int n, c;
9      long long t;
10     cin >> n >> c >> t;
11
12     vector<long long> p(n);
13     long long lo = 0, hi = 0;
14     for (int i = 0; i < n; i++) {

```

```

5     cin >> p[i];
6     lo = max(lo, p[i]); // la bolsa m s grande fija una cota inferior
7     hi += p[i];        // un solo competidor comiendo todo fija la cota superior
8 }
9
10 // Dada una cota S, calcula cu ntos grupos contiguos (competidores)
11 // se necesitan para que ning n grupo tenga suma mayor que S.
12 auto gruposNecesarios = [&](long long S) -> long long {
13     long long grupos = 1;
14     long long acumulado = 0;
15     for (int i = 0; i < n; i++) {
16         if (acumulado + p[i] > S) {
17             grupos++;
18             acumulado = p[i];
19         } else {
20             acumulado += p[i];
21         }
22     }
23     return grupos;
24 };
25
26 // B squeda binaria de la m nima "suma m xima" S alcanzable
27 // usando a lo sumo C grupos (competidores).
28 while (lo < hi) {
29     long long mid = lo + (hi - lo) / 2;
30     if (gruposNecesarios(mid) <= c) {
31         hi = mid;
32     } else {
33         lo = mid + 1;
34     }
35 }
36
37 long long S = lo; // suma m xima  ptima que come un competidor
38 long long respuesta = (S + t - 1) / t; // techo(S / T)
39
40 cout << respuesta << "\n";
41 return 0;
42 }

```

10.2 Bit Manipulation - Lecographically Maximum

C++\Training_Camp_2025\Day 3\L - Lecographically Maximum.cpp

Resuelve "Lecographically Maximum" (Day 3, problema L - Maratona SBC de Programacao 2024): dada una lista de N enteros y una operacion que intercambia el k -esimo bit entre dos elementos cualquiera de la lista, hay que encontrar la secuencia lexicograficamente maxima alcanzable aplicando la operacion tantas veces como se quiera. Como la operacion permite mover libremente cualquier bit entre cualquier par de posiciones, lo  nico que se conserva es, para cada bit b , cuantos elementos originalmente lo tenian encendido ($\text{cnt}[b]$). El c digo aplica una estrategia greedy: para cada bit b , enciende ese bit en los primeros $\text{cnt}[b]$ elementos del resultado ($\text{ans}[i] \mid= (1 \ll b)$), lo que maximiza lexicograficamente la secuencia al concentrar todos los bits posibles en las posiciones mas a la izquierda.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int N;
9     cin >> N;
10
11     vector<int> cnt(31, 0);
12
13     for (int i = 0; i < N; i++) {
14         int x;
15         cin >> x;
16         for (int b = 0; b <= 30; b++)
17             if (x & (1 << b))
18                 cnt[b]++;
19     }
20
21     vector<int> ans(N, 0);
22
23     for (int b = 0; b <= 30; b++)

```

```

24         for (int i = 0; i < cnt[b]; i++)
25             ans[i] |= (1 << b);
26
27     for (int i = 0; i < N; i++) {
28         if (i) cout << ' ';
29         cout << ans[i];
30     }
31     cout << '\n';
32 }

```

10.3 Budget Optimization - Posters

C++\RPC\Poster.cpp

Dado un presupuesto c y costos a, b , calcula el maximo de posters que se pueden comprar comprando al menos uno de cada tipo.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using int64 = long long;
5
6 int main() {
7     long long a,b,c;
8
9     cin >> a >> b >> c;
10
11     cout << (min((c-a) / b, (c-b) / a) < 0 ? 0 : min((c-a) / b, (c-b) / a)) << endl;
12
13
14
15     return 0;
16 }

```

10.4 Buy Low Sell High (Comprar y Vender Acciones)

C++\Notebook\Buy Low Sell High.cpp

Precio de una accion por dia: comprar un dia y vender un dia POSTERIOR (una transaccion) maximizando la ganancia. Un solo recorrido $O(n)$, $O(1)$ de memoria: se guarda el dia mas barato visto hasta ahora (el mejor momento para haber comprado) y cada dia se prueba vender hoy contra ese minimo. Si el precio solo baja, la respuesta es 0 (no operar). Es Kadane disfrazado: la ganancia es la suma maxima de un subarreglo de $d[i] = p[i] - p[i-1]$. Variantes t picas (en todas no se puede tener dos acciones a la vez): - Ilimitadas: sumar cada subida $\max(0, d[i])$; toda subida se puede cobrar. - Con comision por transaccion: DP de dos estados, cash (sin accion) y hold (con accion). - A lo sumo K transacciones: los mismos dos estados por cada transaccion, $O(n \cdot K)$. Con $K \geq n/2$ ya no limita y equivale a ilimitadas ($O(n)$).

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 typedef long long ll;
6
7 const ll NEG = LLONG_MIN / 4;
8
9 // Una transaccion: {ganancia, {dia de compra, dia de venta}} (d as -1 si no conviene operar)
10 pair<ll, pair<int, int>> maxProfit(const vector<ll> &p)
11 {
12     ll best = 0;
13     int low = 0, buy = -1, sell = -1;
14     for (int i = 1; i < (int)p.size(); i++)
15     {
16         if (p[i] < p[low]) low = i;
17         else if (p[i] - p[low] > best)
18         {
19             best = p[i] - p[low];
20             buy = low;
21             sell = i;
22         }
23     }
24     return {best, {buy, sell}};
25 }
26
27 // Transacciones ilimitadas
28 ll maxProfitUnlimited(const vector<ll> &p)
29 {
30     ll s = 0;
31     for (int i = 1; i < (int)p.size(); i++) s += max(0LL, p[i] - p[i - 1]);
32     return s;

```

```

33 }
34
35 // Ilimitadas, pagando `fee` por cada transaccion completa
36 ll maxProfitFee(const vector<ll> &p, ll fee)
37 {
38     ll cash = 0, hold = NEG;
39     for (ll x : p)
40     {
41         hold = max(hold, cash - x);
42         cash = max(cash, hold + x - fee);
43     }
44     return cash;
45 }
46
47 // A lo sumo K transacciones
48 ll maxProfitK(const vector<ll> &p, int K)
49 {
50     if (K <= 0) return 0;
51     if (2 * K >= (int)p.size()) return maxProfitUnlimited(p);
52     vector<ll> hold(K + 1, NEG), cash(K + 1, 0);
53     for (ll x : p)
54     {
55         for (int t = 1; t <= K; t++)
56         {
57             hold[t] = max(hold[t], cash[t - 1] - x);
58             cash[t] = max(cash[t], hold[t] + x);
59         }
60     }
61     return cash[K];
62 }
63
64 int main()
65 {
66     ios_base::sync_with_stdio(false);
67     cin.tie(nullptr);
68
69     int n;
70     cin >> n;
71     vector<ll> p(n);
72     for (auto &x : p) cin >> x;
73
74     auto r = maxProfit(p);
75     cout << r.first << "\n";
76     cout << r.second.first << " " << r.second.second << "\n"; // -1 -1 si no se opera
77     return 0;
78 }

```

10.5 Constructive Algorithms - Nomad

C++ Training_Camp_2026_Day_10E - Nomad.py

Resuelve "Nomad" (Day 10, problema H - Contest 09 [Avanzados]): Kai debe dormir exactamente D noches alternando entre N lugares, cada uno con un límite d_i de días CONSECUTIVOS que tolera antes de expulsarlo; hay que construir un plan de D días que nunca exceda ningún límite, o reportar -1 si es imposible. Busca los primeros dos índices $d1$, $d2$ con tolerancia distinta de cero: si hay al menos dos lugares utilizables, alterna entre ellos ($d1+1$, $d2+1$, $d1+1$, ...) durante los D días, lo que garantiza como máximo 1 día consecutivo en cada uno y por lo tanto siempre respeta cualquier límite ≥ 1 . Si solo hay un lugar utilizable, la única opción es quedarse fijo ahí todos los D días, lo cual solo es válido si su tolerancia $l[d1]$ alcanza para cubrir los D días consecutivos; en cualquier otro caso (ningún lugar utilizable, o el único disponible no aguanta los D días) imprime -1.

```

1 n,m = list(map(int,input().split()))
2 l = list(map(int,input().split()))
3 d1 = -1
4 d2 = -1
5 for i in range(len(l)):
6     if(l[i]!= 0 and d1 == -1):
7         d1 = i
8     elif(l[i]!= 0 and d1 != -1):
9         d2 = i
10        break
11 if(d2 == -1):
12     if(d1!= -1 and l[d1]>=m):
13         for i in range(m):
14             print(d1+1, end=" ")
15     else:

```

```

6         print(-1)
7     else:
8         for i in range(m):
9             if(i%2 == 0):
20                 print(d1+1, end=" ")
21             else:
22                 print(d2+1, end=" ")

```

10.6 DP - Exhibition Of Clownfish

C++ Training_Camp_2026_Day_10E - Exhibition of Clownfish.cpp

Resuelve "Exhibition of Clownfish" (Day 1, problema E - Maratona de Programacao da SBC 2019): N peceras con M machos y F hembras cada una; si al final de la noche una pecera queda con machos pero sin hembras, uno de ellos muta a hembra. Zeliuz puede mover un pez por noche entre peceras y quiere el numero mínimo de movimientos para que todos los peces terminen siendo hembra. Para cada pecera define $v = 1 + M - F$ (cuanto "aporta" dejarla vacía de machos vs. usarla como colector) y separa peceras vacías, solo-hembras y mixtas (con v positivo o negativo). Arma greedily una "familia" alrededor de un colector maximizando la suma de v de las mixtas positivas mas las vacías disponibles, y cuando faltan colectores vacíos usa una DP tipo knapsack ($dp[j]$ = costo mínimo en v para "cubrir" un déficit de peso j) para decidir que peceras mixtas conviene sacrificar. El total de movimientos es $2 \cdot S$ menos el mejor ahorro encontrado (S = total de machos).

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main(){
5     ios_base::sync_with_stdio(false);
6     cin.tie(NULL);
7
8     int n;
9     if(!(cin >> n)) return 0;
10
11     vector<long long> M(n), F(n);
12     long long S = 0; // total males
13     for(int i = 0; i < n; i++){
14         cin >> M[i] >> F[i];
15         S += M[i];
16     }
17
18     if(S == 0){ cout << 0 << "\n"; return 0; } // ya todas hembras
19
20     long long E = 0, femonly = 0, negmixed = 0;
21     long long PosV = 0, PosM = 0; // suma de v y de M sobre "mixtos positivos"
22     long long bestSingle = LLONG_MIN; // mejor v de una sola cámara
23     vector<pair<long long,long long>> pos; // (peso = M, valor = v) de mixtos con v>0
24
25     for(int i = 0; i < n; i++){
26         long long v = 1 + M[i] - F[i];
27         bestSingle = max(bestSingle, v);
28         if(M[i] == 0 && F[i] == 0) E++; // vacío
29         else if(M[i] == 0) femonly++; // solo hembras
30         else { // mixto (M>0 => F>0 por restricción)
31             if(v > 0){ PosV += v; PosM += M[i]; pos.push_back({M[i], v}); }
32             else negmixed++;
33         }
34     }
35
36     long long FM0 = S - PosM; // machos libres provenientes de mixtos con v<=0
37
38     // Familia principal: max sobre w in [0,E] de bestPv(S-w) + w
39     long long best_family = PosV + min(E, FM0); // w <= FM0 : incluir todos los positivos
40
41     if(E > FM0){
42         long long T = E - FM0; // T <= E <= n <= 3000
43         const long long INF = LLONG_MAX / 4;
44         vector<long long> dp(T + 1, INF);
45         dp[0] = 0;
46         for(auto &pr : pos){ // cover DP: min sum(v) para quitar >= j de peso
47             long long w = pr.first, v = pr.second;
48             for(long long j = T; j >= 0; j--){
49                 if(dp[j] < INF){
50                     long long nj = (j + w >= T) ? T : (j + w);
51                     if(dp[j] + v < dp[nj]) dp[nj] = dp[j] + v;
52                 }
53             }
54         }
55     }

```

```

94     }
95     for(long long j = 1; j <= T; j++)
96         if(dp[j] < INF)
97             best_family = max(best_family, PosV + FM0 + (j - dp[j]));
98     }
99
100     bool family_valid = true;
101     if(PosV == 0 && E == 0) family_valid = false;          // daría 0 cámaras
102
103     // Caso frontera: TODOS los tanques son mixtos positivos -> hay que dejar 1 como drenaje
104     if(E == 0 && femonly == 0 && negmixed == 0){
105         long long minv = LLONG_MAX;
106         for(auto &pr : pos) minv = min(minv, pr.second);
107         best_family = PosV - minv;
108         family_valid = true;
109     }
110
111     long long maxB = bestSingle;                          // cámara única (siempre válida, n>=2)
112     if(family_valid) maxB = max(maxB, best_family);
113
114     cout << (2 * S - maxB) << "\n";
115     return 0;
116 }

```

10.7 Huffman

C++\Notebook\Huffman.cpp

Costo mínimo para combinar N elementos de a dos, donde cada combinacion cuesta la suma de los dos elementos y el resultado vuelve a la mesa. Aparece disfrazado como "unir cuerdas", "mezclar archivos ordenados" o "juntar montones de piedras". El greedy correcto es siempre combinar los dos MAS PEQUENOS disponibles, porque cada elemento paga su valor una vez por cada combinacion en la que participa, y los mas grandes deben participar en la menor cantidad posible. Un priority_queue de mínimo lo resuelve en $O(n \log n)$. Cuidado: el acumulador debe ser long long, la suma crece rapido.

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main()
6  {
7      ios_base::sync_with_stdio(false);
8      cin.tie(nullptr);
9
10     int n;
11     cin >> n;
12
13     priority_queue<long long, vector<long long>, greater<long long>> pq;
14     for (int i = 0; i < n; i++)
15     {
16         long long x;
17         cin >> x;
18         pq.push(x);
19     }
20
21     long long cost = 0;
22     while (pq.size() > 1)
23     {
24         long long x = pq.top(); pq.pop();
25         long long y = pq.top(); pq.pop();
26         cost += x + y;
27         pq.push(x + y);
28     }
29
30     cout << cost << "\n";
31     return 0;
32 }

```

10.8 Huffman sobre Frecuencias Variables en el Tiempo - Efficient Encoding

C++\ICPC\2025E - Efficient Encoding.cpp

Resuelve "Efficient Encoding" (problema E, ICPC 2025): cada simbolo tiene una frecuencia que cambia con el tiempo, $f_i(t)$, lineal a trozos con un quiebre en m_i . Para cada t se arma el mejor codigo prefijo (Huffman) y se paga $C(t)$ = suma de $f_i(t)$ por el largo de su codigo. Hay que dar el MENOR t entero de $[0, T]$ que minimiza $C(t)$, y ese costo con 3 decimales. LA PREGUNTA CLAVE ES CUALES t HAY QUE PROBAR. T llega a 10^6 y hay hasta 1000 casos, así que evaluar todos es imposible. El argumento que lo reduce a unos pocos: - Con las longitudes FIJAS, el costo es suma de $f_i(t) * L_i$, lineal a trozos con quiebres solo en los m_i . - El costo OPTIMO es el mínimo sobre todas las asignaciones validas de longitudes, o sea el mínimo de un monton de funciones lineales: eso es CONCAVO en cada tramo sin quiebres. - Una funcion concava en un intervalo alcanza su mínimo en un EXTREMO, nunca por dentro. Entonces el mínimo global solo puede estar en 0, en T , o en alguno de los m_i : a lo sumo $n+2$ candidatos. Y como los m_i son enteros, todos los candidatos ya son enteros. LA PRIMERA VERSION DE ESTE ARCHIVO SOLO PROBABA $t = 0$ y $t = T$, y fallaba el propio sample: en el segundo caso (un simbolo con $a=2$, $m=1$, $b=1$, $c=2$ y $T=2$) el mínimo esta en $t=1$, donde la frecuencia baja a 1. Daba "0 2.000" en vez de "1 1.000". Medido contra probar TODOS los t enteros en 1200 casos: la version vieja fallaba 443 y esta no falla ninguno. EL OTRO ERROR ERA EL TIPO: en un t intermedio las frecuencias NO son enteras, así que el Huffman tiene que trabajar en double. Se recorren los candidatos ordenados y solo se cambia el mejor con una mejora ESTRICTA, para quedarse con el t mas chico en caso de empate, que es lo que pide el enunciado. Con un solo simbolo el costo es su frecuencia: igual hace falta 1 bit por ocurrencia, y esa es la excepcion que Huffman no cubre solo (el arbol de un nodo tendria largo 0).

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  using ll = long long;
5
6  struct Symbol {
7      ll a, m, b, c;
8  };
9
10 // Costo optimo de un codigo prefijo (Huffman). Las frecuencias son REALES,
11 // porque en un t intermedio  $f_i(t)$  no es entera.
12 double huffman(vector<double> freq) {
13
14     priority_queue<double, vector<double>, greater<double>> pq;
15
16     for (double x : freq)
17         pq.push(x);
18
19     // Con un solo simbolo igual hace falta 1 bit por ocurrencia.
20     if (pq.size() == 1)
21         return pq.top();
22
23     double ans = 0;
24
25     while (pq.size() > 1) {
26         double x = pq.top();
27         pq.pop();
28         double y = pq.top();
29         pq.pop();
30         ans += x + y;
31         pq.push(x + y);
32     }
33     return ans;
34 }
35
36 //  $f_i(t)$ , lineal a trozos con el quiebre en  $m_i$ .
37 double frecuencia(const Symbol &s, double t, double T) {
38     if (t <= s.m)
39         return s.a + (double)(s.b - s.a) / s.m * t;
40     return s.b + (double)(s.c - s.b) / (T - s.m) * (t - s.m);
41 }
42
43 int main() {
44     ios::sync_with_stdio(false);
45     cin.tie(nullptr);
46
47     int K;
48     cin >> K;

```

```

53
54 while (K--) {
55
56     int n;
57     ll T;
58
59     cin >> n >> T;
60
61     vector<Symbol> s(n);
62
63     for (int i = 0; i < n; i++)
64         cin >> s[i].a >> s[i].m >> s[i].b >> s[i].c;
65
66     // Con las longitudes FIJAS, el costo es lineal a trozos con quiebres en
67     // los m_i. El optimo es el minimo de esas lineales, o sea CONCAVO entre
68     // quiebres, y una concava alcanza su minimo en un extremo del intervalo.
69     // Así que basta mirar 0, T y cada m_i.
70     vector<ll> cand;
71     cand.push_back(0);
72     cand.push_back(T);
73     for (int i = 0; i < n; i++)
74         cand.push_back(s[i].m);
75
76     sort(cand.begin(), cand.end());
77     cand.erase(unique(cand.begin(), cand.end()), cand.end());
78
79     ll mejorT = 0;
80     double mejorC = 1e18;
81
82     for (ll t : cand) {
83
84         vector<double> f(n);
85
86         for (int i = 0; i < n; i++)
87             f[i] = frecuencia(s[i], t, T);
88
89         double c = huffman(f);
90
91         // Se queda el t mas chico: por eso la comparacion es estricta y los
92         // candidatos vienen ordenados.
93         if (c < mejorC - 1e-9) {
94             mejorC = c;
95             mejorT = t;
96         }
97     }
98
99     cout << mejorT << ' ' << fixed << setprecision(3) << mejorC << '\n';
100 }
101
102 return 0;
103 }

```

10.9 Interval Scheduling - Interval Scheduling Greedy

C++\Notebook\Interval Scheduling Greedy.cpp

Maximo numero de intervalos que se pueden elegir sin que se solapen (tarear con hora de inicio y fin, salones, reservas). Es el greedy clasico y no debe confundirse con la DP de intervalos de "Yuyuan Market", que optimiza otra cosa (cantidad y luego costo). La clave es ordenar por hora de FIN, no por inicio ni por duracion: al quedarse siempre con el intervalo que termina antes, se deja la mayor cantidad de tiempo libre para los que siguen, y se puede probar por intercambio que ninguna otra eleccion mejora el resultado. Ordenar por inicio o por duracion da respuestas incorrectas.

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 int main()
6 {
7     ios_base::sync_with_stdio(false);
8     cin.tie(nullptr);
9
10    int n;
11    cin >> n;
12
13    vector<pair<int,int>> v(n);

```

```

14    for (int i = 0; i < n; i++)
15    {
16        cin >> v[i].first >> v[i].second;
17    }
18
19    // ordenar por hora de fin
20    sort(v.begin(), v.end(), [](const pair<int,int> &a, const pair<int,int> &b){
21        return a.second < b.second;
22    });
23
24    int cnt = 0;
25    int last = INT_MIN;
26
27    for (auto &it : v)
28    {
29        if (it.first >= last)
30        {
31            cnt++;
32            last = it.second;
33        }
34    }
35
36    cout << cnt << "\n";
37    return 0;
38 }

```

10.10 Maximo Conjunto a Distancia d en Arbol - Oficinas A Distancia D En Arbol

C++\Notebook\Oficinas a Distancia d en Arbol.cpp

Hay n ciudades en forma de arbol, cada una con una oficina, y hay que cerrar la MENOR cantidad posible de oficinas para que cualesquiera dos de las que queden esten a distancia al menos d . Cerrar lo minimo es lo mismo que dejar lo maximo, asi que esto es el maximo conjunto de vertices a distancia mutua $\geq d$ en un arbol. En un grafo cualquiera seria NP-difícil, pero en arbol el greedy de hojas a raiz lo resuelve exacto en $O(n)$. La idea que hace que funcione: procesando de abajo hacia arriba, de todo el subarbol de v lo UNICO que el resto del arbol necesita saber es a que distancia esta la oficina superviviente mas cercana a v , porque cualquier oficina de afuera tiene que pasar por v para llegar. Eso es $dist[v]$, y $rep[v]$ guarda cual es esa oficina para poder marcarla si toca cerrarla. Al pegar un hijo c a su padre p , la oficina mas cercana del lado de c queda a $dc = dist[c] + 1$. Si $dist[p] + dc < d$, esas dos se pisan y hay que cerrar una. Se cierra SIEMPRE la mas cercana a p : quedarse con la mas lejana deja $dist[p]$ lo mas grande posible, y $dist[p]$ grande es pura ganancia porque lo unico que falta por decidir esta hacia arriba. Es un intercambio clasico, ninguna solucion optima empeora al cambiarle la cercana por la lejana. Y basta cerrar UNA sola, no hace falta revisar el resto del subarbol: las demas oficinas de p ya cumplen $\geq d$ contra $rep[p]$, y eso mismo las deja lo bastante lejos de $rep[c]$. Si no hay choque, solo se actualiza el minimo: $dist[p] = \min(dist[p], dc)$. Verificado contra fuerza bruta sobre 2^n y contra un DP exacto de arbol, ademas de la salida en si (que todo par sobreviviente quede a $\geq d$). Detalles que importan: $dist[v]$ arranca en 0 y $rep[v] = v$ porque TODA ciudad empieza con oficina, esa es la condicion inicial del problema. El recorrido es BFS iterativo, no DFS recursivo, asi que un camino de 200 mil nodos no revienta la pila. Medido: $n = 200000$ en menos de 200 ms. OJO con el formato de salida: deja un espacio sobrante antes del salto de línea. La mayoría de jueces lo ignora, pero si uno sale estricto, ahí esta el problema. Entrada: n y d , despues las $n-1$ aristas. Salida: cuantas oficinas quedan y cuales.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int n, d;
9     cin >> n >> d;
10    vector<vector<int>> adj(n + 1);
11    for (int i = 0; i < n - 1; i++) {
12        int a, b;
13        cin >> a >> b;
14        adj[a].push_back(b);
15        adj[b].push_back(a);
16    }
17
18    // Orden BFS desde la raiz 1
19    vector<int> order, par(n + 1, 0);
20    order.reserve(n);
21    vector<char> vis(n + 1, 0);
22    order.push_back(1);
23    vis[1] = 1;
24    for (int i = 0; i < (int)order.size(); i++) {
25        int v = order[i];

```

```

26     for (int u : adj[v]) {
27         if (!vis[u]) {
28             vis[u] = 1;
29             par[u] = v;
30             order.push_back(u);
31         }
32     }
33 }
34
35 vector<int> dist(n + 1, 0), rep(n + 1);
36 vector<char> removed(n + 1, 0);
37 // todas las ciudades empiezan con oficina
38 for (int v = 1; v <= n; v++) rep[v] = v;
39
40 // Procesar de hojas a raiz, uniendo cada nodo con su padre
41 for (int i = n - 1; i >= 1; i--) {
42     int c = order[i], p = par[c];
43     int dc = dist[c] + 1;
44     if (dist[p] + dc < d) {
45         // conflicto: cerrar la oficina mas cercana a p
46         if (dist[p] < dc) {
47             removed[rep[p]] = 1;
48             dist[p] = dc;
49             rep[p] = rep[c];
50         } else {
51             removed[rep[c]] = 1;
52         }
53     } else if (dc < dist[p]) {
54         dist[p] = dc;
55         rep[p] = rep[c];
56     }
57 }
58
59 vector<int> ans;
60 for (int v = 1; v <= n; v++)
61     if (!removed[v]) ans.push_back(v);
62
63 string out = to_string(ans.size()) + "\n";
64 for (int x : ans) out += to_string(x) + ' ';
65 out += '\n';
66 cout << out;
67 return 0;
68 }

```

10.11 Prefijos sobre Secuencias Balanceadas - Incompatible Pairs

C++(Training_Camp_2026\Day 10\D - Incompatible Pairs.cpp

Resuelve "Incompatible Pairs" (Day 10, problema D - Contest 09 [Avanzados]): una secuencia de N bailarines Yang ('(') y Yin (')') donde cada Yang debe emparejarse con un Yin posterior; un par (i, j) es incompatible si al formarlo los bailarines restantes ya no pueden emparejarse todos entre s  (rompe el balance tipo parentesis). Usa el balance de prefijos $prefix[k]$ (+1 por '(', -1 por ')'); para cada Yang en la posici n i , $f[i]$ es la primera posici n $k \geq i$ donde el balance vuelve a 0 (fin del segmento balanceado que arranca en i): cualquier Yin ubicado desp es de $f[i]$ forma un par incompatible con ese Yang, porque deja un '(' suelto sin ')' extra donde absorberlo. Suma, para cada '(' en la posici n i , la cantidad de ')' que hay desp es de $f[i]$ (precomputada en $suffixClose$) para obtener el total de pares incompatibles.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4
5 static char buf[1 << 22];
6 static int bufLen_, bufPos_;
7 inline int gc() {
8     if (bufPos_ == bufLen_) { bufLen_ = fread(buf_, 1, sizeof(buf_), stdin); bufPos_ = 0; if (!bufLen_) return -1; }
9     return buf_[bufPos_++];
10 }
11 inline int readInt() {
12     int c = gc();
13     while (c != -1 && (c < '0' || c > '9') && c != '-') c = gc();
14     bool neg = false; if (c == '-') { neg = true; c = gc(); }
15     int x = 0;
16     while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = gc(); }
17     return neg ? -x : x;

```

```

8 }
9 inline void readString(string &s, int expectedLen) {
10     s.clear();
11     s.reserve(expectedLen);
12     int c = gc();
13     while (c != '(' && c != ')') c = gc();
14     while (c == '(' || c == ')') {
15         s.push_back((char)c);
16         c = gc();
17     }
18 }
19
20 int main() {
21     int N = readInt();
22     string s;
23     readString(s, N);
24     // in case the read stopped early due to buffering edge cases, N is authoritative
25     // for sizing arrays; s.size() should equal N given well-formed input.
26
27     vector<int> prefix(N + 1, 0);
28     for (int i = 1; i <= N; i++) {
29         prefix[i] = prefix[i - 1] + (s[i - 1] == '(' ? 1 : -1);
30     }
31
32     // f[i] = smallest k >= i with prefix[k] == 0, for i = 1..N (computed right to left).
33     vector<int> f(N + 2, 0);
34     f[N] = (prefix[N] == 0) ? N : N + 1;
35     for (int i = N - 1; i >= 1; i--) {
36         f[i] = (prefix[i] == 0) ? i : f[i + 1];
37     }
38
39     // suffixClose[k] = number of ')' at 1-indexed positions > k, for k = 0..N.
40     vector<ll> suffixClose(N + 2, 0);
41     for (int k = N - 1; k >= 0; k--) {
42         suffixClose[k] = suffixClose[k + 1] + (s[k] == ')') ? 1 : 0;
43     }
44
45     ll total = 0;
46     for (int i = 1; i <= N; i++) {
47         if (s[i - 1] == '(') {
48             total += suffixClose[f[i]];
49         }
50     }
51
52     printf("%lld\n", total);
53     return 0;
54 }

```

10.12 Prefix Sums - Maximizing Advertising

C++(Training_Camp_2026\Day 7\B - Maximizing Advertising.cpp

Resuelve "Maximizing Advertising" (Day 7, problema B - Contest 07 [Avanzados]): dados N votantes con coordenadas (x, y) y preferencia 'b' o 'w', hay que elegir dos rect ngulos disjuntos de lados paralelos a los ejes (uno para PSD, otro para PS) que maximicen la suma de votantes 'b' cubiertos por el primero mas los votantes 'w' cubiertos por el segundo. Como la partici n  ptima siempre puede lograrse separando el plano con una l nea vertical u horizontal, bestSplit ordena los puntos por una coordenada (x o y) y usa sumas prefijas ($prefB$, $prefW$) para probar, en cada corte posible, poner los 'b' de un lado y los 'w' del otro (o viceversa); el main toma el mejor resultado entre cortar por X o por Y .

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4
5 static char buf[1 << 25];
6 static int bufLen = 0, bufPos = 0;
7
8 inline int gc() {
9     if (bufPos == bufLen) {
10         bufLen = (int)fread(buf, 1, sizeof(buf), stdin);
11         bufPos = 0;
12         if (bufLen <= 0) return -1;
13     }
14     return buf[bufPos++];
15 }
16
17

```

```

1 inline int readInt() {
2     int c = gc();
3     while (c == ' ' || c == '\n' || c == '\r' || c == '\t') c = gc();
4     bool neg = false;
5     if (c == '-') { neg = true; c = gc(); }
6     int x = 0;
7     while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = gc(); }
8     return neg ? -x : x;
9 }
10
11 inline char readChar() {
12     int c = gc();
13     while (c == ' ' || c == '\n' || c == '\r' || c == '\t') c = gc();
14     return (char)c;
15 }
16
17 int N;
18 vector<int> Xs, Ys;
19 vector<char> Cs;
20
21 // dado un vector de coordenadas (alineado con Cs), calcula el mejor
22 // resultado separando en dos grupos (izq/der) por esa coordenada
23 ll bestSplit(vector<int>& coord) {
24     int n = N;
25     vector<pair<int, char>> arr(n);
26     for (int i = 0; i < n; i++) arr[i] = {coord[i], Cs[i]};
27     sort(arr.begin(), arr.end(), [](const pair<int, char>& a, const pair<int, char>& b) {
28         return a.first < b.first;
29     });
30
31     ll totalB = 0, totalW = 0;
32     for (auto& pr : arr) { if (pr.second == 'b') totalB++; else totalW++; }
33
34     ll best = max(totalB, totalW); // caso: todo a un lado, el otro rectangulo vacio
35
36     ll prefB = 0, prefW = 0;
37     int i = 0;
38     while (i < n) {
39         int j = i;
40         int cur = arr[j].first;
41         ll gB = 0, gW = 0;
42         while (j < n && arr[j].first == cur) {
43             if (arr[j].second == 'b') gB++; else gW++;
44             j++;
45         }
46         prefB += gB; prefW += gW;
47         // corte justo despues de este grupo de coordenada 'cur'
48         ll optA = prefB + (totalW - prefW); // b a la izquierda, w a la derecha
49         ll optB = prefW + (totalB - prefB); // w a la izquierda, b a la derecha
50         best = max(best, max(optA, optB));
51         i = j;
52     }
53     return best;
54 }
55
56 int main() {
57     N = readInt();
58     Xs.resize(N); Ys.resize(N); Cs.resize(N);
59     for (int i = 0; i < N; i++) {
60         Xs[i] = readInt();
61         Ys[i] = readInt();
62         Cs[i] = readChar();
63     }
64
65     ll ans = max(bestSplit(Xs), bestSplit(Ys));
66     printf("%lld\n", ans);
67     return 0;
68 }

```

10.13 Puente y Linterna - The Bridge At Night

C++ ICPC 2024 B - The Bridge At Night.cpp

Resuelve "The Bridge At Night" (problema B, ICPC 2024), el clasico del puente con una sola linterna: cruzan a lo sumo dos a la vez, al paso del mas lento, y alguien tiene que devolver la linterna. El codigo calcula el tiempo minimo para pasar a todos. Se ordenan los tiempos y, mientras queden mas de 3, se mandan al otro lado LOS DOS MAS LENTOS (y, z) con la mejor de dos estrategias, siendo a y b los dos mas rapidos: a + 2b + z cruzan a y b, vuelve a, cruzan y y z juntos, vuelve b 2a + y + z a acompaña a z, vuelve; a acompaña a y, vuelve La primera gana cuando los lentos son MUY lentos, porque paga z una sola vez para los dos; la segunda cuando no, porque solo usa al mas rapido de ida y vuelta. Cada ronda deja el problema igual pero con dos personas menos. Casos base: con 3 quedan a + b + c (a acompaña a c, vuelve, cruza con b); con 2, el mas lento; con 1, el unico. Con 1, 2, 5 y 10, que es el acertijo famoso, da 17 (comprobado corriendo este codigo): la estrategia ingenua de que el rapido acompaña a todos da 19.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int n;
9
10    while (cin >> n && n != 0) {
11
12        vector<int> t(n);
13
14        for (int &x : t)
15            cin >> x;
16
17        sort(t.begin(), t.end());
18
19        int ans = 0;
20
21        while (n > 3) {
22
23            int a = t[0];
24            int b = t[1];
25            int y = t[n - 2];
26            int z = t[n - 1];
27
28            int option1 = a + 2 * b + z;
29            int option2 = 2 * a + y + z;
30
31            ans += min(option1, option2);
32
33            n -= 2;
34        }
35
36        // Quedan 1, 2 o 3 personas
37        if (n == 3) {
38            ans += t[0] + t[1] + t[2];
39        }
40        else if (n == 2) {
41            ans += t[1];
42        }
43        else if (n == 1) {
44            ans += t[0];
45        }
46
47        cout << ans << '\n';
48    }
49
50    return 0;
51 }

```

10.14 Puente y Linterna (version con indices) - Bridge Juan

C++\ICPC\2024\bridge_juan.cpp

La otra solución del problema B de ICPC 2024 ("The Bridge at Night"), el clásico del puente con una sola linterna. Misma estrategia que "B - The Bridge At Night" de esta carpeta, escrita con un contador de cuantos quedan en vez de ir recorriendo el arreglo. Con los tiempos ordenados y mientras queden mas de 3, se cruzan los DOS MAS LENTOS con la mejor de dos jugadas, donde $t[0]$ y $t[1]$ son los dos mas rapidos: $escolta = 2*t[0] + t[quedan-2] + t[quedan-1]$ el mas rapido acompaña a cada lento $parejaLenta = t[0] + 2*t[1] + t[quedan-1]$ los dos lentos cruzan juntos La segunda gana cuando los lentos son MUY lentos, porque paga el mas lento una sola vez para los dos; la primera cuando la diferencia es chica. Cada ronda saca a dos del problema. Casos base: con 3 quedan $t[0]+t[1]+t[2]$; con 2, el mas lento; con 1, el unico. Verificado con el sample (10, 20, 8) y con el acertijo famoso de 1, 2, 5 y 10, que da 17 (la estrategia ingenua de que el rapido acompañe a todos daría 19). La otra version esta verificada ademas contra un Dijkstra exacto sobre todos los estados en 1500 casos; esta comparte la formula, así que vale lo mismo.

```
1 #include <iostream>
2 #include <vector>
3 #include <algorithm>
4
5 using namespace std;
6
7 int main()
8 {
9     long long n;
10
11     while(cin >> n && n != 0)
12     {
13         vector<long long> tiempo(n);
14
15         for(long long i = 0; i < n; i++)
16         {
17             cin >> tiempo[i];
18         }
19
20         sort(tiempo.begin(), tiempo.end());
21
22         long long total = 0;
23         long long quedan = n;
24
25         while(quedan > 3)
26         {
27             long long escolta = 2 * tiempo[0] + tiempo[quedan - 2] + tiempo[quedan - 1];
28             long long parejaLenta = tiempo[0] + 2 * tiempo[1] + tiempo[quedan - 1];
29
30             total += min(escolta, parejaLenta);
31             quedan -= 2;
32         }
33
34         if(quedan == 3)
35         {
36             total += tiempo[0] + tiempo[1] + tiempo[2];
37         }
38         else if(quedan == 2)
39         {
40             total += tiempo[1];
41         }
42         else
43         {
44             total += tiempo[0];
45         }
46
47         cout << total << '\n';
48     }
49
50     return 0;
51 }
```

10.15 Segment Counting - Cerros

C++\Codeforces\Cerros.cpp

Cuenta la cantidad de segmentos contiguos con elementos $\geq k$ en un arreglo. Cada vez que encuentra un elemento $\geq k$ después de uno $< k$, incrementa el contador.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
```

```
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     long long n,k;
9     cin >> n >> k;
10    long long god=0;
11    bool abierto = false;
12    for(int i = 0; i < n;i++)
13    {
14        int a; cin >> a;
15
16        if(a>=k&&!abierto)
17        {
18            god++;
19            abierto = true;
20        }
21        else if(a<k)
22        {
23            abierto = false;
24        }
25    }
26
27    cout << god << "\n";
28
29    return 0;
30 }
```

10.16 Simulation - Kathmandu

C++\Training_Camp_2026\Day 4\F - Kathmandu.py

Resuelve "Kathmandu" (Day 4, problema F - Contest 04 [Avanzados]): durante un vuelo de duracion D hay M comidas servidas en instantes y_i (hay que estar despierto para comer), y hay que decidir si se puede dormir T minutos seguidos sin interrupcion sin perderse ninguna comida. Arma la lista c de instantes relevantes (0, cada y_i , y D) y busca si existe un hueco entre dos instantes consecutivos de largo mayor o igual a T ($abs(c[i]-c[i+1]) \geq t$), ya que ese hueco es la unica ventana disponible para dormir sin faltar a ninguna comida.

```
1 t, v, n = list(map(int,input().split()))
2 c = [0]
3 for i in range(n):
4     c.append(int(input()))
5 c.append(v)
6
7 f = False
8 for i in range(len(c)-1):
9     if(t <= abs(c[i]-c[i+1])):
10         f = True
11         break
12 if(f):
13     print("Y")
14 else:
15     print("N")
```

10.17 Sliding Window

C++\Notebook\Sliding Window.cpp

Ventana de tamaño variable para hallar el subarreglo mas largo que cumple una condicion monotona (suma $\leq K$, a lo sumo K distintos, etc). Cada elemento entra y sale de la ventana a lo sumo una vez, así que el costo total es $O(n)$ aunque haya dos ciclos anidados. La condicion tiene que ser monotona: si un rango cumple, cualquier subrango tambien debe cumplir. Por eso funciona con sumas de numeros no negativos, pero NO si el arreglo tiene negativos (ahí encoger la ventana no garantiza reducir la suma y hay que usar prefijos con un map o un deque monotono).

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 int main()
6 {
7     ios_base::sync_with_stdio(false);
8     cin.tie(nullptr);
9
10    int n;
11    long long K;
12    cin >> n >> K;
```

```

8
14 vector<long long> a(n);
15 for (int i = 0; i < n; i++) cin >> a[i];
16
17 int l = 0;
18 long long cur = 0, ans = 0;
19
20 for (int r = 0; r < n; r++)
21 {
22     cur += a[r];
23     while (cur > K)
24     {
25         cur -= a[l];
26         l++;
27     }
28     ans = max(ans, (long long)(r - l + 1));
29 }
30
31 cout << ans << "\n";
32 return 0;
33 }

```

10.18 Sorted Pairing - Balatro

C++ (Codeforces/balatro.cpp)

Calcula la suma de diferencias absolutas entre pares ordenados de dos arreglos. Ordena ambos y suma las diferencias elemento a elemento.

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 int main() {
6
7     long long n;
8     cin >> n;
9     vector<long long > yo(n);
10    vector<long long > atk(n);
11
12    for(int i = 0; i<n ; i++)
13    {
14        long long a;
15        cin >> a;
16        yo[i] = a;
17    }
18
19    for(int i = 0; i<n ; i++)
20    {
21        long long a;
22        cin >> a;
23        atk[i] = a;
24    }
25
26    long long ans = 0;
27
28    sort(yo.begin(), yo.end());
29    sort(atk.begin(), atk.end());
30
31    for(int i = 0; i<n ; i++)
32    {
33        ans+= abs(atk[i] - yo[i]);
34    }
35
36    cout << ans << endl;
37
38    return 0;
39 }

```

10.19 Sorting - Finding Forks

C++ (Training_Camp_2026/Day 11 - CCPL - Finding Forks.py)

Resuelve "Finding Forks" (Day 11, problema A - CCPL (Kattis)): en un cajon con n tipos de tenedores, cada tipo con a_i unidades, al menos dos lugares del cajon quedaron vacios porque no se recuerda que tipo va en cada lugar tras lavarlos. Pide el minimo numero de tenedores que debieron estar en el lavaplatos para causar esa confusion. Ordena las cantidades con `l.sort()` y suma las dos mas pequenas ($l[0]+l[1]$), pues la ambigüedad minima se logra siempre entre los dos tipos con menos tenedores.

```

1 n = int(input())
2 l = list(map(int, input().split()))
3
4 l.sort()
5
6 print(l[0]+l[1])

```

10.20 Sorting (Prefix Sums) - Coven Complications

C++ (Training_Camp_2026/Day 11 - CCPL - Coven Complications.py)

Resuelve "Coven Complications" (Day 11, problema F - CCPL (Kattis)): un grafo de n comunidades brujas unidas por m portales, donde las comunidades con $f_i=0$ son propias y las que tienen $f_i>0$ ferrets son rivales. Cada día se sella una comunidad propia sin sellar, y ese día todas las demas comunidades propias aun sin sellar pierden brujas iguales a la suma de ferrets de sus comunidades rivales vecinas directas; pide el minimo total de brujas perdidas hasta sellar todas las comunidades propias. Calcula para cada comunidad propia su perdida diaria fija ('citiesW', suma de ferrets de los vecinos rivales via las aristas), y aplica la estrategia greedy clasica de minimizar sumas de tiempos de finalizacion: ordena esas perdidas de forma ascendente y suma las sumas prefijas ('pre') salvo la ultima, sellando primero las comunidades con mayor perdida diaria.

```

1 n, c = list(map(int, input().split()))
2
3 hurones = list(map(int, input().split()))
4
5 citiesW = [0]*(n+1)
6 for i in range(c):
7     a, b = list(map(int, input().split()))
8     a, b = (hurones[a-1], a), (hurones[b-1], b)
9     if a[0] == 0 or b[0] == 0:
10         obj = min(a,b)
11         citiesW[obj[1]] += max(a[0],b[0])
12
13 pre = []
14 citiesW.sort()
15 prev = 0
16 for i in citiesW:
17     prev += i
18     if i != 0:
19         pre.append(prev)
20
21 sol = 0
22 for i in range(len(pre)-1):
23     sol += pre[i]
24
25 print(sol)

```

10.21 Two Pointers (Container With Most Water)

C++ (Notebook/Container With Most Water.cpp)

Paredes verticales de altura $h[i]$ en la posicion i : elegir dos que, junto con el eje x , formen el recipiente de mayor area ($(j - i) * \min(h[i], h[j])$). Fuerza bruta es $O(n^2)$; con dos punteros desde los extremos es $O(n)$. Por que se mueve la pared MAS BAJA: el agua queda topada por ella. Cualquier recipiente entre esa pared y una interior es mas angosto y sigue topado por la misma altura (o menos), asi que nunca supera al actual: esa pared ya dio su mejor recipiente y se puede descartar. Si miden lo mismo, ninguna de las dos mejora con una pared interior: mover cualquiera. El area puede pasar de $2^{\wedge}31$ ($1e9 * 1e5$): usar long long. No confundir con Trapping Rain Water (el agua que retienen TODAS las barras): tambien es de dos punteros, pero en cada posicion suma $\min(\maxLzq, \maxDer) - h[i]$.

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 typedef long long ll;
6
7 // {area maxima, {i, j}} (i = j = -1 si hay menos de dos paredes)
8 pair<ll, pair<int, int>> maxArea(const vector<ll> &h)
9 {
10     int l = 0, r = (int)h.size() - 1, bi = -1, bj = -1;
11     ll best = 0;
12     while (l < r)

```

```

3   {
4       ll a = (ll)(r - 1) * min(h[l], h[r]);
5       if (a > best || bi == -1)
6       {
7           best = a;
8           bi = l;
9           bj = r;
10      }
11      if (h[l] < h[r]) l++;
12      else r--;
13  }
14  return {best, {bi, bj}};
15  }
16
17  int main()
18  {
19      ios_base::sync_with_stdio(false);
20      cin.tie(nullptr);
21
22      int n;
23      cin >> n;
24      vector<ll> h(n);
25      for (auto &x : h) cin >> x;
26
27      auto r = maxArea(h);
28      cout << r.first << "\n";
29      cout << r.second.first << " " << r.second.second << "\n";
30      return 0;
31  }

```

11 - Binary Search

11.1 Greedy (busqueda binaria sobre la respuesta) - Minas Gerais' Walls

C++\Training_Camp_2026\Day 5\B - Minas Gerais' walls.cpp

Resuelve "Minas Gerais' walls" (Day 6, problema M - Maratona SBC de Programacao 2025): una muralla de N segmentos consecutivos con alturas x_i se refuerza una unica vez eligiendo un segmento, que recibe K bloques extra, el segmento a su izquierda $K-1$, y asi en forma de escalera hasta sumar solo 1 bloque o quedarse sin segmentos a la izquierda; la fortaleza de la muralla es su altura minima, y hay que maximizar esa altura minima tras el unico refuerzo. El codigo hace busqueda binaria sobre la altura candidata H usando "feasible(H)": para cada H calcula el rango de posiciones p de refuerzo que alcanzarian a subir todo segmento deficitario ($x_i < H$) hasta al menos H , a partir de la formula del escalon $p \leq i + K - H + x_i$, y verifica que ese rango $[lo, hi]$ sea no vacio y quede dentro de $[1, N]$.

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  typedef long long ll;
4
5  int N;
6  ll K;
7  vector<ll> x; // 0-indexed internamente, x[i] = altura del segmento (i+1)
8
9  // Dado un candidato H, ¿existe algun p tal que reforzando en p,
10 // TODOS los segmentos queden con altura >= H?
11 bool feasible(ll H) {
12     ll l = 0; // mayor indice (1-indexado) con x[i] < H, o 0 si no hay ninguno
13     ll U = LLONG_MAX; // cota superior para p, viene de la condicion de "alcanzar" cada defici
14     it
15
16     for (int i = 1; i <= N; i++) {
17         if (x[i-1] < H) {
18             l = i; // como i crece, la ultima asignacion es el maximo indice con deficit
19             // Necesitamos: K - (p - i) >= H - x[i-1] => p <= i + K - H + x[i-1]
20             ll candidate = (ll)i + K - H + x[i-1];
21             U = min(U, candidate);
22         }
23     }
24     ll lo = max((ll)1, l);
25     ll hi = min((ll)N, U);
26     return lo <= hi;
27 }
28
29 int main(){

```

```

30     ios_base::sync_with_stdio(false);
31     cin.tie(NULL);
32
33     cin >> N >> K;
34     x.resize(N);
35     for (int i = 0; i < N; i++) cin >> x[i];
36
37     ll lo = *min_element(x.begin(), x.end()); // siempre alcanzable (bonus >= 0 siempre)
38     ll hi = *max_element(x.begin(), x.end()) + K; // cota superior segura
39
40     ll ans = lo;
41     while (lo <= hi) {
42         ll mid = lo + (hi - lo) / 2;
43         if (feasible(mid)) {
44             ans = mid;
45             lo = mid + 1;
46         } else {
47             hi = mid - 1;
48         }
49     }
50
51     cout << ans << "\n";
52     return 0;
53 }

```

11.2 Two Pointers - Black Hills Golden Jewels

C++\Training_Camp_2026\Day 5\B - Black Hills golden jewels.cpp

Resuelve "Black Hills golden jewels" (Day 5, problema B - Contest 05 [Avanzados]): dado el precio a_i de cada una de N categorias de joyas, un cliente debe elegir dos categorias distintas, y ningun cliente puede repetir una combinacion ya elegida por otro; hay que hallar el monto minimo que garantiza que el K -esimo cliente en la fila pueda comprar, es decir, la K -esima suma de pares $(a_i + a_j)$ mas pequena entre todos los pares posibles. Hace busqueda binaria sobre la respuesta (variable mid entre lo y hi) y para cada candidato cuenta, con dos punteros (i creciente, j decreciente) sobre el arreglo ordenado, cuantos pares tienen suma $\leq mid$, ajustando el rango hasta converger al K -esimo valor.

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  using ll = long long;
5
6  int main() {
7      ios::sync_with_stdio(false);
8      cin.tie(nullptr);
9
10     int N;
11     ll K;
12     cin >> N >> K;
13
14     vector<ll> a(N);
15
16     for (int i = 0; i < N; i++)
17         cin >> a[i];
18
19     sort(a.begin(), a.end());
20
21     // La respuesta est  entre la suma m nima y m xima posibles.
22     ll lo = a[0] + a[1];
23     ll hi = a[N - 1] + a[N - 2];
24
25     while (lo < hi) {
26
27         ll mid = (lo + hi) / 2;
28
29         /*
30          Contaremos cu ntos pares tienen suma <= mid.
31
32          Usamos dos punteros.
33
34          Para cada i, buscamos el mayor j tal que
35
36              a[i] + a[j] <= mid
37
38          Entonces todos los  ndices entre
39          (i+1 ... j) tambi n sirven.

```

```

40  */
41  ll cnt = 0;
42
43  int j = N - 1;
44
45  for (int i = 0; i < N; i++) {
46      while (j > i && a[i] + a[j] > mid)
47          j--;
48
49      if (j <= i)
50          break;
51
52      cnt += (j - i);
53  }
54
55  if (cnt >= K)
56      hi = mid;
57  else
58      lo = mid + 1;
59  }
60
61  cout << lo << '\n';
62
63  return 0;
64 }

```

12 - Constructive Algorithms

12.1 Hashing - Handling The Blocks

C++(Training_Camp_2026/Day 4/E - Handling the Blocks.py)

Resuelve "Handling the Blocks" (Day 4, problema E - Contest 04 [Avanzados]): N bloques numerados $1..N$ tienen cada uno un color, y en cada turno se pueden intercambiar dos bloques que compartan color; hay que decidir si es posible ordenar la secuencia de forma ascendente por n mero. Como los intercambios solo mueven n meros entre bloques del mismo color, la posici n i solo puede terminar con el n mero $(i+1)$ si ese n mero pertenece al mismo color que el bloque originalmente ubicado en la posici n i ; el c digo arma por color (lista l de sets) los n meros que comparten cada color y verifica esa condici n posici n por posici n.

```

1  n, k = list(map(int,input().split()))
2  l = []
3  for i in range(k):
4      l.append(set())
5  val = []
6  for i in range(n):
7      c, v = list(map(int,input().split()))
8      l[v-1].add(c)
9      val.append([c,v])
10
11
12  f = True
13  for i in range(n):
14      #h = val[i][0]
15      if(i+1 in l[val[i][1]-1]):
16          continue
17      else:
18          f = False
19          break
20
21  if(f):
22      print("Y")
23  else:
24      print("N")

```

12.2 Secuencias de Langford - Doubled Sequence II

C++(Training_Camp_2026/Day 9/C - Doubled Sequence II.cpp)

Resuelve "Doubled Sequence II" (Day 9, problema C - Contest 08 [Inicial & Avanzado]): para un n dado, construir una secuencia de longitud $2n$ donde cada valor de 1 a n aparece exactamente dos veces y, para cada i , la distancia entre sus dos ocurrencias es exactamente $i+1$ (esto es una variante de las secuencias de Langford/Skolem). Solo existe soluci n cuando $n \% 4$ es 0 o 3 ; en otro caso imprime -1 . Cuando existe, usa la construcci n expl cita de Davies (1959) (funciones `build4m` y `build4mMinus1`) que arma la secuencia por bloques crecientes/decrecientes de paridad fija (`appendRange`) seg n n sea $4m$ o $4m-1$, con casos base para $n=3$ y $n=4$.

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  static char buf_[1 << 22];
5  static int buflen_, bufPos_;
6  inline int gc() {
7      if (bufPos_ == buflen_) { buflen_ = fread(buf_, 1, sizeof(buf_), stdin); bufPos_ = 0; if (!buflen_) return -1; }
8      return buf_[bufPos_++];
9  }
10 inline int readInt() {
11     int c = gc();
12     while (c != -1 && (c < '0' || c > '9') && c != '-') c = gc();
13     bool neg = false; if (c == '-') { neg = true; c = gc(); }
14     int x = 0;
15     while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = gc(); }
16     return neg ? -x : x;
17 }
18
19 string out;
20
21 inline void appendInt(int v) {
22     char tmp[12];
23     int len = 0;
24     if (v == 0) { tmp[len++] = '0'; }
25     else {
26         int x = v;
27         while (x > 0) { tmp[len++] = char('0' + x % 10); x /= 10; }
28     }
29     for (int i = len - 1; i >= 0; i--) out += tmp[i];
30     out += ' ';
31 }
32
33 // Appends consecutive even/odd terms from a to b inclusive (a,b same parity),
34 // increasing if a<=b, decreasing if a>b.
35 inline void appendRange(int a, int b) {
36     if (a <= b) {
37         for (int v = a; v <= b; v += 2) appendInt(v);
38     } else {
39         for (int v = a; v >= b; v -= 2) appendInt(v);
40     }
41 }
42
43 // Davies (1959) construction for n = 4m, m >= 2.
44 void build4m(int m) {
45     appendRange(4*m-4, 2*m);
46     appendInt(4*m-2);
47     appendRange(2*m-3, 1);
48     appendInt(4*m-1);
49     appendRange(1, 2*m-3);
50     appendRange(2*m, 4*m-4);
51     appendInt(4*m);
52     appendRange(4*m-3, 2*m+1);
53     appendInt(4*m-2);
54     appendRange(2*m-2, 2);
55     appendInt(2*m-1);
56     appendInt(4*m-1);
57     appendRange(2, 2*m-2);
58     appendRange(2*m+1, 4*m-3);
59     appendInt(2*m-1);
60     appendInt(4*m);
61 }
62

```

```

63 // Davies (1959) construction for n = 4m-1, m >= 2.
64 void build4mMinus1(int m) {
65     appendRange(4*m-4, 2*m);
66     appendInt(4*m-2);
67     appendRange(2*m-3, 1);
68     appendInt(4*m-1);
69     appendRange(1, 2*m-3);
70     appendRange(2*m, 4*m-4);
71     appendInt(2*m-1);
72     appendRange(4*m-3, 2*m+1);
73     appendInt(4*m-2);
74     appendRange(2*m-2, 2);
75     appendInt(2*m-1);
76     appendInt(4*m-1);
77     appendRange(2, 2*m-2);
78     appendRange(2*m+1, 4*m-3);
79 }
80
81 int main() {
82     int t = readInt();
83     out.reserve(1 << 22);
84     while (t--) {
85         int n = readInt();
86         int r = n % 4;
87         if (r != 0 && r != 3) {
88             out += "-1\n";
89             continue;
90         }
91         if (n == 3) {
92             out += "2 3 1 2 1 3\n";
93         } else if (n == 4) {
94             out += "2 3 4 2 1 3 1 4\n";
95         } else if (r == 0) {
96             build4m(n / 4);
97             out.back() = '\n';
98         } else { // r == 3, n = 4m-1 with m = (n+1)/4
99             build4mMinus1((n + 1) / 4);
100             out.back() = '\n';
101         }
102     }
103     fwrite(out.data(), 1, out.size(), stdout);
104     return 0;
105 }

```

13 - Implementation

13.1 Ad Hoc - Toilet Seat

C++\Training_Camp_2026\Day 11 - CCPLC - Toilet Seat.cpp

Resuelve "Toilet Seat" (Day 11, problema C - CCPL (Kattis)): dada una secuencia de preferencias 'U' (arriba) o 'D' (abajo) de personas que usan el baño una tras otra, donde el primer carácter es la posición inicial del asiento, pide calcular cuantos ajustes de asiento se necesitan bajo tres políticas: (1) siempre dejarlo arriba, (2) siempre dejarlo abajo, (3) dejarlo como a uno le gustaria encontrarlo. Recorre la cadena una sola vez calculando en paralelo ans1 y ans2 (contando cuantas veces el asiento debe moverse segun cada política fija) y ans3 (contando los cambios de preferencia entre personas consecutivas, que es el costo bajo la política de dejarlo "como te gustaria encontrarlo").

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 int main() {
7     ios::sync_with_stdio(false);
8     cin.tie(nullptr);
9
10    string s;
11    cin >> s;
12    ll ans1=0, ans2=0, ans3=0;
13
14    //1
15    if(s[0]=='U' && s[1]=='U') ans1+=0;
16    else if(s[0]=='U' && s[1]=='D') ans1+=2;
17    else if(s[0]=='D' && s[1]=='D') ans1+=1;

```

```

18    else ans1+=1;
19    for(int i = 2; i < s.size(); i++)
20    {
21        if(s[i] == 'D') ans1+=2;
22    }
23
24    //2
25    if(s[0]=='U' && s[1]=='U') ans2+=1;
26    else if(s[0]=='U' && s[1]=='D') ans2+=1;
27    else if(s[0]=='D' && s[1]=='D') ans2+=0;
28    else ans2+=2;
29    for(int i = 2; i < s.size(); i++)
30    {
31        if(s[i] == 'U') ans2+=2;
32    }
33
34    //3
35    char act = s[0];
36    for(int i = 1; i < s.size(); i++)
37    {
38        if(s[i] != act)
39        {
40            ans3++;
41            act = s[i];
42        }
43    }
44
45    cout << ans1 << endl;
46    cout << ans2 << endl;
47    cout << ans3 << endl;
48
49    return 0;
50 }

```

13.2 Ad Hoc - Rock Paper Scissors Tournament

C++\Training_Camp_2026\Day 11 - CCPLD - Rock-Paper-Scissors Tournament.py

Resuelve "Rock-Paper-Scissors Tournament" (Day 11, problema D - CCPL (Kattis)): en un torneo de piedra-papel-tijera con n jugadores, cada par de jugadores disputa k partidas ($k(n-1)/2$ partidas en total); dado el movimiento de cada jugador en cada partida, pide el promedio de victorias $w/(w+l)$ de cada jugador, redondeado a tres decimales, o "-" si nunca gana ni pierde. Simula partida por partida, aplicando las reglas de piedra-papel-tijera para acumular victorias en 'w' y derrotas en 'l' por jugador, y al final calcula e imprime el promedio de cada uno.

```

1 n = list(map(int, input().split()))
2 if len(n) != 1:
3     n, k = n
4     while(n != 0):
5         if n == 1:
6             print("-")
7             n = list(map(int, input().split()))
8             if len(n) == 1:
9                 break
10            else:
11                n, k = n
12            w = [0]*(n+1)
13            l = [0]*(n+1)
14
15            for i in range((k*n*(n-1))/2):
16                num1, res1, num2, res2 = input().split()
17                num1, num2 = int(num1), int(num2)
18                if res1 == res2:
19                    continue
20                if res1 == "rock" and res2 == "paper":
21                    w[num2] += 1
22                    l[num1] += 1
23                elif res1 == "rock" and res2 == "scissors":
24                    w[num1] += 1
25                    l[num2] += 1
26                elif res1 == "paper" and res2 == "rock":
27                    w[num1] += 1
28                    l[num2] += 1
29                elif res1 == "paper" and res2 == "scissors":
30                    w[num2] += 1

```

```
31         l[num1] += 1
32     elif res1 == "scissors" and res2 == "rock":
33         w[num2] += 1
34         l[num1] += 1
35     elif res1 == "scissors" and res2 == "paper":
36         w[num1] += 1
37         l[num2] += 1
38
39     for i in range(1, n+1):
40         wins = w[i]
41         loses = l[i]
42
43         if wins+loses == 0:
44             print("-")
45         else:
46             print(f"({wins/(wins+loses)}:.3f)")
47     print("")
48     n = list(map(int, input().split()))
49     if len(n) == 1:
50         break
51     else:
52         n, k = n
```

13.3 Ad Hoc - Zero Or One

C++\Training_Camp_2026\Day 2\A - Zero or One.py

Resuelve "Zero or One" (Day 2, problema A - Maratona de Programacao da SBC 2013): Alice, Beto y Clara juegan "cero o uno" mostrando cada uno la mano con un valor 0 o 1; gana quien elige un valor distinto al de los otros dos, y si nadie queda en esa situacion (todos iguales, o empate 2-1 sin un unico distinto) no hay ganador. Usa la suma A+B+C para detectarlo sin comparar par por par: si la suma es 1, el unico que voto 1 es el ganador; si la suma es 2, el unico que voto 0 es el ganador; en cualquier otro caso (suma 0 o 3) no hay ganador y se imprime "*".

```
1 a, b, z = list(map(int, input().split()))
2
3 if 0<(a+b+z)< 3:
4     if (a+b+z == 2 and a == 0) or (a+b+z == 1 and a == 1):
5         print("A")
6     if (a+b+z == 2 and b == 0) or (a+b+z == 1 and b == 1):
7         print("B")
8     if (a+b+z == 2 and z == 0) or (a+b+z == 1 and z == 1):
9         print("C")
10 else:
11     print('*')
```

13.4 Ad Hoc - Collatz Polynomial

C++\Training_Camp_2026\Day 6\C - Collatz polynomial.py

Resuelve "Collatz polynomial" (Day 6, problema C - Maratona SBC de Programacao 2025): Aline crea una variante de la Conjetura de Collatz para polinomios con coeficientes 0 o 1. Si el polinomio tiene termino independiente, se multiplica por $(x+1)$ y se le suma 1, descartando cualquier termino cuyo coeficiente resulte 2; si no tiene termino independiente, se divide entre x . Hay que contar cuantas operaciones hacen falta para llegar a $P(x) = 1$. El codigo simula el proceso literalmente sobre la lista "val" (coeficiente de x^i en la posicion i): hace `val.pop(0)` para dividir entre x , o construye una nueva lista "n" desplazando y sumando (multiplicacion por $x+1$, mas 1 en el termino independiente) y poniendo en 0 los coeficientes que dan 2, contando los pasos en "c" hasta que el polinomio quede reducido a 1.

```
1 grado = int(input())
2 val = list(reversed(list(map(int,input().split()))))
3
4 m = [1, 1]
5 c = 0
6 while val.count(1) > 1 or val[0] != 1:
7     #print(val)
8     if(val[0] == 1):
9         n = [0] * len(val)
10        for i in range (len(val)):
11            if(val[i]==0):
12                continue
13            for j in range(len(m)):
14                try:
15                    n[i+j]+=1
16                except:
17                    n.append(1)
18        n[0]+=1
19        for i in range (len(val)):
```

```
20        if(n[i]>=2):
21            n[i]=0
22        val = n
23    else:
24        val.pop(0)
25    #if(c==5):
26        # break
27    c+= 1
28 print(c)
```

13.5 Ad Hoc - Unraveling Monty Hall

C++\Training_Camp_2026\Day 8\D - Unraveling Monty Hall.cpp

Resuelve "Unraveling Monty Hall" (Day 8, problema D - Contest 08 [Avanzados]): se simulan N juegos del problema de Monty Hall donde el jugador siempre elige la puerta 1 y luego siempre cambia a la otra puerta cerrada tras que el presentador revela una cabra; dado en que puerta esta el auto en cada juego, hay que contar en cuantos de ellos el jugador gana el auto. Como el jugador parte de la puerta 1 y el presentador solo puede abrir una puerta con cabra entre las dos no elegidas, cambiar de puerta gana el auto exactamente cuando el auto esta en la puerta 2 o en la puerta 3; el codigo simplemente cuenta cuantas de las N entradas valen 2 o 3 (variable "sum").

```
1 #include <iostream>
2 using namespace std;
3 int main()
4 {
5     int n; cin >> n;
6     int sum = 0;
7     while(n--)
8     {
9         int a; cin >>a;
10        if(a==3 or a==2)
11            {
12                sum++;
13            }
14    }
15    cout << sum << endl;
16    //a
17    return 0;
18 }
```

13.6 Arreglos - Buffoon

C++\Training_Camp_2026\Day 1\B - Buffoon.py

Resuelve "Buffoon" (Day 1, problema B - Maratona de Programacao da SBC 2019): en la eleccion del bufon de la corte, N candidatos reciben votos y gana el mas votado, rompiendo empates a favor de quien se registro primero. Carlos se asegura de ser el primero en registrarse, así que basta comparar si sus votos (primer valor de la lista) son iguales al maximo del arreglo para saber si gano ("S") o no ("N").

```
1 n = int(input())
2 l = []
3 for i in range(n):
4     l.append(int(input()))
5
6 ma = max(l)
7 if(ma == l[0]):
8     print("S")
9 else:
10    print("N")
```

13.7 Backtracking (Enumeracion de bits desconocidos) - Harmonics With Interference

C++\Training_Camp_2026\Day 3\H - Harmonics with Interference.py

Resuelve "Harmonics with Interference" (Day 3, problema H - Maratona SBC de Programacao 2024): Arthur envia un mensaje binario M junto con una secuencia de control N tal que el entero representado por M es divisible por el entero representado por N ; Bruna recibe M' y N' donde algunos bits se perdieron y aparecen como '*' (a lo sumo 16 simbolos '*' en total entre ambos), y hay que reconstruir un M valido compatible con lo recibido, o determinar que no existe. El codigo genera por backtracking (`generateBins`) todos los valores posibles de N' rellenando sus '*' con 0 o 1, y luego, tambien por backtracking (`generateAndCheck`), prueba todas las combinaciones de los '*' restantes en M' (peso guarda el valor posicional de cada '*'), devolviendo el primer M cuyo valor entero sea divisible por algun N generado; como el total de incognitas nunca supera 16, la enumeracion completa (2^{16} en el peor caso) es factible.

```
1 import copy
2 import sys
3
4 sys.setrecursionlimit(3*10**6)
5 m = input().strip()
```

```

6 n = input().strip()
7
8 ns = []
9 def generateBins(s, i, objective):
10     if i < 0:
11         objective.append(int("".join(s), 2))
12         return
13     elif s[i] == "*":
14         way = copy.copy(s)
15         way[i] = "1"
16         generateBins(way, i-1, objective)
17         way = copy.copy(s)
18         way[i] = "0"
19         generateBins(way, i-1, objective)
20     else:
21         generateBins(s, i-1, objective)
22
23 generateBins(list(n), len(n)-1, ns)
24
25
26 L = len(m)
27 estrellas = [idx for idx, c in enumerate(m) if c == "*"]
28 base = int("".join("0" if c == "*" else c for c in m), 2) # M con todos los * = 0
29 peso = [1 << (L - 1 - idx) for idx in estrellas] # lo que suma poner un 1 en ese *
30
31 def generateAndCheck(t, val):
32     if t == len(estrellas):
33         for k in ns:
34             if k != 0 and val % k == 0:
35                 print(bin(val)[2:].zfill(L)) # una sola vez, ya con la longitud correcta
36                 sys.exit(0)
37         return
38     generateAndCheck(t + 1, val + peso[t]) # ese * = 1
39     generateAndCheck(t + 1, val) # ese * = 0
40
41 generateAndCheck(0, base)
42 print(-1)

```

13.8 Brute Force - Cake Hater

C++ Training_Camp_2020/Day 10/G - Cake Hater.py

Resuelve "Cake Hater" (Day 10, problema G - Contest 09 [Avanzados]): una receta de pastel con N pasos, cada uno con su propia lista de ingredientes; Antonio omite un conjunto fijo de B ingredientes que no le gustan (donts) de un total de A . El pastel se arruina en el primer paso donde faltan MAS de un tercio de sus ingredientes. Simula paso por paso: para cada paso cuenta cuantos de sus ingredientes estan en el set donts (counter) y compara contra $\text{ingreds}[0]/3$ (el tamaño del paso); en cuanto counter supera ese umbral, imprime el índice del paso ($i+1$) y detiene la busqueda; si ningun paso lo supera, imprime -1 .

```

1 n = int(input())
2
3 a, b = list(map(int, input().split()))
4
5 donts = set(map(int, input().split()))
6
7 sol = False
8 for i in range(n):
9     ingreds = list(map(int, input().split()))
10
11     counter = 0
12     if not sol:
13         for j in range(1, ingreds[0]+1):
14             if ingreds[j] in donts:
15                 counter += 1
16             if counter > ingreds[0]/3:
17                 print(i+1)
18                 sol = True
19                 break
20 if not sol:
21     print(-1)

```

13.9 Brute Force - LLMs

C++ Training_Camp_2020/Day 6/L - LLMs.cpp

Resuelve "LLMs" (Day 6, problema L - Maratona SBC de Programacao 2025): Bruno arma un predictor de palabras "SBC" a partir de un diccionario de palabras con vectores 2D, un texto base de conocimiento y consultas; para cada consulta, se buscan en el texto base todas las ocurrencias consecutivas de las ultimas K palabras (ventana de contexto), se toman las palabras siguientes como "candidatas" (si no hay ninguna, se baja K en 1 y se repite hasta $K=0$), y se elige la palabra del diccionario que maximiza el producto punto de su vector contra la suma de los vectores candidatos (empate: la mas comun, primera en el diccionario). El codigo hace esto de forma directa: busqueda por fuerza bruta de "ctx" dentro de "kb" con bucles anidados para armar "candidates", suma sus vectores en (sumX, sumY), y recorre linealmente las N palabras del diccionario para hallar el maximo "bestS" de $\text{dictVec}[i].x * \text{sumX} + \text{dictVec}[i].y * \text{sumY}$.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main(){
5     ios_base::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int N;
9     cin >> N;
10
11     vector<string> dictWord(N);
12     vector<pair<long long, long long>> dictVec(N);
13     unordered_map<string, int> firstIndex; // first occurrence of a word in the dictionary
14
15     for(int i = 0; i < N; i++){
16         string w; long long x, y;
17         cin >> w >> x >> y;
18         dictWord[i] = w;
19         dictVec[i] = {x, y};
20         if(firstIndex.find(w) == firstIndex.end())
21             firstIndex[w] = i;
22     }
23
24     int M;
25     cin >> M;
26     vector<string> kb(M);
27     for(int i = 0; i < M; i++)
28         cin >> kb[i];
29
30     int Q, K;
31     cin >> Q >> K;
32
33     for(int q = 0; q < Q; q++){
34         int F;
35         cin >> F;
36         vector<string> query(F);
37         for(int i = 0; i < F; i++)
38             cin >> query[i];
39
40         int curK = K;
41         vector<string> candidates;
42
43         while(curK >= 1){
44             // last curK words of the ORIGINAL query
45             vector<string> ctx(query.end() - curK, query.end());
46             candidates.clear();
47
48             // search knowledge base for consecutive occurrences of ctx
49             for(int j = 0; j + curK - 1 < M; j++){
50                 bool match = true;
51                 for(int t = 0; t < curK; t++){
52                     if(kb[j + t] != ctx[t]){ match = false; break; }
53                 }
54                 if(match){
55                     int nextPos = j + curK;
56                     if(nextPos < M){
57                         candidates.push_back(kb[nextPos]);
58                     }
59                 }
60             }
61         }

```

```

62         if(!candidates.empty()) break;
63         curK--;
64     }
65
66     // print the query words
67     for(int i = 0; i < F; i++){
68         cout << query[i];
69         cout << " ";
70     }
71
72     if(candidates.empty()){
73         cout << "\n";
74         continue;
75     }
76
77     // Precompute candidate vectors (0,0) if word not in dictionary
78     vector<pair<long long,long long>> candVecs;
79     candVecs.reserve(candidates.size());
80     for(auto &c : candidates){
81         auto it = firstIndex.find(c);
82         if(it != firstIndex.end())
83             candVecs.push_back(dictVec[it->second]);
84         else
85             candVecs.push_back({0,0});
86     }
87
88     // sum of candidate vectors, so S(d) = d . sum(candidates)
89     long long sumX = 0, sumY = 0;
90     for(auto &cv : candVecs){
91         sumX += cv.first;
92         sumY += cv.second;
93     }
94
95     long long bestS = LLONG_MIN;
96     int bestIdx = -1;
97
98     for(int i = 0; i < N; i++){
99         long long s = dictVec[i].first * sumX + dictVec[i].second * sumY;
100         if(s > bestS){
101             bestS = s;
102             bestIdx = i;
103         }
104     }
105
106     cout << dictWord[bestIdx] << "\n";
107 }
108
109 return 0;
110 }

```

13.10 Circular Queue - Eeny Meeny

C++\Training_Camp_2026\Day 11 - CCPL\B - Eeny Meeny.py

Resuelve "Eeny Meeny" (Day 11, problema B - CCPL (Kattis)): un grupo de niños en círculo es dividido en dos equipos aplicando repetidamente una rima (como "eeny meeny miny moe"): se cuenta en sentido horario, saltando a los niños ya elegidos, hasta la última palabra de la rima, que selecciona al siguiente niño para el equipo actual, alternando de equipo en cada ronda. Pide reconstruir los dos equipos en el orden en que fueron elegidos. Simula la ronda con un índice `curr` que recorre circularmente la lista `people`, el conjunto `fuera` para saltar a los ya seleccionados, y un contador `currN` que cuenta palabras hasta llegar a `n` (longitud de la rima), alternando el equipo destino `group` cada vez que se elige a alguien.

```

1 n = input().split()
2
3 n = len(n)
4
5 k = int(input())
6
7 people = []
8
9 for i in range(k):
10     nombre = input()
11     people.append(nombre)
12
13 curr = 0
14 fuera = set()

```

```

5 group = 0
6 groups = [[], []]
7 curr = 0
8 currN = 0
9 while(len(fuera) != len(people)):
10     if curr == k:
11         curr = 0
12     while curr in fuera:
13         curr+=1
14     if curr == k:
15         curr = 0
16     currN += 1
17     if currN == n:
18         groups[group].append(people[curr])
19         fuera.add(curr)
20         currN = 0
21     if group == 1:
22         group = 0
23     else:
24         group = 1
25     curr += 1
26
27 for i in groups:
28     print(len(i))
29     for j in i:
30         print(j)

```

13.11 Clasificación de Cadenas por Prefijo - Joker Or Judger

C++\Codeforces\JokerOrJudger.cpp

Cada reporte que llega es una cadena de una lista fija y vale cierto puntaje: *UnreasonableProblemArrangement* vale 10, *WrongProblemX* vale 100, *SameProblemX* vale 30, *UnreasonableLimitForProblemX* vale 5, *WeakTestsForProblemX* vale 3 y *BadProblemX* vale 1, donde X es una letra de la A a la L (los 12 problemas del contest). Cualquier otra cosa vale 0. Se suman los reportes del caso y se responde Joker si el total pasa de P, y Judger si no. De algoritmo no tiene nada: es lectura cuidadosa. Lo único que decide si pasa o no es no equivocarse contando caracteres, y por eso vale la pena tener la plantilla lista. LA CLAVE: validar `s.size()` ANTES de comparar el prefijo, y ese `size()` no es decorativo. `s.substr(0, 12)` sobre una cadena mas corta NO lanza excepción, simplemente devuelve lo que alcance, así que comparar solo el prefijo dejaría pasar "BadProblemAA" o "BadProblemABC". Fijar el largo exacto es lo que obliga a que haya EXACTAMENTE una letra después del prefijo, y por eso `s.back()` de una vez es la letra del problema. Las seis ramas son mutuamente excluyentes porque cada una fija un largo distinto: 11, 12, 13, 20, 28 y 30. O sea que el orden de los if da igual, no hay una que tape a otra. Vale la pena revisarlo cuando se escribe algo así, porque si dos plantillas compartieran largo Y prefijo, el orden sí mandaría y el bug sería invisible. Verificado que los seis largos cuadran con sus prefijos $(\text{len}(\text{prefijo}) + 1 == \text{size})$, que es justo el off-by-one que se cuele solo en este tipo de problema. Alternativas para comparar prefijos, por si el largo no se puede fijar: `s.find(prefijo, 0) == 0` // busca el prefijo solo en la posición 0 `s.compare(0, k, prefijo) == 0` // sin crear el substring temporal Y ojo con el comparador final: es total > P estricto, empatar con P es Judger.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int score(string s) {
5     if (s == "UnreasonableProblemArrangement")
6         return 10;
7
8     if (s.size() == 13 && s.substr(0, 12) == "WrongProblem" &&
9         s.back() >= 'A' && s.back() <= 'L')
10         return 100;
11
12     if (s.size() == 12 && s.substr(0, 11) == "SameProblem" &&
13         s.back() >= 'A' && s.back() <= 'L')
14         return 30;
15
16     if (s.size() == 28 && s.substr(0, 27) == "UnreasonableLimitForProblem" &&
17         s.back() >= 'A' && s.back() <= 'L')
18         return 5;
19
20     if (s.size() == 20 && s.substr(0, 19) == "WeakTestsForProblem" &&
21         s.back() >= 'A' && s.back() <= 'L')
22         return 3;
23
24     if (s.size() == 11 && s.substr(0, 10) == "BadProblem" &&
25         s.back() >= 'A' && s.back() <= 'L')
26         return 1;

```

```

27
28     return 0;
29 }
30
31 int main() {
32     ios::sync_with_stdio(false);
33     cin.tie(nullptr);
34
35     int T;
36     cin >> T;
37
38     while (T--) {
39         int n, P;
40         cin >> n >> P;
41
42         int total = 0;
43
44         while (n--) {
45             string s;
46             cin >> s;
47             total += score(s);
48         }
49
50         cout << (total > P ? "Joker" : "Judge") << '\n';
51     }
52 }

```

13.12 Conjuntos - João João

C++ \Training_Camp_2025\Day 6\ - João João.py

Resuelve "Joao Joao" (Day 6, problema J - Maratona SBC de Programacao 2025): ya se crearon 10 tareas para un examen, cada una con un nivel de dificultad entre 1 y 4, y se necesita saber cuantas tareas nuevas hacen falta para poder armar un examen con exactamente 4 tareas, una de cada nivel de dificultad distinto. El código mete las 10 dificultades en un set "n" (que colapsa los duplicados y deja solo los niveles distintos ya cubiertos) e imprime 4 menos el tamaño de ese set, es decir, cuantos niveles de dificultad todavía faltan por cubrir.

```

1 n = set(map(int,input().split()))
2 print(4-len(n))

```

13.13 Constructive - A Story About Tea

C++ \Training_Camp_2025\Day 7\F - A story about tea.cpp

Resuelve "A story about tea" (Day 7, problema F - Contest 07 [Avanzados]): N botes deben viajar de Portugal a Inglaterra usando China como puerto auxiliar, siguiendo las reglas de las Torres de Hanoi (cada viaje mueve el bote mas chico entre las dos pilas involucradas, y nunca hay dos viajes simultaneos), en exactamente K viajes; hay que decidir si es posible y, de serlo, imprimir la secuencia de K viajes. El mínimo de viajes es $2^N - 1$ (funcion h, la recursion clasica de Hanoi); si K es mayor, completa la diferencia con pares de movimientos "de relleno" A-B-A que no cambian el estado (delta/2 veces) y, si sobra un viaje impar, usa la variante g que gasta un movimiento extra en el ultimo bote; si $K < 2^N - 1$ no hay solucion posible ("N").

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 string out;
5
6 inline void addMove(char from, char to) {
7     out += from;
8     out += ' ';
9     out += to;
10    out += '\n';
11 }
12
13 // mueve n botes de X a Y usando Z como auxiliar, minimal ( $2^n - 1$  movimientos)
14 void h(int n, char X, char Y, char Z) {
15     if (n == 0) return;
16     h(n - 1, X, Z, Y);
17     addMove(X, Y);
18     h(n - 1, Z, Y, X);
19 }
20
21 // mueve n botes de X a Y usando Z como auxiliar, con exactamente 1 movimiento extra ( $2^n$  movimient
22 os)
23 void g(int n, char X, char Y, char Z) {
24     if (n == 1) {
25         addMove(X, Z);

```

```

25     addMove(Z, Y);
26     return;
27 }
28 h(n - 1, X, Z, Y);
29 addMove(X, Y);
30 g(n - 1, Z, Y, X);
31 }
32
33 int main() {
34     long long N, K;
35     cin >> N >> K;
36
37     long long minK = (1LL << N) - 1;
38
39     if (K < minK) {
40         cout << "N\n";
41         return 0;
42     }
43
44     long long delta = K - minK;
45     bool useG = (delta % 2 == 1);
46     long long padCount = useG ? (delta - 1) / 2 : delta / 2;
47
48     out.reserve((size_t)K * 4 + 16);
49
50     for (long long i = 0; i < padCount; i++) {
51         addMove('A', 'B');
52         addMove('B', 'A');
53     }
54
55     if (useG) {
56         g((int)N, 'A', 'C', 'B');
57     } else {
58         h((int)N, 'A', 'C', 'B');
59     }
60
61     cout << "\n";
62     cout << out;
63     return 0;
64 }

```

13.14 Conteo de Records - Harder Horizons

C++ \ICPC\Regionals_2025\H - Harder Horizons.cpp

Resuelve "Harder Horizons" (problema H, Regionals 2025): cuenta cuantos elementos son mayores que TODOS los anteriores, o sea los "records" o maximos de prefijo. Una sola pasada llevando el maximo visto: si el actual lo supera, se cuenta y se actualiza. $O(n)$ y memoria $O(1)$, sin guardar el arreglo. El maxi arranca en -1 en vez de 0, que importa si los valores pueden ser 0: con maxi = 0 un primer elemento igual a 0 no se contaria. Si los valores pudieran ser negativos habria que arrancar en LLONG_MIN, o tratar el primer elemento aparte. Ojo con el ESTRUCTO: se cuenta solo si $x > maxi$. Si el enunciado considerara record a un empate habria que usar $\geq$, y es el tipo de detalle que cambia la respuesta sin que el código se vea distinto. Comprobado a mano: con 1 3 2 5 4 los records son 1, 3 y 5, o sea 3. El mismo barrido sirve para "cuantas veces cambia el maximo", "desde cuantas posiciones se ve el horizonte" y los problemas de edificios que tapan a otros.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4
5 int main() {
6     ios::sync_with_stdio(false);
7     cin.tie(nullptr);
8     ll n;
9     cin >> n;
10    ll maxi = -1;
11    ll res = 0;
12    while(n--)
13    {
14        ll x; cin >> x;
15        if(x>maxi)
16        {
17            res++;
18            maxi = x;
19        }
20    }

```

```

21     }
22
23     cout << res << endl;
24
25     return 0;
26 }

```

13.15 Cycle Detection - Switches

C++ Training Camp 2020 Day 8 - Switches.py

Resuelve "Switches" (Day 8, problema 1 - Contest 08 [Avanzados]): hay N interruptores que alternan (encienden/apagan) subconjuntos de M focos; el conserje los presiona en el orden fijo $1, 2, \dots, N, 1, 2, \dots$ y se detiene apenas todos los focos quedan apagados a la vez; dado el estado inicial de los focos y los focos asociados a cada interruptor, hay que contar cuantas veces presiona interruptores hasta lograrlo, o -1 si con esa estrategia nunca se apagan todos a la vez. Simula presionando los interruptores en orden cíclico, y guarda en el set "pos" cada estado visto como (configuración de focos + índice del interruptor); si ese mismo par (estado, interruptor) se repite antes de apagar todo, esta en un ciclo sin solución y responde -1 .

```

1  n,m = list(map(int,input().split()))
2  bomb = ["0"]*m
3  k = list(map(int,input().split()))
4  for i in range(1,len(k)):
5      bomb[k[i]-1]="1"
6
7  q = bomb.count("0")
8  c = 0
9  if(q == m):
10     print(0)
11 else:
12     entrada = []
13     pos = set()
14     c = 0
15     for j in range(n):
16         #print(bomb)
17         w = input().split()
18         s = list(map(int,w))
19         entrada.append(s)
20         for i in range(1,len(s)):
21             if(bomb[s[i]-1] == "0"):
22                 bomb[s[i]-1] = "1"
23             else:
24                 bomb[s[i]-1] = "0"
25             q = bomb.count("0")
26             c += 1
27             if(q == m):
28                 break
29             pos.add(str("".join(bomb))+str(j))
30
31 if(q == m):
32     print(c)
33 else:
34     i = 0
35     while True:
36         #print(bomb)
37         s = entrada[i]
38         for j in range(1,len(s)):
39             if(bomb[s[j]-1] == "0"):
40                 bomb[s[j]-1] = "1"
41             else:
42                 bomb[s[j]-1] = "0"
43             q = bomb.count("0")
44             c += 1
45             if(q == m):
46                 print(c)
47                 break
48             inte = str("".join(bomb))+str(i)
49             if(inte in pos):
50                 print(-1)
51                 break
52             pos.add(str("".join(bomb))+str(i))
53
54             i+=1
55             if(i >= len(entrada)):
56                 i = 0

```

13.16 Date Parsing - Crochet

C++ VRC (Crochet).cpp

Calcula la diferencia de tiempo (hacia adelante) entre dos momentos dados como día de la semana + hora. Usa un arreglo de 14 días (2 semanas) para manejar el wrap-around cuando el segundo momento cae en la siguiente semana.

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  using int64 = long long;
5
6  int main() {
7      string d1, d2;
8      string h1, h2;
9      cin >> d1 >> h1 >> d2 >> h2;
10
11     string dias[] = {"Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun",
12                     "Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"};
13
14     // Parsear horas HH:MM
15     int hh1, mm1, hh2, mm2;
16     sscanf(h1.c_str(), "%d:%d", &hh1, &mm1);
17     sscanf(h2.c_str(), "%d:%d", &hh2, &mm2);
18
19     int t1 = hh1 * 60 + mm1;
20     int t2 = hh2 * 60 + mm2;
21
22     // Encontrar el índice de d1 en la primera semana
23     int idx1 = 0;
24     for (int i = 0; i < 7; i++) {
25         if (dias[i] == d1) { idx1 = i; break; }
26     }
27
28     // Buscar d2 desde idx1 en adelante (en el arreglo de 14 días)
29     // Si es el mismo día y la hora de fin es menor o igual, avanzar a la siguiente semana
30     int idx2 = -1;
31     for (int i = idx1; i < 14; i++) {
32         if (dias[i] == d2) {
33             if (i == idx1 && t2 <= t1) continue;
34             idx2 = i;
35             break;
36         }
37     }
38
39     int64 diff = (int64)(idx2 - idx1) * 24 * 60 + (t2 - t1);
40
41     int64 days = diff / (24 * 60);
42     int64 hours = (diff % (24 * 60)) / 60;
43     int64 mins = diff % 60;
44
45     vector<string> parts;
46     if (days) parts.push_back(to_string(days) + (days == 1 ? " day" : " days"));
47     if (hours) parts.push_back(to_string(hours) + (hours == 1 ? " hour" : " hours"));
48     if (mins) parts.push_back(to_string(mins) + (mins == 1 ? " minute" : " minutes"));
49
50     if (parts.size() == 3)
51         cout << parts[0] << ", " << parts[1] << ", " << parts[2] << "\n";
52     else if (parts.size() == 2)
53         cout << parts[0] << " and " << parts[1] << "\n";
54     else
55         cout << parts[0] << "\n";
56
57     return 0;
58 }

```

13.17 Fuerza Bruta $O(n^2)$ sobre Subperiodos - Accounting Qualifying

C++ ICPC 2025 A - Accounting Qualifying.cpp

La otra forma de resolver "Account Qualifying" (problema A, ICPC 2025): probar TODOS los subperiodos. Se mapea deposito a +1, retiro a -1 y saldo a 0, y cada tramo con suma 0 tiene tantos depositos como retiros. Es $O(n^2)$, que con $n = 10000$ son 10^8 sumas. Medido: 5 casos del tamaño máximo en 94 ms, o sea que pasa, pero la versión con sumas de prefijo (el otro archivo A de esta carpeta) hace lo mismo en $O(n)$ y tarda 20 ms. POR QUE NO TIENE PODAS: la primera versión de este archivo intentaba cortar el ciclo interno contando cuantos positivos y negativos quedaban disponibles, y estaba mal: el break se evaluaba ANTES de comprobar si la suma era 0, así que descartaba tramos válidos. Con el primer caso del sample daba $r = 3$ en vez de 4. Una fuerza bruta sin podas es corta y no se equivoca; si hace falta velocidad, lo que corresponde es cambiar de algoritmo, no parchear el ciclo. Lo de d y w es igual que en la otra versión: si no hay depositos d es 0 y si no hay retiros w es 0, de ahí el $\max(0, \dots)$ y el $\min(0, \dots)$. Verificado contra fuerza bruta en 2900 casos, sin fallos.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int n;
9
10    while (cin >> n && n != 0) {
11
12        vector<int> a(n);
13
14        for (int &x : a)
15            cin >> x;
16
17        // Si no hay depositos d es 0, y si no hay retiros w es 0: por eso se
18        // acotan con 0 en vez de tomar el máximo y el mínimo a secas.
19        int d = max(0, *max_element(a.begin(), a.end()));
20        int w = min(0, *min_element(a.begin(), a.end()));
21
22        // +1 = deposito
23        // 0 = balance
24        // -1 = retiro
25        for (int &x : a) {
26            if (x > 0) x = 1;
27            else if (x < 0) x = -1;
28        }
29
30        int r = 0;
31
32        // Todos los subperiodos, sin podas: si la suma de signos es 0, hay
33        // tantos depositos como retiros.
34        for (int i = 0; i < n; i++) {
35
36            int sum = 0;
37
38            for (int j = i; j < n; j++) {
39
40                sum += a[j];
41
42                if (sum == 0)
43                    r = max(r, j - i + 1);
44            }
45        }
46
47        cout << d << ' ' << w << ' ' << r << '\n';
48    }
49
50    return 0;
51 }
```

13.18 Generacion de Numeros con Digos Distintos - Division Digos Distintos

C++ ICPC 2025 A - Division Digos Distintos.cpp

Para cada N del caso de prueba, busca todos los pares $(s1, s2)$ con $s1 / s2 = N$ donde $s1$ y $s2$ tienen, cada uno por su lado, TODOS sus dígitos distintos, y los imprime como " $s1 / s2 = N$ ". La idea es precalcular UNA SOLA VEZ la lista de todos los números con dígitos distintos, y después cada consulta solo recorre esa lista probando $s1 = s2 * N$. CUANTOS SON: el primer dígito tiene 9 opciones (1-9, el 0 no abre), el segundo vuelve a tener 9 (los diez menos el ya usado, y ahí el 0 si vale), el tercero 8, y así bajando. Sumando los largos de 1 a 10 dígitos: $9 + 81 + 648 + 4536 + 27216 + 136080 + 544320 + 1632960 + 3265920 + 3265920 = 8877690$. Los de 9 y 10 dígitos son la misma cantidad porque al decimo ya solo queda una opción, y entre esos dos largos está el 73% del total. Son ~68 MB de vector, así que el reserve no es adorno: sin él, el vector duplica capacidad hasta 128 MB y puede costar el límite de memoria. LA GENERACION es backtracking puro con el esqueleto de siempre: marcar el dígito, recursar, desmarcar. Se usa un bool usado[10] en vez de una máscara de bits a propósito, porque usado[d] se lee de una y $\text{mask} \& (1 \ll d)$ hay que decodificarlo. El desmarcado explícito es lo que deja ver que la rama del 2 puede volver a usar el 1. No hace falta comparar contra MAX_NUM al generar: el mayor número con 10 dígitos distintos ES 9876543210, así que la recursión nunca se pasa sola. EL CORTE DE LA CONSULTA: como la lista está ordenada, apenas $s2 > \text{MAX_NUM} / N$ el producto ya se sale y todos los siguientes también, así que se rompe el ciclo. Con N grande eso deja el recorrido en nada; con $N = 2$ todavía visita 7 millones, que es el peor caso. OJO CON DOS COSAS QUE CUESTAN TIEMPO Y NO SE VEN: - Imprimir con endl hace flush en CADA línea. Con $N = 2$ salen 680425 líneas y ahí se va la mayor parte del tiempo. Con '\n' la salida es idéntica y el caso completo baja de 2084 ms a 1390 ms (medido). - Si a la función de generar se le pone un tercer parámetro y se llama generate(0, 0, 0), choca con std::generate de <algorithm> por el using namespace std y NO compila. Por eso aquí se llama armar. Es el mismo problema que tiene llamar remove a una función propia. Verificado: la salida es idéntica, línea por línea, a la versión original que pasó el juez, sobre 709320 líneas incluyendo el peor caso $N = 2$.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 const ll MAX_NUM = 9876543210LL;
7
8 vector<ll> nums;
9
10 // usado[d] = true si el dígito d ya está puesto en el número que armamos.
11 bool usado[10];
12
13 // Arma números dígito por dígito, probando todas las opciones.
14 // numero = lo que llevamos armado hasta ahora
15 // cuantos = cuantos dígitos tiene ese número
16 // Cada llamada decide CUAL es el siguiente dígito que se pega al final.
17 void armar(ll numero, int cuantos) {
18     // Si ya pusimos por lo menos un dígito, lo que llevamos YA es un número
19     // válido (todos sus dígitos son distintos), así que se guarda.
20     if (cuantos > 0) {
21         nums.push_back(numero);
22     }
23
24     // Con 10 dígitos distintos ya se usaron todos, no hay más que pegar.
25     if (cuantos == 10) {
26         return;
27     }
28
29     // Se prueba cada dígito que todavía no hayamos usado.
30     for (int d = 0; d <= 9; d++) {
31         if (usado[d]) {
32             continue;
33         }
34
35         // El 0 no puede ir de primero (021 no es un número de 3 dígitos).
36         if (cuantos == 0 && d == 0) {
37             continue;
38         }
39
40         usado[d] = true; // se marca: ya lo usamos
41
42         armar(numero * 10 + d, cuantos + 1); // se pega al final y se sigue
43
44         usado[d] = false; // se libera para la otra rama
45     }
46 }
47
48 // Revisa si x tiene todos sus dígitos distintos, sacándolos uno por uno.
```

```

49 bool distinctDigits(ll x) {
50     int mask = 0;
51
52     while (x > 0) {
53         int d = x % 10;
54         x /= 10;
55
56         if (mask & (1 << d))
57             return false;
58
59         mask |= (1 << d);
60     }
61
62     return true;
63 }
64
65 int main() {
66     ios::sync_with_stdio(false);
67     cin.tie(nullptr);
68
69     nums.reserve(9000000);    // el total es 8877690, sin realocar
70
71     armar(0, 0);
72
73     sort(nums.begin(), nums.end());
74
75     int T;
76     cin >> T;
77
78     while (T--) {
79         ll N;
80         cin >> N;
81
82         for (ll s2 : nums) {
83             // Si s2 * N se pasa del maximo, los siguientes tambien: corta.
84             if (s2 > MAX_NUM / N)
85                 break;
86
87             ll s1 = s2 * N;
88
89             if (!distinctDigits(s1))
90                 continue;
91
92             cout << s1 << " / " << s2 << " = " << N << '\n';
93         }
94
95         if (T)
96             cout << '\n';
97     }
98
99     return 0;
100 }

```

13.19 Greedy - Escalator

C++\Training_Camp_2020\Day 4\C - Escalator.cpp

Resuelve "Escalator" (Day 4, problema C - Contest 04 [Avanzados]): una escalera doble tarda 10 segundos en cruzar a cualquier persona, arranca en la dirección del primero que llega estando detenida, y si alguien llega en dirección contraria a la que se mueve debe esperar a que pare; hay que hallar el momento en que la escalera se detiene por última vez. Simula el estado de la escalera con variables `busy_until` (cuando quedaria libre), `dir` (dirección actual) y `waiting` (si hay alguien esperando un cambio de sentido), resolviendo en $O(N)$ los cambios de dirección pendientes antes de procesar la llegada de cada persona.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main(){
5     int n;
6     cin >> n;
7
8     long long busy_until = 0;
9     int dir = -1;
10    bool waiting = false;
11
12    for (int i = 0; i < n; i++){

```

```

8        long long t; int d;
9        cin >> t >> d;
10
11        while (waiting && t >= busy_until){
12            dir = 1 - dir;
13            busy_until = busy_until + 10;
14            waiting = false;
15        }
16
17        if (busy_until <= t){
18            dir = d;
19            busy_until = t + 10;
20        } else {
21            if (d == dir){
22                busy_until = max(busy_until, t + 10);
23            } else {
24                waiting = true;
25            }
26        }
27    }
28
29    if (waiting){
30        busy_until = busy_until + 10;
31    }
32
33    cout << busy_until << "\n";
34    return 0;
35 }

```

13.20 Lectura de Entrada

C++\Notebook\Lectura de Entrada.cpp

Catálogo de patrones para leer entradas "raras": hasta EOF, con centinela, con T casos, bloques separados por líneas en blanco, grids de caracteres, tokens con separadores mixtos y enteros gigantes como `string`. La regla práctica: si el problema NO da un T inicial, piensa en EOF o centinela; si la entrada trae blancos o bloques, usa `getline` mas `istringstream` en vez de `cin >>`; y si alternas `cin >>` con `getline`, consume siempre el salto de línea pendiente con un `getline` extra o el parseo se descuadra. Cada patron esta en su propia funcion para poder copiar solo el que se necesita.

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 // 1) Hasta EOF: no hay T y el archivo termina sin aviso
6 void leerHastaEOF()
7 {
8     int a, b;
9     while (cin >> a >> b)
10     {
11         // procesar (a, b)
12     }
13 }
14
15 // 2) Hasta centinela (clasico "0 0" para terminar)
16 void leerHastaCentinela()
17 {
18     int a, b;
19     while (cin >> a >> b)
20     {
21         if (a == 0 && b == 0) break;
22         // procesar (a, b)
23     }
24 }
25
26 // 3) T al inicio y luego T casos
27 void leerTCasos()
28 {
29     int T;
30     if (cin >> T)
31     {
32         while (T--)
33         {
34             int n, m;
35             cin >> n >> m;
36             // procesar

```

```

37     }
38 }
39 }
40
41 // 4) Cabecera por caso: cada caso empieza con "n m" y trae m lineas
42 void leerCabeceraPorCaso()
43 {
44     int n, m;
45     while (cin >> n >> m)
46     {
47         vector<pair<int,int>> edges;
48         edges.reserve(m);
49         for (int i = 0; i < m; i++)
50         {
51             int u, v;
52             cin >> u >> v;
53             edges.push_back({u, v});
54         }
55         // procesar caso
56     }
57 }
58
59 // 5) Bloques separados por lineas en blanco, sin saber cuantos hay
60 void leerBloques()
61 {
62     string line;
63     while (true)
64     {
65         // saltar lineas vacias entre bloques
66         while (getline(cin, line) && line.empty()) {}
67         if (!cin) break;
68
69         vector<string> bloque;
70         do
71         {
72             if (!line.empty()) bloque.push_back(line);
73         } while (getline(cin, line) && !line.empty());
74
75         // parsear cada linea del bloque con istringstream
76         for (auto &L : bloque)
77         {
78             istringstream iss(L);
79             int x;
80             while (iss >> x)
81             {
82                 // procesar x
83             }
84         }
85     }
86 }
87
88 // 6) Mezclar cin >> con getline: hay que consumir el '\n' colgante
89 void leerLineaConEspacios()
90 {
91     int n;
92     cin >> n;
93     string line;
94     getline(cin, line); // consume el salto de linea pendiente
95     getline(cin, line); // esta si es la linea util (puede traer espacios)
96 }
97
98 // 7) Leer N numeros aunque vengan repartidos en varias lineas
99 void leerNNumeros()
100 {
101     int n;
102     cin >> n;
103     vector<long long> a;
104     a.reserve(n);
105     while ((int)a.size() < n)
106     {
107         long long x;
108         if (!(cin >> x)) break;
109         a.push_back(x);

```

```

110     }
111 }
112
113 // 8) Grid de caracteres R x C (getline es mas robusto que >> por fila)
114 void leerGrid()
115 {
116     int R, C;
117     cin >> R >> C;
118     string dummy;
119     getline(cin, dummy); // consume el '\n'
120     vector<string> g(R);
121     for (int i = 0; i < R; i++)
122     {
123         getline(cin, g[i]);
124     }
125 }
126
127 // 9) Leer linea por linea y tokenizar, saltando vacias y comentarios
128 void leerYTokenizar()
129 {
130     string line;
131     while (getline(cin, line))
132     {
133         if (line.empty() || line[0] == '#') continue;
134         istringstream iss(line);
135         int x;
136         while (iss >> x)
137         {
138             // procesar x
139         }
140     }
141 }
142
143 // 10) Tokens con separadores mixtos (comas, tabs, espacios)
144 void leerSeparadoresMixtos()
145 {
146     string line;
147     getline(cin, line);
148     for (char &c : line)
149     {
150         if (c == ',' || c == '\t') c = ' ';
151     }
152     istringstream iss(line);
153     vector<string> tok;
154     for (string t; iss >> t; ) tok.push_back(t);
155 }
156
157 // 11) Enteros gigantes: leerlos como string para no desbordar
158 void leerBigInt()
159 {
160     string P;
161     while (cin >> P)
162     {
163         // procesar P digito por digito (ver modString)
164     }
165 }
166
167 // 12) Consumir TODO el archivo como un solo string (parser propio)
168 void leerTodo()
169 {
170     string all((istreambuf_iterator<char>(cin), istreambuf_iterator<char>()));
171 }
172
173 // 13) Normalizar fin de linea de Windows (CRLF) y espacios finales
174 void rtrim(string &s)
175 {
176     while (!s.empty() && (s.back() == '\r' || s.back() == ' ' || s.back() == '\t'))
177     {
178         s.pop_back();
179     }
180 }
181
182 int main()

```

```

83 {
84     ios_base::sync_with_stdio(false);
85     cin.tie(nullptr);
86     return 0;
87 }

```

13.21 Lights Out (Primera Fila Forzada) - Turnswitch

C++\ICPC\2024F - Turnswitch.cpp

Resuelve "Turnswitch" (problema F, ICPC 2024). Una grilla $n \times n$ de interruptores que est n en '|' o en '-'; girar uno lo cambia a el y a sus cuatro vecinos. El c digo busca el m nimo de giros para dejar todo igual, probando las dos opciones (todo '|' o todo '-'). Es un "Lights Out", y tiene dos datos que lo desarmen. Primero, el orden de los giros no importa y girar dos veces el mismo es no girarlo, as  que cada interruptor se gira 0 o 1 vez. Segundo, y es la clave: SI SE DECIDE LA PRIMERA FILA, TODO LO DEM S QUEDA FORZADO. Una vez resuelta la fila $r-1$, la  nica forma de arreglar la casilla $(r-1, c)$ que siga mal es girar (r, c) , porque es el  ltimo interruptor que la toca. Entonces la fila siguiente no se elige: se deduce. As  la b squeda baja de $2^{(n \times n)}$ a 2^n combinaciones de la primera fila, cada una completada en $O(n^2)$, y al final solo hay que ver si la  ltima fila qued  bien. CUANDO USAR: cualquier puzzle de "presionar cambia a los vecinos" en una grilla. Si n es grande para 2^n , el mismo sistema se resuelve con eliminaci n gaussiana sobre $GF(2)$.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int n;
9
10    while (cin >> n && n != 0) {
11
12        vector<string> a(n);
13
14        for (auto &row : a)
15            cin >> row;
16
17        int ans = 1e9;
18
19        // Probamos los dos objetivos:
20        // 1. Todas verticales '|'
21        // 2. Todas horizontales '-'
22        for (int target = 0; target < 2; target++) {
23
24            char wanted = (target == 0 ? '|' : '-');
25
26            // Probamos todas las formas de girar la primera fila
27            for (int mask = 0; mask < (1 << n); mask++) {
28
29                vector<string> b = a;
30                int cnt = 0;
31
32                // Funci n para girar una casilla
33                auto turn = [&](int r, int c) {
34                    cnt++;
35
36                    // Propia
37                    b[r][c] = (b[r][c] == '|') ? '-' : '|';
38
39                    // Arriba
40                    if (r > 0)
41                        b[r - 1][c] =
42                            (b[r - 1][c] == '|') ? '-' : '|';
43
44                    // Abajo
45                    if (r + 1 < n)
46                        b[r + 1][c] =
47                            (b[r + 1][c] == '|') ? '-' : '|';
48
49                    // Izquierda
50                    if (c > 0)
51                        b[r][c - 1] =
52                            (b[r][c - 1] == '|') ? '-' : '|';
53
54                    // Derecha

```

```

55                    if (c + 1 < n)
56                        b[r][c + 1] =
57                            (b[r][c + 1] == '|') ? '-' : '|';
58                };
59
60                // Decidimos que switches girar en la primera fila
61                for (int c = 0; c < n; c++) {
62                    if (mask & (1 << c))
63                        turn(0, c);
64                }
65
66                // Desde la segunda fila, los movimientos son obligatorios
67                for (int r = 1; r < n; r++) {
68                    for (int c = 0; c < n; c++) {
69
70                        // Si la casilla de arriba esta mal,
71                        // debemos girar esta.
72                        if (b[r - 1][c] != wanted)
73                            turn(r, c);
74                    }
75                }
76
77                // Comprobamos si la  ltima fila qued  correcta
78                bool ok = true;
79
80                for (int c = 0; c < n; c++) {
81                    if (b[n - 1][c] != wanted) {
82                        ok = false;
83                        break;
84                    }
85                }
86
87                if (ok)
88                    ans = min(ans, cnt);
89            }
90        }
91
92        cout << ans << '\n';
93    }
94
95    return 0;
96 }

```

13.22 Simulación con Parsing de Código - Impossible Primebox

Chen HPC 2025V - Impossible Primebox.cpp

Resuelve "Impossible Primebox" (problema I, ICPC 2025): hay L "primelocks", cada uno con la forma $prlc\ i: x = x [* Ai] [+ Bi]$ if $Pi \text{ div } x \text{ jumpto } \{prlc\ ji \mid end\}$ else $jumpto \{prlc\ mi \mid end\}$ Se arranca en el candado 0 con un x inicial y se sigue saltando hasta caer en un end. Hay que hallar el MENOR x inicial con el que el candado i se ejecuta exactamente Ki veces, para todos. El enunciado promete que la clave existe y es menor que 50000, así que la solución es probar $x = 0, 1, 2, \dots$ y quedarse con el primero que cumpla: no hay que invertir nada. LO QUE DE VERDAD CUESTA ES LEER LA ENTRADA, porque el formato tiene partes OPCIONALES: el $** Ai$ puede no estar, el $+ Bi$ puede no estar, y los destinos son "prlc j" o "end", que ocupan distinta cantidad de palabras. Leer con $>>$ palabra por palabra y decidir según lo que aparece es mucho más seguro que intentar un patrón fijo con getline. línea 2: $x = x$ y después, mientras siga habiendo operador, se lee $** A$ o $+ B$ línea 3: "if P div x jumpto <destino> else jumpto <destino>", donde <destino> es "end" (una palabra) o "prlc" seguido del número (dos palabras) Para saber donde termina la línea 2 sin depender de saltos de línea, se mira si la siguiente palabra es "if": ahí empieza la línea 3. "Pi div x" significa "Pi divide a x", o sea $x \% Pi == 0$. Leerlo al revés es el error fácil. EL CORTE POR CICLO INFINITO ES OBLIGATORIO: con un x malo el proceso puede no terminar nunca. Como cada Ki es a lo sumo 1000 y hay a lo sumo 10 candados, un recorrido válido no puede dar más de 10000 pasos; si se pasa de ahí, ese x no sirve y se corta. Además, apenas un contador supera su Ki ya se puede abandonar, y eso es lo que hace que la mayoría de los x se descarten en pocos pasos. SOBRE EL TIEMPO, PARA QUE NO SORPRENDA: el peor caso teórico es 50000 candidatos por 10000 pasos. Medido con 5 casos armados a propósito para que ninguna x sirva y el recorrido siempre llegue a los 10000 pasos: 15.5 s en total, unos 3 s por caso. El enunciado promete que la clave EXISTE y es menor que 50000, así que en cualquier entrada legítima se encuentra antes y no se paga ese costo; el sample completo corre en 12 ms. Si aun así diera TLE, lo siguiente sería detectar ciclos guardando los estados (candado, residuos) ya vistos. EL VALOR DE x NO SE PUEDE GUARDAR: con A hasta 1000 por paso y hasta 10000 pasos, x llega a tener miles de dígitos. Pero de x solo se pregunta si P lo divide, así que se lleva x MODULO el primo de cada candado. La transformación $x = x*A + B$ se traduce a los residuos sin cambiar nada, porque $(x*A + B) \bmod P$ depende solo de $x \bmod P$. Con $L \leq 10$ son 10 residuos por paso. Verificado con los dos casos del sample (10 y 255) y con una fuerza bruta independiente escrita aparte que simula lo mismo: 600 primeboxes aleatorios, sin diferencias.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 typedef long long ll;
5
6 struct Lock {
7     ll A, B, P;
8     int siTrue, siFalse;    // -1 significa "end"
9 };
10
11 int main() {
12     ios::sync_with_stdio(false);
13     cin.tie(nullptr);
14
15     int L;
16
17     while (cin >> L && L != 0) {
18
19         vector<Lock> lk(L);
20
21         for (int i = 0; i < L; i++) {
22
23             string s;
24
25             // "prlc i:" (dos palabras)
26             cin >> s >> s;
27
28             // "x = x" y después los opcionales
29             cin >> s >> s >> s;
30
31             lk[i].A = 1;
32             lk[i].B = 0;
33
34             // Se leen operadores hasta toparse con el "if" de la línea 3.
35             while (cin >> s && s != "if") {
36                 ll v;
37                 cin >> v;
38
39                 if (s == "**") lk[i].A = v;
40                 else lk[i].B = v;
41             }
42
43             // "P div x jumpto <destino> else jumpto <destino>"
44             cin >> lk[i].P;
```

```
45         cin >> s >> s >> s;    // div x jumpto
46
47         cin >> s;
48         if (s == "end") {
49             lk[i].siTrue = -1;
50         } else {
51             cin >> lk[i].siTrue;    // venia "prlc j"
52         }
53
54         cin >> s >> s;    // else jumpto
55
56         cin >> s;
57         if (s == "end") {
58             lk[i].siFalse = -1;
59         } else {
60             cin >> lk[i].siFalse;
61         }
62     }
63
64     vector<int> K(L);
65     for (int i = 0; i < L; i++) cin >> K[i];
66
67     int pasosMax = 0;
68     for (int i = 0; i < L; i++) pasosMax += K[i];
69
70     // Con los primos DISTINTOS (a lo sumo 10 y cada uno <= 1000) se arma
71     // M = su producto. Si M cabe con margen para multiplicar por A, basta
72     // llevar UN residuo x mod M: como cada P divide a M, preguntar por
73     // r % P sigue valiendo, y el paso cuesta una multiplicación en vez de
74     // una por candado. Si M no cabe, se cae al vector de residuos.
75     vector<ll> primos;
76     for (int i = 0; i < L; i++) primos.push_back(lk[i].P);
77     sort(primos.begin(), primos.end());
78     primos.erase(unique(primos.begin(), primos.end()), primos.end());
79
80     const ll LIMITE = (ll)9e18 / 1001;    // deja espacio para r*A + B
81     ll M = 1;
82     bool unico = true;
83
84     for (ll p : primos) {
85         if (M > LIMITE / p) { unico = false; break; }
86         M *= p;
87     }
88
89     ll clave = -1;
90
91     vector<int> veces(L, r(L));
92
93     for (ll x0 = 0; x0 < 50000 && clave < 0; x0++) {
94
95         // x se dispara (se multiplica por A hasta 1000 en cada paso) y no
96         // cabe en ningún entero. Pero de x solo se pregunta si es divisible
97         // por P, así que basta llevar x modulo esos primos:
98         // (x*A + B) mod P se calcula con (x mod P), sin conocer x.
99         for (int i = 0; i < L; i++) veces[i] = 0;
100
101         ll ru = x0 % M;
102
103         if (!unico)
104             for (int i = 0; i < L; i++) r[i] = (int)(x0 % lk[i].P);
105
106         int cur = 0;
107         int pasos = 0;
108         bool sirve = true;
109
110         while (cur != -1) {
111
112             if (++pasos > pasosMax + 1) {    // ciclo o demasiadas vueltas
113                 sirve = false;
114                 break;
115             }
116
117             veces[cur]++;
```

```
18
19         if (veces[cur] > K[cur]) {           // ya se paso de su cuota
20             sirve = false;
21             break;
22         }
23
24         bool divide;
25
26         if (unico) {
27             ru = (ru * lk[cur].A + lk[cur].B) % M;
28             divide = (ru % lk[cur].P == 0);
29         } else {
30             for (int i = 0; i < L; i++)
31                 r[i] = (int)((r[i] * lk[cur].A + lk[cur].B) % lk[i].P);
32             divide = (r[cur] == 0);
33         }
34
35         if (divide) cur = lk[cur].siTrue;
36         else cur = lk[cur].siFalse;
37     }
38
39     if (sirve && veces == K) clave = x0;
40 }
41
42 cout << clave << '\n';
43 }
44
45 return 0;
46 }
```

```
8
8 int main()
20 {
21     ios_base::sync_with_stdio(false);
22     cin.tie(nullptr);
23     cout.tie(nullptr);
24
25     int filas = 3, columnas = 4;
26     vector<vector<int>> matriz(filas, vector<int>(columnas, 0));
27
28     vector<int> v = {5, 1, 9, 3, 7};
29
30     // indice del maximo y del minimo (no el valor)
31     int idx_max = max_element(all(v)) - v.begin();
32     int idx_min = min_element(all(v)) - v.begin();
33     cout << idx_max << " " << idx_min << endl;
34
35     // borrar desde el indice 1 hasta el 3 (sin incluir el 3)
36     v.erase(v.begin() + 1, v.begin() + 3);
37
38     // imprimir con decimales fijos
39     double value = 1.0 / 3.0;
40     cout << fixed << setprecision(6) << value << endl;
41
42     // gcd ya viene en la STL (__gcd, o std::gcd/std::lcm si el juez tiene C++17).
43     // Para el lcm, dividir ANTES de multiplicar evita desbordar.
44     int a = 4, b = 6;
45     cout << __gcd(12, 18) << " " << (a / __gcd(a, b)) * b << endl;
46
47     return 0;
48 }
```

14 - Formulario

14.1 Algebra y Ecuaciones

Resolver e igualar ecuaciones: cuadratica con discriminante, Vieta para sacar suma y producto de raices sin calcularlas, sistemas 2x2 por Cramer, productos notables, leyes de exponentes y logaritmos, y proporciones y promedios. Incluye la version numericamente estable de la cuadratica, que importa cuando b^2 domina a $4ac$ y la resta directa pierde todos los digitos. Vieta y Cramer estan verificados.

Ecuacion cuadratica $ax^2 + bx + c = 0$

$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$
discriminante $D = b^2 - 4ac$
 $D > 0$: dos raices reales distintas
 $D = 0$: una raiz doble, $x = -b/(2a)$
 $D < 0$: ninguna raiz real
Vieta (verificado) : $x_1 + x_2 = -b/a$ $x_1 \cdot x_2 = c/a$
Sirve para responder sobre las raices sin calcularlas.
Version estable, cuando b^2 es mucho mayor que $4ac$:
 $q = -(b + \text{sign}(b) \cdot \sqrt{D}) / 2$
 $x_1 = q / a$ $x_2 = c / q$
Si $a = 0$ no es cuadratica: es $bx + c = 0$, y si $b = 0$ tambien hay que tratarlo aparte. Ese caso borde tumba muchas soluciones.

Sistema de dos ecuaciones (Cramer, verificado)

$a_1x + b_1y = c_1$
 $a_2x + b_2y = c_2$
 $\det = a_1b_2 - a_2b_1$
 $\det \neq 0$: solucion unica
 $\det == 0$: o no hay solucion, o hay infinitas (misma recta)
 $x = (c_1b_2 - c_2b_1) / \det$ $y = (a_1c_2 - a_2c_1) / \det$
Igualar dos funciones $y = f(x)$, $y = g(x)$ es resolver $f(x)-g(x)=0$.

Productos notables

$(a + b)^2 = a^2 + 2ab + b^2$
 $(a - b)^2 = a^2 - 2ab + b^2$
 $(a + b)(a - b) = a^2 - b^2$
 $(a + b)^3 = a^3 + 3a^2b + 3ab^2 + b^3$
 $a^3 + b^3 = (a + b)(a^2 - ab + b^2)$

13.23 Simulation - Enigma Of The Jewelry Case

C++\Training_Camp_2026\Day 3\E - Enigma of the Jewelry Case.py
Resuelve "Enigma of the Jewelry Case" (Day 3, problema E - Maratona SBC de Programacao 2024): la princesa de Nlogonia tenia una caja $N \times N$ donde los valores crecen estrictamente por fila y por columna (de izquierda a derecha y de arriba a abajo); su hermana roto la caja 90 grados en sentido horario un numero desconocido de veces (0 a 3), y hay que hallar el minimo numero de rotaciones antihorarias necesarias para devolverla a su estado original. Como en el arreglo original el valor minimo siempre esta en la esquina superior izquierda, el codigo solo necesita mirar las 4 esquinas de la matriz recibida (pos), encontrar cual de ellas tiene el valor minimo ($me = \min(pos)$) y su indice dentro de esa lista indica directamente, por la simetria de la rotacion, cuantas rotaciones antihorarias hacen falta.

```
1 n = int(input())
2 matrix = []
3 for i in range(n):
4     matrix.append(list(map(int,input().split())))
5 pos = [matrix[0][0], matrix[0][n-1], matrix[n-1][n-1], matrix[n-1][0]]
6 me = min(pos)
7 print(pos.index(me))
```

13.24 Template Base

C++\Notebook\Template Base.cpp
Plantilla de arranque para cualquier problema: includes, alias de tipos, constantes (MOD, INF, LLINF, EPS, PI), macros de recorrido (all, for, forr, ford) y la desincronizacion de cin/cout que hace la lectura casi tan rapida como scanf. Incluye ademas los utilitarios que mas se olvidan en competencia: crear una matriz, imprimir con decimales fijos, borrar un rango de un vector, y obtener el indice (no el valor) del maximo o minimo. Ojo: __gcd / std::gcd y std::lcm ya vienen en la STL, no hace falta escribirlos a mano.

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 using ll = long long;
6
7 const int MOD = 1e9 + 7;
8 const ll LLINF = 0x3f3f3f3f3f3f3f3fLL;
9 const int INF = 0x3f3f3f3f;
10 const double EPS = 1e-9;
11 const double PI = acos(-1.0);
12
13 #define all(x) (x).begin(), (x).end()
14 #define forn(i, n) for(int i = 0; i < int(n); i++)
15 #define forr(i, a, b) for(int i = a; i <= b; i++)
16 #define ford(i, n) for(int i = int(n) - 1; i >= 0; i--)
17 #define endl '\n'
```

$a^3 - b^3 = (a - b)(a^2 + ab + b^2)$
 $a^n - b^n$ siempre es divisible por $(a - b)$
 binomio de Newton : $(a+b)^n = \sum_k C(n,k) a^{n-k} b^k$

Exponentes y logaritmos

$a^m * a^n = a^{(m+n)}$ $a^m / a^n = a^{(m-n)}$
 $(a^m)^n = a^{(m*n)}$ $(ab)^n = a^n b^n$
 $a^0 = 1$ $a^{(-n)} = 1 / a^n$
 $a^{(1/n)} = \text{raiz } n\text{-esima de } a$
 $\log(x*y) = \log x + \log y$ $\log(x/y) = \log x - \log y$
 $\log(x^n) = n * \log x$ $\log_b(x) = \log(x) / \log(b)$
 En C++ $\log()$ es natural; existen $\log2()$ y $\log10()$ aparte.

OJO: $\log2(n)$ puede dar 2.9999 por flotante. Si necesitas el exponente exacto, usa $31 - _builtin_clz(n)$ o un bucle.

Proporciones y porcentajes

regla de tres: si a corresponde a b, entonces c da $b*c/a$
 aumentar p% : $x * (1 + p/100)$ disminuir : $x * (1 - p/100)$
 variacion porcentual = $(\text{nuevo} - \text{viejo}) / \text{viejo} * 100$

Subir p% y luego bajar p% NO devuelve al valor original.

promedio simple = suma / n
 promedio ponderado = $\text{sum}(w_i * x_i) / \text{sum}(w_i)$
 media geometrica = $(\text{producto de los } x_i)^{(1/n)}$

se usa para tasas de crecimiento encadenadas

media armonica = $n / \text{sum}(1/x_i)$

se usa para promediar velocidades sobre la misma distancia

Siempre: armonica <= geometrica <= aritmetica

Valor absoluto e inecuaciones

$|x| < a$ equivale a $-a < x < a$
 $|x| > a$ equivale a $x < -a$ o $x > a$
 $|a + b| \leq |a| + |b|$ desigualdad triangular

Al multiplicar o dividir una inecuacion por un NEGATIVO, el sentido se voltea. Es el error clasico al despejar.

En C++

Cuadratica numericamente estable. Devuelve cuantas raices reales hay.

```
int cuadratica(double a, double b, double c, double& x1, double& x2) {
    double D = b*b - 4*a*c;
    if (D < 0) return 0;
    double q = -(b + copysign(sqrt(D), b)) / 2;
    x1 = q / a;
    x2 = (fabs(q) > 1e-300) ? c / q : x1;
    if (x1 > x2) swap(x1, x2);
    return D > 0 ? 2 : 1;
}
```

El copysign evita restar dos numeros casi iguales. Con a=1, b=1e8, c=1 la formula directa da 0 en una raiz; esta da -1e-8.

Sistema 2x2 por Cramer. false = no hay solucion unica.

```
bool cramer(double a1, double b1, double c1,
             double a2, double b2, double c2, double& x, double& y) {
    double d = a1*b2 - a2*b1;
    if (fabs(d) < 1e-12) return false;
    x = (c1*b2 - c2*b1) / d;
    y = (a1*c2 - a2*c1) / d;
    return true;
}
```

14.2 Areas y Volúmenes

C++\Formulario\Areas y Volúmenes.cpp

Areas de figuras planas y volúmenes de cuerpos, incluidas las que uno cree recordar y termina equivocando: el rombo es $d1*d2/2$ (no lado por lado), el cono y la piramide llevan el tercio, y el perimetro de la elipse NO tiene formula cerrada elemental, solo la aproximacion de Ramanujan. Todas las formulas de aqui se comprobaron contra integracion numerica o contra el area por shoelace del poligono correspondiente.

Figuras planas (A = area, P = perimetro)

cuadrado $A = L^2$ $P = 4L$ diagonal = $L*\sqrt{2}$
 rectangulo $A = b*h$ $P = 2(b+h)$ diagonal = $\sqrt{b^2+h^2}$
 rombo $A = d1*d2 / 2$ $P = 4L$

las diagonales se cortan perpendicularmente en el centro

el lado sale de ellas: $L = \sqrt{(d1/2)^2 + (d2/2)^2}$
 tambien $A = L^2 * \sin(\text{angulo})$
 romboide $A = b*h$ $P = 2(a+b)$
 trapecio $A = (B + b) * h / 2$
 triangulo $A = b*h/2$ (Heron esta en la ficha Geometria)
 circulo $A = \pi r^2$ $P = 2 \pi r$
 corona circular $A = \pi (R^2 - r^2)$
 sector de angulo t en radianes
 $A = r^2 * t / 2$ arco = $r * t$
 elipse $A = \pi a b$
 El perimetro no tiene formula cerrada elemental.
 Ramanujan, APROXIMADO (error ~1e-5 con a=5, b=2, medido):
 $P \sim \pi (3(a+b) - \sqrt{(3a+b)(a+3b)})$
 poligono regular de n lados de largo L (verificado)
 $A = n * L^2 / (4 * \tan(\pi/n))$
 apotema = $L / (2 * \tan(\pi/n))$
 angulo interior = $(n-2)*180/n$ suma total = $(n-2)*180$
 angulo central = $360/n$

Cuerpos (V = volumen, S = area total)

cubo $V = a^3$ $S = 6a^2$
 diagonal = $a*\sqrt{3}$
 ortoedro $V = a*b*c$ $S = 2(ab + ac + bc)$
 diagonal = $\sqrt{a^2 + b^2 + c^2}$
 prisma $V = (\text{area base}) * h$
 cilindro $V = \pi r^2 h$ lateral = $2 \pi r h$
 $S = 2 \pi r (r + h)$
 cono $V = \pi r^2 h / 3$ lateral = $\pi r g$
 $g = \sqrt{r^2 + h^2}$ $S = \pi r (r + g)$
 esfera $V = 4/3 \pi r^3$ $S = 4 \pi r^2$
 casquete esferico de altura h (verificado)
 $V = \pi h^2 (3r - h) / 3$
 piramide $V = (\text{area base}) * h / 3$
 tetraedro regular de arista a (verificado)
 $V = a^3 / (6*\sqrt{2})$ $S = \sqrt{3} * a^2$
 toro, R al centro del tubo y r el radio del tubo (verificado)
 $V = 2 \pi^2 R r^2$ $S = 4 \pi^2 R r$

Reglas que se olvidan

Al escalar una figura por k: longitudes x k, areas x k^2 , volúmenes x k^3 . Sirve para problemas de semejanza.

Pappus: al girar una figura plana alrededor de un eje externo, V = area de la figura * recorrido de su centroide.

El cono y la piramide son un tercio del prisma de igual base y altura; la esfera es dos tercios del cilindro que la encierra.

14.3 Combinatoria

C++\Formulario\combinatoria.cpp

Conteos clasicos con sus primeros valores para identificar la sucesion en la salida de ejemplo: 1, 2, 5, 14, 42 son Catalan; 1, 1, 2, 5, 15, 52 son Bell; 1, 0, 1, 2, 9, 44 son desarreglos. Incluye el triangulo de Pascal hasta n=10 y las identidades que permiten cerrar una sumatoria antes de escribir una sola linea de codigo.

Los cuatro conteos base

$P(n,k) = n!/(n-k)!$	ordenado, sin repetir
$C(n,k) = n!/(k!(n-k)!)$	sin orden, sin repetir
n^k	ordenado, con repeticion
$C(n+k-1, k)$	sin orden, con repeticion
$n!/(a! b! c! \dots)$	permutaciones de un multiconjunto

Identidades de binomiales (sirven para colapsar sumatorias)

$C(n,k) = C(n-1,k-1) + C(n-1,k)$	Pascal
$C(n,k) = C(n,n-k)$	simetria
$\sum_k C(n,k) = 2^n$	fila completa
$\sum_k (-1)^k C(n,k) = 0$	para $n \geq 1$
$\sum_{i=k..n} C(i,k) = C(n+1,k+1)$	hockey stick
$\sum_i C(m,i)C(n,k-i) = C(m+n,k)$	Vandermonde
$k*C(n,k) = n*C(n-1,k-1)$	absorcion

Triangulo de Pascal

n=0 1

n=1	1	1																		
n=2	1	2	1																	
n=3	1	3	3	1																
n=4	1	4	6	4	1															
n=5	1	5	10	10	5	1														
n=6	1	6	15	20	15	6	1													
n=7	1	7	21	35	35	21	7	1												
n=8	1	8	28	56	70	56	28	8	1											
n=9	1	9	36	84	126	126	84	36	9	1										
n=10	1	10	45	120	210	252	210	120	45	10	1									

Stars and bars: repartir n objetos iguales en k grupos

permitiendo grupos vacíos : $C(n+k-1, k-1)$
 todos con al menos uno : $C(n-1, k-1)$

Catalan $Cat(n) = C(2n,n)/(n+1) = \sum_i Cat(i)*Cat(n-1-i)$

Parentesis balanceados, arboles binarios de n nodos,
 triangulaciones de un poligono de n+2 lados, caminos de Dyck.

n	0	1	2	3	4	5	6	7	8	9
Cat(n)	1	1	2	5	14	42	132	429	1430	4862
Cat(12)=208012	Cat(13)=742900	Cat(14)=2674440								

Desarreglos: permutaciones sin ningun punto fijo

$n!$	$(n-1)(!(n-1) + !(n-2))$	$\sim n!/e$
n	0	1
$!n$	1	0
		2
		1

Bell: formas de particionar un conjunto de n elementos

n	0	1	2	3	4	5	6	7	8
B(n)	1	1	2	5	15	52	203	877	4140

Inclusion-exclusion

$|A \cup B| = |A| + |B| - |A \cap B|$
 $|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |A \cap C| - |B \cap C| + |A \cap B \cap C|$
 General: sumar los de tamaño impar, restar los de tamaño par.

En C++

```

11 nCr(ll n, ll k) { // O(k), sin precomputar nada
    if (k < 0 || k > n) return 0;
    k = min(k, n - k);
    ll r = 1;
    for (ll i = 1; i <= k; i++) r = r * (n - i + 1) / i;
    return r; // la division siempre es exacta
}
11 nPr(ll n, ll k) { // O(k), con orden
    if (k < 0 || k > n) return 0;
    ll r = 1; // SIN el min(k, n-k): esa
    for (ll i = 0; i < k; i++) // simetria es solo de C
        r *= (n - i);
    return r; // crece rapido: P(60,10) cabe,
} // P(60,11) ya se desborda
11 catalan(int n) { // O(n), sin binomiales
    ll c = 1;
    for (int i = 0; i < n; i++) c = c * 2 * (2*i + 1) / (i + 2);
    return c;
}
11 desarreglos(int n) { // O(n), sin memoria extra
    ll a = 1, b = 0;
    for (int i = 2; i <= n; i++) {
        ll c = (i-1) * (a + b); a = b; b = c;
    }
    return n == 0 ? 1 : b;
}
11 bell(int n) { // triangulo de Bell, O(n^2)
    vector<ll> f = {1};
    for (int i = 0; i < n; i++) {
        vector<ll> g = {f.back()};
        for (ll x : f) g.push_back(g.back() + x);
        f = g;
    }
}
    
```

```

    return f[0];
}
    
```

Si n es grande y hay muchas consultas, precomputa factoriales e inversos
 (esta en la ficha Binomial Coefficients) y cada nCr sale en $O(1)$.

14.4 Cotas y Limites

C++(Formulario)Cotas y Limites.cpp

La tabla que se mira ANTES de escribir codigo: dado el n del enunciado, que complejidad cabe en el tiempo limite, hasta donde llega cada tipo entero y cuanto memoria ocupa un arreglo. Leer $n \leq 20$ y pensar en bitmask, o $n \leq 450$ y pensar en Floyd-Warshall, ahorra media hora de rumbo equivocado.

n maximo que aguanta cada complejidad (~10^8 operaciones, 1 seg)

O(n!)	n <= 11
O(2^n * n)	n <= 20
O(2^n)	n <= 25
O(n^3)	n <= 450
O(n^2 log n)	n <= 5000
O(n^2)	n <= 10^4
O(n sqrt n)	n <= 10^5
O(n log^2 n)	n <= 10^5
O(n log n)	n <= 10^6
O(n)	n <= 10^8
Si n <= 10^18 la respuesta es formula cerrada, log o binaria.	

Rangos de los enteros

int	hasta 2147483647	(~2.1*10^9)
unsigned int	hasta 4294967295	
long long	hasta 9223372036854775807	
unsigned long long hasta 18446744073709551615		
__int128	hasta ~1.7*10^38	(no se imprime con cout)
~9.2*10^18 es el techo de long long: dos numeros de 10^9 multiplicados caben, tres no.		

Precision de punto flotante

float ~7 digitos significativos
 double ~15-16 digitos (usar siempre este)
 long double ~18-19 en GCC
 Comparar con EPS: fabs(a-b) < 1e-9, nunca a == b
 Si el problema pide enteros exactos, evitar sqrt y pow.

Modulos comunes

1000000007 (10^9+7) primo, el mas usado
 998244353 primo, el de NTT (= 119*2^23 + 1)
 1000000009 primo
 Con 10^9+7 dos residuos multiplicados llegan a ~10^18:
 cabe en long long, pero sumar tres productos sin reducir NO.

Tamanos de memoria (limite tipico 256 MB)

int[10^7]	40 MB	long long[10^7]	80 MB
int[10^8]	400 MB	(no cabe)	
bool[10^8]	100 MB	bitset<10^8>	12.5 MB
vector<vector<int>> de 10^4 x 10^4 = 400 MB, no cabe			

14.5 Datos que no se Deducen

C++(Formulario)Datos que no se Deducen.cpp

Hechos sueltos que si no te los sabes no los inventas en media hora de competencia: el xor de 1..n tiene periodo 4, el ultimo digito de a^b cicla cada 4, un numero es cuadrado perfecto si y solo si tiene una cantidad impar de divisores, y la cantidad de nodos de grado impar de cualquier grafo siempre es par. Las tablas de xor y de ultimo digito estan verificadas corriendo el codigo.

XOR de 1 hasta n (verificado)

n % 4 == 0	-> n
n % 4 == 1	-> 1
n % 4 == 2	-> n + 1
n % 4 == 3	-> 0
n	: 1 2 3 4 5 6 7 8 9 10 11 12
1^..^n	: 1 3 0 4 1 7 0 8 1 11 0 12
Xor de un rango [l,r] = f(r) ^ f(l-1)	

Reglas de divisibilidad

2 : ultimo digito par 5 : termina en 0 o 5
 3 : suma de digitos div 3 9 : suma de digitos div 9
 4 : los ultimos 2 digitos 8 : los ultimos 3 digitos
 6 : divisible por 2 y por 3
11 : suma alternada de digitos divisible por 11
 7 : no hay regla comoda; saca el modulo digito a digito (Horner)

Ultimo digito de a^b : el ciclo dura a lo sumo 4 (verificado)

2: 2 4 8 6 3: 3 9 7 1 7: 7 9 3 1 8: 8 4 2 6
 4: 4 6 9: 9 1 5: 5 6: 6
 0: 0 1: 1
 Entonces $a^b \bmod 10$ depende de $b \bmod 4$, tratando $b = 0$ aparte.

Cantidad de digitos

digitos de $n = \text{floor}(\log_{10}(n)) + 1$ para $n \geq 1$
Mas seguro sin flotantes: $\text{to_string}(n).size()$
 digitos de $n!$ = suma de $\log_{10}(i)$ para $i=1..n$, redondeado arriba

Josephus: n en circulo, se elimina cada k-esima persona

$k = 2$: escribe $n = 2^m + L$, la respuesta es $2L + 1$
equivale a rotar a la izquierda el bit mas alto de n
 k general (0-indexado):
 $J = 0$; for ($i = 2$; $i \leq n$; $i++$) $J = (J + k) \% i$;
 la respuesta 1-indexada es $J + 1$

Principio del palomar

$n+1$ objetos en n cajas $\rightarrow$ alguna caja tiene 2 o mas
 Entre $n+1$ numeros cualesquiera de $1..2n$ hay dos consecutivos
 Con $n+1$ prefijos y n residuos, dos comparten residuo
 $\rightarrow$ siempre existe un subarreglo con suma divisible por n

Grafos: hechos que se olvidan

arbol de n nodos: exactamente $n-1$ aristas y ningun ciclo
 grafo completo de n nodos: $n(n-1)/2$ aristas
 suma de todos los grados = $2 \cdot$ cantidad de aristas
 la cantidad de nodos de grado impar siempre es PAR
 camino euleriano existe si hay 0 o 2 nodos de grado impar
 ciclo euleriano existe si todos los grados son pares
 arboles etiquetados con n nodos: n^{n-2} (Cayley)
 un grafo bipartito no tiene ciclos de longitud impar

Varios que aparecen disfrazados

ceros al final de $n!$ = $n/5 + n/25 + n/125 + \dots$
 n es cuadrado perfecto $\Leftrightarrow$ tiene una cantidad IMPAR de divisores
(de ahí el problema clasico de los casilleros que se abren)
 suma de los primeros n impares = n^2
 la mediana minimiza la suma de distancias absolutas
 el promedio minimiza la suma de distancias al cuadrado
 para repartir en dos grupos lo mas parejo posible: subset sum
 $(a+b) \% m = ((a \% m) + (b \% m)) \% m$, igual con el producto,
 pero NO con la division: ahí va el inverso modular

En C++

```
ll xorHasta(ll n) { // 1^2^...^n en O(1)
    switch (n % 4) {
        case 0: return n;
        case 1: return 1;
        case 2: return n + 1;
    }
    return 0;
}
xor del rango [l,r] = xorHasta(r) ^ xorHasta(l-1)
int josephus(int n, int k) { // O(n), 1-indexado
    int j = 0;
    for (int i = 2; i <= n; i++) j = (j + k) % i;
    return j + 1;
}
```

```
int josephus2(int n) { // caso k=2 en O(1)
    return 2 * (n - (1 << (31 - __builtin_clz(n)))) + 1;
}
int numDigitos(ll n) { // sin log10, sin error de flotante
    int d = 0;
    while (n) d++, n /= 10;
    return d ? d : 1;
}
```

14.6 Geometria

C++(Formulario)Geometria.cpp

Formulas de geometria que se usan sin pensar pero se olvidan bajo presion: areas, Heron, shoelace, Pick, sector y segmento circular, y el significado del signo del producto cruz. La regla que mas salva: si los datos son enteros, trabajar con cross/dot y distancias al cuadrado en vez de sqrt, y no comparar doubles con ==

Triangulo

area = base * altura / 2
 area = $|\text{cross}(B-A, C-A)| / 2$ con coordenadas
 Heron: $s=(a+b+c)/2$, area = $\sqrt{s(s-a)(s-b)(s-c)}$
 ley de cosenos : $c^2 = a^2 + b^2 - 2ab \cos(C)$
 ley de senos : $a/\sin(A) = b/\sin(B) = c/\sin(C) = 2R$
 desigualdad triangular: $a+b > c$ para que exista

Poligono simple (vertices en orden)

Shoelace: area = $|\sum (x_i * y_{i+1} - x_{i+1} * y_i)| / 2$
El signo de la suma dice la orientacion: + antihorario.
 Regular de n lados y lado L : area = $n * L^2 / (4 * \tan(\pi/n))$
 angulo interior = $(n-2) * 180 / n$ grados

Pick: para poligono con vertices en enteros

area = $I + B/2 - 1$
 I = puntos enteros interiores, B = puntos enteros en el borde
 puntos enteros en el segmento $(x1,y1)-(x2,y2)$:
 $\gcd(|x2-x1|, |y2-y1|) + 1$ (contando los dos extremos)

Circulo de radio r

perimetro = $2 * \pi * r$ area = $\pi * r^2$
 sector de angulo t (rad) : area = $r^2 * t / 2$
 segmento circular : area = $r^2 (t - \sin t) / 2$
 cuerda de angulo t : largo = $2r * \sin(t/2)$
 $\text{PI} = \text{acos}(-1.0)$ 1 rad = $180/\pi$ grados

Producto cruz y punto (la base de todo)

$\text{cross}(A,B) = ax*by - ay*bx$
 $\text{cross}(A,B,C) = \text{cross}(B-A, C-A)$
 > 0 giro antihorario, < 0 horario, $= 0$ colineales
 $\text{dot}(A,B) = ax*bx + ay*by$
 > 0 angulo agudo, < 0 obtuso, $= 0$ perpendicular

Con enteros no uses sqrt: compara distancias al cuadrado.

Distancias

punto a recta por A,B : $|\text{cross}(B-A, P-A)| / |B-A|$
 Manhattan $|dx|+|dy|$, Chebyshev $\max(|dx|, |dy|)$
 Rotar 45 grados convierte Manhattan en Chebyshev:
 $(x,y) \rightarrow (x+y, x-y)$

Euler para grafos planos y poliedros

$V - E + F = 2$
 Con n rectas en posicion general el plano queda en
 $1 + n + C(n,2)$ regiones: 2, 4, 7, 11, 16, 22, 29, 37

14.7 Probabilidad Geometrica y Areas Raras

C++ Formulas/Probabilidad Geometrica.cpp

Para los problemas que en realidad son matematica pura y lo dificil no es programarlos sino plantear el area: "cual es la probabilidad de que un punto al azar del jardin caiga dentro de algun circulo", "cuanto vale en promedio", "que tan seguido se encuentran dos personas". La idea unica de toda la ficha: probabilidad = area favorable / area total, y el valor esperado es la suma de cada valor por su probabilidad, sin necesitar independencia. Trae las areas que no se sacan de memoria: interseccion de dos circulos, circulo con rectangulo, y los clasicos de la cita, el palito y la aguja de Buffon. Todo verificado con Monte Carlo en este mismo equipo.

La idea base

$P(\text{evento}) = \text{medida favorable} / \text{medida total}$
 En 1D la medida es largo, en 2D area, en 3D volumen.
 Si el punto se elige UNIFORME, no hay nada mas que calcular areas. El problema "de programacion" es en realidad geometria.

Valor esperado

$E[X] = \text{suma de } (\text{valor} * \text{probabilidad de ese valor})$
 LINEALIDAD: $E[A + B] = E[A] + E[B]$ SIEMPRE, aunque A y B esten correlacionados. Es lo que permite sumar la contribucion de cada objeto por separado en vez de analizar el conjunto.
 Ejemplo tipico: cada circulo i tiene valor v_i y cubre un area a_i del jardin de $W \times H$, entonces
 $\text{respuesta} = \text{suma de } v_i * (a_i / (W*H))$
 Si los circulos se traslapan NO importa: la linealidad sigue valiendo, cada uno aporta su parte por separado.

Interseccion de DOS circulos (area de la lente) (verificado)

d = distancia entre centros
 si $d \geq r_1 + r_2$ $\rightarrow 0$, no se tocan
 si $d < |r_1 - r_2|$ $\rightarrow \pi * \min(r_1, r_2)^2$, uno dentro del otro
 si no:
 $a_1 = 2 * \arccos((d*d + r_1*r_1 - r_2*r_2) / (2*d*r_1))$
 $a_2 = 2 * \arccos((d*d + r_2*r_2 - r_1*r_1) / (2*d*r_2))$
 $A = 0.5*r_1*r_1*(a_1 - \sin(a_1)) + 0.5*r_2*r_2*(a_2 - \sin(a_2))$
 O sea: la lente son dos segmentos circulares pegados.
 union de los dos = $\pi r_1^2 + \pi r_2^2 - A$

Segmento y sector (los ladrillos de todo lo anterior)

sector de angulo t : $A = r^2 * t / 2$
 segmento (lo que queda entre la cuerda y el arco):
 $A = r^2 * (t - \sin t) / 2$
 cuerda del angulo t : largo = $2 r \sin(t/2)$

Circulo contra rectangulo o poligono

No hay formula cerrada corta. Se recorre el borde del poligono sumando areas con signo, tratando cada arista aparte segun si el tramo cae dentro o fuera del circulo.
 Esta implementado en la seccion Geometry, ficha "Area Circulo-Poligono". Es el metodo del problema del jardin.

Que una FIGURA QUEPA: traducir "cabe" a desigualdades

Cuando piden la probabilidad de que algo quepa dentro de otra cosa, casi nunca se integra a lo bruto: conviene escribir el "cabe" como desigualdades sobre las coordenadas, que suelen simplificarse muchisimo.

LA HERRAMIENTA CLAVE (arco capaz): el circulo que tiene al segmento AB como DIAMETRO cabe dentro de un convexo K si y solo si todo punto de la frontera de K ve a AB con angulo de a lo sumo 90 grados. Es porque ese circulo es justo donde el angulo vale 90, y por fuera de el es menor.

"Angulo ≤ 90 desde Q" se escribe sin trigonometria:
 $(A - Q) \cdot (B - Q) \geq 0$

y si Q recorre un lado recto, se minimiza ese producto punto.

Ejemplo, para el lado $x = 0$ con $Q = (0, q)$:
 $x_1*x_2 + (y_1-q)*(y_2-q) \geq 0$ para todo q
 el minimo cae en $q = (y_1+y_2)/2$ y vale $x_1*x_2 - (y_1-y_2)^2/4$
 o sea la condicion es $4*x_1*x_2 \geq (y_1-y_2)^2$

En un rectangulo $L \times W$ salen cuatro asi, sin una sola raiz:

$4*x_1*x_2 \geq (y_1-y_2)^2$ $4*(W-x_1)*(W-x_2) \geq (y_1-y_2)^2$

$4*y_1*y_2 \geq (x_1-x_2)^2$ $4*(L-y_1)*(L-y_2) \geq (x_1-x_2)^2$

(verificado contra la condicion directa de centro y radio en 3000000 de pruebas, sin una sola diferencia)

Y como las desigualdades son homogeneas de grado 2, escalar el rectangulo no cambia la probabilidad: solo importa la PROPORCION entre los lados. Para dos piedras uniformes en $L \times W$, con

$t = \min(L, W) / \max(L, W)$, la probabilidad de que el circulo quepa es
 $P = (\pi/6) * t * (2 - t)$

(formula verificada numericamente en 8 proporciones, no demostrada; el problema es "I - Omens" de ICPC 2024, esta en este cuaderno)

Lo transferible es el metodo, no la formula: primero traducir "cabe" a desigualdades, despues mirar si colapsan.

Uniones de varias figuras

$|A \cup B| = |A| + |B| - |A \cap B|$
 Con tres o mas, inclusion-exclusion (2^n terminos): solo sirve si n es chico. Si no, integracion por barrido o Monte Carlo.
 Para "cuanta area cubre al menos un circulo" con muchos circulos, lo estandar es barrido angular, no inclusion-exclusion.

Clasicos que se resuelven dibujando el espacio de casos

Cita: dos personas llegan uniformemente en $[0, T]$ y se esperan t minutos. El espacio es el cuadrado $T \times T$, y se encuentran en la franja $|x - y| \leq t$. (verificado)
 $P = 1 - ((T - t)/T)^2$

Palito partido en tres pedazos al azar: forman triangulo con probabilidad $1/4$. (verificado)

Aguja de Buffon: aguja de largo L sobre lineas separadas d , con $L \leq d$, cruza una linea con $P = 2L / (\pi * d)$ (verificado)

Como atacar uno de estos en competencia

1. Identifica que se elige al azar y con que distribucion.
 2. Dibuja el espacio completo de posibilidades y su medida.
 3. Sombrea la region favorable y calcula SU medida.
 4. Si hay varios objetos con valor, usa linealidad y suma valor por probabilidad, uno por uno.
 5. Si el area no sale cerrada, piensa en barrido o en integrar.
- Antes de entregar, comprueba con Monte Carlo: es cinco lineas y te dice si la formula que planteaste esta bien.

En C++

Area de la interseccion de dos circulos (la lente).

```
double areaLente(double r1, double r2, double d) {
    if (d >= r1 + r2) return 0; // separados
    if (d <= fabs(r1 - r2)) // uno dentro del otro
        return PI * min(r1, r2) * min(r1, r2);
    double a1 = 2 * acos((d*d + r1*r1 - r2*r2) / (2*d*r1));
    double a2 = 2 * acos((d*d + r2*r2 - r1*r1) / (2*d*r2));
    return 0.5*r1*r1*(a1 - sin(a1)) + 0.5*r2*r2*(a2 - sin(a2));
}
```

union de los dos = $\pi r_1^2 + \pi r_2^2 - \text{areaLente}(r_1, r_2, d)$
 Los dos primeros if no son adorno: sin ellos, acos recibe un valor fuera de $[-1, 1]$ y devuelve NaN.

14.8 Rectas y Conicas

C++ Formulas/Rectas y Conicas.cpp

Geometria analitica de toda la vida: pendiente, las cuatro formas de escribir una recta, cuando dos son paralelas o perpendiculares, donde se cortan, y las ecuaciones de circunferencia, parabola, elipse e hiperbola. El corte de dos rectas se resuelve por Cramer, que es lo mismo que "igualar las dos ecuaciones" pero sin despejar a mano y sin dividir por cero sin darse cuenta: si el determinante da 0, son paralelas. Formulas verificadas numericamente.

Pendiente por dos puntos

$m = (y_2 - y_1) / (x_2 - x_1)$ indefinida si $x_1 == x_2$ (vertical)
 $m > 0$ sube, $m < 0$ baja, $m = 0$ horizontal
 angulo con el eje x : $\theta = \text{atan2}(y_2 - y_1, x_2 - x_1)$

Formas de la recta

pendiente-intercepto : $y = m x + b$ con $b = y_1 - m*x_1$
 punto-pendiente : $y - y_1 = m (x - x_1)$

```

general      : A x + B y + C = 0
desde dos puntos: A = y2-y1, B = -(x2-x1),
                  C = -(A*x1 + B*y1)
pendiente = -A/B ; es vertical cuando B = 0
simetrica    : x/p + y/q = 1 (corta los ejes en p y q)
    
```

Dos rectas

```

paralelas      : m1 == m2      o A1*B2 - A2*B1 == 0
perpendiculares : m1 * m2 == -1 o A1*A2 + B1*B2 == 0
angulo entre ellas : tan(t) = |(m2 - m1) / (1 + m1*m2)|
Con datos enteros compara pendientes cruzando fracciones y
evita dividir: dy1*dx2 == dy2*dx1
    
```

Interseccion: igualar las dos ecuaciones (Cramer, verificado)

```

A1 x + B1 y = C1
A2 x + B2 y = C2
det = A1*B2 - A2*B1
si det == 0 -> paralelas (o la misma recta, sin punto unico)
x = (C1*B2 - C2*B1) / det
y = (A1*C2 - A2*C1) / det
Para cortar y = f(x) con y = g(x), resuelve f(x) - g(x) = 0.
    
```

Distancias y punto medio

```

|P1P2| = sqrt((x2-x1)^2 + (y2-y1)^2)
punto medio = ((x1+x2)/2, (y1+y2)/2)
punto a recta Ax+By+C=0 : |A*px + B*py + C| / sqrt(A^2 + B^2)
entre dos paralelas      : |C1 - C2| / sqrt(A^2 + B^2)
El SIGNO de A*px+B*py+C dice de que lado del corte esta el punto.
    
```

Circunferencia

```

(x - h)^2 + (y - k)^2 = r^2      centro (h,k), radio r
general : x^2 + y^2 + Dx + Ey + F = 0
          centro (-D/2, -E/2)
          r = sqrt(D^2/4 + E^2/4 - F)
Dos circunferencias, con d la distancia entre centros:
se cortan en 2 puntos si |r1-r2| < d < r1+r2
tangentes exteriores si d == r1+r2
tangentes interiores si d == |r1-r2|
una dentro de la otra sin tocarse si d < |r1-r2|
    
```

Parabola

```

y = a x^2 + b x + c
vertice : x = -b/(2a) , y = c - b^2/(4a)
abre hacia arriba si a > 0, hacia abajo si a < 0
canonica : y - k = a (x - h)^2 con vertice (h,k)
    
```

Elipse e hipérbola centradas en (h,k)

```

elipse : (x-h)^2/a^2 + (y-k)^2/b^2 = 1
a y b son los semiejes; si a = b es una circunferencia
hipérbola : (x-h)^2/a^2 - (y-k)^2/b^2 = 1
asintotas : y - k = +/- (b/a)(x - h)
    
```

En C++

```

double distPuntoRecta(double A, double B, double C,
                     double px, double py) {
    return fabs(A*px + B*py + C) / hypot(A, B);
}
bool corte(double A1, double B1, double C1,
           double A2, double B2, double C2, double& x, double& y) {
    double d = A1*B2 - A2*B1;
    if (fabs(d) < 1e-12) return false; // paralelas
    x = (C1*B2 - C2*B1) / d;
    y = (A1*C2 - A2*C1) / d;
    return true;
}
    
```

Con coordenadas enteras, para saber si tres puntos son colineales NO calcules pendientes: usa el producto cruz, que no divide ni pierde precision. $(x2-x1)*(y3-y1) - (y2-y1)*(x3-x1) == 0$

14.9 Secuencias Notables

C++FormularioSecuenciasNotables.cpp

Las sucesiones que hay que reconocer de vista en un caso de ejemplo, con los primeros valores y el punto exacto donde se desbordan: 20! es el ultimo factorial que cabe en long long, F(92) es el ultimo Fibonacci que cabe, y 2^{31} ya no cabe en int. Saber el límite de antemano evita el overflow silencioso, que es el bug mas caro en competencia.

Fibonacci F(0)=0, F(1)=1, F(n)=F(n-1)+F(n-2)

```

n      0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
F(n)   0  1  1  2  3  5  8 13 21 34 55 89 144 233 377 610
F(20)=6765 F(30)=832040 F(40)=102334155
F(92) = 7540113804746346429 es el ULTIMO que cabe en long long;
F(93) = 12200160415121876738 ya se pasa (el techo es 9.22*10^18).
Identidades:
F(m+n) = F(m)F(n+1) + F(m-1)F(n)      fast doubling
sum_{i<=n} F(i) = F(n+2) - 1
F(n-1)F(n+1) - F(n)^2 = (-1)^n         Cassini
gcd(F(m),F(n)) = F(gcd(m,n))
    
```

Lucas L(0)=2, L(1)=1, misma recurrencia

```

n      0  1  2  3  4  5  6  7  8  9 10 11 12 13
L(n)   2  1  3  4  7 11 18 29 47 76 123 199 322 521
L(n) = F(n-1) + F(n+1)
    
```

Numeros figurados

```

n      1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
triangular  1  3  6 10 15 21 28 36 45 55 66 78 91 105 120
cuadrado    1  4  9 16 25 36 49 64 81 100 121 144 169 196 225
pentagonal  1  5 12 22 35 51 70 92 117 145 176 210 247 287 330
hexagonal   1  6 15 28 45 66 91 120 153 190 231 276 325 378 435
triangular T(n)=n(n+1)/2, T(n)+T(n-1) = n^2
    
```

Factoriales (20! es el ultimo que cabe en long long)

```

n      0  1  2  3  4  5  6  7  8  9
n!     1  1  2  6 24 120 720 5040 40320 362880
10! = 3628800      13! = 6227020800 (ya se pasa de int)
11! = 39916800     15! = 1307674368000
12! = 479001600    20! = 2432902008176640000
21! desborda long long. Si aparece n! con n > 20, es modular.
    
```

Potencias de 2

```

k      0  1  2  3  4  5  6  7  8  9 10
2^k    1  2  4  8 16 32 64 128 256 512 1024
2^20 = 1048576 (~10^6)
2^30 = 1073741824 (~10^9, limite de int)
2^31 = 2147483648 2^32 = 4294967296
2^62 = 4611686018427387904 (long long aguanta hasta 2^63-1)
    
```

Numeros perfectos (iguales a la suma de sus divisores propios)

6, 28, 496, 8128, 33550336
 Todos los pares son $2^p(p-1)*(2^p - 1)$ con 2^p-1 primo (Mersenne).

En C++

```

Fibonacci por fast doubling: O(log n), aguanta n = 10^18.
pair<ll,ll> fib(ll n) { // devuelve {F(n), F(n+1)} mod M
    if (!n) return {0, 1};
    pair<ll,ll> p = fib(n >> 1);
    ll a = p.first, b = p.second;
    ll c = a * ((2*b % M - a + M) % M) % M;
    ll d = (a*a + b*b) % M;
    return (n & 1) ? make_pair(d, (c+d) % M) : make_pair(c, d);
}
Los figurados son O(1) directo:
ll triangular(ll n) { return n*(n+1)/2; }
ll pentagonal(ll n) { return n*(3*n-1)/2; }
ll hexagonal(ll n) { return n*(2*n-1); }
    
```

Para factoriales, tabla hasta 20 y listo: mas alla no cabe sin modulo.

14.10 STL Indispensable

C++FormularioSTLIndispensable.cpp

Las funciones de la librería estandar que la gente reimplementa a mano por no saber que existen, y las que se usan mal por un detalle: `next_permutation` solo recorre las $n!$ si el vector arranca ordenado (verificado: desde {1,2,3} da 6 permutaciones, desde {3,1,2} solo 2), y `substr` toma (inicio, CANTIDAD) y no (inicio, fin). Tener esta lista a mano ahorra escribir veinte líneas que ya vienen hechas y probadas.

Ordenar y buscar (el vector debe estar ordenado para los bound)

```
sort(all(v))                O(n log n)
stable_sort(all(v))         conserva el orden de los iguales
sort(all(v), greater<int>()) descendente
lower_bound(all(v), x)      primer elemento >= x
upper_bound(all(v), x)      primer elemento > x
    cuantos x hay = upper_bound - lower_bound
    posicion = lower_bound(all(v), x) - v.begin()
binary_search(all(v), x) devuelve solo true/false
nth_element(v.begin(), v.begin()+k, v.end())
    deja el k-esimo en su sitio en O(n) promedio, sin ordenar todo
partial_sort(v.begin(), v.begin()+k, v.end()) solo los k primeros
```

Permutaciones

```
next_permutation(all(v)) siguiente en orden lexicografico
devuelve false cuando llega a la ultima y reinicia
ORDENA PRIMERO si quieres recorrer las n! (verificado)
prev_permutation(all(v))
```

Lo que no hay que reimplementar

```
__gcd(a,b)                  en C++17 tambien gcd(a,b) y lcm(a,b)
iota(all(v), 0)             llena con 0,1,2,3,...
accumulate(all(v), 0LL) suma (usa 0LL para no desbordar)
max_element / min_element devuelven ITERADOR: resta v.begin()
count(all(v), x)            find(all(v), x) reverse(all(v))
rotate(v.begin(), v.begin()+k, v.end()) rota a la izquierda
fill(all(v), x)             v.assign(n, x) swap(a, b)
minmax_element(all(v)) devuelve los dos de una
is_sorted(all(v)) next(it) prev(it)
```

Conversiones

```
to_string(x) stoi(s) stoll(s) stod(s)
s.substr(i, LARGO) <- cantidad, NO posicion final
s.find(t) devuelve string::npos si no aparece
getline(cin, s) lee la línea entera con espacios
stringstream ss(línea); while (ss >> token) ... partir por espacio
```

Estructuras y cuando usarlas

```
vector<int> v(n, 0)          vector<vector<int>> g(n)
set / multiset              ordenado, O(log n), tiene .lower_bound propio
unordered_set              O(1) promedio, sin orden
map / unordered_map         igual pero clave -> valor
deque                      push_front y push_back en O(1)
priority_queue              max-heap; greater<> para convertirlo en min-heap
bitset<N>                   operaciones de 64 bits por vez, 8x menos memoria
array<int,N>               tamaño fijo, sin costo de heap
```

Detalles que sirven

```
v.reserve(n) evita realocar si sabes el tamaño de antemano
emplace_back construye en el sitio, evita una copia
structured bindings: for (auto &k, val : mapa) (C++17)
lambda: [](int a, int b){ return a > b; }
capturar por referencia para comparadores con datos externos:
    sort(all(idx), [&](int i, int j){ return v[i] < v[j]; });
```

14.11 Sumas y Series

C++FormularioSumasYSeries.cpp

Las sumas cerradas que mas se repiten, cada una con sus primeros valores para reconocer el patron directamente en el ejemplo del enunciado: si la salida crece 1, 3, 6, 10, 15 es Gauss; 1, 4, 9, 16 son cuadrados; 1, 5, 14, 30 es suma de cuadrados; 1, 9, 36, 100 es suma de cubos. Identificar la sucesion en los casos de prueba suele ser mas rapido que derivar.

Gauss: suma de los primeros n enteros

```
1 + 2 + ... + n = n(n+1)/2
n    1  2  3  4  5  6  7  8  9 10 11 12
S(n) 1  3  6 10 15 21 28 36 45 55 66 78
```

Suma de cuadrados

```
1^2 + 2^2 + ... + n^2 = n(n+1)(2n+1)/6
n    1  2  3  4  5  6  7  8  9 10 11 12
S(n) 1  5 14 30 55 91 140 204 285 385 506 650
```

Suma de cubos = el cuadrado de la suma de Gauss

```
1^3 + 2^3 + ... + n^3 = [n(n+1)/2]^2
n    1  2  3  4  5  6  7  8  9 10 11 12
S(n) 1  9 36 100 225 441 784 1296 2025 3025 4356 6084
```

Impares dan cuadrados perfectos, pares dan n(n+1)

```
1+3+5+...+(2n-1) = n^2          2+4+...+2n = n(n+1)
n    1  2  3  4  5  6  7  8  9 10 11 12
impares 1  4  9 16 25 36 49 64 81 100 121 144
pares   2  6 12 20 30 42 56 72 90 110 132 156
```

Progresion aritmetica (a, a+d, a+2d, ...)

```
termino k : a + (k-1)d
suma de n : n(2a + (n-1)d)/2 = n(primer + ultimo)/2
```

Progresion geometrica (a, ar, ar^2, ...)

```
termino k : a*r^(k-1)
suma de n : a(r^n - 1)/(r - 1) si r != 1
suma inf : a/(1 - r) si |r| < 1
En modular /(r-1) es inverso modular; si r = 1 mod p no existe.
```

Potencias de 2: la suma es la siguiente menos 1

```
1 + 2 + 4 + ... + 2^n = 2^(n+1) - 1
n    0  1  2  3  4  5  6  7  8  9 10 11 12
2^n  1  2  4  8 16 32 64 128 256 512 1024 2048 4096
suma 1  3  7 15 31 63 127 255 511 1023 2047 4095 8191
```

Serie armonica H(n) = 1 + 1/2 + ... + 1/n ~ ln(n) + 0.5772

```
H(10) ~ 2.93 H(100) ~ 5.19 H(10^6) ~ 14.39
Aparece al contar divisores: sum_d=1..n n/d ~ n*ln(n).
```

Conteos sobre un arreglo de n elementos

```
pares i<j : n(n-1)/2
pares ordenados i!=j : n(n-1)
subarreglos contiguos : n(n+1)/2
subconjuntos : 2^n no vacios: 2^n - 1
```

En C++ (todo O(1); dividir ANTES de multiplicar evita el overflow)

```
ll gauss(ll n) { // 1+2+...+n
    return n % 2 ? (n+1)/2*n : n/2*(n+1);
}
ll sumaCuadrados(ll n) { // 1^2+...+n^2
    ll a = n, b = n+1, c = 2*n+1;
    if (a % 2 == 0) a /= 2; else b /= 2;
    if (a % 3 == 0) a /= 3;
    else if (b % 3 == 0) b /= 3; else c /= 3;
    return a * b * c;
}
ll sumaCubos(ll n) { ll g = gauss(n); return g * g; }
Con n = 10^9, gauss(n) = 500000000500000000 y cabe en long long;
pero n*(n+1) sin dividir primero YA se desborda.
```

```
Suma geometrica modular, O(log n):
ll sumaGeoMod(ll a, ll r, ll n) {
    // a(r^n-1)/(r-1) mod M
    if (r % M == 1) return a % M * (n % M) % M;
    ll num = (modpow(r, n, M) - 1 + M) % M;
    return a % M * num % M * modpow((r-1) % M, M-2, M) % M;
}
```

14.12 Tabla ASCII

C++>Formulario>Tabla ASCII.cpp

Tabla ASCII de los caracteres imprimibles (32 a 126) en cuatro columnas, mas los valores y trucos que se usan de verdad: c - '0' para el dígito, c - 'a' para el índice de la letra, y que 'a' - 'A' = 32 permite cambiar de caso con un xor. Incluye la nota del '\r' de Windows, que rompe comparaciones al leer con getline.

dec	chr	dec	chr	dec	chr	dec	chr
32	SP	56	8	80	P	104	h
33	!	57	9	81	Q	105	i
34	"	58	:	82	R	106	j
35	#	59	;	83	S	107	k
36	\$	60	<	84	T	108	l
37	%	61	=	85	U	109	m
38	&	62	>	86	V	110	n
39	'	63	?	87	W	111	o
40	(	64	@	88	X	112	p
41	)	65	A	89	Y	113	q
42	*	66	B	90	Z	114	r
43	+	67	C	91	[	115	s
44	,	68	D	92	\	116	t
45	-	69	E	93	]	117	u
46	.	70	F	94	^	118	v
47	/	71	G	95	_	119	w
48	0	72	H	96	`	120	x
49	1	73	I	97	a	121	y
50	2	74	J	98	b	122	z
51	3	75	K	99	c	123	{
52	4	76	L	100	d	124	
53	5	77	M	101	e	125	}
54	6	78	N	102	f	126	~
55	7	79	O	103	g		

Lo que de verdad se usa

```
'0' = 48      '9' = 57      digito: c - '0'
'A' = 65      'Z' = 90
'a' = 97      'z' = 122     indice: c - 'a' (0..25)
'a' - 'A' = 32 -> minuscula: c | 32, mayuscula: c & ~32
' ' = 32      '\n' = 10     '\0' = 0      '\t' = 9
Cambiar de caso con xor: c ^ 32
```

Chequeos de <ctype>

```
isalpha isdigit isalnum isupper islower isspace ispunct
toupper tolower
Rápido sin librería: (c>='0' && c<='9')
```

Control mas comunes (no imprimibles)

```
0 NUL  9 TAB 10 LF (\n) 13 CR (\r) 27 ESC
Archivos de Windows traen CR+LF: si lees con getline puede
quedar un '\r' al final de la línea. Quitarlo antes de comparar.
```

14.13 Teoría de Números

C++>Formulario>Teoria de Números.cpp

Divisores, phi, gcd, aritmetica modular y CRT, con las tablas hasta 20 para revisar a mano un caso chico. La mitad de los problemas de números se resuelven sabiendo que d(n), sigma(n) y phi(n) salen directo de la factorización, y que en modular dividir significa multiplicar por el inverso.

Con la factorización n = p1^a1 * p2^a2 * ...

```
cantidad de divisores d(n) = (a1+1)(a2+1)...
suma de divisores sigma(n) = prod (p^(a+1)-1)/(p-1)
funcion de Euler phi(n) = n * prod (1 - 1/p)
Ejemplo n=12=2^2*3 : d=3*2=6, sigma=7*4=28, phi=12*(1/2)(2/3)=4
```

Tablas hasta 20

n	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
d(n)	1	2	2	3	2	4	2	4	3	4	2	6	2	4	4	5	2	6	2	6
sig(n)	1	3	4	7	6	12	8	15	13	18	12	28	14	24	24	31	18	39	20	42
phi(n)	1	1	2	2	4	2	6	4	6	4	10	4	12	6	8	8	16	6	18	8

gcd y lcm

```
gcd(a,b) * lcm(a,b) = a*b      (dividir antes de multiplicar)
gcd(a,b) = gcd(b, a mod b)    Euclides
gcd(F(m), F(n)) = F(gcd(m,n)) Fibonacci
a y b coprimos <=> gcd(a,b) = 1
```

Modular

```
Fermat : a^(p-1) = 1 mod p      si p primo y p no divide a
inverso : a^(p-2) mod p        si p primo
Euler : a^phi(m) = 1 mod m      si gcd(a,m) = 1
Wilson : (p-1)! = -1 mod p      si y solo si p es primo
(a/b) mod p = a * b^(p-2) mod p
```

OJO: restar en modular siempre ((a-b) % m + m) % m

Teorema chino del resto (CRT)

```
x = r1 mod m1, x = r2 mod m2, con m1, m2 coprimos
-> solucion unica modulo m1*m2
```

Legendre: exponente de p en n!

```
e = n/p + n/p^2 + n/p^3 + ...      (divisiones enteras)
Sirve para contar ceros finales de n! (exponente de 5).
```

Primos menores que 100 (hay 25)

```
2 3 5 7 11 13 17 19 23 29 31 37
41 43 47 53 59 61 67 71 73 79 83 89
97
```

Criba: O(n log log n). Cantidad de primos ~ n/ln(n).

```
hasta 10^3 hay 168      hasta 10^6 hay 78498
hasta 10^4 hay 1229     hasta 10^7 hay 664579
hasta 10^5 hay 9592     hasta 10^9 hay ~5.08*10^7
```

En C++ (factorizar en O(sqrt n) da los tres de una)

```
ll numDivisores(ll n) {
    ll r = 1;
    for (ll p = 2; p*p <= n; p++) if (n % p == 0) {
        ll e = 0; while (n % p == 0) n /= p, e++;
        r *= e + 1;
    }
    if (n > 1) r *= 2;          // quedo un primo grande
    return r;
}

ll sumaDivisores(ll n) {
    ll r = 1;
    for (ll p = 2; p*p <= n; p++) if (n % p == 0) {
        ll t = 1, q = 1;
        while (n % p == 0) n /= p, q *= p, t += q;
        r *= t;
    }
    if (n > 1) r *= 1 + n;
    return r;
}

ll phi(ll n) {
    ll r = n;
    for (ll p = 2; p*p <= n; p++) if (n % p == 0) {
        while (n % p == 0) n /= p;
        r -= r / p;
    }
    if (n > 1) r -= r / n;
    return r;
}

ll modpow(ll b, ll e, ll m) {      // O(log e)
```

```

ll r = 1; b %= m;
while (e) { if (e & 1) r = r*b % m; b = b*b % m; e >>= 1; }
return r;
}
inverso modular con m primo: modpow(a, m-2, m)
ll legendre(ll n, ll p) { // exponente de p en n!
    ll e = 0;
    for (ll q = p; q <= n; q *= p) e += n / q;
    return e;
}
Criba lineal, O(n): cada compuesto se tacha UNA sola vez.
vector<int> criba(int n) {
    vector<int> pr; vector<char> c(n+1, 1); c[0] = c[1] = 0;
    for (int i = 2; i <= n; i++) {
        if (c[i]) pr.push_back(i);
        for (int p : pr) {
            if ((ll)i*p > n) break;
            c[i*p] = 0;
            if (i % p == 0) break;
        }
    }
    return pr;
}

```

14.14 Trampas de C++

C++\Formulario\Trampas de C++.cpp

Los errores que compilan sin una sola advertencia y dan respuesta equivocada: division entera con negativos, overflow porque $int * int$ se calcula en int antes de guardarse en $long long$, pow devolviendo 999999999999999 en vez de 10^{15} , y comparadores de sort que rompen el programa por usar $<=$ en vez de $<$. Todos los valores de aqui estan verificados corriendo el codigo en este mismo equipo.

Division y modulo con negativos (verificado)

```

-7 / 2 = -3      1a division trunca HACIA CERO, no hacia abajo
-7 % 2 = -1      el resto se lleva el signo del dividendo
7 / -2 = -3      7 % -2 = 1
floor(-7.0/2) = -4 -> distinto de -7/2
modulo siempre positivo : ((a % m) + m) % m
division hacia abajo : (a - ((a % m) + m) % m) / m

```

Esto rompe hashing, indices circulares y aritmetica modular.

Overflow silencioso (verificado)

```

int a = 100000, b = 100000;
long long x = a * b;          -> 1410065408 MAL
long long x = (long long)a * b; -> 100000000000 bien
La conversion ocurre DESPUES de multiplicar: castea un operando.
1 << 40 -> 0                  1LL << 40 -> 1099511627776
accumulate(v.begin(), v.end(), 0) suma en int, desborda
accumulate(v.begin(), v.end(), 0LL) suma en long long
Regla: si el resultado pasa de  $2^{10^9}$ , todo el camino en ll.

```

pow y sqrt son de punto flotante (verificado que falla)

```

(long long)pow(10,15) = 999999999999999 le falta 1
(long long)pow(5,15) = 30517578124 le falta 1

```

Para enteros usa exponenciacion binaria propia.

Para raiz entera, redondea y ajusta:

```

long long r = sqrtl(n);
while (r*r > n) r--;
while ((r+1)*(r+1) <= n) r++;

```

Comparadores de sort

Tiene que ser Estricto: return a < b; nunca a <= b
 Con <= el sort puede salirse del arreglo y tumbar el programa.
 Descendente: return a > b;
 Por varios campos:
 if (a.x != b.x) return a.x < b.x;
 return a.y < b.y;

Contenedores

$m[k]$ CREA la clave con valor por defecto si no existe.

Para solo consultar: $m.count(k)$ o $m.find(k)$.

unique() exige el vector ORDENADO y no borra por si solo:

```

sort(all(v)); v.erase(unique(all(v)), v.end());
En set y map usa el miembro s.lower_bound(x): es O(log n).
std::lower_bound(s.begin(), s.end(), x) recorre en O(n).

```

priority_queue es MAXIMO por defecto. Para minimo:

```
priority_queue<int, vector<int>, greater<int>> pq;
```

Borrar de un vector mientras lo recorres invalida iteradores.

Entrada y salida

cin >> x salta espacios y DEJA el '\n' pendiente. Si despues usas getline, lees una linea vacia: consume el salto primero.
 endl hace flush cada vez; dentro de un bucle usa '\n'.
 Al inicio del main:
 ios::sync_with_stdio(false); cin.tie(nullptr);
 printf de long long es %lld, no %d.
 Si mezclas printf y cout con sync desactivado, el orden se rompe.

Punto flotante

Nunca compares con ==. Usa fabs(a-b) < 1e-9.
 double guarda ~15-16 digitos; por encima de 2^{53} pierde enteros.
 Imprimir con decimales: cout << fixed << setprecision(k)
 Si el problema es de enteros, resuelvelo con enteros.

14.15 Trigonometria

C++\Formulario\Trigonometria.cpp

Conversion grados-radianes, la tabla de valores exactos, las identidades que se usan para simplificar antes de programar, y las leyes de senos y cosenos para resolver un triangulo del que solo se conocen algunos lados o angulos. El detalle practico mas importante: en C++ sin, cos y tan reciben RADIANTES, y para sacar el angulo de un vector hay que usar atan2(y,x), no atan(y/x), que pierde el cuadrante. Valores e identidades verificados numericamente.

Conversion

```

radianes = grados * pi / 180
grados = radianes * 180 / pi
PI = acos(-1.0) (mas exacto que escribir 3.14159...)
En C++ sin/cos/tan reciben RADIANTES, no grados.

```

Valores exactos (verificados)

grados	0	30	45	60	90
radianes	0	pi/6	pi/4	pi/3	pi/2
sin	0	1/2	r2/2	r3/2	1
cos	1	r3/2	r2/2	1/2	0
tan	0	r3/3	1	r3	inf
r2 = sqrt(2)	~ 1.4142136		r3 = sqrt(3)		~ 1.7320508

Triangulo rectangulo

sin = cateto opuesto / hipotenusa
 cos = cateto adyacente / hipotenusa
 tan = opuesto / adyacente = sin / cos
 Pitagoras : $a^2 + b^2 = c^2$
 Ternas enteras comunes: 3-4-5, 5-12-13, 8-15-17, 7-24-25

Identidades (verificadas)

```

sin^2 t + cos^2 t = 1
sin(2t) = 2 sin t cos t
cos(2t) = cos^2 t - sin^2 t = 1 - 2 sin^2 t = 2 cos^2 t - 1
sin(a+b) = sin a cos b + cos a sin b
cos(a+b) = cos a cos b - sin a sin b
tan(a+b) = (tan a + tan b) / (1 - tan a tan b)
sin(-t) = -sin t      cos(-t) = cos t
sin(pi - t) = sin t   cos(pi - t) = -cos t
periodo de sin y cos : 2pi      periodo de tan : pi

```

Triangulo cualquiera (verificados con el triangulo 3-4-5)

ley de senos : $a/\sin A = b/\sin B = c/\sin C = 2R$
 ley de cosenos : $c^2 = a^2 + b^2 - 2ab \cos C$
 despejando : $\cos C = (a^2 + b^2 - c^2) / (2ab)$
 si $\cos C > 0$ el angulo es agudo, si < 0 es obtuso
 area = $(1/2) * a * b * \sin C$
 radio circunscrito $R = a*b*c / (4 * area)$

radio inscrito $r = \text{area} / s$ con s el semiperimetro
 Los angulos suman 180 grados: sabiendo dos, el tercero sale solo.

atan2 contra atan

atan2(y, x) devuelve el angulo correcto en los cuatro cuadrantes, en el rango $(-\pi, \pi]$. Ejemplo: atan2(-1,-1) = -135 grados. atan(y/x) pierde el cuadrante y se rompe cuando $x = 0$.

Angulo CON SIGNO entre dos vectores:

atan2(cross(a,b), dot(a,b))

Para ordenar puntos alrededor de un centro se puede usar atan2, pero con enteros es mas exacto comparar por cuadrante y luego por producto cruz, sin tocar flotantes.

14.16 Trucos de Bits

C++\Formulario\Trucos de Bits.cpp

Identidades de bits con la situacion concreta en la que sirve cada una, que es la parte que no se deduce: $n \& (n-1)$ es "potencia de 2", $n \& -n$ es el avance del Fenwick, y el xor se usa porque $a^a=0$ deja solo lo que aparece un numero impar de veces. Los valores de los builtins estan verificados corriendo el codigo, no de memoria. La trampa mas cara de esta seccion: $1 \ll 40$ da 0 porque el literal 1 es int; hay que escribir $1LL \ll 40$.

Operar el bit k (k empieza en 0)

```

leer      : (n >> k) & 1
prender  : n |= 1LL << k
apagar   : n &= ~(1LL << k)
voltear  : n ^= 1LL << k
mascara de k bits en 1 : (1LL << k) - 1
OJO: 1 << 40 da 0 porque el 1 es int. Usa 1LL << 40.
    
```

Identidades y para que sirve cada una

```

n & (n-1)  apaga el bit encendido mas bajo
            -> n & (n-1) == 0 dice si n es potencia de 2
            -> contar bits: while (n) { n &= n-1; c++; }
n & -n      deja SOLO el bit mas bajo (lowbit)
            -> es el avance del Fenwick tree: i += i & -i
n | (n+1)  prende el cero mas bajo
n & (n+1)  apaga la cola de unos del final
n ^ (n>>1) codigo Gray: dos consecutivos difieren en 1 bit
            -> recorrer subconjuntos cambiando un elemento
a ^ a = 0  y a ^ 0 = a
            -> el que aparece un numero impar de veces sale solo
            -> xor de rango [l,r] = pre[r] ^ pre[l-1]
a ^ b >= 0 a y b tienen el mismo signo
    
```

Codigo Gray, primeros valores

```

i      0 1 2 3 4 5 6 7
i^(i>>1) 0 1 3 2 6 7 5 4
binario 000 001 011 010 110 111 101 100
    
```

Builtins de GCC (verificados con x = 40 = 101000b)

```

__builtin_popcount(40) = 2  bits en 1
__builtin_ctz(40)      = 3  ceros al final = indice del lowbit
__builtin_clz(40)      = 26 ceros al inicio (sobre 32 bits)
__builtin_parity(7)    = 1  1 si la cantidad de bits es impar
Para long long agrega ll: __builtin_popcountll, __builtin_ctzll
PELIGRO: ctz y clz con n = 0 son comportamiento indefinido.
bit mas alto de n = 31 - clz(n) = floor(log2(n))
siguiente potencia de 2 >= n : 1 << (32 - clz(n-1))
    
```

Subconjuntos

recorrer TODAS las submascaras de m (sin el vacio):

```

for (int s = m; s; s = (s-1) & m)
hacerlo para todas las mascararas cuesta  $3^n$ , no  $4^n$ 
complemento dentro de n bits :  $m \wedge ((1 \ll n) - 1)$ 
recorrer los bits: for (b=0;b<n;b++) if (m >> b & 1) ...
agregar elemento b: m | (1<<b)  quitarlo: m & ~(1<<b)
    
```

Desplazamientos como aritmetica

```

n << 1 = n*2      n << k = n * 2^k
n >> 1 = n/2      solo si n >= 0
    
```

OJO: >> redondea hacia abajo y la division hacia cero:
 $-7 \gg 1 = -4$ pero $-7 / 2 = -3$

Cuando conviene pensar en bitmask

$n \leq 20 \rightarrow$ DP sobre 2^n estados es viable ($\sim 10^6$)
 $n \leq 25 \rightarrow 2^n$ cabe, pero ya va justo
 Estado tipico: dp[mascara] = mejor valor usando ese subconjunto.
 Si el enunciado dice $n \leq 20$, casi siempre es bitmask DP.
 bitset<N> para cribas o DP booleana: 64 veces menos memoria y opera and/or/shift de a 64 bits por vez.

15 - Divide and Conquer

15.1 Conteo de Inversiones con Merge Sort - Frosh Merge Sort

C++\CCPL\FroshMergeSort.cpp

La otra forma de resolver "Frosh Week" (problema F, CCPL), la misma del archivo "frosh" pero sin ninguna estructura de datos. La equivalencia del problema es la misma: el minimo de intercambios de vecinos para ordenar la fila es exactamente el numero de INVERSIONES. LA IDEA, Y ES DE LAS MAS BONITAS QUE HAY: un merge sort ya compara todas las parejas que importan, solo que normalmente tira esa informacion. Al mezclar dos mitades YA ORDENADAS, en el momento en que se toma un elemento de la derecha (R[i]) porque es menor que L[i], ese elemento es menor que L[i] Y QUE TODOS LOS QUE LE SIGUEN EN L, porque L esta ordenada. Como todos ellos estan a la izquierda en el arreglo original, ahi hay $n1 - i$ inversiones de golpe. Esa sola linea, respuesta += $n1 - i$, convierte un ordenamiento en un contador. POR QUE SE PUEDEN SUMAR LAS PARTES: toda inversion (i, j) cae en exactamente uno de tres casos, las dos dentro de la mitad izquierda, las dos dentro de la derecha, o una en cada mitad. Las dos primeras las cuentan las llamadas recursivas y la tercera la cuenta el merge, sin solaparse ni dejar ninguna afuera. De ahi sale el $O(n \log n)$. EL DETALLE QUE PARECE TRIVIAL Y NO LO ES: la comparacion tiene que ser $L[i] \leq R[j]$, con el igual del lado izquierdo. Si se pone $<$, los elementos iguales se toman de la derecha y se cuentan como inversiones, lo que da de mas. En este problema los numeros de estudiante son todos distintos y no se nota, pero en cualquier otro con repetidos si. long long en el contador: con $n = 10^6$ al reves salen casi $5 \cdot 10^{11}$ inversiones. DOS COSAS QUE LE CUESTAN TIEMPO A ESTA VERSION Y NO SE VEN (medido, $n = 10^6$ aleatorio): tal como esta 624 ms agregandole ios::sync_with_stdio(false) 452 ms ademas con UN solo buffer en vez de crear L y R 360 ms Lo primero es que leer 10^6 enteros con cin sin desactivar la sincronizacion con stdio cuesta mas que todo el algoritmo. Lo segundo es que crear dos vectores DENTRO de merge significa ~2 millones de asignaciones de memoria; con un unico buffer del tamano del arreglo reservado una sola vez, se hace una. Asi arreglada es la version mas rapida y la que menos memoria usa (~8 MB contra los ~16 MB del Fenwick, que guarda el arreglo, su copia ordenada y el bit de long long). La de Fenwick se queda en el cuaderno porque se generaliza a otras preguntas; esta solo cuenta inversiones. Verificado contra la version de Fenwick con $n = 10^6$ en tres escenarios (aleatorio, invertido y ya ordenado): identicas, 250188280112 en el aleatorio.

```

1  #include <bits/stdc++.h>
2  #include <vector>
3
4  using namespace std;
5
6  long long respuesta = 0;
7
8  void merge(vector<int>& arr, int left, int mid, int right) {
9      int n1 = mid - left + 1;
10     int n2 = right - mid;
11
12
13
14     vector<int> L(n1);
15     vector<int> R(n2);
16
17     for (int i = 0; i < n1; i++)
18         L[i] = arr[left + i];
19     for (int j = 0; j < n2; j++)
20         R[j] = arr[mid + 1 + j];
21
22
23     int i = 0, j = 0, k = left;
24
25     while (i < n1 && j < n2) {
26         if (L[i] <= R[j]) {
27             arr[k] = L[i];
28             i++;
29         } else {
30             arr[k] = R[j];
31             respuesta += n1 - i;
32             j++;
33         }
34     }
35 }
    
```

```
33     k++;
34 }
35
36 while (i < n1) {
37     arr[k] = L[i];
38     i++;
39     k++;
40 }
41
42 while (j < n2) {
43     arr[k] = R[j];
44     j++;
45     k++;
46 }
47 }
48
49 void mergeSort(vector<int>& arr, int left, int right) {
50     if (left >= right)
51         return;
52
53     int mid = left + (right - left) / 2;
54
55     mergeSort(arr, left, mid);
56     mergeSort(arr, mid + 1, right);
57
58     merge(arr, left, mid, right);
59 }
60
61 int main() {
62
63     int n = 0;
64
65     while(cin >> n){
66
67         respuesta = 0;
68         vector <int> arr;
69
70
71         for(int i = 0; i < n; i++){
72             int num = 0;
73             cin >> num;
74             arr.push_back(num);
75         }
76
77         mergeSort(arr, 0, arr.size() - 1);
78         cout << respuesta << endl;
79     }
80
81     return 0;
82 }
```

16 - Uncategorized

16.1 A - Apple Pie

C++\ICPC\Regionals 2025\A - Apple Pie.cpp

```
1
```

16.2 B - Balanced Balloons

C++\ICPC\Regionals 2025\B - Balanced Balloons.cpp

```
1
```

16.3 B - Binary Dozens

C++\ICPC\2025\B - Binary Dozens.py

```
1 while True:
2     b = input()
3
4     if b == "":
5         break
6
7     n = int(b, 2)
8     print(n % 12)
```

16.4 C - Clean Streets

C++\ICPC\Regionals 2025\C - Clean Streets.cpp

```
1
```

16.5 D - Displaying Decimals

C++\ICPC\Regionals 2025\D - Displaying Decimals.cpp

```
1
```

16.6 E - Emergency Rations

C++\ICPC\Regionals 2025\E - Emergency Rations.cpp

```
1
```

16.7 F - Fuzzy Factorization

C++\ICPC\Regionals 2025\F - Fuzzy Factorization.cpp

```
1
```

16.8 G - Gridoland Power Gauge

C++\ICPC\Regionals 2025\G - Gridoland Power Gauge.cpp

```
1
```

16.9 I - Infiltration Route

C++\ICPC\Regionals 2025\I - Infiltration Route.cpp

```
1
```

16.10 K - Kings Conquest

C++\ICPC\Regionals 2025\K - Kings Conquest.cpp

```
1
```

16.11 L - Lonely Creatures

C++\ICPC\Regionals 2025\L - Lonely Creatures.cpp

```
1
```

17 - CSes: Graph Algorithms

17.1 Building Roads - DSU (Union-Find)

C++\CSes\Graph Algorithms\Building Roads.cpp

DSU con compresion de camino y union por tamano, que juntas dan casi $O(1)$ amortizado por operacion. Se unen todas las aristas dadas y despues se recorre $1..n$ preguntando quien es su propio padre: esos son los representantes, uno por componente. Con c componentes hacen falta $c-1$ aristas nuevas, y basta encadenar los representantes en fila. CUANDO USAR DSU: cuando la pregunta es "estos dos estan conectados" o "cuantas componentes hay" y las aristas solo se AGREGAN, nunca se quitan. Si hay que quitar aristas, DSU no sirve tal cual y toca procesar al revés en el tiempo, o irse a link-cut trees. Para solo contar componentes un BFS o DFS tambien alcanza; DSU gana cuando las consultas se intercalan con las uniones, o cuando es la base de Kruskal.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 vector<long long> padre;
5 vector<long long> tamano;
6
7 long long encontrar(long long x)
8 {
9     if(padre[x] == x)
10     {
11         return x;
12     }
13
14     return padre[x] = encontrar(padre[x]);
15 }
16
17 void unir(long long a, long long b)
18 {
19     a = encontrar(a);
20     b = encontrar(b);
21
22     if(a == b)
23     {
```

```

24     return;
25 }
26
27 if(tamano[a] < tamano[b])
28 {
29     swap(a, b);
30 }
31
32 padre[b] = a;
33 tamano[a] += tamano[b];
34 }
35
36 int main()
37 {
38     ios::sync_with_stdio(false);
39     cin.tie(nullptr);
40
41     long long n, m;
42     cin >> n >> m;
43
44     padre.resize(n + 1);
45     tamano.resize(n + 1, 1);
46
47     for(long long i = 1; i <= n; i++)
48     {
49         padre[i] = i;
50     }
51
52     long long a, b;
53
54     for(long long i = 0; i < m; i++)
55     {
56         cin >> a >> b;
57         unir(a, b);
58     }
59
60     vector<long long> familias;
61
62     for(long long i = 1; i <= n; i++)
63     {
64         if(encontrar(i) == i)
65         {
66             familias.push_back(i);
67         }
68     }
69
70     cout << (familias.size()-1) << '\n';
71
72     for(long long i = 0; i + 1 < familias.size(); i++)
73     {
74         cout << familias[i] << ' ' << familias[i + 1] << '\n';
75     }
76
77     return 0;
78 }

```

17.2 Building Teams - BFS de Bicoloreo (Bipartito)

C++\CSES\Graph Algorithms\Building Teams.cpp

Bicoloreo por BFS: se pinta el primer nodo de 1 y cada vecino del color opuesto. Si en algun momento aparece una arista entre dos nodos del MISMO color, el grafo tiene un ciclo impar y no es bipartito. El for de afuera reinicia el BFS en cada componente, que es lo que se olvida. CUANDO USAR: cada vez que un problema pida partir en DOS grupos con "estos dos no pueden ir juntos", o preguntar si se puede 2-colorear. Bipartito equivale a no tener ciclos de longitud impar. Y es el prerrequisito para matching bipartito: si el problema habla de emparejar dos conjuntos, lo primero es verificar que el grafo sea bipartito.

```

1 #include <iostream>
2 #include <vector>
3 #include <queue>
4
5 using namespace std;
6
7 int main()
8 {
9     long long n, m;

```

```

10
11     cin >> n >> m;
12
13     vector<vector<long long>> vecinos(n + 1);
14
15     long long a, b;
16
17     while(m--)
18     {
19         cin >> a >> b;
20
21         vecinos[a].push_back(b);
22         vecinos[b].push_back(a);
23     }
24
25     vector<long long> valores(n + 1, -1);
26
27     for(long long i = 1; i <= n; i++)
28     {
29         if(valores[i] == -1)
30         {
31             queue<long long> cola;
32
33             valores[i] = 1;
34             cola.push(i);
35
36             while(!cola.empty())
37             {
38                 long long actual = cola.front();
39                 cola.pop();
40
41                 for(long long conexion : vecinos[actual])
42                 {
43                     if(valores[conexion] == -1)
44                     {
45                         if(valores[actual] == 1)
46                         {
47                             valores[conexion] = 2;
48                         }
49                         else
50                         {
51                             valores[conexion] = 1;
52                         }
53
54                         cola.push(conexion);
55                     }
56                     else if(valores[conexion] == valores[actual])
57                     {
58                         cout << "IMPOSSIBLE\n";
59                         return 0;
60                     }
61                 }
62             }
63         }
64     }
65
66     for(long long i = 1; i <= n; i++)
67     {
68         cout << valores[i];
69
70         if(i < n)
71         {
72             cout << ' ';
73         }
74     }
75
76     cout << '\n';
77
78     return 0;
79 }

```

17.3 Counting Rooms - Flood Fill en Grilla (BFS)

C++\CSES\Graph Algorithms\Counting Rooms.cpp

Contar componentes conexas en una grilla. Se barre celda por celda y, al encontrar una libre sin visitar, se lanza un BFS que marca todo su cuarto y se suma uno al total. Marcar la celda como visitada AL ENCOLARLA (no al sacarla) es lo que evita que entre dos veces a la cola. Aqu  se reusa la matriz de entrada como arreglo de visitados poniendo 'X': ahorra memoria y una estructura aparte. CUANDO USAR: cualquier problema de grilla que pregunte cuantas regiones, islas, lagos o manchas hay, o el tama o de la region que contiene a una celda. BFS y DFS dan lo mismo aqu ; se prefiere BFS iterativo porque con 1000x1000 celdas un DFS recursivo puede reventar la pila.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main()
5 {
6     ios::sync_with_stdio(false);
7     cin.tie(nullptr);
8
9     long long n, m;
10    cin >> n >> m;
11
12    vector<string> matriz(n);
13
14    for(long long i = 0; i < n; i++)
15    {
16        cin >> matriz[i];
17    }
18
19    queue<pair<long long, long long>> cuartos;
20
21    long long numCuartos = 0;
22
23    for(long long i = 0; i < n; i++)
24    {
25        for(long long j = 0; j < m; j++)
26        {
27            if(matriz[i][j] == '.')
28            {
29                matriz[i][j] = 'X';
30                cuartos.push({i, j});
31
32                while(!cuartos.empty())
33                {
34                    long long x = cuartos.front().first;
35                    long long y = cuartos.front().second;
36                    cuartos.pop();
37
38                    if(x - 1 >= 0 && matriz[x - 1][y] == '.')
39                    {
40                        matriz[x - 1][y] = 'X';
41                        cuartos.push({x - 1, y});
42                    }
43
44                    if(x + 1 < n && matriz[x + 1][y] == '.')
45                    {
46                        matriz[x + 1][y] = 'X';
47                        cuartos.push({x + 1, y});
48                    }
49
50                    if(y - 1 >= 0 && matriz[x][y - 1] == '.')
51                    {
52                        matriz[x][y - 1] = 'X';
53                        cuartos.push({x, y - 1});
54                    }
55
56                    if(y + 1 < m && matriz[x][y + 1] == '.')
57                    {
58                        matriz[x][y + 1] = 'X';
59                        cuartos.push({x, y + 1});
60                    }
61                }
62
63                numCuartos++;
```

```
64            }
65        }
66    }
67
68    cout << numCuartos;
69
70    return 0;
71 }
```

17.4 Course Schedule - Orden Topologico (Kahn)

C++\CSES\Graph Algorithms\Course Schedule.cpp

Orden topologico con el algoritmo de Kahn: se meten a la cola los nodos con grado de entrada 0 (nadie los obliga a esperar), y cada vez que se saca uno se le "quita" la arista a sus vecinos; el que se queda sin prerrequisitos entra a la cola. El orden en que salen de la cola es un orden topologico valido. LO QUE HACE A KAHN MEJOR QUE EL DFS PARA ESTO: detecta el ciclo gratis. Si hay un ciclo, sus nodos nunca llegan a grado 0 y jamas entran a la cola, asi que al final orden tiene MENOS de n nodos. Esa sola comparacion (orden.size() != n) es todo el chequeo de IMPOSSIBLE. CUANDO USAR: prerrequisitos, dependencias, "en que orden se hacen las tareas", y como primer paso de cualquier DP sobre un DAG (ver "Game Routes" y "Longest Flight Route"). Si piden el orden lexicograficamente menor, se cambia la queue por una priority_queue de minimos y ya.

```
1 #include <iostream>
2 #include <vector>
3 #include <queue>
4
5 using namespace std;
6
7 int main()
8 {
9     ios::sync_with_stdio(false);
10    cin.tie(nullptr);
11
12    long long n, m;
13    cin >> n >> m;
14
15    vector<vector<long long>> conexiones(n);
16    vector<long long> grado(n, 0);
17
18    for(long long i = 0; i < m; i++)
19    {
20        long long a, b;
21        cin >> a >> b;
22
23        a--;
24        b--;
25
26        conexiones[a].push_back(b);
27        grado[b]++;
28    }
29
30    queue<long long> cola;
31
32    for(long long i = 0; i < n; i++)
33    {
34        if(grado[i] == 0)
35        {
36            cola.push(i);
37        }
38    }
39
40    vector<long long> orden;
41
42    while(!cola.empty())
43    {
44        long long actual = cola.front();
45        cola.pop();
46
47        orden.push_back(actual);
48
49        for(long long siguiente : conexiones[actual])
50        {
51            grado[siguiente]--;
52
53            if(grado[siguiente] == 0)
```

```

94         {
95             cola.push(siguiente);
96         }
97     }
98 }
99
100 if(orden.size() != n)
101 {
102     cout << "IMPOSSIBLE\n";
103     return 0;
104 }
105
106 for(long long nodo : orden)
107 {
108     cout << nodo + 1 << ' ';
109 }
110
111 cout << '\n';
112
113 return 0;
114 }

```

17.5 Cycle Finding - Bellman-Ford (Ciclo Negativo)

C++\CSES\Graph Algorithms\Cycle Finding.cpp

Bellman-Ford usado no para hallar distancias sino para DETECTAR un ciclo negativo y devolverlo. Se relaja n veces; si en la ultima pasada todavia hubo una mejora, esa mejora solo pudo venir de un ciclo negativo, y el nodo donde ocurrio queda guardado. EL TRUCO DE LA RECONSTRUCCION, que es lo que hay que copiar: el nodo donde se detecto la mejora NO tiene por que estar dentro del ciclo, puede ser uno colgado mas adelante. La solucion es subir n veces por el arreglo de padres: despues de n saltos es seguro que ya se entro al ciclo, porque la cadena de padres tiene a lo sumo n nodos antes de enrollarse. Desde ahi se sigue a los padres hasta volver al mismo nodo y ese es el ciclo. Aqui las distancias arrancan TODAS en 0, no en infinito, y es a proposito: equivale a poner un origen virtual conectado a todos con peso 0, asi se encuentran ciclos negativos en cualquier componente y no solo en la que alcanza el nodo 1. CUANDO USAR: detectar ciclos negativos, que es lo unico que Dijkstra no puede hacer. Aparece en arbitraje de monedas (con logaritmos negados) y en sistemas de restricciones de diferencias. OJO: usa structured bindings (auto [a, b]), que piden C++17. En CSES compila, pero con un g++ viejo hay que volver a .first y .second.

```

1 #include <iostream>
2 #include <vector>
3
4 using namespace std;
5
6 int main()
7 {
8     ios::sync_with_stdio(false);
9     cin.tie(nullptr);
10
11     long long n, m;
12     cin >> n >> m;
13
14     vector<vector<pair<long long, long long>>> conexion(n);
15
16     for(long long i = 0; i < m; i++)
17     {
18         long long a, b, c;
19         cin >> a >> b >> c;
20
21         conexion[--a].push_back({--b, c});
22     }
23
24     vector<long long> distancia(n, 0);
25     vector<long long> padre(n, -1);
26
27     long long ciclo = -1;
28
29     for(long long i = 0; i < n; i++)
30     {
31         ciclo = -1;
32
33         for(long long a = 0; a < n; a++)
34         {
35             for(auto [b, c] : conexion[a])
36             {
37                 if(distancia[a] + c < distancia[b])

```

```

38             {
39                 distancia[b] = distancia[a] + c;
40                 padre[b] = a;
41                 ciclo = b;
42             }
43         }
44     }
45
46     if(ciclo == -1)
47         break;
48 }
49
50 if(ciclo == -1)
51 {
52     cout << "NO\n";
53     return 0;
54 }
55
56 for(long long i = 0; i < n; i++)
57     ciclo = padre[ciclo];
58
59 vector<long long> respuesta;
60
61 long long actual = ciclo;
62
63 do
64 {
65     respuesta.push_back(actual);
66     actual = padre[actual];
67 }
68 while(actual != ciclo);
69
70 respuesta.push_back(ciclo);
71
72 cout << "YES\n";
73
74 for(long long i = respuesta.size() - 1; i >= 0; i--)
75     cout << respuesta[i] + 1 << ' ';
76
77 cout << '\n';
78
79 return 0;
80 }

```

17.6 Flight Discount - Dijkstra sobre Grafo de Capas

C++\CSES\Graph Algorithms\Flight Discount.cpp

Dijkstra donde el estado no es solo la ciudad sino el par (ciudad, ya use el descuento). Es la tecnica de GRAFO DE CAPAS o grafo producto: se duplica el grafo, en la capa 0 nada esta usado y en la capa 1 el descuento ya se gasto, y las aristas que aplican el descuento son las unicas que cruzan de una capa a la otra. Un Dijkstra normal sobre ese grafo mas grande resuelve todo. POR QUE NO SE PUEDE MAS BARATO: la respuesta no es "el camino minimo y despues descuento la arista mas cara", porque el mejor camino con descuento puede ser otro camino completamente distinto. Hay que decidir los dos a la vez, y para eso se mete la decision dentro del estado. CUANDO USAR: cada vez que al camino minimo se le agrega un recurso limitado. "Puedes usar k teletransportes", "puedes hacer una arista gratis", "hay que llegar con tanque suficiente". La regla es multiplicar los nodos por los estados del recurso: $n*(k+1)$ nodos, mismo Dijkstra. Si el recurso tiene muchos valores el estado explota; ahi ya toca pensar en otra cosa.

```

1 #include <iostream>
2 #include <vector>
3 #include <queue>
4 #include <array>
5 #include <climits>
6
7 using namespace std;
8
9 int main()
10 {
11     ios::sync_with_stdio(false);
12     cin.tie(nullptr);
13
14     long long n, m;
15
16     cin >> n >> m;

```

```

7
8     vector<vector<pair<long long, long long>>> vuelos(n);
9
10    long long a,b,c;
11
12    while(m--)
13    {
14        cin >> a >> b >> c;
15
16        vuelos[--a].push_back({--b,c});
17    }
18
19    //Estados
20    long long USADO = 1, NO_USADO = 0;
21
22    vector<vector<long long>> costoMinimo(n, vector<long long>(2, LLONG_MAX));
23
24    priority_queue<array<long long, 3>,
25                  vector<array<long long, 3>>,
26                  greater<array<long long, 3>>> cola;
27
28    costoMinimo[0][NO_USADO] = 0;
29
30    cola.push({0,0,NO_USADO});
31
32    while(!cola.empty())
33    {
34        long long costo, ciudad, estado;
35
36        costo = cola.top()[0];
37        ciudad = cola.top()[1];
38        estado = cola.top()[2];
39
40        cola.pop();
41
42        if(costoMinimo[ciudad][estado] < costo) continue;
43
44        for(pair<long long, long long> vuelo : vuelos[ciudad])
45        {
46            long long adonde = vuelo.first;
47            long long cuanto = vuelo.second;
48
49            if(costoMinimo[adonde][estado] > costo + cuanto)
50            {
51                costoMinimo[adonde][estado] = costo + cuanto;
52
53                cola.push({costo+cuanto,adonde,estado});
54            }
55
56            if(estado == NO_USADO)
57            {
58                if(costoMinimo[adonde][USADO] > costo + cuanto/2)
59                {
60                    costoMinimo[adonde][USADO] = costo + cuanto/2;
61
62                    cola.push({costo+cuanto/2,adonde,USADO});
63                }
64            }
65        }
66    }
67
68    cout << min(costoMinimo[n-1][NO_USADO], costoMinimo[n-1][USADO]) << endl;
69
70    return 0;
71 }

```

17.7 Flight Routes - K Caminos Mas Cortos

C++ CSDESA/Graph Algorithms/Flight Routes.cpp

Los k caminos mas cortos al nodo n , con la variante de Dijkstra que en vez de UNA distancia por nodo guarda las k mejores en un `priority_queue` de maximos por nodo. Cada nodo se puede sacar de la cola hasta k veces, y ahí esta la clave: la k -esima vez que un nodo sale de la cola, sale con su k -esima distancia mas corta. Por eso el contador `usados[]` y el corte cuando pasa de k . El `priority_queue` de MAXIMOS por nodo es para poder botar el peor de los k que se llevan cuando llega uno mejor. Es un heap acotado a tamaño k . CUANDO USAR: "los k caminos mas cortos", "el k -esimo mas corto", con repeticiones permitidas. Costo $O(k*m*\log)$. Ojo que aquí los caminos pueden repetir nodos; si el problema pide caminos DISJUNTOS o simples, esto no sirve y el problema es muchísimo mas duro.

```

1 #include <iostream>
2 #include <vector>
3 #include <queue>
4 #include <functional>
5
6 using namespace std;
7
8 int main()
9 {
10     ios::sync_with_stdio(false);
11     cin.tie(nullptr);
12
13     long long n, m, k;
14     cin >> n >> m >> k;
15
16     vector<vector<pair<long long, long long>>> conexion(n);
17
18     for(long long i = 0; i < m; i++)
19     {
20         long long a, b, c;
21         cin >> a >> b >> c;
22
23         conexion[--a].push_back({--b, c});
24     }
25
26     vector<priority_queue<long long>> distancia(n);
27     vector<long long> usados(n, 0);
28
29     priority_queue<
30         pair<long long, long long>,
31         vector<pair<long long, long long>>,
32         greater<pair<long long, long long>>
33     > cola;
34
35     distancia[0].push(0);
36     cola.push({0, 0});
37
38     while(!cola.empty())
39     {
40         long long costo = cola.top().first;
41         long long actual = cola.top().second;
42         cola.pop();
43
44         usados[actual]++;
45
46         if(usados[actual] > k)
47             continue;
48
49         for(auto vecino : conexion[actual])
50         {
51             long long siguiente = vecino.first;
52             long long peso = vecino.second;
53
54             long long nuevoCosto = costo + peso;
55
56             if(distancia[siguiente].size() < k)
57             {
58                 distancia[siguiente].push(nuevoCosto);
59                 cola.push({nuevoCosto, siguiente});
60             }
61             else if(nuevoCosto < distancia[siguiente].top())
62             {

```

```

63         distancia[siguiente].pop();
64         distancia[siguiente].push(nuevoCosto);
65         cola.push({nuevoCosto, siguiente});
66     }
67 }
68 }
69
70 vector<long long> respuesta;
71
72 while(!distancia[n - 1].empty())
73 {
74     respuesta.push_back(distancia[n - 1].top());
75     distancia[n - 1].pop();
76 }
77
78 for(long long i = respuesta.size() - 1; i >= 0; i--)
79     cout << respuesta[i] << ' ';
80
81 cout << '\n';
82
83 return 0;
84 }

```

17.8 Game Routes - DP sobre DAG en Orden Topologico

C++\CSES\Graph Algorithms\Game Routes.cpp

Contar caminos de 1 a n en un DAG: $\text{caminos}[v]$ = suma de $\text{caminos}[u]$ sobre las aristas $u \rightarrow v$. La recurrencia es obvia; lo que la hace correcta es el ORDEN: hay que procesar un nodo solo cuando ya se sumaron TODOS sus predecesores, y eso es exactamente lo que garantiza recorrer en orden topologico con Kahn. Cuando $\text{grado}[v]$ llega a 0, $\text{caminos}[v]$ ya esta completo. El detalle de arranque: solo $\text{caminos}[0] = 1$. Los otros nodos que tambien empiezan con grado 0 entran a la cola igual, pero con 0 caminos, asi que empujan ceros y no contaminan la cuenta: solo cuentan los caminos que salen del nodo 1. CUANDO USAR: cualquier "cuantas formas", "camino mas largo", "camino mas barato" sobre un grafo SIN ciclos. En un DAG todas esas son una DP en orden topologico, $O(n + m)$. Con ciclos contar caminos puede ser infinito, y el camino mas largo pasa a ser NP-dif cil.

```

1 #include <iostream>
2 #include <vector>
3 #include <queue>
4
5 using namespace std;
6
7 int main()
8 {
9     ios::sync_with_stdio(false);
10    cin.tie(nullptr);
11
12    const long long MOD = 1000000007;
13
14    long long n, m;
15    cin >> n >> m;
16
17    vector<vector<long long>> conexiones(n);
18    vector<long long> grado(n, 0);
19
20    for(long long i = 0; i < m; i++)
21    {
22        long long a, b;
23        cin >> a >> b;
24
25        a--;
26        b--;
27
28        conexiones[a].push_back(b);
29        grado[b]++;
30    }
31
32    vector<long long> caminos(n, 0);
33    queue<long long> cola;
34
35    caminos[0] = 1;
36
37    for(long long i = 0; i < n; i++)
38    {
39        if(grado[i] == 0)

```

```

40    {
41        cola.push(i);
42    }
43 }
44
45 while(!cola.empty())
46 {
47     long long actual = cola.front();
48     cola.pop();
49
50     for(long long siguiente : conexiones[actual])
51     {
52         caminos[siguiente] += caminos[actual];
53         caminos[siguiente] %= MOD;
54
55         grado[siguiente]--;
56
57         if(grado[siguiente] == 0)
58         {
59             cola.push(siguiente);
60         }
61     }
62 }
63
64 cout << caminos[n - 1] << '\n';
65
66 return 0;
67 }

```

17.9 High Score - Bellman-Ford (Camino Maximo)

C++\CSES\Graph Algorithms\High Score.cpp

Camino de peso MAXIMO con aristas negativas, que es Bellman-Ford con la desigualdad volteada. Se relaja n veces; si en la pasada n todavia mejora algo, hay un ciclo de ganancia infinita. El detalle que hace correcto el problema: un ciclo positivo solo importa si se puede LLEGAR a el desde el origen y desde el SALIR hacia el destino. Por eso los dos BFS previos, uno en el grafo normal desde 1 y otro en el grafo INVERSO desde n : solo los nodos marcados en ambos cuentan para detectar el ciclo. Sin ese filtro, un ciclo positivo en una parte inalcanzable del grafo daria -1 de mentiras. CUANDO USAR BELLMAN-FORD: pesos negativos (Dijkstra ya no sirve) o hay que DETECTAR ciclos negativos, que es su unica ventaja real. Es $O(n^3m)$, mucho mas lento que Dijkstra, asi que con pesos no negativos nunca se usa.

```

1 #include <iostream>
2 #include <vector>
3 #include <queue>
4 #include <climits>
5
6 using namespace std;
7
8 int main()
9 {
10    ios::sync_with_stdio(false);
11    cin.tie(nullptr);
12
13    // INPUT
14    long long n, m;
15    cin >> n >> m;
16
17    vector<vector<pair<long long, long long>>> conexion(n);
18    vector<vector<pair<long long, long long>>> conexionInversa(n);
19
20    for(long long i = 0; i < m; i++)
21    {
22        long long a, b, c;
23        cin >> a >> b >> c;
24
25        a--;
26        b--;
27
28        conexion[a].push_back({b, c});
29        conexionInversa[b].push_back({a, c});
30    }
31
32    vector<bool> de0(n, false);
33    vector<bool> antesN(n, false);

```

```

34
35 // BFS desde 0
36 queue<long long> cola;
37 cola.push(0);
38 de0[0] = true;
39
40 while(!cola.empty())
41 {
42     long long actual = cola.front();
43     cola.pop();
44
45     for(auto vecino : conexion[actual])
46     {
47         long long siguiente = vecino.first;
48
49         if(!de0[siguiente])
50         {
51             de0[siguiente] = true;
52             cola.push(siguiente);
53         }
54     }
55 }
56
57 // BFS desde n-1 usando el grafo inverso
58 cola.push(n - 1);
59 antesN[n - 1] = true;
60
61 while(!cola.empty())
62 {
63     long long actual = cola.front();
64     cola.pop();
65
66     for(auto vecino : conexionInversa[actual])
67     {
68         long long siguiente = vecino.first;
69
70         if(!antesN[siguiente])
71         {
72             antesN[siguiente] = true;
73             cola.push(siguiente);
74         }
75     }
76 }
77
78 // OUTPUT
79 vector<long long> distancia(n, LLONG_MIN);
80 distancia[0] = 0;
81
82 bool ciclo = false;
83
84 for(long long i = 0; i < n; i++)
85 {
86     ciclo = false;
87
88     for(long long a = 0; a < n; a++)
89     {
90         if(!de0[a] || !antesN[a] || distancia[a] == LLONG_MIN)
91             continue;
92
93         for(auto vecino : conexion[a])
94         {
95             long long b = vecino.first;
96             long long c = vecino.second;
97
98             if(!de0[b] || !antesN[b])
99                 continue;
100
101             if(distancia[b] < distancia[a] + c)
102             {
103                 distancia[b] = distancia[a] + c;
104                 ciclo = true;
105             }
106 }

```

```

07     }
08 }
09
10 if(ciclo)
11     cout << -1 << '\n';
12 else
13     cout << distancia[n - 1] << '\n';
14
15 return 0;
16 }

```

17.10 Investigation - Dijkstra con Cuatro Cantidades a la Vez

C++\CSES\Graph Algorithms\Investigation.cpp

Un solo Dijkstra que además del costo mínimo lleva otras tres cosas: cuantos caminos mínimos hay (modulo $1e9+7$), el mínimo de aristas de un camino mínimo y el máximo. LA IDEA ES QUE LAS CUATRO SE PROPAGAN CON LA MISMA REGLA, mirando si la distancia MEJORA o EMPATA: - si mejora: se reinicia todo copiando lo del nodo de donde se viene (caminos, y los conteos de aristas mas uno) - si empata: se ACUMULA, sumando los caminos y quedandose con el mínimo y el máximo de aristas. Ese "si mejora reinicio, si empata acumulo" es el patron general para colgar estadísticas de un Dijkstra, y sirve igual para contar caminos, sumar pesos o llevar cualquier cosa asociativa. El if (peso != distancia[actual]) continue descarta las entradas viejas de la cola, que es lo que permite usar priority_queue sin decrease-key. Sin el se procesarían nodos con distancias obsoletas y los acumulados saldrían mal, no solo lentos. Los caminos se cuentan modulo $1e9+7$ porque pueden ser astronómicos, mientras que las distancias van en long long sin modulo: mezclar las dos cosas es el error clásico aquí. Solo funciona porque los pesos son no negativos, que es lo que garantiza que un nodo ya sacado de la cola no vuelva a mejorar. Con pesos negativos habría que irse a Bellman-Ford y la lógica de acumulación se complica.

```

1 #include <iostream>
2 #include <vector>
3 #include <queue>
4 #include <utility>
5 #include <climits>
6 #include <algorithm>
7
8 using namespace std;
9
10 int main()
11 {
12     ios::sync_with_stdio(false);
13     cin.tie(nullptr);
14
15     const long long MOD = 1000000007;
16
17     long long n, m;
18     cin >> n >> m;
19
20     vector<vector<pair<long long, long long>>> conexiones(n);
21
22     for(long long i = 0; i < m; i++)
23     {
24         long long a, b, c;
25         cin >> a >> b >> c;
26
27         a--;
28         b--;
29
30         conexiones[a].push_back({b, c});
31     }
32
33     vector<long long> caminos(n, 0);
34     vector<long long> distancia(n, LLONG_MAX);
35     vector<long long> minvuelos(n, LLONG_MAX);
36     vector<long long> maxvuelos(n, 0);
37
38     priority_queue<
39         pair<long long, long long>,
40         vector<pair<long long, long long>>,
41         greater<pair<long long, long long>>
42     > cola;
43
44     caminos[0] = 1;
45     distancia[0] = 0;
46     minvuelos[0] = 0;
47

```

```

48 cola.push({0, 0});
49
50 while(!cola.empty())
51 {
52     long long peso = cola.top().first;
53     long long actual = cola.top().second;
54     cola.pop();
55
56     if(peso != distancia[actual])
57     {
58         continue;
59     }
60
61     for(pair<long long, long long> siguiente : conexiones[actual])
62     {
63         long long hacia = siguiente.first;
64         long long cuanto = siguiente.second;
65
66         long long nuevaDistancia = peso + cuanto;
67         long long nuevosVuelos = minvuelos[actual] + 1;
68
69         if(distancia[hacia] > nuevaDistancia)
70         {
71             distancia[hacia] = nuevaDistancia;
72             caminos[hacia] = caminos[actual];
73
74             minvuelos[hacia] = nuevosVuelos;
75             maxvuelos[hacia] = maxvuelos[actual] + 1;
76
77             cola.push({nuevaDistancia, hacia});
78         }
79         else if(distancia[hacia] == nuevaDistancia)
80         {
81             caminos[hacia] += caminos[actual];
82             caminos[hacia] %= MOD;
83
84             minvuelos[hacia] = min(minvuelos[hacia], nuevosVuelos);
85             maxvuelos[hacia] = max(maxvuelos[hacia], maxvuelos[actual] + 1);
86         }
87     }
88 }
89
90 cout << distancia[n - 1] << " "
91      << caminos[n - 1] << " "
92      << minvuelos[n - 1] << " "
93      << maxvuelos[n - 1] << '\n';
94
95 return 0;
96 }

```

17.11 Labyrinth - BFS con Reconstruccion de Camino

C++ \CSES\Graph Algorithms\Labyrinth.cpp

BFS en grilla guardando de donde vino cada celda en la matriz anterior[], y al final se sube desde el destino hasta el origen leyendo esos padres y se invierte. La direccion de cada paso se deduce comparando coordenadas con el padre. CUANDO USAR: cuando el problema no pide solo la DISTANCIA sino el camino en si. La regla es que BFS da el camino m nimo solo si todas las aristas pesan lo mismo; con pesos distintos toca Dijkstra, y con pesos 0 y 1 la version barata es 0-1 BFS con deque. Ojo con marcar: aqu  se marca al encolar, y la celda 'B' se detecta ANTES de pisarla para no perder el destino.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main()
5 {
6     ios::sync_with_stdio(false);
7     cin.tie(nullptr);
8
9     long long n, m;
10    cin >> n >> m;
11
12    vector<string> matriz(n);
13
14    long long Ax = -1;
15    long long Ay = -1;

```

```

6
7    for(long long i = 0; i < n; i++)
8    {
9        cin >> matriz[i];
10
11        if(Ax == -1)
12        {
13            for(long long j = 0; j < m; j++)
14            {
15                if(matriz[i][j] == 'A')
16                {
17                    Ax = i;
18                    Ay = j;
19                    break;
20                }
21            }
22        }
23    }
24
25    vector<vector<pair<long long, long long>>> anterior(
26        n,
27        vector<pair<long long, long long>>(m, {-1, -1}));
28
29    queue<pair<long long, long long>> bfs;
30
31    bfs.push({Ax, Ay});
32
33    bool encontrado = false;
34
35    long long Bx = -1;
36    long long By = -1;
37
38    while(!bfs.empty() && !encontrado)
39    {
40        long long x = bfs.front().first;
41        long long y = bfs.front().second;
42
43        bfs.pop();
44
45        if(x > 0 && (matriz[x - 1][y] == '.' || matriz[x - 1][y] == 'B'))
46        {
47            anterior[x - 1][y] = {x, y};
48
49            if(matriz[x - 1][y] == 'B')
50            {
51                Bx = x - 1;
52                By = y;
53                encontrado = true;
54                break;
55            }
56
57            matriz[x - 1][y] = 'X';
58            bfs.push({x - 1, y});
59        }
60
61        if(x + 1 < n && (matriz[x + 1][y] == '.' || matriz[x + 1][y] == 'B'))
62        {
63            anterior[x + 1][y] = {x, y};
64
65            if(matriz[x + 1][y] == 'B')
66            {
67                Bx = x + 1;
68                By = y;
69                encontrado = true;
70                break;
71            }
72
73            matriz[x + 1][y] = 'X';
74            bfs.push({x + 1, y});
75        }
76
77        if(y > 0 && (matriz[x][y - 1] == '.' || matriz[x][y - 1] == 'B'))

```

```

89 {
90     anterior[x][y - 1] = {x, y};
91
92     if(matriz[x][y - 1] == 'B')
93     {
94         Bx = x;
95         By = y - 1;
96         encontrado = true;
97         break;
98     }
99
100     matriz[x][y - 1] = 'X';
101     bfs.push({x, y - 1});
102 }
103
104 if(y + 1 < m && (matriz[x][y + 1] == '.' || matriz[x][y + 1] == 'B'))
105 {
106     anterior[x][y + 1] = {x, y};
107
108     if(matriz[x][y + 1] == 'B')
109     {
110         Bx = x;
111         By = y + 1;
112         encontrado = true;
113         break;
114     }
115
116     matriz[x][y + 1] = 'X';
117     bfs.push({x, y + 1});
118 }
119 }
120
121 if(!encontrado)
122 {
123     cout << "NO\n";
124     return 0;
125 }
126
127 string recorrido = "";
128
129 long long x = Bx;
130 long long y = By;
131
132 while(x != Ax || y != Ay)
133 {
134     long long anteriorX = anterior[x][y].first;
135     long long anteriorY = anterior[x][y].second;
136
137     if(x - anteriorX > 0)
138     {
139         recorrido += "D";
140     }
141     else if(x - anteriorX < 0)
142     {
143         recorrido += "U";
144     }
145     else if(y - anteriorY > 0)
146     {
147         recorrido += "R";
148     }
149     else
150     {
151         recorrido += "L";
152     }
153
154     x = anteriorX;
155     y = anteriorY;
156 }
157
158 reverse(recorrido.begin(), recorrido.end());
159
160 cout << "YES\n";
161 cout << recorrido.size() << '\n';

```

```

62 cout << recorrido << '\n';
63
64 return 0;
65 }

```

17.12 Longest Flight Route - Camino Mas Largo en DAG (DFS con Memo)

C++\CSES\Graph Algorithms\Longest Flight Route.cpp

Camino con mas nodos de 1 a n en un DAG, con DFS memoizado: $mejor[u] = 1 + \max \text{ de } mejor[\text{hijo}]$. En un grafo cualquiera el camino mas largo es NP-difícil; aquí es lineal SOLO porque no hay ciclos, y eso hay que tenerlo presente antes de intentarlo en otro problema. EL TRUCO DEL CERO: $mejor[u] = 0$ significa "desde u no se llega a n". Por eso un hijo solo se toma si $\text{camino} > 0$; así los callejones sin salida no contaminan el maximo, y si al final $mejor[0]$ vale 0 la respuesta es IMPOSSIBLE. El arreglo siguiente[] guarda por donde se fue el maximo, y con eso se reconstruye la ruta. OJO CON LA RECURSION: con un camino de 10^5 nodos la pila de llamadas llega a esa profundidad. En CSES pasa, pero en un juez con pila chica puede dar error de ejecución. La version sin riesgo es la misma DP recorriendo en orden topologico con Kahn, como "Game Routes".

```

1 #include <iostream>
2 #include <vector>
3 #include <algorithm>
4
5 using namespace std;
6
7 long long n, m;
8
9 vector<vector<long long>> conexiones;
10 vector<long long> mejor;
11 vector<long long> siguiente;
12
13 long long dfs(long long actual)
14 {
15     if(actual == n - 1)
16         return 1;
17
18     if(mejor[actual] != -1)
19         return mejor[actual];
20
21     mejor[actual] = 0;
22
23     for(long long hijo : conexiones[actual])
24     {
25         long long camino = dfs(hijo);
26
27         if(camino > 0 && camino + 1 > mejor[actual])
28         {
29             mejor[actual] = camino + 1;
30             siguiente[actual] = hijo;
31         }
32     }
33
34     return mejor[actual];
35 }
36
37 int main()
38 {
39     ios::sync_with_stdio(false);
40     cin.tie(nullptr);
41
42     cin >> n >> m;
43
44     conexiones.resize(n);
45
46     for(long long i = 0; i < m; i++)
47     {
48         long long a, b;
49         cin >> a >> b;
50
51         conexiones[a - 1].push_back(b - 1);
52     }
53
54     mejor.assign(n, -1);
55     siguiente.assign(n, -1);
56
57     long long largo = dfs(0);

```

```

58
59     if(largo == 0)
60     {
61         cout << "IMPOSSIBLE\n";
62         return 0;
63     }
64
65     cout << largo << '\n';
66
67     long long actual = 0;
68
69     while(actual != -1)
70     {
71         cout << actual + 1 << ' ';
72
73         if(actual == n - 1)
74             break;
75
76         actual = siguiente[actual];
77     }
78
79     cout << '\n';
80
81     return 0;
82 }

```

17.13 Message Route - BFS de Camino Mínimo con Reconstrucción

C++\CS2\Graph Algorithms\Message Route.cpp

Mismo patrón que Labyrinth pero en grafo general: BFS desde n hacia 1 guardando siguiente[] y la distancia en cuantos[]. Arrancar del destino tiene su gracia: los punteros ya quedan apuntando hacia adelante y el camino se imprime directo, sin invertir nada. El cuantos[] hace doble trabajo, de contador de pasos y de marca de visitado (0 = no visto), que es un truco común para no llevar dos arreglos. CUANDO USAR: grafo sin pesos y hay que devolver el camino, no solo su largo. Si el grafo tuviera pesos, esto mismo pero con Dijkstra.

```

1  #include <iostream>
2  #include <vector>
3  #include <queue>
4
5  using namespace std;
6
7  int main()
8  {
9      ios::sync_with_stdio(false);
10     cin.tie(nullptr);
11
12     long long n, m;
13
14     cin >> n >> m;
15
16     vector<vector<long long>> vecinos(n + 1);
17
18     long long a, b;
19
20     while(m--)
21     {
22         cin >> a >> b;
23
24         vecinos[a].push_back(b);
25         vecinos[b].push_back(a);
26     }
27
28     vector<long long> siguiente(n + 1, -1);
29     vector<long long> cuantos(n + 1, 0);
30
31     queue<long long> cola;
32
33     cola.push(n);
34     cuantos[n] = 1;
35
36     while(!cola.empty())
37     {
38         long long actual = cola.front();
39         cola.pop();

```

```

40
41         if(actual == 1)
42         {
43             break;
44         }
45
46         for(long long vecino : vecinos[actual])
47         {
48             if(cuantos[vecino] == 0)
49             {
50                 siguiente[vecino] = actual;
51                 cuantos[vecino] = cuantos[actual] + 1;
52                 cola.push(vecino);
53             }
54         }
55     }
56
57     if(cuantos[1] == 0)
58     {
59         cout << "IMPOSSIBLE\n";
60         return 0;
61     }
62
63     cout << cuantos[1] << '\n';
64
65     long long actual = 1;
66
67     while(actual != n)
68     {
69         cout << actual << ' ';
70         actual = siguiente[actual];
71     }
72
73     cout << n << '\n';
74
75     return 0;
76 }

```

17.14 Monsters - BFS Multifuente + BFS de Escape

C++\CS2\Graph Algorithms\Monsters.cpp

Dos BFS encadenados, y el primero es el truco que vale aprender: un BFS MULTIFUENTE, con TODOS los monstruos encolados con distancia 0 a la vez. Eso da, en una sola pasada $O(n \cdot m)$, la distancia de cada celda al monstruo más cercano, que es lo mismo que correr un BFS por monstruo pero sin pagar el costo de cada uno. Después va el BFS del jugador, que solo entra a una celda si llega Estrictamente antes que el monstruo más cercano. Guardando la dirección de llegada se reconstruye la salida. CUANDO USAR EL MULTIFUENTE: siempre que la pregunta sea "distancia al más cercano de un conjunto" en vez de "distancia desde un punto". Aparece en incendios que se propagan, celdas contaminadas, o el típico "cuanto tarda en llenarse todo".

```

1  #include <iostream>
2  #include <vector>
3  #include <queue>
4  #include <algorithm>
5
6  using namespace std;
7
8  int main()
9  {
10     ios::sync_with_stdio(false);
11     cin.tie(nullptr);
12
13     long long n, m;
14
15     cin >> n >> m;
16
17     vector<string> matriz(n);
18
19     vector<pair<long long, long long>> monstruos;
20
21     long long Ax = -1;
22     long long Ay = -1;
23
24     for(long long i = 0; i < n; i++)
25     {
26         cin >> matriz[i];

```

```

27
28     for(long long j = 0; j < m; j++)
29     {
30         if(matriz[i][j] == 'M')
31         {
32             monstruos.push_back({i, j});
33         }
34         else if(matriz[i][j] == 'A')
35         {
36             Ax = i;
37             Ay = j;
38         }
39     }
40 }
41
42 if(Ax == 0 || Ax == n - 1 || Ay == 0 || Ay == m - 1)
43 {
44     cout << "YES\n";
45     cout << "0\n";
46     cout << '\n';
47     return 0;
48 }
49
50 long long dx[4] = {-1, 1, 0, 0};
51 long long dy[4] = {0, 0, -1, 1};
52
53 vector<vector<long long>> distanciaM(n, vector<long long>(m, -1));
54
55 queue<pair<long long, long long>> cola;
56
57 for(pair<long long, long long> monstruo : monstruos)
58 {
59     distanciaM[monstruo.first][monstruo.second] = 0;
60     cola.push(monstruo);
61 }
62
63 while(!cola.empty())
64 {
65     long long x = cola.front().first;
66     long long y = cola.front().second;
67     cola.pop();
68
69     for(long long k = 0; k < 4; k++)
70     {
71         long long nx = x + dx[k];
72         long long ny = y + dy[k];
73
74         if(nx < 0 || nx >= n || ny < 0 || ny >= m)
75         {
76             continue;
77         }
78
79         if(matriz[nx][ny] == '#')
80         {
81             continue;
82         }
83
84         if(distanciaM[nx][ny] == -1)
85         {
86             distanciaM[nx][ny] = distanciaM[x][y] + 1;
87             cola.push({nx, ny});
88         }
89     }
90 }
91
92 vector<vector<long long>> distanciaA(n, vector<long long>(m, -1));
93 vector<vector<char>> direccion(n, vector<char>(m));
94
95 distanciaA[Ax][Ay] = 0;
96 cola.push({Ax, Ay});
97
98 long long salidaX = -1;
99 long long salidaY = -1;

```

```

100
101 while(!cola.empty())
102 {
103     long long x = cola.front().first;
104     long long y = cola.front().second;
105     cola.pop();
106
107     for(long long k = 0; k < 4; k++)
108     {
109         long long nx = x + dx[k];
110         long long ny = y + dy[k];
111
112         if(nx < 0 || nx >= n || ny < 0 || ny >= m)
113         {
114             continue;
115         }
116
117         if(matriz[nx][ny] == '#')
118         {
119             continue;
120         }
121
122         if(distanciaA[nx][ny] != -1)
123         {
124             continue;
125         }
126
127         long long nuevaDistancia = distanciaA[x][y] + 1;
128
129         if(distanciaM[nx][ny] != -1 &&
130            nuevaDistancia >= distanciaM[nx][ny])
131         {
132             continue;
133         }
134
135         distanciaA[nx][ny] = nuevaDistancia;
136
137         if(k == 0)
138         {
139             direccion[nx][ny] = 'U';
140         }
141         else if(k == 1)
142         {
143             direccion[nx][ny] = 'D';
144         }
145         else if(k == 2)
146         {
147             direccion[nx][ny] = 'L';
148         }
149         else
150         {
151             direccion[nx][ny] = 'R';
152         }
153
154         if(nx == 0 || nx == n - 1 || ny == 0 || ny == m - 1)
155         {
156             salidaX = nx;
157             salidaY = ny;
158             break;
159         }
160
161         cola.push({nx, ny});
162     }
163
164     if(salidaX != -1)
165     {
166         break;
167     }
168 }
169
170 if(salidaX == -1)
171 {
172     cout << "NO\n";

```

```

173     return 0;
174 }
175
176 string recorridoFinal;
177
178 long long x = salidaX;
179 long long y = salidaY;
180
181 while(x != Ax || y != Ay)
182 {
183     char movimiento = direccion[x][y];
184
185     recorridoFinal += movimiento;
186
187     if(movimiento == 'R')
188     {
189         y--;
190     }
191     else if(movimiento == 'L')
192     {
193         y++;
194     }
195     else if(movimiento == 'D')
196     {
197         x--;
198     }
199     else
200     {
201         x++;
202     }
203 }
204
205 reverse(recorridoFinal.begin(), recorridoFinal.end());
206
207 cout << "YES\n";
208 cout << recorridoFinal.size() << '\n';
209 cout << recorridoFinal << '\n';
210
211 return 0;
212 }

```

17.15 Planetqueries - Binary Lifting sobre Grafo Funcional

C++\CSES\Graph Algorithms\planetqueries.cpp

Cada planeta tiene EXACTAMENTE un teletransporte de salida, así que la estructura es un grafo funcional: una función $x \rightarrow siguiente[x]$. Las consultas piden donde se termina tras k saltos, con k hasta 10^9 . BINARY LIFTING: $siguiente[j][i]$ es a donde se llega desde i con 2^j saltos, y se construye con $siguiente[j][i] = siguiente[j-1][siguiente[j-1][i]]$ (dar 2^j saltos es dar 2^{j-1} dos veces). Después cada consulta descompone k en binario y aplica los saltos de los bits encendidos: $O(\log k)$ por consulta en vez de caminar k pasos. POR QUE NO HACE FALTA DETECTAR CICLOS: aunque el grafo funcional siempre acaba en un ciclo, el doubling lo atraviesa sin enterarse. La alternativa (hallar el ciclo y usar modulo) es mas rápida en memoria pero mucho mas fácil de romper en los bordes. $LOG = 31$ cubre k hasta $2^{31} - 1$, suficiente para 10^9 . Si k fuera mayor hay que subirlo, y ese es el error silencioso de esta plantilla: con un LOG corto los bits altos de k se ignoran y la respuesta sale mal sin ningun aviso. La tabla ocupa $LOG * n$; con $n = 2 * 10^5$ son unos 6 millones de long long, así que si aprieta la memoria conviene guardarla en int. Es el mismo mecanismo de "Visible Building Queries" y "Movie Festival Queries" de este cuaderno: si un puntero define una función, sus potencias se precalculan con doubling.

```

1 #include <iostream>
2 #include <vector>
3
4 using namespace std;
5
6 int main()
7 {
8     ios::sync_with_stdio(false);
9     cin.tie(nullptr);
10
11     long long n, q;
12     cin >> n >> q;
13
14     const long long LOG = 31;
15
16     vector<vector<long long>> siguiente(LOG, vector<long long>(n));
17

```

```

8     for(long long i = 0; i < n; i++)
9     {
20         cin >> siguiente[0][i];
21         siguiente[0][i]--;
22     }
23
24     for(long long j = 1; j < LOG; j++)
25     {
26         for(long long i = 0; i < n; i++)
27         {
28             siguiente[j][i] = siguiente[j - 1][siguiente[j - 1][i]];
29         }
30     }
31
32     while(q--)
33     {
34         long long x, k;
35         cin >> x >> k;
36
37         x--;
38
39         for(long long j = 0; j < LOG; j++)
40         {
41             if(k & (1LL << j))
42             {
43                 x = siguiente[j][x];
44             }
45         }
46
47         cout << x + 1 << '\n';
48     }
49
50     return 0;
51 }

```

17.16 Round Trip - Detección de Ciclo con BFS

C++\CSES\Graph Algorithms\Round Trip.cpp

Se busca un ciclo en grafo no dirigido: si durante el recorrido aparece una arista hacia un nodo YA visitado que no es el padre inmediato, ahí se cierra un ciclo, y se reconstruye subiendo por los padres desde los dos extremos de esa arista. La trampa del no dirigido es justamente excluir al padre: si no, toda arista $u-v$ se ve como ciclo de largo 2 apenas se vuelve a mirar. CUANDO USAR: "existe un ciclo", "hay que devolverlo". En grafo DIRIGIDO esto no aplica: ahí se usa DFS con tres colores (blanco, gris, negro) y el ciclo es una arista hacia un gris.

```

1 #include <iostream>
2 #include <vector>
3 #include <queue>
4 #include <algorithm>
5
6 using namespace std;
7
8 int main()
9 {
10     ios::sync_with_stdio(false);
11     cin.tie(nullptr);
12
13     long long n, m;
14
15     cin >> n >> m;
16
17     vector<vector<long long>> matriz(n + 1);
18
19     long long a, b;
20
21     while(m--)
22     {
23         cin >> a >> b;
24
25         matriz[a].push_back(b);
26         matriz[b].push_back(a);
27     }
28
29     vector<long long> anterior(n + 1, -1);
30     vector<bool> recorrido(n + 1, false);

```

```

31
32 for(long long i = 1; i <= n; i++)
33 {
34     if(recorrido[i]) continue;
35
36     queue<long long> cola;
37
38     recorrido[i] = true;
39     cola.push(i);
40
41     while(!cola.empty())
42     {
43         long long x = cola.front();
44         cola.pop();
45
46         for(long long z : matriz[x])
47         {
48             if(!recorrido[z])
49             {
50                 recorrido[z] = true;
51                 anterior[z] = x;
52                 cola.push(z);
53             }
54             else if(anterior[x] != z)
55             {
56                 vector<long long> recorridoX;
57                 vector<long long> recorridoZ;
58
59                 long long actual = x;
60
61                 while(actual != -1)
62                 {
63                     recorridoX.push_back(actual);
64
65                     if(actual == i)
66                     {
67                         break;
68                     }
69
70                     actual = anterior[actual];
71                 }
72
73                 actual = z;
74
75                 while(actual != -1)
76                 {
77                     recorridoZ.push_back(actual);
78
79                     if(actual == i)
80                     {
81                         break;
82                     }
83
84                     actual = anterior[actual];
85                 }
86
87                 reverse(recorridoX.begin(), recorridoX.end());
88                 reverse(recorridoZ.begin(), recorridoZ.end());
89
90                 long long posicion = 0;
91
92                 while(posicion < recorridoX.size() &&
93                     posicion < recorridoZ.size() &&
94                     recorridoX[posicion] == recorridoZ[posicion])
95                 {
96                     posicion++;
97                 }
98
99                 posicion--;
100
101                 cout << recorridoX.size() + recorridoZ.size()
102                     - 2 * posicion << '\n';
103

```

```

104         for(long long j = posicion; j < recorridoX.size(); j++)
105         {
106             cout << recorridoX[j] << ' ';
107         }
108
109         for(long long j = recorridoZ.size() - 1; j >= posicion; j--)
110         {
111             cout << recorridoZ[j] << ' ';
112         }
113
114         cout << '\n';
115
116         return 0;
117     }
118 }
119 }
120 }
121
122 cout << "IMPOSSIBLE\n";
123
124 return 0;
125 }

```

17.17 Round Trip (Stack) - Deteccion de Ciclo con Pila (DFS Iterativo)

C++(CSES)Graph Algorithms(Round Trip (Stack).cpp

El mismo problema que Round Trip pero cambiando la cola por una PILA, o sea DFS en vez de BFS, escrito de forma iterativa. Para detectar ciclos da igual cual de los dos se use, y esa es la enseñanza: la estructura es lo unico que cambia entre BFS y DFS. Vale tener las dos versiones porque a veces el problema pide el ciclo mas corto (BFS ayuda) y a veces cualquiera (DFS es mas directo). POR QUE ITERATIVO: con n hasta 10^5 un DFS recursivo se arriesga a stack overflow. Pasar la recursion a una pila explicita es la salida estandar, y es lo que hace este archivo.

```

1 #include <iostream>
2 #include <vector>
3 #include <stack>
4 #include <algorithm>
5
6 using namespace std;
7
8 int main()
9 {
10     ios::sync_with_stdio(false);
11     cin.tie(nullptr);
12
13     long long n, m;
14
15     cin >> n >> m;
16
17     vector<vector<long long>> matriz(n + 1);
18
19     long long a, b;
20
21     while(m--)
22     {
23         cin >> a >> b;
24
25         matriz[a].push_back(b);
26         matriz[b].push_back(a);
27     }
28
29     vector<long long> anterior(n + 1, -1);
30     vector<bool> recorrido(n + 1, false);
31
32     for(long long i = 1; i <= n; i++)
33     {
34         if(recorrido[i]) continue;
35
36         stack<long long> pila;
37
38         recorrido[i] = true;
39         pila.push(i);
40
41         while(!pila.empty())
42         {

```

```

43     long long x = pila.top();
44     pila.pop();
45
46     for(long long z : matriz[x])
47     {
48         if(!recorrido[z])
49         {
50             recorrido[z] = true;
51             anterior[z] = x;
52             pila.push(z);
53         }
54         else if(anterior[x] != z)
55         {
56             vector<long long> recorridoX;
57             vector<long long> recorridoZ;
58
59             long long actual = x;
60
61             while(actual != -1)
62             {
63                 recorridoX.push_back(actual);
64
65                 if(actual == i)
66                 {
67                     break;
68                 }
69
70                 actual = anterior[actual];
71             }
72
73             actual = z;
74
75             while(actual != -1)
76             {
77                 recorridoZ.push_back(actual);
78
79                 if(actual == i)
80                 {
81                     break;
82                 }
83
84                 actual = anterior[actual];
85             }
86
87             reverse(recorridoX.begin(), recorridoX.end());
88             reverse(recorridoZ.begin(), recorridoZ.end());
89
90             long long posicion = 0;
91
92             while(posicion < recorridoX.size() &&
93                   posicion < recorridoZ.size() &&
94                   recorridoX[posicion] == recorridoZ[posicion])
95             {
96                 posicion++;
97             }
98             posicion--;
99
100            cout << recorridoX.size() + recorridoZ.size()
101                << " 2 * posicion << '\n';
102
103            for(long long j = posicion; j < recorridoX.size(); j++)
104            {
105                cout << recorridoX[j] << ' ';
106            }
107
108            for(long long j = recorridoZ.size() - 1; j >= posicion; j--)
109            {
110                cout << recorridoZ[j] << ' ';
111            }
112
113            cout << '\n';
114
115

```

```

116         return 0;
117     }
118 }
119 }
120 }
121
122 cout << "IMPOSSIBLE\n";
123
124 return 0;
125 }

```

17.18 Round Trip II - Ciclo en Grafo Dirigido (DFS de Tres Colores)

C++\CSES\Graph Algorithms\Round Trip II.cpp

Encontrar un ciclo en un grafo DIRIGIDO con DFS de tres colores: 0 = sin visitar, 1 = en la pila (gris), 2 = terminado (negro). Hay ciclo si y solo si aparece una arista hacia un nodo GRIS, porque ese nodo es un ancestro en la rama actual. Una arista hacia un negro NO es ciclo: ese nodo ya se exploró entero por otra rama. Confundir gris con negro es EL error clásico aquí, y por eso el "visitado sí/no" de los grafos no dirigidos no sirve. LO QUE HACE BIEN ESTE DFS ITERATIVO, y conviene copiarlo tal cual: la pila guarda el par (nodo, índice de la siguiente arista por mirar). Así un nodo sigue gris hasta que se revisaron TODAS sus aristas, y solo ahí se vuelve negro. El DFS iterativo ingenuo (sacar el nodo de la pila apenas se visita, como en "Round Trip (Stack)") no sirve para colores, porque pierde el momento en que el nodo termina. El ciclo se reconstruye subiendo por anterior[] desde x hasta z, e invirtiendo. CUANDO USAR: detectar dependencias circulares, o como chequeo de que un grafo es un DAG antes de hacerle DP. Si solo hace falta saber SI hay ciclo, Kahn (ver "Course Schedule") es más corto.

```

1 #include <iostream>
2 #include <vector>
3 #include <stack>
4 #include <algorithm>
5
6 using namespace std;
7
8 int main()
9 {
10     ios::sync_with_stdio(false);
11     cin.tie(nullptr);
12
13     long long n, m;
14
15     cin >> n >> m;
16
17     vector<vector<long long>>> matriz(n + 1);
18
19     long long a, b;
20
21     while (m--)
22     {
23         cin >> a >> b;
24         matriz[a].push_back(b);
25     }
26
27     vector<long long> anterior(n + 1, -1);
28     vector<long long> estado(n + 1, 0);
29
30     for (long long i = 1; i <= n; i++)
31     {
32         if (estado[i] != 0) continue;
33
34         stack<pair<long long, long long>> pila;
35
36         estado[i] = 1;
37         pila.push({i, 0});
38
39         while (!pila.empty())
40         {
41             long long x = pila.top().first;
42             long long posicion = pila.top().second;
43
44             if (posicion == matriz[x].size())
45             {
46                 estado[x] = 2;
47                 pila.pop();
48                 continue;
49             }

```

```

90
91     long long z = matriz[x][posicion];
92
93     pila.top().second++;
94
95     if (estado[z] == 0)
96     {
97         anterior[z] = x;
98         estado[z] = 1;
99         pila.push({z, 0});
100     }
101     else if (estado[z] == 1)
102     {
103         vector<long long> ciclo;
104
105         ciclo.push_back(z);
106
107         long long actual = x;
108
109         while (actual != z)
110         {
111             ciclo.push_back(actual);
112             actual = anterior[actual];
113         }
114
115         ciclo.push_back(z);
116         reverse(ciclo.begin(), ciclo.end());
117
118         cout << ciclo.size() << '\n';
119
120         for (long long ciudad : ciclo)
121             cout << ciudad << ' ';
122
123         cout << '\n';
124
125         return 0;
126     }
127 }
128 }
129
130 cout << "IMPOSSIBLE\n";
131
132 return 0;
133 }

```

17.19 Shortest Routes I - Dijkstra con Priority Queue

C++\CSES\Graph Algorithms\Shortest Routes I.cpp

Dijkstra clasico en $O(m \log n)$ con priority_queue de minimos. Los dos detalles que importan: la cola guarda el par (distancia, nodo) en ese orden para que ordene por distancia, y el $\text{if}(\text{dist} > \text{distancia}[\text{actual}])$ continue descarta las entradas viejas que quedaron en la cola cuando un nodo mejoro su distancia. Sin ese filtro no esta mal, pero se procesa basura. CUANDO USAR: camino minimo desde UN origen con pesos NO NEGATIVOS. Con un peso negativo Dijkstra da respuestas mal y hay que irse a Bellman-Ford. Si todos los pesos son iguales, BFS hace lo mismo mas rapido y sin cola de prioridad. Ojo con LLONG_MAX como infinito: si se suma algo se desborda. Aqui no pasa porque solo se compara, pero es el error clasico de esta plantilla.

```

1 #include <iostream>
2 #include <vector>
3 #include <queue>
4 #include <climits>
5
6 using namespace std;
7
8 int main()
9 {
10     // INPUT
11     long long n, m;
12     cin >> n >> m;
13
14     vector<vector<pair<long long, long long>>> matriz(n + 1);
15
16     for(long long i = 0; i < m; i++)
17     {
18         long long a, b, c;
19         cin >> a >> b >> c;

```

```

20
21         matriz[a].push_back({b, c});
22     }
23
24     // OUTPUT
25     vector<long long> distancia(n + 1, LLONG_MAX);
26     priority_queue<pair<long long, long long>,
27                 vector<pair<long long, long long>>,
28                 greater<pair<long long, long long>>> cola;
29
30     distancia[1] = 0;
31     cola.push({0, 1});
32
33     while(!cola.empty())
34     {
35         long long dist = cola.top().first;
36         long long actual = cola.top().second;
37         cola.pop();
38
39         if(dist > distancia[actual])
40             continue;
41
42         for(auto vecino : matriz[actual])
43         {
44             long long siguiente = vecino.first;
45             long long peso = vecino.second;
46
47             if(distancia[actual] + peso < distancia[siguiente])
48             {
49                 distancia[siguiente] = distancia[actual] + peso;
50                 cola.push({distancia[siguiente], siguiente});
51             }
52         }
53     }
54
55     for(long long i = 1; i < n+1; i++)
56     {
57         cout << distancia[i];
58         if(i < n) cout << " ";
59     }
60
61     cout << endl;
62
63     return 0;
64 }

```

17.20 Shortest Routes II - Dijkstra desde Cada Nodo

C++\CSES\Graph Algorithms\Shortest Routes II.cpp

Distancias entre TODOS los pares resueltas con n Dijkstras, uno por nodo, en vez del Floyd-Warshall que suele ser la respuesta esperada. Trae un atajo: si un nodo tiene grado 1, su unico camino al resto pasa por su vecino, asi que se copia la fila del vecino sumando el peso de esa arista y se ahorra un Dijkstra completo. CUANDO CADA UNO: Floyd-Warshall es $O(n^3)$ y se escribe en tres lineas, asi que con $n \leq 500$ es lo que uno hace. n Dijkstras es $O(n^2 m \log n)$, que gana cuando el grafo es DISPERSO (m del orden de n), y pierde feo si es denso. Tambien hay que recordar quedarse con la arista mas barata entre cada par y que Floyd aguanta pesos negativos sin ciclos negativos, mientras Dijkstra no.

```

1 #include <iostream>
2 #include <vector>
3 #include <queue>
4 #include <climits>
5
6 using namespace std;
7
8 int main()
9 {
10     ios::sync_with_stdio(false);
11     cin.tie(nullptr);
12     // INPUT
13     long long n, m, q;
14     cin >> n >> m >> q;
15
16     vector<vector<pair<long long, long long>>> conexion(n);
17
18     vector<vector<long long>> conexiindirecta(

```

```

18     n, vector<long long>(n, LLONG_MAX));
19
20
21     for(long long i = 0; i < m; i++)
22     {
23         long long a, b, c;
24         cin >> a >> b >> c;
25
26         a--;
27         b--;
28
29         if(conexiondirecta[a][b] > c)
30         {
31             conexiondirecta[a][b] = c;
32             conexiondirecta[b][a] = c;
33         }
34     }
35
36     for(long long i = 0; i < n; i++)
37     {
38         for(long long j = i + 1; j < n; j++)
39         {
40             if(conexiondirecta[i][j] != LLONG_MAX)
41             {
42                 conexion[i].push_back({j, conexiondirecta[i][j]});
43                 conexion[j].push_back({i, conexiondirecta[i][j]});
44             }
45         }
46     }
47
48     // OUTPUT
49     vector<vector<long long>> camino(n, vector<long long>(n, -1));
50     vector<bool> dijkstrados(n, false);
51
52     for(long long x = 0; x < n; x++)
53     {
54         if(conexion[x].size() == 1)
55         {
56             long long z = conexion[x][0].first;
57             long long distanciaXZ = conexion[x][0].second;
58
59             if(!dijkstrados[z])
60             {
61                 vector<long long> distancia(n, LLONG_MAX);
62
63                 priority_queue<
64                     pair<long long, long long>,
65                     vector<pair<long long, long long>>,
66                     greater<pair<long long, long long>>
67                 > cola;
68
69                 distancia[z] = 0;
70                 cola.push({0, z});
71
72                 while(!cola.empty())
73                 {
74                     long long distanciaActual = cola.top().first;
75                     long long actual = cola.top().second;
76                     cola.pop();
77
78                     if(distanciaActual > distancia[actual])
79                         continue;
80
81                     for(auto vecino : conexion[actual])
82                     {
83                         long long siguiente = vecino.first;
84                         long long peso = vecino.second;
85
86                         if(distancia[actual] + peso < distancia[siguiente])
87                         {
88                             distancia[siguiente] = distancia[actual] + peso;
89                             cola.push({distancia[siguiente], siguiente});
90                         }
91                     }

```

```

92         }
93     }
94     for(long long i = 0; i < n; i++)
95     {
96         if(distancia[i] != LLONG_MAX)
97             camino[z][i] = distancia[i];
98     }
99
100     dijkstrados[z] = true;
101 }
102
103 for(long long i = 0; i < n; i++)
104 {
105     if(camino[z][i] != -1)
106         camino[x][i] = camino[z][i] + distanciaXZ;
107 }
108
109 camino[x][x] = 0;
110 dijkstrados[x] = true;
111 }
112 }
113
114 for(long long x = 0; x < n; x++)
115 {
116     if(!dijkstrados[x])
117     {
118         vector<long long> distancia(n, LLONG_MAX);
119
120         priority_queue<
121             pair<long long, long long>,
122             vector<pair<long long, long long>>,
123             greater<pair<long long, long long>>
124         > cola;
125
126         distancia[x] = 0;
127         cola.push({0, x});
128
129         while(!cola.empty())
130         {
131             long long distanciaActual = cola.top().first;
132             long long actual = cola.top().second;
133             cola.pop();
134
135             if(distanciaActual > distancia[actual])
136                 continue;
137
138             for(auto vecino : conexion[actual])
139             {
140                 long long siguiente = vecino.first;
141                 long long peso = vecino.second;
142
143                 if(distancia[actual] + peso < distancia[siguiente])
144                 {
145                     distancia[siguiente] = distancia[actual] + peso;
146                     cola.push({distancia[siguiente], siguiente});
147                 }
148             }
149         }
150     }
151
152     for(long long i = 0; i < n; i++)
153     {
154         if(distancia[i] != LLONG_MAX)
155             camino[x][i] = distancia[i];
156     }
157
158     dijkstrados[x] = true;
159 }
160 }
161
162 // INPUT
163 for(long long i = 0; i < q; i++)
164 {
165     long long a, b;

```

```

65     cin >> a >> b;
66
67     cout << camino[a - 1][b - 1] << '\n';
68 }
69
70 return 0;
71 }

```

17.21 Shortest Routes II (Floyd Warshall) - Floyd-Warshall

C++\CSSES\Graph Algorithms\Shortest Routes II (Floyd-Warshall).cpp

Floyd-Warshall, la solución canónica de todos-contra-todos: tres for anidados donde el de AFUERA es el nodo intermedio k . Ese orden no es negociable, y es el error número uno de esta plantilla: si k va por dentro, el algoritmo no considera caminos con varios intermedios y da respuestas mal. La invariante es que después de la iteración k ya están todos los caminos que solo usan $\{0..k\}$ como intermedios, y de ahí sale la inducción. Se inicializa $dist[i][i] = 0$ y se guarda la arista MAS BARATA entre cada par, porque puede haber carreteras repetidas. El continue cuando $dist[i][k]$ es infinito no es solo optimización: evita que se sumen dos infinitos y se desborde. ESTE ARCHIVO ES LA OTRA SOLUCION del mismo problema que "Shortest Routes II", que ahí está resuelto con n Dijkstras. Vale la pena tener las dos y ver la diferencia: Floyd es $O(n^3)$ y se escribe en tres líneas, n Dijkstras es $O(n^m \log n)$ y gana solo si el grafo es disperso. Con $n \leq 500$, que es el límite típico, Floyd es lo que uno escribe. CUANDO USAR: $n \leq 500$ y hacen falta todos los pares. También sirve para cierre transitivo (cambiando $\min$ y $+$ por or y and) y para el cuello de botella minimax (cambiando $+$ por $\max$). Aguanta pesos negativos siempre que no haya ciclos negativos.

```

1  #include <iostream>
2  #include <vector>
3
4  using namespace std;
5
6  int main()
7  {
8      ios::sync_with_stdio(false);
9      cin.tie(nullptr);
10
11     long long n, m, q;
12     cin >> n >> m >> q;
13
14     const long long INF = 1e18;
15
16     vector<vector<long long>> dist(n, vector<long long>(n, INF));
17
18     for(long long i = 0; i < n; i++)
19     {
20         dist[i][i] = 0;
21     }
22
23     for(long long i = 0; i < m; i++)
24     {
25         long long a, b, c;
26         cin >> a >> b >> c;
27
28         a--;
29         b--;
30
31         // puede haber varias carreteras entre a y b: me quedo con la menor
32         if(c < dist[a][b])
33         {
34             dist[a][b] = c;
35             dist[b][a] = c;
36         }
37     }
38
39     // Floyd-Warshall
40     for(long long k = 0; k < n; k++) // nodo intermedio
41     {
42         for(long long i = 0; i < n; i++)
43         {
44             if(dist[i][k] == INF) // no llego a k desde i
45             {
46                 continue;
47             }
48
49             for(long long j = 0; j < n; j++)
50             {
51                 if(dist[i][k] + dist[k][j] < dist[i][j])

```

```

52             {
53                 dist[i][j] = dist[i][k] + dist[k][j];
54             }
55         }
56     }
57 }
58
59 for(long long i = 0; i < q; i++)
60 {
61     long long a, b;
62     cin >> a >> b;
63
64     a--;
65     b--;
66
67     if(dist[a][b] == INF)
68     {
69         cout << -1 << '\n';
70     }
71     else
72     {
73         cout << dist[a][b] << '\n';
74     }
75 }
76
77 return 0;
78 }

```

18 - CSSES: Introductory Problems

18.1 Apple Divisions - Fuerza Bruta 2^n

C++\CSSES\Introductory Problems\Apple Divisions.cpp

$n \leq 20$, o sea $2^{20} = \sim 10^6$ subconjuntos: cabe de sobra, así que se prueban todos. La recursión decide para cada posición si el elemento va a un grupo o al otro. EL DETALLE BONITO: no hace falta llevar las dos sumas. Con el total fijo, la diferencia entre grupos es $|total - 2 * sum|$, así que basta una sola variable. COMO RECONOCER QUE CABA LA FUERZA BRUTA: mirar n en las restricciones. $n \leq 20$ grita 2^n (o bitmask DP); $n \leq 10$ grita $n!$ (permutaciones); $n \leq 40$ grita $\text{meet in the middle}$. Ese mapeo de n a técnica es de lo más rentable que hay en competitiva. La versión con máscara de bits (for mask = 0 hasta $1 < n$) hace lo mismo sin recursión.

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  long long ans = LLONG_MAX;
5  long long total = 0;
6  vector<long long> a;
7
8  void solve(int pos, long long sum) {
9      if (pos == a.size()) {
10         ans = min(ans, abs(total - 2 * sum));
11         return;
12     }
13
14     solve(pos + 1, sum + a[pos]);
15     solve(pos + 1, sum);
16 }
17
18 int main() {
19     int n;
20     cin >> n;
21
22     a.resize(n);
23
24     for (auto &x : a) {
25         cin >> x;
26         total += x;
27     }
28
29     solve(0, 0);
30
31     cout << ans << '\n';
32 }

```

18.2 Bit Strings - Exponenciacion Modular Rapida

C++(CSESIntroductory Problems/Bit Strings.cpp)

$2^n \bmod (10^9+7)$ con exponenciación binaria: se eleva al cuadrado y se va recorriendo el exponente bit por bit, $O(\log n)$ multiplicaciones. POR QUE NO $\text{pow}(2, n)$: pow es de `double`, pierde precisión pasando de 2^{53} y no sabe de módulos. Para exponentes grandes con módulo SIEMPRE esta plantilla. DETALLE DEL TIPO: $\text{res} * a$ puede llegar a $(10^9)^2 = 10^{18}$, que cabe en `long long` pero NO en `int`. Si a y res fueran `int` el producto se desborda antes del módulo. Esta es la razón de que todo en la función sea `long long`. CUANDO USAR: cualquier respuesta "módulo 10^9+7 " que involucre potencias, y es la base del inverso modular por Fermat, $a^{(p-2)} \bmod p$.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const long long MOD = 1e9 + 7;
5
6 long long modpow(long long a, long long b) {
7     long long res = 1;
8
9     while (b) {
10         if (b & 1)
11             res = res * a % MOD;
12
13         a = a * a % MOD;
14         b >>= 1;
15     }
16
17     return res;
18 }
19
20 int main() {
21     long long n;
22     cin >> n;
23
24     cout << modpow(2, n) << '\n';
25 }
```

18.3 Chessboard And Queens - Backtracking con Poda por Diagonales

C++(CSESIntroductory Problems/Chessboard and Queens.cpp)

Las 8 reinas con backtracking fila por fila, que es lo que baja el espacio de búsqueda: poniendo una reina por fila, las filas ya no hay que revisarlas. LO QUE VALE COPIAR SON LOS INDICES DE DIAGONAL, que es donde todo el mundo se equivoca: `diag2[row + col]` -> las diagonales que bajan hacia la derecha, índice $0..2n-2$ `diag1[row - col + n-1]` -> las que bajan hacia la izquierda, con el $+n-1$ para no dar negativo Marcar, bajar, y DESMARCAR al volver: los tres arreglos se restauran siempre. CUANDO USAR: colocar piezas u objetos con restricciones de conflicto, en tableros chicos. La clave del backtracking útil es que la poda se pueda verificar en $O(1)$ por candidato, y para eso son estos arreglos de ocupado.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 char board[8][8];
5 bool col[8], diag1[15], diag2[15];
6 int ans = 0;
7
8 void solve(int row) {
9     if (row == 8) {
10         ans++;
11         return;
12     }
13
14     for (int c = 0; c < 8; c++) {
15         if (board[row][c] == '*')
16             continue;
17
18         if (col[c] || diag1[row - c + 7] || diag2[row + c])
19             continue;
20
21         col[c] = true;
22         diag1[row - c + 7] = true;
23         diag2[row + c] = true;
24
25         solve(row + 1);
26
27         col[c] = false;
28         diag1[row - c + 7] = false;
```

```
29         diag2[row + c] = false;
30     }
31 }
32
33 int main() {
34     for (int i = 0; i < 8; i++)
35         for (int j = 0; j < 8; j++)
36             cin >> board[i][j];
37
38     solve(0);
39
40     cout << ans << '\n';
41 }
```

18.4 Coin Piles - Invariantes Aritmeticos

C++(CSESIntroductory Problems/Coin Piles.cpp)

Simular nada: dos condiciones cerradas. Cada movimiento quita 3 monedas en total, así que $(a+b)$ tiene que ser divisible por 3. Y cada movimiento quita al menos 1 de cada pila, así que ninguna puede ser más del DOBLE de la otra, $\max(a,b) \leq 2 \cdot \min(a,b)$. Las dos juntas son suficientes. COMO SE LLEGA A ESO: buscar qué cantidad se conserva o cambia de forma fija en cada movimiento. Aquí el total baja de 3 en 3 (invariante de divisibilidad) y la diferencia entre pilas está acotada (invariante de desigualdad). CUANDO USAR: el problema pregunta si un estado es ALCANZABLE, con t casos grandes que hacen imposible simular. Buscar el invariante es lo que convierte una simulación en un par de ifs.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int t;
9     cin >> t;
10
11     while (t--) {
12         long long a, b;
13         cin >> a >> b;
14
15         if ((a + b) % 3 == 0 && max(a, b) <= 2 * min(a, b))
16             cout << "YES\n";
17         else
18             cout << "NO\n";
19     }
20 }
```

18.5 Creating Strings - Backtracking sobre Frecuencias

C++(CSESIntroductory Problems/Creating Strings.cpp)

Generar todas las permutaciones DISTINTAS, y el truco está en recorrer el ALFABETO en cada posición en vez de recorrer las posiciones del string. Como se elige "cuál letra va aquí" y cada letra se considera una sola vez por nivel, las repetidas no generan duplicados: no hay que filtrar nada después. El `freq[i]--` antes de bajar y el `freq[i]++` al volver es el patrón de préstamo de siempre. Y como el alfabeto se recorre de la 'a' a la 'z', las salidas ya vienen en orden lexicográfico. ALTERNATIVA: ordenar la cadena y usar `next_permutation` en un `do-while`, que también salta duplicados y es más corto. Esta versión se prefiere cuando hay que podar por algo. CUANDO USAR: "todas las permutaciones distintas" con letras repetidas. Permutar posiciones y meter en un set funciona pero desperdicia trabajo.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 vector<string> ans;
5 string s;
6 int freq[26];
7
8 void solve(string cur) {
9     if (cur.size() == s.size()) {
10         ans.push_back(cur);
11         return;
12     }
13
14     for (int i = 0; i < 26; i++) {
15         if (freq[i] == 0) continue;
16
17         freq[i]--;
18         solve(cur + char('a' + i));
```

```

18      freq[i]++;
19  }
20 }
21 }
22
23 int main() {
24     cin >> s;
25
26     for (char c : s)
27         freq[c - 'a']++;
28
29     solve("");
30
31     cout << ans.size() << '\n';
32
33     for (auto &x : ans)
34         cout << x << '\n';
35 }

```

18.6 Digit Queries - Búsqueda por Bloques de Longitud

C++/CS/ES/Introductory Problems/Digit Queries.cpp

La cadena infinita 123456789101112... se recorre por BLOQUES segun la cantidad de digitos: hay 9 numeros de 1 digito, 90 de 2, 900 de 3, o sea $9 \cdot 10^{(d-1)}$ numeros de d digitos, que aportan $d \cdot 9 \cdot 10^{(d-1)}$ caracteres. Se le resta a k bloque por bloque hasta que quepa, y ahi division y modulo dan el numero exacto y la posicion dentro de el. CUANDO USAR: cualquier "cual es el k -esimo elemento de esta secuencia infinita" donde la secuencia crece por tramos regulares. El patron es siempre el mismo, saltar tramos completos restando su tamano y despues aterrizar con division entera y modulo. El while corre a lo sumo 18 veces con k de 10^{18} , asi que no hay que optimizar nada. long long en todo: k llega a 10^{18} .

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      int q;
6      cin >> q;
7
8      while (q--) {
9          long long k;
10         cin >> k;
11
12         long long digits = 1;
13         long long count = 9;
14         long long start = 1;
15
16         while (k > digits * count) {
17             k -= digits * count;
18             digits++;
19             count *= 10;
20             start *= 10;
21         }
22
23         long long num = start + (k - 1) / digits;
24         long long pos = (k - 1) % digits;
25
26         string s = to_string(num);
27
28         cout << s[pos] << '\n';
29     }
30 }

```

18.7 Gray Code - Identidad i XOR (i >> 1)

C++/CS/ES/Introductory Problems/Gray Code.cpp

El codigo Gray de i es exactamente $i \oplus (i \gg 1)$. Eso es todo: no hay recursion ni construccion por niveles, es una sola operacion por numero. POR QUE FUNCIONA: al pasar de i a $i+1$ cambia una racha de bits bajos, y el xor con el desplazamiento hace que ese cambio se reduzca a UN solo bit de diferencia, que es justo la definicion del codigo Gray. La vuelta (de Gray a binario) es acumular xor de todos los bits mas altos, que no es simetrica y conviene recordar que existe. CUANDO USAR: recorrer todos los subconjuntos cambiando UN elemento a la vez. Es util cuando recalcular desde cero cuesta caro pero agregar o quitar un elemento cuesta $O(1)$, el mismo espiritu de Mo o de los problemas de torres de Hanoi.

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      int n;

```

```

6      cin >> n;
7
8      for (int i = 0; i < (1 << n); i++) {
9          int x = i ^ (i >> 1);
10
11         for (int j = n - 1; j >= 0; j--)
12             cout << ((x >> j) & 1);
13
14         cout << '\n';
15     }
16 }

```

18.8 Grid Coloring I - Greedy Local con 4 Colores

C++/CS/ES/Introductory Problems/Grid Coloring I.cpp

Greedy celda por celda: se prueba A, B, C, D y se toma el primero que no choque con el color ORIGINAL de la celda, con la celda de arriba y con la de la izquierda. POR QUE NUNCA SE ATASCA: son 3 restricciones y 4 colores, asi que por conteo siempre queda al menos uno libre. Ese argumento es el que hace que no haga falta backtracking. Y por que basta mirar arriba e izquierda: las celdas de abajo y de la derecha todavia no se han decidido, y cuando les toque van a mirar hacia aca. Cada pareja de vecinos se revisa una sola vez, desde el que va despues. CUANDO USAR: coloreos y asignaciones donde el numero de colores supera al de restricciones locales. Si los colores fueran 3 con 3 restricciones, esto ya no cerraria y habria que probar con busqueda.

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      int n, m;
6      cin >> n >> m;
7
8      vector<string> grid(n);
9
10     for (auto &row : grid)
11         cin >> row;
12
13     string colors = "ABCD";
14
15     for (int i = 0; i < n; i++) {
16         for (int j = 0; j < m; j++) {
17
18             for (char c : colors) {
19                 if (c == grid[i][j])
20                     continue;
21
22                 if (i > 0 && c == grid[i - 1][j])
23                     continue;
24
25                 if (j > 0 && c == grid[i][j - 1])
26                     continue;
27
28                 grid[i][j] = c;
29                 break;
30             }
31         }
32     }
33
34     for (auto row : grid)
35         cout << row << '\n';
36 }

```

18.9 Grid Path Description - Backtracking con Poda de Zona Encerrada

C++/CS/ES/Introductory Problems/Grid Path Description.cpp

Backtracking sobre las 48 posiciones del camino, con los '?' abriendo las 4 direcciones y las letras fijas forzando una sola. Sin poda esto no pasa ni de lejos. LA PODA ES TODO EL PROBLEMA: si la celda actual tiene libres arriba y abajo pero ocupadas izquierda y derecha (o al revés), el camino acaba de partir el tablero en dos y una de las mitades queda inalcanzable. Se corta ahi mismo. Esa sola condicion baja el arbol de busqueda de inviable a instantaneo. CUANDO USAR ESTE TIPO DE PODA: en caminos hamiltonianos o recorridos que deben cubrir todo, la poda util casi siempre es de CONECTIVIDAD, o sea detectar temprano que quedo una region a la que ya no se puede llegar. La version fuerte es correr un flood fill en cada nodo; esta version barata mira solo los cuatro vecinos y alcanza.

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  string s;

```

```

5  bool vis[7][7];
6  int ans = 0;
7
8  int dx[] = {1, -1, 0, 0};
9  int dy[] = {0, 0, 1, -1};
10
11 void dfs(int x, int y, int pos) {
12     if (x == 6 && y == 0) {
13         if (pos == 48)
14             ans++;
15         return;
16     }
17
18     if (pos == 48)
19         return;
20
21     // Si estamos encerrando una zona, no sirve continuar
22     if (x > 0 && x < 6 && !vis[x - 1][y] && !vis[x + 1][y] &&
23         (y == 0 || vis[x][y - 1]) && (y == 6 || vis[x][y + 1]))
24         return;
25
26     if (y > 0 && y < 6 && !vis[x][y - 1] && !vis[x][y + 1] &&
27         (x == 0 || vis[x - 1][y]) && (x == 6 || vis[x + 1][y]))
28         return;
29
30     for (int d = 0; d < 4; d++) {
31         if (s[pos] != '?' && s[pos] != "DURL"[d])
32             continue;
33
34         int nx = x + dx[d];
35         int ny = y + dy[d];
36
37         if (nx < 0 || nx >= 7 || ny < 0 || ny >= 7)
38             continue;
39
40         if (vis[nx][ny])
41             continue;
42
43         vis[nx][ny] = true;
44         dfs(nx, ny, pos + 1);
45         vis[nx][ny] = false;
46     }
47 }
48
49 int main() {
50     cin >> s;
51
52     vis[0][0] = true;
53
54     dfs(0, 0, 0);
55
56     cout << ans << '\n';
57 }

```

18.10 Increasing Array - Greedy de Maximo Prefijo

C++\CS\Introductory Problems\Increasing Array.cpp

Greedy: recorriendo de izquierda a derecha, cada elemento que sea menor que el maximo visto tiene que subir hasta ese maximo, y el costo se acumula. Nunca conviene subir un elemento por encima de lo necesario ni bajar ninguno, porque las operaciones solo suman. **POR QUE ES OPTIMO:** el maximo de prefijo es una cota inferior, cada posicion tiene que llegar por lo menos ahí, y el greedy paga exactamente esa cota. No hay decision que tomar, y eso es lo que distingue a un greedy trivialmente correcto de uno que hay que demostrar. long long en el acumulador: n posiciones con diferencias de hasta 10^9 se pasan de int.

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main()
5  {
6      long long n ; cin >> n;
7      long long ant=0;
8      long long god =0;
9      for(int i = 0 ; i < n ; i++)
10     {
11         long long a; cin >> a;

```

```

12         if(i!=0)
13         {
14             if(a < ant)
15             {
16                 god+= ant-a;
17
18                 ant = ant;
19                 continue;
20             }
21         }
22         ant = a;
23     }
24     cout << god << endl;
25
26     return 0;
27 }

```

18.11 Knight Moves Grid - BFS en Grilla

C++\CS\Introductory Problems\Knight Moves Grid.cpp

BFS de caballo sobre la grilla: la distancia en SALTOS es el numero de aristas, y BFS la da porque todos los movimientos cuestan lo mismo. Los 8 movimientos van en los arreglos dx y dy, que es como se evitan 8 bloques de codigo repetido. Se usa dist inicializada en -1 haciendo doble papel de distancia y de visitado, y se marca al ENCOLAR. CUANDO USAR: distancia minima en grilla con pasos de costo uniforme. Si los pasos costaran distinto seria Dijkstra, y con costos 0 y 1 seria 0-1 BFS con deque. OJO: usa auto [x, y] (structured bindings), que necesita C++17. Con un compilador viejo hay que volver a q.front().first y .second.

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      int n;
6      cin >> n;
7
8      vector<vector<int>> dist(n, vector<int>(n, -1));
9
10     int dx[] = {2, 2, -2, -2, 1, 1, -1, -1};
11     int dy[] = {1, -1, 1, -1, 2, -2, 2, -2};
12
13     queue<pair<int, int>> q;
14
15     q.push({0, 0});
16     dist[0][0] = 0;
17
18     while (!q.empty()) {
19         auto [x, y] = q.front();
20         q.pop();
21
22         for (int k = 0; k < 8; k++) {
23             int nx = x + dx[k];
24             int ny = y + dy[k];
25
26             if (nx < 0 || nx >= n || ny < 0 || ny >= n)
27                 continue;
28
29             if (dist[nx][ny] != -1)
30                 continue;
31
32             dist[nx][ny] = dist[x][y] + 1;
33             q.push({nx, ny});
34         }
35     }
36
37     for (int i = 0; i < n; i++) {
38         for (int j = 0; j < n; j++)
39             cout << dist[i][j] << ' ';
40         cout << '\n';
41     }
42 }

```

18.12 Mex Grid Construction - Identidad MEX = XOR

C++\CSES\Introductory Problems\Mex Grid Construction.cpp

Se pide la grilla donde cada celda es el MEX de lo que ya hay a su izquierda en la fila y arriba en la columna, y resulta que eso es EXACTAMENTE $i \wedge j$. Una sola línea, sin construir nada ni calcular ningún MEX. Verificado calculando el MEX de verdad en una grilla de 60x60. DE DONDE SALE: es la tabla de valores de Grundy del Nim con dos montones. La regla de Grundy es "el MEX de los estados alcanzables", y para dos montones da el xor, que es el teorema de Sprague-Grundy en su caso mas chico. Los dos problemas son literalmente la misma tabla. CUANDO SOSPECHARLO: si un problema define una tabla o secuencia como "el MEX de los anteriores", vale la pena calcular los primeros valores a mano y buscar el patron: casi siempre es un xor o algo periodico. Calcular el MEX de verdad suele ser innecesario.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n;
6     cin >> n;
7
8     for (int i = 0; i < n; i++) {
9         for (int j = 0; j < n; j++) {
10             cout << (i ^ j) << ' ';
11         }
12         cout << '\n';
13     }
14 }
```

18.13 Missing Number - Suma de Gauss

C++\CSES\Introductory Problems\Missing Number.cpp

La suma de $1..n$ se sabe de formula, $n*(n+1)/2$, así que el que falta es esa suma menos la suma de lo que llevo. Una sola pasada, memoria $O(1)$, sin ordenar nada. OJO CON EL TIPO: con $n = 2*10^5$ la suma pasa de $2*10^{10}$, o sea que int se desborda. long long. ALTERNATIVA QUE EVITA EL PROBLEMA: hacer XOR de $1..n$ con XOR de la entrada. Los repetidos se cancelan y queda el que falta, sin sumas grandes y sin riesgo de desbordar. Vale tenerla presente cuando n sea tan grande que hasta long long apriete. CUANDO USAR ESTE PATRON: "falta exactamente uno", "hay exactamente uno repetido". La idea es comparar contra el total que DEBERIA haber.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main()
5 {
6     long long n; cin >> n;
7     long long god = 1;
8     long long pre = 0;
9     for(int i = 0; i < n - 1; i++)
10     {
11         god += i + 2;
12         long long a;
13         cin >> a;
14         pre += a;
15     }
16     cout << god-pre << endl;
17
18     return 0;
19 }
```

18.14 Number Spiral - Formula por Capas

C++\CSES\Introductory Problems\Number Spiral.cpp

La espiral se lee por CAPAS: la capa k son las celdas con $\max(\text{fila}, \text{columna}) = k$, y toda la capa va entre $(k-1)^2+1$ y k^2 . Segun la paridad de k la capa se recorre en un sentido o en el otro, y de ahí salen las cuatro formulas de los ifs. CUANDO USAR: el problema pide el valor en una coordenada de un patron infinito, con t consultas y coordenadas hasta 10^9 . Construir la tabla es imposible, así que hay que encontrar la formula. COMO SE ENCUENTRA: dibujar los primeros 4×4 o 5×5 a mano y buscar que se repite. Aquí lo que se ve es que los cuadrados perfectos caen en la diagonal y que la direccion alterna con la paridad. Sin dibujarlo no sale. long long: y^2 con $y = 10^9$ es 10^{18} .

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int t;
```

```
9     cin >> t;
10
11     while (t--) {
12         long long y, x;
13         cin >> y >> x;
14
15         if (y > x) {
16             if (y % 2 == 0)
17                 cout << y * y - x + 1 << '\n';
18             else
19                 cout << (y - 1) * (y - 1) + x << '\n';
20         } else {
21             if (x % 2 == 0)
22                 cout << (x - 1) * (x - 1) + y << '\n';
23             else
24                 cout << x * x - y + 1 << '\n';
25         }
26     }
27 }
```

18.15 Palindrome Reorder - Conteo de Frecuencias

C++\CSES\Introductory Problems\Palindrome Reorder.cpp

Un palindromo se puede armar si y solo si a lo sumo UNA letra tiene frecuencia impar, la que quedaria en el centro. Se cuentan frecuencias, se cuentan las impares y si hay mas de una no hay solucion. Para construirlo: media frecuencia de cada letra en orden va a la primera mitad, la letra impar al centro, y la primera mitad invertida al final. Recorrer el alfabeto en orden da la lexicograficamente menor gratis. CUANDO USAR: cualquier problema de anagramas o reordenar caracteres. La frecuencia es lo unico que importa, el orden original de la cadena se puede tirar. El criterio de "a lo sumo una impar" es el que hay que tener memorizado.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     string s;
6     cin >> s;
7
8     vector<int> freq(26);
9
10    for (char c : s)
11        freq[c - 'A']++;
12
13    int odd = 0;
14    for (int x : freq)
15        odd += x % 2;
16
17    if (odd > 1) {
18        cout << "NO SOLUTION\n";
19        return 0;
20    }
21
22    string half, mid;
23
24    for (int i = 0; i < 26; i++) {
25        half += string(freq[i] / 2, 'A' + i);
26
27        if (freq[i] % 2)
28            mid += char('A' + i);
29    }
30
31    string rev = half;
32    reverse(rev.begin(), rev.end());
33
34    cout << half << mid << rev << '\n';
35 }
```

18.16 Permutations - Construccion Directa

C++\CSSE\Introductory Problems\Permutations.cpp

En vez de buscar, se **CONSTRUYE**: primero todos los pares y despues todos los impares. Dos numeros de la misma paridad difieren en al menos 2, asi que dentro de cada mitad no hay problema, y en la frontera se juntan el ultimo par con el 1, que para $n \geq 4$ tambien difieren en mas de 1. LOS CASOS IMPOSIBLES: $n = 2$ y $n = 3$ no tienen solucion y hay que responderlo aparte. $n = 1$ si la tiene (un solo numero, sin vecinos que comparar), y confundirse ahi es el error tipico. CUANDO PENSAR EN CONSTRUCCION: el enunciado dice "imprima CUALQUIER solucion" o "muestre una que cumpla". Eso casi siempre significa que existe un patron y que buscar con backtracking es perder el tiempo. Y conviene mirar los n chicos a mano: ahi salen los casos imposibles.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int n;
9     cin >> n;
10
11     if (n == 2 || n == 3) {
12         cout << "NO SOLUTION\n";
13         return 0;
14     }
15
16     for (int i = 2; i <= n; i += 2)
17         cout << i << ' ';
18
19     for (int i = 1; i <= n; i += 2)
20         cout << i << ' ';
21
22     cout << '\n';
23 }
```

18.17 Rab Game I - Construccion por Rotacion de Bloque

C++\CSSE\Introductory Problems\Rab Game I.cpp

Construccion con dos condiciones de imposibilidad: $a + b$ no puede pasar de n , y a y b tienen que ser cero los dos o ninguno, que es lo que verifica $(a == 0) != (b == 0)$. La primera fila es la identidad. La segunda rota un bloque de tamano $a+b$: se sacan los a primeros y se ponen al final del bloque, y lo que queda despues de m se deja igual. Eso deja exactamente las coincidencias pedidas en cada lado. CUANDO USAR: "construya un ejemplo con exactamente X de esto y Y de aquello". El metodo es siempre el mismo, primero las condiciones que hacen imposible el caso, y despues un bloque chico que se manipula dejando el resto intacto. Rotar es la operacion mas util porque controla las coincidencias de forma exacta.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int t;
6     cin >> t;
7
8     while (t--) {
9         int n, a, b;
10        cin >> n >> a >> b;
11
12        if (a + b > n || (a == 0) != (b == 0)) {
13            cout << "NO\n";
14            continue;
15        }
16
17        cout << "YES\n";
18
19        for (int i = 1; i <= n; i++)
20            cout << i << ' ';
21        cout << '\n';
22
23        int m = a + b;
24
25        for (int i = a + 1; i <= m; i++)
26            cout << i << ' ';
27
28        for (int i = 1; i <= a; i++)
29            cout << i << ' ';
30    }
```

```
31        for (int i = m + 1; i <= n; i++)
32            cout << i << ' ';
33
34        cout << '\n';
35    }
36 }
```

18.18 Repetitions - Barrido Lineal de Rachas

C++\CSSE\Introductory Problems\Repetitions.cpp

Una pasada llevando el largo de la racha actual y el maximo visto. Al cambiar el caracter se cierra la racha y se reinicia en 1. EL BUG CLASICO: la ultima racha nunca se cierra dentro del for, porque no hay un cambio de caracter que la corte. Por eso hay que comparar contra el maximo OTRA VEZ despues del ciclo, y es justamente lo que se olvida. Cada vez que se escribe un barrido de rachas hay que preguntarse quien cierra la ultima. CUANDO USAR: cualquier "cuantos consecutivos iguales", "el bloque mas largo de", en una pasada $O(n)$ y sin memoria.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main()
5 {
6     string s; cin >> s;
7     long long maxi = 0;
8     char ant = '-';
9     long long temp = 0;
10    for(int i = 0 ; i < (int) s.size() ; i ++){
11        {
12            if(s[i]== ant)
13            {
14                temp++;
15            }
16            else
17            {
18                if(temp>maxi)
19                {
20                    maxi = temp;
21                }
22                temp =1;
23                ant = s[i];
24            }
25        }
26        if(temp>maxi)
27        {
28            maxi = temp;
29        }
30    }
31    cout << maxi << endl;
32
33    return 0;
34 }
```

18.19 String Reorder - Greedy Lexicografico con Cota de Factibilidad

C++\CSSE\Introductory Problems\String Reorder.cpp

Se arma la respuesta caracter por caracter tomando siempre la letra mas chica posible, pero antes de fijarla se verifica que lo que queda TODAVIA se pueda terminar. La cota es la clave: con r caracteres por poner, ninguna letra puede quedar con mas de $(r+1)/2$ copias, porque si no dos de ellas quedarian pegadas por fuerza. POR QUE ESTO ES EL PATRON GENERAL: un greedy lexicografico solo funciona si se puede decidir en $O(1)$ o $O(\text{alfabeto})$ si la eleccion deja el problema resoluble. Sin ese chequeo, tomar siempre la letra mas chica lleva a callejones sin salida. La estructura "elegir el menor + verificar factibilidad del resto" aparece en un monton de problemas de construccion lexicografica. CUANDO USAR: "la cadena lexicograficamente menor que cumple X ". Casi nunca hace falta backtracking si se encuentra la cota de factibilidad.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     string s;
6     cin >> s;
7
8     int freq[26] = {};
9
10    for (char c : s)
11        freq[c - 'A']++;
12 }
```

```

3 string ans;
4 char last = '#';
5
6 for (int pos = 0; pos < s.size(); pos++) {
7     bool found = false;
8
9     for (int c = 0; c < 26; c++) {
10        if (freq[c] == 0 || 'A' + c == last)
11            continue;
12
13        freq[c]--;
14
15        int remaining = s.size() - pos - 1;
16        int mx = 0;
17
18        for (int i = 0; i < 26; i++)
19            mx = max(mx, freq[i]);
20
21        if (mx <= (remaining + 1) / 2) {
22            ans += char('A' + c);
23            last = 'A' + c;
24            found = true;
25            break;
26        }
27
28        freq[c]++;
29    }
30
31    if (!found) {
32        cout << -1 << '\n';
33        return 0;
34    }
35
36    cout << ans << '\n';
37 }

```

18.20 Tower Of Hanoi - Recursion Clasica

C++(CSES)Introductory Problems\Tower of Hanoi.cpp

La recursion de siempre: mover $n-1$ al auxiliar, mover el disco grande al destino, mover los $n-1$ del auxiliar al destino. El caso base es $n = 0$ y no hace nada. El total es $2^n - 1$ movimientos, que se imprime antes porque se sabe de formula. LO QUE HAY QUE VER: los tres parametros ROTAN en cada llamada. El auxiliar de un nivel es el destino del de abajo. Escribir eso bien es todo el problema, y equivocarse en el orden de los argumentos es el unico error posible. CUANDO USAR: el patron es "resolver dos subproblemas de tamano $n-1$ con los roles cambiados". Es el ejemplo canonico para ver que la recursion que genera 2^n salidas es inevitable cuando la SALIDA misma tiene tamano 2^n .

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 void hanoi(int n, int origen, int destino, int auxiliar) {
5     if (n == 0) return;
6
7     hanoi(n - 1, origen, auxiliar, destino);
8     cout << origen << ' ' << destino << '\n';
9     hanoi(n - 1, auxiliar, destino, origen);
10 }
11
12 int main() {
13     int n;
14     cin >> n;
15
16     cout << (1 << n) - 1 << '\n';
17
18     hanoi(n, 1, 3, 2);
19 }

```

18.21 Trailing Zeros - Formula de Legendre

C++(CSES)Introductory Problems\Trailing Zeros.cpp

Los ceros al final de $n!$ los deciden los factores 5, no los 2, porque un cero necesita un par (2,5) y los 2 sobran siempre. La cuenta es la formula de Legendre: $\text{floor}(n/5) + \text{floor}(n/25) + \text{floor}(n/125) + \dots$, que el while hace dividiendo n entre 5 repetidamente y acumulando. POR QUE HAY QUE SEGUIR DIVIDIENDO: 25 aporta dos cinco, 125 aporta tres. Contar solo $\text{floor}(n/5)$ es el error clasico y se queda corto. CUANDO USAR: exponente de un PRIMO p en $n!$, cambiando el 5 por p . Sirve para saber si un binomial es divisible por algo, para el teorema de Kummer, y para cualquier pregunta de divisibilidad sobre factoriales sin calcular el factorial.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     long long n;
6     cin >> n;
7
8     long long ans = 0;
9
10    while (n) {
11        n /= 5;
12        ans += n;
13    }
14
15    cout << ans << '\n';
16 }

```

18.22 Two Knights - Conteo por Complemento

C++(CSES)Introductory Problems\Two Knights.cpp

Contar por el complemento: total de parejas menos las que se atacan. El total es $C(k^2, 2) = k^2 * (k^2 - 1) / 2$, y las parejas que se atacan son $4 * (k-1) * (k-2)$, porque cada rectangulo de 2×3 aporta exactamente 2 ataques y hay $(k-1) * (k-2)$ de ellos en cada una de las dos orientaciones. Formula verificada contra fuerza bruta para k de 1 a 25. CUANDO CONTAR POR COMPLEMENTO: cuando lo que se pide ("que no se ataquen") es difícil de contar directo pero su negacion ("que se ataquen") es facil y estructurada. Es el primer reflejo que hay que tener en conteo, junto con inclusion-exclusion. long long obligatorio: con $k = 10^4$, $k^2 * (k^2 - 1) / 2$ anda por $5 * 10^{15}$.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     long long n;
6     cin >> n;
7
8     for (long long k = 1; k <= n; k++) {
9         long long total = k * k * (k * k - 1) / 2;
10        long long attack = 4 * (k - 1) * (k - 2);
11
12        cout << total - attack << '\n';
13    }
14 }

```

18.23 Two Sets - Construccion por Bloques de 4

C++(CSES)Introductory Problems\Two Sets.cpp

La suma total es $n * (n+1) / 2$, y para partirla en dos mitades iguales tiene que ser PAR, lo que solo pasa si $n \bmod 4$ es 0 o 3. Ese es todo el criterio de imposibilidad. La construccion aprovecha que cuatro consecutivos se parten perfecto: $\{i, i+3\}$ y $\{i+1, i+2\}$ suman lo mismo. Con $n \bmod 4 == 0$ se cubre todo asi; con $n \bmod 4 == 3$ sobran los tres primeros, que se acomodan a mano como $\{1,2\}$ contra $\{3\}$ y desde el 4 siguen los bloques. CUANDO USAR: reparticiones en grupos de suma igual. Lo primero es siempre la condicion de paridad o divisibilidad de la suma total, que descarta los imposibles de una; despues se busca el bloque chico que se parte solo y se repite.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n;
6     cin >> n;
7
8     if (n % 4 == 1 || n % 4 == 2) {
9         cout << "NO\n";
10        return 0;
11    }
12

```

```

13 vector<int> a, b;
14
15 if (n % 4 == 3) {
16     a.push_back(1);
17     a.push_back(2);
18     b.push_back(3);
19 }
20 for (int i = 4; i <= n; i += 4) {
21     a.push_back(i);
22     a.push_back(i + 3);
23     b.push_back(i + 1);
24     b.push_back(i + 2);
25 }
26 } else {
27     for (int i = 1; i <= n; i += 4) {
28         a.push_back(i);
29         a.push_back(i + 3);
30         b.push_back(i + 1);
31         b.push_back(i + 2);
32     }
33 }
34
35 cout << "YES\n";
36
37 cout << a.size() << '\n';
38 for (int x : a) cout << x << ' ';
39 cout << '\n';
40
41 cout << b.size() << '\n';
42 for (int x : b) cout << x << ' ';
43 cout << '\n';
44 }

```

18.24 Weird Algorithm - Simulacion Directa

C++ CSES/Introductory Problems/Weird Algorithm.cpp

Simular el proceso de Collatz tal cual lo dice el enunciado, sin mas. LO UNICO QUE PUEDE FALLAR: el tipo. Con n hasta 10^6 los valores INTERMEDIOS de la secuencia se van muy por encima de 10^9 , así que con `int` se desborda y el programa se cuelga o imprime basura. `long long` y listo. La lección general: antes de simular, estimar hasta donde crece la variable, no hasta donde crece la ENTRADA. Son dos cosas distintas y esta es la trampa mas comun de los problemas faciles.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main()
5 {
6     long long n; cin >> n;
7     cout << n << " ";
8     while(n!=1)
9     {
10         if(n%2==0)
11         {
12             n=n/2;
13         }
14         else
15         {
16             n=(n*3)+1;
17         }
18         cout << n << " ";
19     }
20
21     return 0;
22 }

```

19 - CSES: Range Queries

19.1 Distinct Value Queries II - Segment Tree sobre Ocurrencia Anterior

C++ CSES/Range Queries/Distinct Value Queries II.cpp

Saber si en $[x,y]$ hay valores repetidos, y la idea es preciosa: para cada posición i se calcula `prev[i]`, la última posición antes de i con el mismo valor (o 0 si no hay). Entonces el rango $[x,y]$ tiene un repetido si y solo si existe algún i en $[x,y]$ con `prev[i] >= x`. Eso se convierte en una consulta de MAXIMO de `prev[i]` sobre el rango, o como esta aquí, de minimo sobre el arreglo complementario. De cualquier forma, el punto es que una pregunta que parecía necesitar contar valores distintos se reduce a un maximo de rango, que un segment tree responde en $O(\log n)$ y ONLINE, sin necesidad de Mo. COMPARAR CON "Distinct Values Queries": ese cuenta CUANTOS distintos hay y por eso necesita Mo o un BIT offline. Este solo pregunta SI hay repetidos, que es mucho mas debil, y esa debilidad es justo lo que permite la reduccion a un maximo. Vale la pena mirar los dos juntos: la lección es que antes de sacar la artilleria conviene ver si el problema pide menos de lo que parece. EL PATRON DE `prev[i]`: aparece en casi todos los problemas de "distintos en un rango". Se calcula con un map de última posición vista en una pasada. OJO: usa structured bindings (auto [a, b]), que piden C++17. En CSES compila, pero con un g++ viejo hay que volver a `first` y `second`.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const int INF = 1e9;
5
6 struct SegmentTree {
7     int n;
8     vector<int> tree;
9
10     SegmentTree(int n) : n(n), tree(4 * n, INF) {}
11
12     void update(int node, int l, int r, int pos, int val) {
13         if (l == r) {
14             tree[node] = val;
15             return;
16         }
17
18         int mid = (l + r) / 2;
19
20         if (pos <= mid)
21             update(node * 2, l, mid, pos, val);
22         else
23             update(node * 2 + 1, mid + 1, r, pos, val);
24
25         tree[node] = min(tree[node * 2], tree[node * 2 + 1]);
26     }
27
28     void update(int pos, int val) {
29         update(1, 1, n, pos, val);
30     }
31
32     int query(int node, int l, int r, int ql, int qr) {
33         if (qr < l || r < ql)
34             return INF;
35
36         if (ql <= l && r <= qr)
37             return tree[node];
38
39         int mid = (l + r) / 2;
40
41         return min(
42             query(node * 2, l, mid, ql, qr),
43             query(node * 2 + 1, mid + 1, r, ql, qr)
44         );
45     }
46
47     int query(int l, int r) {
48         return query(1, 1, n, l, r);
49     }
50 };
51
52 struct Operation {
53     int type, x, y;
54 };
55
56 int main() {
57     ios::sync_with_stdio(false);
58     cin.tie(nullptr);
59

```

```

80 int n, q;
81 cin >> n >> q;
82
83 vector<int> a(n + 1);
84 vector<int> values;
85
86 for (int i = 1; i <= n; i++) {
87     cin >> a[i];
88     values.push_back(a[i]);
89 }
90
91 vector<Operation> ops(q);
92
93 for (auto &[type, x, y] : ops) {
94     cin >> type >> x >> y;
95
96     if (type == 1)
97         values.push_back(y);
98 }
99
100 sort(values.begin(), values.end());
101 values.erase(unique(values.begin(), values.end()), values.end());
102
103 auto get = [&](int x) {
104     return lower_bound(values.begin(), values.end(), x)
105            - values.begin();
106 };
107
108 for (int i = 1; i <= n; i++)
109     a[i] = get(a[i]);
110
111 int m = values.size();
112 vector<set<int>> pos(m);
113
114 for (int i = 1; i <= n; i++)
115     pos[a[i]].insert(i);
116
117 vector<int> nxt(n + 1, INF);
118
119 for (int x = 0; x < m; x++) {
120     auto &s = pos[x];
121
122     for (auto it = s.begin(); it != s.end(); ++it) {
123         auto nx = next(it);
124
125         if (nx != s.end())
126             nxt[*it] = *nx;
127     }
128 }
129
130 SegmentTree st(n);
131
132 for (int i = 1; i <= n; i++)
133     st.update(i, nxt[i]);
134
135 for (auto [type, x, y] : ops) {
136
137     if (type == 2) {
138
139         if (st.query(x, y) <= y)
140             cout << "NO\n";
141         else
142             cout << "YES\n";
143     } else {
144
145         int k = x;
146         int newValue = get(y);
147         int oldValue = a[k];
148
149         if (oldValue == newValue)
150             continue;

```

```

151
152 // Quitar k del valor anterior
153 {
154     auto &s = pos[oldValue];
155     auto it = s.find(k);
156
157     int prevPos = -1;
158     int nextPos = INF;
159
160     if (it != s.begin()) {
161         auto p = prev(it);
162         prevPos = *p;
163     }
164
165     auto nx = next(it);
166
167     if (nx != s.end())
168         nextPos = *nx;
169
170     if (prevPos != -1)
171         st.update(prevPos, nextPos);
172
173     st.update(k, INF);
174     s.erase(it);
175 }
176
177 // Agregar k al nuevo valor
178 {
179     auto &s = pos[newValue];
180
181     auto it = s.lower_bound(k);
182
183     int nextPos = INF;
184
185     if (it != s.end())
186         nextPos = *it;
187
188     if (it != s.begin()) {
189         auto p = prev(it);
190         st.update(*p, k);
191     }
192
193     st.update(k, nextPos);
194
195     s.insert(k);
196 }
197
198 a[k] = newValue;
199 }
200 }

```

19.2 Distinct Values Queries - Mo's Algorithm

C++ (CSES) Range Queries Distinct Values Queries.cpp

CUANDO USAR MO: esta es la pregunta importante, porque Mo no es la primera opcion casi nunca. Antes de llegar aqui hay que descartar lo demas, en este orden: - Suma, xor o cualquier operacion con INVERSA -> prefix sums, $O(1)$ por consulta. - min, max, gcd, o cualquier operacion asociativa e idempotente sin updates -> sparse table, $O(1)$ por consulta. - Operacion asociativa CON updates -> segment tree o BIT, $O(\log n)$. Mo entra cuando la respuesta NO se puede combinar a partir de las mitades. "Cuantos valores distintos", "cual es la moda", "cuantos valores aparecen exactamente k veces", "suma de los cuadrados de las frecuencias": de conocer la respuesta de $[l, m]$ y la de $[m+1, r]$ no sale la de $[l, r]$, asi que ningun segment tree la aguanta. Lo que si se puede es pasar de un rango al de al lado metiendo o sacando UN elemento en $O(1)$. Eso es exactamente lo que Mo explota. Las tres condiciones que se tienen que cumplir para usarlo: 1. OFFLINE. Hay que tener las q consultas antes de empezar a responder, porque lo primero que se hace es reordenarlas. Si el problema es interactivo o la consulta $i+1$ depende de la respuesta i , Mo queda descartado de entrada. 2. SIN actualizaciones al arreglo (existe Mo con updates, pero es $O(n^{5/3})$) y es otra plantilla distinta). 3. add y remove de UN elemento en $O(1)$, o a lo sumo $O(\log n)$. Los limites tambien mandan: Mo es $O((n+q)*\sqrt{n})$, asi que vive comodo en $n, q \leq 2*10^5$ con 1 o 2 segundos. Con n de 10^6 ya no cabe y toca buscar otra cosa. COMO FUNCIONA: se mantiene una ventana $[L, R]$ con el conteo cnt[] de cada valor y el total de distintos. Cada consulta se atiende moviendo L y R hasta su posicion. Movidos en desorden eso seria $O(n)$ por consulta, pero ordenando por (bloque de l, r) cada puntero se mueve poco: R solo avanza dentro de cada bloque (n por bloque, $\sqrt{n}$ bloques) y L nunca se aleja mas de un bloque. De ahi sale el sqrt. El orden de los cuatro while es el canonico: primero los dos que EXPANDEN y despues los dos que CONTRAEN, para que la ventana no quede invertida (L por delante de R) en medio del ajuste. Detalles de esta implementacion: - $B = \sqrt{n}$ es lo estandar; el tamaño de bloque que minimiza el costo teorico es $n/\sqrt{n}$, que ayuda cuando q es mucho mas chico que n. - La compresion de coordenadas NO es opcional aqui: los valores llegan a 10^9 y cnt[] se indexa por valor, asi que sin comprimir habria que reservar 10^9 enteros. - remove() se llama igual que std::remove de <algorithm>; con using namespace std compila igual porque ninguna sobrecarga de la STL acepta un solo int, pero es un nombre a evitar. Para este problema puntual existe algo mas rapido: offline ordenando las consultas por r y llevando un BIT con la ultima aparicion de cada valor, $O((n+q)*\log n)$. Mo se queda porque es la plantilla general: el dia que cambien "distintos" por "moda" o por "frecuencia de frecuencias", el BIT no sirve y esto de aqui solo cambia en add y remove.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int B; // tamaño de bloque
5 vector<int> a, cnt;
6 int distinct = 0;
7
8 struct Query { int l, r, idx; };
9
10 void add(int i) { if (cnt[a[i]]++ == 0) distinct++; }
11 void remove(int i) { if (--cnt[a[i]] == 0) distinct--; }
12
13 vector<int> mo(vector<Query>& qs) {
14     sort(qs.begin(), qs.end(), [](const Query& x, const Query& y) {
15         if (x.l / B != y.l / B) return x.l / B < y.l / B; // por bloque de l
16         return x.r < y.r; // luego por r
17     });
18
19     vector<int> ans(qs.size());
20     int l = 0, r = -1; // ventana actual vacia
21     for (auto& q : qs) {
22         while (R < q.r) add(++R);
23         while (L > q.l) add(--L);
24         while (R > q.r) remove(R--);
25         while (L < q.l) remove(L++);
26         ans[q.idx] = distinct;
27     }
28     return ans;
29 }
30
31 int main() {
32     ios::sync_with_stdio(false); cin.tie(nullptr);
33     int n, q; cin >> n >> q;
34     a.resize(n);
35     for (int& x : a) cin >> x;
36
37     // compresion de coordenadas
38     vector<int> v(a);
39     sort(v.begin(), v.end());
40     v.erase(unique(v.begin(), v.end()), v.end());
41     for (int& x : a) x = lower_bound(v.begin(), v.end(), x) - v.begin();
42     cnt.assign(v.size(), 0);
43 }
```

```
44 B = max(1, (int)sqrt(n));
45 vector<Query> qs(q);
46 for (int i = 0; i < q; i++) {
47     cin >> qs[i].l >> qs[i].r;
48     qs[i].l--; qs[i].r--;
49     qs[i].idx = i;
50 }
51
52 for (int x : mo(qs)) cout << x << '\n';
53 }
```

19.3 Dynamic Range Minimum Queries - Segment Tree (Minimo)

C++ (CSES) Range Queries Dynamic Range Minimum Queries.cpp

Segment tree de minimos con actualizacion puntual, en $O(\log n)$ cada operacion. El nodo p cubre $[l, r]$ y sus hijos son $2p$ y $2p+1$; por eso el arreglo se reserva de tamaño $4n$, que es la cota segura para cualquier n (no solo potencias de 2). CUANDO USAR SEGMENT TREE Y NO OTRA COSA: la operacion es asociativa pero NO tiene inversa (min, max, gcd), asi que restar no es opcion y Fenwick queda descartado, y ADEMÁS el arreglo cambia, lo que descarta la sparse table. Ese cruce es exactamente el nicho del segment tree. Es la plantilla de la que salen todas las variantes: cambiando el min por otra operacion asociativa sirve igual, y agregandole lazy se actualizan rangos completos.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 struct SegmentTree {
7     int n;
8     vector<ll> tree;
9
10     SegmentTree(int n) : n(n), tree(4 * n) {}
11
12     void build(int node, int l, int r, vector<ll>& arr) {
13         if (l == r) {
14             tree[node] = arr[l];
15             return;
16         }
17
18         int mid = (l + r) / 2;
19
20         build(node * 2, l, mid, arr);
21         build(node * 2 + 1, mid + 1, r, arr);
22
23         tree[node] = min(tree[node * 2], tree[node * 2 + 1]);
24     }
25
26     void update(int node, int l, int r, int pos, ll val) {
27         if (l == r) {
28             tree[node] = val;
29             return;
30         }
31
32         int mid = (l + r) / 2;
33
34         if (pos <= mid)
35             update(node * 2, l, mid, pos, val);
36         else
37             update(node * 2 + 1, mid + 1, r, pos, val);
38
39         tree[node] = min(tree[node * 2], tree[node * 2 + 1]);
40     }
41
42     ll query(int node, int l, int r, int ql, int qr) {
43         if (ql <= l && r <= qr)
44             return tree[node];
45
46         int mid = (l + r) / 2;
47         ll ans = LLONG_MAX;
48
49         if (ql <= mid)
50             ans = min(ans, query(node * 2, l, mid, ql, qr));
51
52         if (qr > mid)
53             ans = min(ans, query(node * 2 + 1, mid + 1, r, ql, qr));
54     }
55 }
```

```

94
95     return ans;
96 }
97 };
98
99 int main() {
100     ios::sync_with_stdio(false);
101     cin.tie(nullptr);
102
103     int n, q;
104     cin >> n >> q;
105
106     vector<ll> arr(n + 1);
107
108     for (int i = 1; i <= n; i++)
109         cin >> arr[i];
110
111     SegmentTree st(n);
112
113     st.build(1, 1, n, arr);
114
115     while (q--) {
116         int a, b;
117         ll c;
118         cin >> a >> b >> c;
119
120         if (a == 1) {
121             st.update(1, 1, n, b, c);
122         } else {
123             cout << st.query(1, 1, n, b, c) << '\n';
124         }
125     }
126 }

```

19.4 Dynamic Range Sum Queries - Fenwick Tree (BIT)

C++\CS5\Range Queries\Dynamic Range Sum Queries.cpp

Fenwick (o BIT): sumas de prefijo con actualizaci3n puntual, ambas en $O(\log n)$, en ~ 10 l3neas. Toda la magia esta en $i \& -i$, que aisla el bit m3s bajo encendido: sumando ese valor se sube a los padres (add) y restandolo se baja recorriendo bloques (sum). CUANDO USAR FENWICK EN VEZ DE SEGMENT TREE: si la operaci3n es SUMA (o cualquiera con inversa) y las actualizaciones son puntuales, Fenwick es la respuesta: menos c3digo, menos memoria y m3s r3pido en la pr3ctica. El segment tree se necesita para min, max o gcd, que no se pueden restar, y para lazy propagation. Fenwick es 1-indexado a la fuerza: con $i = 0$, $i \& -i$ es 0 y el for no avanza nunca.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 struct Fenwick {
7     int n;
8     vector<ll> bit;
9
10    Fenwick(int n) : n(n), bit(n + 1, 0) {}
11
12    void add(int i, ll x) {
13        for (; i <= n; i += i & -i)
14            bit[i] += x;
15    }
16
17    ll sum(int i) {
18        ll res = 0;
19        for (; i > 0; i -= i & -i)
20            res += bit[i];
21        return res;
22    }
23
24    ll query(int l, int r) {
25        return sum(r) - sum(l - 1);
26    }
27 };
28
29 int main() {
30     ios::sync_with_stdio(false);

```

```

31     cin.tie(nullptr);
32
33     int n, q;
34     cin >> n >> q;
35
36     vector<ll> arr(n + 1);
37     Fenwick fw(n);
38
39     for (int i = 1; i <= n; i++) {
40         cin >> arr[i];
41         fw.add(i, arr[i]);
42     }
43
44     while (q--) {
45         int a, b;
46         ll c;
47         cin >> a >> b >> c;
48
49         if (a == 1) {
50             ll diff = c - arr[b];
51             arr[b] = c;
52             fw.add(b, diff);
53         } else {
54             cout << fw.query(b, c) << '\n';
55         }
56     }
57 }

```

19.5 Forest Queries - Prefix Sums 2D

C++\CS5\Range Queries\Forest_Queries.cpp

Prefix sums en dos dimensiones, donde lo 3nico nuevo es la inclusion-exclusion: al construir se suma arriba e izquierda y se RESTA la diagonal, que se conto dos veces, y al consultar el rect3ngulo se hace lo mismo al revs con las cuatro esquinas. CUANDO USAR: consultas de suma sobre submatrices en una matriz que no cambia. Construcci3n $O(n^2)$, cada consulta $O(1)$. Si la matriz cambia hay que irse a Fenwick 2D o segment tree 2D, que es bastante m3s c3digo. El truco de reservar $(n+1) \times (n+1)$ con la fila y columna 0 en cero es lo que hace que los 3ndices $i-1$ y $j-1$ nunca se salgan, sin un solo if de borde.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int n, q;
9     cin >> n >> q;
10
11     vector<vector<int>> pref(n + 1, vector<int>(n + 1));
12
13     for (int i = 1; i <= n; i++) {
14         string s;
15         cin >> s;
16
17         for (int j = 1; j <= n; j++) {
18             int tree = (s[j - 1] == '*');
19
20             pref[i][j] = tree
21                 + pref[i - 1][j]
22                 + pref[i][j - 1]
23                 - pref[i - 1][j - 1];
24         }
25     }
26
27     while (q--) {
28         int y1, x1, y2, x2;
29         cin >> y1 >> x1 >> y2 >> x2;
30
31         cout << pref[y2][x2]
32             - pref[y1 - 1][x2]
33             - pref[y2][x1 - 1]
34             + pref[y1 - 1][x1 - 1]
35             << '\n';
36     }
37 }

```

19.6 Forest Queries II - Fenwick 2D

C++ (CSES/Range Queries/Forest Queries II.cpp)

Fenwick en dos dimensiones: un for anidado dentro de otro, cada uno con el mismo i & $-i$ de siempre. Actualizar una celda y consultar el rectangulo de $(1,1)$ a (x,y) cuestan $O(\log^2 n)$, y un rectangulo cualquiera sale por inclusion-exclusion con las cuatro esquinas, igual que en las sumas prefijas 2D. COMPARAR CON "Forest Queries": aquella no tenia cambios y le bastaban sumas prefijas 2D con consulta $O(1)$. Aqui los arboles se cortan y se plantan, y ese cambio es lo que obliga a pasar a Fenwick: las sumas prefijas se tendrían que reconstruir enteras, $O(n^2)$ por actualizacion. El toggle se maneja guardando el estado actual en $a[i][j]$ para saber si sumar $+1$ o -1 . Sin ese arreglo no hay forma de saber que hay en una celda sin hacer una consulta. CUANDO USAR: sumas sobre rectangulos con actualizaciones puntuales y n de hasta unos 1000 (la memoria es n^2). Si hay que actualizar RECTANGULOS enteros, esto no alcanza.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int n, q;
5 int bit[1005][1005];
6 bool a[1005][1005];
7
8 void add(int x, int y, int v) {
9     for (int i = x; i <= n; i += i & -i)
10         for (int j = y; j <= n; j += j & -j)
11             bit[i][j] += v;
12 }
13
14 int sum(int x, int y) {
15     int ans = 0;
16
17     for (int i = x; i > 0; i -= i & -i)
18         for (int j = y; j > 0; j -= j & -j)
19             ans += bit[i][j];
20
21     return ans;
22 }
23
24 int query(int x1, int y1, int x2, int y2) {
25     return sum(x2, y2)
26         - sum(x1 - 1, y2)
27         - sum(x2, y1 - 1)
28         + sum(x1 - 1, y1 - 1);
29 }
30
31 int main() {
32     ios::sync_with_stdio(false);
33     cin.tie(nullptr);
34
35     cin >> n >> q;
36
37     for (int i = 1; i <= n; i++) {
38         string s;
39         cin >> s;
40
41         for (int j = 1; j <= n; j++) {
42             if (s[j - 1] == '*') {
43                 a[i][j] = true;
44                 add(i, j, 1);
45             }
46         }
47     }
48
49     while (q--) {
50         int type;
51         cin >> type;
52
53         if (type == 1) {
54             int y, x;
55             cin >> y >> x;
56
57             if (a[y][x]) {
58                 a[y][x] = false;
59                 add(y, x, -1);
60             } else {
61                 a[y][x] = true;
```

```
62         add(y, x, 1);
63     }
64     } else {
65         int y1, x1, y2, x2;
66         cin >> y1 >> x1 >> y2 >> x2;
67
68         cout << query(y1, x1, y2, x2) << '\n';
69     }
70 }
71 }
```

19.7 Hotel Queries - Segment Tree con Descenso

C++ (CSES/Range Queries/Hotel Queries.cpp)

Segment tree de MAXIMOS, pero lo interesante no es la consulta sino el DESCENSO: para hallar el primer hotel con capacidad suficiente, en vez de buscar con una consulta y despues actualizar (dos $O(\log n)$ y una busqueda binaria por fuera), se baja una sola vez desde la raiz preguntando si el hijo izquierdo alcanza; si alcanza se va por ahi, si no por el derecho. Al llegar a la hoja se descuenta de una. Todo en un solo $O(\log n)$. CUANDO USAR EL DESCENSO: cada vez que el problema pida "el PRIMER indice que cumple algo" y el predicado se pueda decidir mirando el maximo (o el minimo, o la suma) de un nodo. Es el patron que reemplaza a "busqueda binaria + consulta", que seria $O(\log^2 n)$.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct ST {
5     int n;
6     vector<int> t;
7
8     ST(vector<int>& a) {
9         n = a.size() - 1;
10        t.resize(4 * n);
11        build(1, 1, n, a);
12    }
13
14    void build(int p, int l, int r, vector<int>& a) {
15        if (l == r) {
16            t[p] = a[l];
17            return;
18        }
19
20        int m = (l + r) / 2;
21
22        build(p * 2, l, m, a);
23        build(p * 2 + 1, m + 1, r, a);
24
25        t[p] = max(t[p * 2], t[p * 2 + 1]);
26    }
27
28    int find(int p, int l, int r, int x) {
29        if (l == r) {
30            t[p] -= x;
31            return l;
32        }
33
34        int m = (l + r) / 2;
35        int pos;
36
37        if (t[p * 2] >= x)
38            pos = find(p * 2, l, m, x);
39        else
40            pos = find(p * 2 + 1, m + 1, r, x);
41
42        t[p] = max(t[p * 2], t[p * 2 + 1]);
43
44        return pos;
45    }
46
47    int find(int x) {
48        if (t[1] < x)
49            return 0;
50
51        return find(1, 1, n, x);
52    }
53 };
```

```

94
95 int main() {
96     ios::sync_with_stdio(false);
97     cin.tie(nullptr);
98
99     int n, m;
100    cin >> n >> m;
101
102    vector<int> a(n + 1);
103
104    for (int i = 1; i <= n; i++)
105        cin >> a[i];
106
107    ST st(a);
108
109    while (m--) {
110        int x;
111        cin >> x;
112        cout << st.find(x) << ' ';
113    }
114 }

```

19.8 Increasing Array Queries - Monotonic Stack + Binary Lifting con Costos

C++ (CS5) Range Queries Increasing Array Queries.cpp

Para cada consulta $[l, r]$, el mínimo de sumas para que el tramo quede no decreciente. La solución óptima es subir cada elemento al MÁXIMO DE PREFIJO del tramo, y ese máximo solo cambia en los "siguientes mayores". Entonces el tramo se parte en bloques: desde i , todo hasta antes de $up[0][i]$ (el siguiente estrictamente mayor) se sube al valor $a[i]$, y cuesta $a[i] * (j - i - 1) - (pref[j-1] - pref[i])$. El siguiente mayor sale con una pila monotona de derecha a izquierda, con $a[n+1] = 1e18$ de centinela para que todos tengan a donde apuntar. EL BINARY LIFTING GUARDA EL COSTO JUNTO CON EL SALTO: $up[k][i]$ es a donde se llega con 2^k saltos, y $cost[k][i]$ lo que se paga en ese camino. Cada consulta baja de la potencia más alta a la más baja mientras el salto no se pase de r , y el tramo final (de x hasta r , sin llegar al siguiente mayor) se cobra aparte con la misma fórmula. Es la misma combinación que "Visible Building Queries", con una sola diferencia: aquí cada salto además acumula un valor. Ese es el patrón general: si un puntero define una función y cada paso tiene un costo, el doubling precalcula a la vez el destino y la suma.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 const int N = 200005;
7 const int LOG = 20;
8
9 ll a[N], pref[N], cost[LOG][N];
10 int up[LOG][N];
11
12 int main() {
13     ios::sync_with_stdio(false);
14     cin.tie(nullptr);
15
16     int n, q;
17     cin >> n >> q;
18
19     for (int i = 1; i <= n; i++) {
20         cin >> a[i];
21         pref[i] = pref[i - 1] + a[i];
22     }
23
24     a[n + 1] = 1e18;
25
26     for (int k = 0; k < LOG; k++)
27         up[k][n + 1] = n + 1;
28
29     stack<int> st;
30     st.push(n + 1);
31
32     for (int i = n; i >= 1; i--) {
33         while (a[st.top()] <= a[i])
34             st.pop();
35
36         up[0][i] = st.top();
37
38         int j = up[0][i];

```

```

39
40         cost[0][i] =
41             a[i] * (j - i - 1)
42             - (pref[j - 1] - pref[i]);
43
44         st.push(i);
45     }
46
47     for (int k = 1; k < LOG; k++) {
48         for (int i = 1; i <= n; i++) {
49             int mid = up[k - 1][i];
50
51             up[k][i] = up[k - 1][mid];
52
53             cost[k][i] =
54                 cost[k - 1][i] +
55                 cost[k - 1][mid];
56         }
57     }
58
59     while (q--) {
60         int l, r;
61         cin >> l >> r;
62
63         int x = l;
64         ll ans = 0;
65
66         for (int k = LOG - 1; k >= 0; k--) {
67             if (up[k][x] <= r) {
68                 ans += cost[k][x];
69                 x = up[k][x];
70             }
71         }
72
73         if (x < r) {
74             ans += a[x] * (r - x)
75                 - (pref[r] - pref[x]);
76         }
77
78         cout << ans << '\n';
79     }
80 }

```

19.9 List Removals - Fenwick con Descenso Binario (k-esimo)

C++ (CS5) Range Queries List Removals.cpp

Fenwick de unos y ceros (1 = la posición todavía está) donde lo interesante es $kth()$: encontrar la k -ésima posición viva sin búsqueda binaria por fuera. EL DESCENSO SOBRE EL BIT: se arranca en la potencia de 2 más grande y se va bajando, probando en cada paso si el bloque que empieza en $pos+p$ acumula menos de k . Si acumula menos, se avanza y se le resta ese acumulado a k . Como los bloques del Fenwick son exactamente potencias de 2, ese recorrido es un descenso por el árbol implícito y sale en $O(\log n)$ de una, contra $O(\log^2 n)$ de hacer búsqueda binaria llamando a $sum()$ en cada paso. CUANDO USAR: "el k -ésimo elemento que queda", "el k -ésimo más chico", con inserciones y borrados. Es el mismo patrón que el descenso del segment tree (ver "Hotel Queries"): en vez de buscar por fuera, se baja por la estructura aprovechando lo que ya guarda cada nodo. OJO con el $1 < k < 18$: tiene que ser una potencia de 2 mayor o igual a n . Con n más grande hay que subirlo, y ese es el error silencioso de esta plantilla.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Fenwick {
5     int n;
6     vector<int> bit;
7
8     Fenwick(int n) : n(n), bit(n + 1, 0) {}
9
10    void add(int i, int x) {
11        for (; i <= n; i += i & -i)
12            bit[i] += x;
13    }
14
15    int sum(int i) {
16        int res = 0;
17        for (; i > 0; i -= i & -i)

```

```

8         res += bit[i];
9         return res;
10    }
11
12    int kth(int k) {
13        int pos = 0;
14
15        for (int p = 1 << 18; p; p >>= 1) {
16            int next = pos + p;
17
18            if (next <= n && bit[next] < k) {
19                pos = next;
20                k -= bit[next];
21            }
22        }
23
24        return pos + 1;
25    }
26 };
27
28 int main() {
29     ios::sync_with_stdio(false);
30     cin.tie(nullptr);
31
32     int n;
33     cin >> n;
34
35     vector<int> a(n + 1);
36
37     for (int i = 1; i <= n; i++) {
38         cin >> a[i];
39     }
40
41     Fenwick fw(n);
42
43     for (int i = 1; i <= n; i++)
44         fw.add(i, 1);
45
46     for (int i = 0; i < n; i++) {
47         int k;
48         cin >> k;
49
50         int pos = fw.kth(k);
51
52         cout << a[pos] << ' ';
53
54         fw.add(pos, -1);
55     }
56
57     cout << '\n';
58 }

```

19.10 Missing Coin Sum Queries - Cubetas por Potencia de 2 + Segment Tree

C++ (CS3) Range Queries, Missing Coin Sum Queries.cpp

La version por rangos del clasico "menor suma que no se puede formar". El greedy de siempre es ordenar las monedas y mantener sum: si la siguiente moneda es $\leq \text{sum}+1$ se absorbe, si no la respuesta es $\text{sum}+1$. Ordenar cada rango seria $O(n \log n)$ por consulta; lo que lo evita son las CUBETAS: la moneda x va a la cubeta $\text{floor}(\log_2 x)$, o sea que en la cubeta k todo esta en $[2^k, 2^{k+1})$. LA OBSERVACION QUE HACE FUNCIONAR LAS CUBETAS: dentro de una misma cubeta, cualquier moneda es menor que el doble de la mas chica. Asi que si la mas chica de la cubeta cabe (es $\leq \text{sum}+1$), despues de absorberla TODAS las demas de esa cubeta tambien caben, y se pueden sumar de golpe con una suma prefija. Solo hace falta saber el MINIMO de cada cubeta en el rango, y para eso va un segment tree de minimos por cubeta. Con 30 cubetas cada consulta cuesta $O(30 \log n)$ en vez de ordenar. Es pesado en memoria: las dos tablas de 30 columnas suman cerca de 100 MB. En CS3 entra (512 MB); en un juez de 256 tambien, pero no sobra mucho.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 const int N = 200005;
7 const int K = 30;
8 const int INF = 2e9;

```

```

9
10 int st[2 * N][K];
11 ll pref[N][K];
12
13 int main() {
14     ios::sync_with_stdio(false);
15     cin.tie(nullptr);
16
17     int n, q;
18     cin >> n >> q;
19
20     for (int i = 0; i < 2 * N; i++)
21         for (int j = 0; j < K; j++)
22             st[i][j] = INF;
23
24     for (int i = 1; i <= n; i++) {
25         int x;
26         cin >> x;
27
28         int k = 31 - __builtin_clz(x);
29
30         st[N + i][k] = x;
31         pref[i][k] = x;
32     }
33
34     for (int i = 1; i <= n; i++)
35         for (int k = 0; k < K; k++)
36             pref[i][k] += pref[i - 1][k];
37
38     for (int i = N - 1; i >= 1; i--)
39         for (int k = 0; k < K; k++)
40             st[i][k] = min(st[i * 2][k], st[i * 2 + 1][k]);
41
42     while (q--) {
43         int l, r;
44         cin >> l >> r;
45
46         int L = l + N;
47         int R = r + N;
48
49         int mn[K];
50
51         for (int k = 0; k < K; k++)
52             mn[k] = INF;
53
54         while (L <= R) {
55             if (L & 1) {
56                 for (int k = 0; k < K; k++)
57                     mn[k] = min(mn[k], st[L][k]);
58                 L++;
59             }
60
61             if (!(R & 1)) {
62                 for (int k = 0; k < K; k++)
63                     mn[k] = min(mn[k], st[R][k]);
64                 R--;
65             }
66
67             L /= 2;
68             R /= 2;
69         }
70
71         ll sum = 0;
72
73         for (int k = 0; k < K; k++) {
74             if (sum + 1 < mn[k] &&
75                 sum + 1 < (1LL << (k + 1))) {
76                 cout << sum + 1 << '\n';
77                 goto done;
78             }
79
80             sum += pref[r][k] - pref[l - 1][k];
81         }

```

```

82
83     cout << sum + 1 << '\n';
84
85     done;;
86 }
87 }

```

19.11 Movie Festival Queries - Greedy + Binary Lifting sobre el Tiempo

C++ (VCSES)Range Queries\Movie Festival Queries.cpp

Maximo de peliculas que se pueden ver enteras dentro de $[a, b]$. El greedy de intervalos es el de siempre: ver primero la que TERMINA mas temprano entre las que empiezan despues del momento actual (intercambio clasico: cambiar cualquier eleccion por la que termina antes nunca empeora). Aqui el truco es indexar por TIEMPO, no por pelicula: $nxt[t]$ = el fin mas temprano de una pelicula que empieza en t o despues. Se calcula con un minimo de sufijos, y queda como una funcion $t \rightarrow nxt[t]$ que es justo "ver la siguiente pelicula". Sobre esa funcion se monta binary lifting: $up[k][t]$ es el momento tras ver 2^k peliculas, y cada consulta cuenta cuantos saltos caben antes de pasarse de b . $O(\log T)$ por consulta. Es el mismo patron que "Visible Building Queries": un puntero, doubling encima, y contar saltos. La tabla $up[20][10^6]$ ocupa unos 80 MB, que es el precio de indexar por tiempo.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const int MAXT = 1000000;
5 const int LOG = 20;
6
7 int up[LOG][MAXT + 2];
8 int nxt[MAXT + 2];
9 int en[MAXT + 2];
10
11 int main() {
12     ios::sync_with_stdio(false);
13     cin.tie(nullptr);
14
15     int n, q;
16     cin >> n >> q;
17
18     for (int i = 0; i <= MAXT + 1; i++)
19         en[i] = MAXT + 1;
20
21     for (int i = 0; i < n; i++) {
22         int a, b;
23         cin >> a >> b;
24         en[a] = min(en[a], b);
25     }
26
27     nxt[MAXT + 1] = MAXT + 1;
28
29     for (int i = MAXT; i >= 1; i--)
30         nxt[i] = min(en[i], nxt[i + 1]);
31
32     for (int i = 1; i <= MAXT; i++)
33         up[0][i] = nxt[i];
34
35     up[0][MAXT + 1] = MAXT + 1;
36
37     for (int k = 1; k < LOG; k++) {
38         up[k][MAXT + 1] = MAXT + 1;
39
40         for (int i = 1; i <= MAXT; i++) {
41             int mid = up[k - 1][i];
42             up[k][i] = up[k - 1][mid];
43         }
44     }
45
46     while (q--) {
47         int a, b;
48         cin >> a >> b;
49
50         int pos = a;
51         int ans = 0;
52
53         for (int k = LOG - 1; k >= 0; k--) {
54             if (up[k][pos] <= b) {
55                 ans += (1 << k);

```

```

56         pos = up[k][pos];
57     }
58 }
59
60 cout << ans << '\n';
61 }
62 }

```

19.12 Pizzeria Queries - Truco de $p[i]$ mas y menos i

C++ (VCSES)Range Queries\Pizzeria Queries.cpp

Hay que minimizar $p[j] + |i - j|$ sobre todo j , con actualizaciones. El valor absoluto es lo que estorba, y el truco es partirlo en los dos casos y meter el indice DENTRO del valor guardado: si $j \leq i$: $p[j] + i - j = (p[j] - j) + i$ si $j \geq i$: $p[j] + j - i = (p[j] + j) - i$ Como i es fijo durante la consulta, minimizar cada caso es minimizar $(p[j] - j)$ a la izquierda y $(p[j] + j)$ a la derecha. O sea DOS segment trees de minimos, uno sobre $p[j] - j$ y otro sobre $p[j] + j$, y la respuesta es el menor de los dos resultados sumandole o restandole i . CUANDO USAR ESTE TRUCO: cada vez que aparece un $|i - j|$ junto a algo que depende de j . Sale en problemas de "la tienda mas cercana con su costo", en DP con costos de distancia, y en cualquier minimizacion sobre una recta. La regla es partir en $j \leq i$ y $j > i$ y absorber el indice en el valor que se guarda; lo que quedaba dependiendo de i sale del minimo como constante.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 struct SegmentTree {
7     int n;
8     vector<ll> tree;
9
10     SegmentTree(vector<ll>& a) {
11         n = a.size() - 1;
12         tree.resize(4 * n);
13         build(1, 1, n, a);
14     }
15
16     void build(int node, int l, int r, vector<ll>& a) {
17         if (l == r) {
18             tree[node] = a[l];
19             return;
20         }
21
22         int mid = (l + r) / 2;
23
24         build(node * 2, l, mid, a);
25         build(node * 2 + 1, mid + 1, r, a);
26
27         tree[node] = min(tree[node * 2], tree[node * 2 + 1]);
28     }
29
30     void update(int node, int l, int r, int pos, ll value) {
31         if (l == r) {
32             tree[node] = value;
33             return;
34         }
35
36         int mid = (l + r) / 2;
37
38         if (pos <= mid)
39             update(node * 2, l, mid, pos, value);
40         else
41             update(node * 2 + 1, mid + 1, r, pos, value);
42
43         tree[node] = min(tree[node * 2], tree[node * 2 + 1]);
44     }
45
46     ll query(int node, int l, int r, int ql, int qr) {
47         if (ql <= l && r <= qr)
48             return tree[node];
49
50         int mid = (l + r) / 2;
51
52         ll ans = LLONG_MAX;
53
54         if (ql <= mid)

```

```

95     ans = min(ans, query(node * 2, l, mid, ql, qr));
96
97     if (qr > mid)
98         ans = min(ans, query(node * 2 + 1, mid + 1, r, ql, qr));
99
100    return ans;
101 }
102
103 void update(int pos, ll value) {
104     update(1, 1, n, pos, value);
105 }
106
107 ll query(int l, int r) {
108     if (l > r)
109         return LLONG_MAX;
110
111     return query(1, 1, n, l, r);
112 }
113 };
114
115 int main() {
116     ios::sync_with_stdio(false);
117     cin.tie(nullptr);
118
119     int n, q;
120     cin >> n >> q;
121
122     vector<ll> p(n + 1);
123     vector<ll> a(n + 1), b(n + 1);
124
125     for (int i = 1; i <= n; i++) {
126         cin >> p[i];
127
128         a[i] = p[i] - i;
129         b[i] = p[i] + i;
130     }
131
132     SegmentTree leftTree(a);
133     SegmentTree rightTree(b);
134
135     while (q--) {
136         int type, k;
137         ll x;
138
139         cin >> type >> k;
140
141         if (type == 1) {
142             cin >> x;
143
144             p[k] = x;
145
146             leftTree.update(k, x - k);
147             rightTree.update(k, x + k);
148         }
149         else {
150             ll left = leftTree.query(1, k) + k;
151             ll right = rightTree.query(k, n) - k;
152
153             cout << min(left, right) << '\n';
154         }
155     }
156 }

```

19.13 Polynomial Queries - Lazy Propagation de Progresiones Aritmeticas

C:\Users\Bartolomeo\Documents\Queries.cpp

Lazy propagation donde la actualizacion no suma una constante sino 1, 2, 3, ... a lo largo del rango. La clave es que una progresion aritmetica se describe con DOS numeros, el primer termino A y la diferencia D, y que dos progresiones se SUMAN componente a componente. Por eso el lazy es el par (A, D), y componer dos actualizaciones pendientes es sumar los pares. Al empujar a los hijos, el izquierdo recibe (A, D) tal cual, pero el derecho arranca mas adelante en la progresion: recibe (A + lenLeft*D, D). Ese corrimiento es lo unico nuevo respecto a un lazy de suma normal. La suma de la progresion sobre un nodo de largo len es $\text{len} * (2A + (\text{len} - 1) * D) / 2$, y la division es exacta siempre: si len es par, listo; si es impar, $(\text{len} - 1) * D$ es par. EL PATRON GENERAL: se puede hacer lazy de cualquier actualizacion que (1) se describa con pocos numeros, (2) se pueda componer con otra del mismo tipo, y (3) permita recalculr la suma del nodo sin bajar a las hojas. Las progresiones cumplen las tres; por eso sirve.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 const int N = 200005;
7
8 ll st[4 * N];
9 ll lazyA[4 * N];
10 ll lazyD[4 * N];
11
12 void build(int p, int l, int r, vector<ll>& a) {
13     if (l == r) {
14         st[p] = a[l];
15         return;
16     }
17
18     int m = (l + r) / 2;
19
20     build(p * 2, l, m, a);
21     build(p * 2 + 1, m + 1, r, a);
22
23     st[p] = st[p * 2] + st[p * 2 + 1];
24 }
25
26 void apply(int p, int l, int r, ll A, ll D) {
27     ll len = r - l + 1;
28
29     st[p] += len * (2 * A + (len - 1) * D) / 2;
30
31     lazyA[p] += A;
32     lazyD[p] += D;
33 }
34
35 void push(int p, int l, int r) {
36     if (lazyA[p] == 0 && lazyD[p] == 0)
37         return;
38
39     int m = (l + r) / 2;
40     int lenLeft = m - l + 1;
41
42     apply(p * 2, l, m, lazyA[p], lazyD[p]);
43
44     apply(
45         p * 2 + 1,
46         m + 1,
47         r,
48         lazyA[p] + lenLeft * lazyD[p],
49         lazyD[p]
50     );
51
52     lazyA[p] = 0;
53     lazyD[p] = 0;
54 }
55
56 void update(int p, int l, int r, int ql, int qr) {
57     if (qr < l || r < ql)
58         return;
59
60     if (ql <= l && r <= qr) {
61         ll A = 1 - ql + 1;

```

```

62     apply(p, l, r, A, 1);
63     return;
64 }
65
66 push(p, l, r);
67
68 int m = (l + r) / 2;
69
70 update(p * 2, l, m, ql, qr);
71 update(p * 2 + 1, m + 1, r, ql, qr);
72
73 st[p] = st[p * 2] + st[p * 2 + 1];
74 }
75
76 ll query(int p, int l, int r, int ql, int qr) {
77     if (qr < l || r < ql)
78         return 0;
79
80     if (ql <= l && r <= qr)
81         return st[p];
82
83     push(p, l, r);
84
85     int m = (l + r) / 2;
86
87     return query(p * 2, l, m, ql, qr)
88         + query(p * 2 + 1, m + 1, r, ql, qr);
89 }
90
91 int main() {
92     ios::sync_with_stdio(false);
93     cin.tie(nullptr);
94
95     int n, q;
96     cin >> n >> q;
97
98     vector<ll> a(n + 1);
99
100     for (int i = 1; i <= n; i++)
101         cin >> a[i];
102
103     build(1, 1, n, a);
104
105     while (q--) {
106         int type, l, r;
107         cin >> type >> l >> r;
108
109         if (type == 1)
110             update(1, 1, n, l, r);
111         else
112             cout << query(1, 1, n, l, r) << '\n';
113     }
114 }

```

19.14 Prefix Sum Queries - Segment Tree con Nodo Compuesto

C++\CS\Range Queries\Prefix Sum Queries.cpp

Segment tree donde cada nodo guarda DOS cosas: la suma del rango y el mejor prefijo del rango. Es el ejemplo mas chico de la tecnica que hace al segment tree util de verdad: si la respuesta sola no se puede combinar, se guarda tambien lo que haga falta para poder combinarla. EL MERGE ES TODO: el mejor prefijo de la union es o el mejor prefijo de la izquierda, o toda la izquierda mas el mejor prefijo de la derecha. $\max(a.pref, a.sum + b.pref)$. Sin llevar la suma no habria forma de escribir esa segunda opcion, y por eso van juntas. COMO SE DISEÑA UN NODO ASI: se escribe a mano como seria la respuesta de la union en funcion de las dos mitades, y cada cosa que aparece en esa formula y no se tiene, se agrega al nodo. Se repite hasta que cierre. Es un proceso mecanico, no hay que adivinar. CUANDO USAR: la pregunta del rango no es una operacion asociativa simple pero SI se puede componer con un poco mas de informacion. Maximo prefijo, maximo subarreglo, cantidad de parentesis sin cerrar, la subsecuencia creciente mas larga por bloques.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 struct Node {

```

```

7     ll sum, pref;
8 };
9
10 Node merge(Node a, Node b) {
11     return {
12         a.sum + b.sum,
13         max(a.pref, a.sum + b.pref)
14     };
15 }
16
17 struct SegmentTree {
18     int n;
19     vector<Node> tree;
20
21     SegmentTree(vector<ll>& a) {
22         n = a.size() - 1;
23         tree.resize(4 * n);
24         build(1, 1, n, a);
25     }
26
27     void build(int node, int l, int r, vector<ll>& a) {
28         if (l == r) {
29             tree[node] = {a[l], max(0LL, a[l])};
30             return;
31         }
32
33         int mid = (l + r) / 2;
34
35         build(node * 2, l, mid, a);
36         build(node * 2 + 1, mid + 1, r, a);
37
38         tree[node] = merge(tree[node * 2], tree[node * 2 + 1]);
39     }
40
41     void update(int node, int l, int r, int pos, ll value) {
42         if (l == r) {
43             tree[node] = {value, max(0LL, value)};
44             return;
45         }
46
47         int mid = (l + r) / 2;
48
49         if (pos <= mid)
50             update(node * 2, l, mid, pos, value);
51         else
52             update(node * 2 + 1, mid + 1, r, pos, value);
53
54         tree[node] = merge(tree[node * 2], tree[node * 2 + 1]);
55     }
56
57     Node query(int node, int l, int r, int ql, int qr) {
58         if (ql <= l && r <= qr)
59             return tree[node];
60
61         int mid = (l + r) / 2;
62
63         if (qr <= mid)
64             return query(node * 2, l, mid, ql, qr);
65
66         if (ql > mid)
67             return query(node * 2 + 1, mid + 1, r, ql, qr);
68
69         return merge(
70             query(node * 2, l, mid, ql, qr),
71             query(node * 2 + 1, mid + 1, r, ql, qr)
72         );
73     }
74
75     void update(int pos, ll value) {
76         update(1, 1, n, pos, value);
77     }
78
79     Node query(int l, int r) {

```

```

80     return query(1, 1, n, 1, r);
81 }
82 };
83
84 int main() {
85     ios::sync_with_stdio(false);
86     cin.tie(nullptr);
87
88     int n, q;
89     cin >> n >> q;
90
91     vector<ll> a(n + 1);
92
93     for (int i = 1; i <= n; i++)
94         cin >> a[i];
95
96     SegmentTree st(a);
97
98     while (q--) {
99         int type, a, b;
100         cin >> type >> a >> b;
101
102         if (type == 1) {
103             st.update(a, b);
104         } else {
105             cout << st.query(a, b).pref << '\n';
106         }
107     }
108 }

```

19.15 Range Interval Queries - Barrido Offline con Fenwick

C++\VCSES\Range Queries\Range Interval Queries.cpp

Contar cuantos elementos de $[l, r]$ tienen valor en $[a, b]$, sin actualizaciones. Es una consulta en DOS dimensiones (posicion y valor), y la tecnica es bajarla a una sola con un barrido offline. COMO: cada consulta se parte en dos eventos por la resta de prefijos sobre el VALOR, uno con signo + en b y otro con signo - en a-1. Todos los eventos y todos los elementos se ordenan por valor, y se barre de menor a mayor insertando en el Fenwick la POSICION de cada elemento que ya entro. Cuando el barrido llega a un evento, el Fenwick contiene exactamente los elementos con valor <= ese umbral, asi que una consulta de rango de posiciones responde el evento. Ese es el patron general: una de las dos dimensiones se convierte en TIEMPO (el orden del barrido) y la otra se guarda en el Fenwick. CUANDO USAR: consultas de rango en dos dimensiones sin actualizaciones. Si hubiera actualizaciones seria una tercera dimension y tocaria BIT 2D o CDQ divide and conquer.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Fenwick {
5     int n;
6     vector<int> bit;
7
8     Fenwick(int n) : n(n), bit(n + 1, 0) {}
9
10    void add(int i, int x) {
11        for (; i <= n; i += i & -i)
12            bit[i] += x;
13    }
14
15    int sum(int i) {
16        int res = 0;
17
18        for (; i > 0; i -= i & -i)
19            res += bit[i];
20
21        return res;
22    }
23 };
24
25 struct Event {
26     int value, pos, id, sign;
27
28     bool operator<(const Event& other) const {
29         return value < other.value;
30     }
31 };

```

```

32
33 int main() {
34     ios::sync_with_stdio(false);
35     cin.tie(nullptr);
36
37     int n, q;
38     cin >> n >> q;
39
40     vector<pair<int, int>> a(n);
41
42     for (int i = 1; i <= n; i++) {
43         int x;
44         cin >> x;
45         a[i - 1] = {x, i};
46     }
47
48     sort(a.begin(), a.end());
49
50     vector<Event> events;
51
52     for (int i = 0; i < q; i++) {
53         int l, r, c, d;
54         cin >> l >> r >> c >> d;
55
56         events.push_back({d, r, i, 1});
57         events.push_back({d, l - 1, i, -1});
58         events.push_back({c - 1, r, i, -1});
59         events.push_back({c - 1, l - 1, i, 1});
60     }
61
62     sort(events.begin(), events.end());
63
64     Fenwick fw(n);
65     vector<int> ans(q);
66
67     int p = 0;
68
69     for (auto e : events) {
70
71         while (p < n && a[p].first <= e.value) {
72             fw.add(a[p].second, 1);
73             p++;
74         }
75
76         ans[e.id] += e.sign * fw.sum(e.pos);
77     }
78
79     for (int x : ans)
80         cout << x << '\n';
81 }

```

19.16 Range Queries And Copies - Segment Tree Persistente

C++\VCSES\Range Queries\Range Queries and Copies.cpp

Segment tree PERSISTENTE: cada actualizacion no modifica nodos, crea copias nuevas SOLO del camino de la raiz a la hoja, $O(\log n)$ nodos, y todo lo demas se comparte con la version anterior. Asi cada version tiene su propia raiz y se puede consultar cualquiera. LO QUE HACE ESTE PROBLEMA ELEGANTE: copiar un arreglo entero (la operacion 3) cuesta $O(1)$. Basta copiar el PUNTERO a la raiz, porque las dos versiones comparten todos los nodos hasta que alguna se modifique, y en ese momento la modificacion crea caminos nuevos sin tocar la otra. Los nodos no pueden ir en el esquema $2p / 2p+1$ de siempre, porque un nodo tiene varios padres segun la version. Por eso cada nodo guarda los INDICES de sus hijos (l, r) y se sacan de un arreglo grande con un contador. EL TAMANO DEL POOL: el build usa $2n$ nodos y cada actualizacion unos $\log_2(n)+1 = 19$. Con $2 \cdot 10^5$ actualizaciones son ~4.2 millones, y el arreglo reserva 25 por posicion (5 millones). Quedarse corto aqui da un error silencioso al pisar memoria. Ocupa unos 80 MB. CUANDO USAR: "consultar el arreglo como estaba en el momento k", versiones, o la k-esima estadistica en un rango (el clasico con persistencia sobre prefijos).

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 struct Node {
7     int l, r;
8     ll sum;

```

```

9  };
10
11  const int MAXN = 200005;
12  const int MAXNODE = MAXN * 25;
13
14  Node st[MAXNODE];
15  int root[MAXN];
16  int nodes = 0;
17
18  int build(int l, int r, vector<ll>& a) {
19      int p = ++nodes;
20
21      if (l == r) {
22          st[p].sum = a[l];
23          return p;
24      }
25
26      int m = (l + r) / 2;
27
28      st[p].l = build(l, m, a);
29      st[p].r = build(m + 1, r, a);
30
31      st[p].sum = st[st[p].l].sum + st[st[p].r].sum;
32
33      return p;
34  }
35
36  int update(int old, int l, int r, int pos, ll x) {
37      int p = ++nodes;
38      st[p] = st[old];
39
40      if (l == r) {
41          st[p].sum = x;
42          return p;
43      }
44
45      int m = (l + r) / 2;
46
47      if (pos <= m)
48          st[p].l = update(st[old].l, l, m, pos, x);
49      else
50          st[p].r = update(st[old].r, m + 1, r, pos, x);
51
52      st[p].sum = st[st[p].l].sum + st[st[p].r].sum;
53
54      return p;
55  }
56
57  ll query(int p, int l, int r, int ql, int qr) {
58      if (qr < l || r < ql)
59          return 0;
60
61      if (ql <= l && r <= qr)
62          return st[p].sum;
63
64      int m = (l + r) / 2;
65
66      return query(st[p].l, l, m, ql, qr)
67          + query(st[p].r, m + 1, r, ql, qr);
68  }
69
70  int main() {
71      ios::sync_with_stdio(false);
72      cin.tie(nullptr);
73
74      int n, q;
75      cin >> n >> q;
76
77      vector<ll> a(n + 1);
78
79      for (int i = 1; i <= n; i++)
80          cin >> a[i];
81

```

```

82      root[1] = build(1, n, a);
83
84      int versions = 1;
85
86      while (q--) {
87          int type;
88          cin >> type;
89
90          if (type == 1) {
91              int k, a, x;
92              cin >> k >> a >> x;
93
94              root[k] = update(root[k], 1, n, a, x);
95          }
96          else if (type == 2) {
97              int k, a, b;
98              cin >> k >> a >> b;
99
100             cout << query(root[k], 1, n, a, b) << '\n';
101         }
102         else {
103             int k;
104             cin >> k;
105
106             root[++versions] = root[k];
107         }
108     }
109 }

```

19.17 Range Update Queries - Fenwick de Diferencias

C++\CSES\Range Queries\Range Update Queries.cpp

La vuelta de tuerca: en vez de guardar los valores, el Fenwick guarda las DIFERENCIAS. Sumar x a todo el rango $[a, b]$ son dos operaciones puntuales, $add(a, x)$ y $add(b+1, -x)$, y consultar la posición k es la suma de prefijo hasta k . O sea que el mismo Fenwick que hace "actualización puntual + consulta de rango" también hace lo contrario, "actualización de rango + consulta puntual", solo cambiando lo que se guarda. CUANDO USAR: el problema actualiza rangos completos y pregunta por posiciones sueltas. Si necesita actualizar rangos Y consultar rangos, ahí si toca el Fenwick doble o un segment tree con lazy propagation.

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  using ll = long long;
5
6  struct Fenwick {
7      int n;
8      vector<ll> bit;
9
10     Fenwick(int n) : n(n), bit(n + 1, 0) {}
11
12     void add(int i, ll x) {
13         for (; i <= n; i += i & -i)
14             bit[i] += x;
15     }
16
17     ll sum(int i) {
18         ll res = 0;
19         for (; i > 0; i -= i & -i)
20             res += bit[i];
21         return res;
22     }
23 };
24
25 int main() {
26     ios::sync_with_stdio(false);
27     cin.tie(nullptr);
28
29     int n, q;
30     cin >> n >> q;
31
32     vector<ll> arr(n + 1);
33
34     for (int i = 1; i <= n; i++)
35         cin >> arr[i];
36

```

```

37 Fenwick fw(n + 1);
38
39 while (q--) {
40     int type;
41     cin >> type;
42
43     if (type == 1) {
44         int a, b;
45         ll c;
46         cin >> a >> b >> c;
47
48         fw.add(a, c);
49         fw.add(b + 1, -c);
50     } else {
51         int k;
52         cin >> k;
53
54         cout << arr[k] + fw.sum(k) << '\n';
55     }
56 }
57 }

```

19.18 Range Updates And Sums - Lazy Propagation con Suma y Asignacion

C++ (GCC) Range Queries Range Updates and Sums.cpp

Lazy propagation con DOS tipos de actualizacion (sumar x al rango y ASIGNAR x al rango), y todo el problema es como se componen cuando se encuentran en un mismo nodo. Se mantiene el invariante "lo pendiente es: primero asignar lazySet (si hasSet), despues sumar lazyAdd": - Una ASIGNACION borra cualquier suma pendiente: applySet pone lazyAdd en 0. La asignacion pisa todo lo anterior. - Una SUMA sobre un nodo que ya tiene una asignacion pendiente no se guarda como suma: se le suma al valor asignado (lazySet += x). Asignar S y despues sumar x es asignar S + x. - Al empujar, se baja primero la asignacion y despues la suma, en ese orden. Equivocarse en cualquiera de las tres da respuestas que parecen bien en casos chicos y fallan cuando las dos operaciones se mezclan sobre el mismo rango. Se usa hasSet aparte en vez de un valor centinela porque asignar 0 es una operacion valida. EL PATRON GENERAL: con varios tipos de lazy, antes de programar se escribe la tabla de "si llega B encima de A, que queda". Si esa tabla cierra, el lazy funciona.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 const int N = 200005;
7
8 ll st[4 * N];
9 ll lazyAdd[4 * N];
10 ll lazySet[4 * N];
11 bool hasSet[4 * N];
12
13 void build(int p, int l, int r, vector<ll>& a) {
14     if (l == r) {
15         st[p] = a[l];
16         return;
17     }
18
19     int m = (l + r) / 2;
20
21     build(p * 2, l, m, a);
22     build(p * 2 + 1, m + 1, r, a);
23
24     st[p] = st[p * 2] + st[p * 2 + 1];
25 }
26
27 void applySet(int p, int l, int r, ll x) {
28     st[p] = x * (r - l + 1);
29
30     lazySet[p] = x;
31     lazyAdd[p] = 0;
32     hasSet[p] = true;
33 }
34
35 void applyAdd(int p, int l, int r, ll x) {
36     st[p] += x * (r - l + 1);
37
38     if (hasSet[p])

```

```

39         lazySet[p] += x;
40     else
41         lazyAdd[p] += x;
42 }
43
44 void push(int p, int l, int r) {
45     if (l == r)
46         return;
47
48     int m = (l + r) / 2;
49
50     if (hasSet[p]) {
51         applySet(p * 2, l, m, lazySet[p]);
52         applySet(p * 2 + 1, m + 1, r, lazySet[p]);
53
54         hasSet[p] = false;
55     }
56
57     if (lazyAdd[p] != 0) {
58         applyAdd(p * 2, l, m, lazyAdd[p]);
59         applyAdd(p * 2 + 1, m + 1, r, lazyAdd[p]);
60
61         lazyAdd[p] = 0;
62     }
63 }
64
65 void updateAdd(int p, int l, int r, int ql, int qr, ll x) {
66     if (qr < l || r < ql)
67         return;
68
69     if (ql <= l && r <= qr) {
70         applyAdd(p, l, r, x);
71         return;
72     }
73
74     push(p, l, r);
75
76     int m = (l + r) / 2;
77
78     updateAdd(p * 2, l, m, ql, qr, x);
79     updateAdd(p * 2 + 1, m + 1, r, ql, qr, x);
80
81     st[p] = st[p * 2] + st[p * 2 + 1];
82 }
83
84 void updateSet(int p, int l, int r, int ql, int qr, ll x) {
85     if (qr < l || r < ql)
86         return;
87
88     if (ql <= l && r <= qr) {
89         applySet(p, l, r, x);
90         return;
91     }
92
93     push(p, l, r);
94
95     int m = (l + r) / 2;
96
97     updateSet(p * 2, l, m, ql, qr, x);
98     updateSet(p * 2 + 1, m + 1, r, ql, qr, x);
99
100     st[p] = st[p * 2] + st[p * 2 + 1];
101 }
102
103 ll query(int p, int l, int r, int ql, int qr) {
104     if (qr < l || r < ql)
105         return 0;
106
107     if (ql <= l && r <= qr)
108         return st[p];
109
110     push(p, l, r);
111

```

```

12 int m = (l + r) / 2;
13
14 return query(p * 2, l, m, ql, qr)
15         + query(p * 2 + 1, m + 1, r, ql, qr);
16 }
17
18 int main() {
19     ios::sync_with_stdio(false);
20     cin.tie(nullptr);
21
22     int n, q;
23     cin >> n >> q;
24
25     vector<ll> a(n + 1);
26
27     for (int i = 1; i <= n; i++)
28         cin >> a[i];
29
30     build(1, 1, n, a);
31
32     while (q--) {
33         int type, l, r;
34         cin >> type >> l >> r;
35
36         if (type == 1) {
37             ll x;
38             cin >> x;
39             updateAdd(1, 1, n, l, r, x);
40         }
41         else if (type == 2) {
42             ll x;
43             cin >> x;
44             updateSet(1, 1, n, l, r, x);
45         }
46         else {
47             cout << query(1, 1, n, l, r) << '\n';
48         }
49     }
50 }

```

19.19 Range XOR Queries - Prefix XOR

C++\CSES\Range Queries\Range XOR Queries.cpp

Idem prefix sums pero con xor, y funciona por la misma razon: el xor es su propia inversa, así que $\text{pref}[r] \oplus \text{pref}[l-1]$ cancela todo lo de antes de l. Los elementos repetidos se anulan de a pares. CUANDO USAR: ver que el xor tiene inversa es lo que abre la puerta. Mismo patron para cualquier grupo abeliano: suma, xor, producto modular con inversos. Para min o max NO aplica, porque no hay nada que "deshaga" un mínimo.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int n, q;
9     cin >> n >> q;
10
11     vector<long long> pref(n + 1);
12
13     for (int i = 1; i <= n; i++) {
14         long long x;
15         cin >> x;
16         pref[i] = pref[i - 1] ^ x;
17     }
18
19     while (q--) {
20         int l, r;
21         cin >> l >> r;
22         cout << (pref[r] ^ pref[l - 1]) << '\n';
23     }
24 }

```

19.20 Salary Queries - Fenwick con Compresion Offline

C++\CSES\Range Queries\Salary Queries.cpp

Fenwick sobre VALORES en vez de posiciones, para contar cuantos salarios caen en un rango. Como los valores llegan a 10^9 hay que comprimirlos, y aqui esta el detalle que hace el problema: las consultas se leen TODAS primero, porque los valores que van a aparecer en las actualizaciones futuras tambien tienen que entrar en la compresion. Si uno comprime solo con los valores iniciales, la primera actualizacion a un valor nuevo no tiene indice donde ir. Eso obliga a que el problema sea OFFLINE: leer todo, comprimir, y despues procesar en orden. El Fenwick guarda la CANTIDAD de empleados con cada valor comprimido, así que cambiar un salario es $\text{add}(\text{viejo}, -1)$ y $\text{add}(\text{nuevo}, +1)$, y contar en un rango es la resta de dos prefijos. CUANDO USAR: contar elementos en un rango de VALORES con actualizaciones. Es el patron de "cuantos hay menores que x" con cambios. La alternativa online es un segment tree dinamico o sobre los valores sin comprimir, que gasta mas memoria pero no obliga a leer todo antes. OJO: usa structured bindings (auto [a, b]), que piden C++17. En CSES compila, pero con un g++ viejo hay que volver a .first y .second.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Fenwick {
5     int n;
6     vector<int> bit;
7
8     Fenwick(int n) : n(n), bit(n + 1, 0) {}
9
10    void add(int i, int x) {
11        for (; i <= n; i += i & -i)
12            bit[i] += x;
13    }
14
15    int sum(int i) {
16        int res = 0;
17        for (; i > 0; i -= i & -i)
18            res += bit[i];
19        return res;
20    }
21 };
22
23 int main() {
24     ios::sync_with_stdio(false);
25     cin.tie(nullptr);
26
27     int n, q;
28     cin >> n >> q;
29
30     vector<int> salary(n + 1);
31     vector<tuple<char, int, int>> queries;
32     vector<int> values;
33
34     for (int i = 1; i <= n; i++) {
35         cin >> salary[i];
36         values.push_back(salary[i]);
37     }
38
39     for (int i = 0; i < q; i++) {
40         char type;
41         int a, b;
42         cin >> type >> a >> b;
43
44         queries.push_back({type, a, b});
45
46         if (type == '!')
47             values.push_back(b);
48     }
49
50     sort(values.begin(), values.end());
51     values.erase(unique(values.begin(), values.end()), values.end());
52
53     Fenwick fw(values.size());
54
55     for (int i = 1; i <= n; i++) {
56         int pos = lower_bound(values.begin(), values.end(), salary[i])
57                 - values.begin() + 1;
58
59         fw.add(pos, 1);

```

```

60     }
61
62     for (auto [type, a, b] : queries) {
63
64         if (type == '!') {
65             int oldPos = lower_bound(values.begin(), values.end(), salary[a])
66                 - values.begin() + 1;
67
68             int newPos = lower_bound(values.begin(), values.end(), b)
69                 - values.begin() + 1;
70
71             fw.add(oldPos, -1);
72             fw.add(newPos, 1);
73
74             salary[a] = b;
75         }
76
77         else {
78             int right = upper_bound(values.begin(), values.end(), b)
79                 - values.begin();
80
81             int left = lower_bound(values.begin(), values.end(), a)
82                 - values.begin();
83
84             cout << fw.sum(right) - fw.sum(left) << '\n';
85         }
86     }
87 }

```

19.21 Static Range Minimum Queries - Sparse Table

C++\CSES\Range Queries\Static Range Minimum Queries.cpp

Sparse table: $st[k][i]$ es el mínimo de los 2^k elementos que arrancan en i . Se construye en $O(n \log n)$ partiendo cada rango en dos mitades de 2^{k-1} , y cada consulta se responde en $O(1)$ cubriendo $[l, r]$ con DOS bloques de tamaño 2^k que se solapan. Solaparse no es problema, y esa es la clave: el mínimo es IDEMPOTENTE, contar un elemento dos veces no cambia nada. CUANDO USAR: arreglo estático y operación idempotente (min, max, gcd, and, or). Consulta $O(1)$, mas rápido que cualquier segment tree. CUANDO NO: si el arreglo cambia, la sparse table no se puede actualizar y toca segment tree. Y para SUMA no sirve, porque solapar los dos bloques contaría de mas.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 int main() {
7     ios::sync_with_stdio(false);
8     cin.tie(nullptr);
9
10    int n, q;
11    cin >> n >> q;
12
13    // K = cantidad de niveles que necesitamos.
14    // st[k][i] = mínimo de 2^k elementos empezando en i.
15    int K = 1;
16
17    while ((1 << K) <= n)
18        K++;
19
20    vector<vector<ll>> st(K, vector<ll>(n + 1));
21
22    // Nivel 0: rangos de tamaño 1.
23    for (int i = 1; i <= n; i++)
24        cin >> st[0][i];
25
26    // Construimos los demas niveles.
27    for (int k = 1; k < K; k++) {
28
29        // Tamaño del rango = 2^k
30        int len = 1 << k;
31
32        for (int i = 1; i + len - 1 <= n; i++) {
33
34            // Dividimos el rango en dos partes
35            // de tamaño 2^(k-1).

```

```

36            st[k][i] = min(
37                st[k - 1][i],
38                st[k - 1][i + (len / 2)]
39            );
40        }
41    }
42
43    while (q--) {
44        int a, b;
45        cin >> a >> b;
46
47        int len = b - a + 1;
48
49        // Mayor potencia de 2 que cabe en len.
50        int k = 0;
51
52        while ((1 << (k + 1)) <= len)
53            k++;
54
55        // Primer bloque de tamaño 2^k
56        ll x = st[k][a];
57
58        // Segundo bloque de tamaño 2^k,
59        // empezando desde el final.
60        ll y = st[k][b - (1 << k) + 1];
61
62        cout << min(x, y) << '\n';
63    }
64 }

```

19.22 Static Range Sum Queries - Prefix Sums

C++\CSES\Range Queries\Static Range Sum Queries.cpp

La estructura mas barata que existe: $pref[i]$ = suma de los primeros i , y la suma de $[a, b]$ sale como $pref[b] - pref[a - 1]$. Construcción $O(n)$, consulta $O(1)$, y ni una línea de estructura. CUANDO USAR: el arreglo NO cambia y la operación tiene INVERSA (suma, xor, producto sin ceros). Eso es todo. Si hay actualizaciones se necesita Fenwick; si la operación no tiene inversa (min, max, gcd) no se puede restar y toca sparse table o segment tree. Lo del índice 1 no es capricho: con $pref$ de tamaño $n + 1$ y $pref[0] = 0$ desaparece el if del borde cuando $a = 1$. Y long long, porque n valores de 10^9 se pasan de int sin avisar.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 int main() {
7     ios::sync_with_stdio(false);
8     cin.tie(nullptr);
9
10    ll n, q;
11    cin >> n >> q;
12
13    vector<ll> pref(n + 1);
14
15    for (int i = 1; i <= n; i++) {
16        ll x;
17        cin >> x;
18        pref[i] = pref[i - 1] + x;
19    }
20
21    while (q--) {
22        ll a, b;
23        cin >> a >> b;
24
25        cout << pref[b] - pref[a - 1] << '\n';
26    }
27 }

```

19.23 Sub Array Sum Queries - Segment Tree de Maximo Subarreglo

C++ (CSES) Range Queries SubArray Sum Queries.cpp

El nodo compuesto clasico, con cuatro campos: sum, pref (mejor prefijo), suff (mejor sufijo) y best (el mejor subarreglo de adentro). Es Kadane metido en un segment tree, y por eso aguanta actualizaciones que Kadane suelto no aguanta. EL MERGE, que es lo unico que hay que entender: $sum = L.sum + R.sum$, $pref = \max(L.pref, L.sum + R.pref)$ el prefijo se queda o cruza $suff = \max(R.suff, R.sum + L.suff)$ el sufijo, simetrico $best = \max(L.best, R.best, L.suff + R.pref)$ o esta en una mitad, o CRUZA el corte Ese tercer caso del best es la razon de llevar suff y pref: el subarreglo que cruza es el mejor sufijo de la izquierda pegado al mejor prefijo de la derecha. El $\max(0, a[i])$ en las hojas permite el subarreglo VACIO, o sea que la respuesta nunca es negativa. Si el problema exige al menos un elemento hay que sacar ese max y poner la hoja en $a[i]$ pelado. CUANDO USAR: maximo subarreglo con actualizaciones. Sin actualizaciones basta Kadane en $O(n)$.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 struct Node {
7     ll sum, pref, suff, best;
8 };
9
10 Node merge(Node L, Node R) {
11     Node res;
12
13     res.sum = L.sum + R.sum;
14
15     res.pref = max(L.pref, L.sum + R.pref);
16
17     res.suff = max(R.suff, R.sum + L.suff);
18
19     res.best = max({
20         L.best,
21         R.best,
22         L.suff + R.pref
23     });
24
25     return res;
26 }
27
28 struct SegmentTree {
29     int n;
30     vector<Node> tree;
31
32     SegmentTree(vector<ll>& a) {
33         n = a.size() - 1;
34         tree.resize(4 * n);
35         build(1, 1, n, a);
36     }
37
38     void build(int node, int l, int r, vector<ll>& a) {
39         if (l == r) {
40             tree[node] = {
41                 a[l],
42                 max(0LL, a[l]),
43                 max(0LL, a[l]),
44                 max(0LL, a[l])
45             };
46             return;
47         }
48
49         int mid = (l + r) / 2;
50
51         build(node * 2, l, mid, a);
52         build(node * 2 + 1, mid + 1, r, a);
53
54         tree[node] = merge(tree[node * 2], tree[node * 2 + 1]);
55     }
56
57     void update(int node, int l, int r, int pos, ll value) {
58         if (l == r) {
59             tree[node] = {
60                 value,
61                 max(0LL, value),
```

```
62                 max(0LL, value),
63                 max(0LL, value)
64             };
65             return;
66         }
67
68         int mid = (l + r) / 2;
69
70         if (pos <= mid)
71             update(node * 2, l, mid, pos, value);
72         else
73             update(node * 2 + 1, mid + 1, r, pos, value);
74
75         tree[node] = merge(tree[node * 2], tree[node * 2 + 1]);
76     }
77
78     void update(int pos, ll value) {
79         update(1, 1, n, pos, value);
80     }
81
82     ll answer() {
83         return tree[1].best;
84     }
85 };
86
87 int main() {
88     ios::sync_with_stdio(false);
89     cin.tie(nullptr);
90
91     int n, m;
92     cin >> n >> m;
93
94     vector<ll> a(n + 1);
95
96     for (int i = 1; i <= n; i++)
97         cin >> a[i];
98
99     SegmentTree st(a);
100
101     while (m--) {
102         int k;
103         ll x;
104
105         cin >> k >> x;
106
107         st.update(k, x);
108
109         cout << st.answer() << '\n';
110     }
111 }
```

19.24 Sub Array Sum Queries II - Maximo Subarreglo en un Rango

C++ (CSES) Range Queries SubArray Sum Queries II.cpp

El MISMO nodo de cuatro campos que "SubArray Sum Queries", con el merge identico, pero respondiendo otra pregunta: alla se actualiza una posicion y se pide el mejor subarreglo de TODO el arreglo (se lee la raiz), y aqui no hay actualizaciones y se pide el mejor subarreglo DENTRO de un rango $[a,b]$ (se combinan los nodos que cubren el rango). POR ESO VALE LA PENA TENER LOS DOS JUNTOS: es la misma estructura sirviendo para dos problemas distintos, y lo unico que cambia es de donde se lee la respuesta. Si el merge esta bien escrito, que la consulta sea global o de rango no le importa a la estructura. La leccion practica: cuando se disena un nodo compuesto, conviene escribir la consulta de rango aunque el problema solo pida la raiz. Cuesta cinco lineas mas y cubre las dos variantes.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 struct Node {
7     ll sum, pref, suff, best;
8 };
9
10 Node merge(Node L, Node R) {
11     return {
```

```

12     L.sum + R.sum,
13     max(L.pref, L.sum + R.pref),
14     max(R.suff, R.sum + L.suff),
15     max({
16         L.best,
17         R.best,
18         L.suff + R.pref
19     })
20 };
21 }
22
23 struct SegmentTree {
24     int n;
25     vector<Node> tree;
26
27     SegmentTree(vector<ll>& a) {
28         n = a.size() - 1;
29         tree.resize(4 * n);
30         build(1, 1, n, a);
31     }
32
33     void build(int node, int l, int r, vector<ll>& a) {
34         if (l == r) {
35             tree[node] = {
36                 a[l],
37                 max(0LL, a[l]),
38                 max(0LL, a[l]),
39                 max(0LL, a[l])
40             };
41             return;
42         }
43
44         int mid = (l + r) / 2;
45
46         build(node * 2, l, mid, a);
47         build(node * 2 + 1, mid + 1, r, a);
48
49         tree[node] = merge(tree[node * 2], tree[node * 2 + 1]);
50     }
51
52     Node query(int node, int l, int r, int ql, int qr) {
53         if (ql <= l && r <= qr)
54             return tree[node];
55
56         int mid = (l + r) / 2;
57
58         if (qr <= mid)
59             return query(node * 2, l, mid, ql, qr);
60
61         if (ql > mid)
62             return query(node * 2 + 1, mid + 1, r, ql, qr);
63
64         Node left = query(node * 2, l, mid, ql, qr);
65         Node right = query(node * 2 + 1, mid + 1, r, ql, qr);
66
67         return merge(left, right);
68     }
69
70     Node query(int l, int r) {
71         return query(1, 1, n, l, r);
72     }
73 };
74
75 int main() {
76     ios::sync_with_stdio(false);
77     cin.tie(nullptr);
78
79     int n, q;
80     cin >> n >> q;
81
82     vector<ll> a(n + 1);
83
84     for (int i = 1; i <= n; i++)

```

```

85         cin >> a[i];
86
87     SegmentTree st(a);
88
89     while (q--) {
90         int a, b;
91         cin >> a >> b;
92
93         cout << st.query(a, b).best << '\n';
94     }
95 }

```

19.25 Visible Building Queries - Monotonic Stack + Binary Lifting

C++ (VS2019) Range Queries (Visible Building Queries.cpp)

Dos técnicas encadenadas, y la combinación es lo que vale. Primero una PILA MONOTONA de derecha a izquierda calcula $nxt[i]$, el siguiente edificio mas alto que i . Eso deja un arbol implicito donde el padre de i es $nxt[i]$: desde i , los edificios que se ven son i , $nxt[i]$, $nxt[nxt[i]]$... Y como hay que contar cuantos de esos caben antes de pasarse de r , se le monta BINARY LIFTING encima: $up[j][i]$ es el resultado de saltar 2^j veces, así que contar los saltos se hace en $O(\log n)$ bajando de la potencia mas alta a la mas baja, en vez de caminar uno por uno. CUANDO USAR CADA UNA: la pila monotona para "el siguiente mayor o menor" en $O(n)$, que es la base de los histogramas y de los rangos donde un elemento es el maximo. El binary lifting cuando hay que seguir una cadena de punteros MUCHAS veces y cada salto es fijo: el mismo mecanismo del LCA. LA IDEA QUE SE REPITE: si un puntero define una funcion $i \rightarrow f(i)$, entonces las potencias de esa funcion se precaculan con doubling y cualquier cantidad de saltos sale en $O(\log n)$. Sirve para LCA, para funciones de permutacion y para los "k pasos adelante" de cualquier grafo funcional.

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      ios::sync_with_stdio(false);
6      cin.tie(nullptr);
7
8      int n, q;
9      cin >> n >> q;
10
11     vector<int> h(n + 1);
12
13     for (int i = 1; i <= n; i++)
14         cin >> h[i];
15
16     vector<int> nxt(n + 1, 0);
17     stack<int> st;
18
19     for (int i = n; i >= 1; i--) {
20         while (!st.empty() && h[st.top()] <= h[i])
21             st.pop();
22
23         if (!st.empty())
24             nxt[i] = st.top();
25
26         st.push(i);
27     }
28
29     int LOG = 18;
30     vector<vector<int>> up(LOG, vector<int>(n + 1));
31
32     for (int i = 1; i <= n; i++)
33         up[0][i] = nxt[i];
34
35     for (int j = 1; j < LOG; j++) {
36         for (int i = 1; i <= n; i++) {
37             if (up[j - 1][i])
38                 up[j][i] = up[j - 1][up[j - 1][i]];
39         }
40     }
41
42     while (q--) {
43         int a, b;
44         cin >> a >> b;
45
46         int pos = a;
47         int ans = 1;
48

```

```

49     for (int j = LOG - 1; j >= 0; j--) {
50         int x = up[j][pos];
51
52         if (x != 0 && x <= b) {
53             pos = x;
54             ans += (1 << j);
55         }
56     }
57
58     cout << ans << '\n';
59 }
60 }

```

20 - CSES: String Algorithms

20.1 All Palindromes - Manacher (Palindromo Mas Largo que Termina en Cada Posicion)

C++\CSES\String Algorithms\All Palindromes.cpp

Para cada posicion, el palindromo mas largo que TERMINA ahi. Primero Manacher con centinelas distintos en los extremos ('\$' al inicio, '^' al final), que es el truco para que el while de expansion no necesite chequear limites: al llegar a los bordes los centinelas nunca coinciden. EL BARRIDO QUE LO RESUELVE: si un palindromo centrado en i llega hasta j o mas, entonces tambien hay un palindromo centrado en i que termina EXACTAMENTE en j (se recorta simetrico). Y entre dos centros que alcanzan j, el de mas a la izquierda da el palindromo mas largo. Asi que para cada j basta el PRIMER centro, de izquierda a derecha, que lo alcanza. Por eso el puntero j nunca retrocede: cada centro asigna las posiciones nuevas que alcanza y que nadie antes alcanzo. Cada posicion se asigna una vez por pasada, y todo queda O(n). Se hace una pasada para impares y otra para pares, quedandose con el maximo.

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  vector<int> manacher(string s) {
5      string t = "$";
6
7      for (char c : s) {
8          t += '#';
9          t += c;
10     }
11
12     t += "#^";
13
14     vector<int> p(t.size());
15
16     int l = 1;
17     int r = 1;
18
19     for (int i = 1; i < (int)t.size() - 1; i++) {
20
21         p[i] = max(0, min(r - i, p[l + r - i]));
22
23         while (t[i - p[i]] == t[i + p[i]])
24             p[i]++;
25
26         if (i + p[i] > r) {
27             l = i - p[i];
28             r = i + p[i];
29         }
30     }
31
32     return p;
33 }
34
35 int main() {
36     ios::sync_with_stdio(false);
37     cin.tie(nullptr);
38
39     string s;
40     cin >> s;
41
42     vector<int> p = manacher(s);
43
44     int n = s.size();
45

```

```

46     vector<int> ans(n);
47
48     /*
49     PALINDROMOS IMPARES
50
51     i representa el centro en la cadena original.
52
53     p[(i + 1) * 2] / 2
54     nos dice cuanto puede extenderse el palindromo.
55
56     Para cada centro:
57
58     i, i+1, i+2, ...
59
60     podemos obtener palindromos de longitudes:
61
62     1, 3, 5, 7, ...
63
64     Pero usamos j para no volver a procesar posiciones
65     que ya fueron actualizadas.
66     */
67
68     int j = 0;
69
70     for (int i = 0; i < n; i++) {
71
72         j = max(j, i);
73
74         int limit = i + (p[(i + 1) * 2] / 2);
75
76         for (; j < limit; j++) {
77             ans[j] = 1 + 2 * (j - i);
78         }
79     }
80
81     /*
82     PALINDROMOS PARES
83
84     Ahora hacemos lo mismo para palindromos pares.
85
86     El centro esta entre i-1 e i.
87     */
88
89     j = 0;
90
91     for (int i = 1; i < n; i++) {
92
93         j = max(j, i);
94
95         int limit = i + (p[1 + i * 2] / 2);
96
97         for (; j < limit; j++) {
98             ans[j] = max(ans[j], 2 * (j - i + 1));
99         }
100     }
101
102     for (int x : ans)
103         cout << x << ' ';
104
105     cout << '\n';
106 }

```

20.2 Finding Borders - Funcion de Prefijos (Cadena de Bordes)

C++\CSES\String Algorithms\Finding Borders.cpp

Todos los bordes de s (prefijos que tambien son sufijos) salen de la funcion de prefijos, sin buscar nada mas. El borde mas largo es $p_i[n-1]$; y el siguiente mas largo es el borde mas largo DE ESE BORDE, o sea $p_i[p_i[n-1] - 1]$; y asi hasta llegar a 0. POR QUE LA CADENA LOS DA TODOS: un borde de s mas corto que el borde mas largo b tambien es borde de b (es prefijo de s, luego prefijo de b; y es sufijo de s, luego sufijo de b). Asi que los bordes de s son b y los bordes de b, recursivamente, y no se salta ninguno. Salen de mayor a menor, por eso el reverse al final. Esta cadena j -> $p_i[j-1]$ es la misma que usa KMP para retroceder, y es de las ideas mas reusadas de strings: aparece en periodos, en el automata de KMP y en contar apariciones de cada prefijo.

```

1  #include <bits/stdc++.h>
2  using namespace std;

```

```

3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     string s;
9     cin >> s;
10
11     int n = s.size();
12
13     vector<int> pi(n);
14
15     for (int i = 1; i < n; i++) {
16         int j = pi[i - 1];
17
18         while (j > 0 && s[i] != s[j])
19             j = pi[j - 1];
20
21         if (s[i] == s[j])
22             j++;
23
24         pi[i] = j;
25     }
26
27     vector<int> ans;
28
29     int j = pi[n - 1];
30
31     while (j > 0) {
32         ans.push_back(j);
33         j = pi[j - 1];
34     }
35
36     reverse(ans.begin(), ans.end());
37
38     for (int x : ans)
39         cout << x << ' ';
40
41     cout << '\n';
42 }

```

20.3 Finding Periods - Periodos = n - Bordes

C++\CSES\String Algorithms\Finding Periods.cpp

Un periodo p de s es un desplazamiento tal que $s[i] = s[i+p]$ para todo i valido. La equivalencia que resuelve el problema: p es periodo $\Leftrightarrow n - p$ es un borde porque " $s[i] = s[i+p]$ para todo i " dice exactamente que el prefijo de largo $n-p$ es igual al sufijo de largo $n-p$. Así que los periodos son n menos cada borde de la cadena de "Finding Borders", mas el periodo trivial n (que corresponde al borde vacio). Se juntan y se ordenan. CUANDO USAR: "la cadena es repeticion de un bloque", "el periodo mas corto", "cuantas veces se repite". El periodo mas corto es $n - pi[n-1]$; y s es una repeticion EXACTA de un bloque si y solo si ese periodo divide a n . Esa ultima condicion es la que se olvida.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 vector<int> KMP(string s) {
5     int n = s.size();
6     vector<int> pi(n);
7
8     for (int i = 1; i < n; i++) {
9         int j = pi[i - 1];
10
11         while (j > 0 && s[i] != s[j])
12             j = pi[j - 1];
13
14         if (s[i] == s[j])
15             j++;
16
17         pi[i] = j;
18     }
19
20     return pi;
21 }
22
23 int main() {

```

```

24     ios::sync_with_stdio(false);
25     cin.tie(nullptr);
26
27     string s;
28     cin >> s;
29
30     int n = s.size();
31
32     vector<int> pi = KMP(s);
33
34     vector<int> ans;
35
36     int j = pi[n - 1];
37
38     while (j > 0) {
39         ans.push_back(n - j);
40         j = pi[j - 1];
41     }
42
43     ans.push_back(n);
44
45     sort(ans.begin(), ans.end());
46
47     for (int x : ans)
48         cout << x << ' ';
49
50     cout << '\n';
51 }

```

20.4 Longest Palindrome - Manacher

C++\CSES\String Algorithms\Longest Palindrome.cpp

Manacher: el palindromo mas largo en $O(n)$. Se intercala '#' entre letras para que los palindromos pares y los impares se vuelvan todos impares, y para cada centro se guarda $p[i]$, el radio del palindromo mas largo ahí. LO QUE LO HACE LINEAL: se mantiene el palindromo que llega mas a la derecha (center, right). Si i cae dentro de él, su espejo $2*center - i$ ya se calculo, y $p[i]$ arranca en $\min(right - i, p[espejo])$ en vez de en 0. Solo se expande lo que sobresale de right, y right nunca retrocede. Para volver a la cadena original: con separadores, el palindromo de radio $p[i]$ centrado en i mide $p[i]$ letras reales y empieza en $(i - p[i]) / 2$. OJO, ESTE ARCHIVO TIENE UN BUCLE DE DEPURACION AL FINAL: despues de imprimir la respuesta recorre p e imprime $p[i]$ para cada i impar. Con la entrada "aybabtu" sale "bab" y 7 lineas de mas (comprobado). En CSES eso es Wrong Answer; hay que borrar ese for antes de enviar.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 vector<int> manacher(string s) {
5     string t = "#";
6
7     for (char c : s) {
8         t += c;
9         t += '#';
10    }
11
12    int n = t.size();
13
14    vector<int> p(n);
15
16    int center = 0;
17    int right = 0;
18
19    for (int i = 0; i < n; i++) {
20        int mirror = 2 * center - i;
21
22        if (i < right)
23            p[i] = min(right - i, p[mirror]);
24
25        while (i + 1 + p[i] < n &&
26              i - 1 - p[i] >= 0 &&
27              t[i + 1 + p[i]] == t[i - 1 - p[i]]) {
28            p[i]++;
29        }
30
31        if (i + p[i] > right) {
32            center = i;

```

```

33         right = i + p[i];
34     }
35 }
36
37 return p;
38 }
39
40 int main() {
41     ios::sync_with_stdio(false);
42     cin.tie(nullptr);
43
44     string s;
45     cin >> s;
46
47     vector<int> p = manacher(s);
48
49     int pos = max_element(p.begin(), p.end()) - p.begin();
50     int len = p[pos];
51
52     int start = (pos - len) / 2;
53
54     cout << s.substr(start, len) << '\n';
55
56     for(int i = 0; i < p.size(); i++)
57     {
58         if(i%2!=0)
59         {
60             cout << p[i] << endl;
61         }
62     }
63
64
65     return 0;
66 }

```

20.5 Minimal Rotation - Rotacion Minima (Dos Punteros)

C++\CSES\String Algorithms\Minimal Rotation.cpp

La rotacion lexicograficamente menor en $O(n)$ con dos candidatos i y j . Se trabaja sobre $s + s$ (asi toda rotacion es un substring de largo n) y se comparan las rotaciones que empiezan en i y en j caracter por caracter hasta que difieren en la posicion k . EL SALTO QUE HACE QUE SEA LINEAL: si $s[i+k] > s[j+k]$, no solo la rotacion en i pierde; tambien pierden las que empiezan en $i+1, \dots, i+k$, porque cada una se puede emparejar con la que empieza en $j+1, \dots, j+k$ y pierde por la misma posicion. Por eso i salta a $i+k+1$ de una. Como i y j solo avanzan y ninguno pasa de n , el total es $O(n)$. Si k llega a n , las dos rotaciones son iguales (la cadena es periodica) y cualquiera sirve. El if ($i == j$) $j++$ evita comparar un candidato consigo mismo. Alternativas: el algoritmo de Booth (con funcion de fallo) o el sufijo minimo de Lyndon. Esta version de dos punteros es la mas corta de escribir y la mas facil de recordar.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 string minimalRotation(string s) {
5     int n = s.size();
6
7     s += s;
8
9     int i = 0, j = 1;
10
11     while (i < n && j < n) {
12         int k = 0;
13
14         while (k < n && s[i + k] == s[j + k])
15             k++;
16
17         if (k == n)
18             break;
19
20         if (s[i + k] > s[j + k])
21             i = i + k + 1;
22         else
23             j = j + k + 1;
24
25         if (i == j)
26             j++;

```

```

27     }
28
29     int pos = min(i, j);
30
31     return s.substr(pos, n);
32 }
33
34 int main() {
35     ios::sync_with_stdio(false);
36     cin.tie(nullptr);
37
38     string s;
39     cin >> s;
40
41     cout << minimalRotation(s) << '\n';
42 }

```

20.6 Required Substring - DP sobre el Automata de KMP

C++\CSES\String Algorithms\Required Substring.cpp

Cuenta cuantas cadenas de largo n sobre un alfabeto de 26 letras CONTIENEN un patron dado, modulo $1e9+7$. LA IDEA CENTRAL: en vez de contar las que lo contienen, se cuenta con una DP donde el estado es "cuanto del patron llevo casado". Se construye el AUTOMATA de KMP: $nxt[j][c]$ dice a que estado se pasa si llevo j caracteres del patron casados y llega la letra c . Eso se calcula con la funcion de prefijos, retrocediendo igual que en la busqueda de KMP. Con el automata, la DP es directa: $dp[i][j]$ = cuantas cadenas de largo i dejan el automata en el estado j . Desde cada estado se prueban las 26 letras y se avanza. Al llegar al estado m (patron completo) esa cadena ya sirve, y todas las formas de completarla tambien: se suman de golpe multiplicando por $26^{\text{(lo que falta)}}$, sin seguir simulando. POR QUE HACE FALTA EL AUTOMATA Y NO BASTA CONTAR: dos cadenas distintas pueden dejar el mismo "sufijo util" del patron, y de ahi en adelante se comportan igual. El estado de KMP es exactamente esa informacion, y es lo que colapsa un espacio de 26^n a $n \cdot m$ estados. La construccion de nxt se hace UNA vez, en $O(m \cdot 26)$, retrocediendo con pi ; hacerlo dentro de la DP seria pagarlo n veces. Este patron (automata de KMP + DP) es el que resuelve toda la familia de "cuantas cadenas contienen / evitan un patron", y con varios patrones a la vez el equivalente es Aho-Corasick.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 const ll MOD = 1000000007;
7
8
9 // -----
10 // KMP - Prefix Function
11 // -----
12
13 vector<int> prefixFunction(const string& s) {
14
15     int n = s.size();
16
17     vector<int> pi(n);
18
19     for (int i = 1; i < n; i++) {
20
21         int j = pi[i - 1];
22
23         while (j > 0 && s[i] != s[j])
24             j = pi[j - 1];
25
26         if (s[i] == s[j])
27             j++;
28
29         pi[i] = j;
30     }
31
32     return pi;
33 }
34
35
36 // -----
37 // Transiciones de KMP
38 // -----
39
40 vector<vector<int>> buildKMPTransitions(const string& p) {
41

```

```

42 int m = p.size();
43
44 vector<int> pi = prefixFunction(p);
45
46 vector<vector<int>> nxt(m, vector<int>(26));
47
48 for (int j = 0; j < m; j++) {
49     for (int c = 0; c < 26; c++) {
50         char x = 'A' + c;
51
52         int k = j;
53
54         while (k > 0 && p[k] != x)
55             k = pi[k - 1];
56
57         if (p[k] == x)
58             k++;
59
60         nxt[j][c] = k;
61     }
62 }
63
64 return nxt;
65 }
66
67 // -----
68 // Main
69 // -----
70
71 int main() {
72     ios::sync_with_stdio(false);
73     cin.tie(nullptr);
74
75     int n;
76     string p;
77
78     cin >> n >> p;
79
80     int m = p.size();
81
82     auto nxt = buildKMPTransitions(p);
83
84     // Estados:
85     // 0 ... m-1 = cuanto del patron tenemos
86     // m = el patron ya aparecio
87     vector<ll> dp(m + 1);
88     vector<ll> ndp(m + 1);
89
90     dp[0] = 1;
91
92     for (int len = 0; len < n; len++) {
93         fill(ndp.begin(), ndp.end(), 0);
94
95         for (int j = 0; j <= m; j++) {
96             if (dp[j] == 0)
97                 continue;
98
99             // Ya aparecio el patron.
100             // Cualquier caracter que agreguemos mantiene
101             // este estado.
102             if (j == m) {
103                 ndp[m] += dp[j] * 26;
104                 ndp[m] %= MOD;
105
106                 continue;
107             }
108         }
109     }

```

```

106 // Todavia no aparecio.
107 for (int c = 0; c < 26; c++) {
108     int k = nxt[j][c];
109
110     ndp[k] += dp[j];
111     ndp[k] %= MOD;
112 }
113
114 dp.swap(ndp);
115 }
116
117 cout << dp[m] << '\n';
118
119 return 0;
120 }

```

20.7 String Matching - KMP (Busqueda de Patron)

C++\CSES\String Algorithms\String Matching.cpp

KMP: cuantas veces aparece p en s , en $O(n + m)$. Primero la funcion de prefijos de p , $pi[i]$ = el largo del borde mas largo de $p[0..i]$. Despues se recorre s manteniendo j , cuanto del patron ya coincide; cuando falla, en vez de volver a empezar se retrocede a $pi[j-1]$, que es cuanto del patron sigue coincidiendo gratis. DOS DETALLES QUE DECIDEN SI CUENTA BIEN: despues de un match se hace $j = pi[j-1]$ y no $j = 0$, y eso es lo que permite contar apariciones SOLAPADAS ("aa" en "aaa" son dos). Y el while de retroceso va ANTES del if que avanza, siempre. Por que es lineal: j sube a lo sumo 1 por caracter, y cada retroceso lo baja; no puede bajar mas de lo que subio. CUANDO USAR: buscar un patron fijo. Si son muchos patrones a la vez, Aho-Corasick; si hay que comparar substrings arbitrarios, hashing; si piden los bordes o periodos, la misma pi sirve (ver "Finding Borders" y "Finding Periods").

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 // Retorna cuantas veces aparece p dentro de s.
5 // Permite apariciones solapadas.
6 int KMP(const string& s, const string& p) {
7
8     int n = s.size();
9     int m = p.size();
10
11     // Prefix function del patron
12     vector<int> pi(m);
13
14     for (int i = 1; i < m; i++) {
15         int j = pi[i - 1];
16
17         while (j > 0 && p[i] != p[j])
18             j = pi[j - 1];
19
20         if (p[i] == p[j])
21             j++;
22
23         pi[i] = j;
24     }
25
26     // Buscar p dentro de s
27     int ans = 0;
28     int j = 0;
29
30     for (int i = 0; i < n; i++) {
31         while (j > 0 && s[i] != p[j])
32             j = pi[j - 1];
33
34         if (s[i] == p[j])
35             j++;
36
37         if (j == m) {
38             ans++;
39
40             // Importante para permitir solapamientos
41             j = pi[j - 1];
42         }
43     }
44 }

```

```

44     }
45 }
46
47 return ans;
48 }
49
50 int main() {
51
52     ios::sync_with_stdio(false);
53     cin.tie(nullptr);
54
55     string s, p;
56
57     cin >> s >> p;
58
59     cout << KMP(s, p) << '\n';
60
61     return 0;
62 }

```

20.8 Word Combinations - Trie + DP de Segmentacion (Conteo)

C++\CSES\String Algorithms\Word Combinations.cpp

Cuántas formas hay de partir s en palabras del diccionario. $dp[i]$ = formas de armar el prefijo $s[0..i]$, con $dp[0] = 1$, y cada palabra que empiece en i y termine en j suma $dp[i]$ a $dp[j+1]$. EL TRIE ES LO QUE HACE QUE QUEPA: probar cada palabra del diccionario en cada posición sería $O(n * k * \text{largo})$. Con las palabras en un trie, desde cada i se camina UNA vez por el trie siguiendo s , y cada nodo marcado como fin de palabra en el camino es una palabra que encaja. El recorrido se corta apenas el trie no tiene la letra, así que desde cada i se avanza a lo sumo lo que mide la palabra mas larga. El if ($dp[i] == 0$) continue ahorra caminar desde posiciones a las que no se puede llegar. CUANDO USAR: segmentar un texto en palabras de un diccionario, contando formas (como aquí) o maximizando un puntaje (como "E - Spacebar Tokenizer" de ICPC 2024, que es este mismo esqueleto con max en vez de suma).

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  const long long MOD = 1e9 + 7;
5
6  struct Node {
7      int nxt[26];
8      bool end;
9
10     Node() {
11         memset(nxt, -1, sizeof(nxt));
12         end = false;
13     }
14 };
15
16 int main() {
17     ios::sync_with_stdio(false);
18     cin.tie(nullptr);
19
20     string s;
21     cin >> s;
22
23     int k;
24     cin >> k;
25
26     vector<Node> trie(1);
27
28     while (k--) {
29         string w;
30         cin >> w;
31
32         int v = 0;
33
34         for (char c : w) {
35             int x = c - 'a';
36
37             if (trie[v].nxt[x] == -1) {
38                 trie[v].nxt[x] = trie.size();
39                 trie.emplace_back();
40             }
41

```

```

42             v = trie[v].nxt[x];
43         }
44
45         trie[v].end = true;
46     }
47
48     int n = s.size();
49
50     vector<long long> dp(n + 1);
51     dp[0] = 1;
52
53     for (int i = 0; i < n; i++) {
54         if (dp[i] == 0) continue;
55
56         int v = 0;
57
58         for (int j = i; j < n; j++) {
59             int x = s[j] - 'a';
60
61             if (trie[v].nxt[x] == -1)
62                 break;
63
64             v = trie[v].nxt[x];
65
66             if (trie[v].end) {
67                 dp[j + 1] += dp[i];
68
69                 if (dp[j + 1] >= MOD)
70                     dp[j + 1] -= MOD;
71             }
72         }
73
74         cout << dp[n] << '\n';
75     }
76 }

```

Indice alfabetico de problemas

A

A Healthy Menu	Day 6	5.11
A Mazing!	Day 11	3.19
A Story About Tea	Day 7	13.13
A String Problem	Codeforces	9.17
Accounting Qualifying	ICPC 2025	13.17
Accounting Qualifying Con Prefix	ICPC 2025	1.15
Aesthetics In Poetry	Day 7	6.8
Algebra Y Ecuaciones	Formulario	14.1
All Palindromes	CSES	20.1
Antiarithmetic	Day 11	5.5
Apple Divisions	CSES	18.1
Apple Pie	ICPC	16.1
Aquarium Bubble Lights	Codeforces	1.10
Archmage's Crystals	Day 9	6.7
Area Circulo Poligono	Notebook	4.2
Areas Y Volumes	Formulario	14.2
AROD	RPC	4.13
Artic Virus	ICPC 2024	2.9
Artwork	Day 1	3.17
Attacking Rooks	Day 7	3.8
Attention To Meeting	Day 3	5.2

B

Bacon Number	Day 3	3.5
Balanced Balloons	ICPC	16.2
Balatro	Codeforces	10.18
Beautiful Words	Day 4	9.15
Binary Dozens	ICPC 2025	6.15
Binary Dozens	ICPC 2025	16.3
Binary Tree	LeetCode	1.22
Binomial Coefficients	Day 10	7.1
Bit Strings	CSES	18.2
Black Hills Golden Jewels	Day 5	11.2
Block Placement Queries	LeetCode	1.16
Boss	Day 2	3.4
Bridge Juan	ICPC 2024	10.14
Broul	CCPL	6.12
Brute	CCPL	3.29
Buffoon	Day 1	13.6
Building Roads	CSES	17.1
Building Teams	CSES	17.2
Buses	Day 2	5.14
Buy Low Sell High	Notebook	10.4
Bytelandia's Stones	Day 9	7.15

C

Cahokia Ruins	Day 5	5.22
Cake Hater	Day 10	13.8
Celestial Veins	ICPC 2025	4.6
Cerros	Codeforces	10.15
Chequeo De Palindromo	Notebook	9.2
Chessboard And Queens	CSES	18.3
Chinese Innovation	Day 10	3.21

Clean Streets	ICPC	16.4
Coach	Codeforces	5.17
Coin Piles	CSES	18.4
Collatz Polynomial	Day 6	13.4
Colmillos	Codeforces	3.32
Combinatoria	Formulario	14.3
Combinatoria Avanzada	Notebook	7.2
Compresion De Coordenadas	Notebook	1.2
Container With Most Water	Notebook	10.21
Convex Hull Trick	Notebook	2.5
Coordenadas Polares	Notebook	4.4
Cotas Y Limites	Formulario	14.4
Count Majority Subarrays	LeetCode	1.1
Count Majority Subarrays2	LeetCode	1.14
Counting Divisors	Day 10	6.16
Counting Rooms	CSES	17.3
Couple Of Bip Bop	Day 3	9.14
Course Schedule	CSES	17.4
Coven Complications	Day 11	10.20
Creating Multiples	Day 4	6.13
Creating Strings	CSES	18.5
Criba De Eratostenes (Solo Impares)	Notebook	6.4
Crochet	RPC	13.16
Cycle Finding	CSES	17.5

D

D Query	Day 5	1.5
Dangerous Dive	Day 2	1.9
Datos Que No Se Deducen	Formulario	14.5
Denouncing Mafia	Day 1	3.18
Digit DP	Notebook	2.6
Digit Queries	CSES	18.6
Digitsduel	Codeforces	8.3
Discrete Catalog	ICPC 2025	7.13
Displaying Decimals	ICPC	16.5
Distancia De Damerau Levenshtein	Notebook	2.7
Distinct Colors	Day 4	1.19
Distinct Value Queries II	CSES	19.1
Distinct Values Queries	CSES	19.2
Divide And Conquer Optimization	Notebook	2.8
Division Digitos Distintos	CCPL	13.18
Doubled Sequence II	Day 9	12.2
Download Speed	Day 7	3.26
Drug Test	ICPC 2024	5.1
Dynamic Range Minimum Queries	CSES	19.3
Dynamic Range Sum Queries	CSES	19.4

E

Eeny Meeny	Day 11	13.10
Efficient Encoding	ICPC 2025	10.8
El Juego Del Calamar	Day 9	2.3
Eliminacion Gaussiana	Notebook	5.7
Emergency Rations	ICPC	16.6
Enigma	Day 8	9.1

Enigma Of The Jewelry Case	Day 3	13.23
Escalator	Day 4	13.19
Exhibition Of Clownfish	Day 1	10.6
Expansion Of The Road Network	Day 6	3.33
Exponentiation II	Day 10	6.9
Ezraft And Array	Codeforces	5.6

F

Factorizacion En Digitos	Notebook	6.10
Fantastic Robot	Day 9	3.20
Farmpiggie And Subset Sum	Codeforces	7.3
Fibonacci Sum	Day 10	5.8
Find Ladders	LeetCode	3.6
Finding Borders	CSES	20.2
Finding Forks	Day 11	10.19
Finding Periods	CSES	20.3
Fingerprints	ICPC 2025	9.12
Flight Discount	CSES	17.6
Flight Routes	CSES	17.7
Folding Machine	Day 2	3.30
Forest Queries	CSES	19.5
Forest Queries II	CSES	19.6
Fractions Are Better When Continue	Day 3	5.19
Frangolino Ali Na Mesa	Day 6	7.11
Frequent	CCPL	1.20
Frosh	CCPL	1.3
Frosh Merge Sort	CCPL	15.1
Fuzzy Factorization	ICPC	16.7

G

Game Routes	CSES	17.8
Gasoline	Day 8	3.28
Gatuno's Descent Into Psychopathy	Day 9	5.13
Geometria	Formulario	14.6
Getting In Shape	Day 4	5.9
Gray Code	CSES	18.7
Grid Coloring I	CSES	18.8
Grid Path Description	CSES	18.9
Gridoland Power Gauge	ICPC	16.8
Group Work	Day 7	7.14
Guard Deployment	ICPC 2025	4.3

H

Handling The Blocks	Day 4	12.1
Harder Horizons	ICPC	13.14
Harmonics With Interference	Day 3	13.7
Has Valid Path	LeetCode	2.4
Hash Map Y Ordenamiento	Notebook	1.8
Hedge Mazes	Day 4	3.7
High Score	CSES	17.9
Holes	Day 4	1.21
Holy Network	ICPC 2025	3.12
Holy Network (clique Iterativo)	ICPC 2025	3.13
Holy Network (DSU, Incorrecta)	ICPC 2025	3.22
Hotel Queries	CSES	19.7

Indice alfabetico (cont.)

Hour For A Run	Day 1	5.12	Means	Notebook	4.7	Range Interval Queries	CSES	19.15
How Many Teams	Day 6	2.12	Meme Wars	Day 7	5.4	Range Queries And Copies	CSES	19.16
Huffman	Notebook	10.7	Message Route	CSES	17.13	Range Update Queries	CSES	19.17
I			Metal Detector	Day 5	5.21	Range Updates And Sums	CSES	19.18
Impossible Primebox	ICPC 2025	13.22	Mex Grid Construction	CSES	18.12	Range XOR Queries	CSES	19.19
Incompatible Pairs	Day 10	10.11	Minas Gerais' Walls	Day 6	11.1	Rectas Y Conicas	Formulario	14.8
Increasing Array	CSES	18.10	Minimal Rotation	CSES	20.5	Remainders	RPC	6.19
Increasing Array Queries	CSES	19.8	Misere Nim Y Grundy Acotado	Notebook	8.1	Remove Duplicates	LeetCode	9.11
Infiltration Route	ICPC	16.9	Missing Coin Sum Queries	CSES	19.10	Remove Duplicates1	LeetCode	9.6
Ingredients That May Harm You	Day 3	6.5	Missing Number	CSES	18.13	Repetitions	CSES	18.18
Interval Scheduling Greedy	Notebook	10.9	Modulo De Entero Gigante	Notebook	6.14	Required Substring	CSES	20.6
Inversions	Codeforces	7.5	Monarchy In Vertigo	Day 4	1.17	Revenge Of The Salary Of At Coder	Day 10	2.15
Investigating Quadrádómeda	Day 6	5.20	Monsters	CSES	17.14	Rock Paper Scissors Tournament	Day 11	13.2
Investigation	CSES	17.10	Mount Rushmore And Birthdays	Day 5	7.10	Rotating Calipers	Notebook	4.11
J			Movie Festival Queries	CSES	19.11	Round Trip	CSES	17.16
Joining Capitals	Day 8	2.13	Music Festival	Day 8	2.2	Round Trip (Stack)	CSES	17.17
Joker Or Judge	Codeforces	13.11	N			Round Trip II	CSES	17.18
Journeygrapes	Codeforces	6.11	Nomad	Day 10	10.5	Running A Penitentiary	Day 7	1.18
João João	Day 6	13.12	Number Of Sets	LeetCode	7.7	S		
Juan Vs Frank	Day 9	8.2	Number Spiral	CSES	18.14	Salary Queries	CSES	19.20
Judgmental Crowd	ICPC	9.3	Numeros Romanos	Notebook	5.15	Satoru Gojo	Codeforces	5.16
Just Palindromes	ICPC 2025	9.7	O			School Dance	Day 5	3.10
K			Office Space	Day 11	4.1	Secuencias Notables	Formulario	14.9
K Query	Day 4	1.6	Oficinas A Distancia D En Arbol	Notebook	10.10	Shoelace	Notebook	4.12
Karamell	Day 3	2.14	Omens	ICPC 2024	4.9	Shortest Routes I	CSES	17.19
Kathmandu	Day 4	10.16	Only1s0s	ICPC 2024	6.1	Shortest Routes II	CSES	17.20
Kilometric Intersection	Day 9	4.5	Ordered Set (PBDS)	Notebook	1.12	Shortest Routes II (Floyd Warshall)	CSES	17.21
Kings Conquest	ICPC	16.10	P			Sign	CCPL	9.5
Knight Moves Grid	CSES	18.11	Palindrome Reorder	CSES	18.15	Signal Coverage	ICPC 2024	3.1
Knights In The Board	Day 7	3.9	Palindromic Lexi Small	LeetCode	7.12	Sky Line	ICPC 2024	1.4
L			Parentesis Balanceados	Notebook	9.9	Sliding Window	Notebook	10.17
Labyrinth	CSES	17.11	Patches	Day 2	2.1	Song Of The Sirens	Codeforces	9.4
Ladder Length	LeetCode	3.3	Permutation Rounds	Day 10	6.18	Spacebar Tokenizer	ICPC 2024	9.16
Lecographically Maximum	Day 3	10.2	Permutations	CSES	18.16	Square Diff	ICPC 2024	6.6
Lectura De Entrada	Notebook	13.20	Pizza Cutter	Day 8	7.6	Static Range Minimum Queries	CSES	19.21
Less Coint Tosses	Day 1	7.8	Pizzeria Queries	CSES	19.12	Static Range Sum Queries	CSES	19.22
Lines Of Containers	Day 2	7.9	Planetqueriesi	CSES	17.15	STL Indispensable	Formulario	14.10
List Removals	CSES	19.9	Planets And Kingdoms	Day 5	3.16	String Matching	CSES	20.7
LLMs	Day 6	13.9	Policianacional	Codeforces	9.10	String Reorder	CSES	18.19
Lonely Creatures	ICPC	16.11	Polynomial Queries	CSES	19.13	Sub Array Sum Queries	CSES	19.23
Longest Flight Route	CSES	17.12	Posters	RPC	10.3	Sub Array Sum Queries II	CSES	19.24
Longest Palindrome	CSES	20.4	Pou	Codeforces	5.10	Submerging Islands	Day 4	3.2
Los Ratones III	Day 9	3.23	Prefix Sum Queries	CSES	19.14	Subway Lines	Day 8	3.25
Lumina	ICPC 2024	3.15	Prime Sieve	Day 11	6.20	Sumas Y Series	Formulario	14.11
Lumina Juan	ICPC 2024	3.14	Primitivas Y Convex Hull	Notebook	4.8	Switches	Day 8	13.15
M			Probabilidad Geometrica	Formulario	14.7	T		
Mail Delivery	Day 5	3.11	Process Str	LeetCode	9.13	Tabla ASCII	Formulario	14.12
Maratona Brasileira De Popcorn	Day 1	10.1	Proving Equivalences	Day 4	3.31	Template Base	Notebook	13.24
Marbles	Day 8	8.4	Punto En Poligono Convexo	Notebook	4.10	Teoria De Numeros	Formulario	14.13
March Of The Penguins	Day 7	3.27	Pythagorean	RPC	6.21	The Bridge At Night	ICPC 2024	10.13
Max Palindromes	LeetCode	9.8	R			The Day Of The Competitors	Day 4	1.7
Maximizing Advertising	Day 7	10.12	Rab Game I	CSES	18.17	The Declaration Of Independence	Day 5	1.13

Indice alfabetico (cont.)

The Seals Of Shanghai	Day 10	6.17
The Wisdom Of Master Wei	Day 10	5.3
Toilet Seat	Day 11	13.1
Total Numbers	LeetCode	7.4
Tower Of Hanoi	CSES	18.20
Trailing Zeros	CSES	18.21
Trampas De C++	Formulario	14.14
Triangulos	Day 2	5.18
Triangulos En Poligono Convexo	Notebook	7.16
Trigonometria	Formulario	14.15
Trucks	Day 2	3.24
Trucos De Bits	Formulario	14.16
Turnswitch	ICPC 2024	13.21
Two Knights	CSES	18.22
Two Sets	CSES	18.23
U		
Uj	CCPL	6.2
Uj	CCPL	6.3
Unraveling Monty Hall	Day 8	13.5
V		
Visible Building Queries	CSES	19.25
W		
Weird Algorithm	CSES	18.24
Wine Production	Day 7	1.11
Word Combinations	CSES	20.8
Y		
Yuyuan Market	Day 10	2.10
Z		
Zero Or One	Day 2	13.3
Zig Zag Array	LeetCode	2.11